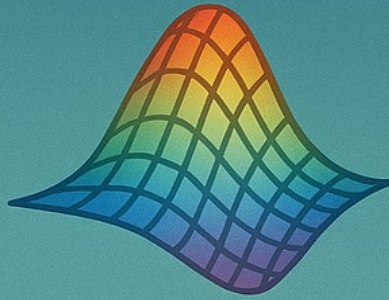


KOMPUTASI MATEMATIKA DENGAN SOFTWARE EULER MATH TOOLBOX (EMT)

Teori dan Praktik



$$\int_a^b f(x) dx$$

$$\pi = 3,14$$

```
>t=linspace(0,2pi,1000);  
>x1=1.5*cos(10*t)+cos(23*t); y1=1.5*sin(10*t)-sin(23*t);  
>x2=cos(3*t); y2=sin(4*t);  
>aspect(2); figure(1,2);  
>figure(1); plot2d(x2,y2,>filled,fillcolor=5,<frame,<grid);  
>figure(2); plot2d(x1,y1,<grid,<frame,>filled,fillcolor=10);  
>figure(0);
```



$$x_1 = \cos(3t), \quad y_1 = \sin(4t)$$

$$x_2 = 1.5 \times \cos(10t) + \cos(23t), \quad y_2 = 1.5 \times \sin(10t) - \sin(23t)$$

Drs. Sahid, M.Sc.



Copyright © 2025 Sahid

[Nama Penerbit]

ISBN [1234567890XYZ]

**Komputasi Matematika dengan Software Euler Math Toolbox (EMT):
Teori dan Praktik**

© Hak Cipta Dilindungi Undang-Undang, 2025

Penulis: Drs. Sahid, M.Sc.

Editor: Sahid

Perancang Sampul: Sahid (dibantu AI ChatGPT)

Penata letak: Sahid (menggunakan software L^AT_EX)

Cetakan pertama, Mei 2025

Diterbitkan melalui: [Nama Penerbit]

Perpustakaan Nasional RI Data Katalog dalam Terbitan (KDT)

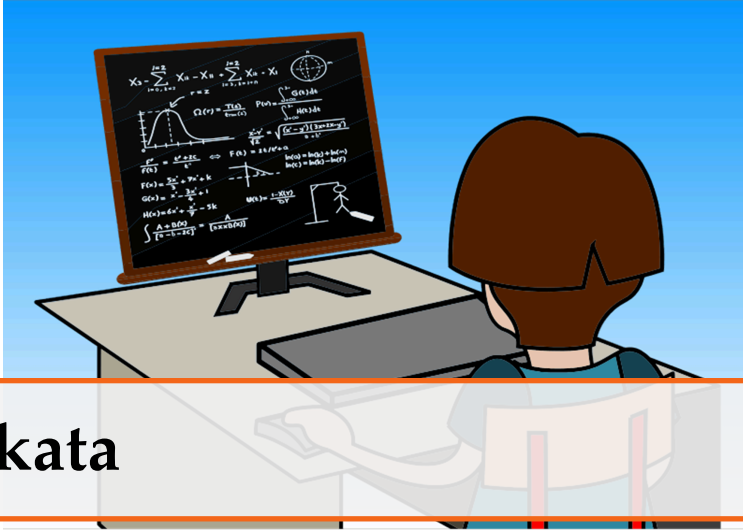
Drs. Sahid, M.Sc.

Komputasi Matematika dengan Software Euler Math Toolbox (EMT):
Teori dan Praktik 2025 / Penyusun, Drs. Sahid, M.Sc., – Yogyakarta:
[Nama Penerbit] 2025

ix + 513 hlm.; 18,2 × 25,7 cm²

ISBN [1234567890XYZ]

1. Komputasi Matematika, Referensi I. Judul



Prakata

Puji syukur Alhamdulillah penulis panjatkan ke hadirat Allah SWT atas limpahan rahmat, hidayah, dan taufik serta kekuatan lahir dan batin, sehingga buku ini dapat diselesaikan dan terbit. Memakan waktu yang cukup lama untuk menyelesaikan naskah buku ini sampai akhirnya hadir di hadapan pembaca dengan isi dan tampilan yang seperti Anda lihat.

Buku ini merupakan buku pertama dalam bahasa Indonesia yang menjelaskan topik-topik dalam komputasi matematika dan penyelesaian berbagai masalah komputasi matematika dengan menggunakan software Euler Math Toolbox (EMT). Software EMT adalah aplikasi matematika gratis yang belum banyak dikenal di Indonesia (bahkan internasional), padahal software ini sangat bagus, memungkinkan perhitungan matematika dilakukan secara terpadu dengan teks penjelasan yang dapat dilengkapi dengan notasi/ekspresi matematika menggunakan $\text{\LaTeX}$.

Cakupan isi buku ini meliputi:

1. Komputasi Matematika dan Software Aplikasinya,
2. Pengenalan software EMT,
3. Komputasi Aljabar dengan EMT,
4. Visualisasi Grafik 2 Dimensi (2D) dengan EMT,
5. Visualisasi Grafik 3 Dimensi (3D) dengan EMT,

Penyajian materi setiap topik mencakup penjelasan teori, contoh-contoh masalah komputasi dan penyelesaiannya menggunakan EMT (menggunakan perintah-perintah EMT dan program dengan bahasa EMT) beserta penjelasan hasilnya. Penulis telah berusaha menyajikan isi buku ini selengkap dan sedetil mungkin untuk setiap topik, sehingga menjadikan buku layak sebagai referensi yang bermanfaat bagi pembaca, baik guru, dosen, siswa, maupun mahasiswa. Meski-

pun topik utama yang disajikan pada buku ini hanya sebanyak 5 (lima), namun keseluruhan isi buku ini terbagi menjadi 7 bab, karena dua topik terpaksa masing-masing disajikan dalam dua bab.

Buku ini menyajikan referensi yang sangat bermanfaat bagi para dosen, mahasiswa, dan para peneliti yang harus menyelesaikan masalah-masalah matematika sekaligus menemukan cara praktis untuk menyelesaikan masalah-masalah matematika tersebut dengan software EMT yang memiliki antarmuka berbentuk *notebook*, yang menggabungkan teks deskriptif mendukung $\text{\LaTeX}$ dan komputasi matematis, termasuk pemrogramannya.

Proses penulisan buku ini memakan waktu yang cukup lama. Hal ini dikarenakan penulis harus mengumpulkan bahan-bahan yang harus ditulis sesuai dengan cakupan materi di atas dari berbagai sumber, baik dari buku-buku matematika, panduan software EMT, maupun sumber-sumber di Internet. Penulisan buku ini juga melibatkan penggunaan berbagai teknologi, tidak hanya software EMT itu sendiri yang menjadi fokus penulisan buku ini, namun juga penggunaan software editor Markdown, dan pengolah dokumen $\text{\LaTeX}$.

Proses penulisan buku ini dilakukan dengan langkah-langkah sebagai berikut.

1. Untuk bab-bab yang menjelaskan tentang komputasi matematika, naskah asli ditulis langsung menggunakan software EMT, karena semua contoh perhitungan langsung dilakukan dengan EMT. File EMT disimpan dengan ekstensi `.en` yang merupakan file *notebook* EMT (berisi teks penjelasan dan contoh-contoh perhitungan dan hasilnya).
2. Setelah selesai penulisan naskah asli, file *notebook* EMT dikonversi menjadi file **Markdown** (dengan ekstensi `.md`)
3. File Markdown diedit dengan editor Markdown untuk menambahkan dan merapikan teks penjelasan, termasuk teks uraian dalam bentuk tabel dan persamaan-persamaan matematika dengan $\text{\LaTeX}$, yang terlalu rumit untuk dilakukan di EMT.
4. Untuk bab yang tidak menjelaskan atau memuat perhitungan matematika, naskah asli diketik menggunakan editor Markdown.
5. File-file Markdown yang sudah selesai diedit selanjutnya dikonversi menjadi file $\text{\LaTeX}$ (dengan ekstensi `.tex`)
6. File-file $\text{\LaTeX}$ selanjutnya dirakit (dan dilakukan beberapa penyuntingan untuk penomoran tabel dan gambar secara otomatis) menggunakan editor $\text{\LaTeX}$.
7. Terakhir, file-file $\text{\LaTeX}$ tersebut dirakit menjadi satu file PDF yang merupakan naskah jadi buku ini.

Format, tata letak, dan tampilan buku ini, mulai dari halaman sampul sampai halaman terakhir, dirakit dengan $\text{\LaTeX}$ menggunakan file kelas yang penulis modifikasi dari file-file kelas dokumen $\text{\LaTeX}$ yang tersedia di situs Template $\text{\LaTeX}$ dan menggunakan berbagai paket $\text{\LaTeX}$ sehingga menghasilkan tampilan buku

seperti yang sedang Anda lihat dan baca ini.

Penulisan buku ini tidak menggunakan MS Word, karena MS Word sangat sulit digunakan untuk menulis buku yang masing-masing bab ditulis dalam file terpisah dengan penomoran halaman, gambar, dan tabel, serta daftar isi seluruh buku otomatis berkesinambungan. Selain itu, dengan MS Word, konsistensi penulisan sangat sulit dilakukan karena semua harus dilakukan secara manual. Misalnya, semua contoh penulisan perintah komputer di dalam buku ini ditulis dengan jenis huruf yang mirip tulisan mesin ketik, jika diketik dengan MS Word, sangat sulit untuk membuat konsisten dengan mengatur jenis huruf untuk masing-masing tulisan.

Penulis mengucapkan terima kasih yang sebesar-besarnya kepada Rene Grothmann, yang telah mengembangkan software EMT dan memberikannya kepada umum secara gratis, Donald Knuth, yang telah mengembangkan software $\text{\TeX}$ dan menjadikannya software gratis, para pengembang software editor Mark-down, dan para pengembang paket $\text{\LaTeX}$ yang menjadikannya software pengolah dokumen yang sangat andal.

Semoga kehadiran buku ini dapat memberikan manfaat yang sebesar-besarnya bagi dunia akademis di Indonesia.

Yogyakarta, 10 Juli 2025

Penulis

Sahid

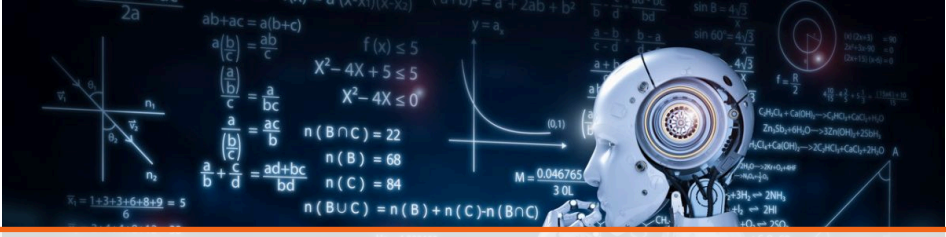


Daftar Isi

1	Komputasi Matematika dan Software Aplikasinya	1
1.1	Apakah Komputasi Matematika?	1
1.2	Cakupan Masalah Komputasi Matematika	2
1.3	Software untuk Komputasi Matematika	4
1.3.1	Software Kalkulator	5
1.3.2	Software Sistem Aljabar Komputer (CAS: <i>Computer Algebra System</i>)	6
1.3.3	Software Geometri Dinamis (DGS: <i>dynamic geometry system</i>)	12
1.3.4	Software Analisis Numerik	26
1.3.5	Software Statistika	56
2	Pengenalan dan Panduan Penggunaan Software EMT	71
2.1	Apakah EMT?	71
2.2	Memasang Software EMT	72
2.2.1	Maxima	74
2.2.2	L ^A T _E X	74
2.2.3	Python	75
2.2.4	TinyC	75
2.2.5	Povray	75
2.2.6	Editor Eksternal	76
2.2.7	Euler Portabel	76
2.2.8	Instalasi Jaringan	76
2.3	Komponen dan Fitur EMT	77
2.3.1	Fitur Utama	78
2.3.2	EMT Sebagai Bahasa Pemrograman	79

2.4	Mulai Bekerja dengan EMT	81
2.4.1	Antarmuka Pengguna	82
2.4.2	Jendela <i>Notebook</i> (Ruang Kerja EMT)	86
2.5	Mengenai perintah-perintah EMT	94
2.5.1	Sintaks penulisan perintah Euler	94
2.5.2	Matematika Simbolik	116
2.5.3	Pemrograman dalam EMT	124
2.5.4	Skrip	137
3	Komputasi Aljabar dengan EMT	139
3.1	Pengertian Aljabar dan Bentuk Aljabar	140
3.2	Operasi Bentuk Aljabar dan Menyederhanakan Hasilnya	141
3.3	Faktorisasi Bentuk Aljabar	149
3.4	Fungsi Polinomial dan Rasional	156
3.5	Fungsi Komposisi dan Fungsi Invers	171
3.6	Fungsi Eksponensial	176
3.7	Fungsi Logaritma	179
3.8	Operasi Fungsi Trigonometri dan Hiperbolik	189
3.9	Operasi dan Fungsi Bilangan Kompleks	201
3.9.1	Bentuk Polar Bilangan Kompleks	207
4	Komputasi Aljabar Lanjut dengan EMT	213
4.1	Menyelesaikan Persamaan dan Sistem Persamaan	213
4.1.1	Sistem Persamaan Linier	220
4.1.2	Persamaan Linier Tiga Variabel	222
4.1.3	Persamaan Kuadrat	224
4.2	Penyelesaian Langkah demi Langkah	228
4.3	Pertidaksamaan dan Sistem Pertidaksamaan	231
4.3.1	Pertidaksamaan Linear Satu Variabel	231
4.3.2	Pertidaksamaan Linear Dua Variabel	235
4.3.3	Pertidaksamaan Kuadrat	237
4.3.4	Pertidaksamaan Logaritma	240
4.3.5	Pertidaksamaan Trigonometri	243
4.3.6	Pertidaksamaan Eksponen	244
4.4	Manipulasi dan Perhitungan Matriks dan Vektor	245
4.4.1	Fungsi-fungsi Dasar Matriks	267
4.4.2	Matriks Simbolik	270
4.5	Contoh Aplikasi Aljabar: Perhitungan Bunga Majemuk	275
4.6	Penyelesaian Simbolik Masalah Bunga Bank	282

5	Visualisasi Grafik 2 Dimensi (2D) dengan EMT	287
5.1	Layar untuk menggambar	288
5.2	Menggambar grafik 2D dengan <code>plot2d()</code>	289
5.3	Pengaturan rasio bidang koordinat	290
5.4	Menggambar grafik/kurva $y = f(x)$	292
5.5	Grafik fungsi yang memuat parameter	302
5.6	Menggambar beberapa kurva sekaligus	308
5.7	Gaya/model (<i>style</i>) plot 2D	317
5.7.1	Teks label	323
5.7.2	Teks label, keterangan, judul dengan LaTeX	328
5.8	Menggambar grafik interaktif	333
5.9	Menggambar data 2D (titik)	339
6	Visualisasi Grafik 2 Dimensi (2D) Lebih Lanjut	343
6.1	Kurva fungsi implisit	343
6.2	Grafik fungsi logaritmik	355
6.3	Grafik fungsi parametrik dan persamaan polar	358
6.4	Menggambar grafik bilangan kompleks	371
6.5	Menggambar daerah yang dibatasi dua kurva dan segi banyak	383
6.6	Rujukan lengkap fungsi <code>plot2d()</code>	394
7	Visualisasi Grafik 3 Dimensi (3D) dengan EMT	399
	Daftar Pustaka	401



Bab 1.

Komputasi Matematika dan Software Aplikasinya

1.1 Apakah Komputasi Matematika?

Di dalam matematika terdapat dua istilah yang menggunakan dua kata sama namun memiliki pengertian yang sangat berbeda, *komputasi matematika* dan *matematika komputasi*. Apabila Anda menggunakan **Google** untuk mencari tahu tentang kedua istilah tersebut, mungkin Anda akan lebih banyak mendapatkan informasi atau sumber informasi tentang matematika komputasi. Mungkin juga Anda akan bingung membaca penjelasan tentang kedua istilah tersebut, ada yang mengatakan keduanya sama, ada juga yang mengatakan keduanya berbeda, namun pengertiannya saling terbalik. Buku ini berisi tentang komputasi matematika, bukan matematika komputasi. Berikut adalah beberapa definisi tentang istilah komputasi yang dapat ditemukan pada beberapa sumber di Internet.

Situs **Mathisfun** (<https://www.mathsisfun.com/definitions/computation.html>) mendefinisikan **komputasi** sebagai mencari jawaban atau hasil penjumlahan, pengurangan, perkalian, dan sebagainya menggunakan matematika atau logika. Perhitungan-perhitungan yang lebih rumit memerlukan komputer. Sementara itu, situs **Storyofmathematics** (<https://www.storyofmathematics.com/glossary/computation/>) mendefinisikan **komputasi** sebagai prosedur yang menggunakan operasi matematika, seperti penjumlahan, pengurangan, perkalian, dan pembagian, atau langkah-langkah logis untuk mendapatkan jawaban atau penyelesaian suatu masalah. Proses komputasi dapat dilakukan secara manual, dengan bantuan alat hitung mekanik (seperti **abakus**) maupun alat hitung elektronik (seperti kalkulator), maupun menggunakan perangkat lunak (*software*) komputer. Buku ini berisi tentang komputasi dengan menggunakan (*software*) komputer. Dengan demikian, untuk kejelasan makna, maka digunakan pengertian sebagai berikut.

Komputasi matematika yang dimaksud di dalam buku ini adalah semua prosedur atau langkah-langkah untuk menyelesaikan masalah

dengan menggunakan operasi matematika dan/atau penalaran logis yang dilakukan dengan bantuan perangkat lunak komputer.

Komputasi matematika berkaitan dengan penggunaan komputer untuk memecahkan masalah matematika, mulai dari yang sederhana sampai masalah yang kompleks atau melibatkan perhitungan yang sangat besar. Oleh karena itu, komputasi matematika mencakup perhitungan-perhitungan sederhana dengan menggunakan operasi dasar aritmetika (seperti penjumlahan, pengurangan, perkalian, dan pembagian), perhitungan-perhitungan yang menggunakan fungsi-fungsi matematika (misalnya fungsi aljabar, fungsi trigonometri, fungsi logaritma, fungsi eksponensial, dan fungsi-fungsi khusus bilangan bulat), dan perhitungan-perhitungan yang menggunakan algoritma tertentu. Termasuk bagian dari komputasi matematika adalah visualisasi matematika (misalnya menggambar kurva, baik kurva dua dimensi maupun permukaan tiga dimensi, menggambar aneka grafik untuk visualisasi data).

Dilihat dari hasil perhitungan, komputasi matematika terdiri atas komputasi numerik dan komputasi simbolik (eksak). Komputasi numerik menggunakan bilangan-bilangan riil dan hasilnya berupa bilangan riil. Maksud bilangan riil adalah bilangan yang ditulis menggunakan titik desimal (titik pemisah bagian bulat dan bagian pecahan), misalnya 5.25, 12.0, 0.123. Di dalam komputer istilah yang digunakan untuk menyatakan bilangan desimal semacam ini adalah **bilangan titik mengambang** (*floating point number*). Komputasi simbolik atau eksak menggunakan simbol-simbol atau variabel maupun nilai-nilai eksak (tidak memuat titik desimal) dan hasilnya juga dalam bentuk simbol-simbol atau nilai-nilai eksak.

1.2 Cakupan Masalah Komputasi Matematika

Cakupan masalah komputasi matematika adalah seluruh perhitungan yang dilakukan di dalam matematika, baik perhitungan yang melibatkan bilangan riil, bilangan bulat, bilangan asli, bilangan kompleks, vektor dan matriks, variabel/simbol numerik, perhitungan yang melibatkan operasi hitung dasar (penjumlahan, pengurangan, perkalian, pembagian), perpangkatan, penarikan akar, maupun perhitungan menggunakan fungsi-fungsi khusus dalam matematika (termasuk fungsi trigonometri, eksponensial, logaritma, dan kombinasinya).

Secara umum, masalah komputasi dalam matematika mencakup seluruh cabang matematika, mulai dari yang paling sederhana sampai perhitungan yang kompleks. Hampir semua cabang matematika memerlukan komputasi dan pembuktian teorema-teorema terkait. Di antara cabang-cabang matematika yang paling umum dan macam-macam komputasi yang terkait adalah sebagai berikut. Masing-masing cabang mungkin saja memiliki subcabang (ranting) yang lebih spesifik.

- **Aritmetika:** perhitungan dengan operasi dasar aritmetika (penjumlahan,

pengurangan, perkalian, pembagian), dll.

- **Aljabar:** perhitungan yang melibatkan penggunaan angka dan variabel dan bentuk-bentuk aljabar (penjumlahan, pengurangan, perkalian, pembagian, perpangkatan, penarikan akar, dll.), penyelesaian persamaan dan pertidaksamaan, penyelesaian sistem persamaan, operasi fungsi (termasuk fungsi eksponensial, fungsi logaritma, fungsi trigonometri, fungsi polinomial), operasi pada bilangan riil dan bilangan kompleks, operasi dan perhitungan-perhitungan terkait matriks/vektor (penjumlahan, pengurangan, perkalian, determinan, invers, nilai eigen, dll.), dsb.
- **Geometri:** perhitungan yang terkait dengan objek-objek geometri (titik koordinat, jarak, luas, keliling, volume, sudut, dimensi, persamaan garis, persamaan bidang, persamaan tempat kedudukan (lokus), transformasi, dsb.), masalah mengkonstruksi bangun-geometri, dll.
- **Kalkulus:** perhitungan yang terkait dengan aneka macam fungsi (sifat-sifat fungsi, grafik fungsi, komposisi fungsi, invers fungsi, kurva fungsi, limit fungsi, turunan fungsi, integral fungsi dan aplikasinya, dll.), termasuk barisan dan deret tak hingga (perhitungan suku-suku barisan, limit barisan, jumlah deret, deret fungsi, dll.)
- **Matematika diskrit:** operasi logika (dan, atau, negasi), aljabar logika (proposisi), penarikan kesimpulan, pembuktian, operasi himpunan (gabungan, irisan, komplemen), perhitungan banyak anggota suatu himpunan, perhitungan nilai faktorial, banyak urutan/susunan (permutasi) dan aturan perkalian, banyak pilihan (kombinasi) dan aturan penjumlahan, prinsip sangkar burung, perhitungan suku-suku perpangkatan binomial dan multinomial, perhitungan fungsi-fungsi khusus terkait bilangan cacah, penyelesaian relasi rekurens, barisan bilangan bulat dan fungsi pembangkit barisan, perhitungan nilai peluang suatu kejadian, perhitungan-perhitungan terkait dengan graf, konversi sistem bilangan, pengkodean dan persandian, dsb.
- **Teori bilangan:** perhitungan-perhitungan yang terkait dengan bilangan asli, operasi bilangan bulat, mencari faktor persekutuan terbesar, kelipatan persekutuan terkecil, menghitung banyak pembagi suatu bilangan, sisa pembagian, keterbagian, modulo bilangan dan aritmetika modulo, kongruensi, penyelesaian persamaan kongruensi, bilangan prima, uji bilangan prima, bilangan-bilangan bulat yang saling prima, faktorisasi prima, penjumlahan bilangan prima, distribusi bilangan prima, perhitungan fungsi-fungsi khusus bilangan asli, bilangan rasional dan bilangan irasional, pecahan berlanjut, perhitungan terkait pola-pola bilangan tertentu, barisan bilangan Fibonacci dan Lucas, dsb.
- **Optimisasi:** penyajian model-model matematika untuk masalah-masalah menentukan nilai optimal dalam kehidupan sehari-hari, penyelesaian masalah-masalah optimisasi, penyelesaian program linier, penyelesaian pemrograman kuadratik, penyelesaian pemrograman bulat, penyelesaian pemrograman non-linier, algoritma simplex, metode gradien, metode *branch and bound*, analisis sensitivitas, pemrograman konveks, pemrograman

dinamis, pemrograman stokastik, algoritma heuristik, algoritma metaheuristik, algoritma genetika, optimisasi sistem antrian, simulasi, dsb.

- **Statistika:** perhitungan dalam statistika (jumlah data, nilai rata-rata, median, modus, kuartil, simpangan baku, varians, uji statistika, interval kepercayaan, perhitungan regresi, koefisien korelasi, perhitungan nilai fungsi distribusi), penyajian data, visualisasi data, analisis varians, uji Chi kuadrat, uji normalitas, regresi logistik, regresi berganda, analisis faktor, analisis komponen utama, analisis klaster, analisis runtun waktu, analisis survei, metode Bootstrap, metode Monte Carlo, dsb.
- **Aljabar abstrak:** perhitungan-perhitungan terkait sistem matematika (grup, medan, gelanggang), operasi elemen-elemen grup, medan, gelanggang, perhitungan grup Abelian, grup non-Abelian, grup siklis, grup transformasi, dsb.

Masih banyak cabang matematika lain yang juga memerlukan komputasi terkait dan dapat dilakukan dengan bantuan program komputer. Pada bagian berikutnya diuraikan berbagai software untuk komputasi matematika. Sudah tentu daftar yang disajikan bukanlah mencakup seluruh software untuk komputasi matematika yang sudah tersedia, karena terdapat ratusan, bahkan mungkin ribuan software untuk aplikasi matematika. Kiranya daftar yang disajikan cukup memberikan gambaran dan pilihan bagi pengguna yang memerlukan software untuk menyelesaikan masalah-masalah matematika yang dihadapi.

1.3 Software untuk Komputasi Matematika

Software untuk komputasi matematika dapat dikategorikan berdasarkan cara pendistribusiannya dan berdasarkan cakupan permasalahan matematika yang dapat diselesaikan. Berdasarkan cara pendistribusiannya, software matematika, seperti halnya software lain, dapat diklasifikasi menjadi software komersial (berbayar), software *shareware* (berbayar relatif murah, sebelum membeli dapat mencoba), dan software gratis (baik *open source* maupun *proprietary*).

Berdasarkan cakupan masalah matematika yang dapat diselesaikan, software komputasi matematika dapat diklasifikasi menjadi beberapa kategori. Di antara kategori yang paling umum adalah:

- Software kalkulator;
- Software sistem aljabar komputer (CAS: *computer algebra system*);
- Software geometri dinamis (DGS: *dynamic geometry system*);
- Software analisis numerik;
- Software optimisasi;
- Software statistika;
- Software pembuktian teorema dan asisten pembuktian;
- Software matematika musik.

1.3.1 Software Kalkulator

Tabel 1.1: Contoh-contoh software kalkulator berbasis tombol

Nama	Lisensi Software	Sistem Operasi	Presisi	Modus Sainifik	Modus Hex/oct/bin
DeskCalc	Gratis (MIT)	Haiku	Sebarang bilangan desimal	Ya	Tidak
Mac OS Calculator	Komersial	macOS	Dobel (64 bit)	Ya	Ya
Windows Calculator	Gratis (MIT)	Windows	Desimal paling sedikit 32 digit	Ya	Ya
KCalc	Gratis (GPL-2.0)	Linux, BSD, macOS	Sebarang bilangan desimal	Ya	Ya
xcalc	Gratis (X11)	Linux	Sebarang bilangan desimal	Ya	Ya
GNOME Calculator	Gratis (GPL-3.0)	Linux, BSD, macOS	Sebarang bilangan desimal	Ya	Ya

Tabel 1.2: Contoh-contoh software kalkulator berbasis perintah

Nama	Lisensi Software	Sistem Operasi	Presisi	Modus hex/oct/bin	Definisi variabel	Definisi fungsi	Menyopot sintaks	Konversi satuan	Langkah ganda
bc	Gratis (GNU GPL)	Unix	Variabel	Ya	Ya	Tidak	Tidak	Tidak	-
kalker	Gratis (MIT)	Linux, BSD, Android, macOS, Windows	Variabel	Ya	Ya	Ya	Ya	Sebagian	-
Auto-Calcs	Gratis	Web	Variabel	Tidak	Ya	Tidak	Tidak	Otomatis, Manual	Ya
Calc-pad	Gratis (MIT)	Windows, Web	Dobel	Tidak	Ya	Ya	Ya	Ya	-

Software kalkulator adalah kalkulator dalam bentuk program komputer, bukan dalam bentuk fisik sebagai peralatan elektronik. Software ini memiliki fungsi

persis seperti kalkulator elektronik fisik, mulai dari perhitungan sederhana berupa operasi-operasi penjumlahan, pengurangan, perkalian, pembagian, sampai dengan berbagai perhitungan canggih yang dapat dilakukan oleh kalkulator saintifik, seperti perpangkatan, penarikan akar, logaritma, fungsi-fungsi trigonometri, perhitungan statistika, dan sebagainya. Terdapat dua kategori software kalkulator, yakni software kalkulator berbasis tombol (memiliki antarmuka grafis) dan kalkulator berbasis baris perintah terminal (Wikipedia, 2023).

Tabel 1.1 menyajikan informasi beberapa contoh software kalkulator berbasis tombol dan karakteristik utamanya. Tabel 1.2 menyajikan informasi beberapa contoh software kalkulator berbasis perintah/ekspresi/rumus dan karakteristik utamanya.

1.3.2 Software Sistem Aljabar Komputer (CAS: *Computer Algebra System*)

Istilah CAS (*Computer Algebra System*) digunakan untuk merujuk nama-nama software yang memiliki kemampuan komputasi matematika simbolik atau eksak. Software CAS dapat digunakan untuk menyelesaikan masalah-masalah matematika, baik secara numerik maupun eksak. Beberapa software CAS juga dapat digunakan untuk menggambar grafik matematis. Tabel 1.3 menyajikan daftar beberapa software CAS beserta karakteristik utamanya (adaptasi dan *update* dari Wikipedia, 2024).

Tabel 1.3: Daftar software CAS

Nama CAS (Pertama mulai/rilis)	Pembuat	Versi stabil terbaru (Tanggal)	Lisensi (Harga, USD)	Catatan	Sistem Operasi (Platform)
Axioma (1971/1993, 2002)	Richard Jenks	Agustus 2014	Gratis (BSD termodifikasi)	CAS umum	Linux, Windows (WSL), Mac OS X Docker
Cadabra (2001/2007)	Kasper Peters	2.4.4 (20-9-2023)	Gratis (GNU GPL)	CAS khusus untuk teori medan	Linux, MacOS, Windows, BSD, Jupyter Notebook
CoCoA (1987/1995)	Abbott, J.; Bigatti, A. M. and Robbiano, L. (Universitas Genewa, Italia)	5.4.0 (5-7-2023)	Gratis (GNU GPL)	CAS khusus untuk aljabar komutatif	Linux/UNIX, Windows, MacOS

Tabel 1.3: Daftar software CAS (... lanjutan)

Nama CAS (Pertama mulai/rilis)	Pembuat	Versi stabil terbaru (Tanggal)	Lisensi (Harga, USD)	Catatan	Sistem Operasi (Platform)
Fermat (1986/1993)	Robert H.Lewis	7.3 (11-1-2024)	Gratis (GNU GPL), donasi \$60 (jika mau)	CAS khusus untuk komputasi polinomial dan matriks	Linux, Windows (WSL), Mac OS X
FORM (1984/1989)	JAM Vermaseren (Nikhef, Institut Fisika Subatom Belanda)	4.3.1 (11-4-2023)	Gratis (GNU GPL)	CAS dirancang terutama untuk fisika partikel	<i>Source code</i> dalam bahasa C di https://github.com/vermaseren/form
FriCAS (2007)	Waldek Hebisch	1.3.10 (9-1-2024)	Gratis (BSD termodifikasi)	CAS umum berfitur lengkap untuk riset matematika dan desain algoritma baru, dikembangkan dari Axiom	Linux/UNIX/ BSD, MacOS, Windows (Cygwin, WSL)
GAP (1986)	Universitas St Andrews, RWTH Aachen, TU Braunschweig, Colorado State University, TU Kaiserslautern	4.12.2 (18-12-2022)	Gratis (GNU GPL)	CAS khusus untuk teori grup dan kombinatorik diperluas dengan berbagai paket pendukung	Linux, Windows, MacOS
CAS GeoGebra (2013)	Markus Ho- henwarter, dkk.	6.0.820.0 (3-1-2023)	Lisensi GeoGebra, Freeware non- komersial, sebagian GPL, CC- BY-NC-SA	Penggabungan Xcas Bernard Pariisse dengan GeoGebra	Windows, Linux, MacOS, Android, iOS, Web, Chromebook

Tabel 1.3: Daftar software CAS (... lanjutan)

Nama CAS (Pertama mulai/rilis)	Pembuat	Versi stabil terbaru (Tanggal)	Lisensi (Harga, USD)	Catatan	Sistem Operasi (Platform)
GNU Octave (1992/1993)	John W. Eaton	8.4.0 (5-11-2023)	Gratis (GPLv3+)	Bahasa pemrograman tingkat tinggi untuk komputasi ilmiah dan komputasi numerik sebagian besar kompatibel dengan MATLAB diperluas dengan berbagai paket pendukung, termasuk paket matematika simbolik (CAS)	Linux/BSD, Windows, MacOS
LiveMath (1991/1993)	MathMonkeys, LLC	3.6 (Agustus 2020)	Komersial (29 - 399 USD + (20-70 USD)/ aktivasi)	CAS dengan antarmuka GUI, berbasis notebook mirip Mathematica dan MathCAD	Linux, Windows, MacOS, Solaris
Macaulay2 (1992/1994)	Daniel Grayson dan Michael Stillman	1.22 (1-7-2021)	Gratis (GNU GPL)	CAS khusus untuk geometri aljabar dan aljabar komutatif diperluas dengan berbagai paket pendukung	Windows, MacOSX, Linux/BSD
Macsyma (1968/1982)	Proyek MAC MIT dan Symbolics	2.4(1999)	Komersial (500 USD)	CAS tujuan umum tertua, masih hidup sebagai Maxima yang gratis.	Linux, Windows, MacOS
Magma (~1990/1993)	Universitas Sydney	2.28-5 (MacOS) (22-12-2023) 2.25-3 (Windows)	non-komersial, perlu biaya langganan untuk dapat mengunduhnya	CAS tujuan umum, awalnya dirancang untuk masalah aljabar, teori bilangan, geometri dan kombinatorik	Linux, Windows, MacOS, Solaris
Maple (1980/1984)	Maplesoft, divisi Waterloo Maple Inc.	2023 (9-3-2023)	Komersial (99 - 6715 USD)	CAS tujuan umum untuk komputasi simbolik dan numerik, bahasa pemrograman tingkat tinggi	Linux, Windows, MacOS

Tabel 1.3: Daftar software CAS (... lanjutan)

Nama CAS (Pertama mulai/rilis)	Pembuat	Versi stabil terbaru (Tanggal)	Lisensi (Harga, USD)	Catatan	Sistem Operasi (Platform)
Mathcad (1985)	Allen Razdow & Josh Bernoff di Mathsoft (pendiri: David Blohm & Razdow)	PTC Mathcad Prime 9 (14-3-2023)	Lisensi: 735 USD/tahun Edisi siswa: 63USD/tahun	CAS umum dengan kemampuan numerik dan simbolik, ruang kerja notebook	Windows
Mathematica (1986/1988)	Stephen Wolfram, Wolfram Research	14.0.0 (9-1-2024)	Komersial (194-6510 USD)	CAS umum untuk komputasi matematika dan teknik dengan kemampuan sangat luas, ruang kerja notebook	Linux, Windows, MacOS, Cloud (Web)
Mathics (2016)	Jan Pöschko	6.0.4 (6-12-2023)	Gratis (GPL3)	CAS umum sebagai alternatif gratis untuk Mathematica dan berbasis Python	Linux/BSD, Windows, MacOS, Web
Maxima (1967/1998)	Proyek MIT MAC dan Bill Schelter, dkk.	5.47.0 (31-5-2023)	Gratis (GNU GPL)	CAS umum turunan Macsyma . GUI: wxMaxima , Euler Math Toolbox (EMT), WIMS , SageMath , LabPlot2 , dll.	Linux, Windows, MacOS, Android, Web (Jupyter Notebook)
OpenAxiom (2007)	Gabriel Dos Reis	1.4.2 (1-4-2013)	Gratis (BSD termodifikasi)	CAS umum untuk komputasi simbolik, aljabar, dan numerik, turunan dari Aksioma.	Unix/Linux, Windows (MinGW/MSYS)
PARI/GP (1985/1990)	Henri Cohen, Karim Belabas, Bill Allombert dkk.	2.15.4 (27-6-2023)	Gratis (GNU GPL)	CAS khusus untuk teori bilangan	Windows, MacOS, Android, Linux/UNIX (<i>source code</i>)
Reduce (1960-an/ 1968, 2008)	Anthony C. Hearn	PSL-6658 (18-12-2023)	Gratis (BSD termodifikasi)	CAS umum untuk komputasi skalar, vektor, matriks, dan komputasi numerik, dan grafik (dengan Gnuplot)	Linux, Windows, MacOS, Web

Tabel 1.3: Daftar software CAS (... lanjutan)

Nama CAS (Pertama mulai/rilis)	Pembuat	Versi stabil terbaru (Tanggal)	Lisensi (Harga, USD)	Catatan	Sistem Operasi (Platform)
SageMath (2005)	William A.Stein	10.2.rc3 (14-11-2023)	Gratis (GNU GPL)	CAS yang menggabungkan sejumlah paket software komputasi numerik, aljabar, geometri, kalkulus, kombinatorik, teori bilangan, statistik, dan grafik	Linux, MacOS, Windows, Solaris, Android, iOS, Web (Jupyter notebook)
Scilab (1990/1994)	INRIA (sekarang bagian dari perusahaan Dassault Systèmes) Perancis	2024.0.0 (24-10-2023)	Gratis (CeCILL) (kompatibel dengan GPL) hingga versi 5.5.2 GPL v2.0 sejak versi 6.0.2	Software untuk komputasi numerik dan bahasa pemrograman tingkat tinggi, alternatif MATLAB.	Linux/BSD, Windows, MacOS
SICMUtil (2014/2016)	Colin Smith, Sam Richie	0.23.0 (9-11-2020)	Gratis (GNU GPL-3.0)	SICMUtils adalah CAS untuk matematika dan fisika dalam bahasa Clojure dan Clojurescript	Web
SINGULAR (1984/1997)	W. Decker, G.-M. Greuel, G. Pfister, H. Schoene- mann dari Universitas Kaiserslau- tern Jerman	4-3-2 (24-2-2023)	Gratis (GNU GPL)	CAS untuk perhitungan polinomial, dengan penekanan khusus pada aljabar komutatif dan non-komutatif, geometri aljabar, dan teori singularitas	multiplatform (<i>source code</i>), Linux/ UNIX, Windows (Cgywin), MacOS, Web (Web SINGU- LAR, Jupyter notebook), sebagai bagian dari MACAU- LAY2, SAGEMA- TH

Tabel 1.3: Daftar software CAS (... lanjutan)

Nama CAS (Pertama mulai/rilis)	Pembuat	Versi stabil terbaru (Tanggal)	Lisensi (Harga, USD)	Catatan	Sistem Operasi (Platform)
SMath_Studio (2004/2006)	Andrey Ivashov (SMath LLC)	1.1.8763 (29-12-2023)	Gratis (Proprietary/ <i>closed-source</i>)	Software komputasi matematika numerik dan simbolik dengan editor berupa notebook mirip Mathcad	Windows, Linux, iOS, Android, Universal Windows Platform, beberapa perangkat telepon genggam, Web
Symbolic Math Toolbox MATLAB (aka MuPAD) (1989/2008)	MuPAD research group Universitas Paderborn, Jerman, SciFace Software GmbH & Co. KG (1997), MathWorks (2008)	R2023b (2023)	Komersial	Paket (Toolbox) software MATLAB untuk komputasi matematika simbolik terintegrasi dengan antarmuka Live Editor	Linux, Windows, MacOS
Symbolics.jl (2021)	Shashi Gowda, Yingbo Ma, Chris Rackauckas	v5.3.1 (25-4-2023)	Gratis (MIT)	CAS cepat dan modern untuk bahasa pemrograman Julia	<i>multiplatform</i>
SimPy (2006/2007)	Ondrej Certik (2005) dan Aaron Meurer (Tim Pengem- bang SimPy)	1.12 (10-5-2023)	Gratis (BSD termodifikasi)	Software pustaka (paket) Python untuk komputasi matematika simbolik	<i>multiplatform</i>
Wolfram Alpha (2009)	Wolfram Research	2013	Komersial (versi Pro)	Sistem pintar berbasis Web yang mampu menjawab pertanyaan tentang matematika, sains dan teknologi, sosial budaya, serta kehidupan sehari-hari, dikembangkan menggunakan bahasa Wolfram.	Web, antarmuka dapat dipasang di PC (Windows), Android, iOS,

Tabel 1.3: Daftar software CAS (... lanjutan)

Nama CAS (Pertama mulai/rilis)	Pembuat	Versi stabil terbaru (Tanggal)	Lisensi (Harga, USD)	Catatan	Sistem Operasi (Platform)
Xcas/Giac (2000)	Bernard Parisse	1.9.0-91 (Februari 2024)	Gratis (GPL)	Xcas adalah antarmuka grafik (GUI) untuk CAS Giac,	Windows, macOS, Linux, FreeBSD, Android, iOS
Yacas (1998/1999)	Ayal Z. Pinkus, Serge Winitzki, dkk.	1.9.1 (4-7-2020)	Gratis (GNU GPL)	Yacas (<i>Yet Another Computer Algebra System</i>) adalah software CAS dan bahasa pemrograman serba guna yang kecil dan sangat fleksibel.	Linux, JAVA, Web
EMT (Euler Math Toolbox, Euler) (1987)	René Grothmann dari Universitas Katolik Eichstätt- Ingolstadt, Jerman	2024-01-12	Gratis(GNU GPLv2)	Software komputasi matematika umum dengan kemampuan komputasi numerik dan simbolik (menggunakan Maxima)	Windows, Linux (mengguna- kan emulator Windows seperti Wine)

1.3.3 Software Geometri Dinamis (DGS: *dynamic geometry system*)

Perangkat lunak **sistem geometri interaktif (IGS)** atau **lingkungan geometri dinamis (DGE)** atau *geometry dynamic system (DGS)* adalah program komputer yang memungkinkan seseorang membuat dan kemudian memanipulasi bangun-bangun geometris, terutama dalam geometri bidang. Di sebagian besar IGS, seseorang memulai mengkonstruksi suatu bangun geometri dengan meletakkan beberapa titik dan menggunakannya untuk menentukan objek baru seperti garis, lingkaran, atau titik lainnya. Setelah beberapa konstruksi selesai, seseorang dapat memindahkan titik awal dan melihat bagaimana bangun tersebut berubah.

IGS yang paling awal adalah **Geometric Supposer**, yang dikembangkan pada awal tahun 1980-an, kemudian diikuti oleh **Cabri** pada tahun 1986 dan **The Geometer's Sketchpad**.

Terdapat tiga jenis utama lingkungan komputer untuk mempelajari geometri sekolah: supposer, lingkungan geometri dinamis (DGEs) dan program berbasis Logo. Sebagian besar software geometri merupakan DGE: perangkat lunak

yang memungkinkan pengguna memanipulasi ("menye-ret") objek geometris ke dalam bentuk atau posisi berbeda. Contoh utama suatu supposer adalah **Geometry Supposer**, yang tidak memiliki objek yang dapat diseret, namun memungkinkan siswa mempelajari bentuk yang telah ditentukan sebelumnya. Hampir semua program yang dijelaskan pada bagian berikutnya merupakan DGE. Logo adalah bahasa pemrograman yang perintah-perintah dasarnya merupakan perintah-perintah untuk menggerakkan "kura-kura" dan hasil gerakannya dapat meninggalkan jejak berupa gambar, termasuk bangun-bangun geometri. Objek-objek yang digambar tidak bersifat dinamis, karena mirip menggambar pada selembar kertas.

Geometric Supposer

Geometric Supposer merupakan serangkaian program komputer mikro yang dikembangkan di Educational Development Center, Newton, MA 02160, dan diterbitkan oleh Sunburst Communication pada 1985. Program ini terdiri atas *Geometric PreSupposer: Points and Lines*, *Geometric Supposer: Triangles*, *Geometric Supposer: Quadrilaterals*, dan *Geometric Supposer: Circles*.

Geometric Supposer memungkinkan penggunaanya untuk menggambar bangun geometri yang dapat dibuat menggunakan jangka dan penggaris. Program ini memungkinkan siswa memilih bentuk primitif, seperti segitiga atau segi empat, lalu membuat bangun geometri lain dalam geometri Euclid. Program ini mengingat konstruksi sebagai prosedur yang dapat dilakukan pada bentuk primitif lain. Dengan menggunakan teknik ini, siswa dapat dengan cepat dan mudah memeriksa berbagai bangun dengan berbagai jenis segitiga atau segi empat.

Geometric Supposer memungkinkan siswa membuat titik, ruas garis, lingkaran, garis-garis sejajar, garis-garis saling tegak lurus, garis bagi sudut, dan perpanjangan segmen garis. Jika bentuk primitifnya adalah segitiga, maka konstruksi median, segmen tengah, ketinggian, dan garis bagi yang tegak lurus dapat dibuat. Program ini mencakup opsi untuk memberi label berbagai jenis titik, seperti titik potong, titik acak (pada, di luar, dan di bagian dalam suatu bangun), dan pantulan titik dan ruas garis. Titik dan ruas garis dapat dengan mudah dihapus. Selain konstruksi prosedural, program ini juga dapat mengukur sudut dan panjang ruas garis, menghitung keliling dan luas segitiga dan segi empat, membagi ruas menjadi bagian-bagian yang sama panjang, dan buatlah gambar skala.

Program ini tidak banyak dikenal dan informasi tentang software ini juga sangat terbatas, tidak tersedia situs softwarena.

Cabri

Cabri (*Interactive Notebook*), yang merupakan software IGS pertama di dunia, dikembangkan oleh sebuah tim peneliti Perancis yang diketuai Jean-Marie Laborde, Direktur *Research for the National Science Research Center* (CNRS), pada

1986 untuk platform komputer Apple Macintosh. **Cabri Geometry** (nama aslinya **Cabri Géomètre** atau **Cabri I**) merupakan perangkat lunak geometri interaktif komersial dan diproduksi oleh perusahaan Perancis **Cabrilog**. Perusahaan Cabrilog didirikan oleh Jean-Marie Laborde dan Max Marcadet pada 2000 di Grenoble, Perancis, dan mengembangkan **Cabri II Plus** dan **Cabri 3D**.

Tabel 1.4: Daftar harga lisensi software Cabri

Software Cabri	Lisensi Personal 1 Tahun	Lisensi Personal Tanpa Batas	Lisensi Kelas 1 Tahun	Lisensi Kelas Tanpa Batas	Lisensi Lembaga
Cabri II Plus	25 euro + PPN (1 PC)	100 euro + PPN (1 PC)	-	275 euro + PPN (10 PC sekolah)	165 euro + PPN (semua PC sekolah 1 tahun) 570 euro + PPN (semua PC sekolah, tanpa batas) 249 euro + PPN (semua PC sekolah dan murid, 1 tahun) 950 euro (semua PC sekolah dan murid, tanpa batas)
Cabri 3D	25 euro + PPN (1 PC)	75 euro + PPN (1 PC)	-	270 euro + PPN (10 PC sekolah)	165 euro + PPN (semua PC sekolah 1 tahun) 550 euro + PPN (semua PC sekolah, tanpa batas) 249 euro + PPN (semua PC sekolah dan murid, 1 tahun) 900 euro (semua PC sekolah dan murid, tanpa batas)
New Cabri	39 euro + PPN (2 pengguna)	149 euro + PPN (2 pengguna)	199 euro + PPN (42 pengguna)	699 euro + PPN (42 pengguna)	sesuai banyak pengguna
Cabri Express	Gratis	Gratis	Gratis	Gratis	Gratis

Cabri adalah perangkat lunak geometri dinamis yang memungkinkan pengguna untuk melakukan konstruksi geometri, eksplorasi, dan demonstrasi konsep matematika secara interaktif. Perangkat lunak ini dirancang untuk memungkinkan pengguna secara visual dan intuitif memahami prinsip-prinsip geometri dan matematika.

Program ini dirancang dengan mempertimbangkan kemudahan penggunaannya, digunakan untuk pembelajaran geometri dan trigonometri. Program ini memungkinkan pengguna untuk menganimasikan bentuk geometris, menunjukkan keunggulan program ini dibandingkan dengan gambar yang dibuat di papan tulis. Pendekatan geometri dinamis yang diusung oleh Cabri memungkinkan pengguna untuk secara interaktif memanipulasi objek geometris dan mengamati

perubahan yang terjadi secara langsung. Hubungan antar titik pada suatu objek geometris dapat dengan mudah didemonstrasikan, sehingga dapat berguna dalam proses pembelajaran. Terdapat juga fasilitas untuk menggambar grafik fungsi yang memungkinkan eksplorasi hubungan antara geometri dan aljabar. Program ini dapat dijalankan pada Windows atau Mac OS.

Program ini juga sudah diimplementasikan pada beberapa jenis kalkulator, seperti Texas Instruments TI-92 (1995), TI-83, TI-84 (2003), dan juga tersedia pada tablet Android, iPad, dan Windows (2014), serta pada 2016 tersedia juga Cabri berbasis Web yang gratis (**Cabri Express**). Selain dapat diakses melalui Web (*Cloud*) Cabri Express juga dapat diinstal dan dijalankan secara *offline* pada PC. Perkembangan terakhir adalah *New Cabri*, yang merupakan kumpulan alat yang handal untuk pengembangan konten pendidikan, untuk mengerjakan **semua matematika** (aljabar, grafik fungsi, geometri bidang, geometri ruang, statistika, trigonometri, dll.). *New Cabri* dapat dijalankan pada PC Windows maupun Mac OS.

The Geometer's Sketchpad (GSP)

The Geometer's Sketchpad (GSP) adalah perangkat lunak geometri dinamis yang dirancang untuk membantu siswa dan guru memahami konsep geometri melalui eksplorasi visual dan interaktif. **GSP** adalah program perangkat lunak geometri interaktif komersial untuk menjelajahi geometri Euclidean, aljabar, kalkulus, dan bidang matematika lainnya. Program ini dibuat sebagai bagian dari **Proyek Geometri Visual** yang dipimpin oleh Eugene Klotz dan Doris Schattschneider dari tahun 1986 hingga 1991 di Swarthmore College dan didanai **NSF**. Nicholas Jackiw, seorang mahasiswa pada saat itu, adalah perancang dan pemrogram asli perangkat lunak tersebut, dan penemu pendekatan "geometri dinamis" yang menjadi merek dagangnya. Ia kemudian pindah ke **Key Curriculum Press**, **KCP Technologies**, dan **McGraw-Hill Education** untuk melanjutkan desain dan implementasi perangkat lunak yang sedang berlangsung pada berbagai rilis besar dan platform perangkat keras.

Versi terakhir GSP adalah 5.06 yang dirilis 26 April 2013. Versi GSP ini berjalan di Microsoft Windows dan MacOS Ventura. Software ini juga berjalan di Linux (melalui emulator Wine) namun memiliki beberapa *bug*. Terdapat juga versi GSP yang dikembangkan untuk seri Kalkulator TI-89 dan TI-92. Pada bulan Juni 2019, McGraw-Hill mengumumkan bahwa mereka tidak lagi menjual lisensi baru untuk GSP. Dengan demikian sekarang GSP, baik versi Windows maupun MacOS, dapat diunduh (melalui situs **KeyCurriculum McGraw-Hill Education**) dan digunakan secara gratis (bebas lisensi) meskipun bersifat *closed source*.

KeyCurriculum juga menyediakan sumber-sumber aktivitas (file-file yang dikembangkan dengan software GSP) yang dikelompokkan berdasarkan usia (3-5 tahun, 6-8 tahun) maupun topik matematika (aljabar, geometri, pengantar kalkulus, dan kalkulus). File-file ini dapat diunduh secara gratis. Selain itu juga

tersedia beberapa tutorial penggunaan GSP dan mengembangkan aktivitas pembelajaran dengan GSP. Situs **Resource Center GSP** menyediakan informasi dan sumber-sumber aktivitas berbasis GSP serta tutorial yang sekarang juga sudah dapat diunduh secara gratis. Repositori Sketchpad berisi lebih dari 200 video, dengan tutorial Sketchpad dan Web Sketchpad serta arsip webinar Sketchpad yang ditawarkan oleh Key Curriculum Press.

McGraw-Hill Education juga memproduksi teknologi berbasis Web, **Web Sketchpad (WSP)** yang didasarkan pada software GSP dan dapat diakses serta digunakan secara gratis. WSP semula merupakan bagian dari proyek Dynamic Number yang didanai NSF DRK-12. WSP terdiri atas kumpulan software (*tool library*) yang mudah digunakan dan ditargetkan secara tepat untuk membuat aktivitas yang mirip di GSP, namun dengan antarmuka yang sederhana, dan juga untuk membuka/ menampilkan file GSP. WSP menghilangkan antarmuka yang mengganggu—tidak memiliki menu, tidak menampilkan kotak dialog, dan tidak mengharuskan pengguna untuk “memilih” objek untuk menerapkan perintah pada objek tersebut. Aktivitas WSP kompatibel dengan browser web modern apa pun yang berkemampuan HTML5, baik pada perangkat seluler atau komputer desktop, tidak memerlukan perangkat lunak atau lisensi lain. McGraw-Hill Education menjadikan WSP gratis untuk penggunaan non-komersial.

Cinderella

Cinderella, semula merupakan software geometri dinamis komersial dan awalnya dikembangkan oleh Jürgen Richter-Gebert dan Henry Crapo, digunakan untuk memasukkan teorema dan konjektur titik potong guna pembuktian teorema secara otomatis menggunakan metode pembuktian binomial Richter-Gebert. Software awalnya ditulis dengan bahasa Objective-C] pada komputer NeXT.

Pada tahun 1996, software ini ditulis ulang dari awal dengan bahasa Java oleh Jürgen Richter-Gebert dan Ulrich Kortenkamp. Program ini masih memasukkan pembuktian binomial, tetapi belum cocok untuk pembelajaran di kelas karena masih berupa prototipe. Versi ini memenangkan Penghargaan Inovasi Multimedia di Learntec '97 di Karlsruhe, Jerman. Setelah itu, penerbit perangkat lunak pendidikan Jerman Heureka-Klett dan penerbit ilmiah Springer-Verlag, Heidelberg setuju untuk memproduksi versi komersial perangkat lunak tersebut. Versi sekolah Cinderella 1.0 diterbitkan pada tahun 1998, menyertakan sekitar 150 contoh, animasi dan latihan yang dibuat dengan Cinderella, sedangkan versi universitas dirilis pada tahun 1999.

Pada tahun 2006, versi baru Cinderella, yakni **Cinderella.2**, diterbitkan dalam versi online saja. Manual cetak untuk versi 2.6 telah diterbitkan oleh Springer-Verlag pada tahun 2012. Dokumentasi lengkap Cinderella.2 tersedia online di <https://doc.cinderella.de/tiki-index.php>. Pada tahun 2013, versi pro Cinderella telah tersedia secara gratis. Versi Cinderella.2 yang lebih baru juga menyertakan mesin simulasi fisika dan bahasa skrip. Cinderella.2 dapat dijalankan pada

komputer dengan sistem operasi Windows, Mac OS, Linux, dan Solaris. Versi terbaru (versi 3 masih dalam tahap pengembangan atau *beta* dirilis 21 Februari 2024) tersedia gratis, baik untuk Windows 32/64 bit, Unix/Linux, maupun Mac OS, dan dapat diunduh dari situs <https://beta.cinderella.de/>. Cinderella juga mendukung menggunakan alat masukan pena untuk PC tablet,

Selain sebagai aplikasi geometri interaktif untuk geometri Euclid, geometri hiperbolik, dan geometri eliptik, Cinderella.2 (juga versi 3 beta yang sekarang) dilengkapi dengan dua bagian lain, yakni: CindyLab, perangkat untuk melakukan eksperimen fisika secara interaktif, dan CindyScript, bahasa pemrograman tingkat tinggi yang memungkinkan skenario interaksi secara cepat, fleksibel, dan dapat diprogram secara bebas. Selain itu, Cinderella juga dapat diperluas kemampuannya dengan program tambahan (*plugins*) yang berupa program kumpulan kelas dalam bahasa Java yang sudah dikompail menjadi file `.jar`. Plugin ini dapat diakses melalui CindyScript.

Informasi yang lebih lengkap tentang software Cinderella dapat dilihat melalui situs-situs berikut ini.

- <https://cinderella.de/>: Situs resmi Cinderella Interactive Geometry.
- Download Cinderella: Situs distribusi Cinderella versi terkini (untuk Windows 32/64 bit, Mac OS X, Unix, Linux).
- Cinderella Documentation: Cinderella Documentation: Situs resmi Panduan dan Tutorial Cinderella.
- Cinderella (software) - Wikipedia: informasi software Cinderella di Wikipedia.
- (20+) The Interactive Geometry Software Cinderella | Facebook: akun resmi software Cinderella di Facebook.
- Cindy 3D (gagern.github.io): Repositori plugin Cindy 3D untuk Cinderella.
- CindyJS: situs resmi CindyJS (aplikasi JavaScript, analog CindyScript yang digunakan di Cinderella), untuk mengembangkan media pembelajaran matematika dan simulasi fisika interaktif.
- CindyJS Project (github.com): repositori software-software aplikasi berbasis CindyJS di **Github.com**.
- MatheVital (mathe-vital.de): koleksi materi matematika interaktif yang dikembangkan dengan menggunakan Cinderella.2" dan CindyJS.
- Toolbox Teacher Education (tum.de): koleksi media pembelajaran matematika interaktif berbasis CindyJS.
- ALICE:fractions (tum.de): koleksi media pembelajaran pecahan interaktif berbasis CindyJS.
- <https://github.com/IMAGINARY/cinderella-applets-2011-edition>
Koleksi applet (contoh-contoh) media yang dibuat dengan Cinderella 2.

CaR (Compas and Ruler)

Software **C.a.R.** (*Compass and Ruler*) adalah sebuah paket program DGS dalam bentuk Applet Java yang dapat digunakan untuk melakukan simulasi pengkonstruksian bangun-geometri sekolah seperti halnya menggunakan jangka dan penggaris. Software ini dapat dijalankan pada hampir semua komputer modern, baik versi Windows, Macintosh, maupun Unix. Software ini juga dapat menampilkan hasil konstruksi geometri di media online (Internet/intranet) maupun media offline (CD-ROM) dan dijadikan latihan interaktif. Software ini memiliki fitur-fitur seperti alat-alat geometri dasar (seperti titik, garis, lingkaran, dll), alat-alat transformasi (seperti translasi, rotasi, refleksi, dll), alat-alat pengukuran (seperti jarak, luas, sudut, dll), alat-alat analisis (seperti teorema Pythagoras, teorema Thales, dll), alat-alat visualisasi (seperti warna, label, ukuran, dll), alat-alat perintah (untuk membuat konstruksi secara visual atau teks), alat-alat jejak (untuk menggambar jejak perjalanan suatu titik yang bergerak), dan alat-alat lainnya. Anda dapat mengunduh software C.a.R. secara gratis dari situs web resminya <http://car.rene-grothmann.de/>.

C.a.R. juga dikenal sebagai **Z.u.L.**, yang merupakan singkatan dari bahasa Jerman "*Zirkel und Lineal*" — adalah aplikasi geometri interaktif sumber terbuka dan gratis yang dapat digunakan untuk mengonstruksi bangun-geometri dalam geometri Euclidean maupun geometri non-Euclidean. Perangkat lunak ini berbasis bahasa pemrograman Java. Programernya adalah René Grothmann dari Catholic University of Eichstätt-Ingolstadt Jerman, yang juga programmer software EMT yang digunakan di dalam buku ini. Software ini disebarluaskan dengan lisensi berbasis Lisensi Publik Umum GNU (GPL). C.a.R. merupakan alat yang ampuh untuk mengajar dan mengeksplorasi geometri baik di sekolah maupun perguruan tinggi.

Program C.a.R. ditulis dalam bahasa Java mulai tahun 1996, bersamaan dengan hadirnya bahasa pemrograman tersebut. Versi pertama menggunakan Java 1.0 dan versi-versi berikutnya mengikuti Java sepanjang sejarahnya. Java sudah banyak berubah sejak saat itu, sebagian besar perubahan menuju kebaikan. Lingkungan pemrogramannya menjadi lebih cepat, lebih kaya, dan lebih dapat diandalkan. Gambar grafik menjadi lebih halus. Beberapa perubahan dalam bahasa memerlukan penulisan ulang C.a.R. secara besar-besaran, selain menambahkan fitur baru dan disambut oleh penggunanya dengan baik.

Sebelumnya terdapat versi C.a.R., atau lebih tepatnya pendahulunya. Program geometri dinamis pertama yang dibuat oleh programmer yang sama adalah untuk Atari ST dengan bahasa C, meniru gaya pemrograman berorientasi objek pada sekitar tahun 1989. Ketika Atari ST dan sistem operasi GEM-nya mati mendadak, program kemudian dipindahkan ke OS/2, dan pada saat itu programnya berjalan lambat di mesin IBM pertama yang dirancang untuk DOS. OS/2 segera kalah dalam persaingan melawan Windows. Versi Windows C.a.R. lebih merupakan penulisan ulang daripada proses pemindahan, dengan menggunakan C++, ba-

hasa pemrograman berorientasi objek (OOP). Versi Windows sudah agak maju waktu itu, namun tidak dapat dibandingkan dengan versi Java yang digunakan sekarang.

Menurut Rene, program C.a.R. ditulis lebih untuk hobi. Pada awalnya, penulisnya tidak berpikir bahwa program ini akan menjadi alat yang berguna, sehingga penulisnya tidak tertarik untuk mengembangkan fitur terlalu jauh. Versi Java tersebut hanya tersimpan selama sekitar dua tahun, sampai Hans Fischer menanyakannya, mengapa proyek tersebut tidak dilanjutkan. Menurut Hans, program-program yang tersedia tidak memiliki antarmuka yang sebagus dan semudah C.a.R.

Hans sangat tertarik dengan Web. Pada saat itu dan sampai beberapa tahun kemudian, Java adalah bahasa pemrograman yang hebat untuk mengembangkan aplikasi berbasis Web, dalam bentuk applet Java. Meskipun pada waktu itu Flash tampaknya lebih cepat daripada Applet, kombinasi aplikasi Java dan applet Java untuk web tidak dapat dikalahkan dengan mudah. Akibatnya, sebagian besar upaya pengembangan C.a.R. difokuskan pada kemampuan ekspor, ekspor web, dan juga ekspor grafik. Baru kemudian ditambahkan kemampuan ekspor LaTeX.

Versi terakhir C.a.R. adalah 2023-3 yang dirilis pada 15 Maret 2023 dapat diunduh dari situs **Sourceforge** (<https://sourceforge.net/projects/zirkel/files/>).

CaRMetal

CaRMetal adalah program geometri interaktif gratis yang diturunkan dari C.a.R. CaRMetal dibuat dan dikembangkan oleh Eric Hakenholz, Patrice Debrabant, Pierre-Marc Mazat, dan Alain Busser dengan menggunakan bahasa pemrograman Java. Perangkat lunak ini mempertahankan beberapa fungsi C.a.R, tetapi menggunakan antarmuka grafis berbeda yang dirancang untuk menghapus beberapa dialog perantara. Konstruksi dilakukan dengan menggunakan palet utama, yang berisi beberapa pintasan konstruksi yang berguna selain alat jangka dan penggaris standar. Ini termasuk garis bagi tegak lurus, lingkaran melalui tiga titik, busur lingkaran melalui tiga titik, dan bagian kerucut melalui lima titik. Yang juga menarik adalah lokus, fungsi, kurva parametrik, dan plot implisit. Ketebalan elemen, warna, label, dan atribut lainnya (termasuk yang disebut sifat magnetik) dapat diatur menggunakan panel terpisah.

CaRMetal juga mendukung palet konstruksi terbatas yang dapat dikonfigurasi dan memiliki kemampuan penugasan, yang menggunakan fitur unik yang disebut *Monkey*. CaRMetal memiliki bahasa skrip (JavaScript) yang memungkinkan pengguna membuat gambar yang agak rumit seperti fraktal. Versi terakhir CaRMetal adalah 4.3, yang dirilis 11 September 2020, dan dapat diunduh dari situs **UptoDown** (<https://carmetal.en.uptodown.com/windows>). Sejauh ini tidak ditemukan informasi perkembangan terkini untuk software ini, bahkan situs resminya juga sudah tidak aktif lagi. Di antara informasi tentang CaRMetal yang masih dapat diakses adalah Wikipedia dan situs ar-

sip CaRMetal, https://web.archive.org/web/20170707071912/http://db-maths.nuxit.net/CaRMetal/index_en.html (hanya menyediakan informasi CaRMetal sampai versi 3.8.2).

GeoGebra

GeoGebra (nama yang menggabungkan kata *Geometry* dan *Algebra*) semula merupakan software DGS, namun sekarang sudah menjadi software aplikasi terpadu untuk geometri, aljabar, statistika, dan kalkulus interaktif, yang ditujukan untuk pembelajaran matematika dan sains mulai dari tingkat sekolah dasar hingga universitas. GeoGebra tersedia di berbagai *platform*, dengan aplikasi untuk desktop (Windows, macOS dan Linux), tablet (Android, iPad dan Windows) dan web. Saat ini GeoGebra dimiliki oleh perusahaan pendidikan India, **Byju's**.

Pencipta GeoGebra, Markus Hohenwarter, memulai proyek ini pada tahun 2001 sebagai bagian dari tesis masternya di Universitas Salzburg. Setelah kampanye Kickstarter yang sukses, GeoGebra memperluas ketersediaannya dengan menyertakan versi aplikasi iPad, Android, dan Windows Store. Pada tahun 2013, GeoGebra memasukkan Xcas karya Bernard Parisse ke dalam tampilan CAS-nya. Proyek ini sekarang bersifat *freeware* (bagian sumber terbuka) dan multibahasa, dan Hohenwarter terus memimpin pengembangannya di Universitas Linz.

GeoGebra meliputi dua organisasi komersial dan nirlaba yang bekerja sama dari kantor pusat di Linz, Austria, untuk memperluas perangkat lunak dan layanan *cloud* yang tersedia bagi para pengguna. **International GeoGebra Institute (IGI)** adalah cabang nirlaba dari GeoGebra Group. Lembaga ini mengoordinasikan upaya penelitian, pengembangan, penerjemahan dan penerapan sistem GeoGebra di seluruh jaringan global kelompok pengguna di universitas dan organisasi nirlaba, serta memberikan sertifikasi kepada para ahli dan pelatih GeoGebra.

Pada bulan Desember 2021, GeoGebra diakuisisi oleh konglomerat *edtech* **Byju's** dengan nilai sekitar 100 juta USD. Pada tanggal 25 Januari 2024, pemberi dana pinjaman memproses pailit perusahaan induk GeoGebra, yakni Byju's, dalam upaya untuk mendapatkan kembali dana pinjamannya. Pada tanggal 1 Februari 2024, divisi Byju di AS mengajukan penundaan tuntutan pailit dengan Bab 11 Undang-undang Pailit USA di Delaware. Byju's akan mengumpulkan sekitar \$200 juta dalam upaya melunasi "kewajiban langsung" dan biaya operasional lainnya.

GeoGebra merupakan serangkaian perangkat lunak matematika interaktif untuk pembelajaran sains, teknologi, teknik, dan matematika dari tingkat sekolah dasar hingga universitas. Konstruksi dapat dibuat dengan titik, vektor, segmen, garis, poligon, irisan kerucut, pertidaksamaan, polinomial implisit, dan fungsi, yang semuanya dapat diedit secara dinamis. Setiap objek dapat dikonstruksi dan dimodifikasi menggunakan mouse dan sentuhan, atau melalui bilah input. GeoGebra dapat menggunakan variabel-variabel untuk menyimpan bilangan, vektor dan titik, menghitung turunan dan integral fungsi, serta memiliki perintah

lengkap seperti *Root* atau *Extremum*. Guru dan siswa dapat menggunakan GeoGebra sebagai alat bantu dalam merumuskan dan membuktikan dugaan geometri.

Fitur utama GeoGebra adalah mencakup:

- lingkungan geometri interaktif (2D dan 3D);
- spreadsheet (lembar kerja elektronik);
- sistem aljabar komputer (CAS);
- alat statistik dan kalkulus;
- pemrograman dan perintah skrip;
- sumber belajar dan pengajaran interaktif di **GeoGebra Materials**.

Situs GeoGebra (www.geogebra.org) menyediakan layanan **Platform Materials GeoGebra** yang memungkinkan pengguna mengunggah dan berbagi *applet* GeoGebra dengan orang lain. Semula, layanan ini diluncurkan sebagai **GeoGebraTube** pada bulan Juni 2011, dan diganti namanya pada tahun 2016. Pada bulan April 2016, layanan ini menampung lebih dari 1 juta sumber daya, 400.000+ di antaranya bersifat publik. “Materi” meliputi lembar kerja interaktif, simulasi, permainan dan *e-book* (buku interaktif) yang dibuat menggunakan **GeoGebraBook**.

Materi GeoGebra juga dapat diekspor dalam beberapa format, termasuk SVG, GIF Animasi, Windows Metafile, PNG, PDF dan EPS, serta disalin langsung ke *clipboard*. GeoGebra juga dapat menghasilkan kode untuk digunakan dalam file LaTeX.

Sebelum versi 4.2, GeoGebra diterbitkan di bawah Lisensi Publik Umum GNU (GPL-3.0+). Setelah versi 4.2, lisensi diubah. Kode sumber GeoGebra, kecuali *instaler*, layanan web, file-file gambar dan pengaturan antarmuka pengguna, serta file dokumentasi dan bahasa, dilisensikan di bawah Lisensi Publik Umum GNU (GPL-3.0-atau-yang lebih baru). Instal器和 layanan web dirilis di bawah lisensi milik GeoGebra sendiri. File gambar dan pengaturan antarmuka pengguna, serta file dokumentasi dan bahasa diterbitkan di bawah lisensi Creative Commons NonCommercial (CC BY-NC-SA 3.0). Penggunaan komersial diatur dengan lisensi khusus dan perjanjian kerja sama.

Versi terakhir GeoGebra adalah 6.0.826 yang dirilis 13 Februari 2024, dan dapat dijalankan pada sistem operasi Windows, macOS, ChromeOS, Linux. Selain itu, GeoGebra juga dapat dijalankan melalui web dari situs www.geogebra.org.

Dr. Geo

Dr. Geo GNU adalah perangkat lunak geometri interaktif yang ditujukan untuk siswa sekolah (usia 7-15 tahun), dan memungkinkan penggunaannya menggambar & memanipulasi bangun-geometri secara interaktif, termasuk model Fisika dinamis. Dengan Dr. Geo pengguna dapat mempelajari dan memodifikasi bahkan memperluas kode programnya sambil menggunakannya untuk membuat

gambar bangun geometri.

Software ini merupakan perangkat lunak gratis (kode sumber, terjemahan, ikon dan penginstal dirilis di bawah lisensi GNU GPL). Dr. Geo dibuat oleh Hilaire Fernandes, dan merupakan bagian dari proyek GNU. Program ini dapat berjalan pada sistem grafis Morfik (yang berarti berjalan di Linux, Mac OS, Windows, Android). Dr Geo awalnya dikembangkan dalam bahasa C++ dengan skrip *Scheme* kemudian ditulis ulang dalam berbagai versi bahasa pemrograman Smalltalk dengan Squeak, Etoys untuk proyek Pharo **Satu Laptop per Anak**, kemudian juga menggunakan bahasa Cuis-Smalltalk. Dr. Geo pertama kali dirilis pada 31 Desember 1996. Versi terakhir adalah Dr. Geo 23.12a-beta yang dirilis pada 24 Desember 2023.

Sama seperti software DGS lain, Dr Geo memanipulasi berbagai jenis objek seperti titik, garis, lingkaran, vektor, nilai, transformasi geometris, skrip. Dr. Geo memiliki beberapa jenis titik: titik bebas, yang dapat digerakkan dengan mouse (tetapi dapat ditempelkan pada kurva) dan titik yang ditentukan oleh koordinatnya. Titik juga dapat sebagai hasil perpotongan dua kurva atau sebagai titik tengah suatu segmen. Dr. Geo dilengkapi dengan garis klasik, sinar, segmen dan vektor. Objek lengkung lainnya termasuk lingkaran (ditentukan oleh 2 titik, pusat dan ruas atau jari-jari), busur (ditentukan oleh tiga titik atau pusat dan sudut), poligon (beraturan atau tidak, ditentukan oleh titik sudut), dan lokus. Garis juga dapat ditentukan oleh objek geometri lain, misalnya garis sejajar atau garis yang tegak lurus garis lain dan melalui suatu titik. Dr. Geo dapat menerapkan salah satu transformasi berikut pada titik atau garis: refleksi, simetri, translasi, rotasi, dilatasi.

Dr. Geo hadir dengan konstruksi makro: cara untuk mengajari Dr. Geo konstruksi baru. Dimungkinkan untuk menambahkan objek baru ke Dr. Geo: transformasi baru seperti inversi lingkaran (transformasi yang memetakan lingkaran atau garis ke lingkaran atau garis lain dan yang mempertahankan sudut-sudut antara kurva-kurva yang berpotongan), konstruksi membosankan yang melibatkan banyak objek perantara atau konstruksi yang melibatkan skrip (juga disebut skrip makro).

Ketika beberapa objek, yang disebut *final*, bergantung pada objek lain, yang disebut *inisial*, dimungkinkan untuk membuat konstruksi kompleks yang menghasilkan objek akhir dari objek awal yang diberikan pengguna. Ini adalah konstruksi makro, grafik objek yang saling bergantung.

Akses ke pemrograman pengguna adalah inti dari Dr. Geo: dari program aplikasi, pengguna dapat langsung membaca, mempelajari, memodifikasi, dan mendistribusikan ulang versi Dr. Geo yang telah dimodifikasi. Selain itu juga tersedia pemrograman skrip yang dipadukan dengan objek geometri.

Dr. Geo terbaru ditulis dalam bahasa Smalltalk, yang juga merupakan bahasa pemrograman bagi pengguna untuk memperluas Dr. Geo dengan operasi kom-

putasi sebarang (skrip Smalltalk) dan untuk mendefinisikan sketsa geometris yang sepenuhnya menggunakan instruksi pemrograman (sketsa Smalltalk).

Geometry Expressions

Geometry Expressions adalah sistem geometri simbolik interaktif pertama di dunia yang menyajikan representasi aljabar dan diagram model dengan sketsa berbasis kendala yang mudah. Bangun geometri dapat digambar dengan menggunakan kendala simbolik (misalnya persamaan) atau nilai-nilai numerik (misalnya titik koordinat). Pengukuran pada gambar disajikan secara numerik maupun simbolik dalam bentuk ekspresi matematika. Selain dapat dijalankan secara mandiri, *Geometry Expressions* dapat dijalankan melalui sistem aljabar komputer (CAS) melalui sistem *input* dan *output* MathML, serta mampu mengekspor dan berbagi melalui aplikasi widget dasbor HTML5, Lua, dan OS X yang interaktif. Ekspresi matematika simbolik yang dihasilkan oleh *Geometry Expressions* dapat dikirim ke *Mathematica* untuk dianalisis, kemudian disalin dan dimasukkan kembali ke dalam model *Geometry Expressions* (Gx). Selanjutnya, model dapat diubah menjadi aplikasi browser JavaScript/HTML atau *widget* dasbor tanpa perlu menulis skrip. Jika *Mathematica* dapat digunakan untuk mengotomatiskan analisis ekspresi matematika, *Geometry Expressions* mengotomatiskan pembuatan ekspresi simbolik dalam geometri.

Fitur utama *Geometry Expressions* adalah:

- Geometri simbolik berbasis kendala (ekspresi simbolik);
- antarmuka pengguna berbasis sketsa;
- animasi penuh;
- perhitungan lokus dan kurva singgung (*envelope*);
- perhitungan simbolik parameter seperti jarak, sudut, luas, koordinat, kemiringan, dan radius;
- perhitungan simbolik persamaan implisit dan eksplisit untuk lokus;
- kemampuan ekspor *Mathematica* dan MathML asli;
- aplikasi browser JavaScript/HTML otomatis dan pembuatan *widget* dasbor;
- impor ekspresi *Mathematica* menggunakan salin dan tempel (*Copy* dan *Paste*).

Tabel 1.5: Daftar harga lisensi software Saltire

	Geometry Expressions	Math Illustrations	Mechanical Expressions
Lisensi Personal (Murid/Guru)	59 USD (2 PC)	35 USD (2 PC)	59 USD (untuk murid)
Lisensi Akademik	349 USD (2 PC)	-	495 USD (untuk guru/dosen)

Tabel 1.5: Daftar harga lisensi software Saltire

	Geometry Expressions	Math Illustrations	Mechanical Expressions
Lisensi Industri/ Komersial	595 USD (2 PC)	-	995 USD (untuk ilmuwan dan insinyur di industri)
Lisensi Sekolah	1449 USD (2 PC)	-	<i>quote</i>

Geometry Expressions menyediakan berbagai fungsi yang dapat digunakan sebagai landasan dasar untuk mengeksplorasi hubungan spasial dan mempelajari konsep geometris.

- **Geometri Primitif:** titik, garis, ruas garis, vektor, poligon, lingkaran, elips, parabola, hiperbola, busur, segi- n , aproksimasi kurva, fungsi, fungsi sepotong-sepotong;
- **Konstruksi:** titik tengah, perpotongan, garis sumbu, garis bagi sudut, garis sejajar, garis tegak lurus, garis singgung, kurva lokus, kurva singgung (*envelope*);
- **Kendala Kuantitatif:** jarak, panjang, radius, sudut, arah, kemiringan, koordinat, vektor koefisien, proporsi ruas garis atau kurva, persamaan implisit;
- **Kendala Kualitatif:** tegak lurus, sejajar, menyinggung, kongruen (ruas garis) (segmen);
- **Transformasi:** refleksi, rotasi, dilatasi, translasi;
- **Pengukuran:** jarak, panjang, radius, sudut, arah, gradien, koordinat, luas, keliling, vektor koefisien, persamaan parametrik kurva, persamaan implisit kurva;
- **Ekspor Impor:** kirim ekspresi langsung ke *Mathematica*; impor & ekspor MathML; ekspor gambar: BMP, JPEG, TIFF, PNG, XMPE; ekspor PostScript yang dienkapsulasi; ekspor Windows Metafile; ekspor grafik vektor yang dapat diskalakan; ekspor HTML; ekspor animasi (GIF); ekspor aplikasi HTML5/JavaScript; ekspor aplikasi Lua; ekspor widget Dasbor OS X.

Geometry Expressions (Gx) merupakan software komersial yang dikembangkan dan didukung oleh **Saltire Software**, AS. Saltire Software didirikan pada tahun 1989 sebagai pendukung **Tektronix Computer Research Lab**. Versi terbaru Gx adalah **Geometry Expressions 3.2**, tersedia untuk sistem operasi Windows dan MacOS. Tersedia juga versi ringan Gx tanpa fitur simbolik, yakni Math Illustrations. Versi Gx yang diperluas dengan menambahkan fitur-fitur dalam mekanika (Fisika) seperti gaya, massa, kecepatan, dan percepatan ke dalam model geometri adalah Mechanical Expressions. Versi ini memungkinkan Anda mendapatkan hasil simbolik untuk gaya reaksi dan persamaan gerak, bahkan untuk membuat simulasi dinamis sebagai aplikasi. Selain versi-versi desktop,

Geometry Expressions saat ini juga tersedia versi Web, yang dapat diakses secara gratis, yakni **GxWeb** (<https://geometryexpressions.com/gxweb/>).

GEUP

GEUP merupakan software DGS komersial untuk komputer Windows. Aslinya menggunakan bahasa Spanyol, diprogram oleh Ramón Alvarez Galván. Versi terbarunya adalah **GEUP 10.0.1.1**. Terdapat juga **GEUP3D** untuk geometri tiga dimensi, versi terakhirnya adalah **GEUP3D 9.0.1.1**. Konstruksi dan perhitungan serta visualisasi interaktif

GEUP 10 adalah perangkat lunak geometri bidang interaktif untuk visualisasi matematika dan untuk melakukan perhitungan matematika dinamis. Hal ini memungkinkan untuk membuat konstruksi/aplikasi yang dinamis dan umum secara visual dengan mendefinisikan elemen matematika melalui alat konstruksi yang disertakan. GEUP 10 memungkinkan modifikasi konstruksi secara visual (langsung di layar) dan menghitung setiap kasus tertentu secara real time. Kisaran penerapan GEUP 10 sangat luas, berikut penjelasan aplikasi utamanya.

Elemen geometris yang ditentukan oleh alat dasarnya memverifikasi aksioma geometri Euclidean dan seperangkat alat pada tingkat yang sama memungkinkan penerapannya dalam geometri analitik, transformasional, dan fraktal. Juga kemampuan untuk menentukan alat konstruksi Anda sendiri (makro) melalui penerapan alat konstruksi bawaan, memungkinkan untuk membuat aplikasi geometri tentang geometri non-Euclidean.

Kemampuan untuk mendefinisikan elemen non-geometris memungkinkan penerapan konsep konstruksi dalam Aljabar, Kalkulus, dll. Gunakan potensinya untuk membuat perhitungan dan visualisasi matematika yang interaktif dan dinamis dan Anda akan menemukan cara baru untuk mempelajari Matematika dalam berbagai bidang.

Penggunaan makro, iterasi dan skrip: kemampuan untuk menentukan alat konstruksi Anda sendiri (makro), mengulangi konstruksi, dan membuat skrip memungkinkan untuk menyederhanakan pembuatan konstruksi yang kompleks. Fungsionalitas skrip memungkinkan untuk membuat konstruksi GEUP melalui skrip. Pembuatan konstruksi melalui skrip memungkinkan algoritma pemrograman untuk mengotomatisasi proses konstruktif yang kompleks dan menerapkannya secara sistematis, kondisional, dan berulang. Berikut adalah harga lisensi GEUP10 dan SEUP3D9.

1. Untuk lisensi personal: GEUP10 (24.95 euro), GEUP3D9 (24.95 euro), GEUP10+GEUP3D9 (49.00 euro).
2. Untuk lisensi institusi (lembaga pendidikan/perusahaan): GEUP10 (125 euro), GEUP3D9 (125 euro), kombinasi GEUP10+GEUP3D9 (245 euro).

1.3.4 Software Analisis Numerik

Software analisis numerik adalah program komputer yang dapat digunakan untuk melakukan perhitungan-perhitungan numerik (hasilnya dalam bentuk bilangan desimal atau *floating point*) serta dilengkapi dengan bahasa pemrograman untuk mengimplementasikan algoritma-algoritma di dalam metode numerik. Metode numerik itu sendiri merupakan metode penyelesaian masalah-masalah matematika yang hanya menggunakan operasi dasar aritmetika (penjumlahan, pengurangan, perkalian, dan pembagian), dimulai dari nilai tertentu, dilakukan secara berulang (*rekursif*) dengan rumus yang sama. Metode numerik merupakan alternatif penyelesaian masalah-masalah matematika yang tidak mungkin diselesaikan secara eksak (analitik). Perangkat lunak analisis numerik biasanya menyediakan perintah-perintah yang mengimplementasikan algoritma dan metode numerik untuk memecahkan persamaan matematika, menganalisis data, dan melakukan simulasi numerik.

Beberapa software CAS yang sudah disebutkan pada bagian sebelumnya, khususnya software CAS untuk tujuan umum, merupakan software untuk analisis numerik. Di antara software-software tersebut adalah EMT, FreeMat, GNU Octave, GroovyLab, Julia, Maple, MathCAD, Mathematica, MATLAB, SageMath, Scilab, dan SMATH Studio.

Euler Math Toolbox (EMT)

Euler Mathematical Toolbox (EMT) atau **EuMathT** atau **Euler** adalah paket perangkat lunak numerik gratis dan sumber terbuka yang berisi bahasa matriks, menggunakan antarmuka grafik (GUI) mode buku catatan (*notebook*), dan jendela plot. Euler dirancang untuk matematika tingkat tinggi seperti kalkulus, optimisasi, dan statistika. EMT dapat menangani bilangan real, kompleks dan interval, vektor dan matriks, dapat menghasilkan plot 2D/3D, serta menggunakan Maxima untuk perhitungan simbolik/eksak. EMT hanya tersedia untuk sistem operasi Windows (beberapa versi terakhir hanya untuk sistem 64 bit) dan dapat dijalankan di Linux melalui emulator (misalnya Wine). Terdapat versi Unix dan Linux (versi lama), namun tidak mendukung subsistem aljabar komputer.

Pengembangan EMT dimulai pada tahun 1988 sebagai program untuk Atari ST. Saat itu, namanya hanya Euler, tetapi ternyata terlalu tidak spesifik untuk Internet. Tujuan utama program ini adalah sebagai alat untuk menguji algoritma numerik, untuk memvisualisasikan hasil, dan untuk mendemonstrasikan konten matematika di kelas. EMT menggunakan bahasa matriks yang mirip dengan MATLAB. Pengembang utama Euler dari semula hingga sekarang adalah René Grothmann, profesor matematika dari Universitas Katolik Eichstätt-Ingolstadt, Jerman. Pada tahun 2007, Euler menyatukan sistem aljabar komputer Maxima. Ekspresi simbolik dan fungsi lainnya ditambahkan untuk berkomunikasi dengan Maxima, dan untuk mencapai tingkat integrasi yang baik ke dalam inti numerik Euler. Versi terbaru EMT adalah EMT 2024-01-14 yang dirilis pada 14 Januari

2024.

Inti program Euler adalah sistem numerik yang ditulis dalam bahasa C/C++. Euler dapat menangani nilai yang berupa bilangan nyata, bilangan kompleks, dan interval, serta vektor dan matriks dengan elemen-elemen jenis tersebut. Tipe data lain yang tersedia adalah matriks terkompresi dan matriks renggang, akumulator panjang untuk produk skalar eksak, dan string. String digunakan untuk menyimpan ekspresi, nama file, dll. Berdasarkan inti ini, fungsi tambahan diimplementasikan dalam bahasa matriks Euler, yang merupakan bahasa pemrograman interpreter dalam gaya dialek BASIC tingkat lanjut. Euler memuat pustaka-pustaka (kumpulan program/perintah/fungsi) untuk statistika, perhitungan numerik eksak yang dapat berupa interval, persamaan diferensial dan persamaan diferensial *stiff*, fungsi astronomi, geometri, dan masih banyak lagi.

Antarmuka EMT yang sederhana dan bersih terdiri atas jendela teks, dan jendela grafis. Jendela teks berisi buku catatan yang dapat diedit dengan editor teks terpisah dan jendela grafik berisi tampilan grafik. Grafik juga dapat ditampilkan di jendela *notebook*, atau dapat diekspor dalam berbagai format (PNG, SVG, WMF, *Clipboard*). Jenis grafik mencakup kurva, diagram garis, diagram batang, atau titik dalam 2D dan 3D, termasuk plot *anaglyph* permukaan 3D dan plot 3D lainnya. Euler memiliki API untuk menggunakan *raytracer* POV-Ray terbuka.

Euler dapat melakukan komputasi simbolik dengan menggunakan Maxima, yang dimuat sebagai proses terpisah, berkomunikasi dengan Euler melalui *pipe*. Kedua program dapat bertukar variabel dan nilai. Memang, Maxima digunakan dalam berbagai fungsi Euler (misalnya metode Newton) untuk membantu perhitungan turunan, ekspansi Taylor, dan integral. Selain itu, Maxima dapat dipanggil pada waktu definisi fungsi Euler.

EMT juga dapat menampilkan ekspresi-ekspresi atau rumus-rumus matematika dengan menggunakan LaTeX, baik di dalam teks yang dapat diedit maupun untuk menampilkan hasil perhitungan eksak. Untuk mengekspor rumus ke HTML, ekspresi/rumus matematika dengan LaTeX dapat disimpan sebagai file gambar atau menggunakan MathJax. Terdapat juga opsi khusus untuk mengekspor semua grafik ke format SVG (grafik vektor berskala). Euler juga menyertakan bahasa perakit (*compiler*) **Tiny C**, yang memungkinkan program dalam bahasa C dirakiti, dan disertakan melalui Windows DLL. Euler memiliki banyak kemiripan dengan MATLAB dan rivalnya yang gratis (GNU Octave), tetapi tidak saling kompatibel.

FreeMat

FreeMat adalah perangkat lunak komputasi numerik dan bahasa pemrograman gratis sumber terbuka dengan lisensi GPL, mirip dengan software komersial MATLAB dan IDL serta software gratis lain GNU Octave atau Scilab. Selain mendukung banyak fungsi MATLAB dan beberapa fungsi IDL, FreeMat memiliki antarmuka tanpa kode ke kode eksternal C, C++, dan Fortran, pengembangan

algoritma terdistribusi paralel lebih lanjut (melalui MPI), dan memiliki kemampuan plotting dan visualisasi 3D. FreeMat dapat dijalankan pada sistem operasi Windows, Linux dan Mac OS X. Versi terakhir adalah FreeMat 4.2 yang dirilis pada 30 Juni 2013. Sudah cukup lama software ini tidak mengalami pembaruan. Dokumen manual yang tersedia hanya untuk versi 4.1 yang dikeluarkan 10 Juli 2011, namun cukup lengkap setebal lebih dari 2500 halaman.

Berikut adalah di antara fitur-fitur FreeMat:

- Manipulasi array berdimensi- n (aslinya n dibatasi hingga 6);
- Dukungan untuk tipe data *integer* 8,16, dan 32 bit (bertanda dan tidak bertanda), tipe data *floating point* 32 dan 64 bit, serta tipe data kompleks 64 dan 128 bit;
- Aritmatika bawaan untuk manipulasi semua tipe data yang didukung;
- Dukungan untuk menyelesaikan sistem persamaan linier melalui operator pembagian;
- Nilai *eigen* dan dekomposisi nilai *singular*;
- Dukungan struktur pengendalian penuh (*for*, *while*, *break*, *continue*, dll.);
- Plot 2D dan menampilkan gambar;
- Tipe *array* heterogen (disebut "array sel" dalam bahasa MATLAB) didukung sepenuhnya;
- Dukungan penuh untuk array struktur dinamis;
- Dukungan FFT berbasis split-radix;
- Dukungan *pass-by-reference* (fitur IDL);
- Dukungan kata kunci (fitur IDL);
- Antarmuka tanpa kode untuk kode C/C++/FORTRAN eksternal;
- Dukungan Windows asli;
- Dukungan matriks renggang asli;
- Dukungan asli untuk Mac OS X (tidak diperlukan server X11);
- Penunjuk fungsi (*eval* dan *feval* didukung penuh);
- Plotting dan visualisasi 3D melalui OpenGL;
- Kemampuan *rendering* volume 3D (melalui VTK).

GNU Octave

GNU Octave adalah bahasa pemrograman tingkat tinggi untuk komputasi ilmiah dan komputasi numerik. Octave membantu dalam menyelesaikan masalah linier dan nonlinier secara numerik, dan untuk melakukan eksperimen numerik lainnya menggunakan bahasa yang sebagian besar kompatibel dengan MATLAB. Octave menyediakan fasilitas yang luas untuk memecahkan masalah umum aljabar linier numerik, menemukan akar persamaan nonlinier, mengintegrasikan fungsi biasa, memanipulasi polinomial, dan mengintegrasikan persamaan diferensial biasa dan persamaan aljabar diferensial. Kemampuannya dapat diperluas dan disesuaikan melalui fungsi/program buatan pengguna yang ditulis dalam bahasa Octave, atau menggunakan modul/paket yang ditulis dalam C++, C, Fortran, atau bahasa lainnya dan dapat dipanggil secara dinamis. Tersedia

lebih dari 100 paket yang dapat diunduh dan digunakan di Octave. Repositori paket-paket Octave terdida di situs <https://gnu-octave.github.io/packages/>.

Sebagai bagian dari Proyek GNU, Octave merupakan perangkat lunak gratis sumber terbuka di bawah lisensi GNU GPL. Proyek ini dimulai sekitar tahun 1988. Pada awalnya Octave dimaksudkan sebagai software pandamping buku teks untuk mata kuliah Desain Reaktor Kimia yang sedang ditulis oleh James B. Rawlings dari the Universitas Wisconsin-Madison dan John G. Ekerdt dari Universitas Texas. Pengembangan penuh dimulai oleh John W. Eaton pada tahun 1992. Rilis alfa pertama dimulai pada 4 Januari 1993 dan pada 17 Februari 1994 versi 1.0 dirilis. Sejak itu, Octave telah melalui beberapa revisi besar dan disertakan dengan banyak distribusi GNU/Linux. Versi terakhir Octave adalah 8.4.0, dirilis pada 5 November 2023. Program ini berjalan di Microsoft Windows dan sebagian besar sistem operasi mirip Unix dan Unix, termasuk Linux, Android, dan macOS.

Nama program ini diambil dari nama Octave Levenspiel, mantan profesor penulis utamanya, yang menulis buku teks terkenal tentang teknik reaksi kimia. Levenspiel juga dikenal karena kemampuannya melakukan perhitungan cepat di atas kertas.

Selain digunakan pada desktop untuk komputasi ilmiah pribadi, Octave digunakan di dunia akademis dan industri. Misalnya, Octave digunakan pada komputer paralel besar di Pittsburgh Supercomputing Center untuk menemukan kerentanan terkait tebakan nomor jaminan sosial.

Berikut ini adalah detail teknis GNU Octave.

- Octave ditulis dalam bahasa C++ menggunakan pustaka standar C++.
- Octave menggunakan interpreter untuk menjalankan bahasa skrip Octave.
- Octave dapat diperluas menggunakan modul yang dapat dimuat secara dinamis.
- Penerjemah Octave memiliki mesin grafis berbasis OpenGL atau Gnuplot untuk membuat plot, grafik, dan diagram serta menyimpan atau mencetaknya.
- Octave menyediakan baik antarmuka pengguna grafis (GUI) maupun antarmuka baris perintah tradisional (CLI).

Bahasa Octave adalah bahasa pemrograman *interpreter* yang terstruktur (mirip dengan C) dan mendukung banyak fungsi pustaka standar C yang umum, dan juga panggilan dan fungsi sistem UNIX tertentu. Sayangnya, Octave tidak mendukung penggunaan argumen dengan referensi meskipun argumen fungsi bersifat *copy-on-write* untuk menghindari duplikasi yang tidak perlu. Program Octave terdiri atas daftar pemanggilan fungsi atau skrip. Sintaksnya berbasis matriks dan menyediakan berbagai fungsi untuk operasi matriks. Octave mendukung berbagai struktur data dan memungkinkan pemrograman berorientasi objek. Sintaksnya sangat mirip dengan MATLAB, dan pemrograman skrip secara

teliti akan memungkinkannya berjalan pada Octave maupun MATLAB.

GroovyLab (aka jLab)

GroovyLab, sebelumnya bernama **jLab**, adalah software komputasi numerik yang ditulis dalam bahasa pemrograman Java. Mesin skrip utama jLab adalah GroovySci, yang sebagian besar didasarkan pada GroovyShell. Selain itu, jLab juga mendukung bahasa interpreter **J-Script** (mirip dengan MATLAB) dan penggunaan kode kelas Java. GroovyLab mendukung **indy** (panggil dinamis) karena menggunakan Groovy versi **indy**. Selain itu, aslinya GroovyLab melakukan komputasi menggunakan kecepatan ganda, bukan BigDecimals. Pengaturan ini dapat dikonfigurasi.

Software aplikasi jLab bertujuan untuk menyediakan platform komputasi ilmiah seperti MATLAB/Scilab yang didukung oleh mesin skrip yang diimplementasikan dalam bahasa Java. Dalam implementasi jLab saat ini terdapat dua mesin skrip: bahasa skrip interpreter j-Script dan bahasa skrip perakitan Groovy. Yang terakhir (yaitu Groovy) tampaknya menjadi pilihan yang lebih disukai, karena jauh lebih cepat, dapat mengeksekusi kode Java secara langsung hanya dengan menggunakan aturan pengemasan Java yang sudah dikenal, dan merupakan bahasa yang kaya fitur, yaitu Groovy yang ditingkatkan dengan operasi matriks seperti MATLAB dan dukungan sekitarnya lingkungan. Skrip GroovyLab dapat dengan mudah digabungkan dengan MATLAB, sehingga ilmuwan dapat menggabungkan fleksibilitas Groovy dengan efektivitas komputasi MATLAB.

Software aplikasi GroovyLab bertujuan untuk menyediakan platform komputasi ilmiah seperti MATLAB/Scilab yang didukung oleh mesin skrip yang diimplementasikan dalam bahasa Groovy. Pengguna GroovyLab dapat bekerja dengan jendela terminal perintah seperti MATLAB, atau dengan editor fleksibel berdasarkan komponen **rsyntaxarea**, yang menawarkan pengembangan kode yang lebih nyaman. GroovyLab mendukung fasilitas pembuatan berbagai grafik kurva dan diagram dan dapat memanfaatkan secara efektif banyak pustaka (program) Java yang handal, seperti J LAPACK, Apache Common Maths, EJML, MTJ, terjemahan NUMAL dalam Java, terjemahan Numerical Recipes Java, Colt, dll. Selain itu, GroovyLab mendukung Aljabar Komputer berdasarkan pada proyek **symja** (<http://code.google.com/p/symja/>).

Pengembang GroovyLab adalah Stergios Papadimitriou dari Institut Pendidikan Teknologi Macedonia Timur dan Thraki, Jurusan Teknik Informatika dan Komputer, Yunani.

Keunggulan utama GroovySci yang memberikan potensi besar kepada GroovyLab adalah sebagai berikut.

- Fleksibilitas bahasa Groovy yang menawarkan banyak peluang untuk mengimplementasikan modul-modul komputasi ilmiah yang siap pakai. Modul-modul komputasi ilmiah berbasis Java yang luas dengan berbagai

fungsi untuk banyak domain aplikasi. Pustaka ini tersedia langsung dari GroovySci yang dapat dipadukan secara dinamis dengan modul-modul Java yang sesuai.

- Program GroovyLab mirip MATLAB yang ramah pengguna dan menyediakan fungsi-fungsi untuk menggambar kurva dan diagram berkualitas tinggi.
- Sintaks Groovy sangat mirip dengan Java, dan kode Java dapat dengan mudah dipadukan dengan Groovy, sehingga pengguna dengan latar belakang Java dapat menjadi produktif dalam waktu singkat.
- Kecepatan skrip berbasis Groovy mendekati kecepatan kode Java.

GroovyLab menyediakan banyak bantuan online dan contoh demo. Filosofi GroovyLab adalah memanfaatkan perangkat lunak ilmiah Java berkualitas tinggi. Meskipun setiap pustaka ilmiah Java dapat digunakan sebagai modul tambahan, GroovyLab aslinya menyertakan beberapa pustaka ilmiah Java yang sangat baik. Pustaka terintegrasi tersebut adalah:

1. Implementasi ke dalam bahasa Java program-program *Numerical Recipes*, edisi ketiga, yang semula dalam bahasa C++ oleh Huang Wen Hui;
2. Pustaka matriks yang efisien dalam bahasa Java, <http://code.google.com/p/efisien-java-matrix-library/>
3. Modul-modul matriks untuk bahasa Java, <http://code.google.com/p/matrix-toolkits-java/>
4. Pustaka Paralel **Colt**, <http://sites.google.com/site/piotrwendykier/software/parallelcolt>
5. **Jsci** - API sains untuk Java, <http://jsci.sourceforge.net/>, menyediakan sebagian besar program-program Analisis Wavelet
6. Sistem plot **JFreeChart**, <http://www.jfree.org/jfreechart/>, menyediakan plot yang beberapa di antaranya dilengkapi antarmuka seperti MATLAB.

Julia

Julia adalah bahasa pemrograman dinamis tingkat tinggi untuk tujuan umum, paling lazim digunakan untuk analisis numerik dan ilmu komputasi. Meskipun Julia adalah bahasa perakitan (melalui LLVM, *Low Level Virtual Machine*), Julia dapat dijalankan mirip dengan bahasa skrip atau interpreter (yaitu Julia memiliki jendela/layar perintah bahasa pemrograman REPL, *read-eval-print loop*). Program atau kode sumber Julia dapat dirakit menjadi file siap instal/dijalankan, yang dapat dibuat dengan cepat, tidak memerlukan apa pun yang sudah diinstal sebelumnya. Program Julia juga dapat dirakit secara terpisah menjadi file biner yang dapat dijalankan, tanpa program aslinya. Kompilasi seperti itu tidak diperlukan untuk kecepatan, karena Julia juga dikompilasi saat dijalankan secara interaktif, namun kompilasi berarti menyembunyikan program aslinya.

REPL dapat diperluas dengan mode tambahan, dan telah disertakan dengan paket, misalnya dengan mode SQL, untuk akses database, dan RCall.jl menam-

bahkan mode R, untuk bekerja dengan bahasa R.

Program Julia dapat dioperasikan dengan bahasa lain, digunakan untuk memanggil fungsi pustaka bersama secara individual (seperti yang ditulis dalam C atau Fortran), dan paket-paket untuk memanggil bahasa lain, misalnya Python (dengan **PythonCall.jl**), R, MATLAB, C# (dan bahasa .NET lainnya dengan **DotNET.jl**, dengan **JdotNET**), JavaScript, Java (dan bahasa JVM lainnya, seperti Scala dengan **JavaCall.jl**) atau Rust menggunakan makro `@ccall`. Juga tersedia paket-paket bahasa lain untuk memanggil Julia, misalnya dari Python (misalnya dengan **PythonCall.jl**, **pyjulia**), R (misalnya **JuliaCall**), Rust, Ruby, atau C#.

Julia juga memiliki paket yang mendukung bahasa markup seperti HTML (dan juga untuk HTTP), XML, JSON dan BSON, dan untuk database (seperti PostgreSQL, Mongo, Oracle, termasuk untuk TimesTen, MySQL, SQLite, Microsoft SQL Server, Amazon Redshift, Vertica, ODBC) dan penggunaan web secara umum.

Inti program Julia ditulis dalam bahasa dengan Julia itu sendiri dan C, termasuk C++ untuk paket-paket yang dibutuhkan LLVM. Penguraian kode, penurunan kode, dan *bootstrapping* Julia hingga versi 1.10 ditulis dengan **FemtoLisp**, salah satu dialek bahasa Scheme. Sejak versi itu, paket Julia murni baru **JuliaSyntax.jl** digunakan untuk *parsing*, yang meningkatkan kecepatan dan penampilan pesan kesalahan dalam berbagai kasus. Proyek infrastruktur kompiler LLVM digunakan sebagai tulang punggung untuk menghasilkan kode mesin yang optimal untuk semua *platform* umum.

Julia dimulai pada tahun 2009 oleh Jeff Bezanson, Stefan Karpinski, Viral B. Shah, dan Alan Edelman. Pada 14 Februari 2012, tim meluncurkan situs web *blog* yang menjelaskan misi bahasa tersebut. Tiga dari rekan pembuat Julia adalah penerima **Penghargaan James H. Wilkinson untuk Perangkat Lunak Numerik 2019** (diberikan setiap empat tahun) atas kreasi Julia, sebuah lingkungan inovatif untuk pembuatan alat berkinerja tinggi yang memungkinkan analisis dan solusi masalah ilmu komputasi. Selain itu, Alan Edelman, profesor matematika terapan di MIT, telah terpilih untuk menerima **Penghargaan Sidney Fernbach dari IEEE Computer Society 2019** untuk terobosan luar biasa dalam komputasi kinerja tinggi, aljabar linier, dan ilmu komputasi dan atas kontribusinya pada bahasa pemrograman Julia.

Julia 0.7 dan versi 1.0 dirilis pada tanggal yang sama, yakni 8 Agustus 2018. Pengerjaan Julia 0.7 merupakan "usaha besar" (misalnya, karena "pengoptimal yang sepenuhnya baru"), dan beberapa perubahan dilakukan pada semantik, misalnya iterasi antarmuka disederhanakan. Julia 1.1 dirilis pada Januari 2019 dengan fitur "*stack* pengecualian" baru. Julia 1.2 dirilis pada Agustus 2019 dengan beberapa dukungan bawaan untuk browser web. Julia 1.3 menambahkan paralelisme *multi-thread* yang dapat disusun dan sistem artefak biner untuk paket Julia. Julia 1.4 menambahkan sintaks untuk pengindeksan *array* generik untuk menangani misalnya *array* dengan indeks mulai 0. Model memori juga diubah. Julia 1.5

yang dirilis pada Agustus 2020 menambahkan dukungan *debugging* catat dan jalankan kembali, untuk **rr** Mozilla. Rilis ini mengubah cara kerja REPL (*soft scope*) menjadi seperti yang digunakan pada Jupyter, tetapi sepenuhnya kompatibel dengan kode non-REPL. Julia 1.5 menargetkan kinerja (kecepatan kompilasi dibandingkan dengan kinerja kode yang dihasilkan) yang disebut "*time-to-first-plot*" (TTFP, juga disebut TTFX) dan menambahkan alat bantu bagi pengembang untuk meningkatkan kecepatan pemanggilan paket. Julia 1.6 merupakan rilis terbesar sejak 1.0, lebih cepat dalam banyak hal, misalnya memperkenalkan prakompilasi paralel dan memanggil paket lebih cepat, misalnya peningkatan 50 kali kecepatan memanggil *artefak* yang berupa pohon biner besar. Pada versi 1.7, pengembangan Julia kembali ke rilis berbasis waktu. Julia 1.7.0 dirilis pada November 2021 dengan banyak perubahan, misalnya penghasil bilangan acak baru yang lebih cepat. Julia 1.7.3, yang dirilis pada 25 Mei 2022, memperbaiki beberapa masalah, termasuk setidaknya satu pembaruan keamanan. Julia 1.8 dirilis pada tahun 2022, sedangkan versi-versi perbaikannya hingga 1.8.5 dirilis pada Januari 2023, memperbaiki *bug* (*backporting*) dan "*invalidations*", sehingga waktu kompilasi lebih cepat. Julia 1.9.0 (dan versi stabil yang lebih baru 1.9.4) dirilis pada 7 Mei 2023. Julia 1.10.0 dirilis pada 25 Desember 2023, sedangkan pembaruannya, versi 1.10.1 dirilis pada 14 Februari 2024, dengan banyak fitur baru, misalnya waktu panggil paket yang lebih cepat. Julia 1.10.1 dan 1.10.0 memiliki sejumlah regresi, beberapa dengan solusi seperti untuk menggambar grafik. Julia 1.10.2 telah dirilis pada 1 Maret 2024.

Bahasa Julia menjadi proyek yang disponsori secara fiskal oleh **NumFOCUS** pada tahun 2014 dalam upaya memastikan keberlanjutan jangka panjang. Semula, Jeremy Kepner dari MIT Lincoln Laboratory adalah sponsor pendiri proyek Julia. Selain itu, dana dari Gordon and Betty Moore Foundation, Alfred P. Sloan Foundation, Intel, dan lembaga-lembaga lain seperti NSF, DARPA, NIH, NASA, dan FAA sangat berperan penting dalam pengembangan Julia. Mozilla, pembuat browser web Firefox, dengan hibah penelitiannya untuk H1 2019, mensponsori anggota tim resmi Julia untuk proyek "**Membawa Julia ke Browser**". Bahasa Julia juga didukung oleh para penyumbang pribadi di **GitHub**.

Pada tahun 2015 didirikan perusahaan **Julia Computing, Inc.**, yang sekarang bernama **JuliaHub, Inc.**, oleh Viral B. Shah, Deepak Vinchhi, Alan Edelman, Jeff Bezanson, Stefan Karpinski, dan Keno Fischer. Pada bulan Juni 2017, Julia Computing mengumpulkan dana awal sebesar 4,6 juta USD dari **General Catalyst** dan **Founder Collective**. Pada bulan yang sama juga diperoleh dana 910.000 USD dari **Alfred P. Sloan Foundation** untuk mendukung pengembangan sumber terbuka Julia, dan 160.000 USD digunakan untuk mempromosikan keberagaman di komunitas Julia. Pada bulan Desember 2019 perusahaan tersebut mendapat dana 1,1 juta USD dari pemerintah AS untuk mengembangkan komponen syarat pembelajaran mesin guna mengurangi total konsumsi energi sistem pemanas, ventilasi, dan pendingin udara (HVAC) di gedung. Pada bulan Juli 2021, Julia Computing mengumumkan bahwa mereka mengumpulkan putaran **Seri A** se-

nilai 24 juta USD yang dipimpin oleh Dorilton Ventures, yang juga memiliki tim **Formula 1 Williams Racing**, yang bermitra dengan Julia Computing. Pada bulan Juni 2023, JuliaHub (nama baru Julia Computing) menerima investasi baru yang strategis senilai 13 juta USD dari perusahaan gabungan yang dipimpin oleh **AE Industrial Partners HorizonX (AEI HorizonX)**. AEI HorizonX adalah *platform* investasi modal ventura yang dibentuk bekerja sama dengan **The Boeing Company**, salah satu perusahaan pengguna Julia.

Sejak versi 1.0 pada tahun 2018, sintaks bahasa Julia sudah dianggap stabil, dan Julia memiliki jaminan kompatibilitas mundur untuk 1.x dan jaminan stabilitas API yang terdokumentasi (stabil). Semua ekosistem (paket terdaftar) menggunakan sintaksis yang baru dan lebih baik, dan dalam banyak kasus bergantung pada API baru yang telah ditambahkan secara berkala, dan dalam beberapa kasus, beberapa sintaksis kecil ditambahkan dengan cara yang kompatibel ke depan, misalnya di Julia 1.7.

Julia adalah bahasa pemrograman untuk tujuan umum, dan awalnya dirancang untuk komputasi numerik/teknis. Julia juga berguna untuk pemrograman sistem tingkat rendah, sebagai bahasa spesifikasi (bahasa yang digunakan untuk analisis sistem, analisis kebutuhan, dan perancangan sistem), alat sintesis tingkat tinggi (HLS) (untuk perangkat keras, misalnya FPGA), dan untuk pemrograman web, baik di sisi server maupun di sisi klien.

Ciri-ciri utama bahasa Julia adalah:

- *Dispatch* berganda: menyediakan kemampuan untuk menentukan perilaku fungsi dengan berbagai jenis argumen;
- Penggunaan jenis data dinamis: tipe untuk dokumentasi, optimasi, dan pengiriman;
- Performanya mendekati bahasa yang menggunakan jenis data statis seperti C;
- Manajer paket bawaan;
- Makro mirip bahasa Lisp dan fasilitas *metaprogramming* lainnya;
- Dirancang untuk komputasi paralel dan terdistribusi;
- *Coroutine: green threading* ringan;
- Pembuatan kode otomatis untuk jenis-jenis argumen berbeda;
- Konversi dan promosi yang dapat diperluas untuk jenis numerik dan lainnya;

Julia menggunakan kode UTF-8 dan $\text{\LaTeX}$, sehingga mendukung penggunaan simbol-simbol umum dalam matematika untuk operator, seperti $\in$ untuk operator `in`, dapat diketik `\in` lalu ditekan tombol Tab (yaitu menggunakan perintah $\text{\LaTeX}$, atau juga dapat dilakukan dengan menyalin-menempel, misalnya $\sqrt{}$ dan $\sqrt[3]{}$ untuk fungsi `sqrt` dan `cbrt`. Julia memiliki dukungan untuk rilis besar terbaru Unicode 15.0 (Julia 1.11-DEV mendukung rilis poin 15.1 terbaru untuk bahasa-bahasa di dunia, bahkan untuk kode sumber, misalnya nama variabel)

(sementara disarankan menggunakan bahasa Inggris untuk kode publik, dan misalnya nama paket).

Julia didukung oleh Jupyter, lingkungan *notebook* berbasis web, dan **Pluto.jl**, sebuah *notebook reaktif* (di mana *notebook* disimpan sebagai file Julia murni), mungkin akan menjadi pengganti Jupyter. Selain itu, sistem penerbitan **Quarto** milik Posit (sebelumnya **RStudio Inc**) mendukung Julia, Python, R, dan Observable JavaScript (bahasa-bahasa tersebut mendapat dukungan resmi dari Posit, dan bahkan dapat dirangkai bersama dalam dokumen buku catatan yang sama, lebih banyak bahasa yang didukung secara tidak resmi).

Julia telah digunakan di banyak universitas, termasuk MIT, Stanford, UC Berkeley dan Universitas Cape Town. Perusahaan swasta besar di berbagai sektor, termasuk Amazon, IBM, JP Morgan AI Research, dan ASML, telah mengadopsi bahasa ini. Julia juga telah digunakan oleh lembaga pemerintah, termasuk NASA dan FAA, serta setiap laboratorium energi nasional AS. Berikut adalah daftar beberapa aplikasi Julia yang dilakukan di berbagai perusahaan dan institusi lain.

- Amazon menggunakan Julia untuk komputasi kuantum dan pembelajaran mesin melalui Amazon SageMaker.
- ASML menggunakan Julia untuk pemrograman real-time yang sulit dengan mesin mereka.
- Aliansi Pemodelan Iklim menggunakan Julia untuk pemodelan perubahan iklim.
- CERN menggunakan Julia untuk analisis data dari **Large Hadron Collider** (percobaan **LHCb**).
- NASA dan Laboratorium Propulsi Jet menggunakan Julia untuk memodelkan dinamika pemisahan pesawat ruang angkasa, menganalisis kumpulan data eksoplanet TRAPPIST, dan menganalisis data latar belakang gelombang mikro kosmik dari Big Bang.
- INPE Brasil menggunakan Julia untuk misi luar angkasa dan simulasi satelit.
- Laboratorium Penelitian Angkutan Udara AS juga menggunakan Julia guna menjalankan perangkat keras yang tertanam untuk merencanakan dan melaksanakan penerbangan drone VTOL secara otomatis.
- Moderna, Pfizer, AstraZeneca, Procter & Gamble, dan United Therapeutics menggunakan Julia untuk pengembangan obat.
- *Federal Reserve Bank of New York* menggunakan Julia untuk pemodelan makroekonomi sejak tahun 2015, termasuk perkiraan guncangan COVID-19 pada tahun 2021.
- Bank Sentral Kanada menggunakan Julia untuk pemodelan makroekonomi.
- BlackRock, manajer aset terbesar di dunia, menggunakan Julia untuk analisis deret waktu keuangan.
- Aviva, perusahaan asuransi umum terbesar di Inggris, menggunakan Julia untuk perhitungan aktuarial.

- Mitre Corporation, menggunakan Julia untuk verifikasi hasil pemilu yang dipublikasikan.
- Pemenang Nobel Thomas J. Sargent, menggunakan Julia untuk pemodelan makroekonometrik.

Maple

Maple adalah software komputasi simbolik dan numerik serta bahasa pemrograman multi-paradigma. Kemampuan Maple mencakup beberapa bidang komputasi teknis, seperti matematika simbolik, analisis numerik, pemrosesan data, visualisasi, dan lain-lain. Maple menyediakan program pendamping **MapleSim** untuk pemodelan fisik multidomain dan penghasil kode-kode program.

Kemampuan Maple untuk komputasi simbolik mencakup kemampuan-kemampuan software sistem aljabar komputer tujuan umum. Maple dapat memanipulasi ekspresi matematika dan menemukan solusi simbolis untuk permasalahan tertentu, seperti permasalahan terkait persamaan diferensial biasa dan parsial. Di Maple, pengguna dapat memasukkan ekspresi matematika dengan notasi matematika biasa. Tersedia dukungan untuk perhitungan numerik, presisi sebarang, serta perhitungan dan visualisasi simbolik.

Maple menggabungkan bahasa pemrograman gaya imperatif berjenis dinamis (menyerupai Pascal), yang memungkinkan variabel lingkup leksikal. Maple menyediakan antarmuka ke bahasa lain (C, C#, Fortran, Java, MATLAB, dan Visual Basic), serta ke Microsoft Excel.

Maple mendukung MathML 2.0, yang merupakan format W3C untuk menampilkan dan membaca ekspresi matematika, termasuk tampilannya di halaman Web. Terdapat juga fungsi untuk mengubah ekspresi dari notasi matematika biasa menjadi perintah-perintah LaTeX.

Maple dikembangkan secara komersial oleh perusahaan perangkat lunak Kanada **Maplesoft**. Nama "Maple" mengacu pada warisan perangkat lunak Kanada.

Maple didasarkan pada kernel (program inti) kecil yang ditulis dalam bahasa C, yang menyediakan bahasa Maple. Sebagian besar kemampuan Maple didukung oleh program-program kecil (pustaka) berasal dari berbagai sumber. Sebagian besar pustaka ditulis dalam bahasa Maple, yang memiliki kode program yang dapat dilihat. Banyak komputasi numerik dilakukan dengan menggunakan pustaka (kumpulan program) numerik komersial NAG (*Numerical Analysis Group*) dan pustaka numerik gratis ATLAS (*Automatically Tuned Linear Algebra Software*) atau GMP (*GNU Multiple Precision Arithmetic*).

Konsep pertama Maple muncul dari pertemuan pada akhir tahun 1980 di Universitas Waterloo. Para peneliti di universitas tersebut ingin membeli komputer yang cukup mumpuni untuk menjalankan sistem aljabar komputer berbasis **Lisp**, yakni **Macsyma**. Alih-alih membelinya, mereka memilih untuk mengembangkan sistem aljabar komputer sendiri, bernama Maple, yang dapat dijalankan

pada komputer yang lebih murah. Karena menginginkan portabilitas yang luas, mereka mulai menulis Maple dalam bahasa pemrograman dari keluarga BCPL (awalnya menggunakan subset dari B dan C, dan kemudian hanya C). Versi terbatas pertama muncul setelah tiga pekan, dan versi yang lebih lengkap mulai digunakan secara umum pada tahun 1982. Pada akhir tahun 1983, lebih dari 50 universitas telah memasang salinan Maple di komputer mereka.

Pada tahun 1984, kelompok peneliti tersebut bekerja sama dengan Watcom Products Inc untuk melisensikan dan mendistribusikan versi pertama yang tersedia secara komersial, Maple 3.3. Pada tahun 1988 didirikan perusahaan Waterloo Maple Inc. (**Maplesoft**) untuk mendistribusikan Maple, namun akhirnya berkembang hingga memiliki departemen R&D sendiri dan mengembangkan sebagian besar Maple (sedangkan sisanya dikembangkan di berbagai laboratorium universitas).

Pada tahun 1989, antarmuka pengguna grafis (GUI) pertama Maple dikembangkan dan disertakan pada versi 4.3 untuk Macintosh. GUI baru versi X11 dan Windows disertakan pada tahun 1990 pada Maple V. Pada tahun 1992, Maple V Rilis 2 memperkenalkan "lembar kerja" Maple yang menggabungkan teks, grafik, baris-baris perintah serta hasilnya.

Pada tahun 1999 dirilis Maple 6 yang menyertakan beberapa pustaka numerik NAG. Pada tahun 2003 diperkenalkan antarmuka "standar" pada Maple 9. Antarmuka ini terutama ditulis dalam Java (walaupun sebagian, seperti aturan untuk menulis rumus matematika, ditulis dalam bahasa Maple). Karena antarmuka Java dikritik lambat; perbaikan dilakukan pada versi sesudahnya, meskipun dokumentasi Maple 11 merekomendasikan antarmuka sebelumnya ("klasik") untuk pengguna dengan memori kurang dari 500 MB. Antara tahun 1995 dan 2005 Maple kehilangan pangsa pasar yang signifikan karena antarmuka pengguna yang lebih lemah. Dengan Maple 10 pada tahun 2005, Maple memperkenalkan antarmuka "modus dokumen" baru, yang sejak itu telah dikembangkan lebih lanjut dalam beberapa rilis. Pada bulan September 2009 Maple dan Maplesoft diakuisisi oleh penjual perangkat lunak Jepang **Cybernet Systems**. Versi terbaru Maple adalah Maple 2024 yang dirilis pada 6 Maret 2024.

Fitur Maple meliputi:

- dukungan untuk perhitungan simbolik dan numerik dengan sebarang presisi;
- pustaka (kumpulan) fungsi matematika dasar dan khusus;
- bilangan kompleks dan aritmatika interval;
- aritmatika, faktor persekutuan terbesar (FPB) dan faktorisasi polinomial rasional multivariat, medan berhingga, medan bilangan aljabar, dan medan fungsi aljabar;
- limit, deret, dan deret fungsi asimtotik;
- basis Grobner;

- aljabar diferensial;
- fungsi-fungsi manipulasi matriks termasuk dukungan untuk matriks jarang;
- fungsi-fungsi untuk menggambar grafik dan animasi fungsi matematika;
- penyelesaian sistem persamaan, persamaan Diophantine, ODE, PDE, DAE, DDE dan relasi rekurens;
- fungsi-fungsi numerik dan simbolik untuk kalkulus diskrit dan kontinu termasuk integral tentu dan tak tentu, penjumlahan berhingga dan tak berhingga, diferensiasi otomatis, dan transformasi integral kontinu dan diskrit;
- optimisasi lokal dan global dengan kendala dan tidak berkendala;
- analisis statistika termasuk pencocokan model, pengujian hipotesis, dan distribusi probabilitas;
- fungsi-fungsi (perintah) untuk manipulasi data, visualisasi dan analisis;
- fungsi-fungsi untuk masalah probabilitas dan kombinatorik;
- dukungan untuk data berbasis runtun waktu dan satuan;
- koneksi dengan kumpulan data keuangan dan ekonomi secara online;
- fungsi-fungsi untuk perhitungan keuangan termasuk obligasi, anuitas, derivatif, opsi, dll.;
- perhitungan dan simulasi proses acak;
- fungsi-fungsi untuk penambangan teks termasuk ekspresi reguler;
- fungsi-fungsi untuk pemrosesan sinyal dan sistem kendali linier dan non linier;
- fungsi-fungsi matematika, diskrit termasuk teori bilangan;
- fungsi-fungsi untuk memvisualisasikan dan menganalisis graf berarah dan graf tak berarah;
- teori grup, termasuk grup permutasi dan grup berhingga;
- fungsi tensor simbolik;
- impor dan ekspor filter untuk format data, gambar, suara, CAD, dan dokumen;
- editor teks termasuk pengeditan rumus;
- bahasa pemrograman yang mendukung konstruksi prosedural, fungsional dan berorientasi objek;
- perintah-perintah untuk menambahkan antarmuka pengguna ke penghitungan dan aplikasi;
- fungsi-fungsi untuk menghubungkan ke SQL, Java, .NET, C++ dan http;
- perintah-perintah untuk menghasilkan kode untuk C, C#, Fortran, Java, JavaScript, Julia, MATLAB, Perl, Python, R, dan Visual Basic;
- fungsi-fungsi untuk pemrograman paralel.

Harga lisensi Maple untuk perorangan adalah sebagai berikut.

- Maple edisi personal untuk perorangan: 349 USD.
- Maple edisi pelajar/mahasiswa: 149 USD (lisensi standar, tanpa batas waktu), 75 USD (lisensi 1 tahun).

- Maple edisi mahasiswa riset pascasarjana: 495 USD (lisensi standar).
- Maple edisi akademik: 1360 USD (lisensi akademik perorangan untuk lembaga pendidikan terakreditasi).
- Maple edisi komersial: 3140 USD (lisensi komersial perorangan).

Informasi harga lisensi Maple untuk multi-komputer tidak tersedia secara umum, dapat diperoleh melalui *quote* dengan menghubungi bagian penjualan Maplesoft.

Mathcad

Mathcad adalah perangkat lunak komputer untuk verifikasi, validasi, dokumentasi, dan penggunaan kembali perhitungan matematis di bidang teknik dan sains, terutama teknik mesin, kimia, listrik, dan sipil. Mathcad pertama dirilis pada tahun 1986 untuk sistem operasi DOS. Mathcad merupakan aplikasi matematika pertama dengan antarmuka berupa editor teks yang bersifat WYSIWYG (hasil cetakan seperti yang terlihat di layar) terpadu dengan editor notasi/ekspresi matematika yang langsung dapat dikerjakan secara interaktif. Pendekatan ini diadopsi oleh sistem lain seperti Mathematica, Maple, Macsyma, MATLAB (*live script*), dan Jupyter. Mathcad juga yang pertama memeriksa konsistensi satuan-satuan dalam bidang teknik melalui perhitungan penuh. Mathcad saat ini mencakup beberapa kemampuan sistem aljabar komputer, namun tetap berorientasi pada kemudahan penggunaan dan dokumentasi aplikasi metode-metode numerik dalam bidang teknik.

Awalnya Mathcad disusun dan dikembangkan oleh Allen Razdow dan Josh Bernoff di Mathsoft yang didirikan oleh David Blohm dan Razdow. Ini dirilis pada tahun 1986. Mathcad hanya berjalan di Microsoft Windows. Hingga tahun 1998, Mathcad juga mendukung MacOS. Pada April 2006 Mathcad diakuisisi oleh dan telah menjadi produk milik **Parametric Technology Corporation (PTC)**.

Rilis terakhir lini produk tradisional (pra "Prime"), yakni Mathcad 15.0, keluar pada bulan Juni 2010 dan berbagi struktur file lembar kerja yang sama dengan Mathcad 14.0. Rilis layanan terakhir, yakni Mathcad 15.0 M050, yang menambahkan dukungan untuk Windows 10, dirilis pada Desember 2017. Mathcad 15.0 tidak lagi dikembangkan secara aktif tetapi dalam "dukungan berkelanjutan". Setelah 2010 produk Mathcad diberi nama **Mathcad Prime**. Mathcad Prime 1.0 dirilis pada 10 Januari 2011. Mathcad Prime 6.0, dirilis 1 Oktober 2019, memerlukan versi 64-bit Windows 7, Windows 8.1 atau Windows 10. Pada 15 Maret 2022 dirilis versi PTC Mathcad Prime 8.0.0.0, yang merupakan varian freemium (jika tidak diaktifkan setelah mencoba gratis Mathcad Prime selama 30 hari, Anda dapat terus menggunakan PTC Mathcad Express untuk waktu yang tidak terbatas sebagai "Perangkat Lunak Penghitungan Teknik gratis-seumur-hidup PTC Mathcad Express". Percontohan freemium ini adalah pendekatan pemasaran baru untuk PTC. Peninjauan dan penandaan catatan teknik kini dapat dilakukan langsung oleh anggota tim tanpa semuanya memerlukan lisensi penuh Mathcad Prime. Versi terbaru Mathcad adalah Mathcad Prime 9.0, dirilis pada 14 Maret

2023.

Mathcad adalah bagian dari sistem pengembangan produk yang lebih luas yang dikembangkan oleh PTC, mampu menangani langkah-langkah analitis dalam rekayasa sistem. Mathcad terintegrasi dengan Creo Elements/Pro, Windchill, dan Creo Elements/View PTC. Integrasi tingkat fitur langsungnya dengan Creo Elements/Pro memungkinkan model analitik Mathcad digunakan secara langsung dalam menggerakkan geometri CAD, dan kesadaran strukturalnya dalam Windchill memungkinkan penghitungan langsung digunakan kembali dan diterapkan kembali pada beberapa model desain.

Antarmuka Mathcad memungkinkan pengguna untuk menggabungkan berbagai elemen berbeda (matematika, teks deskriptif, dan citra pendukung) ke dalam lembar kerja, di mana rumus-rumus atau ekspresi-ekspresi yang tergantung pada rumus-rumus lain dihitung ulang secara dinamis seiring perubahan input. Hal ini memungkinkan manipulasi sederhana terhadap variabel masukan, asumsi, dan ekspresi.

Fungsionalitas Mathcad meliputi:

- Berbagai fungsi numerik untuk statistik, analisis data, pemrosesan gambar, dan pemrosesan sinyal;
- Pemeriksaan dan penyederhanaan dimensi di mana-mana;
- Penyelesaian sistem persamaan, seperti ODE dan PDE menggunakan beberapa metode;
- Pencarian akar untuk polinomial dan fungsi lainnya;
- Manipulasi simbolik ekspresi matematika;
- Pembuatan plot parametrik 2D dan 3D serta pembuatan plot data diskrit;
- Memanfaatkan ekspresi matematika standar yang dapat dibaca dalam konstruksi program tertanam;
- Operasi vektor dan matriks, termasuk nilai eigen dan vektor eigen;
- Penyesuaian kurva dan analisis regresi;
- Statistik dan desain fungsi eksperimen dan jenis plot, serta evaluasi distribusi probabilitas;
- Impor dari dan ekspor ke aplikasi dan jenis file lain, seperti Microsoft Excel dan MathML;
- Referensi silang ke lembar kerja Mathcad lainnya;
- Integrasi dengan aplikasi teknik lainnya, seperti CAD, FEM, BIM, dan alat Simulasi, untuk membantu desain produk, seperti Autocad, Ansys, Revit.

Meskipun Mathcad sebagian besar berorientasi pada non-pemrogram, Mathcad juga digunakan dalam proyek yang lebih kompleks untuk memvisualisasikan hasil pemodelan matematika dengan menggunakan komputasi terdistribusi dan digabungkan dengan program yang ditulis menggunakan bahasa yang lebih tradisional seperti C++.

Lisensi PTC MathCAD Prime 9.0 adalah 63 USD/tahun (lisensi perorangan untuk

pelajar/mahasiswa) dan 735 USD/tahun (lisensi standar komersial perorangan). Untuk lisensi multi-pengguna, informasinya harga lisensinya dapat diperoleh dengan menghubungi pihak PTC.

Mathematica

Wolfram Mathematica adalah sistem perangkat lunak dengan pustaka program bawaan untuk berbagai bidang komputasi teknis yang memungkinkan pembelajaran mesin, statistika, komputasi simbolik, manipulasi data, analisis jaringan, analisis runtun waktu, NLP (pengolahan bahasa alami), optimasi, fungsi plot dan berbagai jenis data, implementasi algoritma, pembuatan antarmuka pengguna, dan antarmuka program lain. Mathematica disusun oleh Stephen Wolfram, dan dikembangkan oleh perusahaan komputasi multinasional Amerika **Wolfram Research** Champaign, Illinois. Mathematica menggunakan bahasa pemrograman Wolfram.

Mathematica dibagi menjadi dua bagian, yakni **kernel** dan **tampilan** (antarmuka pengguna). Kernel menerjemahkan ekspresi (kode bahasa Wolfram) dan menampilkan ekspresi hasil pada bagian tampilan. Tampilan asli Mathematica, yang dirancang oleh Theodore Gray pada tahun 1988, terdiri atas antarmuka *notebook* untuk mengedit dokumen yang dapat berisi kode (perintah-perintah Mathematica), teks biasa, gambar, dan grafik. Alternatif antarmuka Mathematica yang lain adalah Wolfram Workbench, IDE berbasis IntelliJ IDEA, dan JMath.

Wolfram Workbench merupakan software pengembangan terpadu (IDE) berbasis Eclipse yang diperkenalkan pada tahun 2006. Antarmuka ini menyediakan alat pengembangan kode berbasis proyek untuk Mathematica, termasuk manajemen revisi, debugging, pembuatan profil, dan pengujian. *Plugin* untuk IDE berbasis IntelliJ IDEA bekerja dengan bahasa Wolfram yang memiliki fitur penyorotan sintaksis, menganalisis dan melengkapi otomatis variabel dan fungsi buatan pengguna. Kernel Mathematica juga menyertakan antarmuka baris perintah (jendela terminal). Antarmuka JMath menggunakan GNU Readline dan WolframScript yang menjalankan program-program Mathematica (dengan argumen) dari baris perintah UNIX. Ekstensi file *notebook* Mathematica adalah `.nb`, sedangkan untuk file konfigurasi menggunakan ekstensi `.m`.

Mathematica dirancang agar sepenuhnya stabil dan kompatibel dengan versi sebelumnya. Kemampuan komputasi berkinerja tinggi diperluas dengan diperkenalkannya *array* yang dikemas dalam versi 4 (1999) dan matriks *renggang* (versi 5, 2003), dan dengan mengadopsi pustaka aritmatika presisi ganda GNU untuk mengevaluasi komputasi presisi tinggi.

Mathematica 1.0 dirilis pada tanggal 23 Juni 1988 di Champaign, Illinois dan Santa Clara, California. Versi 5.2 (2005) menambahkan kemampuan menjalankan bagian-bagian program otomatis secara terpisah ketika komputasi dilakukan pada komputer multi-core. Rilis ini menyertakan pustaka yang dioptimalkan untuk CPU tertentu. Selain itu Mathematica didukung oleh perangkat keras

akselerasi spesialis pihak ketiga seperti ClearSpeed.

Pada tahun 2002 diperkenalkan **gridMathematica** untuk memungkinkan pemrograman paralel pada komputer kluster heterogen dan sistem multiprosesor. Pada tahun 2008 teknologi komputasi paralel disertakan dalam semua lisensi Mathematica termasuk dukungan untuk teknologi grid seperti Windows HPC Server 2008, Microsoft Compute Cluster Server dan Sun Grid. Dukungan untuk perangkat keras GPU CUDA dan OpenCL ditambahkan pada tahun 2010.

Pada Versi 13 tersedia 6.051 fungsi dan simbol bawaan dalam bahasa Wolfram. Stephen Wolfram mengumumkan peluncuran Repositori Fungsi Wolfram (<https://resources.wolframcloud.com/FunctionRepository/>) pada 11 Juni 2019 sebagai tempat bagi komunitas publik Wolfram untuk berbagi fungsi-fungsi atau program-program dalam bahasa Wolfram. Pada saat buku ini ditulis, sudah tersedia 2948 fungsi yang dikontribusikan sebagai Fungsi Sumber Daya. Selain Repositori Fungsi Wolfram, terdapat Repositori Data Wolfram dengan data yang dapat dihitung dan Repositori Jaringan Neural Wolfram untuk pembelajaran mesin. Versi terbaru Mathematica adalah mathematica 14.0 yang dirilis pada 9 Januari 2024.

Komunikasi dengan aplikasi lain dapat dilakukan menggunakan protokol yang disebut Wolfram Symbolic Transfer Protocol (WSTP). Protokol ini memungkinkan komunikasi antara kernel Wolfram Mathematica dan antarmuka yang sekaligus sebagai antarmuka umum antara kernel dan aplikasi lain.

Wolfram Research mendistribusikan secara gratis kit bagi pengembang untuk menghubungkan aplikasi yang ditulis dalam bahasa pemrograman C ke kernel Mathematica melalui WSTP menggunakan J/Link, sebuah program Java yang dapat meminta Mathematica untuk melakukan komputasi. Hal serupa dapat dilakukan dengan program .NET, yakni .NET/Link. Bahasa lain yang dapat terhubung ke Mathematica termasuk Haskell, AppleScript, Racket, Visual Basic, Python, dan Clojure. Mathematica juga mendukung pembuatan dan pelaksanaan model-model Modelica untuk pemodelan sistem yang dapat terhubung dengan Wolfram System Modeler. Berbagai tautan juga tersedia ke banyak paket perangkat lunak dan API pihak ketiga.

Mathematica dapat mengambil data langsung dari berbagai sumber dan dapat membaca dan menulis ke *blockchain* publik (misalnya, Bitcoin, Ethereum, dan ARK). Mathematica mendukung impor dan ekspor lebih dari 220 data, gambar, video, suara, desain berbantuan komputer (CAD), sistem informasi geografis (GIS), dokumen, dan format biomedis.

Mathematica juga terintegrasi dengan Wolfram Alpha, mesin penjawab online yang menyediakan data tambahan, beberapa di antaranya terus-menerus diperbarui, bagi pengguna yang menggunakan Mathematica dengan koneksi Internet. Beberapa kumpulan data tersebut mencakup data astronomi, kimia, geopolitik, bahasa, biomedis, pesawat terbang, dan cuaca, selain data matematika (seperti

knot dan *polihedra*).

Lisensi Mathematica tersedia untuk berbagai tujuan/kepentingan. Berikut adalah informasi beberapa harga lisensi software Mathematica yang diterbitkan pada situs Wolfram <https://www.wolfram.com/>.

1. Lisensi rumah dan penggemar (personal non-profesional): 2 kunci aktivasi, desktop+cloud, 4 kernel/desktop, 6000 kali pemanggilan Wolfram Alpha per bulan, 2 GB penyimpanan cloud, 1 GB batas ukuran file cloud, 5 menit batas perhitungan cloud, 1 GB batas memori cloud, 20000 kredit cloud
 1. Paket Premium: 195 USD/tahun;
 2. Paket Bulanan: 26 USD/bulan;
 3. Paket Premium Plus: 390 USD + registrasi ulang 175 USD/tahun, 4 kernel/desktop tanpa batas waktu;
2. Lisensi Pelajar (non-profesional): 2 kunci aktivasi, desktop+cloud, 4 kernel/desktop, 6000 kali pemanggilan Wolfram Alpha per bulan, 2 GB penyimpanan cloud, 1 GB batas ukuran file cloud, 5 menit batas perhitungan cloud, 1 GB batas memori cloud, 20000 kredit cloud
 1. Paket Tahunan: 75 USD/tahun;
 2. Paket 4 Tahun: 195 USD;
 3. Paket Semester: 50 USD/6 bulan;
 4. Paket Bulanan: 10 USD/bulan;
3. Lisensi Pendidikan untuk Pendidikan Dasar dan Menengah: 2 kunci aktivasi, desktop+cloud, 8 kernel/desktop, 12000 kali pemanggilan Wolfram Alpha per bulan, 15 GB penyimpanan cloud, 1 GB batas ukuran file cloud, 10 menit batas perhitungan cloud, 2 GB batas memori cloud, 50000 kredit cloud
 1. Paket Premium: 153 USD/tahun;
 2. Paket Bulanan: 19 USD/bulan;
 3. Paket Premium Plus: 306 USD + registrasi ulang 115 USD/tahun, 8 kernel/desktop tanpa batas waktu;
4. Lisensi Pendidikan untuk Junior & Community Colleges: 2 kunci aktivasi, desktop+cloud, 8 kernel/desktop, 12000 kali pemanggilan Wolfram Alpha per bulan, 15 GB penyimpanan cloud, 1 GB batas ukuran file cloud, 10 menit batas perhitungan cloud, 2 GB batas memori cloud, 50000 kredit cloud
 1. Paket Premium: 153 USD/tahun;
 2. Paket Bulanan: 19 USD/bulan;
 3. Paket Premium Plus: 306 USD + registrasi ulang 115 USD/tahun, 8 kernel/desktop tanpa batas waktu;
5. Lisensi Pendidikan untuk Akademi & Universitas: 2 kunci aktivasi, desktop+cloud, 8 kernel/desktop, 12000 kali pemanggilan Wolfram Alpha per bulan, 15 GB penyimpanan cloud, 1 GB batas ukuran file cloud, 10 menit batas perhitungan cloud, 2 GB batas memori cloud, 50000 kredit cloud
 1. Paket Premium: 815 USD/tahun;

2. Paket Bulanan: 98 USD/bulan;
3. Paket Premium Plus: 1630 USD + registrasi ulang 448 USD/tahun, 8 kernel/desktop tanpa batas waktu;
6. Lisensi Pendidikan untuk Pemerintah: 2 kunci aktivasi, desktop+cloud, 8 kernel/desktop, 12000 kali pemanggilan Wolfram Alpha per bulan, 15 GB penyimpanan cloud, 1 GB batas ukuran file cloud, 10 menit batas perhitungan cloud, 2 GB batas memori cloud, 50000 kredit cloud
 1. Paket Premium: 1620 USD/tahun;
 2. Paket Bulanan: 194 USD/bulan;
 3. Paket Premium Plus: 3240 USD + registrasi ulang 888 USD/tahun, 8 kernel/desktop tanpa batas waktu;
7. Lisensi Pendidikan untuk Komersial: 2 kunci aktivasi, desktop+cloud, 8 kernel/desktop, 12000 kali pemanggilan Wolfram Alpha per bulan, 15 GB penyimpanan cloud, 1 GB batas ukuran file cloud, 10 menit batas perhitungan cloud, 2 GB batas memori cloud, 50000 kredit cloud
 1. Paket Premium: 1800 USD/tahun;
 2. Paket Bulanan: 216 USD/bulan;
 3. Paket Premium Plus: 3600 USD + registrasi ulang 941 USD/tahun, 8 kernel/desktop tanpa batas waktu.

MATLAB

MATLAB (singkatan dari "**MAT**rix **LAB**oratory") adalah bahasa pemrograman multi-paradigma dan lingkungan komputasi numerik yang dikembangkan oleh MathWorks. MATLAB memungkinkan manipulasi matriks, pembuatan plot fungsi dan data, implementasi algoritma, pembuatan antarmuka pengguna, dan antarmuka program yang ditulis dalam bahasa lain.

MATLAB diprogram oleh ahli matematika dan programmer komputer Cleve Moler, berawal dari tesis PhD -nya, pada tahun 1960-an. Moler menjadi profesor matematika di Universitas New Mexico dan mulai mengembangkan MATLAB untuk mahasiswa sebagai hobi. Ia mengembangkan program aljabar linier awal MATLAB pada tahun 1967 bersama supervisor tesisnya, George Forsythe. Selanjutnya ditambahkan program Fortran untuk persamaan linier pada tahun 1971.

Sebelum versi 1.0, MATLAB bukan merupakan bahasa pemrograman, melainkan sebagai kalkulator matriks interaktif sederhana dengan 71 fungsi bawaan. Pada saat itu MATLAB belum dapat digunakan untuk membuat program, belum tersedia program pendamping, tidak ada grafik, juga belum tersedia program ODE (persamaan diferensial biasa) maupun FFT (transformasi Fourier cepat). Versi awal MATLAB yang pertama diselesaikan pada akhir tahun 1970-an. MATLAB pertama kali diumumkan kepada publik pada bulan Februari 1979 di Sekolah Pascasarjana Angkatan Laut di California. Pada saat itu, MATLAB didistribusikan secara gratis ke universitas-universitas. Moler membagikan salinannya di universitas yang dia kunjungi dan MATLAB banyak digunakan di departemen

matematika di kampus universitas.

Pada tahun 1980-an, Cleve Moler bertemu dengan seorang insinyur teknik elektro, John N. Little. Mereka memutuskan untuk memprogram ulang MATLAB dalam bahasa C dan memasarkannya untuk desktop IBM yang menggantikan komputer *mainframe* pada saat itu. John Little dan programer Steve Bangert menulis ulang MATLAB dalam bahasa C, membuat bahasa pemrograman MATLAB, dan mengembangkan fitur untuk program-program pelengkap.

MATLAB dirilis pertama kali sebagai produk komersial pada tahun 1984 dalam Konferensi Kontrol Otomatis di Las Vegas. Perusahaan **MathWorks, Inc.** didirikan untuk mengembangkan MATLAB, kemudian bahasa pemrograman MATLAB dirilis. Penjualan MATLAB pertama terjadi pada tahun berikutnya, ketika Nick Trefethen dari Institut Teknologi Massachusetts membeli sepuluh salinan. Menjelang akhir tahun 1980-an, ratusan salinan MATLAB telah terjual ke beberapa universitas untuk digunakan mahasiswa. MATLAB menjadi populer sebagian besar berkat adanya program-program pendamping yang dibuat oleh para ahli di berbagai bidang untuk melakukan perhitungan khusus dalam matematika. Banyak program pendamping yang dikembangkan merupakan hasil dari mahasiswa Stanford yang menggunakan MATLAB, kemudian membawa perangkat lunak tersebut ke sektor swasta.

Seiring berjalannya waktu, MATLAB ditulis ulang untuk sistem operasi awal yang dibuat oleh perusahaan Digital Equipment Corporation, VAX, Sun Microsystems, dan untuk komputer-komputer personal dengan sistem operasi Unix. Versi 3 dirilis pada tahun 1987. Perakit MATLAB pertama dikembangkan oleh Stephen C. Johnson pada tahun 1990-an.

Pada tahun 2000, MathWorks menambahkan pustaka berbasis Fortran untuk aljabar linier di MATLAB 6, menggantikan program pustaka aslinya, yakni LINPACK dan EISPACK yang ditulis dengan C. Program pendamping MATLAB untuk komputasi paralel dirilis pada Konferensi Supercomputing 2004 dan dukungan untuk unit pemrosesan grafis (GPU) ditambahkan ke dalamnya pada tahun 2010.

Meskipun MATLAB ditujukan terutama untuk komputasi numerik, namun software ini juga mampu melakukan perhitungan simbolik dengan menggunakan program pendamping MuPAD, Maple, atau Mathematica. Paket Simulink menambahkan simulasi multi-domain grafis dan desain berbasis model untuk sistem dinamis dan tertanam.

Beberapa perubahan besar dilakukan pada MATLAB versi 8, yang dirilis tahun 2012. Antarmuka pengguna dikerjakan ulang dan fungsionalitas Simulink diperluas. Pada tahun 2016, MATLAB memperkenalkan beberapa perbaikan teknis dan antarmuka pengguna, termasuk *notebook* **Live Editor** MATLAB, dan fitur lainnya. Pada tahun 2017, lebih dari 5000 perguruan tinggi dan universitas global menggunakan MATLAB untuk mendukung pembelajaran dan penelitian.

Pada tahun 2020, MATLAB memiliki lebih dari empat juta pengguna di seluruh dunia yang berasal dari berbagai latar belakang teknik, sains, dan ekonomi. Versi terbaru MATLAB adalah R2024a dirilis 21 Maret 2024.

Aplikasi MATLAB dibangun dengan bahasa pemrograman MATLAB. Antarmuka MATLAB memiliki "Jendela Perintah" yang berisi baris-baris perintah matematika interaktif, termasuk perintah untuk membuka dan menjalankan file teks yang berisi program MATLAB. MATLAB dapat memanggil fungsi dan program yang ditulis dalam bahasa pemrograman C atau Fortran. File MEX (MATLAB *executable*) adalah file objek yang dapat dipanggil secara dinamis yang dibuat dengan mengkompilasi fungsi-fungsi tersebut. Sejak tahun 2014 peningkatan antarmuka dua arah dengan Python telah ditambahkan.

Pustaka MATLAB yang ditulis dalam bahasa Perl, Java, ActiveX atau .NET dapat langsung dipanggil dari MATLAB, dan banyak pustaka MATLAB (misalnya pustaka terkait dukungan XML atau SQL) diimplementasikan sebagai pemanggil program Java atau ActiveX. Memanggil MATLAB dari Java lebih rumit, tetapi dapat dilakukan dengan program tambahan untuk MATLAB yang dijual terpisah oleh MathWorks, atau menggunakan mekanisme yang tidak didokumentasikan yang disebut JMI (*Java-to-MATLAB Interface*). Jangan bingung dengan istilah JMI lain, *Java Metadata Interface*, yang ini tidak terkait sama sekali. API MATLAB resmi untuk Java ditambahkan pada tahun 2016.

Berikut adalah harga lisensi software MATLAB dan Simulink.

1. Lisensi MATLAB:

1. Lisensi Standar MATLAB perorangan: 980 USD/tahun/pengguna atau 2450 USD tanpa batas waktu, untuk komersial, pemerintah, atau lainnya.
2. Lisensi *Startup* (Perusahaan pemula): 3950 USD/tahun (MATLAB dan Simulink + 90+ produk tambahan), untuk perusahaan tahap awal yang disetujui.
3. Lisensi Akademik: 275 USD/tahun/pengguna atau 550 USD tanpa batas waktu per pengguna, untuk pembelajaran dan penelitian akademis di lembaga pemberi gelar.
4. Lisensi Mahasiswa: 55 USD tanpa batas waktu per mahasiswa (MATLAB dan Simulink + 10 produk tambahan), untuk penunjang pembelajaran di lembaga pemberi gelar.
5. Lisensi Rumah: 95 USD tanpa batas waktu per pengguna, hanya untuk individu (perorangan).

2. Lisensi Simulink:

1. Lisensi Standar Simulink perorangan: 1480 USD/tahun/pengguna atau 3700 USD tanpa batas waktu, untuk komersial, pemerintah, atau lainnya.
2. Lisensi *Startup* (Perusahaan pemula): 3950 USD/tahun (MATLAB dan Simulink + 90+ produk tambahan), untuk perusahaan tahap awal

yang disetujui.

3. Lisensi Akademik: 275 USD/tahun/pengguna atau 550 USD tanpa batas waktu per pengguna, untuk pembelajaran dan penelitian akademis di lembaga pemberi gelar.
4. Lisensi Mahasiswa: 55 USD tanpa batas waktu per mahasiswa (MATLAB dan Simulink + 10 produk tambahan), untuk penunjang pembelajaran di lembaga pemberi gelar.
5. Lisensi Rumah: 95 USD tanpa batas waktu per pengguna, hanya untuk individu (perorangan).

SageMath

SageMath (sebelumnya **Sage** atau **SAGE**, singkatan "*System for Algebra and Geometry Experimentation*") adalah sistem aljabar komputer (CAS) dengan fitur yang mencakup banyak aspek matematika, termasuk aljabar, kombinatorik, teori graf, teori grup, manifold terdiferensiasi, analisis numerik, teori bilangan, kalkulus dan statistika.

SageMath adalah perangkat lunak gratis, didistribusikan di bawah ketentuan Lisensi Publik Umum GNU versi 3. Versi pertama SageMath dirilis pada 24 Februari 2005 sebagai perangkat lunak gratis dan sumber terbuka di bawah ketentuan Lisensi Publik Umum GNU versi 2. Tujuan semula SageMath adalah sebagai "alternatif sumber terbuka untuk Magma, Maple, Mathematica, dan MATLAB". Pencetus dan pemimpin proyek SageMath, yakni William Stein, adalah seorang ahli matematika di Universitas Washington. Meskipun Microsoft mensponsori versi asli SageMath untuk sistem operasi Windows, sebelum tahun 2016 tidak ada rencana untuk mengimplementasikan SageMath pada sistem operasi Windows secara murni, dan pengguna Windows harus menggunakan teknologi *virtualisasi* seperti **VirtualBox** untuk menjalankan SageMath. Dengan pengembangan yang didanai oleh proyek OpenDreamKit berhasil dikembangkan versi SageMath 8.0 (Juli 2017) untuk Windows 64-bit (menggunakan Cygwin). Mulai SageMath 10.0 (Mei 2023) dijalankan melalui Subsistem Windows untuk Linux (WSL), yakni Linux yang dijalankan melalui Windows, tidak melalui *booting* sendiri. Versi stabil terbaru SageMath adalah 10.2 yang dirilis pada 3 Desember 2023.

Distribusi Linux yang menyertakan paket SageMath adalah Fedora, Arch Linux, Debian, Ubuntu dan NixOS. Di Gentoo, SageMath tersedia melalui *overlay* "sage-on-gentoo". Paket SageMath yang digunakan di NixOS dapat digunakan pada distribusi lain, karena sifat manajer paketnya, Nix, yang *agnostik*. Awalan Gentoo juga menyediakan Sage pada sistem operasi lain.

SageMath menggunakan sintaks yang menyerupai Python, mendukung konstruksi prosedural, fungsional, dan berorientasi objek. Stein menyadari ketika merancang Sage bahwa sudah tersedia banyak paket perangkat lunak matematika sumber terbuka yang telah ditulis dalam berbagai bahasa, seperti C, C++,

Common Lisp, Fortran dan Python.

Daripada mengembangkan ulang sesuatu yang sudah ada, Sage (yang sebagian besar ditulis dalam Python dan Cython) mengintegrasikan banyak paket perangkat lunak CAS khusus ke dalam sebuah antarmuka umum, yang dengannya pengguna hanya perlu mengetahui Python. Filosofi SageMath adalah menggunakan pustaka software sumber terbuka yang tersedia. Oleh karena itu, ia menggunakan banyak pustaka sftware dari proyek lain. Sage sendiri berisi ratusan ribu baris kode unik yang menambahkan fungsi baru dan membuat antarmuka antar komponennya. Meskipun SageMath dikembangkan dari berbagai kode sumber, banyak software pustaka yang disertakan seperti OpenBLAS, FLINT, GAP (sistem aljabar komputer), dan NTL akan disesuaikan dan dioptimalkan untuk komputer target, dengan mempertimbangkan jumlah prosesor, ukuran *cache*-nya, apakah tersedia dukungan perangkat keras untuk instruksi SSE, dll. Penggunaan bahasa Cython dapat meningkatkan kecepatan program SageMath, karena kode Python diubah menjadi C.

Pengembangan SageMath melibatkan baik para mahasiswa maupun programmer profesional. Pengembangan SageMath didukung oleh kerja sukarela dan dana hibah. Mulai 2016 pengembangan Sage dilakukan oleh para pekerja penuh waktu dan didanai oleh hibah Uni Eropa. Pada tahun yang sama, Stein menggambarkan kekecewaannya terhadap kurangnya dana akademik dan kredensial untuk pengembangan perangkat lunak, sehingga ia memutuskan untuk meninggalkan posisi akademisnya dan bekerja penuh waktu pada proyek di perusahaan yang baru didirikan, **SageMath, Inc.**

Versi SageMath yang sudah dirakit (sudah menjadi program biner untuk instalasi, hanya untuk versi Linux, termasuk WSL, dan macOS) dan kode sumber program aslinya tersedia dari situs SageMath <https://www.sagemath.org/>.

1. Paket matematika yang terdapat dalam SageMath:

- Aljabar (GAP, Singular, FLINT);
- Geometri aljabar (Singular);
- Aritmatika presisi sebarang (GMP, MPFR, MPFI, NTL, mpmath, Arb);
- Geometri aritmatika (PARI/GP, NTL, mwrank, ECM);
- Kalkulus (Maxima, SymPy, GiNaC, Giac, FriCAS);
- Kombinatorik (Symmetrica, Sage-Combinat);
- Aljabar linier (ATLAS, BLAS, LAPACK, NumPy, LinBox, IML, GSL);
- Teori graf (NetworkX);
- Teori kelompok (GAP);
- Perhitungan numerik (GSL, SciPy, NumPy, ATLAS);
- Teori bilangan (PARI/GP, FLINT, NTL);
- Komputasi statistik (R, SciPy).

2. Paket lain yang terdapat dalam SageMath:

- Terminal baris perintah (IPython);
- Basis data (ZODB, SQLite)

- Antarmuka grafis (SageMath Notebook, MathJax (sebelumnya jsMath));
 - Grafik (matplotlib, Tachyon, GD, Jmol);
 - Bahasa pemrograman interaktif (Python)
 - Jaringan (Twisted)
3. Paket Matematika lainnya tersedia untuk SageMath:
- Geometri diferensial dan kalkulus tensor (Manifold Sage)

Scilab

Scilab adalah software komputasi numerik lintas platform (Windows, Linux, macOS) yang gratis dan bersumber terbuka serta merupakan bahasa pemrograman berorientasi numerik tingkat tinggi. Kemampuan Scilab mencakup bidang-bidang matematika, optimisasi, statistika, pemrosesan sinyal, dan sistem kendali. Scilab dapat digunakan untuk analisis numerik, visualisasi data, pengembangan algoritma dan aplikasi. Dengan Scilab Anda dapat melakukan pemrosesan sinyal, analisis statistika, perbaikan citra, simulasi dinamika fluida, optimasi numerik, dan pemodelan, simulasi sistem dinamik eksplisit dan implisit.

Scilab merupakan salah satu dari dua software utama dengan sumber terbuka sebagai pengganti MATLAB, yang lainnya adalah GNU Octave. Scilab kurang menekankan kompatibilitas sintaksis dengan MATLAB dibandingkan Octave, tetapi cukup mirip sehingga peralihan dari MATLAB ke Scilab dan sebaliknya dapat dilakukan dengan mudah.

Scilab adalah bahasa pemrograman tingkat tinggi yang berorientasi numerik. Bahasa ini menyediakan lingkungan pemrograman interpreter, dengan tipe data utama berupa matriks. Dengan menggunakan komputasi berbasis matriks, jenis data dinamis, dan manajemen memori otomatis, berbagai perhitungan untuk masalah numerik dapat dilakukan dengan sedikit (baris) perintah, dibandingkan dengan solusi serupa yang menggunakan bahasa tradisional, seperti Fortran, C, atau C++. Hal ini memungkinkan pengguna dengan cepat membuat model untuk berbagai masalah matematika. Selain menyediakan operasi matriks sederhana seperti perkalian, Scilab juga menyediakan berbagai operasi untuk perhitungan tingkat tinggi seperti korelasi dan aritmatika bilangan kompleks multidimensi.

Scilab juga menyertakan paket gratis yang disebut **Xcos** untuk pemodelan dan simulasi sistem dinamis eksplisit dan implisit, termasuk subsistem kontinu dan diskrit. Xcos adalah *open source* yang setara dengan **Simulink** dari **MathWorks**.

Karena sintaksis Scilab mirip dengan MATLAB, Scilab menyertakan penerjemah kode sumber untuk membantu konversi kode dari MATLAB ke Scilab. Scilab tersedia gratis dengan lisensi *GNU General Public License (GPL) v2.0*. Karenanya, beberapa kontribusi pengguna telah diintegrasikan ke dalam program utama.

Sintaks Scilab sebagian besar didasarkan pada bahasa MATLAB. Cara paling

sederhana untuk menjalankan perintah-perintah Scilab adalah dengan menuliskannya pada baris perintah, `-->`, di layar/jendela perintah grafis. Dengan cara ini, Scilab dapat digunakan sebagai *shell* (terminal) matematika interaktif.

Scilab menyediakan ratusan fungsi matematika, termasuk fungsi-fungsi untuk menggambar grafik 2-D dan 3-D. Berikut adalah daftar sebagian kemampuan Scilab.

- Matematika & Simulasi: Scilab untuk aplikasi teknik dan sains biasa termasuk operasi matematika dan analisis data.
- Visualisasi 2D & 3D: Scilab untuk menggambar grafik fungsi, memberi anotasi, dan mengeksplor data serta membuat dan menyesuaikan berbagai jenis plot dan diagram.
- Optimasi: Scilab untuk menyelesaikan masalah optimasi kontinu dan diskrit, baik yang berkendala maupun yang tidak berkendala.
- Statistika: Scilab untuk melakukan analisis dan pemodelan data.
- Desain & Analisis Sistem Kontrol: Scilab menyediakan algoritma dan alat standar untuk mempelajari sistem kendali.
- Pemrosesan Sinyal: Scilab mampu memvisualisasikan, menganalisis, dan memfilter sinyal dalam domain waktu dan frekuensi.
- Pengembangan Aplikasi: Fungsionalitas asli Scilab dapat dikembangkan lebih lanjut dan mampu melakukan pertukaran data dengan alat eksternal.
- Xcos - Pemodel dan simulator sistem dinamis hibrid: Paket Scilab untuk pemodelan dan simulasi sistem mekanik, sirkuit hidrolik, sistem kontrol, dan sebagainya.

Scilab memiliki beberapa software tambahan untuk berbagai tujuan tertentu, seperti

- Scilab Image Processing Toolbox (SIP) dan variannya (seperti SIVP);
- Toolbox Wavelet Scilab;
- Modul Java dan .NET Scilab;
- Modul Akses Jarak Jauh Scilab.

Paket-paket tersebut, dan paket-paket Scilab lainnya, tersedia di portal repositori ATOMS (*AuTomatic mOdules Management for Scilab*) atau Scilab forge.

Sejarah perangkat lunak Scilab dimulai pada tahun 80-an, dengan nama **Blaise**, perangkat lunak CACSD (*Computer Aided Control System Design*) yang dibuat di IRIA (Institut Perancis untuk Penelitian Ilmu Komputer dan Kontrol) dan dikembangkan terutama oleh François Delebecque dan Serge Steer dengan tujuan menyediakan alat dalam Kendali Otomatis untuk para peneliti. Pengembangan perangkat lunak ini terinspirasi oleh perangkat lunak MATLAB Fortran yang dikembangkan oleh Cleve Moler, yang kemudian bersama John Little menjadi pendiri perusahaan "The MathWorks".

Pada tahun 1984, nama Blaise menjadi **Basile** dan didistribusikan selama bebe-

rapa tahun oleh **Simulog**, perusahaan *startup* pertama **Inria** (Institut Nasional Perancis untuk Penelitian Ilmu Komputer dan Kontrol).

Pada awal tahun 90-an, Simulog berhenti mendistribusikan Basile dan nama perangkat lunaknya menjadi **Scilab** (awalnya **Ψlab**, *Psilab*), kemudian dikembangkan oleh Inria dalam **Grup Scilab** yang terdiri atas enam peneliti, yakni: Jean-Philippe Chancelier dari ENPC (*École Nationale des Ponts et Chaussées*), François Delebecque, Claude Gomez, Maurice Goursat, Ramine Nikoukhah dan Serge Steer dari Inria.

Selanjutnya, Inria memutuskan untuk mendistribusikan Scilab sebagai software *open source* gratis. Scilab 1.1, versi pertama Scilab yang dirilis, dipasang di situs *ftp anonim* pada tanggal 2 Januari 1994. Grup Scilab, dengan kolaborasi aktif pengembang eksternal, mengembangkan Scilab hingga akhir tahun 2002 (Scilab versi 2.7), mendistribusikan program mentah dan program biner Scilab di Internet.

Pada bulan Mei 2003, setelah mempertimbangkan peningkatan jumlah orang yang mengunduh dan menggunakan Scilab, dan untuk memastikan masa depan, pengembangan, pemeliharaan, dukungan dan promosi, Inria memutuskan untuk membentuk Konsorsium Scilab dengan dukungan perusahaan dan organisasi akademik.

Konsorsium Scilab kemudian mengintegrasikan jaringan penelitian **Digiteo Foundation** pada bulan Juli 2008, untuk menyediakan lingkungan pengembangan Scilab secara berkelanjutan. Scilab kemudian dikembangkan, dipelihara dan dipromosikan oleh Konsorsium Scilab di dalam Digiteo. Sejak 2008 juga Scilab didistribusikan dengan lisensi CeCILL, jenis lisensi *open source* yang kompatibel dengan GPL.

Untuk menjamin masa depan Scilab, pada bulan Juni 2010 didirikan perusahaan **Scilab Enterprises** dengan dukungan Inria. Scilab Enterprises bertanggung jawab penuh atas edisi dan pengembangan Scilab sejak Juli 2012. Berdasarkan model bisnis *open source* klasik, Scilab Enterprises juga menawarkan layanan komprehensif dan dukungan profesional pada Scilab. Tujuan utama Scilab Enterprises adalah membantu menjadikan penggunaan Scilab lebih efektif dan mudah.

Setelah 5 tahun Scilab dikembangkan oleh Scilab Enterprises, pada awal tahun 2017 tim pengembang bergabung dengan **ESI Group** karena Scilab Enterprises diakuisisi oleh perusahaan tersebut. ESI Group adalah pionir dan penyedia terkemuka dunia dalam Virtual Prototyping, yang memanfaatkan fisika material. Tim tetap berkomitmen pada perangkat lunak Scilab yang gratis dan *open source* untuk komputasi numerik. Dengan pengalaman OpenFOAM, ESI Group telah menunjukkan komitmennya terhadap perangkat lunak sumber terbuka bagi para insinyur dan ilmuwan.

Sejak 2019 dan Scilab 6.0.2, Universitas Teknologi Compiègne menyediakan sumber daya untuk membangun dan memelihara versi macOS. Sejak pertengahan tahun 2022 tim Scilab menjadi bagian dari Dassault Systèmes. Versi stabil terkini Scilab adalah Scilab 2024.0.0 yang dirilis pada 24 Oktober 2023.

SMath Studio

SMath Studio adalah software matematika kecil namun handal, berbasis buku catatan matematika (editor WYSIWYG), sumber tertutup, *freeware* (gratis untuk pengguna pribadi, tetapi tidak bebas, harus mendaftar), mirip dengan Mathcad. SMath Studio dibuat oleh Andrey Ivashov (seorang ahli matematika dari St. Petersburg Rusia, sebagai CEO **SMath LLC**, perusahaan yang mengembangkan dan mengedarkannya) sejak 2005. SMath Studio dapat dijalankan pada komputer dan perangkat berbasis Windows, Linux, iOS, Android, Platform Windows Universal, dan pada beberapa perangkat genggam, serta dijalankan melalui Web (*Cloud*). Versi terbaru SMath Studio adalah 1.1.8763.0 yang dirilis pada 29 Desember 2023.

Meskipun tidak ada perbedaan dari sisi kemampuan software, versi gratis SMath Studio memiliki keterbatasan untuk mengintegrasikan *add-ons* tambahan komersial (berbayar) dan fitur SMath Viewer. Versi SMath berbayar tidak membatasi kedua hal tersebut. Berikut adalah daftar harga lisensi SMath Studio "Advanced" dan "Unlimited".

Lisensi Personal:

Nama Lisensi (Durasi)	Unit	Kuantitas	Harga (USD)
SMath Studio "Advanced", 1 bulan	PC	1	5.00
SMath Studio "Advanced", 6 bulan	PC	1	30.00
SMath Studio "Advanced", 12 bulan	PC	1	48.00

Lisensi Pelajar/Mahasiswa:

Nama Lisensi (Durasi)	Unit	Kuantitas	Harga (USD)
SMath Studio "Advanced", 1 bulan	PC	1	4.00
SMath Studio "Advanced", 6 bulan	PC	1	18.00
SMath Studio "Advanced", 12 bulan	PC	1	24.00

Lisensi untuk institusi resmi:

Nama Lisensi (Durasi)	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD
(Banyak PC):	1 – 4	≥ 5	≥ 20	≥ 50	≥ 100	≥ 1000
SMath Studio "Unlimited", 12 bulan	564.00	504.00	456.00	396.00	312.00	192.00
SMath Studio "Unlimited", 12 bulan, <i>Floating license</i>	-	756.00	684.00	588.00	468.00	288.00
SMath Studio "Unlimited", selamanya.	1,032.00	936.00	840.00	720.00	576.00	360.00
SMath Studio "Unlimited", selamanya, <i>Floating license</i>	-	1,392.00	1,248.00	1,080.00	864.00	528.00

Lisensi untuk lembaga pendidikan:

Nama Lisensi (Durasi)	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD
(Banyak PC):	1 – 4	≥ 5	≥ 20	≥ 50	≥ 100	≥ 1000
SMath Studio "Unlimited", 12 bulan	192.00	180.00	156.00	132.00	108.00	72.00
SMath Studio "Unlimited", 12 bulan, <i>Floating license</i>	-	264.00	228.00	192.00	156.00	108.00
SMath Studio "Unlimited", selamanya.	288.00	264.00	240.00	216.00	168.00	96.00
SMath Studio "Unlimited", selamanya, <i>Floating license</i>	-	384.00	360.00	312.00	240.00	144.00

Lisensi untuk institusi pemerintah:

Nama Lisensi (Durasi)	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD	Harga (per PC), USD
(Banyak PC):	1 – 4	≥ 5	≥ 20	≥ 50	≥ 100	≥ 1000
SMath Studio "Unlimited", 12 bulan	456.00	420.00	372.00	324.00	252.00	156.00
SMath Studio "Unlimited", 12 bulan, <i>Floating license</i>	-	624.00	552.00	480.00	372.00	228.00
SMath Studio "Unlimited", selamanya	840.00	744.00	672.00	576.00	456.00	288.00
SMath Studio "Unlimited", selamanya, <i>Floating license</i>	-	1,104.00	1,008.00	864.00	672.00	432.00

Software ini menyediakan banyak fitur komputasi dan antarmuka pengguna yang kaya yang diterjemahkan ke dalam sekitar 40 bahasa berbeda. Aplikasi ini juga memuat buku referensi matematika terintegrasi (berupa kumpulan rumus dan identitas matematika) serta contoh-contoh berbagai perhitungan matematika dengan SMath Studio. Di antara kemampuan SMath Studio adalah:

- memecahkan persamaan diferensial;
- grafik fungsi dua atau tiga dimensi;
- perhitungan simbolik, termasuk penyelesaian sistem persamaan;
- operasi matriks, termasuk determinan;
- menemukan akar polinomial dan fungsinya;
- diferensiasi fungsi secara simbolis dan numerik;
- integrasi numerik;
- program pengulangan multi-baris sederhana;
- fungsi buatan pengguna;
- dukungan satuan pengukuran yang lengkap.

Aplikasi ini juga dapat dengan mudah diperluas sesuai kebutuhan penggunanya. Alat Manajer Ekstensi bawaan memungkinkan untuk mendapatkan akses ke ratusan sumber daya resmi dan pihak ketiga yang berupa: contoh penggunaan, *plug-in*, aplikasi berbasis SMath Viewer, cuplikan, terjemahan antarmuka, buku interaktif, buku pegangan, dan tutorial.

Berbagai contoh penggunaan SMath Studio tersedia di portal Wiki SMath (<https://smath.info/wiki/>). Berikut adalah di antara berbagai bidang contoh penerapan SMath. File-file contoh dapat diunduh melalui Web tersebut.

- Aplikasi bidang teknik:
 - Defleksi non-linier kantilever fleksibel;
 - Reaktor kimia *batch*;
 - Gaya lateral ban, menggunakan model Pacejka awal;
 - Model Atmosfer NASA (Satuan SI);
 - Pengocok elektromagnetik untuk menggerakkan sistem kebebasan berderajat tunggal;
 - Perkiraan sederhana kinerja pesawat satu propeler, yang digerakkan oleh angin;
 - Desain Ramp - pengujian momen tekuk dan defleksi maksimal (Satuan USA);
 - Desain Kolom - berdasarkan Manual Konstruksi Baja AISC (Satuan USA);
 - Perhitungan Penolakan Panas Mesin (Satuan USA);
 - Menghitung Sifat Luas Luas Komposit Persegi Panjang (Satuan USA)
 - Menghitung sifat-sifat poligon generik (keliling, luas, pusat massa, momen luas kedua, jari-jari girasi, modulus penampang elastis, modulus penampang plastis, orientasi sumbu inersia utama, momen inersia utama, jari-jari girasi terhadap sumbu inersia utama) multibahasa (ENG/ITA);
 - Teknik elektro: menghitung arus, tegangan, daya nyata, reaktif, dan semu dalam rangkaian AC menggunakan domain kompleks dan menggunakan satuan khusus untuk besaran daya AC.
- Keuangan:
 - Persamaan Keuangan.
- Metode numerik:
 - Metode Newton-Raphson untuk menyelesaikan persamaan;
 - Sistem persamaan aljabar nonlinier, metode Broyden;
 - Polinomial interpolasi Lagrange;
 - Polinomial interpolasi Newton (selisih berhingga maju);
 - Polinomial interpolasi Newton (selisih berhingga mundur);
 - Pencocokan data (regresi linier variabel independen tunggal);
 - Pencocokan data (regresi linier multi-variabel independen);
 - Integrasi Simpson dengan ekstrapolasi Richardson;
 - Metode integrasi Gauss-Legendre;
 - ODE linier orde kedua - metode selisih berhingga;
 - Runge-Kutta orde-4 untuk PDB tingkat 1;
 - Runge-Kutta orde-4 untuk sistem PDB;
 - Runge-Kutta orde-4 untuk PDB tingkat 2;
 - Runge-Kutta orde-5 Adaptif;
 - Grafik fungsi implisit 2 variabel: $f(x,y) = K$ (dengan K adalah konstanta).
- Statistika:
 - Distribusi normal;

- Distribusi t siswa;
- Distribusi chi-kuadrat;
- Distribusi-F;
- Distribusi peluang.
- Manipulasi dan Interpolasi Data:
 - Mengimpor data dari file csv;
 - Interpolasi Bi-linear.
- Teknik Sipil Hidraulik/Mekanika Fluida:
 - Uji Mekanika Fluida;
 - Perhitungan dengan satuan untuk hidrolika;
 - HIDROLIKA (aliran pipa, sistem pipa-pompa, jaringan pipa, pompa, dan aliran saluran terbuka);
 - Distribusi Kecepatan Vertikal pada Saluran Terbuka Halus Hidraulik;
 - Geometri Hidrolik Sungai;
 - Aliran Weir menggunakan Persamaan Rehbock;
 - Persamaan Faktor Gesekan Swamee-Jain untuk Aliran Pipa.
- Rekayasa Struktural:
 - Menghitung Sifat Luas Luas Komposit Persegi Panjang (Satuan USA);
 - Desain Balok Beton Bertulang Menurut NZS 3101-95 (Satuan SI);
 - Desain Sambungan Pelat Ujung Baja;
 - Desain Pelat Beton Bertulang - Tulangan Lentur (Unit AS);
 - Momen bentang tunggal & amplop geser untuk pemuatan HL-93 menggunakan garis pengaruh;
 - Desain bentang tunggal balok kayu yang ditopang sederhana menurut EN 1995-Eurocode 5 (satuan SI).

1.3.5 Software Statistika

Software statistika adalah program komputer yang dapat digunakan untuk melakukan pengelolaan data statistika, menganalisis dan menyajikan data secara efisien dan komprehensif, dan melakukan berbagai uji statistika untuk pengambilan keputusan. Beberapa software CAS dan software analisis numerik tujuan umum, seperti MATLAB, GNU Octave, EMT, Maple, Mathematica dan lain-lain, biasanya juga dapat digunakan untuk melakukan analisis data statistika secara terbatas. Di antara software yang khusus untuk analisis data statistika adalah Minitab, R, SAS, S-PLUS, SPSS, Stan, STATA,

Minitab

Minitab adalah software statistika komersial yang semula dikembangkan dan digunakan untuk mengajar statistika di Universitas Negeri Pennsylvania oleh peneliti Barbara F. Ryan, Thomas A. Ryan, Jr., dan Brian L. Joiner dengan dana dari perusahaan **Triola Statistics** pada tahun 1972. Software Minitab diawali sebagai versi ringan OMNITAB, sebuah program analisis statistika oleh Institut Standar dan Teknologi Nasional (NIST), yang disusun oleh Joseph Hilsenrath pada tahun 1962–1964 untuk IBM 7090.

Minitab didistribusikan oleh **Minitab, LLC**, sebuah perusahaan swasta yang berkantor pusat di State College, Pennsylvania. Minitab digunakan oleh banyak kalangan akademisi maupun industri. Pada bulan Oktober 2020, Minitab meluncurkan versi pertama perangkat lunak statistiknya yang berbasis *cloud*. Mulai Juni 2021, aplikasi Minitab Desktop hanya tersedia untuk Windows, sedangkan versi sebelumnya untuk MacOS (Minitab 19.x) tidak lagi didukung. Versi terbarunya adalah Minitab 21.3 yang dirilis 9 November 2022. Harga software ini tidak diterbitkan secara umum, untuk memperoleh harga lisensi Anda harus menghubungi pihak perusahaan Minitab.

Minitab merupakan software statistika yang sangat lengkap dan handal serta menyediakan alat impor/ekspor data dan serangkaian kemampuan statistika yang komprehensif. Fitur Minitab mencakup asistensi, visualisasi (grafik) statistika termasuk kontur dan plot 3D berputar, statistika dasar, analisis varians, model tetap dan campuran, analisis regresi, analisis sistem pengukuran, analisis kualitas, rancangan percobaan, analisis reliabilitas/survival, dan sebagainya. Daftar fitur lengkap dapat dilihat pada laman <https://www.minitab.com/en-us/products/minitab/features/>. Untuk pengguna tingkat lanjut, tersedia editor baris perintah, yang dapat digunakan untuk mengedit makro (fungsi).

R

R adalah perangkat lunak gratis dan terbuka yang merupakan bahasa pemrograman untuk komputasi statistik dan visualisasi data. R telah diadopsi di bidang penambangan data, bioinformatika, dan analisis data. Software (bahasa) inti R dilengkapi dengan ribuan paket tambahan, yang merupakan kumpulan fungsi atau program yang ditulis dengan R dan siap digunakan di R, dokumentasi, dan data sampel. R didistribusikan dengan lisensi GPL GNU. Pada tanggal 5 Desember 1997, R menjadi proyek GNU ketika versi 0.60 dirilis. Pada tanggal 29 Februari 2000, R versi 1.0 resmi pertama dirilis. Versi terbaru R adalah 4.3.3 (dengan sebutan **Angle Food Cake**) yang dirilis pada 29 Februari 2024. R dapat dijalankan pada sistem operasi UNIX (termasuk Linux), Windows, dan MacOS.

Informasi lengkap software R tersedia di situs resminya (<https://www.r-project.org>), sedangkan distribusi software R dan paket-paketnya dilakukan melalui situs **CRAN (Comprehensive R Archive Network)** (<https://cran.r-project.org>) yang memiliki beberapa situs bayangan (*mirror*) di seluruh dunia. CRAN menyediakan hampir 19 ribu paket kontribusi pada pengguna R. Selain CRAN, terdapat repositori paket-paket R lain, seperti R-Forge, Omegahat, and GitHub.

Untuk memudahkan pengguna mencari dan/atau memilih paket R yang sesuai, CRAN sudah mengklasifikasi paket-paket R berdasarkan topik atau kegunaannya melalui laman CRAN Task Views. Di antara klasifikasi paket yang tersedia adalah paket aktuarial, inferensi Bayes, analisis klaster, persamaan diferensial, distribusi peluang, ekonometrik, epidemi, analisis data fungsional, komputasi super cepat dan paralel, pembelajaran mesin, analisis citra medis, meta-analisis,

matematika numerik, optimisasi, analisis olah raga, analisis survival, dan analisis data spasial. Selain itu, terdapat paket khusus yang merupakan kumpulan paket-paket lain untuk tujuan tertentu, misalnya paket **bioconductor** dan paket **tidyverse**. Paket **bioconductor** menyediakan paket-paket R untuk analisis data genom, DNA komplementer, *microarray*, dan metode pengurutan *throughput* tinggi. Paket **tidyverse** merupakan koleksi paket-paket R untuk sains data.

R pertama kali dikembangkan oleh Profesor Ross Ihaka dan Robert Gentleman sebagai bahasa pemrograman untuk mengajar pengantar statistika di Universitas Auckland, Selandia Baru, pada awal tahun 1990an. Ross Ihaka dan Robert Gentleman memerlukan program statistika untuk digunakan dalam laboratorium pengajaran mereka. Ketika itu, laboratorium komputer mereka hanya memiliki komputer Macintosh dan tidak dilengkapi dengan perangkat lunak yang sesuai. Ihaka dan Gentleman memutuskan untuk menulis program statistika dalam bentuk bahasa yang memiliki sintaks mirip sintaks bahasa S. Versi awal R diberikan kepada *Statlib* di Universitas Carnegie Mellon dan umpan balik pengguna menunjukkan sambutan yang positif.

Bahasa R terinspirasi oleh bahasa pemrograman S, dan sebagian besar program S dapat berjalan tanpa perubahan di R. Bahasa ini juga terinspirasi oleh bahasa Scheme, yang memungkinkan variabel lokal. Nama bahasa R diambil dari dua hal, sebagai penerus bahasa S dan dari huruf pertama nama kedua penulisnya, Ross dan Robert. R ditulis terutama dalam bahasa C, Fortran, dan R itu sendiri (paket-paket tambahan).

Sebagai bahasa interpreter, R memiliki antarmuka baris perintah berbentuk layar terminal berbasis teks. Meskipun, tersedia beberapa antarmuka grafis (GUI) atau lingkungan pengembangan terpadu (IDE), seperti R.app (hanya untuk OSX/macOS), Rattle GUI, R Commander, Rkward, RStudio (antarmuka berbasis dokumen Markdown), Tinn-R, notebook Jupyter (berbasis Web).

SAS

SAS (sebelumnya **Sistem Analisis Statistik**) adalah rangkaian perangkat lunak statistik yang dikembangkan oleh **Institut SAS** untuk manajemen data, analisis lanjutan, analisis multivariat, intelijen bisnis, investigasi kriminal, dan analisis prediktif. SAS dapat menambang, mengubah, mengelola, dan mengambil data dari berbagai sumber dan melakukan analisis statistik terhadapnya. SAS menyediakan antarmuka grafis tunjuk-dan-klik (disebut **SAS Enterprise Guide**) untuk pengguna non-teknis dan lebih banyak lagi melalui bahasa SAS. Perangkat lunak analitik SAS dibangun berdasarkan kecerdasan buatan dan memanfaatkan pembelajaran mesin, pembelajaran mendalam, dan AI generatif untuk mengelola dan memodelkan data.

SAS dikembangkan di Departemen Statistika Eksperimental Universitas Negeri Carolina Utara dari tahun 1966 hingga 1976. Seiring dengan berkembangnya SAS, pada tahun 1976 didirikan **Institut SAS**. SAS dikembangkan lebih lanjut

pada tahun 1980-an dan 1990-an dengan penambahan prosedur statistika baru, komponen baru dan pengenalan JMP (bagian dari Institut SAS). Antarmuka tunjuk-dan-klik ditambahkan mulai versi 9 pada tahun 2004. Produk analisis media sosial ditambahkan pada tahun 2010. Sejak awal, SAS telah berkembang menjadi sistem terintegrasi untuk analisis data dan eksplorasi. Sistem SAS telah digunakan di berbagai bidang bisnis dan institusi akademik di seluruh dunia.

SAS menyediakan paket untuk mendukung berbagai tugas analitik data. Komponen SAS/STAT berisi kemampuan yang biasanya dikaitkan dengan analisis data. SAS/STAT mendukung analisis varians (ANOVA), regresi, analisis data kategorikal, analisis multivariat, analisis kelangsungan hidup, analisis psikometri, analisis kluster, dan analisis non-parametrik. Paket SAS/INSIGHT menerapkan strategi visualisasi. Visualisasi dapat dihubungkan ke beberapa jendela untuk mengungkap tren, menemukan pencilan, dan dengan mudah membedakan pola-pola halus. SAS juga menyediakan bahasa pemrograman matriks melalui Sistem SAS/IML, yang memungkinkan pengembangan algoritma statistika khusus.

Program SAS memiliki langkah-langkah DATA untuk mengambil dan memanipulasi data, PROC (prosedur) untuk menganalisis data, dan juga memiliki fungsi. Setiap langkah terdiri atas serangkaian pernyataan. Langkah DATA memiliki pernyataan untuk mengambil tindakan, dan pernyataan deklaratif untuk membaca kumpulan data atau mengubah tampilan data. Langkah DATA terdiri atas dua tahap: kompilasi dan eksekusi. Pada tahap kompilasi, pernyataan deklaratif diproses dan kesalahan sintaksis diidentifikasi. Setelah itu, tahap eksekusi memproses setiap pernyataan yang dapat dieksekusi secara berurutan. Kumpulan data disusun dalam tabel dengan baris yang disebut "pengamatan" dan kolom yang disebut "variabel". Selain itu, setiap bagian data memiliki deskriptor dan nilai.

Pernyataan PROC memanggil prosedur tertentu untuk melakukan analisis dan pelaporan kumpulan data untuk menghasilkan statistik, analisis, dan grafik. Terdapat lebih dari 300 prosedur dan masing-masing prosedur melakukan sejumlah besar pekerjaan statistika. Pernyataan PROC juga dapat menampilkan hasil, mengurutkan data, atau melakukan operasi lainnya.

SAS juga mendukung penggunaan makro yang berupa potongan kode atau variabel yang telah diberi kode dan dirujuk untuk melakukan tugas berulang.

Data SAS dapat diekspor ke HTML, PDF, Excel, RTF atau format lain menggunakan *Output Delivery System*, yang pertama kali diperkenalkan pada tahun 2007. Untuk menghasilkan kode guna memanipulasi data atau melakukan analisis tanpa menggunakan bahasa pemrograman SAS dapat digunakan antarmuka grafis tunjuk-dan-klik (*SAS Enterprise Guide*).

Rangkaian perangkat lunak SAS, selain paket dasar **Base SAS**, dilengkapi dengan lebih dari 200 paket tambahan (komponen) di antaranya:

- Base SAS – Prosedur dasar dan manajemen data
- SAS/STAT – Analisis statistik
- SAS/GRAPH – Grafik dan presentasi
- SAS/OR – Riset operasi
- SAS/ETS – Analisis Ekonometrika dan Runtun Waktu
- SAS/IML – Bahasa matriks interaktif
- SAS/AF – Fasilitas aplikasi
- SAS/QC – Kendali kualitas
- SAS/INSIGHT - Penambangan data
- SAS/PH – Analisis uji klinis
- Enterprise Miner – penambangan data
- Enterprise Guide – editor kode & manajer proyek berbasis GUI
- SAS EBI – Rangkaian Aplikasi Intelijen Bisnis
- SAS Grid Manager – Manajer lingkungan komputasi grid SAS
- SAS CI360 - Intelijen Pelanggan.

Perkembangan SAS dimulai pada tahun 1966 setelah North Carolina State University mempekerjakan kembali Anthony Barr untuk memprogram perangkat lunak analisis varians dan regresi sehingga dapat dijalankan pada komputer IBM System/360. Proyek ini didanai oleh Institut Kesehatan Nasional (NIH) dan pada awalnya dimaksudkan untuk menganalisis data pertanian guna meningkatkan hasil panen. Barr dibantu oleh mahasiswa James Goodnight, yang mengembangkan program-program kecil statistika, dan keduanya menjadi pemimpin proyek. Pada tahun 1968, Barr dan Goodnight mengintegrasikan program regresi berganda baru dan analisis varians. Pada tahun 1972, setelah rilis pertama SAS, proyek tersebut kehilangan pendanaan. Menurut Goodnight, hal ini karena NIH hanya ingin mendanai proyek dengan aplikasi medis. Goodnight terus mengajar di universitas dengan gaji 1 USD dan akses ke komputer mainframe untuk mengerjakan proyek tersebut, sampai akhirnya proyek tersebut mendapat dana dari University Statisticians of the Southern Experiment Stations (USSES) pada tahun berikutnya. John Sall bergabung dengan proyek ini pada tahun 1973 dan berkontribusi pada perangkat lunak ekonometrika, deret waktu, dan aljabar matriks. Penyumbang awal lainnya, Carroll G. Perkins, berkontribusi pada program awal SAS. Jolayne W. Service dan Jane T. Helwig membuat dokumentasi pertama SAS.

SAS memiliki banyak rilis sejak tahun 1972. Versi pertama SAS, dari SAS 71 hingga SAS 82, diberi nama berdasarkan tahun peluncurannya. Pada tahun 1971, SAS 71 dirilis secara terbatas, hanya untuk mainframe IBM dan memiliki elemen utama pemrograman SAS, seperti langkah DATA dan prosedur yang paling umum, yaitu PROC. Tahun berikutnya versi lengkap dirilis sebagai SAS 72, yang memperkenalkan pernyataan MERGE dan menambahkan fitur untuk menangani data yang hilang atau menggabungkan kumpulan data. Perkembangan SAS telah digambarkan sebagai "titik infleksi" dalam sejarah kecerdasan buatan. Pada tahun 1976, Barr, Goodnight, Sall, dan Helwig menghapus proyek tersebut dari Negara Bagian Carolina Utara dan menggabungkannya sebagai SAS Institute,

Inc.

SAS didesain ulang pada SAS 76. Pernyataan INPUT dan INFILE ditingkatkan sehingga dapat membaca sebagian besar format data yang digunakan oleh main-frame IBM. Pembuatan laporan juga ditambahkan melalui pernyataan PUT dan FILE. Kemampuan untuk menganalisis model linier umum juga ditambahkan seperti prosedur FORMAT, yang memungkinkan pengembang untuk menyesuaikan tampilan data. Pada tahun 1979, SAS 79 menambahkan dukungan untuk sistem operasi IBM VM/ CMS dan memperkenalkan prosedur DATASETS. Tiga tahun kemudian, SAS 82 memperkenalkan bahasa makro awal dan prosedur APPEND.

Dimulai dengan SAS 4, dikeluarkan pada tahun 1984, rilis SAS mengikuti konvensi penamaan berurutan yang tidak berdasarkan tahun rilis. SAS versi 4 memiliki fitur terbatas, namun membuat SAS lebih mudah diakses. Versi 5 memperkenalkan bahasa makro lengkap, subskrip *array*, dan antarmuka pengguna interaktif layar penuh yang disebut **Display Manager**. Pada tahun 1985, SAS ditulis ulang dalam bahasa pemrograman C. Hal ini mengaktifkan Arsitektur MultiVendor SAS yang memungkinkan perangkat lunak berjalan di UNIX, MS-DOS, dan Windows. Sebelumnya, SAS ditulis dalam bahasa PL/I, Fortran, dan bahasa Assembly.

Pada tahun 1980-an dan 1990-an, SAS merilis sejumlah komponen untuk melengkapi Base SAS. SAS/GRAPH, yang menghasilkan grafik, dirilis pada tahun 1980, serta komponen SAS/ETS, yang mendukung analisis ekonometrik dan deret waktu. Komponen yang ditujukan untuk pengguna farmasi, SAS/PH-Clinical, dirilis pada tahun 1990-an. Badan Pengawas Obat dan Makanan melakukan standarisasi penggunaan SAS/PH-Clinical untuk aplikasi obat baru pada tahun 2002. Produk-produk vertikal seperti SAS Financial Management dan SAS Human Capital Management (kemudian disebut CFO Vision dan HR Vision) juga diperkenalkan.

JMP dikembangkan oleh salah satu pendiri SAS, John Sall, dan tim pengembang, untuk memanfaatkan antarmuka pengguna grafis yang diperkenalkan pada Apple Macintosh tahun 1984. Nama JMP awalnya merupakan singkatan dari "John's Macintosh Project". JMP dikirimkan pertama kali pada tahun 1989. Versi JMP yang diperbarui dirilis terus menerus setelah tahun 2002 dengan rilis terbaru pada tahun 2016. Pada bulan Januari 2022, JMP menjadi sepenuhnya anak perusahaan milik Institut SAS, yang sebelumnya merupakan unit bisnis perusahaan.

SAS 6 digunakan sepanjang tahun 1990-an dan tersedia pada sistem operasi yang lebih luas, termasuk Macintosh, OS/2, Silicon Graphics, dan PRIMOS. SAS memperkenalkan fitur-fitur baru melalui *dot-release*. Dari 6.06 hingga 6.09, antarmuka pengguna berdasarkan paradigma Windows diperkenalkan dan dukungan untuk SQL ditambahkan. Versi 7 memperkenalkan Output Delivery System (ODS)

dan editor teks yang ditingkatkan. Rilis berikutnya memperbaiki ODS. Misalnya, lebih banyak opsi keluaran ditambahkan di versi 8. Jumlah sistem operasi yang didukung dikurangi menjadi UNIX, Windows, z/OS, dan Linux. SAS 8 dan SAS Enterprise Miner dirilis pada tahun 1999.

Pada tahun 2002, perangkat lunak SAS Text Miner diperkenalkan. Text Miner menganalisis data teks seperti email untuk mencari pola dalam aplikasi intelijen bisnis. Pada tahun 2004, SAS Versi 9.0 dirilis, disebut sebagai "Project Mercury" secara internal, dan dirancang untuk membuat SAS dapat diakses oleh lebih banyak pengguna bisnis. SAS 9.0 menambahkan antarmuka pengguna khusus berdasarkan peran pengguna dan menetapkan antarmuka pengguna tunjuk-dan-klik dari SAS Enterprise Guide sebagai antarmuka pengguna grafis (GUI) utamanya. Fitur Manajemen Hubungan Pelanggan (CRM) ditingkatkan pada tahun 2004 dengan SAS Interaction Management. Pada tahun 2008, SAS mengumumkan Project Unity, yang dirancang untuk mengintegrasikan kualitas data, integrasi data, dan manajemen data master. Sejak rilis 9.3, SAS/STAT memiliki penomoran rilis sendiri. Versi terbaru SAS adalah 9.4M8 yang dirilis 31 Januari 2023. Informasi harga lisensi SAS tidak diterbitkan secara umum, calon pengguna harus menghubungi pihak perusahaan untuk mengetahui harganya.

Versi gratis SAS diperkenalkan untuk siswa pada tahun 2010. SAS Social Media Analytics, alat untuk pemantauan media sosial, keterlibatan dan analisis sentimen, juga dirilis pada tahun itu. SAS Rapid Predictive Modeler (RPM), yang membuat model analitik dasar menggunakan Microsoft Excel, diperkenalkan pada tahun yang sama. Pada tahun 2010, JMP 9 menyertakan antarmuka baru untuk menggunakan bahasa pemrograman R dan *add-in* untuk MS Excel. Tahun berikutnya, *platform* Komputasi Kinerja Tinggi tersedia melalui kemitraan dengan Teradata dan EMC Greenplum. Pada tahun 2011, perusahaan merilis SAS Enterprise Miner 7.1. Perusahaan memperkenalkan 27 produk manajemen data dari Oktober 2013 hingga Oktober 2014 dan memperbarui 160 produk lainnya. Pada SAS Global Forum 2015, SAS mengumumkan beberapa produk baru yang dikhususkan untuk industri berbeda, serta perangkat lunak pelatihan baru.

Perusahaan ini telah berinvestasi dalam pengembangan kecerdasan umum buatan, atau "AI yang kuat", dengan tujuan memajukan pembelajaran mendalam dan pemrosesan bahasa alami hingga mencapai komputasi kognitif. Pada tahun 2019, SAS mengumumkan akan menginvestasikan 1 miliar USD untuk pengembangan kecerdasan buatan tingkat lanjut, pembelajaran mendalam, pemrosesan bahasa alami, dan pembelajaran mesin. Mereka mengumumkan investasi tambahan sebesar 1 miliar USD untuk pengembangan AI pada September 2023.

SPSS

SPSS (Statistika SPSS) adalah rangkaian perangkat lunak statistika yang dikembangkan oleh IBM untuk manajemen data, analitik tingkat lanjut, analisis multivariat, intelijen bisnis, dan investigasi kriminal. Setelah lama diproduksi

oleh perusahaan **SPSS Inc.**, SPSS diakuisisi oleh IBM pada tahun 2009. Versi SPSS yang dirilis sejak tahun 2015 memiliki nama merek **IBM SPSS Statistics**. Nama **SPSS** awalnya merupakan singkatan dari *Statistical Package for the Social Sciences* (**SPSS**), yang mencerminkan pasar aslinya, kemudian diubah menjadi *Statistical Product and Service Solutions* dengan singkatan yang tidak berubah (**SPSS**).

Semula SPSS dikembangkan oleh Norman H. Nie, C. Hadlai (Tex) Hul, dan Dale Brent pada akhir tahun 1960an. Ketiganya pada waktu itu adalah mahasiswa pascasarjana Universitas Stanford. Versi pertama SPSS dirilis pada 1968. Ketiganya kemudian mendirikan **SPSS Inc.** pada tahun 1975. SPSS diperdagangkan secara publik di 1993. Sekarang, IBM memiliki hak atas SPSS.

Awalnya, SPSS ditulis dalam bahasa Fortran dan dirancang untuk pemrosesan *batch* (berjalan sendiri tanpa pantauan manusia) pada mainframe, termasuk misalnya versi IBM dan ICL, awalnya menggunakan kartu berlubang untuk input data dan program. Proses yang dijalankan membaca file perintah dari perintah SPSS dan file input mentah dari data berformat tetap dengan tipe rekaman tunggal, atau "*getfile*" data yang disimpan oleh proses sebelumnya. Untuk menghemat waktu komputer yang berharga, proses "*edit*" dapat dilakukan untuk memeriksa sintaks perintah tanpa menganalisis data. Mulai versi 10 (**SPSS-X**) pada tahun 1983, file data dapat berisi beberapa tipe *record*.

Pada tahun 1984, SPSS memperkenalkan SPSS/PC+ untuk komputer yang menjalankan MS-DOS, diikuti dengan rilis UNIX pada tahun 1988 dan versi Macintosh pada tahun 1990. SPSS memiliki fitur titik-dan- yang intuitif klik antarmuka. Ini desain memberdayakan basis pengguna yang luas untuk melakukan analisis standar. Sebelum SPSS 16.0, versi SPSS yang berbeda tersedia untuk Windows, Mac OS X dan Unix. Statistika SPSS versi 13.0 untuk Mac OS X tidak kompatibel dengan komputer Macintosh berbasis Intel, karena perangkat lunak emulasi Rosetta menyebabkan kesalahan dalam perhitungan. Statistik SPSS 15.0 untuk Windows memerlukan *hotfix* yang dapat diunduh untuk diinstal agar kompatibel dengan Windows Vista. Mulai versi 16.0, SPSS dapat berjalan di Windows, Mac, dan Linux. Antarmuka pengguna grafis ditulis dalam Java. Versi Mac OS disediakan sebagai *biner Universal*, sehingga sepenuhnya kompatibel dengan perangkat keras Mac berbasis PowerPC dan Intel.

Pada 28 Juli 2009 SPSS Inc. mengumumkan bahwa SPSS diakuisisi oleh IBM senilai 1,2 miliar USD. Karena perselisihan mengenai kepemilikan nama "SPSS", antara tahun 2009 dan 2010, produknya disebut sebagai PASW (*Predictive Analytics SoftWare*). Pada Januari 2010, namanya menjadi "**SPSS: An IBM Company**". Pengalihan bisnis secara menyeluruh ke IBM telah dilakukan pada tanggal 1 Oktober 2010. Mulai tanggal tersebut, nama "SPSS: An IBM Company" tidak ada lagi. IBM SPSS kini terintegrasi penuh ke dalam Perusahaan IBM, dan merupakan salah satu merek di bawah **IBM Software Group's Business Analytics Portfolio**, bersama dengan **IBM Algorithmics**, **IBM Cognos**, dan **IBM OpenPages**.

Statistik SPSS versi 25 diluncurkan pada 8 Agustus 2017. Ini menambahkan statistik baru dan lanjutan, seperti hasil solusi efek acak (GENLINMIXED), kesalahan standar yang kuat (GLM/UNIANOVA), dan plot profil dengan bilah kesalahan dalam Statistik Lanjutan dan Kustom Tambahan tabel. V25 juga mencakup kemampuan statistik Bayesian baru, metode inferensi statistik, dan grafik siap publikasi, seperti kemampuan pembuatan grafik baru yang kuat, termasuk templat default baru dan kemampuan untuk berbagi dengan aplikasi Microsoft Office. Versi terbaru SPSS ada SPSS 29 dirilis pada 13 September 2022.

Tersedia pula *software* pendamping dalam keluarga "IBM SPSS" yang digunakan untuk penambangan data dan analisis teks (IBM SPSS Modeler), layanan penilaian kredit waktu nyata (IBM SPSS Collaboration and Deployment Services), dan pemodelan persamaan struktural (IBM SPSS Amos).

SPSS memiliki beragam kemampuan analitik termasuk regresi, pohon klasifikasi, pembuatan tabel, pengujian eksak, analisis kategori, analisis kecenderungan, analisis konjoin, analisis nilai yang hilang, analisis berbasis peta, dan analisis sampel kompleks. Selain itu, SPSS mendukung berbagai produk yang berdiri sendiri termasuk Amos (paket pemodelan persamaan struktural), SPSS Text Analysis for Surveys (paket analisis survei yang memanfaatkan metodologi pemrosesan bahasa alami (NLP)), SPSS Data Entry (paket entri data berbasis web), AnswerTree (paket penargetan segmen pasar), SmartViewer Web Server (paket pembuatan laporan dan diseminasi), SamplePower (paket perhitungan ukuran sampel), DecisionTime dan What if? (paket analisis skenario untuk non-spesialis), SmartViewer untuk Windows (utilitas berbagi grafik/laporan), SPSS WebApp Framework (paket analisis berbasis web), dan Dimensions Development Library (perpustakaan pengambilan data).

SPSS tetap populer, terutama di dalam berbagai karya ilmiah. Bagi banyak peneliti yang menerapkan model standar, SPSS menyelesaikan pekerjaannya. SPSS tetap menjadi alat yang berguna bagi para praktisi di banyak bidang. SPSS banyak digunakan untuk analisis statistika dalam ilmu sosial. SPSS juga digunakan oleh peneliti pasar, peneliti kesehatan, perusahaan survei, pemerintah, peneliti pendidikan, organisasi pemasaran, penambang data, dan lain-lain. Manual SPSS yang asli (Nie, Bent & Hull, 1970) telah digambarkan sebagai salah satu "buku sosiologi yang paling berpengaruh" karena memungkinkan peneliti biasa melakukan analisis statistika secara mandiri. Selain analisis statistika, manajemen data (pemilihan kasus, pembentukan ulang file, dan pembuatan data turunan) dan dokumentasi data (kamus metadata disimpan dalam file data adalah fitur perangkat lunak dasar).

Banyak fitur Statistik SPSS yang dapat diakses melalui menu *pull-down* atau dapat diprogram dengan *bahasa sintaks perintah* 4GL yang dipatenkan. Pemrograman sintaksis perintah memiliki keuntungan berupa keluaran yang dapat direproduksi, menyederhanakan tugas yang berulang, dan menangani manipulasi dan analisis data yang kompleks. Selain itu, beberapa aplikasi kompleks hanya

dapat diprogram dalam sintaksis dan tidak dapat diakses melalui struktur menu. Antarmuka menu tarik-turun juga menghasilkan sintaks perintah: ini dapat ditampilkan di *output*, meskipun pengaturan *default* harus diubah agar sintaks terlihat oleh pengguna. Perintah tersebut juga dapat disisipkan ke dalam file sintaksis menggunakan tombol "tempel" yang ada di setiap menu. Program dapat dijalankan secara interaktif atau tanpa pengawasan, menggunakan **Fasilitas Pekerjaan Produksi** yang disediakan.

Bahasa "makro" dapat digunakan untuk menulis perintah-perintah baru. Ekstensi kemampuan program Python dapat mengakses informasi dalam kamus data dan data serta secara dinamis membangun program sintaksis perintah. Ekstensi ini, diperkenalkan mulai SPSS 14, menggantikan "skrip" SAX Basic yang kurang berfungsi untuk sebagian besar tujuan, meskipun SaxBasic tetap tersedia. Selain itu, ekstensi Python memungkinkan SPSS mengerjakan statistika apa pun dalam paket perangkat lunak gratis R. Mulai versi 14 dan seterusnya, SPSS dapat dijalankan secara eksternal dengan program Python atau VB.NET menggunakan "*plug-in*" yang disediakan. (Mulai versi 20 dan seterusnya, kedua fasilitas skrip ini, serta banyak skrip, disertakan pada media instalasi dan biasanya otomatis ikut diinstal.)

Antarmuka pengguna grafis memiliki dua tampilan yang dapat diubah. "**Tampilan Data**" menunjukkan tampilan spreadsheet dari kasus (baris) dan variabel (kolom). Berbeda dengan spreadsheet, sel data hanya dapat berisi angka atau teks, dan rumus tidak bisa disimpan di sel ini. "**Tampilan Variabel**" menampilkan kamus metadata, di mana setiap baris mewakili variabel dan menunjukkan nama variabel, label variabel, label nilai, lebar cetak, jenis pengukuran, dan berbagai karakteristik lainnya. Sel di kedua tampilan dapat diedit secara manual, menentukan struktur file dan mengizinkan entri data tanpa menggunakan sintaks perintah. Ini mungkin cukup untuk kumpulan data kecil. Kumpulan data yang lebih besar seperti survei statistika lebih sering dibuat dalam perangkat lunak entri data, atau dimasukkan selama wawancara pribadi dengan bantuan komputer, dengan memindai dan menggunakan perangkat lunak pengenalan karakter optik dan pengenalan tanda optik, atau dengan pengambilan langsung dari kuesioner online. Kumpulan data ini kemudian dibaca ke dalam SPSS.

Statistika SPSS dapat membaca dan menulis data dari file teks ASCII (termasuk file hierarki), paket statistika lainnya, *spreadsheet*, dan database. Ia juga dapat membaca dan menulis ke tabel database relasional eksternal melalui ODBC dan SQ.

Output statistik adalah format file berpemilik (file *.spv, mendukung tabel *pivot*) yang, selain penampil dalam paket, pembaca yang berdiri sendiri dapat diunduh. Output kepemilikan dapat diekspor ke teks atau Microsoft Word, PDF, Excel, dan format lainnya. Alternatifnya, output dapat diambil sebagai data (menggunakan perintah OMS), sebagai teks, teks yang dibatasi tab, PDF, XLS,

HTML, XML, kumpulan data SPSS, atau berbagai format gambar grafik (JPEG, PNG, BMP, dan EMF).

Terdapat beberapa varian Statistik SPSS. **SPSS Statistics Gradpacks** adalah versi diskon tinggi yang hanya dijual kepada pelajar. **SPSS Statistics Server** merupakan versi perangkat lunak dengan arsitektur *client/server*. Paket tambahan dapat menyempurnakan perangkat lunak dasar dengan fitur tambahan (contohnya mencakup **sampel kompleks**, yang dapat menyesuaikan sampel berkerumun dan bertingkat, dan **tabel khusus**, yang dapat membuat tabel siap publikasi). Statistika SPSS tersedia dengan lisensi berlangganan tahunan atau bulanan.

Stan

Stan adalah bahasa pemrograman probabilistik untuk inferensi statistika (menentukan model), paling sering model Bayesian. Bahasa Stan digunakan untuk menentukan model statistik (Bayesian) dengan program penting yang menghitung fungsi kepadatan probabilitas log. Selain inferensi Bayesian penuh, Stan menghitung kepadatan log dan Hessian, variasional Bayes, propagasi ekspektasi, dan integrasi perkiraan. Stan mengambil sampel distribusi *posterior* menggunakan *Hamiltonian Monte Carlo* (HMC) – varian dari *Markov Chain Monte Carlo* (MCMC). Untuk model-model kompleks, HMC menawarkan pendekatan yang lebih kuat dan efisien daripada pengambilan sampel Gibbs atau Metropolis-Hastings, sekaligus memberikan diagnostik mendalam untuk menilai konvergensi dan pencampuran. Hal ini mungkin menjelaskan mengapa Stan mendapatkan popularitas dibandingkan sampler Bayesian lainnya (seperti BUGS dan JAGS).

Pemakaian bahasa Stan dapat dilakukan melalui beberapa antarmuka:

- **CmdStan** – layar terminal yang berisi baris-baris perintah,
- **CmdStanR** dan **RStan** – pustaka (paket) untuk software **R**,
- **CmdStanPy** dan **PyStan** – pustaka (paket) untuk bahasa pemrograman **Python**,
- **CmdStan.rb** - pustaka untuk bahasa pemrograman **Ruby**,
- **MATLABStan** – integrasi dengan lingkungan komputasi numerik **MATLAB**,
- **Stan.jl** – integrasi dengan bahasa pemrograman **Julia**,
- **StataStan** – integrasi dengan **Stata**.

Selain itu, antarmuka tingkat yang lebih tinggi dilengkapi dengan paket yang menggunakan Stan sebagai *tulang punggungnya*, terutama dalam bahasa **R**:

- **rstanarm** menyediakan pengganti *drop-in* untuk model *frequentist* yang disediakan oleh **R** dasar dan **lme4** yang menggunakan sintaks rumus **R**;
- **brms** menyediakan beragam model linier dan nonlinier menggunakan sintaks rumus **R**;
- **prophet** menyediakan prosedur otomatis untuk peramalan deret waktu.

Stan dikembangkan oleh tim yang terdiri atas lebih dari 50 anggota, termasuk

Andrew Gelman, Bob Carpenter, Matt Hoffman, dan Daniel Lee. **Stan** ditulis dalam bahasa C++. Nama **Stan** digunakan untuk menghormati Stanislaw Ulam, pelopor metode Monte Carlo. Rilis pertama Stan dilakukan pada 30 Agustus 2012. Stan diedarkan dengan Lisensi BSD Baru (gratis dan sumber terbuka). Versi terbaru Stan adalah 2.34.1, dirilis pada 23 Januari 2024. Software Stan dapat dijalankan pada sistem operasi Windows, MacOS, UNIX, dan Linux.

Stan mengimplementasikan algoritma *Markov chain Monte Carlo* (MCMC) berbasis gradien untuk inferensi Bayesian, stokastik, metode Bayesian variasi berbasis gradien untuk perkiraan inferensi Bayesian, dan pengoptimalan berbasis gradien untuk estimasi kemungkinan maksimum yang diberi sanksi.

- Algoritma MCMC:
 - Hamiltonian Monte Carlo (HMC)
 - Sampler No-U-Turn (NUTS), varian dari mesin MCMC default HMC dan Stan
- Algoritma inferensi variasi:
 - Inferensi Variasi Diferensiasi Otomatis
- Algoritma optimasi:
 - BFGS dengan memori terbatas (algoritme pengoptimalan default Stan)
 - Algoritma Broyden–Fletcher–Goldfarb–Shanno
 - Perkiraan Laplace untuk perkiraan kesalahan standar klasik dan perkiraan posterior Bayesian

Stan menerapkan diferensiasi otomatis mode terbalik untuk menghitung gradien model, yang diperlukan oleh HMC, NUTS, L-BFGS, BFGS, dan inferensi variasional. Diferensiasi otomatis dalam Stan dapat digunakan di luar bahasa pemrograman probabilistik. Stan digunakan dalam berbagai bidang termasuk ilmu sosial, statistik farmasi, riset pasar, dan analisis citra medis.

STATA

Stata (**STATA**) adalah program statistika komersial dengan tujuan umum, yang dikembangkan oleh **StataCorp** untuk manipulasi data, visualisasi, statistik, dan pelaporan otomatis. Stata digunakan oleh para peneliti pada berbagai bidang, termasuk biomedis, ekonomi, epidemiologi, dan sosiologi. Pengembangan Stata dimulai pada tahun 1984, awalnya dilakukan oleh William (Bill) Gould dan kemudian oleh Sean Beckett dari **Computing Resource Center** di California. Pada tahun 1993, perusahaan tersebut pindah ke College Station, TX dan berganti nama menjadi **Stata Corporation**, yang sekarang dikenal sebagai **StataCorp**. Sekarang Stata menjadi milik **StataCorp**, yang sekaligus sebagai pengembang dan pemasarnya.

Perangkat lunak ini awalnya dimaksudkan untuk bersaing dengan program statistika untuk komputer pribadi seperti **SYSTAT** dan **MicroTSP**. Stata ditulis dalam bahasa pemrograman C, awalnya untuk PC yang menjalankan sistem operasi DOS. Versi pertama dirilis pada tahun 1985 dengan 44 perintah. Ter-

dapat 17 rilis utama Stata antara tahun 1985 dan 2021, serta pembaruan kode dan dokumentasi tambahan di antara rilis utama. Pada tahun-tahun awalnya, rangkaian program Stata tambahan terkadang dijual sebagai "*kit*" atau didistribusikan sebagai *disk dukungan*. Dengan dirilisnya Stata 6 pada tahun 1999, pembaruan Stata 6 mulai dikirimkan ke pengguna melalui Web. Sejak itu, versi Stata telah dirilis untuk sistem yang menjalankan varian Unix seperti distribusi Linux, Windows, dan MacOS. Semua file Stata tidak bergantung pada sistem operasi yang digunakan. Rilis besar pada tahun 2003 menyertakan sistem grafis baru dan jendela dialog untuk semua perintah. Sejak itu, versi baru telah dirilis setiap dua tahun sekali. Versi terbarunya adalah **Stata 18.0**, dirilis pada 25 April 2023.

Sejak pembuatannya, Stata selalu menggunakan antarmuka baris perintah yang terintegrasi. Dimulai dengan versi 8.0, Stata telah menyertakan antarmuka pengguna grafis (GUI) berdasarkan kerangka Qt yang menggunakan menu dan jendela dialog untuk memberikan akses ke banyak perintah bawaan. Kumpulan data dapat dilihat atau diedit dalam format *spreadsheet*. Mulai versi 11 dan seterusnya, perintah lain dapat dijalankan saat *browser* atau editor data dibuka. Hingga rilis versi 16, Stata hanya dapat membuka satu *dataset* pada satu waktu.

Stata memungkinkan fleksibilitas dalam menetapkan data sebagai jenis (tipe) data. Perintah Stata `compress` secara otomatis menetapkan ulang data ke tipe data yang menggunakan lebih sedikit memori tanpa kehilangan informasi. Stata menyimpan jenis data bilangan bulat hanya menggunakan satu atau dua *byte*, bukan empat, sedangkan untuk menyimpan jenis data bilangan riil (*floating-point*) digunakan presisi tunggal (4 *byte*), bukan presisi ganda (8 *byte*).

Stata dapat mengimpor data dalam berbagai format, termasuk format data ASCII (seperti format CSV atau bank data dan format *spreadsheet* (termasuk berbagai format Excel). Format file milik Stata telah berubah seiring waktu, meskipun tidak semua rilis Stata menyertakan format kumpulan data baru. Setiap versi Stata dapat membaca semua format kumpulan data lama, dan dapat menulis format kumpulan data sebelumnya saat ini dan yang terbaru, menggunakan perintah `saveold`. Dengan demikian, rilis Stata saat ini selalu dapat membuka kumpulan data yang dibuat dengan versi yang lebih lama, namun versi yang lebih lama tidak dapat membaca kumpulan data dengan format yang lebih baru. Format data Stata selalu berbentuk tabel. Stata mengacu pada kolom data tabel sebagai variabel. Stata dapat membaca dan menulis kumpulan data berformat SAS XPORT secara asli, menggunakan perintah `fdause` dan `fdasave`. Beberapa aplikasi ekonometrik lain, termasuk **gretl**, dapat langsung mengimpor format file Stata.

Stata menyediakan ratusan alat untuk berbagai aplikasi dan metode, termasuk pemodelan Bayesian dan estimasi kemungkinan maksimum. Target pengguna Stata adalah semua sektor – akademisi, industri, dan pemerintah. Stata menyediakan dukungan aktif dan pengembangan serta memiliki beberapa karakteristik

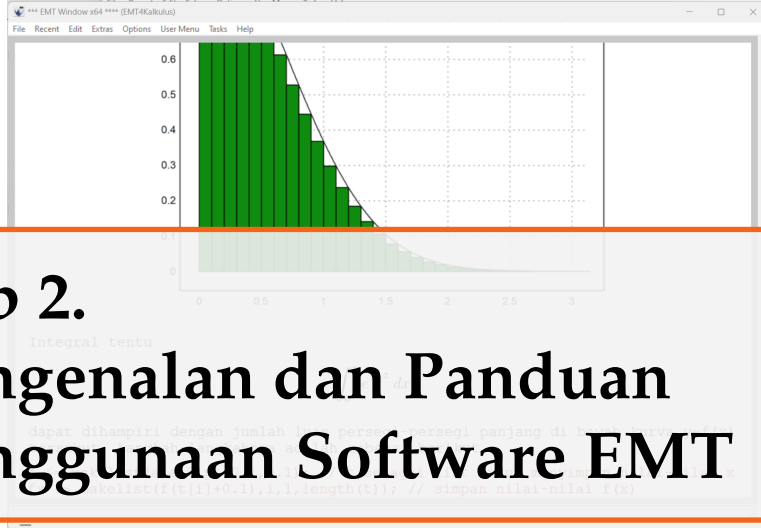
unik, namun kemampuan Big Data Stata masih terbatas.

Ratusan perintah telah ditambahkan ke Stata selama lebih dari 37 tahun sejarahnya. Perkembangan tertentu terbukti sangat penting dan terus membentuk pengalaman pengguna saat ini, termasuk ekstensibilitas, ketidaktergantungan pada sistem operasi, dan komunitas pengguna aktif. Stata memberikan pengguna kemampuan untuk menambahkan perintah mereka sendiri dengan menulis program dengan Stata. Program yang ditulis pengguna dapat dimuat secara otomatis ke dalam memori. StataCorp juga menyediakan perintah `ssc` untuk menambahkan program kontribusi komunitas langsung ke dalam Stata. Stata edisi terbaru memungkinkan pengguna memanggil skrip Python, serta menjalankan perintah Stata melalui IDE Python seperti Jupyter Notebooks. Meskipun aslinya Stata tidak mendukung R, tersedia ekstensi yang ditulis pengguna untuk menggunakan skrip R di Stata.

Lisensi Stata dijual dengan beberapa seri, seperti **Stata/BE**, **Stata/SE**, **Stata/MP**, dan **Numerics by Stata**. Seri SE dan BE berbeda dalam banyak variabel yang dapat digunakan dan kebutuhan memori. Seri Stata/MP, selain mampu menggunakan jumlah variabel dan kebutuhan memori yang lebih besar juga memiliki kecepatan lebih dibandingkan dengan seri BE maupun SE. Seri MP dapat menyimpan hingga 20 miliar observasi dan hingga 120.000 variabel, sedangkan seri Stata/SE dan Stata/BE menyimpan hingga 2,14 miliar observasi dan menangani masing-masing 32.767 variabel dan 2.048 variabel. Jumlah maksimal variabel bebas dalam suatu model adalah 65.532 untuk seri Stata/MP, 10.998 untuk seri Stata/SE, dan 798 untuk seri Stata/BE. **Numerik by Stata** sama dengan seri Stata/MP ditambah kemampuan untuk aplikasi tertanam dan web. Harga lisensi Stata tergantung pada peruntukan (personal, lembaga pendidikan, lembaga pemerintah/non-profit, atau bisnis) dan jangka waktunya. Berikut adalah daftar harga lisensi personal software Stata.

- **Stata/BE**: 48 USD (Lisensi personal untuk pelajar/6 bulan), 94 USD (Lisensi personal untuk pelajar/tahun), 225 USD (Lisensi personal untuk pelajar selamanya).
- **Stata/SE**: 95 USD (Lisensi personal untuk pelajar/6 bulan), 179 USD (Lisensi personal untuk pelajar/tahun), 425 USD (Lisensi personal untuk pelajar selamanya).
- **Stata/MP**: 275 USD (Lisensi personal 2-core untuk pelajar/tahun), 375 USD (Lisensi personal 4-core untuk pelajar/tahun), 595 USD (Lisensi personal untuk pelajar 2-core selamanya), 795 (Lisensi personal untuk pelajar 4-core selamanya).

Untuk lisensi lembaga pendidikan, lembaga pemerintah/non-profit, dan bisnis, harga lisensi software Stata tergantung pada jumlah pengguna dan jangka waktu. Informasi selengkapnya dapat dilihat di laman **Pemesan Software Stata** (<https://www.stata.com/order/dl/>).



Bab 2.

Pengenalan dan Panduan Penggunaan Software EMT

Bab ini menjelaskan software **Euler Math Toolbox** (disingkat **EMT** atau **Euler**) dan panduan cara menggunakannya. Penjelasan tentang EMT mencakup sejarah, tujuan EMT, fitur-fitur EMT, dan cara kerja EMT. Pada bagian pengenalan penggunaan EMT ditunjukkan beberapa kemampuan EMT untuk berbagai perhitungan matematika, baik perhitungan numerik maupun perhitungan simbolik.

2.1 Apakah EMT?

EMT merupakan program matematika canggih yang menggabungkan kemampuan perhitungan numerik dan simbolik (menggunakan sistem aljabar komputer terbuka Maxima) dalam satu paket sumber terbuka dan gratis. Nama software ini diambil dari nama ahli matematika Leonhard Euler (1707-1783) sebagai penghormatan atas prestasinya dalam bidang matematika modern. Idennya mirip dengan MATLAB (menggunakan variabel numerik berbasis matriks), namun sintaks-nya tidak sepenuhnya kompatibel. Software ini mulai dikembangkan pada tahun 1987 oleh Rene Grothmann, seorang profesor matematika di Universitas Katolik Eichstätt-Ingolstadt, Jerman, dan menjadi paket lengkap untuk eksplorasi dan pengajaran matematika selama bertahun-tahun. Pada tahun 2007, Euler menggabungkan sistem aljabar komputer Maxima. Ekspresi simbolik dan fungsi lainnya ditambahkan untuk berkomunikasi dengan Maxima, dan untuk memperkaya kemampuan komputasi numerik Euler. Software ini tersedia secara gratis dan dapat diunduh melalui situs Sourceforge (<https://sourceforge.net/projects/eumat/files/Source/>). Informasi lengkap dan dokumentasi serta panduan penggunaan software ini juga dapat diakses melalui situs resminya (<https://www.euler-math-toolbox.de/>). EMT terus dikembangkan dan diperbarui oleh pengembangnya. Versi terakhir pada saat buku ini ditulis adalah "Version 2024-01-14" dan hanya tersedia untuk sistem operasi Windows 64 bit. EMT dapat dijalankan di Linux dengan menggunakan emulator Windows, misalnya Wine.

Bagi pemula yang ingin mempelajari lebih lanjut tentang program ini, setelah

selesai menginstal EMT, jalankan dan buka menu **Help**, kemudian mulailah mengeksplor. Informasi tentang EMT juga terdapat di Facebook (<https://www.facebook.com/Euler-Math-Toolbox-274936832609749/>) dan Youtube (https://www.youtube.com/channel/UC-dzx4f1XyipZDa94_wOlBg?view_as=subscriber).

Desain antarmuka Euler Math Toolbox sangat sederhana namun dapat memberikan akses yang mudah ke berbagai software lain yang canggih. EMT adalah sistem terintegrasi yang merupakan perpaduan kernel (program inti) numerik Euler dan program komputer aljabar Maxima. EMT sangat cocok bagi Anda yang:

- mencari pengganti yang hebat, mudah digunakan, dan gratis untuk software komersial MATLAB yang sangat mahal;
- perlu mengerjakan perhitungan-perhitungan matematika secara numerik maupun simbolik di sekolah atau universitas;
- ingin menulis matematika yang memuat berbagai perhitungan matematika maupun menyajikan aneka ragam grafik matematis dalam bentuk buku atau laman Web.

Software EMT merupakan perpaduan dari beberapa komponen sebagai berikut.

- Bagian numerik, antarmuka grafis (GUI), dan komunikasi dengan Maxima. Banyak algoritma numerik dan pustaka software *open source* yang digunakan di dalamnya.
- Maxima, program *open source* yang matang dan sangat kaya untuk perhitungan simbolik.
- Software ini dikelola oleh sekelompok pengembang di Internet.
- Beberapa program lain (seperti $\text{\LaTeX}$, Povray, TinyC Compiler, Python) dapat digunakan di Euler untuk memungkinkan perhitungan yang lebih cepat atau grafik dan tampilan hasil yang lebih baik.

Pada bab ini dijelaskan cara kerja dan beberapa kemampuan software EMT dan menggunakannya untuk menyelesaikan berbagai masalah matematika. Cara kerja EMT adalah Anda membuat dokumen yang dikenal dengan berkas notebook di EMT. File notebook EMT memiliki ekstensi `.en`.

Software EMT dilengkapi dengan contoh-contoh file notebook yang dapat dibuka melalui menu `File`, kemudian pilih `Open Tutorials and Examples`. Panduan (dalam bahasa Inggris) dan berbagai contoh aplikasi EMT untuk matematika disediakan dalam file-file notebook contoh tersebut.

2.2 Memasang Software EMT

Euler dilengkapi dengan file instalasi sederhana **EulerSetup.exe**. File ini adalah program instalasi untuk Windows yang telah dibuat menggunakan InnoSetup. EMT dapat berjalan pada semua versi Windows.

Instalasi akan membuat hal-hal berikut di komputer Anda:

- folder instalasi Euler, aslinya adalah `C:\Program Files\Euler x64`,
- folder Euler Files di folder Document pengguna,
- folder Euler di folder `$HOME$` pengguna untuk menyimpan file sementara,
- file `.euler.profile` di folder `$HOME$` pengguna untuk pengaturan,
- ikon di menu Start pengguna,
- entri `uninstall` untuk Windows,
- asosiasi untuk file Euler `*.e` dan `*.en` dalam register Windows.

Pengguna dapat menyimpan file *notebook* dan file lainnya ke folder Euler Files. Euler tidak menulis atau membaca folder ini dengan sendirinya. Untuk file sementara, Euler membuat folder bernama Euler di folder `$HOME$` pengguna (biasanya `C:\Users\NamaPengguna`). Tempat ini digunakan untuk menyimpan file antara Euler (Tekan [F9] atau [F10] di perintah garis kosong untuk melihat file sementara), Maxima (gnuplot), Povray, atau file $\text{\LaTeX}$.

Jika Anda menghapus instalasi Euler, folder instalasi Euler akan dihapus, termasuk asosiasi file Euler. Folder Euler Files Anda tetap utuh, juga folder sementara Euler di folder `$HOME$` pengguna.

Pengaturan EMT disimpan dalam file `.euler.profile` di folder `$HOME$` pengguna. Untuk mereset EMT ke status aslinya cukup hapus file ini.

Instalasi lengkap EMT terdiri atas:

- antarmuka,
- kernel (program inti) numerik EMT,
- bahasa pemrograman,
- file-file Euler dengan utilitas untuk EMT,
- tutorial dan contoh-contoh dalam bentuk *notebook* dan HTML,
- sistem Maxima,
- dokumentasi dalam format HTML,
- file-file dokumentasi untuk bantuan penggunaan program,
- kompailer TinyC,
- antarmuka Python,
- DLL LPSOLVE,
- editor JE untuk file Euler (memerlukan **Java runtime**).

Untuk dapat memanfaatkan semua kemampuan EMT, Anda dapat menginstal software tambahan berikut ini:

- sistem **LaTeX** seperti **MikTeX** atau **TeXLive**,
- software distribusi **Python 2.x**,
- software *raytracer* **Povray**,
- editor kode yang baik (pengganti JE), misalnya Notepad++.

Jika file biner **latex.exe** tidak tersedia di jalur sistem, Anda dapat mengonfigurasi $\text{\LaTeX}$ di EMT. Jika Anda menginstal Python, penerjemah Python-nya akan ditemukan oleh EMT. Hal ini diakses melalui DLL Python di Windows. Lokasi program **Povray** disimpan dalam string jika tidak dalam jalur sistem. Editor eksternal dapat dikonfigurasi di dalam EMT juga.

2.2.1 Maxima

Jika Anda memilih file instalasi yang lengkap `EulerSetup.exe`, Maxima akan diinstal ke lokasi instalasi Euler.

Anda dapat menggunakan versi Maxima Anda sendiri. Karena mungkin terdapat perubahan di Maxima versi terbaru, tidak semua fitur Maxima terbaru dijamin akan berfungsi. Anda dapat menghubungkan Euler ke instalasi Maxima Anda. Selama Maxima tidak mengubah antarmukanya, cara ini akan bekerja seperti Maxima bawaan Euler.

Untuk menghubungkan Euler ke Maxima, buka menu `Options -> Symbolic Setting -> Setup Maxima CallString`. Anda perlu menelusuri file `sbcl.exe` (biasanya di folder `bin`) atau `maxima.exe` (pada versi lama), dan juga mencari lokasi Maxima.

Gnuplot di Maxima menggunakan direktori Euler dalam folder `$HOME$` pengguna.

2.2.2 $\text{\LaTeX}$

Euler dapat menampilkan rumus yang diformat oleh $\text{\LaTeX}$ pada bagian teks komentar maupun pada *output* perintah Maxima. Untuk ini, $\text{\LaTeX}$ harus diinstal. Disarankan Anda menginstal MikTeX atau TeXLive. Euler menggunakan paket AMS. Jadi Anda harus menginstal ini juga.

Anda dapat menemukan dan mengunduh sistem $\text{\TeX}$ (MikTeX maupun TeXLive) versi windows di situs CTAN (*Comprehensive TeX Archive Network*) di <https://ctan.org>. Situs CTAN memiliki banyak server bayangan (*mirror*), sehingga sebaiknya pilih lokasi server CTAN terdekat. Misalnya, untuk daerah Yogyakarta dan Jateng dapat menggunakan situs bayangan CTAN UGM (<https://http://repo.ugm.ac.id/ctan/>).

Anda dapat menggunakan versi 64bit. Setelah instalasi, terdapat dua cara untuk membuat $\text{\LaTeX}$ dikenali Euler. Anda dapat menambahkannya ke jalur sistem Anda, atau Anda dapat memberi tahu Euler di mana menemukannya.

Anda dapat memberitahu Euler di mana untuk menemukan $\text{\LaTeX}$. Telusuri $\text{\LaTeX}$ melalui menu `Options -> Latex Setting -> Setup Latex ....` Anda perlu menelusuri ke lokasi di mana file `latex.exe` dan `dvipng.exe` berada. Lokasi asli ruang kerja adalah folder Euler di folder `$HOME$` pengguna. Anda dapat membiarkan ini kosong. Atau, Anda dapat menambahkan jalur lokasi

L^AT_EX ke jalur sistem (\$path\$).

2.2.3 Python

Anda dapat menulis dan menjalankan perintah dan program Python di Euler. Perintah Python dapat mencapai 10 kali lebih cepat daripada perintah Euler untuk program dasar dengan banyak *loop*. Selain itu, perpustakaan fungsi yang matang dari Python tersedia di Euler dengan antarmuka ini. Perhatikan, bahwa kode C, misalnya dihasilkan dengan *compiler* TinyC yang disertakan, juga dapat mencapai 10 kali lebih cepat daripada Python.

Disarankan agar Anda menginstal Anaconda. Ini adalah sistem Python penuh dengan **MatPlotLib** dan **NumPy**. Instal "untuk semua pengguna", atau salin `python27.dll` ke direktori Windows dengan *priviledges* administrator. Saat ini, hanya Python 2.7 yang didukung. Python 3 akan didukung dalam versi EMT yang lebih baru.

Anda juga dapat menggunakan lingkungan Python standar. Bergantung pada instalasi Euler Anda, Python 2.7 versi 32bit atau 64bit harus sepenuhnya diinstal pada komputer. **MatPlotLib** hanya diperlukan, jika Anda ingin membuat plot melalui Python. Dalam hal ini, Anda juga membutuhkan **NumPy**.

2.2.4 TinyC

Kompiler TinyC diinstal bersamaan (otomatis) dengan EMT. Hal ini dapat digunakan secara otomatis.

2.2.5 Povray

Euler dapat memanggil program *raytracer* Povray untuk menghasilkan output 3D dengan kualitas serupa foto. Anda perlu menginstal Povray dari situsnya (<http://www.povray.org/>). Selama instalasi, perhatikan di mana lokasinya.

Perhatikan dengan seksama di mana Povray dipasang. Anda perlu menghubungkan program Povray ke EMT sebelum Anda dapat menggunakannya.

Terdapat dua cara untuk menghubungkan EMT dengan Povray. Anda dapat menambahkan folder `bin` Povray ke jalur sistem. Atau Anda dapat mengatur program Povray di setiap *notebook* yang menggunakan Povray. Anda perlu melakukan ini dengan menggunakan variabel Euler.

```
defaultpovray =  
"C:\Program Files\POV-Ray\v3.7\bin\pvengine.exe"
```

Povray menyimpan file perantara dan *output* PNG ke folder Euler di folder \$HOME\$ pengguna.

2.2.6 Editor Eksternal

Euler hadir dengan editor eksternal JE, yang dibuat oleh penulis EMT. Ini adalah editor Java. Java harus diinstal pada sistem. Editor eksternal dapat digunakan untuk mengedit file Euler eksternal, atau file Euler sementara.

JE memiliki penyorotan sintaks untuk file Euler (*.e), dan pemeriksa ejaan untuk bahasa Inggris dan Jerman yang hanya berfungsi di dalam komentar/keterangan fungsi atau komentar global.

Anda juga dapat menggunakan editor lainnya, misalnya Notepad++. EMT menyediakan konfigurasi untuk fasilitas menyorot sintaks dan disimpan dalam file `EMT.xml` yang disimpan di sub-direktori instalasi `util`. Anda dapat mengimpornya di Notepad++ sebagai bahasa tambahan yang dikenalnya.

2.2.7 Euler Portabel

Jika Anda ingin menjalankan Euler melalui *flashdisk*, atau tidak ingin menginstal Euler ke komputer, Anda dapat membuat versi yang tidak mengubah komputer.

Untuk menginstal Euler pada stik USB, jangan hanya mengubah folder instalasi. Anda masih akan mendapatkan ikon dan menghapus informasi di komputer yang menunjuk ke data pada *flashdisk*. Sebaliknya, lakukan hal berikut.

- Instal versi lengkap Euler di komputer dan salin folder `Euler` di lokasi program ke stik USB. Ini adalah folder yang berisi file-file untuk menjalankan Euler dan kumpulan file dokumentasi/panduan serta contoh-contoh *notebook*.
- Buat sub-folder `Euler Files` di dalam folder yang disalin pada stik USB tersebut. Jika Euler menemukan sub-folder tersebut, ia tidak akan menggunakan *registry*, atau folder `$HOME$` pengguna. Euler akan membuat file dengan pengaturannya pada USB tersebut.
- Jika Anda ingin menggunakan LPSOLVE atau Python, Anda perlu menyalin DLL yang sesuai (terutama `eulpsolve.dll`, `lpssolve55.dll`, `py27.dll`.) ke dalam USB tersebut. EMT menginstal DLL ke dalam lokasi sistem, biasanya `c:\windows\system32`.
- Selanjutnya, jika mau Anda dapat menghapus program Euler dari komputer.
- Jalankan program Euler dari USB.

2.2.8 Instalasi Jaringan

Untuk menginstal Euler di jaringan lokal, Anda perlu menyediakan tempat untuk menyimpan file-file biner Euler di server, dan tempat lain untuk folder `$HOME$` pengguna dan untuk menyimpan file konfigurasi Euler di folder `$HOME$` pengguna aktif.

- Instal Euler seperti biasa dan salin folder `Euler` ke server. Setelah file

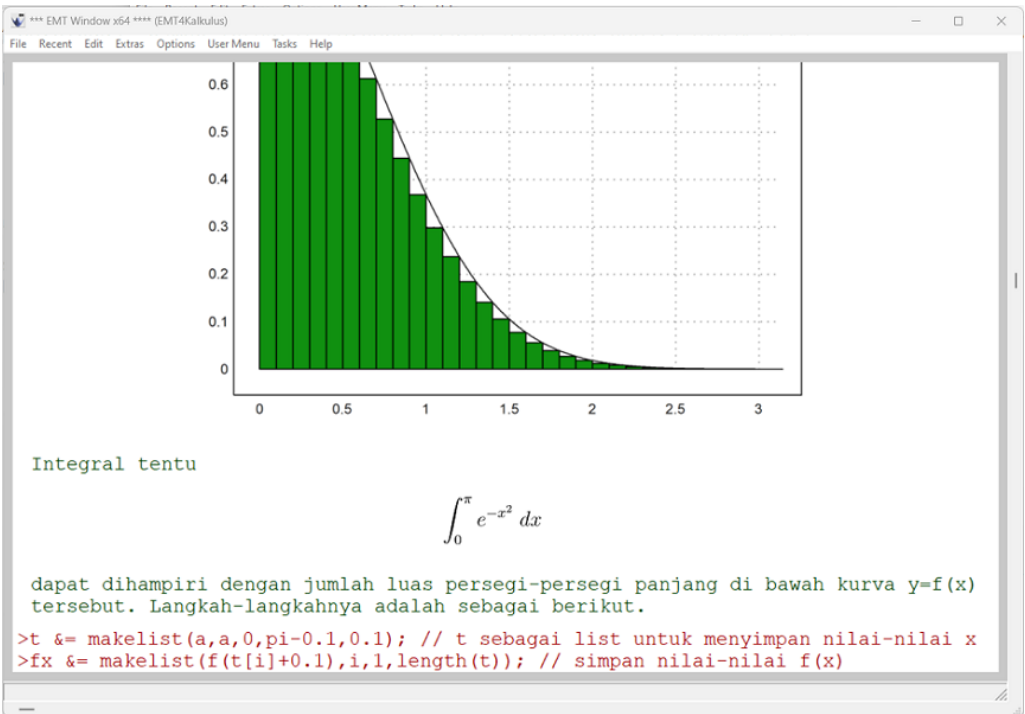
disalin, Anda dapat menghapus instalasi Euler.

- Berikan tautan untuk memulai `euler.exe` pada server dengan parameter `-h homedir`, di mana `homedir` adalah nama lengkap folder pengguna Windows. Euler akan membuat folder Euler Files di lokasi ini, dan menyimpan file `.euler.profile` di sana.

2.3 Komponen dan Fitur EMT

Tabel 2.1: Software inti dan komponen pendukung EMT

Kernel Euler: dengan bahasa pemrograman dan algoritma numerik	Maxima: terintegrasi sebagai sistem simbolik	LaTeX: untuk rumus dan ekspor PDF
Python dan C: untuk skrip yang sangat efisien	Antarmuka EMT: Buku Catatan (<i>Notebook</i>)	Browser: untuk menampilkan panduan, ekspor dan pencetakan HTML
Povray: untuk grafis serealistik foto	PUSTAKA DLL: LPSOLVE untuk optimisasi, DLL pengguna	Editor: editor eksternal, JE untuk mengedit komentar dan file Euler



Antarmuka utama EMT berupa layar editor atau layar notebook (buku catat-

an) yang digunakan untuk menuliskan teks dan perintah-perintah. Teks ditulis sebagai komentar, dapat berupa penjelasan, perintah dan hasil perhitungan menggunakan Maxima, ekspresi matematika menggunakan LaTeX, dan menyisipkan gambar. Baris perintah pada layar notebook EMT dapat diedit setiap saat dan di-Enter untuk mendapatkan hasilnya. Antarmuka EMT juga menyediakan menu-menu sederhana yang mendukung kemampuannya. Komponen utama software EMT terdiri atas:

- Antarmuka untuk bekerja dalam bentuk notebook;
- Kernel Euler yang dilengkapi dengan bahasa pemrograman dan algoritma numerik untuk mendukung komputasi numerik;
- Maxima, untuk mendukung kemampuan komputasi simbolik (eksak),
- Povray, untuk mendukung kemampuan grafik 3 dimensi yang bersifat realistis menyerupai foto;
- Pustaka DLL seperti LPSOLVE untuk mendukung kemampuan optimisasi;
- Python dan C, untuk mendukung skrip dengan efisiensi tinggi;
- LaTeX, untuk menuliskan ekspresi matematika dan ekspor file PDF;
- Browser untuk menampilkan panduan dan petunjuk serta dokumentasi EMT;
- Editor eksternal untuk mengedit komentar (teks).

Beberapa komponen, seperti LaTeX, Python, dan Povray harus diinstal secara terpisah. Maxima secara otomatis akan diinstal bersamaan dengan Euler, namun dapat pula diinstal secara terpisah. Gambar berikut ini menunjukkan tampilan layar notebook EMT dengan baris menu di bagian atas.

2.3.1 Fitur Utama

Berikut adalah daftar fitur utama EMT. Karena EMT adalah sistem yang sangat besar, tidak semua algoritma dan fungsi di dalamnya dapat disebutkan dan dijelaskan pada halaman ini.

- Antarmuka yang sederhana dengan ruang kerja berupa notebook yang terdiri atas baris-baris perintah yang dapat diedit, komentar (teks penjelasan) antar baris perintah, hasil perhitungan EMT dan grafik.
- Fasilitas salin dan tempel (*copy and paste*) juga dapat dilakukan di layar notebook, maupun editor teks, sehingga memudahkan pengetikan.
- File hasil kerja (dengan ekstensi .en) disimpan sebagai file teks dengan format XML, yang dapat diedit dengan sebarang editor teks.
- kemampuan mengekspor hasil kerja ke format HTML (untuk ekspresi matematika dengan LaTeX dapat diekspor ke format MathJax atau grafik SVG) dan berbagai format grafik;
- kernel numerik yang cepat dengan kemampuan perhitungan aritmatika eksak dan interval;
- dapat diprogram menggunakan bahasa interpreter mirip BASIC;
- penggunaan ekspresi dan fungsi numerik maupun simbolik yang pendefi-

- nisiannya mudah, karena semua ditulis pada baris perintah;
- sebagai antarmuka Maxima;
- dilengkapi dengan menu bantuan interaktif dan akses dokumentasi dalam format HTML serta contoh-contoh penggunaan EMT (dalam bentuk notebook) untuk berbagai topik matematika yang dapat diedit;
- sebagai antarmuka untuk berbagai program eksternal (misalnya $\text{\LaTeX}$ untuk menampilkan hasil perhitungan eksak dan menuliskan ekspresi matematika, Povray untuk grafik tiga dimensi, bahasa pemrograman Tiny C dan Python).

2.3.2 EMT Sebagai Bahasa Pemrograman

EMT selain sebagai alat komputasi yang dapat digunakan untuk melakukan perhitungan matematika, baik secara numerik maupun simbolik, juga merupakan bahasa pemrograman tingkat tinggi yang menggunakan dialek bahasa BASIC modern, dengan fitur-fitur:

- sintaks perintah/fungsi sangat sederhana dan interaktif (dapat diedit dan dilihat panduan penggunaannya dengan klik kanan nama perintah/fungsi pada baris perintah),
- mendukung fungsi dengan hasil ganda,
- mendefinisikan fungsi (program) secara inline (pada 1 baris perintah) untuk fungsi-fungsi dengan rumus/hasil sederhana,
- mendefinisikan fungsi (program) multibaris untuk hasil yang tidak dapat dinyatakan dengan satu rumus/ekspresi,
- mendefinisikan fungsi yang memuat parameter dengan nilai pasti apabila tidak ditentukan pada saat memanggil/menggunakannya,
- menggunakan variabel global dan lokal pada fungsi,
- kumpulan program/fungsi dapat disimpan pada file Euler (dengan ekstensi .e) yang dapat dipanggil apabila diperlukan,
- mendukung bahasa C dengan menggunakan kompailer TinyC,
- mendukung bahasa Python.

Berbagai algoritma untuk komputasi numerik dan eksak, mencakup hampir semua fungsi dalam matematika, diimplementasikan di dalam Euler dan Maxima, di antaranya:

- variabel/ekspresi (string) dapat berupa vektor/matriks dengan elemen bilangan riil (desimal) dengan presisi ganda IEEE, bilangan kompleks, interval, dan string (termasuk Unicode);
- mendukung matriks berukuran besar dan matriks jarang (matriks dengan elemen nol dominan);
- operasi aritmetika dasar, fungsi-fungsi matematika, operasi matriks, baik untuk bilangan riil (desimal), bilangan kompleks, interval (aritmetika interval), string, maupun vektor/matriks;
- penyelesaian persamaan (metode Newton-Raphson, metode bagi dua, me-

tode tali busur, metode interval Newton untuk fungsi satu atau multivariabel, metode Broyden, Newton, Nelder-Mead untuk fungsi multivariabel, metode Bauhuber untuk akar polinomial, dll.);

- penyelesaian sistem persamaan linier (metode Jacobi, Gauss-Seidel, dan metode langsung);
- penyelesaian sistem persamaan non-linier (metode Broyden, Nelder, Powell);
- fungsi-fungsi aljabar linier (vektor dan matriks);
- iterasi (perhitungan secara berulang dengan menggunakan rumus sama);
- interpolasi, pencocokan kurva, dan polinomial aproksimasi (polinomial interpolasi, metode selisih terbagi Newton, spline, interpolasi Hermite, kurva Bezier, regresi polinomial);
- perhitungan nilai turunan fungsi (sampai turunan ke 5);
- perhitungan integral fungsi (metode Simpson, Gauss-Legendre, Romberg, metode adaptif, dll.);
- penyelesaian persamaan diferensial/masalah nilai awal (metode Heun, Runge-Kutta, metode LSODA, metode adaptif);
- penyelesaian masalah optimisasi (optimisasi non-linier, pemrograman linier dengan metode simplex dan Newton-Barrier, metode branch and bound, LPSOLVE);
- komputasi statistika (bilangan acak dan fungsi distribusi, statistika deskriptif, grafik/diagram statistika, tabel data, uji statistika, interval kepercayaan, simulasi);
- menggambar grafik dan diagram 2 dimensi (2D) maupun 3 dimensi (3D), termasuk menggunakan **gnuplot**;
- aneka grafik (kurva fungsi, permukaan, aneka grafik/diagram statistika, garis kontur, vektor, gambar geometris, dll.) dan atribut (model garis atau kurva, model arsiran, warna, label sumbu koordinat, teks keterangan dengan $\text{\LaTeX}$, dll.) dapat dihasilkan dan ditampilkan;
- perhitungan matematika simbolik (eksak) dengan bantuan Maxima, yang tampilan hasilnya dapat diformat menggunakan $\text{\LaTeX}$;
- perhitungan simbolik dapat menggunakan variabel simbolik, ekspresi, dan fungsi simbolik, serta perintah-perintah Maxima;
- adanya modus langsung untuk menggunakan perintah-perintah Maxima;
- kemudahan pergantian dan penggunaan hasil simbolik Maxima dengan Euler;
- Maxima dapat dipanggil **pada waktu kompilasi** fungsi Euler;
- banyak algoritma di Euler menggunakan Maxima, misalnya metode Newton;
- String Euler dapat dengan mudah diurai oleh Maxima sebelum digunakan di Euler;
- variabel numerik dapat didefinisikan baik dalam Euler dan Maxima;
- Ekspresi dalam string Euler dapat dengan mudah digunakan dalam perintah Maxima.

2.4 Mulai Bekerja dengan EMT

Dari sudut pandang pengguna, tampilan layar utama Euler terlihat seperti *shell* (jendela) perintah (terminal untuk menuliskan perintah-perintah komputer). Perintah dapat dituliskan dan dijalankan, dan hasil keluarannya akan muncul di bawah baris perintah. Selain itu, setiap perintah dapat memiliki bagian komentar di atas perintah. Inilah yang membedakan layar utama EMT dengan jendela terminal atau bahkan layar program-program aplikasi matematika lain, seperti GNU Octave, Scilab, dan lain-lain.

Terdapat jendela kedua, jendela grafik, untuk keluaran berupa gambar atau grafik. Grafik pada jendela grafik dapat dimasukkan ke dalam layar teks pada posisi kursor aktif. Jendela grafik dapat disembunyikan (`[Ctrl]+[g]`) dan grafik akan selalu ditampilkan di layar teks berdasarkan permintaan.

Semua perintah di layar teks dapat diedit dan jalankan setiap saat, bahkan berkali-kali. Anda juga dapat menambahkan komentar menggunakan editor komentar khusus. Isi layar teks Euler disebut *notebook* (**buku catatan**). Notebook dapat diekspor menjadi file dengan format lain, seperti PDF, HTML, dan lain-lain.

Berikut ini adalah dua contoh perintah Euler dan tampilan hasil keluaran masing-masing. Yang pertama penjumlahan biasa, sedangkan yang kedua perhitungan $e^{-1/2} \sin(3/2 \times \pi)$.

```
>1+1
2
>exp(-0.5)*sin(3/2*pi)
-0.6065306597126
```

Apabila Anda membuat kesalahan (biasanya hasilnya berupa pesan kesalahan), pergi ke baris perintah yang salah dengan mouse, tombol panah, atau tombol `[PgUp]` dan `[PgDn]` dan edit perintah tersebut. Tekan `[Enter]` di mana saja pada baris perintah untuk menjalankan lagi perintah tersebut. Tampilan/keluaran hasil perintah tidak dapat diedit.

Inti EMT adalah sebuah antarmuka berupa *notebook*, yang dapat berisi:

- *input* (perintah) dan *output* atau hasil dari perintah Euler tersebut,
- kode fungsi dan program EMT,
- grafik,
- komentar, yang dapat berisi teks, ekspresi matematika, perhitungan matematika, tautan, dan gambar.

File hasil kerja dengan EMT disimpan dalam format *notebook* EMT dengan ekstensi `.en` (merupakan file teks biasa, dengan format XML, dapat diedit dengan editor teks biasa) dan dapat diekspor ke format dokumen jenis lain seperti HTML atau PDF (menggunakan $\text{\LaTeX}$).

Dari *notebook*, pengguna dapat mengakses kernel numerik EMT dengan bahasa matriks, bahasa pemrograman EMT, dan aljabar simbolik (menggunakan CAS Maxima), serta banyak program dan fitur lain seperti Povray, Python, C, dan Scilab.

Kernel (program inti) numerik EMT dilengkapi dengan:

- bilangan riil presisi ganda IEEE,
- matriks, termasuk matriks jarang,
- bilangan kompleks dan matriks kompleks,
- interval dan matriks interval,
- akumulator panjang,
- string (juga string Unicode),
- bahasa pemrograman yang lengkap,

dan banyak fungsi bawaan untuk semua jenis aplikasi, sebagian besar ditulis dalam bahasa C yang efisien atau dalam bahasa pemrograman EMT. Bahasa matriks diimplementasikan dengan cara yang sangat sistematis dan mudah dipahami.

Kode program EMT disimpan dalam bentuk file Euler (* .e). Ini adalah dialek bahasa BASIC lanjut. File Euler dapat berisi kode dan beberapa fungsi. File ini dapat dipanggil dengan menggunakan perintah `load`. Fungsi dapat tercantum di jendela bantuan menggunakan nama filenya (Cobalah membuka file "**sets.e**").

Antarmuka grafis dapat menghasilkan grafik 2D dan 3D dengan tampilan anti-alias. Grafik dapat disisipkan ke dalam *notebook* atau diekspor ke file PNG, SVG, **Bitmap** atau Windows **Metafile**.

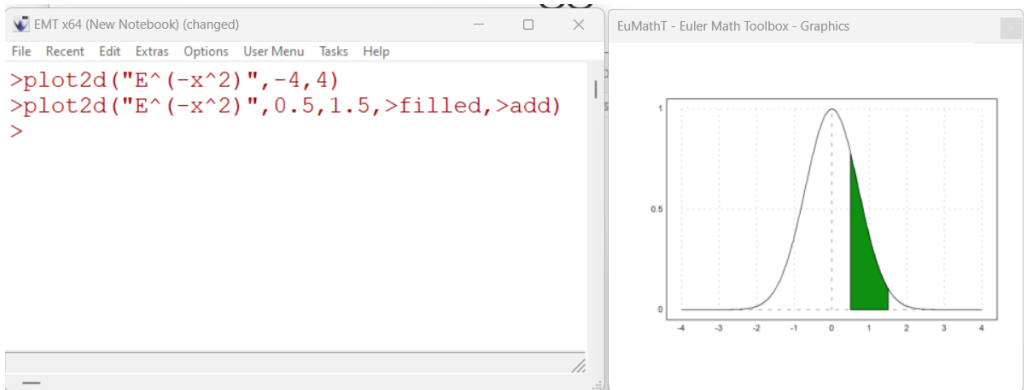
Semua materi di dalam buku ini ditulis dengan software EMT dalam bentuk *notebook* Euler, kemudian diekspor ke format **Markdown** untuk diedit lebih lanjut, selanjutnya diekspor ke format $\text{\LaTeX}$, dan terakhir diproses dengan $\text{\LaTeX}$ menjadi dokumen PDF.

Untuk mulai bekerja dengan EMT, tentu softwarenya sudah harus dipasang (diinstal) di komputer. Selanjutnya, klik tombol/ikon EMT di layar desktop, atau melalui menu Start.

2.4.1 Antarmuka Pengguna

Antarmuka EMT relatif sederhana terdiri atas layar teks (layar *notebook*) dan jendela grafik. Tampilan layar teks EMT terdiri atas baris judul (Logo EMT, nama file yang sedang diedit dan statusnya), baris menu, ruang kerja (jendela/layar *notebook*), dan baris status. Tidak terdapat gambar-gambar ikon perintah layaknya software-software berbasis Windows. Hal ini dikarenakan cara bekerja dengan EMT lebih banyak menggunakan tangan dengan cara mengetik. Jendela grafik tidak memiliki menu, hanya untuk menampilkan gambar/grafik hasil perintah EMT. Kedua jendela dapat diubah ukuran dan posisinya seperti layaknya antar-

muka setiap software Windows. EMT dapat mengingat ukuran dan posisinya tergantung pada ukuran layar monitor. Layar grafik dapat dijadikan satu dengan layar teks (*notebook*) dengan menekan tombol `[Ctrl]+[G]`. Selanjutnya, untuk menampilkan jendela grafik secara terpisah tekan tombol `[Tab]`. Selain itu, grafik dapat disisipkan ke dalam *notebook* secara mudah dengan menggunakan tanda titik dua (`:`) di belakang perintah yang menghasilkan gambar.



Keuntungan antarmuka yang sederhana adalah bahwa pengguna baru tidak perlu belajar banyak untuk mulai menggunakan EMT. Kerugiannya adalah bahwa ia tidak menyediakan ikon grafis sebagai jalan pintas untuk menggunakan perintah-perintah yang tersedia pada menu teks. Pembatasan ini diperlukan karena tidak mungkin untuk menampilkan semua perintah atau opsi Euler dan Maxima ke dalam menu atau ikon. Akan tetapi, terdapat referensi dan bantuan ekstensif di EMT. Tekan `[F1]` guna membuka jendela bantuan untuk memulai.

EMT dapat digunakan untuk bekerja pada beberapa file *notebook* sekaligus, setiap *notebook* diedit pada layar/jendela terpisah. Untuk membuka jendela kedua, ketiga, dan seterusnya, gunakan menu `File` $\rightarrow$ `Open a second windows with EMT`, atau tekan kombinasi tombol `[Ctrl]+[Shift]+[O]`.

Baris menu terdiri atas menu: `File`, `Recent`, `Edit`, `Extras`, `User Menu`, `Tasks`, dan `Help`.

Restart Euler	Ctrl+N
Open ...	Ctrl+O
Open in Euler Files ...	Alt+O
Open Tutorials or Examples	
Open a second window with EMT	Ctrl+Shift+O
Save	Ctrl+S
Save as ...	Shift+Ctrl+S
Save in Euler Files ...	Alt+S
Save and create ZIP	
Load and interpret *.e File	
Save Graphics as PNG ...	
Copy Graphics to Clipboard as Bitmap	
Crop for Graphics Export ...	
Save Graphics as SVG ...	
Copy Graphics to Clipboard as Metafile	
Print Graphics	
Exit	Alt+F4

Menu File berkaitan dengan operasi berkas dan program EMT, terdiri atas submenu:

- `Restrat Euler (Ctrl+N)`: memulai kembali EMT
- `Open ... (Ctrl+O)`: membuka file notebook Euler
- `Open in Euler Files (Alt+O)`: membuka file Euler
- `Open Tutorial or Examples`: membuka file notebook contoh yang sudah disediakan EMT
- `Open a second window with EMT (Ctrl+Shift+O)`: membuka jendela baru EMT (untuk mengedit beberapa file secara bersamaan). EMT menggunakan layar terpisah untuk mengedit setiap file
- `Save (Ctrl+S)`: menyimpan file notebook yang sedang diedit
- `Save as ... (Shift+Ctrl+S)`: menyimpan file notebook yang sedang diedit dengan nama lain (baru)
- `Save in Euler Files ... (Alt+S)`: menyimpan file EMT yang sedang diedit
- `Save and create ZIP`: menyimpan file notebook beserta file-file gambar dan hasil ekspornya dalam bentuk ZIP
- `Load and interpret *.e File`: memuat dan menjalankan file kumpulan program Euler (memiliki ekstensi `.e`)
- `Save Graphics and PNG ...`: Menyimpan file gambar ke format PNG
- `Copy Graphics to Clipboard as Bitmap`: Menyalin file gambar ke memori dalam format bitmap
- `Crop for Graphics Export ...`: Menggunting gambar untuk diekspor
- `Save Graphics as SVG ...`: Menyimpan file gambar dalam format SVG
- `Copy Graphics to Clipboard as Metafile`: Menyalin file gambar ke memori dalam format Metafile
- `Print Graphics`: mencetak gambar
- `Exit (Alt+F4)`: Keluar program EMT.

Menu Recent berisi daftar file-file yang dibuka/diedit sebelumnya.

Menu Edit, berkaitan dengan penyuntingan perintah, terdiri atas sub-submenu sebagai berikut (berserta tombol pintas untuk masing-masing perintah).

Menu Extras, berkaitan dengan penyuntingan teks komentar, mengekspor file *notebook*, dan sebagainya, seperti terlihat pada daftar submenu berikut ini.

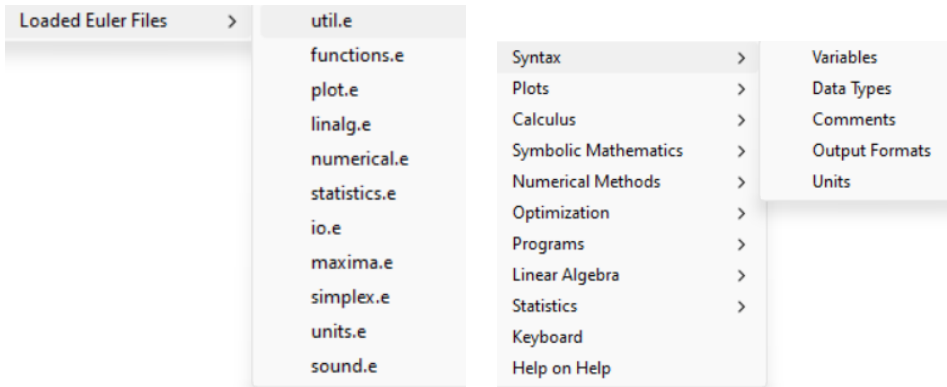
Menu Options, berkaitan dengan pengaturan jendela gambar, rasio gambar, melipat baris-baris perintah, pengaturan program EMT, pengaturan Maxima, sintaks, $\text{\LaTeX}$, dan slide Beamer (*power point*).

	Restore current Line	Ctrl+Z
	Undo Restore	Ctrl+Y
Clear Recent	Run Commands in Section	Shift+Return
00 - First Steps	Update Current Comment	Alt+Return
14 - Exact Computation	Run subsequent Commands and update Comments	Ctrl+R
D4 - EMT For Schools		
Child and Toy Tracks	Copy	Ctrl+C
EMT4aljabar1	Cut	Ctrl+X
SPL	Copy as formatted Output	Ctrl+ Shift+F
akarpersamaan	Copy Commands only	Ctrl+ Shift+C
07 - Complex Numbers	Paste	Ctrl+V
Bab4Visualisasi2D	Paste as Commands	Ctrl+ Shift+V
ArtiPentingMetodeNumerik	Select all	Ctrl-A
EMT4Kalkulus		
EMT4aljabar	Seach for ...	Ctrl+F
Bab2KomputasiAljabar	Search next	F3
EMT4Kalkulus	Output	>
EMT4Plot2D	Command Line	>
EMT4Plot3D	Command History	>
interpolasi		
turunan		
00-PengantarEMT.		
pengantarEMT		

	Show Graphics	TAB
	Graphics in Text Window	Ctrl+ G
	Set Aspect	
	Fold all Multi-Lines	Ctrl+L
Export current Notebook to HTML	Other Graphics Settings	>
Copy HTML Pattern File	Other Program Settings	>
Create HTML Documentation from Euler File	Symbolic Settings	>
Generate Markdown	Syntax Settings	>
Generate PDF with Latex	Latex Settings	>
Delete Notebook and all Files from Disk	Beamer Settings	>
Spell Check	Reset default values for Euler	
Add marked lines to Dictionary		
Languages		
Quick Inserts		

Menu `User Menu`, akan berisi daftar file-file (kumpulan program atau perintah) Euler yang dipanggil dengan perintah `load()`.

Menu `Tasks` berguna untuk menampilkan petunjuk/panduan setiap perintah EMT yang disusun secara sistematis dengan dengan klasifikasi tertentu.



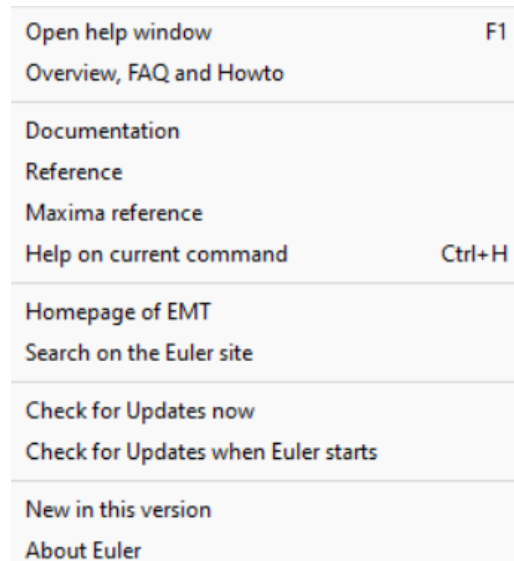
Menu `Help`, berisi sub-submenu untuk menampilkan berbagai dokumen panduan dan rujukan lengkap program EMT dan Maxima.

2.4.2 Jendela *Notebook* (Ruang Kerja EMT)

Ruang kerja pada EMT berupa jendela *notebook*, yang ketika Anda membuka atau menjalankan EMT, jendela tersebut masih kosong hanya berisi tanda `>` berwarna merah dan kursor tegak berwarna hitam. Di tempat itulah pengguna dapat menuliskan perintah-perintah EMT. Anda jangan menganggap remeh penampilan yang sederhana dan mula-mula kosong tersebut. Layar kosong tersebut, selain dapat digunakan untuk menuliskan perintah-perintah EMT pada baris yang bertanda `>`, dapat digunakan untuk menuliskan teks (deskriptif) – bahkan memuat ekspresi matematika yang rumit dengan LaTeX, berisi tampilan hasil perintah (numerik, ekspresi matematika, atau gambar/plot), dan gambar yang disisipkan dari file gambar.

Komentar (Teks Uraian)

Anda dapat menulis teks komentar atau penjelasan di atas setiap baris perintah. Caranya adalah dengan mengklik kanan lokasi di atas baris perintah di mana kursor berada, atau tekan menu `Extras` → `Edit Comment` atau tekan `[F5]`. Cara yang sama juga dilakukan untuk mengedit teks penjelasan tersebut. Teks penjelasan tersebut diketik pada layar/jendela editor teks. Tidak perlu mengatur



tampilan teks. Apa saja yang dituliskan?

Komentar atau teks penjelasan dapat berisi beberapa *markup* dengan sintaks sebagai berikut. Teks paragraf diketik dengan cara biasa, tidak perlu menekan ENTER untuk berganti baris. Untuk berganti paragraf, tambahkan baris kosong. Tanda bintang (*) di awal baris digunakan untuk menulis judul teks. Tanda bintang ganda (**) di awal baris digunakan untuk menulis subjudul. Tulisan `latex:` di awal baris digunakan untuk menulis perintah-perintah L^AT_EX. Sebagian besar perintah L^AT_EX dapat digunakan, bahkan perintah-perintah L^AT_EX yang menggunakan `\begin{}` dan `\end{}`.

```
* Judul
** Sub-Judul
```

Teks paragraf ditulis tanpa menekan tombol ENTER untuk pergantian baris. Pergantian paragraf ditandai dengan baris kosong. Berikutnya adalah paragraf baru.

Berikut adalah cara menampilkan tulisan matematika dengan menggunakan L^AT_EX, MathJax, dan Maxima (perhitungan simbolik).

Butir-butir teks diawali dengan tanda strip (-) dipisahkan oleh spasi dengan teks butirnya.

- Ekspresi matematika dapat diformat menggunakan L^AT_EX, dan semua perintah L^AT_EX dapat digunakan asalkan ditulis dalam satu baris tanpa ENTER.
- Ekspresi matematika juga dapat diformat menggunakan MathJax, namun tidak selengkap L^AT_EX.
- Jika ingin menampilkan ekspresi matematika yang langsung dikerjakan secara eksak (analitik/simbolik), gunakan Maxima.

```
latex: F (x) = \int_a^x f (t) \, dt
mathjax: \frac{x^2-1}{x-1} = x + 1
maxima: '(x^2-1)/(x-1)=ratsimp((x^2-1)/(x-1))
maxima: 'integrate(x^3,x) = integrate(x^3,x) + C
http://www.euler-math-toolbox.de
See: http://www.google.de | Google
image: hati.png
```

Syaratnya, semua perintah L^AT_EX tersebut harus ditulis dalam satu baris (tanpa

menekan ENTER). Tulisan `mathjax`: di awal baris memiliki kegunaan yang sama dengan `latex`, namun tidak semua perintah $\text{\LaTeX}$ dapat digunakan. Selain itu, perintah `latex`: membutuhkan sistem $\text{\LaTeX}$ harus terpasang di komputer, sedangkan perintah `mathjax`: tidak membutuhkan sistem $\text{\LaTeX}$, melainkan menggunakan program JavaScript MathJax. Tulisan `maxima`: di awal baris berfungsi untuk menuliskan perintah-perintah Maxima. Tulisan `See`: digunakan untuk menuliskan teks dengan hiperlink (alamat Internet). Tulisan `image`: di awal baris digunakan untuk menyisipkan gambar di layar *notebook* EMT. Berikut adalah contoh penggunaan sintaks-sintaks tersebut di dalam teks komentar/penjelasan pada *notebook* Euler.

Hasil sintaks-sintaks di atas (diktetik pada jendela editor) adalah sebagai berikut.

Judul

Sub-Judul

Teks paragraf ditulis tanpa menekan tombol ENTER untuk pergantian baris. Pergantian paragraf ditandai dengan baris kosong. Berikutnya adalah paragraf baru.

Berikut adalah cara menampilkan tulisan matematika dengan menggunakan LaTeX, MathJax, dan Maxima (perhitungan simbolik).

Butir-butir teks diawali dengan tanda strip (-) dipisahkan oleh spasi dengan teks butirnya.

- Ekspresi matematika dapat diformat menggunakan LaTeX, dan semua perintah LaTeX dapat digunakan asalkan ditulis dalam satu baris tanpa ENTER.
- Ekspresi matematika juga dapat diformat menggunakan MathJax, namun tidak selengkap LaTeX.
- Jika ingin menampilkan ekspresi matematika yang langsung dikerjakan secara eksak (analitik/symbolik), gunakan Maxima.

$$F(x) = \int_a^x f(t) dt$$

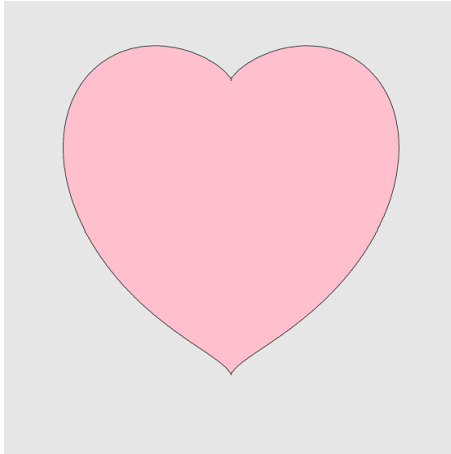
$$\frac{x^2 - 1}{x - 1} = x + 1$$

$$\frac{x^2 - 1}{x - 1} = x + 1$$

$$\int x^3 dx = \frac{x^4}{4} + C$$

<http://www.euler-math-toolbox.de>

Google



Perhatikan, contoh tampilan matematika baris ketiga, meskipun tampilan persis sama dengan baris kedua, namun dihasilkan dengan perintah yang berbeda. Tampilan baris ketiga dihasilkan dengan perintah Maxima dan menunjukkan hasil perhitungan, sedangkan tampilan baris kedua adalah hasil perintah $\text{\LaTeX}$ (diproses oleh MathJax), bukan hasil perhitungan.

Hiperlink dan URL dapat diklik pada file HTML hasil konversi file notebook Euler. Untuk *See :*, tautan (URL) dapat berupa web/file lokal.

Gambar diambil dari folder "images" di tempat file notebook berada dan tidak dapat dibaca dari Web.

Baris Perintah

```
> // baris perintah diawali dengan > ,  
> // komentar (keterangan) diawali dengan //
```

Seperti sudah dijelaskan sebelumnya, ruang kerja EMT berupa *notebook* tempat untuk menuliskan perintah-perintah atau program (fungsi) Euler atau Maxima, menampilkan hasilnya, dan teks penjelasan (komentar).

EMT berorientasi pada baris perintah. Baris-baris perintah Euler diawali dengan tanda `>`. Baris perintah merupakan tempat untuk menuliskan perintah-perintah Euler murni (untuk perhitungan numerik) maupun Maxima (untuk perhitungan simbolik). Perintah-perintah yang dapat dituliskan pada baris perintah dapat

berupa perintah-perintah sederhana maupun program (dinamakan fungsi). Perhatikan, tanda baris perintah dan teks perintah ditampilkan dengan warna merah, untuk memudahkan pengguna melihatnya. Untuk menjalankan suatu perintah, cukup tekan tombol [Enter] pada baris perintah di mana kursor berada.

Hasil keluaran suatu perintah, yang dapat berupa bilangan desimal atau jenis data lain, ekspresi matematika simbolik (dalam bentuk teks ASCII atau format $\text{\LaTeX}$), dan gambar atau plot, ditampilkan di bawah baris perintah.

Anda dapat menuliskan satu atau lebih perintah dalam satu baris perintah. Setiap perintah harus diakhiri dengan koma atau titik koma.

- Titik koma (;) akan menyembunyikan *output* (hasil) dari perintah.
- Sebuah koma (,) mencetak hasilnya.
- Setelah perintah terakhir, koma diasumsikan secara otomatis (boleh tidak ditulis).
- Antar perintah harus dipisahkan dengan spasi.
- Komentar (teks keterangan) dapat dituliskan di akhir baris perintah (diawali dengan tanda //).
- Baris perintah dapat dibagi menjadi multi-baris (dengan menekan [Ctrl]+[Enter]) atau dalam dua baris terpisah ([Shift]+[Ctrl]+[Enter]), dan sebaliknya dua baris perintah dapat digabung menjadi satu baris dengan cara menekan [Ctrl]+[Backspace] di awal baris ke dua. Perintah yang berlanjut ke baris berikutnya akan ditandai dengan ... di akhir baris. Tombol [F9] akan memanggil editor baris perintah dan dapat digunakan untuk mengedit perintah multi-baris atau program/fungsi Euler baru.

Berikut adalah contoh perintah multi-baris.

```
>a=4; b=15; c=2; // menyelesaikan  $a \cdot x^2 + b \cdot x + c = 0$  ...
// secara manual
>D=sqrt(b^2/(a^2*4)-c/a); ...
    -b/(2*a) + D, ...
    -b/(2*a) - D
        -0.138444501319
        -3.61155549868
>plot3d("x^2+y^2",a=0,b=2,c=0,d=2, ...
        niveau=0:0.5:8,zlabel="x^2+y^2",angle=-60°)
```

(Contoh perintah terakhir, hasilnya akan berupa plot permukaan pada jendela plot terpisah.)

Apabila baris pertama perintah multi-baris diawali dengan tanda %+ atau %%, maka otomatis perintah-perintah pada baris kedua dan seterusnya akan tersembunyi ketika kursor meninggalkan baris tersebut.

Tombol [Ctrl]+[L] berguna untuk menyembunyikan atau menampilkan semua perintah multi-baris, kecuali baris di mana kursor sedang berada, mulai baris kedua. Baris perintah multi-baris yang diawali dengan tanda %% akan tersembunyi sepenuhnya.

Untuk membagi baris perintah pada posisi saat ini, gunakan [Ctrl]+[Enter], atau tekan [F9] untuk memanggil editor baris perintah, dan pecahkan baris di sana. Untuk menggabungkan dua baris perintah, tekan [Ctrl]+[Backspace] di awal baris kedua.

```
>%+ "Perintah multi-baris", ...
  "Baris berikutnya", ...
  "Perintah mulai baris kedua tersembunyi jika kursor ...
  pindah baris" ...
Perintah multi-baris ...
Baris berikutnya ...
Perintah mulai baris kedua tersembunyi jika kursor ...
pindah baris
>% x := 1.5; ... // akan tersembunyi jika ditekan ...
tombol [Ctrl]+[L] ...
x := (x+2/x)/2, x := (x+2/x)/2, x := (x+2/x)/2, ...
r:=2; h:=4; pi*r^2*h/3
1.41666666667
1.41421568627
1.41421356237
>x := 1.5; ...
x := (x+2/x)/2, x := (x+2/x)/2, x := (x+2/x)/2, ...
r:=2; h:=4; pi*r^2*h/3
1.41666666667
1.41421568627
1.41421356237
16.7551608191
```

- Anda dapat menyisipkan baris perintah baru (di atas baris perintah saat ini) dengan cara menekan tiga tombol bersamaan: [Shift]+[Ctrl]+[Enter].
- Baris perintah baru otomatis akan muncul setiap kali Anda menjalankan perintah pada baris perintah terakhir.

Gunakan tombol panah (kanan, kiri, atas, bawah) atau *mouse* untuk menggerakkan kursor. Tombol [Ctrl] bersama dengan kursor menggerakkan kursor dari kata ke kata. Lihat menu Edit untuk opsi lainnya. Perintah memiliki sejarah. Untuk mengganti perintah saat ini dengan perintah dalam riwayat, tekan [Ctrl]+[Up]. Tentu saja, Anda juga dapat menyalin ([Ctrl]+[C]), menghapus ([Ctrl]+[X]), dan menempelkan ([Ctrl]+[V]) perintah maupun ko-

mentar. Tombol [Ctrl]+[Backspace] berguna untuk menghapus baris dan [Ctrl]+[U] menyisipkan semua baris yang dihapus sekaligus. [Ctrl]+[Up] akan mengingat perintah sebelumnya ke dalam baris perintah tempat kursor berada. [Enter] menjalankan perintah atau semua perintah pada baris (atau semua perintah dalam satu tanda >), [Shift]+[Enter] menjalankan semua perintah dari awal hingga posisi kursor, dan [Ctrl]+[Enter] menjalankan semua perintah dari posisi kursor hingga akhir.

Kegunaan Beberapa Tombol Papan Ketik

Tabel 2.2 menyajikan ringkasan beberapa tombol papan ketik yang sangat berguna ketika bekerja dengan EMT (mengedit *notebook*).

Tabel 2.2: Tombol-tombol khusus pada EMT

Tombol	Kegunaan
[Esc]	menghentikan perhitungan saat ini, membatalkan pengeditan fungsi
[TAB]	beralih menampilkan grafik
[Ctrl]+[G]	beralih jendela grafis tambahan
Panah atas/bawah	pergi ke baris perintah sebelumnya atau berikutnya
Panah kiri/kanan	pergi ke karakter sebelumnya atau berikutnya
[Ctrl]+panah kiri,	pergi ke spasi sebelumnya atau berikutnya
[Ctrl]+panah kanan	
[Start], [End]	pergi ke awal atau akhir baris
[PageUp],	membuka halaman sebelumnya atau berikutnya
[PageDown]	
[Ctrl]+[PageUp],	membuka judul sebelumnya atau berikutnya
[Ctrl]+[PageDown]	
[F1]	beralih antara jendela bantuan dan notebook
[F3]	cari posisi berikutnya dari pencarian saat ini ([Ctrl]+[F])
[F5]	membuka editor komentar
[F7]	simbol derajat
[F8]	simbol plus/minus
[F9]	membuka editor internal
[F10]	membuka editor eksternal
[F11]	mode layar penuh
[Ctrl]+[F11]	mode pembesaran
[F12]	beralih ejaan

Tabel 2.2: Tombol-tombol khusus pada EMT (... lanjutan)

Tombol	Kegunaan
[Ctrl]+[F12]	menambahkan kesalahan dalam baris yang diblok ke ejaan
[Enter]	menjalankan perintah, atau mengaktifkan definisi fungsi
[Alt]+[Enter]	menjalankan perintah dan perbarui komentar
[Ctrl]+[Enter]	membagi baris perintah, atau masuk ke mode edit fungsi
[Shift]+[Ctrl]+[Enter]	membagi baris perintah tanpa menambahkan "..."
[Shift]+[Enter]	jalankan blok perintah saat ini
[Ctrl]+[Backspace]	menggabung perintah pada baris saat ini ke baris perintah sebelumnya
[Ctrl]+[R]	menjalankan semua perintah berikutnya
[Alt]+[Insert]	menyisipkan perintah atau baris fungsi
[Insert]	menyisipkan saran yang muncul di baris status
[Ctrl]+[Delete]	menghapus perintah pada baris saat ini
[Alt]+[Backspace]	menhapus baris perintah atau fungsi
[Ctrl]+[C]	menyalin teks EMT yang disorot atau gambar yang diblok
[Ctrl]+[V]	menyisipkan teks EMT
[Ctrl]+[X]	menghapus teks EMT yang disorot/diblok
[Ctrl]+[Alt]+[C]	hanya menyalin perintah
[Ctrl]+[Alt]+[V]	tempel teks sebagai perintah
[Ctrl]+[F]	mencari teks (pencarian berikutnya dengan [F3])
[Ctrl]+[N]	mengulang EMT
[Ctrl]+[O]	membuka file <i>notebook</i>
[Ctrl]+[S]	menyimpan <i>notebook</i>
[Shift]+[Ctrl]+[S]	menyimpan <i>notebook</i> ke nama baru dan secara opsional menghapus yang lama
[Ctrl]+[E]	mengekspor <i>notebook</i> ke HTML dan menyimpan <i>notebook</i>
[Ctrl]+[P]	mengekspor <i>notebook</i> menjadi file PDF
[Ctrl]+[L]	melipat setiap perintah multi-garis dan definisi fungsi dalam bentuk multi-baris menjadi satu baris
[Ctrl]+[H]	Membuka dokumentasi di browser (alternatif untuk [F1])

Menampilkan Panduan

Untuk mendapatkan panduan tentang penggunaan perintah atau fungsi di EMT, buka jendela panduan dengan menekan tombol [F1] dan cari fungsinya. Anda juga dapat mengklik dua kali pada fungsi yang tertulis di baris perintah atau di teks untuk membuka jendela panduan.

Misalkan Anda sudah menuliskan perintah seperti di bawah ini. Cobalah klik dua kali pada perintah `intrandom` tersebut!

```
>intrandom(10,6)
      [4,  2,  6,  2,  4,  2,  3,  2,  2,  6]
```

Di jendela panduan, Anda dapat mengklik kata apa saja untuk menemukan referensi atau panduan terkait dengan kata (perintah/fungsi) tersebut.

Misalnya, coba klik kata `random` di jendela panduan. Kata tersebut mungkin berada di dalam teks penjelasan atau di bagian *See:* pada panduan. Anda akan menemukan penjelasan fungsi `random()`, yakni untuk menghasilkan bilangan acak berdistribusi *uniform* antara 0.0 dan 1.0. Dari panduan untuk `random` Anda dapat menampilkan panduan untuk fungsi `normal`, dan sebagainya.

```
>random(4)
      [0.270906,  0.704419,  0.217693,  0.445363]
>normal(5)
      [-0.495418,  1.6463,  -0.390056,  -1.98151,  3.44132]
```

2.5 Mengenal perintah-perintah EMT

2.5.1 Sintaks penulisan perintah Euler

Perintah Euler dapat berupa perintah bawaan (fungsi) Euler atau ekspresi. Suatu ekspresi dapat memuat bilangan, konstanta, variabel, string, operasi, atau perintah/fungsi Euler. Suatu konstanta di dalam EMT adalah nilai tertentu di dalam matematika yang sudah didefinisikan dengan variabel tertentu di dalam EMT. Beberapa konstanta yang sudah tersedia di EMT adalah `pi` (nilai π), `E` (nilai bilangan e). String di EMT adalah tulisan atau teks yang diapit dengan tanda kutip dobel (" "). Apabila string ini berupa ekspresi matematika dalam `x`, maka `x` dapat disubstitusi dengan nilai tertentu dan akan menghasilkan nilai sesuai ekspresi tersebut. Hal ini hanya berlaku untuk string dalam bentuk ekspresi matematika dalam `x`.

```

>"2*x^3"(2)
16
>"s+5"(4)
Variable s not found!
Use global variables or parameters for string evaluation.
Error in expression: s+5
Error in:
"s+5"(4) ...
^

>r=3;
>"Luas = "+(pi*r^2), "Keliling = "+(2*pi*r)
Luas = 28.2743338823
Keliling = 18.8495559215

```

Variabel

Variabel di dalam EMT digunakan untuk menyimpan ekspresi, nilai (baik nilai konstanta, hasil ekspresi, maupun hasil suatu fungsi). Selain itu, variabel juga digunakan sebagai *argumen* atau input suatu fungsi. Variabel di dalam EMT dapat berupa variabel numerik atau variabel simbolik. Variabel numerik ditandai dengan `:` atau `=` untuk menyimpan nilai tertentu.

```

> a = 3; b = 5; // menyimpan nilai ke suatu variabel
> x := (a^5+b^(-2))/sqrt(5); // menyimpan nilai ekspresi
> s := 0:0.1:2*pi; // menyimpan vektor ke suatu variabel
> y := sin(s); // menyimpan nilai fungsi ke variabel
>x
108.690792
>show x
x =
108.690792
>show s
s =
Real 1 x 63 matrix

0.000000  0.100000  0.200000  0.300000  0.400000  ...

```

Nilai variabel dapat ditampilkan dengan 3 cara:

1. setelah definisi variabel, apabila tidak diakhiri dengan `;`
2. dengan menuliskan nama variabel tanda `=`
3. dengan menggunakan perintah `show` diikuti nama variabel.

Cara pertama dan kedua hanya menampilkan nilai variabel, sedangkan cara ketiga akan menampilkan nama variabel diikuti tanda `=` dan nilainya ditampilkan di bawahnya.

Variabel simbolik menggunakan tanda $\&=$ untuk menyimpan nilai eksak atau ekspresi simbolik. Variabel simbolik yang didefinisikan dengan tanda $\&=$ oleh Maxima merupakan suatu ekspresi, sedangkan oleh EMT diberlakukan sebagai string. Apabila ekspresi tersebut merupakan ekspresi matematika, maka nilainya dapat dihitung secara numerik oleh EMT. Artinya, mendefinisikan variabel simbolik (oleh Maxima) $\text{pk } \&=2*x^2-3*x+1$ oleh EMT sama saja dengan $\text{pk} := "2*x^2-3*x+1"$. Seperti sudah dijelaskan di atas, ekspresi ini dapat dihitung nilainya hanya apabila variabel simboliknya adalah x .

```
>pk &=2*x^2-3*x+1; // mendefinisikan variabel simbolik
>pk(u) // salah, ekspresi simbolik tidak dapat ....
// diperlakukan secara numerik
Variable or function u not found.
Error in:
pk(u) ...
>&pk(u) // ekspresi simbolik
                2
            2 x  - 3 x + 1(u)
>pk(2) // menghitung nilai numerik dengan ekspresi ...
// simbolik sebagai string
3
```

Beberapa ketentuan dalam menggunakan variabel adalah sebagai berikut.

- Nama variabel dapat menggunakan karakter dolar (\$), namun sebaiknya jangan di akhir, agar tidak rancu dengan nama satuan.
- Nama variabel dapat memuat karakter spasi khusus yang dihasilkan dengan tombol [Ctrl]+[Sapsi], dan bukan spasi biasa dengan tombol [Spasi].
- Nama variabel juga dapat memuat garis bawah (gunakan garis bawah dobel __).
- Nama variabel juga dapat memuat karakter khusus %, dan variabel % selalu menyimpan hasil perhitungan terakhir yang dilakukan di luar suatu fungsi.

```
>pk = "2*x^2-3*x+1";
>pk(2)
3
>%
3
```

Jika Anda membutuhkan variabel yang akan digunakan oleh EMT (sebagai variabel numerik) sekaligus oleh Maxima (sebagai variabel simbolik), gunakan tanda $\&:=$.

```

> // A sebagai matriks numerik dan simbolik
>A &:= [1,2;3,4]; $A
>A

```

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

```

>det(A) // determinan matriks A (numerik)
-2
>eigenvalues(A) // nilai eigen matriks A (numerik)
[ -0.372281+0i , 5.37228+0i ]
>$eigenvalues(A) // nilai eigen matriks A (simbolik)

```

$$\left[\left[-\frac{\sqrt{33}-5}{2}, \frac{\sqrt{33}+5}{2} \right], [1,1] \right]$$

```

>A.A // hasilnya matriks numerik

```

$$\begin{bmatrix} 7 & 10 \\ 15 & 22 \end{bmatrix}$$

```

>$A.A // hasilnya matriks simbolik

```

$$\begin{pmatrix} 7 & 10 \\ 15 & 22 \end{pmatrix}$$

Variabel juga dapat didefinisikan hanya untuk Maxima dengan &&= (variabel simbolik murni). Perhatikan bahwa tanda &&= tidak akan mengerjakan sisi kanan secara simbolis pada saat didefinisikan. Ekspresi pada ruas kanan hanya akan dikerjakan pada saat variabel yang menyimpannya dipanggil. Hal ini berbeda dengan pendefinisian variabel simbolik menggunakan tanda &=.

```

>Px &&= integrate(x^3,x);
>&Px

```

$$\frac{x^4}{4}$$

Ekspresi

Dalam kernel numerik EMT, ekspresi adalah string yang ditulis di antara tanda kutip double (" "). Ekspresi yang memuat operasi aritmetika, variabel, konstanta, dan/atau fungsi matematika disebut ekspresi matematika. Ekspresi ini dapat menghasilkan nilai numerik (bilangan desimal). Ekspresi simbolik didefinisikan menggunakan tanda &=, &:=, atau &&= dan definisinya tanpa menggunakan tanda kutip double. Ekspresi simbolik dapat ditampilkan melalui Maxima dengan

diawali tanda $\&$, $\$$, atau $\$ \&$. Ekspresi simbolik yang bukan simbolik murni juga dapat ditampilkan oleh EMT tanpa menggunakan tanda $\&$ atau $\$$. Ekspresi dalam bentuk string sering digunakan untuk menggambar kurva dan mendefinisikan fungsi numerik. Dalam hal ini, variabel yang digunakan di dalam ekspresi harus x .

```
>ekspresi := "x^3-x" // ekspresi string
      x^3-x
>plot2d(ekspresi,-1,1); // menggambar kurva y=x^3-x
>eksim &= x^3-x // ekspresi simbolik
      3
      x  - x

>eksim // menampilkan ekspresi simbolik melalui EMT
      3
      x  - x

>&eksim // menampilkan ekspresi simbolik melalui Maxima
      3
      x  - x
```

Jika perlu dapat digunakan tanda kurung untuk menentukan urutan mana yang harus dikerjakan lebih dahulu pada suatu ekspresi. EMT otomatis akan memblok setiap pasangan tanda kurung, sehingga memudahkan penulisan perintah.

```
>2^3^4, (2^3)^4, 2^(3^4)
2.41785163923e+24
4096
2.41785163923e+24
>(cos(pi/4)+1)^3*(sin(pi/4)+1)^2
14.4978445072
```

EMT menyediakan banyak sekali fungsi dan operator matematika. Hampir semua fungsi matematika sudah tersedia di EMT. Anda dapat melihat daftar lengkap fungsi-fungsi matematika di EMT pada berkas Referensi (klik menu Help -> Reference). Di antara operasi-operasi bilangan yang dikenal EMT adalah + (penjumlahan), - (pengurangan), * (perkalian), . (perkalian matriks), ^ atau ** untuk perpangkatan, ! (faktorial), dan masih banyak lagi. Berikut adalah sebagian perintah atau fungsi yang tersedia di EMT.

```
sin, cos, tan, atan, asin, acos, rad, deg
log, exp, log10, sqrt, logbase
bin, logbin, logfac, mod, floor, ceil, round, abs, sign
conj, re, im, arg, conj, real, complex
beta, betai, gamma, complexgamma, ellrf, ellf, ellrd, elle
bitand, bitor, bitxor, bitnot
```

Beberapa perintah memiliki alias atau nama lain yang berguna, misalnya `ln` untuk `log`.

```
>ln(E^2), arctan(tan(0.5))
2
0.5
```

Mari kita tunjukkan cara menggunakan EMT sebagai kalkulator yang sangat canggih. Pada contoh berikut, kita mendefinisikan variabel `r` yang diberi nilai 1,25. *Output* dari definisi ini adalah nilai variabel. Karena perintahnya diakhiri tanda titik koma, nilai ini tidak ditampilkan. Pada kedua perintah di belakangnya, hasil kedua perhitungan tersebut ditampilkan.

```
>r=1.25; pi*r^2, 2*pi*r ...
// luas dan keliling lingkaran dengan jari-jari r
4.90873852123
7.85398163397
```

Perintah `r = 1.25` adalah menyimpan nilai ke variabel di EMT. Cara lain adalah menggunakan sintaks `r := 1.25`. Anda dapat menggunakan spasi sesuka Anda.

```
>r := 1.25 // Menggunakan := sebagai ganti =
1.25
```

Argumen atau input untuk fungsi ditulis di dalam tanda kurung.

```
>sin(45°), cos(pi), log(sqrt(E))
0.707106781187
-1
0.5
```

Seperti yang Anda lihat, fungsi trigonometri bekerja dengan radian, dan derajat dapat diubah dengan `°`. Jika *keyboard* Anda tidak memiliki karakter derajat tekan [F7], atau gunakan fungsi `deg()` untuk mengonversi.

Untuk membuat rangkaian komputasi lebih mudah, Anda dapat merujuk ke hasil sebelumnya dengan `%`. Cara ini sebaiknya hanya digunakan untuk merujuk

hasil perhitungan dalam baris perintah yang sama.

```
>&solve(x^2-3*x+1=0)

      3 - sqrt(5)      sqrt(5) + 3
[x = -----, x = -----]
      2              2

>(sqrt(5)+3)/2, %^2-3*%+1 ...
// Memeriksa solusi x^2-3x+1=0
2.61803398875
0
>x=(sqrt(5)+3)/2; x^2-3*x+1 ...
// Cara lain memeriksa solusi x^2-3x+1=0
0
```

Ekspresi dapat dihitung hasilnya secara numerik. Variabel x , y , z digunakan secara otomatis. Variabel lain digunakan sebagai parameter yang ditentukan nilainya (di dalam definisi suatu ekspresi/fungsi atau ketika memanggil ekspresi/fungsi tersebut) atau melalui variabel global (didefinisikan di luar ekspresi/fungsi).

```
>ekspresi &= a*x^2 ...
// ekspresi simbolik yang memuat parameter a
      2
      a x
>a=4; ekspresi(2) // a sebagai variabel global ...
dengan nilai 4 (= 4*2^2)
16
>ekspresi(2,a=3) // a sebagai parameter yang ...
ditentukan dengan nilai 3 (=3*2^2)
12
>ekspresi(2,3) // nilai 3 tidak berguna, ...
gunakan variabel global (=4*2^2)
16
```

Dalam banyak algoritma EMT lebih baik Anda menggunakan ekspresi dalam x daripada menggunakan fungsi. Akan tetapi, apabila ekspresi memuat parameter yang tidak didefinisikan secara global, lebih baik digunakan fungsi.

```

>a=5; "a*x^3-x"([0,1]) ...
// parameter global, ekspresi dihitung di 2 nilai x
[0, 4]
>function f(x,a) := a*x^3-x ...
// fungsi 1 variabel dengan parameter a
>f([0,1],4) ...
// nilai f(x,4) dengan x=[0,1] hasilnya 2 nilai fungsi
[0, 3]
>"f"([0,1],4) ...
// cara lain menghitung f(x,4) dengan x=[0,1]
[0, 3]

```

Substitusi variabel/parameter pada ekspresi simbolis di Maxima dilakukan dengan kata `with`. Cara ini lebih baik daripada menggunakan `at`.

```

>eksp1 &= exp(-x^2)
          2
          - x
          E
>&eksp1 with x=a
          2
          - a
          E
>&eksp1 with x=5.67
          1.09121450630144e-14
>&eksp1 with x=sqrt(log(y))
          1
          -
          y
>&x^2+y^2 with [x=5,y=4]
          41
>&at(x^2+y^2,[x=5,y=4]) ...
// sintaks asli Maxima menggunakan "at"
          41

```

Penggunaan `with` adalah cara sederhana dalam perhitungan yang menggunakan hasil penyelesaian dengan perintah `solve()`.

```

>&solve(x^2-3*x+1,x), ...
  &(x with %[1])+(x with %[2])|ratsimp // x(1)+x(2)
          3 - sqrt(5)      sqrt(5) + 3
[x = -----, x = -----]
          2                2

3

>&solve(x^2-3*x+1,x), ...
  &(x with %[1])*(x with %[2])|ratsimp // x(1)*x(2)
          3 - sqrt(5)      sqrt(5) + 3
[x = -----, x = -----]
          2                2

1

>&solve([x+y=5,x-2*y=4]); &[x,y] with %[1]

14  1
[--, -]
 3   3

```

Perlu ditegaskan di sini, bahwa suatu masalah akan terjadi apabila terdapat ekspresi dan fungsi dengan nama sama. Untuk menghindari hal ini, sebaiknya gunakan nama variabel seperti `fx`, `fxxy` atau `eksp1`, `eksp2`, dan seterusnya untuk variabel ekspresi, yang tidak digunakan untuk nama fungsi fungsi. Jika terdapat fungsi dan ekspresi dengan nama, maka EMT akan menggunakan ekspresi.

Tipe Data

EMT menyediakan tipe data:

- bilangan riil dan matriks nyata,
- bilangan kompleks dan matriks kompleks,
- interval dan matriks interval,
- string dan vektor string,
- string simbolik,
- matriks terkompresi,
- koleksi (himpunan),
- data biner.

```

>shortformat; // menggunakan format tampilan pendek
>A=[1,2,3;4,5,6] // matriks 2x3
      1      2      3
      4      5      6
>A[1] // baris pertama
      [1, 2, 3]
>A[,2] // kolom kedua
      2
      5
>A[1,2:3] // baris pertama, kolom 2 dan 3
      [2, 3]
>[1,2,3;4] // melengkapi baris kedua dengan 0
      1      2      3
      4      0      0
>[1,2;3,I]; // matriks kompleks
>v=1:10 // vektor baris isinya 1 sampai 10
      [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>v[5:7] // kolom 5 sampai 7
      [5, 6, 7]

```

Vektor numerik adalah matriks yang memiliki satu baris atau satu kolom. Vektor kolom dan vektor baris adalah berbeda. Koleksi adalah kumpulan data lain yang tidak berubah. Terdapat juga daftar yang dapat diperluas melalui `glist()`.

Untuk mendefinisikan vektor dan matriks digunakan tanda kurung siku `[]`. Elemen-elemen dalam satu baris dipisahkan dengan koma `(,)`, baris dipisahkan dengan tanda titik koma `(;)`.

Berikut adalah beberapa contoh tipe data (nilai) di dalam EMT.

```

>-1.234 // bilangan desimal (dengan titik, bukan koma)
-1.234
>1.45e22 // 1.45*10^22
1.45e+22
>[1,2,-5,6] // vektor 1x4
      [1, 2, -5, 6]
>[1,0;0,1] // matriks identitas 2x2 id(2)
      1      0
      0      1
>5+6i // bilangan kompleks 5+6*I
5+6i

```



```

>[I,2*I,3+4*I] // vektor kompleks
[ 0+1i, 0+2i, 3+4i ]
>Pi, E, I // konstanta
3.1416
2.7183
0+1i
>"Ini adalah string"
Ini adalah string
>'Ucapkan "EMT"' // string yang memuat "
Ucapkan "EMT"
>r=5; "Jika r = "+r+", luas lingkaran = "+(pi*r^2)+".
    Jika r = 5, luas lingkaran = 78.53981633972.
>["Satu","Dua","Tiga"]
Satu
Dua
Tiga

```

EMT juga mengenal jenis data interval. Dalam hal ini, interval yang digunakan adalah interval tertutup (termasuk nilai-nilai di kedua ujung interval). Ujung-ujung suatu interval ditandai dengan karakter ~.

```

>~1.5,1.7~ // interval [1.5, 1.7]
~1.5,1.7~
>1±0.1 // interval [1-0.1, 1+0.1], tekan tombol F8 ...
// untuk menghasilkan tanda ±
~0.9,1.1~
>~Pi~ // interval di sekitar pi ([pi-eps, pi+eps])
~3.141592653589792,3.141592653589794~
>~-1,2~*~-1,2~ // semua nilai x*y dengan -1 <= x,y <=2
~-2,4~
>~-1,1~^2 // semua nilai x^2 dengan -1 <= x <= 1
~0,1~
>exp(~-1,1~) // semua nilai exp(x) dengan -1 <= x <= 1
~0.36,2.8~

```

Aturan dasar untuk operasi dan fungsi pada interval adalah:

- Operator dan fungsi menghitung semua hasil yang mungkin untuk semua nilai di dalam setiap interval parameter/masukan, dan menghasilkan interval tertutup yang memuat semua hasilnya.
- Interval yang dihasilkan tidak selalu yang terkecil, tetapi untuk operator biasa, sangat mendekati interval optimal.

```
>~0,2~*~3,4~, ~-1,1~^2, cos(~1,5~)
~0,8~
~0,1~
~-1,0.54~
```

Format Tampilan Nilai

Akurasi internal untuk nilai bilangan di EMT adalah standar IEEE, sekitar 16 digit desimal. Aslinya, EMT tidak mencetak semua digit suatu bilangan. Ini untuk menghemat tempat dan agar terlihat lebih baik. Untuk mengatur tampilan suatu bilangan, operator berikut dapat digunakan.

```
>pi
3.14159265359
>longest pi
3.141592653589793
>long pi
3.14159265359
>short pi
3.1416
>shortest pi
3.1
>fraction pi
312689/99532
>short 1200*1.03^10, long E, longest pi
1612.7
2.71828182846
3.141592653589793
> longest epsilon() // nilai toleransi EMT...
// sebagai batas maksimum nol
2.220446049250313e-12
```

Format aslinya menampilkan nilai menggunakan 12 digit. Format tampilan nilai dapat diatur secara global atau hanya untuk satu nilai. Anda dapat mengganti format tampilan bilangan untuk semua perintah selanjutnya. Untuk mengembalikan ke format aslinya dapat digunakan perintah `defformat` atau `reset`.

```
>longestformat; pi, defformat; pi
3.141592653589793
3.14159265359
```

Kernel (program inti) numerik EMT bekerja dengan bilangan titik mengambang (*floating point*) dalam presisi ganda IEEE (berbeda dengan bagian simbolik EMT). Hasil numerik dapat ditampilkan dalam bentuk pecahan.

```
>1/7+1/4, fraction %
0.392857142857
11/28
```

Jenis data numerik di EMT didasarkan pada standar IEEE dengan akurasi 16 digit desimal. Tampilan asli hasil perhitungan tidak memperlihatkan semua digit signifikan.

```
>1/3 // tampilan asli bilangan presisi tunggal ...
(12 digit signifikan)
0.333333333333
>longest 1/3 // menampilkan semua (16) digit signifikan
0.3333333333333333
>fraction 1/3 // tampilan bilangan dgn format pecahan
1/3
>shortest pi // tampilan bilangan desimal dgn format ...
// singkat/pendek
3.1
>long exp(1+2*I) // bilangan kompleks dengan ...
// format panjang
-1.13120438376+2.471726672i
>sin(pi), longest sin(pi) // pembulatan
0
1.224646799147353e-16
```

Terdapat lebih banyak format untuk *output* dan input. Perintah/fungsi `print(x,k,n,unit,sep,dot)` dapat digunakan untuk memformat tampilan suatu bilangan riil x dengan n digit, termasuk k digit di belakang koma, ditambahkan string `unit`, menggunakan pemisah kelompok digit `sep` dan koma pemisah pecahan `dot`. Perintah-perintah lain termasuk `printf()`, `degprint()`, `prntdate()`, `printheX()`, `prntdual()`.

```
>print(101/891,unit="%")
0.11%
>print(1234567890.23,2,20,>sep,dot=",",unit=" rupiah")
1 234 567 890,23 rupiah
>print(1000*1.03^5,2,15," Juta rupiah") ...
// menggunakan fungsi print
1159.27 Juta rupiah
>printf("pi = %20.18f",pi) // perintah printf C
pi = 3.141592653589793116
```

```

>degprint(21.567°) // menampilkan ukuran sudut ...
// dalam derajat, menit, dan detik
21°34'1.20''
>printdate(date(daynow())) // menampilkan tanggal ...
// dan waktu sekarang
2023-11-17 12:49
>printhex(sqrt(2)) // menampilkan bilangan desimal ...
// dalam format heksadesimal
1.6A09E667F3BCD*16^0
>printhex(1/3) // menampilkan dalam format basis 16
5.5555555555554*16^-1
>printhex(pi)
3.243F6A8885A30*16^0
>prindual(1/3) // menampilkan format bilangan biner
1.01010101010101010101010101010101010101010101*2^-2

```

Perintah/fungsi `format(n,k)` akan menyebabkan semua nilai selanjutnya ditampilkan dengan menggunakan `n` digit (jika sesuai), termasuk `k` digit di belakang koma. Perintah `format(n)` akan menyebabkan semua nilai selanjutnya ditampilkan dengan `n` digit. Untuk mengubah tampilan banyak digit konstanta EMT dapat juga menggunakan perintah `setscalarformat()`.

```

>format(10,6); pi, E^10, 2/7
3.141593
22026.465795
0.285714
>shortestformat; random(3,8)
0.66    0.2    0.89    0.28    0.53    0.31    0.44    0.3
0.28    0.88    0.27    0.7    0.22    0.45    0.31    0.91
0.19    0.46    0.095    0.6    0.43    0.73    0.47    0.32
>setscalarformat(5); pi
3.1416

```

Perintah-perintah lain yang dapat digunakan untuk mengubah tampilan banyak digit nilai-nilai selanjutnya adalah sebagai berikut.

```

shortestformat shortformat longformat, longestformat,
goodformat(n), fracformat(n)

```

Fungsi `printstr()` berguna untuk menampilkan string dengan menggunakan banyak kolom yang ditentukan, ditampilkan rata kiri, rata kanan, atau di tengah, bahkan dengan tambahan karakter/string tertentu.

```

>printstr("EMT",11,0,"-")
----EMT----
>printstr("EMT",11,1,"~")
~~~~~EMT
>printstr("EMT",11,-1,"^")
EMT^^^^^^
>s=""; for i=1:5; s=s+"|"+printstr("Kolom "+i,10,0); ...
end; s+"|",
| Kolom 1 | Kolom 2 | Kolom 3 | Kolom 4 | Kolom 5 |

```

Menampilkan Daftar Variabel

Untuk menampilkan semua variabel yang sudah pernah Anda definisikan sebelumnya (dan dapat dilihat kembali nilainya), gunakan perintah `listvar`.

```

>listvar // hasilnya tergantung variabel-variabel ...
// yang pernah digunakan
r                1.25
a                4
b                15
c                2
D                1.73655549868123

```

Perintah `listvar` hanya menampilkan variabel buatan pengguna. Dimungkinkan untuk menampilkan variabel lain, dengan menambahkan string termuat di dalam nama variabel yang diinginkan.

Perlu Anda perhatikan, bahwa EMT membedakan huruf besar dan huruf kecil. Jadi variabel `d` berbeda dengan variabel `D`.

Contoh berikut ini menampilkan semua unit yang diakhiri dengan `m` dengan mencari semua variabel yang berisi `m$`.

```
>listvar m$
```

```

km$              1000
cm$              0.01
mm$              0.001
nm$              1853.24496
gram$            0.001
m$               1
hquantum$        6.62606957e-34
atm$             101325

```

Untuk menghapus variabel tanpa harus memulai ulang EMT gunakan perintah `remvalue`.

```
>remvalue a,b,c,D
>D
Variable D not found!
Error in:
D ...
^
```

Matriks dan Vektor

EMT merupakan suatu aplikasi matematika yang mengerti "bahasa matriks". Artinya, EMT menggunakan vektor dan matriks untuk perhitungan-perhitungan tingkat lanjut. Seperti sudah dijelaskan sebelumnya, suatu vektor atau matriks dapat didefinisikan dengan tanda kurung siku. Elemen-elemennya dituliskan di dalam tanda kurung siku, antar elemen dalam satu baris dipisahkan oleh koma (,), antar baris dipisahkan oleh titik koma (;).

Vektor dan matriks dapat diberi nama seperti variabel biasa. Selain secara manual, matriks dan vektor dapat didefinisikan dengan menggunakan fungsi-fungsi penghasil matriks yang tersedia di EMT.

```
>v=[4,5,6,3,2,1]
      [4, 5, 6, 3, 2, 1]
>A=[1,2,3;4,5,6;7,8,9]
      1          2          3
      4          5          6
      7          8          9
```

Karena EMT mengerti bahasa matriks, EMT memiliki kemampuan yang sangat canggih untuk melakukan perhitungan matematis untuk masalah-masalah aljabar linier, statistika, dan optimisasi.

Vektor juga dapat didefinisikan dengan menggunakan tanda titik dua (:) untuk menentukan rentang nilai elemen-elemennya dan jarak antar elemen berurutan, seperti contoh berikut ini. Operasi, termasuk penjumlahan (+), pengurangan (-), perkalian (*), pembagian (/), dan perpangkatan (^ atau **), dan fungsi-fungsi matriks di EMT aslinya berbasis operasi elemen. Artinya, operasi dilakukan terhadap masing-masing elemen suatu matriks. Untuk operasi perkalian matriks EMT menggunakan tanda titik (.).

```

>c=1:5
      [1,  2,  3,  4,  5]
>w=0:0.2:1
      [0,  0.2,  0.4,  0.6,  0.8,  1]
>mean(w^2)
      0.35
>A = [1,2;3,4]
           1           2
           3           4

>v = [5,6] // vektor baris 1x2
      [5,  6]
>w = [5;6] // vektor kolom 2x1
           5
           6

>v.A // hasil kali matriks A dan v ...
      // (A.v hasilnya error!)
      [23,  34]
>A.v' // v' adalah transpos v
           17
           39

>A.w
           17
           39

>w'.w // hasil kali skalar suatu matriks
      61
>sqrt(A) // akar kuadrat elemen-elemen matriks A
           1           1.41421
      1.73205           2
>A*A // kuadrat elemen-elemen matriks A
           1           4
           9           16

```

Matriks juga dapat dibentuk dari matriks lain atau vektor.

```

>A=[1,2;3,4]; // matriks 2x2
>b=[1;2]; // vektor kolom
>A|b // penggabungan matriks dan vektor kolom ...
      // secara horisontal
           1           2           1
           3           4           2

```

```

>A_b' // penggabungan secara vertikal
      1      2
      3      4
      1      2
>[b,b^2,b^3] // menggabungkan 3 vektor kolom ...
// menjadi matriks baru
      1      1      1
      2      4      8
>[b';b'^2] // menggabungkan 2 vektor baris menjadi ...
// matriks baru
      1      2
      1      4
>A=[1,2,3;4,5,6;7,8,9] // definisi matriks A
      1      2      3
      4      5      6
      7      8      9
>A[1:2,1:2]=[3,4;5,6] // mengganti elemen-elemen ...
// pada baris dan kolom 1 -2
      3      4      3
      5      6      6
      7      8      9
>A[1]=A[1]+A[2] //baris pertama diganti menjadi ...
// jumlah baris pertama dan kedua
      8      10     9
      5      6      6
      7      8      9
>A[2]=-1 // mengganti semua elemen baris kedua dengan -1
      8      10     9
     -1     -1     -1
      7      8      9

```

Untuk merujuk elemen-elemen pada baris/kolom tertentu suatu matriks digunakan tanda kurung siku. Sudah tentu untuk vektor hanya butuh satu indeks. Untuk matriks, satu indeks merujuk pada semua elemen pada nomor baris yang disebutkan. Suatu matriks dapat disisipkan (untuk mengganti atau mendefinisikan elemen-elemen) pada sub-matriks pada matriks lain (apabila sesuai). Nilai elemen-elemen suatu matriks dapat diganti dengan nilai-nilai yang baru.

Berikut ini adalah contoh beberapa perintah/fungsi penghasil matriks di EMT.


```

>setdiag(setdiag(2*id(4),-1,1),1,1) ...
// matriks tri-diagonal
      2          1          0          0
      1          2          1          0
      0          1          2          1
      0          0          1          2
>diagmatrix(1:4) // matriks diagonal
      1          0          0          0
      0          2          0          0
      0          0          3          0
      0          0          0          4
>A=normal(3,3); A=A/getdiag(A) ...
// menormalkan diagonal menjadi 1
           1      -7.37532      1.1013
      2.80451           1      1.56574
      0.461699    -0.0330072           1

```

Berikut ini adalah contoh penggunaan indeks/posisi elemen-elemen bukan nol suatu vektor/matriks. Contoh yang pertama untuk mencari bilangan-bilangan prima pada rentang tertentu (1 sampai 20). Contoh kedua untuk mencari elemen-elemen suatu matriks yang memenuhi kriteria tertentu.

```

>nonzeros(isprime(1:20))
      [2,  3,  5,  7, 11, 13, 17, 19]
>A=random(3,3); k=mnonzeros(A>0.5)
      0.655416      0.200995      0.893622
      0.281887           0.525      0.314127
      0.444616      0.299474      0.28269
           1           1
           1           3
           2           2

```

Itu artinya, elemen-elemen A yang nilainya lebih dari 0.5 adalah A[1,1], A[1,3], dan A[2,2], seperti tampilan matriks A.

```

>mset(A,k,0) ....
// mengganti elemen-elemen pada posisi k dengan nol
           0      0.200995           0
      0.281887           0      0.314127
      0.444616      0.299474      0.28269

```

Bilangan Kompleks

EMT juga dapat menggunakan bilangan kompleks. Tersedia banyak fungsi untuk bilangan kompleks di EMT. Bilangan imajiner $i = \sqrt{-1}$ dituliskan dengan huruf $\mathbb{I}$ (huruf besar I), namun akan ditampilkan dengan huruf $\mathfrak{i}$ (i kecil). Terdapat beberapa fungsi terkait bilangan kompleks adalah sebagai berikut.

`re(x)`: bagian riil pada bilangan kompleks x .

`im(x)`: bagian imajiner pada bilangan kompleks x .

`complex(x)`: mengubah bilangan riil x menjadi bilangan kompleks.

`conj(x)`: konjugat untuk bilangan bilangan kompleks x .

`arg(x)`: argumen (sudut dalam radian) bilangan kompleks x .

`real(x)`: mengubah x menjadi bilangan riil.

Apabila bagian imajiner x terlalu besar, hasilnya akan menampilkan pesan kesalahan. Euler menggunakan nilai-nilai bagian riil dan imajiner untuk menghitung logaritma dan akar kuadrat bilangan kompleks, sehingga fungsi-fungsi ini analitik pada potongan bidang sepanjang sumbu negatif. Hasil beberapa fungsi aslinya dinyatakan dalam bilangan kompleks. Nilai ini dapat dijadikan bilangan riil apabila memang benar-benar bilangan nyata, artinya bagian imajinernya dalam batas akurasi numerik `epsilon()`. Apabila tidak, maka akan muncul pesan kesalahan.

```
>I^2, 1+3i
-1+0i
1+3i
>sqrt(-1+0.01i), sqrt(-1-0.01i),
0.00499994+1.00001i
0.00499994-1.00001i
>sqrt(-1) // menghasilkan pesan kesalahan
Floating point error!
Error in sqrt
Error in:
sqrt(-1) ...
^
>sqrt(complex(-1))
0+1i
>z=2+3*I, re(z), im(z), conj(z), arg(z), ...
deg(arg(z)), deg(arctan(3/2))
```

```

2+3i
2
3
2-3i
0.982793723247
56.309932474
56.309932474
>deg(arg(I)) // 90°
90
>sqrt(complex(-1))
0+1i

```

EMT selalu menganggap semua hasil perhitungan berupa bilangan riil dan tidak akan secara otomatis mengubahnya ke bilangan kompleks.

Jadi, $\sqrt{-1}$ akan menghasilkan kesalahan, tetapi akar kuadrat kompleks didefinisikan untuk bidang koordinat dengan cara seperti biasa. Untuk mengubah bilangan riil menjadi kompleks, Anda dapat menambahkan $0i$ atau menggunakan fungsi `complex()`.

```

>complex(-1), sqrt(%)
-1+0i
0+1i

```

Satuan

EMT menyediakan semua satuan dan dapat mengubah satuan menjadi sistem standar internasional (SI). Tambahkan nama satuan di belakang nilai (bilangan) untuk mengonversi satuan tersebut ke satuan SI.

```

>1miles // 1 mil = 1609,344 m
1609.344
>5inch // 5 inch = 0.127 m
0.127
>rEarth$, cm$, // jari-jari bumi (km), 1 cm = 0.01 m
6356752
0.01

```

Beberapa satuan yang sudah dikenal di dalam EMT adalah sebagai berikut. Semua nama satuan diakhiri dengan tanda dolar (\$) untuk membedakan dengan nama variabel, namun boleh tidak ditulis jika nama satuan ditulis di belakang suatu nilai (bilangan) atau dengan menggunakan perintah `easyunits` sebelum menuliskan nama-nama satuan.

```

kilometer$:=1000;
km$:=kilometer$;
cm$:=0.01;
mm$:=0.001;
minute$:=60;
min$:=minute$;
minutes$:=minute$;
hour$:=60*minute$;
h$:=hour$;
hours$:=hour$;
day$:=24*hour$;
days$:=day$;
d$:=day$;
year$:=365.2425*day$;
years$:=year$;
y$:=year$;
inch$:=0.0254;

in$:=inch$;
feet$:=12*inch$;
foot$:=feet$;
ft$:=feet$;
yard$:=3*feet$;
yards$:=yard$;
yd$:=yard$;
mile$:=1760*yard$;
miles$:=mile$;
kg$:=1;
sec$:=1;
ha$:=10000;
Ar$:=100;
Tagwerk$:=3408;
Acre$:=4046.8564224;
pt$:=0.376mm;

```

Untuk konversi ke dan antar unit, EMT menggunakan operator khusus, yakni `->`. Apabila nama satuan ditulis sebagai string (dengan tanda kutip double `" "`), maka nama satuan tersebut akan ditampilkan.

```

>4km -> miles, 4inch -> "mm" ...
// mengubah kilometer ke mil, inchi ke milimeter
2.48548476895
101.6 mm
>5km/hour->miles/day, 5kJ->" cal" ...
// mengubah satuan kecepatan dan energi
74.5645430685
5000 cal
>kts=0; 190kts$ -> " km/h" ...
// mengubah satuan knots ke kilometer per jam
0.391240602667 km/h

```

Perintah `print()` dapat digunakan untuk menampilkan suatu nilai dengan satuan tertentu.

```

>print (FahrenheitToCelsius (100), 2, unit="°F")
37.78°F

```

EMT juga dapat digunakan untuk mendefinisikan satuan baru.

```
>impgallon$=4.54609liter, 2.5impgallon->" l"
0.00454609
11.365225 l
```

Euler dapat mengonversi satuan, tetapi tidak memeriksa apakah hasil konversinya masuk akal (benar), karena konversi ini hanya mengubah nilai. Satuan disimpan dalam variabel global, yang diakhiri dengan tanda dolar (\$). Variabel-variabel ini terlihat secara global dalam fungsi.

EMT juga menyediakan beberapa fungsi untuk mengubah satuan, seperti berikut ini.

`FahrenheitToCelsius (x)`: mengubah satuan suhu Fahrenheit ke Celsius dengan rumus $(x - 32) * 5/9$.

`CelsiusToFahrenheit (x)`: mengubah satuan suhu Celsius ke Fahrenheit dengan rumus $x * 9/5 + 32$.

`KelvinToCelsius (x)`: mengubah satuan suhu Kelvin ke Celsius dengan rumus $x - 273.15$.

`CelsiusToKelvin (x)`: mengubah satuan suhu Kelvin ke Celsius dengan rumus $x + 273.15$.

`rearth (lat:number, r1=6378137, r2=6356752)`: jari-jari bumi menurut WSG84 pada posisi garis bujur tertentu.

```
>FahrenheitToCelsius(100) ...
// mengubah satuan suhu Fahrenheit ke Celsius
37.777777778
>3.56yards ...
// mengubah satuan yard ke meter
3.255264
>5.689km/h ...
// mengubah satuan kecepatan km/jam ke meter/detik
1.5802777778
>2km/h -> miles/min ...
// nama satuan hasil konversi tidak ditulis
0.0207123730746
>4.56km/h -> " in/sec" ...
// nama satuan hasil konversi ditulis
49.8687664042 in/sec
```

2.5.2 Matematika Simbolik

EMT dapat melakukan perhitungan matematika simbolis (eksak) dengan bantuan software Maxima. Para pengguna Maxima harus memperhatikan bahwa terdapat sedikit perbedaan antara sintaks asli Maxima dan sintaks ekspresi simbolik di

EMT.

Untuk melakukan perhitungan matematika simbolis di EMT, awali perintah Maxima dengan tanda &. Setiap ekspresi yang dimulai dengan & adalah ekspresi simbolis dan dikerjakan oleh Maxima.

Kita akan bandingkan tampilan hasil perhitungan nilai

$$\left(\frac{\frac{1}{7} + \frac{1}{8} + 2}{\frac{1}{3} + \frac{1}{2}} \right)^2 \pi$$

secara numerik dan secara eksak.

```
>((1/7 + 1/8 + 2) / (1/3 + 1/2))^2 * pi ...
// hasil perhitungan secara numerik oleh Euler
23.2671801626
>&((1/7 + 1/8 + 2) / (1/3 + 1/2))^2 * pi ...
// hasil perhitungan eksak dengan Maxima
145161 pi
-----
19600
```

Berikut adalah contoh menggunakan Maxima untuk menyelesaikan suatu persamaan secara eksak.

$$x^2 + x + 1 = 2x + 3.$$

```
>&solve(x^2+x+1=2*x+3,x)
[x = 2, x = - 1]
```

Berikut adalah contoh menyelesaikan sistem persamaan secara eksak.

$$x^2 - x - y^2 = 1, \quad x + y = 6.$$

```
>&solve([x^2-x-y^2=1,x+y=6],[x,y])
37      29
[[x = --, y = --]]
11      11
```

Sistem persamaan linier dalam bentuk umum juga dapat diselesaikan.

$$a_1x + b_1y = c_1$$

$$a_2x + b_2y = c_2$$

```
>&solve([a1*x+b1*y=c1,a2*x+b2*y=c2],[x,y])
      b1 c2 - b2 c1      a2 c1 - a1 c2
[[x = -----, y = -----]]
      a2 b1 - a1 b2      a2 b1 - a1 b2
```

Berikut adalah contoh mengekspansi dan memfaktorkan bentuk aljabar.

```
>&(a+b)^2
      2
      (b + a)
>&expand((a+b)^2), &factor(x^2+5*x+6)
      2      2
      b + 2 a b + a
      (x + 2) (x + 3)
>&solve(a*x^2+b*x+c,x) // rumus abc
      2      2
      (-sqrt(b-4ac))-b      sqrt(b-4ac)-b
[x = -----, x = -----]
      2a      2a
>&(a^2-b^2)/(a+b), &ratsimp(%) ...
// ratsimp menyederhanakan bentuk pecahan
      2      2
      a - b
      -----
      b + a
      a - b
>10! // nilai faktorial (modus EMT)
3628800
>&10! //nilai faktorial (simbolik dengan Maxima)
3628800
```

Untuk menggunakan perintah Maxima secara langsung (seperti perintah pada layar Maxima) awali perintahnya dengan tanda :: pada baris perintah EMT. Sintaks Maxima disesuaikan dengan sintaks EMT (disebut "modus kompatibilitas").

```

>factor(1000) // mencari semua faktor 1000 (EMT)
      [2,  2,  2,  5,  5,  5]
>&factor(1000) // tidak dikerjakan
      1000
>:: factor(1000) // faktorisasi prima 1000 ...
      (modus kompatibilitas Maxima)
           3  3
           2  5
>:: factor(20!)
           18  8  4  2
           2   3  5  7 11 13 17 19

```

Jika Anda sudah mahir menggunakan Maxima, Anda dapat menggunakan sintaks asli perintah Maxima dengan menggunakan tanda `:::` untuk mengawali setiap perintah Maxima di EMT. Perhatikan, harus ada spasi antara `:::` dan perintahnya.

```

>::: binomial(5,2); // nilai C(5,2)
      10
>::: binomial(m,4); // C(m,4)=m!/(4!(m-4)!)
      (m - 3) (m - 2) (m - 1) m
      -----
              24
>::: trigexpand(cos(x+y)); ...
      // rumus cos(x+y)=cos(x) cos(y)-sin(x)sin(y)
              cos(x) cos(y) - sin(x) sin(y)
>::: trigexpand(sin(x+y));
              cos(x) sin(y) + sin(x) cos(y)
>::: trigsimp(((1-sin(x)^2)*cos(x))/cos(x)^2+ ...
      tan(x)*sec(x)^2)
              4
      sin(x) + cos (x)
      -----
              3
      cos (x)

```

Untuk menyimpan ekspresi simbolik ke dalam suatu variabel digunakan tanda `&=`.


```

>p1 &= (x^3+1)/(x+1)
          3
          x  + 1
          -----
          x + 1

>&ratsimp(p1)
          2
          x  - x + 1

```

Untuk mensubstitusikan suatu nilai ke dalam variabel dapat digunakan perintah `with`.

```

>&p1 with x=3 // (3^3+1)/(3+1)
                                                    7

>&p1 with x=a+b, &ratsimp(%) ...
//substitusi dengan variabel baru
          3
          (b + a)  + 1
          -----
          b + a + 1

          2                2
          b  + (2 a - 1) b + a  - a + 1
>&diff(p1,x) //turunan p1 terhadap x
          2      3
          3 x    x  + 1
          ----- - -----
          x + 1      2
                    (x + 1)
>&integrate(p1,x) // integral p1 terhadap x
          3      2
          2 x  - 3 x  + 6 x
          -----
                    6

```

Tampilan Matematika Simbolik dengan $\text{\LaTeX}$

Anda dapat menampilkan hasil perhitunagn simbolik secara lebih bagus menggunakan $\text{\LaTeX}$. Untuk melakukan hal ini, tambahkan tanda dolar (\$) di depan tanda & pada setiap perintah Maxima. Anda juga dapat hanya menggunakan tanda dolar (\$) saja di depan perintah Maxima untuk menampilkan hasil perhitungan eksak dengan $\text{\LaTeX}$.

Perhatikan, hal ini hanya dapat menghasilkan tampilan yang diinginkan apabila

komputer Anda sudah terpasang software $\text{\LaTeX}$.

Maxima dapat menghasilkan perhitungan yang nilainya sangat besar. Perintah Maxima `showev` dapat digunakan untuk menampilkan perhitungan matematika simbolik secara lengkap (tidak hanya hasilnya saja, namun juga soalnya).

```
>&44!
2658271574788448768043625811014615890319638528000000000
```

Berikut adalah contoh perhitungan koefisien binomial

$$\binom{44}{10} = \frac{44!}{10!34!}.$$

```
>& 44! / (34! * 10!)
2481256778
>$binomial(44,10)

2481256778
```

```
>$binomial(m, 5)


$$\frac{(m-4)(m-3)(m-2)(m-1)m}{120}$$

```

```
>$binomial(m, 10) with m=44

2481256778
```

```
>sol &= solve(x^2+x-4=0,x); $sol ...
// penyelesaian x^2+x-4=0
```

$$\left[x = \frac{-\sqrt{17}-1}{2}, x = \frac{\sqrt{17}-1}{2} \right]$$

```
>$x^2+x-4 with sol[1] // substitusikan x1 ke ruas kiri
```

$$\frac{(\sqrt{17}+1)^2}{4} - \frac{\sqrt{17}+1}{2} - 4$$

```
>$expand(%) // hitung hasilnya
```

$$0$$

```
>&(b+a)^3
```

$$(b + a)^3$$

```
>$&(a+b)^2
```

$$(b+a)^2$$

```
>$expand((a+b)^2), $factor(x^2+5*x+6)
```

$$(x+2)(x+3)$$

```
>$solve(a*x^2+b*x+c,x) // rumus abc
```

$$\left[x = \frac{-\sqrt{b^2 - 4ac} - b}{2a}, x = \frac{\sqrt{b^2 - 4ac} - b}{2a} \right]$$

```
>$&(a^2-b^2)/(a+b), $ratsimp(%)
```

$$a - b$$

```
>$solve([a1*x+b1*y=c1,a2*x+b2*y=c2],[x,y])
```

$$\left[\left[x = \frac{b_1 c_2 - b_2 c_1}{a_2 b_1 - a_1 b_2}, y = \frac{a_2 c_1 - a_1 c_2}{a_2 b_1 - a_1 b_2} \right] \right]$$

Berikut adalah beberapa cara menampilkan soal dan penyelesaian dengan Maxima dan tampilan menggunakan $\text{\LaTeX}$.

```
>$showev('expand((a+b)^2)), $showev('factor(x^2+5*x+6))
```

$$\text{factor}(x^2 + 5x + 6) = (x + 2)(x + 3)$$

```
>$ '(a+b)^2 =expand((a+b)^2), ...
$ '(x^2+5*x+6)=factor(x^2+5*x+6)
```

$$(b+a)^2 = b^2 + 2ab + a^2$$

$$x^2 + 5x + 6 = (x+2)(x+3)$$

```
>$ '(a+b)^2=expand((a+b)^2)
```

$$(b+a)^2 = b^2 + 2ab + a^2$$

```
>$ '(x^2+5*x+6)=factor(x^2+5*x+6)
```

$$x^2 + 5x + 6 = (x+2)(x+3)$$

```
>$showev('diff(exp(sin(x))*cos(x),x))
```

$$\frac{d}{dx} (e^{\sin x} \cos x) = e^{\sin x} \cos^2 x - e^{\sin x} \sin x$$

```
>$showev('diff(exp(sin(x))*cos(x),x,2))
```

$$\frac{d^2}{dx^2} (e^{\sin x} \cos x) = -3e^{\sin x} \cos x \sin x + e^{\sin x} \cos^3 x - e^{\sin x} \cos x$$

```
>$showev('integrate(sin(x)+tan(x),x))
```

$$\int \tan x + \sin x \, dx = \log \sec x - \cos x$$

```
>$showev('ratsimp((a^2-b^2)/(a+b))')
```

$$\text{ratsimp}\left(\frac{a^2 - b^2}{b + a}\right) = a - b$$

```
>$ '(a^2-b^2)/(a+b)=ratsimp((a^2-b^2)/(a+b))
```

$$\frac{a^2 - b^2}{b + a} = a - b$$

2.5.3 Pemrograman dalam EMT

Selain sebagai program aplikasi untuk matematika, EMT juga merupakan bahasa pemrograman tingkat tinggi. EMT merupakan bahasa interpreter, sehingga program dapat dijalankan tanpa melalui proses kompilasi. Program dalam EMT sangat sederhana. Program Euler merupakan suatu fungsi. Fungsi dapat berupa fungsi numerik maupun fungsi simbolik. Program dapat ditulis dalam bentuk fungsi satu baris atau fungsi multi-baris. Program/fungsi satu baris digunakan untuk mendefinisikan fungsi yang memiliki rumus sederhana. Fungsi multi-baris digunakan untuk mendefinisikan fungsi yang memiliki rumus agak kompleks.

Contoh pertama kita adalah fungsi satu baris sederhana. Fungsi seperti itu didefinisikan dengan kata kunci `function` diikuti nama fungsi dengan variabel input/argumen/parameter (ditulis di dalam tanda kurung), tanda `:=` dan ekspresi yang merupakan rumus/hasil fungsi dalam variabel input.

```
>function f(x) := x^3+x^2-x
```

Fungsi ini dapat digunakan di dalam ekspresi numerik apa pun, seperti halnya fungsi bawaan EMT.

```
>f(5)+f(f(3))
37138
>function g(x) := x^x
>g(1:5)
[ 1  4  27  256  3125 ]
```

Apabila rumus fungsi cukup panjang atau kompleks, Anda dapat memecah fungsi satu baris menjadi dua baris menggunakan tanda tiga titik (`...`).

```
>function f(x) := x^4+x^3+x^2 ...
+x;
```

Sebenarnya contoh ini masih termasuk fungsi sederhana, karena memiliki rumus dengan satu ekspresi dan didefinisikan dengan tanda `:=`. Anda dapat memeriksa definisi suatu fungsi atau menampilkan jenis dan definisinya dengan menggunakan perintah `type`.

```
>type f
function f (x)
useglobal; return x^4+x^3+x^2+x;
endfunction
```

Perhatikan perintah `useglobal` yang membuat semua variabel global terlihat. Berikut ini ditunjukkan, bahwa fungsi satu baris juga dapat menjadi bagian dari

perintah multi-baris, jika diakhiri dengan titik koma (;).

```
>aglob=4; ... // perintah multi-baris
function f(x) := aglob*x; ... // fungsi satu baris
f(5) // perintah terakhir
20
```

Fungsi multi-baris

Cara mendefinisikan fungsi yang kedua adalah dalam bentuk fungsi multi-baris, yang memiliki struktur sebagai berikut.

```
function nama_fungsi (variabel masukan/parameter) ...
return ...
...
endfunction
```

Definisi fungsi dimulai dengan kata `function` diikuti nama fungsi dan variabel input/argumen/parameter (ditulis di dalam tanda kurung), kemudian tekan tombol [Ctrl]+[Enter] untuk berganti baris. Kedua tombol ini akan membawa Anda ke modus edit fungsi. Nilai fungsi dinyatakan dengan kata `return` diikuti ekspresi yang menyatakan rumus/nilai fungsi. Definisi fungsi harus diakhiri dengan baris yang berisi kata `endfunction`. Kata penutup definisi fungsi ini dapat secara otomatis muncul dengan cara menekan tombol [Enter] pada baris kosong di akhir definisi fungsi multi-baris.

```
>function f(x)
return x^3+x^2-x
endfunction
```

Berikut adalah fungsi multi-baris yang sama, namun menggunakan tanda ... di akhir baris pertama.

```
>function f(x) ...
return x^3+x^2-x
endfunction
```

Metode lain untuk mendefinisikan fungsi Euler adalah dengan menggunakan editor internal untuk menuliskan definisi fungsi. Dengan menekan tombol [F9] Euler akan menampilkan editor internal yang dengannya Anda dapat menuliskan definisi fungsi.

Untuk mengedit fungsi multi-baris, klik di suatu tempat di dalam definisi fungsi dan edit fungsinya, atau tekan [F9] untuk mengeditnya melalui editor internal. Berikut adalah beberapa tombol yang bermanfaat ketika mengedit fungsi.

- [Alt]+[Insert]: menyisipkan baris.
- [Alt]+[Back]: menghapus baris.
- [Alt]+[U]: menyisipkan baris yang dihapus.
- [Ctrl]+[Enter]: membagi 1 baris menjadi 2 baris
- [Ctrl]+[Back]: menggabungkan baris ke baris sebelumnya.
- [Enter]: keluar dari modus edit dan akhiri definisi fungsi.
- [Escape]: keluar dari modus edit.

```
>function map f(x) // fungsi multi-baris ...  
  if x>0 then return x^2  
  // syarat x>0 tidak berlaku jika x berupa matriks  
  else return x^3  
  endif;  
endfunction
```

Suatu fungsi dapat memuat lebih dari satu pernyataan `return`. Apabila definisi fungsi multi-baris tidak memuat pernyataan `return`, maka hasilnya adalah `none`, string dengan satu karakter sama dengan 1. Hasil ini tidak akan ditampilkan.

```
>function f(x) ...  
  endfunction  
>f(2)  
>
```

Berbagai struktur pengendalian program dapat digunakan di dalam definisi fungsi, terutama untuk fungsi-fungsi multi-baris. Struktur demikian sebenarnya juga dapat digunakan pada perintah satu baris. Di antara struktur pengendalian program yang dapat digunakan di EMT adalah struktur pengambilan keputusan dengan `if ... then ... else ...endif` atau `if ... then ... elseif ...then ... else ... endif`, struktur pengulangan dengan `repeat ... until ...end`, `repeat ... while ...end`, `for ... to ... end`, atau `loop ... to ... end`.

Berikut ini adalah contoh mendefinisikan fungsi tanda untuk bilangan riil, dengan hasil -1 untuk bilangan riil negatif, 0 untuk bilangan riil nol, dan 1 untuk bilangan riil positif.

```

>function map tanda(x) ...
    if x<0 then return -1;
    elseif x==0 then return 0;
    else return 1
    endif;
endfunction
>tanda(-5)
-1
>tanda(0)
0
>tanda(3)
1
>A=[-3,0,1,2,-5]
[-3, 0, 1, 2, -5]
>tanda(A)
[-1, 0, 1, 1, -1]

```

Kata `map` dapat digunakan pada definisi fungsi untuk menandai bahwa fungsinya dapat bekerja dengan input x berupa vektor/matriks.

Berikut ini adalah contoh fungsi yang menggunakan struktur pengulangan yang banyaknya peng-ulangan tidak diketahui sebelumnya. Fungsi `hakar2()` digunakan untuk menghitung hampiran $\sqrt{2}$ dengan algoritma

$$x_{n+1} = \frac{x_n + 2/x_n}{2}, \quad x_0 \text{ ditentukan, dan } x_0 \neq 0.$$

```

>function hakar2(x0) ...
    repeat
        xnew=(x0+2/x0)/2;
        until x0~=xnew;
        x0=xnew;
    end;
    return x0;
endfunction
>hakar2(1)
1.41421356237
>hakar2(5)
1.41421356237

```



```
>hakar2(0)
Floating point error!
Try "trace errors" to inspect local variables
after errors.
hakar2:
      xnew=(x0+2/x0)/2;
```

Pada contoh penggunaan fungsi di atas, hasilnya selalu berupa hampiran $\sqrt{2} \approx 1.41421356237$, berapa pun masukan fungsi, asalkan bukan 0.

Apabila banyak pengulangan sudah diketahui sebelumnya, struktur pengulangan dapat menggunakan sintaks `for ... to ... end`, atau `loop ... to ... end`. Sebagai penghitung proses pengulangan dapat digunakan variabel indeks atau vektor dan dapat dirujuk dengan menggunakan nama variabel indeks, fungsi `index()` atau karakter `#`. Berikut adalah beberapa contoh penggunaan struktur pengulangan ini untuk menghitung jumlah $1+2+3+\dots+10$.

```
>s=0; loop 1 to 10; s=s+#; end; s,
55
>s=0; for i=1 to 10; s=s+i; end; s,
55
>s=0; for i=10 to 1 step -1; s=s+i; end; s,
55
>v=1:10; s=0; for i=v; s=s+i; end; s,
55
>s=0; i=1; repeat; s=s+i; i=i+1; until i>10; end; s,
55
>s=0; i=1; repeat; s=s+i; i=i+1; while i<=10; end; s,
55
```

Proses pengulangan dapat dihentikan dengan menggunakan perintah `break` atau `return` (biasanya dalam pernyataan `if`), yang akan membuat proses melompat ke perintah `end`. Perintah `continue` akan melanjutkan proses pengulangan. Contoh program coba di bawah ini menghitung nilai n terkecil, sedemikian hingga

$$s = \sum_{k=1}^n \frac{1}{k} \geq 10.$$

```

>function coba ...
  s=0; n=1;
  repeat
    s=s+1/n;
  while s<10;
  n=n+1;
  end; s
  return n;
endfunction
>coba
  10.0000430083
  12367

```

Diperoleh nilai $n = 12367$ dan $s = \sum_{k=1}^{12367} \frac{1}{k} = 10.0000430083$.

Suatu fungsi dapat menghasilkan nilai lebih dari satu. Dalam hal ini, ekspresi-ekspresi yang menyatakan hasil suatu fungsi multi-nilai ditulis di dalam tanda kurung kurawal dan dipisahkan dengan koma. Dalam hal ini, pemanggilan fungsi multi-hasil tanpa menggunakan sejumlah variabel yang sesuai, maka hanya nilai ekspresi pertama yang akan disimpan dan dapat digunakan sebagai argumen/masukan ke fungsi lain, kecuali definisi fungsi menggunakan kata `args`.

```

>function f(a,b) := {2*b,a^2}
>{x,y}=f(1,2); x, y,
  4
  1
>function sort2 (a,b) ...
  if a<b then return {a,b}
  else return {b,a}
endfunction
>{c,d}=sort2(5,-14); [c,d]
  [-14  5 ]
>function f(a,b) := {2*b,a^2}
>{x,y}=f(f(1,2)); x, y, // menyebabkan kesalahan
Function f needs at least 2 arguments!
Use: f (a, b)
Error in:
{x,y}=f(f(1,2)); x, y, ...
      ^

```

```
>function args f(a,b) := {2*b,a^2}
>{x,y}=f(f(1,2)); x, y, ...
// berhasil sesuai yang diharapkan
2
16
```

Di dalam EMT terdapat beberapa fungsi yang tidak dapat dipanggil. Fungsi semacam ini digunakan untuk menampilkan penjelasan/dokumentasi fungsi-fungsi yang disimpan di dalam file pustaka DLL, atau fungsi Maxima. Untuk mendefinisikan fungsi demikian, gunakan kata `comment` di belakang kata `function` sebelum nama fungsi. Penjelasan ditulis di dalam definisi fungsi langsung di bawah baris judul fungsi pada baris yang diawali dengan tanda `##` dan spasi.

```
function comment nama_fungsi (variabel masukan)
  ## Penjelasan ...
endfunction
```

Anda dapat menentukan nama alias (nama lain) untuk suatu fungsi dengan menggunakan sintaks `alias namalias namasli` dengan `namalias` adalah nama lain, dan `namasli` adalah nama fungsi aslinya.

```
>alias h hakar2
>h(2)
1.41421356237
>hakar2(7)
1.41421356237
```

Parameter/argumen/variabel masukan fungsi

Variabel masukan suatu fungsi dapat memiliki nilai *default* yang sudah ditentukan (apabila ketika dipanggil tidak ditentukan). Jika argumen suatu fungsi memiliki nilai *default* dalam definisi fungsi seperti `f(x, n=3)`, nilai `n=3` akan digunakan pada saat fungsi `f` dipanggil, kecuali pemanggilan `f` menggunakan dua masukan. Jika suatu argumen fungsi tidak disebutkan seperti `h(x,,4)`, maka nilai *default* untuk parameter kedua akan digunakan.

```
>function f(x,a=5) := a*x
>f(2) // x=2, a=5
10
>f(2,3) // x=2, a=3
6
```

Parameter dengan nilai *default* dapat diatur menggunakan argumen yang diberik-

an nilai seperti $f(10, a=5)$. Cara ini memungkinkan penggunaan fungsi yang sangat fleksibel, karena urutan argumen yang ditetapkan tidak penting. Pemberian nilai suatu argumen boolean (logika) suatu fungsi dapat ditulis dengan cara $>a$ (artinya $a = 1$) atau $<a$ (artinya $a = 0$). Selain itu, opsi $=a$ pada saat memanggil suatu fungsi akan memberikan nilai parameter yang bersesuaian dengan nilai yang sama dengan variabel a .

Aslinya, hanya mungkin untuk menggunakan nilai masukan ketika memanggil suatu fungsi apabila parameter yang sesuai dengan nilai *default* memang tersedia. Apabila di dalam definisi fungsi digunakan kata `allowassigned`, maka ketika fungsi tersebut dipanggil dimungkinkan penggunaan argumen tambahan. Argumen ini bekerja seperti variabel dalam fungsi tersebut.

Parameter suatu fungsi dimasukkan dengan nilai yang sesuai, kecuali nama parameter dimulai dengan tanda `%`, bahkan jika parameter dimasukkan dengan rujukan, rujukan tersebut hanya dapat mengubah nilainya, bukan jenisnya. Apabila parameter tersebut berupa matriks, maka rujukan yang dimasukkan tidak dapat mengubah ukurannya.

```
>a=4;
>function ubah$x (x) ...
    x=5;
endfunction
>ubah$x(a); a,
    4
>function ubah$x (%x) ...
    %x=5;
endfunction
>ubah$x(a); a,
    5
```

Parameter suatu fungsi dapat dibatasi untuk jenis data tertentu.

```
>function map f(a:real) := integrate("sinc(x)",0,a)
>f(0:0.5:2)
    [0, 0.493107, 0.946083, 1.32468, 1.60541]
```

Jika dalam memanggil suatu fungsi digunakan titik koma (;) untuk memisahkan masukan, seperti pada $f(x, y; 4, 5)$, maka parameter 4, 5 bersifat tambahan, yang dapat dilewatkan dari f ke fungsi lain menggunakan fungsi `args()`. Parameter ini disebut **parameter titik koma** atau juga dikenal dengan **parameter tanpa nama**. Fungsi `args()` dapat digunakan untuk membaca parameter tambahan dan menyimpannya dengan nama `arg1`, `arg2`, `arg3`, dan seterusnya. Terdapat beberapa fungsi EMT terkait hal ini.

`argn()`: banyaknya argumen yang diberikan ketika memanggil suatu fungsi,
`args(n)`: semua argumen mulai argumen ke- n ,
`getarg(n)`: argumen ke- n ,
`args()`: semua parameter dari argumen pertama yang tidak disebutkan namanya atau argumen pertama setelah titik koma ";"

Pada contoh berikut ini, kita menggunakan suatu fungsi (h) sebagai masukan untuk fungsi lain (g) dan masukan untuk fungsi h diperoleh dari fungsi `args()` yang nilainya diperoleh ketika memanggil g sebagai parameter tanpa nama.

```

>function f(a,b,c) := a*b*c
>function g(h) := h(args())
>g("f";5,6,7) // sama saja dengan f(5,6,7)
  210
>f(5,6,7)
  210
  
```

Parameter titik koma tidak dapat digunakan di dalam suatu fungsi, dan hanya dapat digunakan melalui fungsi `args()` sebagai parameter (masukan) untuk fungsi lain.

```

>p1="a*x";
>function f(p1) := p1(2,args()); // sama saja f(a)=2a
>function g(p1,a) := f(p1;a) // sama saja g(a)=f(a)
>g(p1,4) // hasilnya akan 2*4
  8
  
```

Parameter suatu fungsi dapat dibatasi pada jenis nilai yang dapat digunakan ketika memanggilnya. Kemungkinan nilai-nilai yang dapat digunakan pembatasan adalah sebagai berikut.

`real`, `complex`, `interval`, `integer`: termasuk skalar, vektor, atau matriks

`scalar`: bukan vektor maupun matriks

`vector`: vektor baris, termasuk skalar

`column`: vektor kolom, termasuk skalar

`numerical`: bukan string matriks terkompres

`string`: tidak termasuk vektor string

`cpx`: matriks terkompres

`positive`, `nonnegative`: clue tambahan.

Jenis-jenis ini dapat dikombinasikan, misalnya `complex vector` akan menerima semua masukan yang berupa vektor baris bilangan kompleks maupun riil.

Terdapat penyingkat untuk menyebut jenis-jenis tersebut.

```
number = real scalar
natural = non-negative integer
index = positive integer scalar
indices = positive integer vector
```

```
>function f(x:number) := x^2
>f(1:5)
Function f needs a scalar for x

Error in :
f(1:5)
^
>f(5)
25
```

Parameter yang memiliki nilai *default*, seperti dalam $f(x, y, c=5)$, harus menjadi parameter terakhir dalam daftar parameter, bahkan setelah parameter titik koma. Parameter khusus $>c$ sama dengan dengan $c = \text{true}$, dan $<c$ setara dengan $c=\text{false}$. Parameter Boolean seperti ini sering digunakan dalam fungsi `plot2d()` dan `plot3d()`.

Variabel dalam Fungsi

Variabel yang digunakan di dalam suatu fungsi dapat berupa variabel lokal maupun variabel global. Variabel lokal adalah variabel yang namanya hanya disebut (digunakan) di dalam definisi fungsi. Variabel global adalah variabel yang didefinisikan di luar fungsi, pada baris perintah EMT. Variabel global dapat diakses di dalam suatu fungsi dengan menggunakan perintah `useglobal` (menggunakan semua variabel global) atau `global nama_variabel` (menggunakan variabel tertentu).

```
>function f(x)
  useglobal; // cara lain: global a;
  return a*x; // x adalah variabel lokal,
  a adalah variabel global
endfunction
>a=4; f(2)
8
```

Untuk menghasilkan variabel global dengan nilai tertentu gunakan `global nama_variabel = nilai`.

```
>function newvar (x) ...
    global nv=x;
endfunction
>newvar(4); nv,
4
```

Variabel yang diakhiri dengan \$\$ secara otomatis akan menjadi variabel global.

```
>a$$ = 1:4 // memberi nilai variabel global a$$
[1, 2, 3, 4]
>v = a$$ ...
// menyimpan nilai variabel global a$$ ke variabel v
[1, 2, 3, 4]
```

Selain itu, suatu variabel dapat ditetapkan sebagai variabel global dengan perintah `setglobal`.

```
>v=1:3
[1, 2, 3]
>setglobal v;
>function changev () ...
    v[2]=5;
endfunction
>changev(); v,
[1, 5, 3]
```

Sebagai catatan, fungsi satu baris menggunakan `useglobal` secara otomatis. Jadi semua variabel global dapat digunakan. Akan tetapi, parameter memiliki prioritas.

```
>function f(x) := a*x // fungsi dengan parameter a
>a=6; f(5) // fungsi dipanggil dengan parameter ...
(variabel global a)
30
```

Evaluasi ekspresi juga dapat melihat variabel global, pada kenyataannya hanya variabel global yang diberikan sebagai argumen untuk evaluasi.

```
>function ev(fx,x) := fx(x) ...
// mendefinisikan fungsi ev dengan ekspresi fx
>a=6; ev("a*x",2) // hasilnya 6*2
12
```

Argumen lebih diutamakan daripada variabel global.

```
>a=5; f(2,3) // hasilnya 3*2 BUKAN 5*3
6
>"a*x"(2,a=3) // hasilnya 3*2 BUKAN 5*3
6
>"a*x"(2) // hasilnya 5*2 karena nilai a ...
        tidak ditentukan di sini
10
```

Selain itu, terdapat jenis daftar (*list*) global. Daftar dapat bertambah anggota sesuai yang diperlukan. Beberapa fungsi terkait daftar di EMT adalah sebagai berikut.

`glist("namadaftar")`: membuat daftar dengan nama "nama"
`glistget("namadaftar",posisi)`: Elemen daftar pada posisi yang diberikan
`glistadd("namadaftar",nilai)`: menambahkan suatu nilai ke daftar "nama"
`glistinsert("namadaftar",posisi,nilai)`: menyisipkan suatu nilai ke daftar
`glistput("namadaftar",posisi,nilai)`: mengganti elemen pada daftar
`glistdelete("namadaftar",posisi)`: menghapus elemen suatu daftar
`glistlength("namadaftar")`: panjang (banyak elemen) suatu daftar
`glistremove("namadaftar")`: menghapus daftar
`listlists`: daftar semua daftar global
`listlists nama`: semua daftar yang namanya memuat "nama"

Berikut adalah contoh menghitung jumlah

$$\sum_{k=1}^{10} k^2$$

dengan menyimpan nilai-nilai k^2 ke dalam daftar v .

```
>glist("v"); // mendefinsikan daftar v ...
        (tidak dapat ditampilkan)
>for k=1 to 10; glistadd("v",k^2); end;
>s=0; for k=1 to 10; s=s+glistget("v",k); end; s,
385
>sum((1:10)^2) // cek hasilnya secara langsung
385
```

EMT menyediakan fungsi-fungsi untuk bekerja dengan variabel global, yakni:

gset("nama", nilai): menyimpan nilai ke variabel global nama,
 gvar("nama"): membaca variabel global nama,
 gremove("nama"): menghapus variabel global nama, dan
 gclear(): menghapus semua variabel global.

Fungsi simbolik

Fungsi simbolik adalah fungsi yang didefinisikan untuk Maxima dan untuk kernel numerik EMT. Fungsi-fungsi ini harus menjadi fungsi satu baris yang didefinisikan dengan $\&=$. Meskipun demikian, penulisan rumus fungsinya tetap dapat berlanjut ke beberapa baris dengan menggunakan tanda $\dots$ sebagai penghubung ke baris berikutnya.

```
>function f(x) &= x^x;
>function df(x) &= diff(f(x), x);
>function h(x) &= diff(f(x), x) + ...
    diff(f(x), x, 2);
```

Fungsi simbolik yang didefinisikan dengan tanda $\&=$ dapat bekerja secara simbolik maupun numerik. Artinya, fungsi tersebut dapat dipanggil dengan masukan simbol maupun bilangan, termasuk matriks skalar.

```
>df(2)
6.77258872224
>df(1:3)
[1, 6.77259, 56.6625]
>$df(a)
```

$$a^a (\log a + 1)$$

Fungsi simbolik juga dapat didefinisikan secara murni dengan tanda $\&\&=$. Fungsi-fungsi ini hanya diketahui oleh Maxima. Mereka tidak dievaluasi pada saat mereka didefinisikan. Fungsi berikut menghitung $\frac{d^2 f}{dx^2} + \frac{d^2 f}{dy^2}$. Ini tentu saja harus dievaluasi (dikerjakan), ketika digunakan, bukan pada saat didefinisikan.

```
>function L(f) &&= diff(f, x, 2) + diff(f, y, 2)
    diff(f, y, 2) + diff(f, x, 2)
>u &= ratsimp(realpart((x + I*y)^3 / exp(x + I*y))); $u
```

$$-e^{-x} ((y^3 - 3x^2 y) \sin y + (3xy^2 - x^3) \cos y)$$

```
>&ratsimp(L(u)) // harus bernilai nol, karena u(x,y) ...
    adalah fungsi harmonik
    0
```

Hasil kerja Maxima diberikan kepada EMT

Ekspre di ruas kanan pada definisi fungsi simbolik menggunakan `&=` dikerjakan oleh Maxima kemudian hasilnya dikirim ke EMT.

```
>function f(x) &= integrate(x^3*cos(x),x)
              3              2
              (x  - 6 x) sin(x) + (3 x  - 6) cos(x)
>type f // cek definisi fungsinya
function f (x)
useglobal; return (x^3-6*x)*sin(x)+(3*x^2-6)*cos(x)
endfunction
```

Ekspre simbolis juga dapat dimasukkan ke dalam fungsi multi-baris. Sekali lagi, ekspre dievaluasi pada waktu kompilasi, dan hasilnya dimasukkan ke dalam fungsi EMT normal. Berhati-hatilah untuk mengatur tanda kurung, karena hasil evaluasi simbolis dimasukkan kata demi kata.

```
>p1 &= x^x-2;
>function iterasinewton (x)
    return x-(&:"p1")/(&:"diff(p1,x)");
endfunction
>type iterasinewton // cek definisi fungsinya
function iterasinewton (x)
    return x-(x^x-2)/(x^x*(log(x)+1));
endfunction
>iterate("iterasinewton",2) // hasil iterasi untuk x=2
1.55961046946
```

2.5.4 Skrip

Sebagian besar fungsi EMT tidak tersedia pada bahasa C aslinya, tetapi ditulis dalam bahasa pemrograman EMT dan disimpan ke serta dipanggil/dimuat/dibaca dari file Euler (*.e) ketika program EMT dijalankan. File Euler dapat diedit dengan editor mana pun. Untuk memulai pengeditan, masukkan perintah

```
>load namafile
```

di dalam file notebook EMT dan tekan F9 (editor internal) atau F10 (editor eksternal, aslinya JE). File akan disimpan di direktori/folder di mana notebook

yang sedang diedit disimpan. Dalam kasus editor internal, EMT akan menunggu pengeditan selesai.

Skrip dapat berisi:

- definisi fungsi (`function ...`)
- Perintah EMT (`>`)
- panggilan ke Maxima (`>::`)
- fungsi komentar (`function comment nama_fungsi ...`)
- fungsi komentar Maxima (`maximafunction nama_fungsi ...`)
- entri untuk menu pengguna (`addmenu ...`)
- bagian komentar (`comment ... endcomment`)
- Komentar baris tunggal (`// ...`)
- perintah memanggil file Euler lain (`load nama_file`)

Pastikan, fungsi memiliki baris komentar yang tepat. Untuk fungsi atau fungsi dalam bahasa C aslinya yang dimuat oleh DLL, terdapat fungsi komentar. Untuk fungsi Maxima yang didefinisikan di dalam EMT, terdapat definisi `maximafunction`.

```
>:: h(x) := x^3; ...  
maximafunction h (x) ## Menghitung x^3 endfunction
```

Kemudian

```
>help &h
```

akan bekerja. Fungsi `h` akan dikenal di jendela bantuan.

Bantuan untuk file Euler tersedia dengan memasukkan nama file di jendela bantuan, atau dengan mengklik dua kali pada baris yang memuat perintah atau fungsi tersebut.

Skrip dapat dijalankan pada baris perintah. *Output* akan hilang kecuali file digunakan. Gunakan parameter `"-script script.e"` untuk memulai skrip EMT.

Bab 3.

Komputasi Aljabar dengan EMT

Bab ini menjelaskan tentang berbagai perhitungan terkait dengan materi atau topik dalam aljabar dengan menggunakan software EMT. Di antara masalah-masalah aljabar adalah:

- melakukan operasi bentuk-bentuk aljabar (menjabarkan, menyederhanakan, memfaktorkan, dll.)
- melakukan perhitungan dengan berbagai operasi dan fungsi matematika
- melakukan perhitungan menggunakan bilangan kompleks
- melakukan perhitungan menggunakan fungsi-fungsi buatan sendiri
- menyelesaikan persamaan dan sistem persamaan
- menyelesaikan pertidaksamaan dan sistem pertidaksamaan
- melakukan manipulasi dan perhitungan menggunakan matriks dan vektor
- menggunakan aljabar untuk menyelesaikan masalah sehari-hari atau dalam matematika dan bidang lain.

Oleh karena pembahasan topik-topik tersebut ternyata cukup panjang, maka pembahasannya dijadikan dua bab. Bab ini secara khusus membahas topik-topik:

1. pengertian aljabar dan bentuk aljabar,
2. operasi bentuk aljabar dan penyederhanaannya,
3. faktorisasi bentuk aljabar,
4. fungsi polinomial dan rasional,
5. fungsi komposisi dan invers,
6. fungsi eksponensial,
7. fungsi logaritma,
8. fungsi trigonometri dan hiperbolik, dan
9. bilangan kompleks.

Topik-topik yang lebih lanjut dibahas pada bab berikutnya.

Semua contoh komputasi aljabar di dalam bab ini dilakukan secara simbolik dengan menggunakan Maxima di EMT. Semua contoh juga ditampilkan dengan

menggunakan LaTeX, yakni dengan cara setiap perintah Maxima diawali dengan tanda dolar (\$).

3.1 Pengertian Aljabar dan Bentuk Aljabar

Matematika dapat dipandang bahasa ilmu pengetahuan yang menggunakan simbol dan aturan-aturan yang telah disepakati. Aljabar merupakan salah satu bidang kajian dalam matematika sebagai bahasa. Aljabar merupakan bahasa lambang dan operasi atau relasinya. Aljabar adalah salah satu cabang matematika terkait ilmu bilangan, geometri dan analisis dengan menggunakan huruf-huruf atau yang biasa disebut sebagai variabel untuk menyatakan objek-objek matematika tertentu.

Pada aljabar dikenal beberapa istilah sebagai berikut:

1. variabel, yaitu lambang pengganti nilai yang belum diketahui (atau nilainya dapat berubah-ubah);
2. koefisien, yaitu bilangan sebagai pengali variabel atau perkalian beberapa variabel;
3. konstanta, yaitu bilangan yang ditulis terpisah dari variabel;
4. bentuk aljabar;
5. suku.

Variabel atau peubah adalah lambang atau simbol (biasanya berupa huruf) yang dapat digantikan oleh bilangan tertentu dalam semesta pembicaraan. Dalam aljabar terdapat konstanta yang menunjuk anggota tertentu berupa bilangan dalam himpunan semestanya.

Bentuk aljabar merupakan bentuk yang memuat angka (lambang bilangan), variabel atau peubah yang digunakan untuk menyajikan bilangan secara umum, dan operasi matematika (penjumlahan, pengurangan, perkalian, pembagian, dll.) sebagai sarana penyerdahan dan alat bantu untuk menyelesaikan suatu masalah. Bentuk aljabar adalah kombinasi bilangan dan beberapa huruf (variabel) yang mewakili nilai-nilai bilangan.

Bentuk aljabar dapat berupa suku satu, suku dua, dan suku banyak. Suku-suku pada bentuk aljabar dipisahkan oleh tanda operasi penjumlahan atau pengurangan. Bentuk aljabar mungkin dapat disederhanakan dengan cara melakukan operasi matematika yang terdapat pada bentuk aljabar tersebut.

Contoh-contoh bentuk aljabar: $3x^2 - 5xy + 2y^4$, $2a^3b^5$, $\frac{5xy+3z}{2a^3-c^2}$.

Setiap perkalian atau pembagian yang melibatkan bilangan dan beberapa huruf (variabel) yang mewakili bilangan disebut suku. Dengan demikian, $6x^2y^3$, $5x/3y^4$, $-3x^7$ adalah contoh-contoh suku. Akan tetapi, $6x^2 + 7xy$ merupakan ekspresi/bentuk aljabar yang terdiri atas dua suku.

Pengali di dalam suatu suku disebut koefisien. Koefisien yang berupa bilangan

disebut koefisien numerik (atau hanya disebut koefisien). Suku-suku yang hanya berbeda koefisien numeriknya disebut suku-suku sejenis. Beberapa suku sejenis dapat digabung menjadi satu suku dengan menjumlahkan koefisien-koefisien numeriknya.

Ekspresi/bentuk aljabar yang hanya terdiri atas satu suku disebut monomial (atau sering disebut suku). Bentuk aljabar yang terdiri atas dua suku disebut binomial. Bentuk aljabar yang terdiri atas tiga suku disebut trinomial. Demikian seterusnya dalam penamaan bentuk-bentuk aljabar. Bentuk aljabar yang terdiri atas lebih dari satu suku disebut multinomial.

Suatu suku dikatakan bulat dan rasional dalam beberapa variabel tertentu apabila suku tersebut terdiri atas hasil kali perpangkatan bulat positif variabel-variabel tersebut dan suatu koefisien numerik, atau tidak memuat variabel.

Contoh suku-suku bulat dan rasional (dalam variabel-variabel yang tertulis): $6x^2y^3$, $-5y^4$, 7 , $-4x$, $\sqrt{3}x^3y^6$.

Sementara, $3\sqrt{x}$ bukan suku rasional dalam x , sedangkan $4/x^2$ bukan suku bulat dalam x .

Contoh monomial (suku): $7x^3y^4$, $3xyz^2$, $4x^2/y$.

Contoh binomial: $2x + 4y$, $3x^4 - 4xyz^3$.

Contoh trinomial: $3x^2 - 5x + 2$, $2x + 6y - 3z$, $x^3 - 3xy/z - 2x^3z^7$.

Contoh multinomial: $7x + 6y$, $3x^3 + 6x^2y - xy + 6$, $7x + 5x^2/y - 3x^3/16$.

Monomial atau multinomial yang setiap sukunya bulat dan rasional disebut polinomial.

Contoh-contoh polinomial: $3x^2y^3 - 5x^4y + 2$, $2x^4 - 7x^3 + 3x^2 - 5x + 2$, $4xy + z$, x^2 .

Di antara yang bukan polinomial adalah: $3x^2 - 4/x$, $4\sqrt{y} + 3$.

Derajat suatu suku adalah jumlah pangkat semua variabel di dalam suku tersebut. Suku konstanta memiliki derajat nol.

3.2 Operasi Bentuk Aljabar dan Menyederhanakan Hasilnya

Menjumlahkan dan mengurangkan bentuk-bentuk aljabar dilakukan dengan cara menggabungkan suku-suku yang sejenis (menjumlahkan koefisien suku-suku sejenis).

Contoh 1. Sederhanakan:

$$(2x + 3y + z - 7) + (4x - 2y - z + 8) + (-3x + y - 2z - 4).$$

Jawab:

```
>$ (2*x+3*y+z-7) + (4*x-2*y-z+8) + (-3*x+y-2*z-4)
```

$$-2z + 2y + 3x - 3$$

Penjelasan:

- $2x$, $4x$, dan $-3x$ merupakan suku sejenis, sehingga akan dikelompokkan dan dijumlahkan, menghasilkan $3x$;
- $3y$, $-2y$, dan y merupakan suku sejenis, sehingga akan dikelompokkan dan dijumlahkan, menghasilkan $3y$;
- z , $-z$, dan $-2z$ merupakan suku sejenis, sehingga akan dikelompokkan dan dijumlahkan, menghasilkan $-2z$;
- -7 , 8 , dan -4 merupakan konstanta, sehingga akan dikelompokkan dan dijumlahkan, menghasilkan -3 ;

Jadi hasil akhirnya adalah

$$3x + 2y - 2z - 3.$$

Perhatikan, EMT aslinya menampilkan bentuk aljabar menggunakan urutan abjad dari z ke a dan dari pangkat tertinggi ke pangkat terendah. Untuk mengubah urutan tersebut dapat digunakan perintah `powerdisp:true`.

Contoh 2. Sederhanakan:

$$3x^2 + 4x^4 + 9x^2 + 6x + x^4.$$

Jawab:

```
>&powerdisp:false;
```

Perintah `powerdisp:false`; akan menyebabkan tampilan polinomial dimulai dari suku berpangkat tertinggi.

```
>$3*x^2+4*x^4+9*x^2+6*x+x^4
```

$$5x^4 + 12x^2 + 6x$$

Jadi,

$$3x^2 + 4x^4 + 9x^2 + 6x + x^4 = 5x^4 + 12x^2 + 6x.$$

Berikut adalah contoh-contoh lain penjumlahan dan pengurangan bentuk aljabar.

```
>$ (3*x^2+4*x^4+9*x^2+6*x+y^4+7*x)
```

$$y^4 + 4x^4 + 12x^2 + 13x$$

```
>&powerdisp:true;
```

Perintah `powerdisp:true;` akan menyebabkan tampilan bentuk aljabar dimulai dari suku dengan variabel $a, b, \dots$, dan suku berpangkat terendah.

```
>$ (3*a^2+4*b^4+9*c^2+6*d+e^4+b^2+a^2)
```

$$4a^2 + b^2 + 4b^4 + 9c^2 + 6d + e^4$$

Jadi,

$$(3a^2 + 4b^4 + 9c^2 + 6d + e^4 + b^2 + a^2) = 4a^2 + b^2 + 4b^4 + 9c^2 + 6d + e^4.$$

```
>$ (2*x^2+12*x*y-11) + (6*x^2-2*x+4) + (-x^2-y-2)
```

$$-9 - 2x + 7x^2 - y + 12xy$$

Jadi,

$$(2x^2 + 12xy - 11) + (6x^2 - 2x + 4) + (-x^2 - y - 2) = -9 - 2x + 7x^2 - y + 12xy.$$

```
>&powerdisp:false;
>$ (3*x^2-2*x-x^3+2) - (5*x^2-8*x-x^3+4)
```

$$-2x^2 + 6x - 2$$

$$(3x^2 - 2x - x^3 + 2) - (5x^2 - 8x - x^3 + 4) = -2x^2 + 6x - 2.$$

```
>$ (5*x^2+4*x*y-3*y^2+2) - (9*x^2-4*x*y+2*y^2-1)
```

$$-5y^2 + 8xy - 4x^2 + 3$$

$$(5x^2 + 4xy - 3y^2 + 2) - (9x^2 - 4xy + 2y^2 - 1) = -5y^2 + 8xy - 4x^2 + 3.$$


```
>$6*x^(-3)*y^5*-7*x^2*y^(-9)
```

$$-\left(\frac{42}{xy^4}\right)$$

$$6x^{-3}y^5 \times -7x^2y^{-9} = -\frac{42}{xy^4}.$$

Dalam aljabar, operasi perkalian biasanya identik dengan menguraikan bentuk aljabarnya.

Mengalikan dua bentuk aljabar dilakukan dengan cara mengalikan setiap suku dari kedua bentuk aljabar, kemudian menjumlahkan suku-suku sejenis hasil perkaliannya.

Misalnya: $x(x+2)$.

Dengan menggunakan sifat distribusi, maka perkalian tersebut jika dijabarkan akan menghasilkan $x^2 + 2x$. Berikut ditunjukkan beberapa contoh perkalian bentuk-bentuk aljabar dan cara-cara menuliskan hasilnya.

$$(x+6)(x+3)$$

```
>$ (x+6) * (x+3) // tidak dijabarkan
```

$$(x+3)(x+6)$$

```
>$expand((x+6)*(x+3)) //dijabarkan
```

$$x^2 + 9x + 18$$

Perintah `expand` akan menjabarkan operasi bentuk aljabar.

```
>$ '(x+6)*(x+3) = expand((x+6)*(x+3)) // bentuk kesamaan
```

$$(x+3)(x+6) = x^2 + 9x + 18$$

Operasi bentuk aljabar yang diawali dengan tanda petik tunggal (') akan ditampilkan apa adanya. Hal ini dapat dimanfaatkan untuk menyajikan hasil perhitungan dan soalnya sebagai suatu kesamaan atau identitas.

```
>$showev('expand((a-b)*(2*a^3-a*b+3*b^2))')
```

$$\text{expand}((a-b)(3b^2-ab+2a^3)) = -3b^3 + 4ab^2 - 2a^3b - a^2b + 2a^4$$

Perintah `showev` akan menampilkan perintah (dan ekspresi yang akan dikerjakan) dan hasilnya. Beberapa perintah yang memiliki simbol matematika akan ditampilkan simbol matematikanya, seperti integral, turunan, dan notasi sigma.

```
>$'(2*x+3*y+4)*(2*x+3*y-4)= ...
>      expand((2*x+3*y+4)*(2*x+3*y-4))
```

$$(3y+2x-4)(3y+2x+4) = 9y^2 + 12xy + 4x^2 - 16$$

```
>$'(y-2)*(y+2)*(y^2+4)=expand((y-2)*(y+2)*(y^2+4))
```

$$(y-2)(y+2)(y^2+4) = y^4 - 16$$

```
>$'(a-8)*(a-1)=expand((a-8)*(a-1))
```

$$(a-8)(a-1) = a^2 - 9a + 8$$

```
>&powerdisp:true;
>$'(a+b)^5=expand((a+b)^5)
```

$$(a+b)^5 = a^5 + 5a^4b + 10a^3b^2 + 10a^2b^3 + 5ab^4 + b^5$$

```
>$ (x-2)^3|expand
```

$$x^3 - 6x^2 + 12x - 8$$

Perintah Maxima juga dapat dituliskan dengan menggunakan tanda garis tegak (`|`) di belakang ekspresi yang harus dikerjakan. Cara ini dikenal sebagai *pipe*.

```
>$'(a+b)^3=expand((a+b)^3)
```

$$(a+b)^3 = a^3 + 3a^2b + 3ab^2 + b^3$$

```
>$ '(1+x)^5=expand((1+x)^5)
```

$$(1+x)^5 = 1 + 5x + 10x^2 + 10x^3 + 5x^4 + x^5$$

Pembagian dua monomial dilakukan dengan cara mengalikan hasil pembagian koefisien-koefisien numerik dan hasil pembagian variabel-variabel pada pembilang dan penyebut.

Contoh pembagian dua monomial:

$$\frac{24x^3y^2z}{4xyz^2} = \left(\frac{24}{4}\right) \left(\frac{x^3}{x}\right) \left(\frac{y^2}{y}\right) \left(\frac{z}{z^2}\right) = \frac{6x^2y}{z}.$$

Operasi perpangkatan dengan pangkat negatif diselesaikan dengan cara sebagai berikut

$$(x+1)^{-2} = \frac{1}{(x+1)^2}$$

sehingga didapat hasilnya:

$$\frac{1}{x^2 + 2x + 1}$$

```
>$ '(6*x^(-3)+y^5)*(-7*x^2-y^(-9))= ...
>expand((6*x^(-3)+y^5)*(-7*x^2-y^(-9)))
```

$$\left(-\left(\frac{1}{y^9}\right) - 7x^2\right) \left(y^5 + \frac{6}{x^3}\right) = -(7x^2y^5) - \frac{1}{y^4} - \frac{6}{x^3y^9} - \frac{42}{x}$$

Perintah `combine` dapat digunakan untuk menyederhanakan penjumlahan ekspresi aljabar (pecahan) menjadi satu suku dengan penyebut sama.

```
>$ 1*f/2*b + 2*c/3*a + 3*f/4*b + c/5*b*a= ...
>combine(1*f/2*b + 2*c/3*a + 3*f/4*b + c/5*b*a)
```

$$\frac{5bf}{4} + \frac{abc}{5} + \frac{2ac}{3} = \frac{75bf + 4(3abc + 10ac)}{60}$$

```
>$ '(a+b)^5=expand((a+b)^5)
```

$$(b+a)^5 = b^5 + 5ab^4 + 10a^2b^3 + 10a^3b^2 + 5a^4b + a^5$$

Bagaimana menyederhanakan perpangkatan bentuk perpangkatan? Perhatikan contoh-contoh di bawah ini.

```
>(a^b)^m // tidak disederhanakan ... mengapa?
```

$$(a^b)^m$$

```
>radexpand:=all
all
```

Untuk menyederhanakan perpangkatan bentuk perpangkatan atau akar, variabel Maxima `radexpand` harus diberi nilai `all`.

```
>(a^b)^m // yang ini berhasil
```

$$a^{bm}$$

```
>a^b*a^c // berhasil
```

$$a^{b+c}$$

Jadi, EMT dapat menunjukkan bahwa

$$(a^b)^m = a^{b.m}$$

dan

$$a^b \times a^c = a^{b+c}.$$

Pada contoh berikut ini ditunjukkan bahwa EMT juga dapat menyederhanakan perkalian bentuk perpangkatan yang sedikit lebih rumit. Sudah tentu hal ini harus didahului dengan perintah Maxima untuk memberi nilai variabel `radexpand` dengan `all`, seperti sudah dilakukan di atas.

```
>'(m^(x-b)*n^(x+b))^x*(m^b*n^(-b))^x= ...
> expand((m^(x-b)*n^(x+b))^x*(m^b*n^(-b))^x)
```

$$m^{bx+x(-b+x)} n^{-(bx)+x(b+x)} = m^{x^2} n^{x^2}$$

Berikut ini menunjukkan bahwa

$$(3m^4)^3(2m^{-5})^2 = 108m^2.$$

```
>$ (3*m^4) ^3* (2*m^(-5)) ^2
```

$$108m^2$$

```
>$ '(a+b+c) ^3=expand((a+b+c) ^3)
```

$$(a+b+c)^3 = a^3 + 3a^2b + 3ab^2 + b^3 + 3a^2c + 6abc + 3b^2c + 3ac^2 + 3bc^2 + c^3$$

```
>&powerdisp:false
false
>$ '(x+1) ^5=expand((x+1) ^5)
```

$$(x+1)^5 = x^5 + 5x^4 + 10x^3 + 10x^2 + 5x + 1$$

```
>$ (27*a^-5*b^-3/(3^5*a^-7*b^-5)) ^-1 // disederhanakan
```

$$\frac{9}{a^2b^2} \left(\frac{27a^{-5}b^{-3}}{3^5a^{-7}b^{-5}} \right)^{-1} = \frac{9}{(ab)^2}.$$

Berikut adalah contoh substitusi nilai-nilai variabel pada suatu ekspresi.

```
>$subst([a=4,b=2,c=1/2],(a^-1) ^2*b^4/c^-4)
```

$$\frac{1}{16}$$

Jika

$$a = 4, b = 2, c = \frac{1}{2},$$

maka

$$(a^{-1})^2 \frac{b^4}{c^{-4}} = \frac{1}{16}.$$

```
>$ '6*(3+sqrt(5))*(3-sqrt(5))/(2+sqrt(6)) = ...
> ratsimp(6*(3+sqrt(5))*(3-sqrt(5))/(2+sqrt(6)))
```

$$\frac{6(3-\sqrt{5})(\sqrt{5}+3)}{\sqrt{6}+2} = \frac{24}{\sqrt{6}+2}$$

```
>$%=12*(sqrt(6)-2)
```

$$\frac{6(3-\sqrt{5})(\sqrt{5}+3)}{\sqrt{6}+2} = \frac{24}{\sqrt{6}+2} = 12(\sqrt{6}-2)$$

3.3 Faktorisasi Bentuk Aljabar

Untuk faktorisasi, kita akan menggunakan konsep faktor persekutuan terbesar (FPB) dan sifat distributif aljabar. Sebagai ilustrasi sederhana, misalkan kita akan memfaktorkan penjumlahan dua bilangan:

$$\begin{aligned} 24 + 60 &= 2(12 + 30) && (12 \text{ dan } 30 \text{ memiliki faktor persekutuan } 2) \\ &= 2 \cdot 2(6 + 15) && (6 \text{ dan } 15 \text{ memiliki faktor persekutuan } 2) \\ &= 2 \cdot 2 \cdot 3(2 + 5). && (2 \text{ dan } 5 \text{ tidak memiliki faktor persekutuan}) \end{aligned}$$

Jadi faktor persekutuan terbesar 24 dan 60 adalah 12 dan $24 + 60 = 12 \times (2 + 5)$. Kenyataannya memang $24 + 80 = 84 = 12 \times 7$. Memfaktorkan operasi bentuk aljabar adalah proses mengubah operasi aljabar menjadi bentuk perkalian dengan suku-suku atau faktor-faktor yang lebih sederhana. Pemfaktoran membantu menyederhanakan operasi bentuk aljabar, mengidentifikasi akar-akar persamaan, dan memahami struktur matematis dengan lebih baik. Terdapat beberapa bentuk faktor aljabar.

1. Bentuk distributif (sifat distributif pada pemfaktoran aljabar)

$$px + py = p(x + y).$$

$$px - py = p(x - y).$$

2. Bentuk selisih kuadrat (pemfaktoran selisih kuadrat dua bilangan)

$$a^2 - b^2 = (a + b)(a - b).$$

3. Bentuk kuadrat sempurna (pemfaktoran persamaan kuadrat sempurna)

$$x^2 + 2xy + y^2 = (x + y)(x + y) = (x + y)^2.$$

$$x^2 - 2xy + y^2 = (x - y)(x - y) = (x - y)^2.$$

4. Bentuk persamaan kuadrat (pemfaktoran bentuk umum persamaan kuadrat)

$$x^2 + bx + c = (x + p)(x + q).$$

Dalam hal ini berlaku hubungan $p + q = b$ dan $p \cdot q = c$.

5. Bentuk persamaan kuadrat dengan pangkat tertinggi berkoefisien a

$$ax^2 + bx + c = a(x + p/a)(x + q/a).$$

Dalam hal ini berlaku hubungan $p + q = a$ dan $p \cdot q = a \cdot c$.

Sekarang, perhatikan pemfaktoran bentuk aljabar berikut ini.

$$\begin{aligned} 5x^3 - 12x^2 &= x(5x^2 - 12x) \\ &= x \cdot x(5x + 12) \\ &= x^2(5x - 12). \end{aligned}$$

Berikut adalah contoh-contoh memfaktorkan bentuk-bentuk aljabar di EMT.

```
>$t^2+8*t+15=factor(t^2+8*t+15)
```

$$t^2 + 8t + 15 = (t + 3)(t + 5)$$

```
>$(t+3)*(t+5)=expand((t+3)*(t+5))
```

$$(t + 3)(t + 5) = t^2 + 8t + 15$$

Itu artinya

$$t^2 + 8t + 15 = (t + 3)(t + 5).$$

```
>&powerdisp:false;
>$5*x^2*y-5*y*z^4=factor(5*x^2*y-5*y*z^4)
```

$$5x^2y - 5yz^4 = -5y(z^2 - x)(z^2 + x)$$

```
>$expand(-5*y*(z^2-x)*(z^2+x))
```

$$5x^2y - 5yz^4$$

```
>$4*p^2-8*p*q+4*q^2=factor(4*p^2-8*p*q+4*q^2)
```

$$4q^2 - 8pq + 4p^2 = 4(q - p)^2$$

```
>&powerdisp:true;
>$expand(4*p^2-8*p*q+4*q^2)
```

$$4p^2 - 8pq + 4q^2$$

```
>&powerdisp:false;
>$3*a^5-24*a^2=factor(3*a^5-24*a^2)
```

$$3a^5 - 24a^2 = 3(a - 2)a^2(a^2 + 2a + 4)$$

```
>$m^6+8*m^3-20=factor(m^6+8*m^3-20)
```

$$m^6 + 8m^3 - 20 = (m^3 - 2)(m^3 + 10)$$

Seperti dapat dilihat pada contoh-contoh sebelumnya, Maxima menyediakan beberapa perintah/fungsi untuk menjabarkan dan memfaktorkan operasi-operasi bentuk aljabar, yakni `expand` dan `factor`. Penyederhanaan bentuk-bentuk aljabar sederhana dan operasi bentuk-bentuk aljabar yang sederhana dilakukan secara otomatis oleh Maxima. Untuk bentuk-bentuk aljabar khusus (seperti bentuk akar) atau operasi bentuk-bentuk aljabar yang memuat fungsi-fungsi trigonometri, logaritma, dan perpangkatan diperlukan perintah-perintah khusus.

Perintah `ratsimp(expr)` berfungsi untuk menyederhanakan ekspresi `expr` dan semua subekspresinya, termasuk argumennya ke fungsi non-rasional. Hasilnya dinyatakan sebagai hasil bagi dua polinomial alam bentuk rekursif, yaitu koefisien variabel utama adalah polinomial variabel lainnya. Variabel dapat mencakup fungsi non-rasional (misalnya, `sin(x^2 + 1)`) dan argumen untuk fungsi tersebut juga disederhanakan secara rasional.

Jika variabel `ratsimpexpons` bernilai `true`, `ratsimp` akan menyederhanakan eksponen ekspresi selama penyederhanaan. Hasil fungsi `ratsimp` dipengaruhi oleh beberapa variabel yang mempengaruhi perintah `rateexpand`.

Perhatikan contoh-contoh berikut ini.

```
>&powerdisp:=false;
>$sin (x/(x^2 + x)) = exp ((log(x) + 1)^2 - log(x)^2)
```

$$\sin\left(\frac{x}{x^2+x}\right) = e^{(\log x+1)^2-(\log x)^2}$$

Perhatikan,

$$e^{(\log(x)+1)^2-\log(x)^2} = e^{\log^2(x)+2\log(x)+1-\log^2(x)} = e^{2\log(x)+1} = e \cdot \left(e^{\log(x)}\right)^2 = e \cdot x^2.$$

Persamaan di atas dapat disederhanakan dengan perintah sebagai berikut.

```
>$ratsimp (%)
```

$$\sin\left(\frac{1}{x+1}\right) = ex^2$$

```
>$((x - 1)^(3/2) - (x + 1)*sqrt(x - 1))/...
> sqrt((x - 1)*(x + 1))
```

$$\frac{(x-1)^{\frac{3}{2}} - \sqrt{x-1}(x+1)}{\sqrt{(x-1)(x+1)}}$$

Ekspresi tersebut dapat disederhanakan dengan `ratsimp`.

```
>$ratsimp(%)
```

$$-\left(\frac{2}{\sqrt{x+1}}\right)$$

Jadi,

$$\frac{(x-1)^{\frac{3}{2}} - (x+1)\sqrt{x-1}}{\sqrt{(x-1)(x+1)}} = -\frac{2}{\sqrt{x+1}}.$$

```
>$x^2*y^2*z+a*x*y^2*z-3*x*y^2*z-3*a*y^2*z...
> -2*x^2*y^2-2*a*x*y^2+6*x*y^2+6*a*y^2
```

$$x^2y^2z + axy^2z - 3xy^2z - 3ay^2z - 2x^2y^2 - 2axy^2 + 6xy^2 + 6ay^2$$

Perintah `ratsimp` akan menyederhanakan ekspresi di atas dalam bentuk penjumlahan hasil kali faktor-faktor yang paling sederhana yang mungkin.

```
>$ratsimp(%)
```

$$(x^2 + (a-3)x - 3a)y^2z + (-(2x^2) + (6-2a)x + 6a)y^2$$

```
>&powerdisp:=true;
>$x+y/(x+2)+2/(x^2+4)
```

$$x + \frac{2}{4+x^2} + \frac{y}{2+x}$$

Perintah `ratsimp` akan menyatakan pecahan parsial tersebut ke bentuk pecahan dengan satu penyebut (pembagian dua suku banyak dalam variabel-variabel yang terlibat). Apabila ekspresi aljabar yang disederhanakan memuat variabel x dan variabel lain, maka suku-suku yang memuat variabel x akan dikumpulkan dan dijabarkan.

```
>$'x+y/(x+2)+2/(x^2+4)=ratsimp(x+y/(x+2)+2/(x^2+4))
```

$$x + \frac{2}{4+x^2} + \frac{y}{2+x} = \frac{4+10x+4x^2+2x^3+x^4+(4+x^2)y}{8+4x+2x^2+x^3}$$

Untuk menyederhanakan ekspresi yang memuat logaritma, eksponensial, dan radikal (akar pangkat) digunakan perintah/fungsi `radcan`. Hasil penyederhanaannya dalam bentuk kanonik (baku/normal), yakni penjumlahan hasil kali/perpangkatan variabel-variabelnya. Berikut adalah beberapa contoh penggunaan perintah `radcan`.

```
>$'(%e^x-1)/(1+%e^(x/2))=radcan((%e^x-1)/(1+%e^(x/2)))
```

$$\frac{e^x - 1}{e^{\frac{x}{2}} + 1} = e^{\frac{x}{2}} - 1$$

```
>$' (%e^(-x)+%e^x) / (%e^(-x)-%e^x) = ...
> radcan((%e^(-x)+%e^x) / (%e^(-x)-%e^x))
```

$$\frac{e^x + e^{-x}}{e^{-x} - e^x} = - \left(\frac{e^{2x} + 1}{e^{2x} - 1} \right)$$

```
>$' (x+2*sqrt(x)+1) / (1+sqrt(x)) = ...
> radcan((x+2*sqrt(x)+1) / (1+sqrt(x)))
```

$$\frac{x+2\sqrt{x}+1}{\sqrt{x}+1} = \sqrt{x}+1$$

Di atas sudah ditunjukkan cara memfaktorkan bentuk aljabar dengan menggunakan perintah/fungsi `factor`. Perintah ini akan memfaktorkan bentuk aljabar sebisa mungkin. Untuk memfaktorkan suatu bentuk aljabar menjadi hasil kali faktor-faktor kompleks dapat digunakan perintah/fungsi `gfactor`. Perhatikan perbedaan hasil kedua perintah tersebut.

```
>$' x^4-9*x^3+9*x^2+85*x-150 = ...
> factor(x^4-9*x^3+9*x^2+85*x-150)
```

$$x^4 - 9x^3 + 9x^2 + 85x - 150 = (x-5)^2 (x-2) (x+3)$$

```
>$' x^4-9*x^3+9*x^2+85*x-150 = ...
> gfactor(x^4-9*x^3+9*x^2+85*x-150)
```

$$x^4 - 9x^3 + 9x^2 + 85x - 150 = (x-5)^2 (x-2) (x+3)$$

```
>$factor(x^2+4) // tidak difaktorkan
```

$$x^2 + 4$$

```
>$' x^2+4=gfactor(x^2+4) // faktor-faktor kompleks
```

$$x^2 + 4 = (x-2i) (x+2i)$$

```
>$'x^4+64=factor(x^4+64) // faktor-faktor riil
```

$$x^4 + 64 = (x^2 - 4x + 8)(x^2 + 4x + 8)$$

```
>$'x^4+64=gfactor(x^4+64) // faktor-faktor kompleks
```

$$x^4 + 64 = (x - 2i - 2)(x - 2i + 2)(x + 2i - 2)(x + 2i + 2)$$

Perintah `factor` juga dapat digunakan untuk memfaktorkan bilangan asli menjadi faktorisasi prima. Dalam hal ini, perintah `factor` harus digunakan sebagai perintah Maxima secara langsung (modus Maxima) dengan menggunakan tanda `::`. Perhatikan beberapa contoh berikut ini.

```
>$factor(10!)
```

$$3628800$$

```
>::'10!=factor(10!)
```

$$3628800 = 2^8 \cdot 3^4 \cdot 5^2 \cdot 7$$

```
>::'2^15+1=factor(2^15+1)
```

$$32769 = 2^3 \cdot 11 \cdot 331$$

Sebagai kebalikan dari faktorisasi bentuk aljabar adalah menjabarkan bentuk aljabar. Di atas sudah ditunjukkan beberapa contoh menjabarkan bentuk aljabar dengan menggunakan perintah `expand`. Perintah ini juga dapat digunakan untuk menjabarkan bentuk pecahan. Perhatikan contoh-contoh di bawah ini.

```
>$'(x^2+2*x+2)/(x^2-2*x+2)=...
```

```
> expand((x^2+2*x+2)/(x^2-2*x+2))
```

$$\frac{x^2 + 2x + 2}{x^2 - 2x + 2} = \frac{x^2}{x^2 - 2x + 2} + \frac{2x}{x^2 - 2x + 2} + \frac{2}{x^2 - 2x + 2}$$

```
>$rhs(%)=ratsimp(rhs(%))
```

$$\frac{x^2}{x^2-2x+2} + \frac{2x}{x^2-2x+2} + \frac{2}{x^2-2x+2} = \frac{x^2+2x+2}{x^2-2x+2}$$

Untuk merujuk penyebut suatu pecahan dapat digunakan perintah `denom`, sedangkan untuk merujuk pembilang suatu pecahan dapat digunakan perintah `num`, seperti contoh berikut ini.

```
>$pecahan: (x^2+%e^x) / (x^3-%e^(-x))
```

$$\frac{e^x + x^2}{x^3 - e^{-x}}$$

```
>$'pecahan=num(pecahan)/denom(pecahan)
```

$$\frac{e^x + x^2}{x^3 - e^{-x}} = \frac{e^x + x^2}{x^3 - e^{-x}}$$

```
>$ (3*x^(-2)+2*x) * (4*x^2-5*x) = ...
> ratsimp((3*x^(-2)+2*x) * (4*x^2-5*x))
```

$$\left(2x + \frac{3}{x^2}\right) (4x^2 - 5x) = \frac{8x^4 - 10x^3 + 12x - 15}{x}$$

3.4 Fungsi Polinomial dan Rasional

Fungsi polinomial adalah fungsi yang hanya melibatkan pangkat bilangan bulat non-negatif atau hanya eksponen bilangan bulat positif dari suatu variabel. Polinomial (suku banyak) adalah suatu bentuk matematika yang merupakan penjumlahan atau pengurangan dari satu atau lebih suku dengan pangkat variabelnya bilangan bulat non-negatif.

Bentuk umum fungsi polinomial

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \dots + a_2 x^2 + a_1 x + a_0.$$

Fungsi rasional adalah fungsi yang merupakan rasio (pembagian) dua polinomial, dan memiliki bentuk umum

$$h(x) = \frac{P(x)}{Q(x)},$$

dengan $P(x)$ dan $Q(x)$ adalah polinomial dengan $Q(x)$ tidak sama dengan nol.

EMT (bersama Maxima) menyediakan beberapa perintah/fungsi untuk memanipulasi polinomial dan bentuk rasional. Sebagian sudah ditunjukkan pada contoh-contoh sebelumnya. Berikut ini adalah beberapa contoh lainnya.

```
>$3*x^3-4*x^2+2*x+4=factor(3*x^3-4*x^2+2*x+4)
```

$$3x^3 - 4x^2 + 2x + 4 = (3x + 2)(x^2 - 2x + 2)$$

```
>$ (3*x^3-4*x^2+2*x+4) / (3*x+2) = ...
>      factor(3*x^3-4*x^2+2*x+4) / (3*x+2)
```

$$\frac{3x^3 - 4x^2 + 2x + 4}{3x + 2} = x^2 - 2x + 2$$

```
>$ (3*x^3-4*x^2+2*x+4) / (3*x+2) = ...
>      radcan((3*x^3-4*x^2+2*x+4) / (3*x+2))
```

$$\frac{3x^3 - 4x^2 + 2x + 4}{3x + 2} = x^2 - 2x + 2$$

```
>$16*x^4-1=factor(16*x^4-1)
```

$$16x^4 - 1 = (2x - 1)(2x + 1)(4x^2 + 1)$$

```
>$ (16*x^4-1) / (2*x-1)=factor(16*x^4-1) / (2*x-1)
```

$$\frac{16x^4 - 1}{2x - 1} = (2x + 1)(4x^2 + 1)$$

```
>$ (16*x^4-1) / (2*x-1)=radcan((16*x^4-1) / (2*x-1))
```

$$\frac{16x^4 - 1}{2x - 1} = 8x^3 + 4x^2 + 2x + 1$$

Suatu polinomial juga dapat difaktorkan terhadap modulo tertentu. Contoh berikut ini menunjukkan faktorisasi $x^6 + x^2 + 1$ terhadap modulo 13.

```
>$x^6+x^2+1=factor(x^6+x^2+1) | modulus:=13
```

$$x^6 + x^2 + 1 = (x^2 + 6)(x^4 - 6x^2 - 2)$$

Ingat,

$$-12 \equiv 1 \quad \text{dan} \quad -38 \equiv 1 \quad (\text{modulo } 13).$$

```
>$mod(-12,13)
```

1

```
>$mod(-38,13)
```

1

```
>$expand((1+x)^4+(1+x)^2+1/(1+x)^2+1/(1+x)^4,2,2)
```

$$\frac{1}{x^2+2x+1} + (x+1)^4 + \frac{1}{(x+1)^4} + x^2 + 2x + 1$$

```
>$p:=expand((a*x^3+a^2*x+b*x+y)*(a*x+y))|ratsimp
```

$$y^2 + (ax^3 + (b+a^2+a)x)y + a^2x^4 + (ab+a^3)x^2$$

```
>$part(p,2) // suku kedua pada ekspresi di atas
```

$$(ax^3 + (b+a^2+a)x)y$$

```
>$collectterms(p,x) // p sbg polinomial dalam x
```

$$y^2 + (ax^3 + (b+a^2+a)x)y + a^2x^4 + (ab+a^3)x^2$$

```
>$factorout(p,x) // setiap suku sebagai f(x).(y)
```

$$y^2 + x(a(x^2 + a + 1) + b)y + a^2x^4 + (ab + a^3)x^2$$

Perintah `factorout(p, x)` akan menyatakan p sebagai jumlah suku-suku yang masing-masing merupakan hasil kali $f(x)$ dan g , masing-masing difaktorkan dan fungsi g tidak memuat x . Sudah tentu hasil `factorout(p, x)` dalam hal ini akan sama dengan hasil `factorout(p, y)`.

Terdapat beberapa fungsi Maxima yang terkait dengan polinomial, di antaranya adalah `coeff`, `divide`, `quotient`, `remainder`, `gcd`, dan `horner`. Berikut adalah penjelasan dan contoh masing-masing fungsi.

Fungsi `coeff` menghitung koefisien suatu variabel dengan pangkat tertentu pada suatu polinomial. Contoh berikut adalah menghitung koefisien setiap suku pada polinomial

$$3x^6 - 3x^4 + 5x^2 - 3.$$

```
>$makelist(coeff(3*x^6-3*x^4+5*x^2-3, x, i), i, ...
>      [6, 5, 4, 3, 2, 1, 0])
```

$$[3, 0, -3, 0, 5, 0, -3]$$

Jelas bahwa koefisien x mulai dari pangkat tertinggi (=6) pada polinomial tersebut adalah berturut-turut 3, 0, -3, 0, 5, 0, -3.

Fungsi `divide(p, q)` akan menghasilkan polinomial h hasil bagi p dan q serta polinomial sisa pembagiannya, r sedemikian hingga

$$p(x) = h(x)q(x) + r(x).$$

```
>$divide(3*x^6-3*x^4+5*x^2-3, x^2+2*x+1)
```

$$[3x^4 - 6x^3 + 6x^2 - 6x + 11, -(16x) - 14]$$

Jadi,

$$3x^6 - 3x^4 + 5x^2 - 3 = (3x^4 - 6x^3 + 6x^2 - 6x + 11)(x^2 + 2x + 1) - 16x - 14.$$

Berikut kita cek kebenaran hasil tersebut.

```
>$expand(%[1] * (x^2+2*x+1) - 16*x - 14)
```


$$3x^6 - 3x^4 + 5x^2 - 3$$

Hasilnya benar. Ingat, notasi `%[1]` artinya hasil pertama dari perintah `divide` (sebagai perintah terakhir), yakni hasil bagi kedua polinomial pada perintah `divide`.

Fungsi `quotient` menghasilkan polinomial hasil bagi dua polinomial, sedangkan `remainder` memberikan sisa pembagian kedua polinomial. Berikut kita cek pembagian di atas.

```
>$quotient(3*x^6-3*x^4+5*x^2-3, x^2+2*x+1)
```

$$3x^4 - 6x^3 + 6x^2 - 6x + 11$$

```
>$remainder(3*x^6-3*x^4+5*x^2-3, x^2+2*x+1)
```

$$-(16x) - 14$$

Hasilnya cocok dengan yang diperoleh pada perintah `divide`.

Fungsi `gcd` berguna untuk menghitung faktor persekutuan (FPB) terbesar beberapa bilangan (asli), dapat pula digunakan untuk menghitung FPB beberapa polinomial. FPB dua polinomial adalah polinomial dengan derajat tertinggi yang membagi polinomial-polinomial yang dicari FPB-nya. Perhatikan contoh berikut ini. Pertama kita faktorkan dua polinomial, untuk melihat FPB-nya. Selanjutnya kita cari FPB kedua polinomial langsung dengan perintah `gcd`.

```
>$p1 : x^4-4*x^3+5*x^2-2*x
```

$$x^4 - 4x^3 + 5x^2 - 2x$$

```
>$p2 : x^3-6*x^2+11*x-6
```

$$x^3 - 6x^2 + 11x - 6$$

```
>$p1=factor(p1)
```

$$x^4 - 4x^3 + 5x^2 - 2x = (x-2)(x-1)^2 x$$

```
>$p2=factor(p2)
```

$$x^3 - 6x^2 + 11x - 6 = (x - 3)(x - 2)(x - 1)$$

Dari hasil faktorisasi kedua polinomial di atas terlihat bahwa FPB kedua polinomialnya adalah

$$(x - 2)(x - 1) = x^2 - 3x + 2.$$

Berikut kita lihat hasil perintah `gcd` terhadap kedua polinomial $p1$ dan $p2$.

```
>$gcd(p1,p2)
```

$$x^2 - 3x + 2$$

Hasilnya benar.

Fungsi `horner` akan menyatakan suatu polinomial ke dalam bentuk perkalian tersarang. Bentuk perkalian tersarang merupakan bentuk polinomial untuk menghitung nilainya secara efisien. Perhatikan contoh-contoh berikut ini.

```
>$p1=horner(p1)
```

$$x^4 - 4x^3 + 5x^2 - 2x = x(x((x - 4)x + 5) - 2)$$

```
>$p2=horner(p2)
```

$$x^3 - 6x^2 + 11x - 6 = x((x - 6)x + 11) - 6$$

```
>$3*x^6-3*x^4+5*x^2-3=horner(3*x^6-3*x^4+5*x^2-3)
```

$$3x^6 - 3x^4 + 5x^2 - 3 = x^2(x^2(3x^2 - 3) + 5) - 3$$

Dengan menggunakan paket Maxima `functs`, terdapat beberapa fungsi/perintah tambahan terkait polinomial. Beberapa di antaranya adalah `lcm`, `gcddivide`, dan `rempart`.

Fungsi `lcm`, yang berguna untuk mencari kelipatan persekutuan terkecil (KPK) beberapa bilangan (asli), dapat pula digunakan untuk mencari KPK beberapa polinomial. Berikut adalah beberapa contohnya.

```
>&load("functs")
      C:/maxima-5.47.0/share/maxima/5.47.0/share/
simplification/functs.mac
>$p1 : x^3-4*x^2+4*x
```

$$x^3 - 4x^2 + 4x$$

```
>$p2 : x^3+3*x^2-10*x
```

$$x^3 + 3x^2 - 10x$$

```
>$p1=factor(p1)
```

$$x^3 - 4x^2 + 4x = (x - 2)^2 x$$

```
>$p2=factor(p2)
```

$$x^3 + 3x^2 - 10x = (x - 2) x (x + 5)$$

$$>\$1\text{cm}(p1, p2)$$

$$(x-2)^2 x (x+5)$$

Apakah berlaku hubungan antara KPK dan FPB seperti pada bilangan, yakni $KPK(a,b) = a.b / FPB(a,b)$? Dari hasil faktorisasi kedua polinomial di atas terlihat bahwa FPB keduanya adalah $(x-2)x$ dan hasil 1 cm di atas memenuhi hubungan KPK dan FPB tersebut.

Fungsi `gcddivide` menyatakan hasil bagi dua polinomial dalam bentuk rasio yang paling sederhana apabila variabel `takegcd` bernilai `true` (nilai aslinya). Apabila variabel `takegcd` bernilai `false`, fungsi `gcddivide` hanya menghasilkan bentuk pecahan dua polinomial masukannya, tanpa disederhanakan. Perhatikan contoh-contoh berikut ini.

```
>&takegcd
true
>$p1:6*x^3+19*x^2+19*x+6
```

$$6x^3 + 19x^2 + 19x + 6$$

```
>$p2:=6*x^5+13*x^4+12*x^3+13*x^2+6*x
```

$$6x^5 + 13x^4 + 12x^3 + 13x^2 + 6x$$

```
>$p1:=factor(p1)
```

$$6x^3 + 19x^2 + 19x + 6 = (x+1)(2x+3)(3x+2)$$

```
>$p2:=factor(p2)
```

$$6x^5 + 13x^4 + 12x^3 + 13x^2 + 6x = x(2x+3)(3x+2)(x^2+1)$$

```
>$p2/p1:=gcddivide(p2,p1)
```

$$\frac{6x^5 + 13x^4 + 12x^3 + 13x^2 + 6x}{6x^3 + 19x^2 + 19x + 6} = \frac{x^3 + x}{x+1}$$

Jadi, fungsi `gcddivide` sama saja dengan hasil bagi dua polinomial setelah keduanya dibagi dengan FPB keduanya. Berikut kita cek hasil di atas dengan cara lain, yakni menggunakan `gcd`.

```
>$p2/p1=(ratsimp(p2/gcd(p1,p2)))/...  
>      (ratsimp(p1/gcd(p1,p2)))
```

$$\frac{6x^5 + 13x^4 + 12x^3 + 13x^2 + 6x}{6x^3 + 19x^2 + 19x + 6} = \frac{x^3 + x}{x+1}$$

Perhatikan, fungsi/perintah `ratsimp` harus digunakan, karena jika tidak pada pembagian tersebut akan muncul pembagi yang sama (polinomial FPB) pada pembilang dan penyebut, sehingga hasilnya pecahan yang tidak disederhanakan. Sudah tentu cara ini memerlukan perintah yang lebih panjang daripada langsung menggunakan fungsi `gcddivide`.

Hasil fungsi `gcddivide` tersebut juga sama dengan hasil fungsi `radcan`, yang sudah dibahas sebelumnya, dan bahkan `ratsimp` juga, hanya beda cara penulisan saja.

```
>$p2/p1=radcan (p2/p1)
```

$$\frac{6x^5 + 13x^4 + 12x^3 + 13x^2 + 6x}{6x^3 + 19x^2 + 19x + 6} = \frac{x^3 + x}{x + 1}$$

```
>$p2/p1=ratsimp (p2/p1)
```

$$\frac{6x^5 + 13x^4 + 12x^3 + 13x^2 + 6x}{6x^3 + 19x^2 + 19x + 6} = \frac{x^3 + x}{x + 1}$$

Fungsi `rempart (p, n)` akan menghapus/menghilangkan suku berderajat n pada polinomial p . Jika digunakan dengan cara `rempart (p, [m, n])`, maka suku berderajat m sampai n akan dihapus pada polinomial p . Fungsi ini tidak mengubah polinomial p itu sendiri. Perhatikan contoh-contoh di bawah ini.

```
>$rempart (p2, 3)
```

$$6x^5 + 13x^4 + 13x^2 + 6x$$

```
>$rempart (p2, [2, 4])
```

$$6x^5 + 6x$$

```
>$p2
```

$$6x^5 + 13x^4 + 12x^3 + 13x^2 + 6x$$

Untuk operasi terkait pecahan, Maxima menyediakan beberapa fungsi sederhana, di antaranya adalah `partfrac`, `combine`, dan `cfdisrep`.

Fungsi `partfrac` berguna untuk menyajikan suatu pecahan dalam bentuk pecahan parsial (jumlah pecahan-pecahan dengan penyebut yang saling prima) dan konstanta. Berikut adalah beberapa contoh penggunaan fungsi `partfrac`. Fungsi `ratsimp` terhadap pecahan parsial akan mengembalikan ke bentuk pecahan aslinya (dengan satu penyebut).

Pada contoh pertama, kita akan menyatakan pecahan

$$\frac{x - 1}{x^2(x + 1)}$$

sebagai pecahan parsial secara langsung dengan perintah Maxima `partfrac`, kemudian kita bandingkan hasilnya dengan cara manual langkah demi langkah sebagaimana dilakukan di dalam aljabar.

```
>p:(x-1)/(x^2*(x+1));
>$p:=partfrac((x-1)/(x^2*(x+1)),x)
```

$$\frac{x-1}{x^2(x+1)} = -\left(\frac{2}{x+1}\right) + \frac{2}{x} - \frac{1}{x^2}$$

Cara manual adalah dengan memisalkan

$$\frac{x-1}{x^2(x+1)} = \frac{A}{x} + \frac{B}{x^2} + \frac{C}{x+1},$$

kemudian kita cari nilai A, B, C dengan menyamakan pembilang dan penyebut ruas kiri dan kanan.

```
>&remvalue(A,B,C);
>$kanan:ratsimp(A/x+B/x^2+C/(x+1))//jumlah ruas kanan
```

$$\frac{(C+A)x^2 + (B+A)x + B}{x^3 + x^2}$$

```
>$binan: num(%) // pembilang ruas kanan
```

$$(C+A)x^2 + (B+A)x + B$$

```
>$biri:x-1 // pembilang ruas kiri
```

$$x-1$$

Kita selesaikan A, B, C dengan membandingkan pembilang ruas kiri dan pembilang ruas kanan.

```
>$bil:solve([coeff(binan,x,2)=0,...
>      coeff(binan,x,1)=1,coeff(binan,x,0)=-1])
```

$$[[B=-1,C=-2,A=2]]$$

Penyelesaian tersebut kita substitusikan ke pecahan parsial pemisalan.

```
>$p=A/x+B/x^2+C/(x+1) with bil[1]
```

$$\frac{x-1}{x^2(x+1)} = -\left(\frac{2}{x+1}\right) + \frac{2}{x} - \frac{1}{x^2}$$

Hasilnya sama dengan yang diperoleh melalui perintah Maxima `partfrac`.

```
>$p: (x^4-2*x^3+x^2+2)/(x^4-2*x^3-x^2+2*x)
```

$$\frac{x^4 - 2x^3 + x^2 + 2}{x^4 - 2x^3 - x^2 + 2x}$$

```
>$p=partfrac(p,x)
```

$$\frac{x^4 - 2x^3 + x^2 + 2}{x^4 - 2x^3 - x^2 + 2x} = -\left(\frac{1}{x+1}\right) + \frac{1}{x} - \frac{1}{x-1} + \frac{1}{x-2} + 1$$

```
>$rhs(%)=ratsimp(rhs(%))
```

$$-\left(\frac{1}{x+1}\right) + \frac{1}{x} - \frac{1}{x-1} + \frac{1}{x-2} + 1 = \frac{x^4 - 2x^3 + x^2 + 2}{x^4 - 2x^3 - x^2 + 2x}$$

```
>$p:-x/(x^3+4*x^2+5*x+2)
```

$$-\left(\frac{x}{x^3 + 4x^2 + 5x + 2}\right)$$

```
>$p=partfrac(p,x)
```

$$-\left(\frac{x}{x^3 + 4x^2 + 5x + 2}\right) = \frac{2}{x+2} - \frac{2}{x+1} + \frac{1}{(x+1)^2}$$

```
>$rhs(%)=ratsimp(rhs(%))
```

$$\frac{2}{x+2} - \frac{2}{x+1} + \frac{1}{(x+1)^2} = -\left(\frac{x}{x^3+4x^2+5x+2}\right)$$

```
>$p:(2 + x)/(3 + x)/(b + x)
```

$$\frac{x+2}{(x+3)(x+b)}$$

```
>$p=partfrac(p,x)
```

$$\frac{x+2}{(x+3)(x+b)} = \frac{b-2}{(b-3)(x+b)} - \frac{1}{(b-3)(x+3)}$$

```
>$rhs(%)=ratsimp(rhs(%))
```

$$\frac{b-2}{(b-3)(x+b)} - \frac{1}{(b-3)(x+3)} = \frac{x+2}{x^2+(b+3)x+3b}$$

```
>$p:f/(2*b) + 2*c/(3*a) + 3*e/(2*b) +c/(5*b*a)
```

$$\frac{f}{2b} + \frac{3e}{2b} + \frac{c}{5ab} + \frac{2c}{3a}$$

```
>$combine(p)
```

$$\frac{f+3e}{2b} + \frac{c}{5ab} + \frac{2c}{3a}$$

Fungsi `combine` berguna untuk menjumlahkan/menyederhakan suatu ekspresi dengan menggabungkan suku-suku yang memiliki penyebut sama. Dalam beberapa kasus hasil fungsi `combine` sama dengan hasil fungsi `ratsimp`, namun tidak selalu demikian. Berikut adalah beberapa contoh penggunaan fungsi `combine`.

```
>$ratsimp(p)
```


$$\frac{15af + 45ae + (20b + 6)c}{30ab}$$

```
>$p=combine(p)
```

$$\frac{f}{2b} + \frac{3e}{2b} + \frac{c}{5ab} + \frac{2c}{3a} = \frac{f+3e}{2b} + \frac{c}{5ab} + \frac{2c}{3a}$$

```
>$%=ratsimp(p)
```

$$\begin{aligned} \frac{f}{2b} + \frac{3e}{2b} + \frac{c}{5ab} + \frac{2c}{3a} &= \frac{f+3e}{2b} + \frac{c}{5ab} + \frac{2c}{3a} \\ &= \frac{15af + 45ae + (20b + 6)c}{30ab} \end{aligned}$$

```
>$p:1*f/2*b + 2*c/3*a + 3*f/4*b + c/5*b*a
```

$$\frac{5bf}{4} + \frac{abc}{5} + \frac{2ac}{3}$$

```
>$ratsimp(p)
```

$$\frac{75bf + (12ab + 40a)c}{60}$$

```
>$p=combine(p)
```

$$\frac{5bf}{4} + \frac{abc}{5} + \frac{2ac}{3} = \frac{75bf + 4(3abc + 10ac)}{60}$$

```
>$p:3/(x+1)+5/(y-2)-3*x/(x+1)+2*y/(y-2)
```

$$\frac{2y}{y-2} + \frac{5}{y-2} - \frac{3x}{x+1} + \frac{3}{x+1}$$

```
>$ratsimp(p)
```

$$-\left(\frac{(x-5)y-11x+1}{(x+1)y-2x-2}\right)$$

```
>$p=combine(p)
```

$$\frac{2y}{y-2} + \frac{5}{y-2} - \frac{3x}{x+1} + \frac{3}{x+1} = \frac{2y+5}{y-2} + \frac{3-3x}{x+1}$$

```
>$%=factor(num(ratsimp(p)))/factor(denom(ratsimp(p)))
```

$$\frac{2y}{y-2} + \frac{5}{y-2} - \frac{3x}{x+1} + \frac{3}{x+1} = \frac{2y+5}{y-2} + \frac{3-3x}{x+1} = -\left(\frac{xy-5y-11x+1}{(x+1)(y-2)}\right)$$

Fungsi `cfdisrep` akan menghasilkan pecahan berlanjut dengan daftar koefisien yang diberikan. Pecahan berlanjut

$$a + \frac{1}{b + \frac{1}{c + \frac{1}{d + \dots}}}$$

dapat disajikan dengan daftar koefisien $[a, b, c, d, \dots]$. Apabila daftar koefisien tersebut berupa vektor bilangan, perintah Maxima `cfdisrep` harus digunakan pada modus Maxima langsung (dengan simbol `::`). Jika digunakan pada modus tak langsung (dengan simbol `&` atau `$`), hasilnya adalah pecahan biasa. Akan tetapi untuk pecahan berlanjut simbolik, modus tidak langsung dapat digunakan, hasilnya sama. Perhatikan contoh-contoh berikut ini.

```
>::cfdisrep([1,0,1,2,1,3,1,4])
```

$$\begin{array}{r}
 1 \\
 1 + \frac{1}{\frac{0 + \frac{1}{1 + \frac{2}{1 + \frac{3}{1 + \frac{4}{1}}}}}}
 \end{array}$$

```
>$cfdisrep([1,0,1,2,1,3,1,4])
```

$$\frac{158}{67}$$

```
>::cfdisrep(cf(158/67))
```

$$\begin{array}{r}
 1 \\
 2 + \frac{1}{\frac{2 + \frac{1}{1 + \frac{3}{1 + \frac{4}{1}}}}}}
 \end{array}$$

Mungkin Anda melihat kedua pecahan berlanjut berbeda, namun jika Anda perhatikan, kedua adalah sama!

```
>::cfdisrep([a,b,c,d])
```

$$a + \frac{1}{b + \frac{1}{c + \frac{1}{d}}}$$

```
>&powerdisp:true;
```

```
>$cfdisrep([a,b,c,d])
```

$$a + \frac{1}{b + \frac{1}{c + \frac{1}{d}}}$$

```
>$P:radcan(%)
```

$$\frac{1 + ab + (a + (1 + ab) c) d}{b + (1 + bc) d}$$

```
>$cf(P)
```

```
cf: continued fractions must be lists or integers;
found b + (1 + b c) d
-- an error. To debug this try: debugmode(true);
```

Fungsi Maxima `cf` tidak dapat digunakan secara simbolik.

Topik tentang pecahan berlanjut merupakan topik matematika yang sangat menarik dan banyak buku referensi yang membahas tentang pecahan berlanjut dan sifat-sifatnya.

3.5 Fungsi Komposisi dan Fungsi Invers

Fungsi komposisi adalah fungsi yang melibatkan lebih dari satu fungsi. Apabila nilai suatu fungsi disubstitusikan ke fungsi lainnya, maka hasilnya merupakan nilai fungsi baru. Fungsi baru ini merupakan komposisi dari kedua fungsi sebelumnya.

Notasi:

$$(f \circ g)(x) \equiv f(g(x)).$$

$$(g \circ f)(x) \equiv g(f(x)).$$

```
>//contoh
>function f(x) &= sqrt(2*x + 5); $'f(x)=f(x)
```

$$f(x) = \sqrt{2x+5}$$

```
>function g(x) &= x^2+1; $'g(x)=g(x)
```

$$g(x) = x^2 + 1$$

```
>f(g(1))
3
>g(f(1))
8
>$f(a)
```

$$\sqrt{2a+5}$$

```
>$g(a)
```

$$a^2 + 1$$

```
>$'f('g(a))=f(g(a))
```

$$f(g(a)) = \sqrt{2(a^2+1)+5}$$

```
>$'g('f(a))=g(f(a))
```

$$g(f(a)) = 2a + 6$$

Fungsi invers atau fungsi kebalikan merupakan suatu fungsi yang merupakan kebalikan dari fungsi asalnya. Suatu fungsi f memiliki fungsi invers (kebalikan) f^{-1} jika f merupakan fungsi satu-satu dan fungsi pada (bijektif). Hubungan tersebut dapat dinyatakan sebagai berikut:

$$(f^{-1})^{-1} = f$$

Lakukan 3 langkah berikut ini untuk menentukan fungsi invers, yaitu:

1. Ubahlah bentuk $y = f(x)$ menjadi bentuk $x = g(y)$,
2. Tuliskan x sebagai $f^{-1}(y)$ sehingga $f^{-1}(y) = g(y)$, dan
3. Ubahlah variabel y dengan x sehingga diperoleh rumus fungsi invers $f^{-1}(x) = g(x)$.

Berikut adalah contoh mendefinisikan fungsi EMT untuk mencari invers (domain) dari suatu fungsi jika diketahui nilai-nilai fungsi (*range*).

```
>// mencari nilai  $x=f^{-1}(y)$ , artinya  $y=f(x)$ .
>function map invf(y;f) := solve(f,y,y=y)
```

Dengan definisi fungsi invers tersebut, berikut dicontohkan mencari invers, yakni nilai $x = f^{-1}(y)$ sedemikian hingga $y = f(x)$. Penggunaan kata `map` di belakang kata `function` membuat fungsinya dapat menerima vektor/matriks sebagai masukan (input).

```
>function f(x) &= x^x; $'f(x)=f(x)
```

$$f(x) = x^x$$

```
>y=1:5; x:=invf(y,"f")
[1, 1.55961, 1.82546, 2, 2.12937]
>f(x) // kita cek apakah benar  $y=f(x)=x^x$ 
[1, 2, 3, 4, 5]
```

Hasilnya benar! Berikut adalah contoh-contoh lain menggunakan fungsi `invf` di atas untuk mencari invers (domain) dari suatu fungsi yang diketahui nilai fungsinya (*range*). Untuk fungsi bijektif hasilnya sesuai yang kita harapkan, namun jika fungsi f tidak bijektif mungkin hasilnya tidak sesuai dengan harapan, meskipun tidak salah.

```
>function f(x):=2*x^2-3*x
>x=1:5; x1:=invf(f(x),"f")
[0.5, 2, 3, 4, 5]
```

Hasilnya tidak sama dengan nilai-nilai x semula, namun itu tidak salah. Perhatikan nilai fungsi pada nilai-nilai x semula dan nilai fungsi pada nilai-nilai x hasil invers.

```
>f(x1), f(x)
    [-1,  2,  9, 20, 35]
    [-1,  2,  9, 20, 35]
>function f(x):=3*x+1
>x=1:5; x1:=invf(f(x),"f")
    [1,  2,  3,  4,  5]
```

Di sini, karena fungsinya bijektif, nilai inversnya sama dengan nilai x semula.

Berikut ditunjukkan bagaimana cara mencari invers fungsi secara simbolik dengan Maxima. Kita akan mencari invers fungsi

$$y = f(x).$$

Pertama kita selesaikan persamaan $y = f(x)$ untuk x . Mungkin solusinya tidak tunggal, sehingga kita akan pilih salah satu solusi untuk mendapatkan inversnya. Untuk contoh kasus kita pilih

$$f(x) = x^2 - 3x.$$

```
>function f(x) &=x^2-3*x; $'f(x)=f(x)
```

$$f(x) = x^2 - 3x$$

```
>$xx:solve(y=f(x),x)
```

$$\left[x = -\left(\frac{\sqrt{4y+9}-3}{2}\right), x = \frac{\sqrt{4y+9}+3}{2} \right]$$

Langkah kedua kita definisikan fungsi invers f dari solusi di atas. Dalam hal ini kita pilih solusi yang positif, sebagai

$$f^{-1}(x) = g(x),$$

dengan mensubstitusikan $y = x$ ke dalam $g(y)$ pada rumus solusi yang kita pilih.

```
>function g(x) &= rhs(xx[2]) with y=x; $'g(x)=g(x)
```

$$g(x) = \frac{\sqrt{4x+9}+3}{2}$$

Jadi, invers f adalah

$$f^{-1}(x) = g(x) = \frac{\sqrt{4x+9}+3}{2}.$$

Kita bandingkan hasil tersebut dengan cara manual mencari invers fungsi.

Diketahui

$$\begin{aligned} f(x) &= x^2 - 3x \\ &= \left(x - \frac{3}{2}\right)^2 - \frac{9}{4} \\ &= \frac{(2x-3)^2 - 9}{4}. \end{aligned}$$

Jadi nilai $f(x)$ diperoleh dari nilai x dengan cara, secara berturut-turut:

1. dikalikan 2,
2. dikurangi 3,
3. dikuadratkan,
4. dikurangi 9, dan
5. dibagi 4.

Dengan demikian, invers f , yakni $f^{-1}(x)$ diperoleh dari x dengan melakukan operasi kebalikan dari langkah-langkah di atas (dari langkah terakhir), yakni:

1. dikalikan 4,
2. ditambah 9,
3. ditarik akarnya,
4. ditambah 3, dan
5. dibagi 2.

Hasil dari langkah-langkah ini cocok dengan rumus $g(x)$ yang diperoleh di atas. Berikut kita cek apakah $g(f(x)) = f(g(x)) = x$.

>\$g (f (x)) | radcan

x

>\$f (g (x)) | radcan

x

Benar bahwa $g(f(x)) = f(g(x)) = x$.

3.6 Fungsi Eksponensial

Fungsi eksponensial adalah fungsi yang memiliki bentuk

$$f(x) = a^x$$

dengan $a > 0$.

Berikut adalah sifat-sifat fungsi eksponen.

1. Pangkat negatif

$$a^{-x} = \frac{1}{a^x}$$

untuk a tidak sama dengan 0 dan $x > 0$.

$$>\$2^{-5}$$

$$\frac{1}{32}$$

Jadi

$$3^{-5} = \frac{1}{32}.$$

2. Pangkat nol

$$a^0 = 1.$$

$$>\$1^0$$

$$1$$

$$>\$10^0$$

$$1$$

$$>\$(-3/7)^0$$

$$1$$

$$>\$d^0$$

$$1$$

$$x^0 = 1$$

Jadi,

$$x^0 = 1$$

berlaku untuk semua nilai x bilangan riil.

3. Hasil kali perpangkatan dengan bilangan pokok sama Jika p dan q adalah bilangan real, maka berlaku sifat-sifat berikut:

$$a^p \times a^q = a^{p+q}.$$

$$a^5 \cdot a^{-2}$$

$$a^3$$

$$a^5 \cdot a^{-2} = a^3.$$

$$a^3 \cdot a^5$$

$$a^8$$

$$a^3 \cdot a^5 = a^8.$$

$$a^p \cdot a^q$$

$$a^{p+q}$$

$$a^p \cdot a^q = a^{p+q}.$$

$$3^8 \cdot 3^{-2}$$

$$729$$

$$3^8 \cdot 3^{-2} = 3^6 = 729.$$

$$(x+2)^4 \cdot (x+2)^{-2}$$

$$(x+2)^2$$

$$(x+2)^4 \cdot (x+2)^{-2} = (x+2)^2.$$

```
>$a^p*a^-p
```

$$a^p \cdot a^{-p} = a^0 = 1.$$

4. Perpangkatan bentuk pangkat

$$(a^p)^q = a^{pq}.$$

```
>$ (x^2)^5
```

$$x^{10}$$

```
>$ (5^2)^3
```

$$15625$$

```
>$ (a^x)^y | radexpand:all
```

$$a^{xy}$$

Penjelasan: Opsi (tambahan perintah) `radexpand:all` menyebabkan semua bentuk perpangkatan dan akar akan dijabarkan oleh EMT (Maxima).

5. Pembagian bentuk pangkat

Untuk a tidak sama dengan 0, berlaku:

$$\frac{a^p}{a^q} = a^{p-q}.$$

```
>$a^x/a^y
```

$$a^{x-y}$$

```
>$ (x^2*y^-2) / (x^-1*y)
```

$$\frac{x^3}{y^3}$$

Jadi,

$$\frac{x^2 y^{-2}}{x^{-1} y} = \frac{x^3}{y^3}.$$

6. Perpangkatan hasil kali

$$(ab)^p = a^p \times b^p.$$

```
>$ (2*x*3*y) ^3
```

$$216x^3y^3$$

Artinya,

$$(2x.3y)^3 = (2x)^3.(3y)^3 = 216x^3y^3.$$

7. Perpangkatan hasil bagi Dengan b tidak sama dengan 0, berlaku:

$$\left(\frac{a}{b}\right)^p = \frac{a^p}{b^p}.$$

```
>$ ((a^2*b^-3*c^5) / (a^2*b^-2*c^3)) ^-2
```

$$\frac{b^2}{c^4}$$

Jadi,

$$\left(\frac{a^2b^{-3}c^5}{a^2b^{-2}c^3}\right)^{-2} = \frac{b^2}{c^4}.$$

```
>&powerdisp:false;
>$ (E^x-1) / (E^(x/2)+1)=radcan((E^x-1) / (E^(x/2)+1))
```

$$\frac{e^x - 1}{e^{\frac{x}{2}} + 1} = e^{\frac{x}{2}} - 1$$

3.7 Fungsi Logaritma

Logaritma merupakan kebalikan (invers) pemangkatan, artinya logaritma adalah fungsi untuk mencari pangkat apabila suatu bilangan positif dinyatakan sebagai perpangkatan dengan bilangan pokok (basis) tertentu. Suatu bentuk perpangkatan dapat diubah menjadi bentuk logaritma dan sebaliknya.

Bentuk Umum: Untuk $0 < a < 1$ atau $a > 1$, dan $b > 0$ berlaku:

$$a^n = b \iff \log_a b = n.$$

Keterangan:

a disebut bilangan pokok (basis),

b disebut numerus (hasil perpangkatan), dan

n disebut pangkat.

EMT menyediakan fungsi/perintah numerik `logbase()` untuk menghitung logaritma dengan basis tertentu, dan perintah `log10()` untuk menghitung logaritma dengan basis 10. Cara mencari nilai logaritma umum dengan menggunakan EMT adalah dengan menuliskan sintaks `logbase(numerus, basis)`. Fungsi ini tidak dapat digunakan secara simbolik.

Untuk logaritma simbolik dengan Maxima, fungsi yang digunakan adalah `log()` yang merupakan fungsi logaritma alami dengan basis e . Untuk perhitungan logaritma dengan basis selain e secara simbolik dapat digunakan definisi:

$$\text{logbase}(a, b) = \log(b) / \log(a).$$

```
>logbase(16, 2)
4
```

Artinya,

$$16 = 2^4 \iff \log_2 16 = 4.$$

```
>logbase(81, 3)
4
```

$$3^4 = 81 \iff \log_3 81 = 4.$$

Sifat-sifat logaritma Untuk a, b, c bilangan riil positif berlaku sifat-sifat sebagai berikut.

1. Untuk a tidak sama dengan 0:

$$\log_a bc = \log_a b + \log_a c.$$

Contoh: Apakah

$$\log_2 4 + \log_2 8 = \log_2(4 \times 8)?$$

Jawab:

```
>logbase(4, 2) + logbase(8, 2)
5
>logbase(8*4, 2)
5
```

Hasil ruas kiri dan ruas kanan sama, yakni 5. Jadi kedua ruas bernilai sama. Berikut ditunjukkan sifat-sifat logaritma secara simbolik dengan menggunakan Maxima di EMT.

```
>&powerdisp:true;
>&logexpand:all; $log(a^x*b^y)
```

$$\log ax + \log by$$

Variabel `logexpand` menentukan hasil penyederhanaan bentuk logaritma. Jika `logexpand:true` (ini adalah nilai aslinya), $\log(a^b)$ menghasilkan $b \cdot \log(a)$. Jika `logexpand:all`, $\log(a \cdot b)$ akan disederhanakan menjadi $\log(a) + \log(b)$. Jika `logexpand:super`, maka $\log(a/b)$ akan disederhanakan menjadi $\log(a) - \log(b)$ untuk bilangan rasional a/b , ($\log(1/b)$, untuk bilangan bulat b , selalu disederhanakan.) Jika `logexpand:false`, semua fungsi logaritma tidak disederhanakan. Ketika nilai `logexpand` adalah `all` atau `super`, logaritma hasil kali disederhanakan menjadi jumlah logaritma.

```
>$log(x*y)
```

$$\log x + \log y$$

```
>$log(x/y)
```

$$\log x - \log y$$

Invers `logexpand` adalah `logcontract`, yakni menyederhanakan bentuk

$$\log(x) + \log(y)$$

menjadi

$$\log(xy).$$

```
>$logcontract(log(x)+log(y))
```

$$\log x + \log y$$

Seharusnya hasilnya adalah $\log(xy)$. Dalam hal ini harus digunakan modus Maxima langsung dengan menggunakan tanda dua titik dua (`::`). Sayangnya dalam modus maxima langsung tampilan dengan LaTeX tidak dapat digunakan.

```

>:: log(x)+log(y)=logcontract(log(x)+log(y))...
                                log(x) + log(y) = log(x y)
>:: 2*(a*log(x) + 2*a*log(y))= ...
                                logcontract(2*(a*log(x) + 2*a*log(y)))
                                                2    4
                                2 (a log(x) + 2 a log(y)) = a log(x y)

```

2. Untuk $a > 0$, a tidak sama dengan 1, dan $b > 0$, berlaku

$$\log_a \left(\frac{b}{c} \right) = \log_a b - \log_a c.$$

Contoh: Apakah

$$\log_3 27 - \log_3 3 = \log_3 \left(\frac{27}{3} \right)?$$

Jawab:

```

>logbase(27,3)-logbase(3,3)
2
>logbase(27/3,3)
2

```

Hasil kedua ruas sama, artinya nilai ruas kiri sama dengan ruas kanan.

```

>&powerdisp:false
                                false
>&logexpand:all; $log(a^x/b^y)

```

$$\log ax - \log by$$

Itu adalah

$$x.\log a - y.\log b$$

bukan

$$\log(ax) - \log(by).$$

Berikut adalah kebalikannya.

```

>::logcontract(log(x)-log(y))
                                x
                                log(-)
                                y
>::logcontract(log(a^x)-log(b^y))
                                log(a) x - log(b) y

```

3. Untuk n bilangan asli, $a > 0$, a tidak sama dengan 1, dan $b > 0$, berlaku:

$$\log_a b^n = n \log_a b.$$

Contoh: Tunjukkan menggunakan EMT, bahwa pernyataan di bawah benar!

$$\log_2 3^5 = 5 \times \log_2 3.$$

Jawab:

```
>logbase(3^5, 2)
7.92481250361
>logbase(3, 2)*5
7.92481250361
```

Benar.

```
>&logexpand:all; $log(a^x)
```

$$\log_a x$$

4. Untuk a , b , c tidak sama dengan 1, berlaku:

$$\log_a b = \frac{\log_c b}{\log_c a} = \frac{1}{\log_b a}.$$

Contoh: Apakah

$$\log_3 7 = \frac{\log_7 7}{\log_7 3} = \frac{1}{\log_7 3}?$$

Jawab:

```
>logbase(7, 3)
1.77124374916
>logbase(7, 7)/logbase(3, 7)
1.77124374916
>1/logbase(3, 7)
1.77124374916
```

Terbukti bahwa

$$\log_3 7 = \frac{\log_7 7}{\log_7 3} = \frac{1}{\log_7 3}.$$

5. Untuk a , b , c tidak sama dengan 1, berlaku:

$$\log_a b \times \log_b c = \log_a c.$$

Contoh: Apakah

$$\log_2 3 \times \log_3 32 = \log_2 32?$$

Tunjukkan bahwa pernyataan di atas benar!

Jawab:

```
>logbase(3,2)*logbase(32,3)
5
>logbase(32,2)
5
```

Benar.

6. Untuk a tidak sama dengan 1, berlaku:

$$a^{\log_a b} = b.$$

Contoh: Tunjukkan bahwa pernyataan berikut benar

$$2^{\log_2 4} = 4.$$

Jawab:

```
>2^logbase(4,2)
4
```

Benar.

```
>&logexpand:all; $E^log(x)
```

x

Itu artinya,

$$e^{\ln x} = x.$$

Bentuk umum persamaan logaritma adalah

$$\log_a f(x) = \log_a g(x)$$

dengan a adalah basis (bilangan pokok), $f(x)$ dan $g(x)$ adalah numerus dalam bentuk fungsi. Agar suatu persamaan logaritma dapat diselesaikan nilai numerus harus lebih besar dari nol. Artinya, untuk mencari solusi suatu persamaan logaritma syarat ini harus digunakan.

Contoh 1: Tentukan nilai x yang memenuhi persamaan

$$\log_3(2x^2 - x) = 1.$$

Penyelesaian:

$$\log_3(2x^2 - x) = 1 = \log_3 3,$$

sehingga solusi persamaan logaritma di atas dapat diperoleh dengan menyelesaikan persamaan

$$2x^2 - x = 3.$$

```
>$solve(2*x^2-x=3,x)
```

$$\left[x = \frac{3}{2}, x = -1 \right]$$

```
>$log(2*x^2-x)/log(3) with x=3/2//cek kebenaran jawaban
```

1

```
>$log(2*x^2-x)/log(3) with x=-1
```

1

Jadi, solusi persamaan logaritma

$$\log_3(2x^2 - x) = 1$$

adalah

$$x = \frac{3}{2} \text{ dan } x = -1.$$

Hasil tersebut sama jika persamaannya diselesaikan secara langsung dengan perintah Maxima `solve`.

```
>$solve(log(2*x^2-x)/log(3)=1,x)
```

$$\left[x = \frac{3}{2}, x = -1 \right]$$

Contoh 2: Tentukan nilai x yang memenuhi persamaan

$$\log_2(2x^2 - 6x - 7) = \log_3(2x^2 - 6x - 7).$$

Penyelesaian:

```
>$solve(log(2*x^2-6*x-7)/log(2)=...
> log(2*x^2-6*x-7)/log(3),x)
```

$$[x = 4, x = -1]$$

```
>$log(2*x^2-6*x-7)/log(2) with x=4|x=-1
```

$$0$$

```
>$log(2*x^2-6*x-7)/log(3) with x=4|x=-1
```

$$0$$

Contoh 3: Tentukan nilai x yang memenuhi persamaan

$$\log_5(2x^2 + 5x - 10) = \log_5(x^2 - 2x + 18).$$

Penyelesaian:

```
>$solve(log(2*x^2+5*x-10)=log(x^2-2*x+18), x)
```

$$[\log(x^2 - 2x + 18) = \log(2x^2 + 5x - 10)]$$

Maxima gagal menyelesaikan persamaan logaritma tersebut. Ingat penyelesaian persamaan logaritma di atas sebenarnya dapat diperoleh dengan menyelesaikan persamaan

$$2x^2 + 5x - 10 = x^2 - 2x + 18$$

asalkan keduanya positif.

```
>$solx:solve(2*x^2+5*x-10=x^2-2*x+18, x)
```

$$\left[x = -\left(\frac{\sqrt{161} + 7}{2}\right), x = \frac{\sqrt{161} - 7}{2} \right]$$

Apabila solusi tersebut disubstitusikan ke persamaan logaritma aslinya, maka akan diperoleh nilai ruas kiri sama dengan nilai ruas kanan, misalnya seperti hasil sebagai berikut.

```
>$log(x^2-2*x+18) with solx[1]|ratsimp
```

$$\log\left(\frac{9\sqrt{161} + 155}{2}\right)$$

```
>$log(2*x^2+5*x-10) with solx[1]|ratsimp
```

$$\log\left(\frac{9\sqrt{161}+155}{2}\right)$$

Contoh 4: Tentukan nilai x yang memenuhi persamaan

$$\log_{x^2-1}(2x^2-2x+20) = \log_{x^2-1}(x^2+6x+5).$$

Penyelesaian:

```
>$solve(log(2*x^2-2*x+20)/log(x^2-1)=...
>      log(x^2+6*x+5)/log(x^2-1), x)
```

$$[\log(2x^2-2x+20) = \log(x^2+6x+5)]$$

```
>$solve(2*x^2-2*x+20=x^2+6*x+5, x)
```

$$[x = 3, x = 5]$$

Hasilnya sama seperti pada contoh sebelumnya, sehingga persamaannya kita selesaikan dengan menyamakan hasil perpangkatannya. Selanjutnya, hasilnya kita substitusikan ke persamaan logaritma aslinya untuk memeriksa kebenarannya.

```
>$log(2*x^2-2*x+20)/log(x^2-1) with x=3|radcan
```

$$\frac{5}{3}$$

```
>$log(x^2+6*x+5)/log(x^2-1) with x=3|radcan
```

$$\frac{5}{3}$$

Contoh 5: Tentukan nilai x yang memenuhi persamaan

$$\log_3^2 x - 7 \log_3 x + 12 = 0.$$

Penyelesaian:

```
>$solve((log(x)/log(3))^2-7*log(x)/log(3)+12, x)
```

$$[x = 27, x = 81]$$

```
>$ (log(x)/log(3)) ^2-7*log(x)/log(3)+12 with x=27|radcan
```

0

```
>$ (log(x)/log(3)) ^2-7*log(x)/log(3)+12 with x=81|radcan
```

0

Berikut ditunjukkan beberapa contoh menyederhanakan bentuk pecahan logaritma dengan menggunakan perintah/fungsi Maxima `radcan()`.

```
>$ (log(1+2*a^x+a^(2*x))/log(1+a^x))=...
> radcan((log(1+2*a^x+a^(2*x))/log(1+a^x)))
```

$$\frac{\log(1+2a^x+a^{2x})}{\log(1+a^x)} = 2$$

```
>&powerdisp:false;
>$ (log(x^2+x)-log(x))^a/log(x+1)^(a/2)=...
> radcan((log(x^2+x)-log(x))^a/log(x+1)^(a/2))
```

$$\frac{(\log(x^2+x) - \log x)^a}{(\log(x+1))^{\frac{a}{2}}} = (\log(x+1))^{\frac{a}{2}}$$

```
>$ (log(sqrt(6))/log(3))/((log(18)/log(3))^2)...
>-(log(2)/log(3))^2; $'%=radcan(%)
```

$$\frac{\log 6}{2 \log 3 \left(\frac{(\log 18)^2}{(\log 3)^2} - \frac{(\log 2)^2}{(\log 3)^2} \right)} = \frac{1}{8}$$

$$\frac{{}^3\log \sqrt{6}}{({}^3\log 18)^2 - ({}^3\log 2)^2} = \frac{1}{8}.$$

```
>$ (log(9)/log(27)+log(3)/log(2)*log(4)/log(sqrt(3)))/...
> (log(2)/log(3)-log(18)/log(3)); $'%=radcan(%)
```

$$\frac{\frac{\log 9}{\log 27} + \frac{2 \log 4}{\log 2}}{\frac{\log 2}{\log 3} - \frac{\log 18}{\log 3}} = -\left(\frac{7}{3}\right)$$

$$\frac{{}^{27}\log 9 + {}^2\log 3 \cdot {}^3\log 4}{{}^3\log 2 - {}^3\log 18} = -\frac{7}{3}.$$

```
>$' (log(x+x^2)-log(x))^a/log(1+x)^(a/2)=...
> radcan((log(x+x^2)-log(x))^a/log(1+x)^(a/2))
```

$$\frac{(\log(x^2+x) - \log x)^a}{(\log(x+1))^{\frac{a}{2}}} = (\log(x+1))^{\frac{a}{2}}$$

```
>$' (log(1+2*a^x+a^(2*x))/log(1+a^x))=...
>radcan((log(1+2*a^x+a^(2*x))/log(1+a^x)))
```

$$\frac{\log(a^{2x} + 2a^x + 1)}{\log(a^x + 1)} = 2$$

```
>$' (log(x+x^2)-log(x))^n/(log(1+x))^(n/2)=...
> radcan((log(x+x^2)-log(x))^n/(log(1+x))^(n/2))
```

$$\frac{(\log(x^2+x) - \log x)^n}{(\log(x+1))^{\frac{n}{2}}} = (\log(x+1))^{\frac{n}{2}}$$

3.8 Operasi Fungsi Trigonometri dan Hiperbolik

Selain fungsi-fungsi yang melibatkan operasi penjumlahan, pengurangan, perkalian, pembagian, dan operasi radikal (penarikan akar), di dalam matematika juga terdapat fungsi-fungsi lain seperti fungsi eksponensial, fungsi logaritma, dan fungsi trigonometri.

Selain fungsi-fungsi tersebut, EMT (bersama Maxima) menyediakan beraneka ragam fungsi lain yang dikenal di dalam matematika, termasuk fungsi trigonometri.

Fungsi *sinus*, *kosinus*, dan *tangen* adalah tiga fungsi utama trigonometri. Pasangan ketiganya *kosekan*, *sekan*, dan *kotangen*.

$$\csc(x) = \frac{1}{\sin(x)}, \quad \sec(x) = \frac{1}{\cos(x)}, \quad \cot(x) = \frac{1}{\tan(x)}.$$

Selain keenam fungsi dasar trigonometri tersebut EMT (Maxima) juga menyediakan fungsi-fungsi trigonometri yang lain, yakni invers fungsi trigonometri (*asin*, *acos*, *atan*, *acsc*, *asec*, *acot*), fungsi hiperbolik (*sinh*, *cosh*, *tanh*, *csch*, *sech*, *coth*), dan invers fungsi hiperbolik (*asinh*, *acosh*, *atanh*, *acsch*, *asech*, *acoth*).

Fungsi trigonometri di dalam EMT memerlukan masukan dalam bentuk derajat (°) maupun radian (bilangan riil).

```
>x := 60°
      1.0471975512
```

Perintah di atas mendefinisikan nilai x adalah 60° secara numerik, sehingga akan ditampilkan sebagai nilai radian.

```
>sin(x) // nilai sin(60°)
      0.866025403784
>$sin(x)
```

$$\sin x$$

Seperti yang dijelaskan sebelumnya bahwa x telah didefinisikan 60° secara numerik, sehingga tidak dapat dihitung secara eksak (simbolik). Jadi, $\sin(x)$ akan dihitung nilai dari $\sin(60)$ secara numerik, bukan simbolik. Berikut adalah contoh-contoh perhitungan nilai fungsi trigonometri secara simbolik (eksak).

```
>$sin(60°)
```

$$\frac{\sqrt{3}}{2}$$

```
>$x=60°
```

$$x = \frac{\pi}{3}$$

Ini tidak menyimpan nilai 60° ke dalam x , melainkan mendefinisikan suatu persamaan $x = 60$.

>\$y:45^\circ

$$\frac{\pi}{4}$$

Di sini, nilai y didefinisikan sama dengan 45° dan secara eksak ditampilkan seperti yang ditunjukkan.

>\$cos (x)

$$\cos x$$

Di sini, secara simbolik nilai x belum terdefinisi.

>\$tan (y)

$$1$$

Perhatikan perbedaan tanda sama dengan ($=$) dan titik dua ($:$) di dalam Maxima. Tanda sama dengan digunakan bukan untuk menyimpan suatu nilai, melainkan untuk membentuk suatu persamaan, sedangkan tanda titik dua digunakan untuk menyimpan suatu nilai. Untuk mendapatkan nilai di ruas kanan suatu persamaan dapat digunakan fungsi `rhs`, sedangkan untuk mendapatkan nilai di ruas kiri suatu persamaan dapat digunakan fungsi `lhs`.

>\$cos (rhs (x=60^\circ))

$$\frac{1}{2}$$

$$\cos(60^\circ) = \frac{1}{2}.$$

>\$cot (y)

$$1$$

$$\cot(45^\circ) = 1.$$

```
>$sec(y)
```

$$\sqrt{2}$$

$$\sec(45^\circ) = \sqrt{2}.$$

```
>$csc(y)
```

$$\sqrt{2}$$

$$\csc(45^\circ) = \sqrt{2}.$$

```
>$sin(y)
```

$$\frac{1}{\sqrt{2}}$$

```
>$tan(y)
```

$$1$$

$$\tan(45^\circ) = 1.$$

```
>$cos(pi/2)
```

$$0$$

$$\cos(\pi/2) = 0.$$

```
>$asin(1/2)
```

$$\frac{\pi}{6}$$

$$\sin^{-1}\left(\frac{1}{2}\right) = \frac{\pi}{6}.$$

```
>$acos (1/2)
```

$$\frac{\pi}{3}$$

$$\cos^{-1}\left(\frac{1}{2}\right) = \frac{\pi}{3}.$$

```
>$atan (1/sqrt (3) )
```

$$\frac{\pi}{6}$$

$$\tan^{-1}\left(\frac{1}{\sqrt{3}}\right) = \frac{\pi}{6}.$$

Fungsi Hiperbolik Perintah (fungsi) `exponentialize()` dapat digunakan untuk mengubah fungsi hiperbolik menjadi fungsi eksponen sesuai definisinya.

```
>$'sinh(x)=exponentialize(sinh(x))
```

$$\sinh x = \frac{e^x - e^{-x}}{2}$$

```
>$'cosh(x)=exponentialize(cosh(x))
```

$$\cosh x = \frac{e^x + e^{-x}}{2}$$

```
>$'tanh(x)=exponentialize(tanh(x))
```

$$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

```
>$'(1+x)^2*sin(x)=exponentialize((1+x)^2*sin(x))
```

$$(x+1)^2 \sin x = - \left(\frac{i(x+1)^2 (e^{ix} - e^{-ix})}{2} \right)$$

Sifat-sifat dan identitas trigonometri dapat ditunjukkan dengan EMT secara simbolik menggunakan Maxima. Dalam hal ini kita dapat menggunakan perintah/ fungsi `trigsimp` untuk menyederhanakan fungsi trigonometri dan `trigexpand` untuk menjabarkan fungsi trigonometri.

```
>&kill(x,y); &powerdisp:=true;
>$'sin(x)^2+cos(x)^2=trigsimp(sin(x)^2+cos(x)^2)
```

$$\cos^2 x + \sin^2 x = 1$$

```
>$'cosh(x)^2-sinh(x)^2=trigsimp(cosh(x)^2-sinh(x)^2)
```

$$\cosh^2 x - \sinh^2 x = 1$$

```
>$'sin(x+y)=trigexpand(sin(x+y))
```

$$\sin(x+y) = \sin x \cos y + \cos x \sin y$$

```
>$'sinh(x+y)=trigexpand(sinh(x+y))
```

$$\sinh(x+y) = \sinh x \cosh y + \cosh x \sinh y$$

```
>$'sin(x-y)=trigexpand(sin(x-y))
```

$$-\sin(-x+y) = \sin x \cos y - \cos x \sin y$$

```
>$'cos(x+y)=trigexpand(cos(x+y))
```

$$\cos(x+y) = \cos x \cos y - \sin x \sin y$$

```
>$'cosh(x+y)=trigexpand(cosh(x+y))
```

$$\cosh(x+y) = \cosh x \cosh y + \sinh x \sinh y$$

```
>$'cos(x-y)=trigexpand(cos(x-y))
```

$$\cos(-x+y) = \cos x \cos y + \sin x \sin y$$

```
>$'tan(x+y)=trigexpand(tan(x+y))
```

$$\tan(x+y) = \frac{\tan x + \tan y}{1 - \tan x \tan y}$$

```
>$'tanh(x+y)=trigexpand(tanh(x+y))
```

$$\tanh(x+y) = \frac{\tanh x + \tanh y}{1 + \tanh x \tanh y}$$

```
>$'tan(x-y)=trigexpand(tan(x-y))
```

$$-\tan(-x+y) = -\left(\frac{-\tan x + \tan y}{1 + \tan x \tan y}\right)$$

```
>&powerdisp:=false;
>$'sin(2*x)=trigexpand(sin(2*x))
```

$$\sin(2x) = 2 \cos x \sin x$$

```
>$'sinh(2*x)=trigexpand(sinh(2*x))
```

$$\sinh(2x) = 2 \cosh x \sinh x$$

```
>$'cos(2*x)=trigexpand(cos(2*x))
```

$$\cos(2x) = \cos^2 x - \sin^2 x$$

```
>$trigsimp(cos(x)^2-sin(x)^2)
```

$$2 \cos^2 x - 1$$

```
>$'cosh(2*x)=trigexpand(cosh(2*x))
```

$$\cosh(2x) = \sinh^2 x + \cosh^2 x$$

```
>$trigsimp(cosh(x)^2+sinh(x)^2)
```

$$2 \cosh^2 x - 1$$

```
>$'tan(2*x)=trigexpand(tan(2*x))
```

$$\tan(2x) = \frac{2 \tan x}{1 - \tan^2 x}$$

```
>$'tanh(2*x)=trigexpand(tanh(2*x))
```

$$\tanh(2x) = \frac{2 \tanh x}{\tanh^2 x + 1}$$

Berikut adalah cara mendapatkan rumus untuk sudut setengah x .

```
>&assume(x>0, x<2*pi); &halfangles:true;
>$sin(x/2), $sinh(x/2)
```

$$\frac{\sqrt{\cosh x - 1}}{\sqrt{2}}$$

Jadi,

$$\sin\left(\frac{x}{2}\right) = \frac{\sqrt{1 - \cos x}}{\sqrt{2}},$$

$$\sinh\left(\frac{x}{2}\right) = \frac{\sqrt{\cosh x - 1}}{\sqrt{2}}.$$

```
>$x+sin(3*x)/sin(x)=trigexpand(x+sin(3*x)/sin(x))
```

$$\frac{\sin(3x)}{\sin x} + x = \frac{3 \cos^2 x \sin x - \sin^3 x}{\sin x} + x$$

```
>$x+sinh(3*x)/sinh(x)=trigexpand(x+sinh(3*x)/sinh(x))
```

$$\frac{\sinh(3x)}{\sinh x} + x = \frac{\sinh^3 x + 3 \cosh^2 x \sinh x}{\sinh x} + x$$

```
>$-sin(x)^2+3*cos(x)^2+x=...
>   trigreduce(-sin(x)^2+3*cos(x)^2+x)
```

$$-\sin^2 x + 3 \cos^2 x + x = \frac{\cos(2x)}{2} + 3 \left(\frac{\cos(2x)}{2} + \frac{1}{2} \right) + x - \frac{1}{2}$$

```
>$-sinh(x)^2+3*cosh(x)^2+x=...
>   trigreduce(-sinh(x)^2+3*cosh(x)^2+x)
```

$$-\sinh^2 x + 3 \cosh^2 x + x = - \left(\frac{\cosh(2x)}{2} \right) + 3 \left(\frac{\cosh(2x)}{2} + \frac{1}{2} \right) + x + \frac{1}{2}$$

Perintah/fungsi `trigreduce()` menggabungkan hasil kali atau/dan perpangkatan fungsi trigonometri, fungsi hiperbolik sinus, kosinus dengan variabel x menjadi fungsi trigonometri atau fungsi hiperbolik sinus/kosinus kelipatan x . Selain itu, fungsi ini juga mencoba menghilangkan fungsi-fungsi tersebut sebagai penyebut.

```
>$'sin(x)^2=trigreduce(sin(x)^2)
```

$$\sin^2 x = \frac{1 - \cos(2x)}{2}$$

```
>$'sinh(x)^2=trigreduce(sinh(x)^2)
```

$$\sinh^2 x = \frac{\cosh(2x) - 1}{2}$$

```
>$'sin(x)*cos(x)=trigreduce(sin(x)*cos(x))
```

$$\cos x \sin x = \frac{\sin(2x)}{2}$$

```
>$'sinh(x)*cosh(x)=trigreduce(sinh(x)*cosh(x))
```

$$\cosh x \sinh x = \frac{\sinh(2x)}{2}$$

```
>$'sin(theta)^2+sin(theta)*cos(theta)+cos(theta)^2= ...
> trigreduce(sin(theta)^2+sin(theta)*cos(theta)+...
> cos(theta)^2)
```

$$\sin^2 \vartheta + \cos \vartheta \sin \vartheta + \cos^2 \vartheta = \frac{\sin(2\vartheta)}{2} + \frac{\cos(2\vartheta) + 1}{2} + \frac{1 - \cos(2\vartheta)}{2}$$

```
>$'sinh(theta)^2+sinh(theta)*cosh(theta)+...
> cosh(theta)^2=trigreduce(sinh(theta)^2+...
> sinh(theta)*cosh(theta)+cosh(theta)^2)
```

$$\sinh^2 \vartheta + \cosh \vartheta \sinh \vartheta + \cosh^2 \vartheta = \frac{\sinh(2\vartheta)}{2} + \frac{\cosh(2\vartheta) + 1}{2} + \frac{\cosh(2\vartheta) - 1}{2}$$

```
>$sin(3*a)/sin(a+%pi/3)=trigrat(sin(3*a)/sin(a+%pi/3))
```

$$\frac{\sin(3a)}{\sin(a + \frac{\pi}{3})} = \sqrt{3} \sin(2a) + \cos(2a) - 1$$

Fungsi/perintah `trigrat()` menyederhanakan bentuk fungsi trigonometri menjadi bentuk quasi linier, yakni bentuk fungsi trigonometri yang tidak memuat perpangkatan fungsi trigonometri, sedapat mungkin. Berikut adalah beberapa contoh lain.

```
>$ (sin(theta)+cos(theta))^2=...
> expand((sin(theta)+cos(theta))^2)
```

$$(\sin \vartheta + \cos \vartheta)^2 = \sin^2 \vartheta + 2 \cos \vartheta \sin \vartheta + \cos^2 \vartheta$$

```
>$%=trigrat(rhs(%))
```

$$(\sin \vartheta + \cos \vartheta)^2 = \sin^2 \vartheta + 2 \cos \vartheta \sin \vartheta + \cos^2 \vartheta = \sin(2 \vartheta) + 1$$

```
>$sin(theta)^3+sin(theta)^2+(sin(theta)+1)^2= ...
> expand(sin(theta)^3+sin(theta)^2+(sin(theta)+1)^2)
```

$$(\sin \vartheta + 1)^2 + \sin^3 \vartheta + \sin^2 \vartheta = \sin^3 \vartheta + 2 \sin^2 \vartheta + 2 \sin \vartheta + 1$$

```
>$%=trigrat(rhs(%))
```

$$\begin{aligned} (\sin \vartheta + 1)^2 + \sin^3 \vartheta + \sin^2 \vartheta &= \sin^3 \vartheta + 2 \sin^2 \vartheta + 2 \sin \vartheta + 1 \\ &= - \left(\frac{\sin(3 \vartheta) + 4 \cos(2 \vartheta) - 11 \sin \vartheta - 8}{4} \right) \end{aligned}$$

```
>$sin(5*x)=trigexpand(sin(5*x))
```

$$\sin(5x) = \sin^5 x - 10 \cos^2 x \sin^3 x + 5 \cos^4 x \sin x$$

```
>$sinh(5*x)=trigexpand(sinh(5*x))
```

$$\sinh(5x) = \sinh^5 x + 10 \cosh^2 x \sinh^3 x + 5 \cosh^4 x \sinh x$$

```
>$sin(x)^5=trigreduce(sin(x)^5)
```

$$\sin^5 x = \frac{\sin(5x) - 5 \sin(3x) + 10 \sin x}{16}$$

```
>$sinh(x)^5=trigreduce(sinh(x)^5)
```

$$\sinh^5 x = \frac{\sinh(5x) - 5 \sinh(3x) + 10 \sinh x}{16}$$


```
>$'cos(x)^4-sin(x)^4= trigsimp(cos(x)^4-sin(x)^4)
```

$$\cos^4 x - \sin^4 x = 2 \cos^2 x - 1$$

```
>$'cosh(x)^4-sinh(x)^4= trigsimp(cosh(x)^4-sinh(x)^4)
```

$$\cosh^4 x - \sinh^4 x = 2 \cosh^2 x - 1$$

```
>$showev('trigsimp(cos(x)*(sec(x)^2*tan(x)+1)-...
>      sec(x)^2*sin(x)-cos(x)))
```

$$\text{trigsimp}(\cos x (\sec^2 x \tan x + 1) - \sec^2 x \sin x - \cos x) = 0$$

```
>$showev('trigsimp(((1-sin(x)^2)*cos(x))/cos(x)^2+...
>      tan(x)*sec(x)^2))
```

$$\text{trigsimp}\left(\sec^2 x \tan x + \frac{1 - \sin^2 x}{\cos x}\right) = \frac{\sin x + \cos^4 x}{\cos^3 x}$$

```
>$showev('trigsimp(((1-sinh(x)^2)*cosh(x))/...
>      cosh(x)^2+tanh(x)*sech(x)^2))
```

$$\text{trigsimp}\left(\text{sech}^2 x \tanh x + \frac{1 - \sinh^2 x}{\cosh x}\right) = -\left(\frac{\sinh^4 x - \sinh x - 1}{\cosh^3 x}\right)$$

```
>$showev('trigsimp((tan(x)^2+sec(x)^2/...
>      (1-tan(x)*sec(x))))
```

$$\text{trigsimp}\left(\frac{\sec^2 x}{1 - \sec x \tan x} + \tan^2 x\right) = \frac{\sin^4 x + \sin^3 x - 1}{\cos^2 x \sin x - \cos^4 x}$$

```
>$showev('trigsimp((tanh(x)^2+sech(x)^2/...
>      (1-tanh(x)*sech(x))))
```

$$\text{trigsimp}\left(\frac{\text{sech}^2 x}{1 - \text{sech } x \tanh x} + \tanh^2 x\right) = \frac{(\cosh^2 x - 1) \sinh x - \cosh^4 x}{\cosh^2 x \sinh x - \cosh^4 x}$$

```
>$showev('trigsimp((sin(x)^4-6*cos(x)^2*sin(x)^2+...
> 4*(cos(x)^2-sin(x)^2)+ ...
> 8*sin(x)+cos(x)^4+3)/(8*cos(x)^3))
```

$$\text{trigsimp}\left(\frac{\sin^4 x - 6 \cos^2 x \sin^2 x + 4 (\cos^2 x - \sin^2 x) + 8 \sin x + \cos^4 x + 3}{8 \cos^3 x}\right)$$

$$= \frac{\sin x + \cos^4 x}{\cos^3 x}$$

```
>$showev('trigsimp((sech(x)^2*sinh(x)*tanh(x))/...
> coth(x)^2+(cosh(x)^2*sech(x)^2*tanh(x))/...
> coth(x)^2+(sech(x)^2*tanh(x))/coth(x)^2))
```

$$\text{trigsimp}\left(\frac{\text{sech}^2 x \sinh x \tanh x}{\coth^2 x} + \frac{\cosh^2 x \text{sech}^2 x \tanh x}{\coth^2 x} + \frac{\text{sech}^2 x \tanh x}{\coth^2 x}\right)$$

$$= \frac{\sinh^5 x + \sinh^4 x + 2 \sinh^3 x}{\cosh^5 x}$$

3.9 Operasi dan Fungsi Bilangan Kompleks

Bilangan kompleks z di dalam matematika merupakan bilangan yang dapat dinyatakan dalam bentuk:

$$z = x + i.y$$

dengan x dan y adalah bilangan-bilangan riil dan $i = \sqrt{-1}$ atau $i^2 = -1$ adalah bilangan imajiner. Bilangan x juga disebut bagian riil bilangan kompleks z , sedangkan y disebut bagian imajiner z .

EMT dapat melakukan berbagai perhitungan bilangan kompleks, baik secara numerik maupun simbolik. Tersedia banyak fungsi untuk bilangan kompleks di EMT.

Bilangan kompleks dimasukkan dengan menambahkan i ke bagian imajiner. Untuk memasukkan bilangan imajiner $i = \sqrt{-1}$ secara terpisah, tuliskan dengan huruf $\mathbb{I}$ (huruf besar $\mathbb{I}$), namun akan ditampilkan dengan huruf i (i kecil). Apabila dituliskan sebagai bilangan kompleks, i dapat ditulis dengan huruf i kecil, seperti $2i$, $2-i$, $3+2i$, dll.

```
>sqrt(-1)//-1 adalah riil, sqrt adalah fungsi riil
Floating point error!
Error in sqrt
Error in:
sqrt(-1)//-1 adalah riil, sqrt adalah fungsi riil...
^
```

Perintah EMT `sqrt(-1)` tidak akan berfungsi. Jadi akar kuadrat -1 akan menghasilkan kesalahan. Untuk mengubah bilangan real x menjadi bilangan kompleks, gunakan perintah `complex(x)`.

```
>i // i sebagai nama variabel, bukan konstanta imajiner
Variable i not found!
Error in:
i ...
^

>sqrt(complex(-1)), I, 1i // ketiganya bernilai sama
0+1i
0+1i
0+1i
>-3i, 2i, 2-3i, 3+2i // diterima EMT
0-3i
0+2i
2-3i
3+2i
```

Untuk bilangan kompleks simbolik (Maxima), konstanta imajiner i dituliskan dengan `%i` (huruf i kecil diawali tanda prosen) atau `I` (huruf I besar).

```
>%i // variabel simbolik i
```

$$i$$

```
>$2+i // ekspresi simbolik yang memuat variabel i
```

$$2+i$$

```
>$$i, $I // bilangan imajiner i
```

$$i$$

```
>$2+%i, $3-2*I // bilangan-bilangan kompleks eksak
```

$$3 - 2i$$

```
>&powerdisp:true; $z:x+y*i // jangan lupa tanda %
```

$$x + iy$$

```
>$z:x-y*I // atau menggunakan huruf I besar
```

$$x - iy$$

```
>$w:a+b*i // bukan bilangan kompleks
```

$$a + bi$$

Perhatikan perbedaan tampilan oleh Maxima antara bilangan kompleks (konstanta imajiner ditampilkan di depan pengalinya) dan bukan bilangan kompleks (i sebagai nama variabel ditampilkan di belakang pengalinya).

Fungsi `re()` berguna untuk mendapatkan bagian riil suatu bilangan kompleks, sedangkan fungsi `im()` berguna untuk mendapatkan bagian imajiner (pengali i) suatu bilangan kompleks. Fungsi yang sama untuk simbolik (Maxima) adalah `realpart()` dan `imagpart()`.

```
>z:=5-3i
      5-3i
>re(z)
      5
>im(z)
     -3
>&powerdisp:true;
>$z:a+b*I
```

$$a + ibz := 5 - 3i$$

```
>$realpart(z)
```

$$az := 5 - 3i$$

```
>$imagpart (z)
```

$$bz := 5 - 3i$$

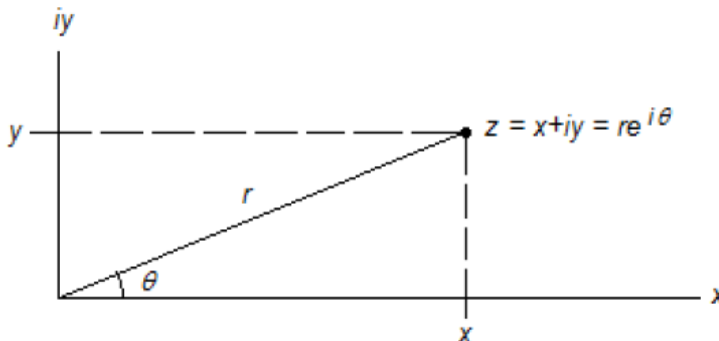
Bilangan kompleks $w = x - i.y$ dinamakan *sekawan* (*konjugat*) bilangan kompleks $z = x + i.y$ dan disimbolkan dengan $\bar{z} = x - i.y$.

EMT menyediakan dua fungsi untuk menghitung konjugat bilangan kompleks, yakni `conj()` untuk sekawan bilangan kompleks secara numerik dan `conjugate()` untuk sekawan bilangan kompleks secara simbolik (dengan Maxima).

```
>z=3+2*I // bilangan kompleks numerik
      3+2i
>conj(z) // konjugat (sekawan z) secara numerik
      3-2i
>&remvalue(z,w);
>//bilangan kompleks simbolik dan sekawannya
>$z:a+b*I, $conjugate(z)
```

$$a - ibz := 5 - 3i$$

Secara geometris, bilangan kompleks z tersebut dapat disajikan sebagai suatu titik pada diagram Argand, yakni sistem koordinat Kartesius dengan ordinat menyajikan bilangan-bilangan imajiner.



Seperti ditunjukkan pada gambar di atas, setiap bilangan kompleks dapat dinyatakan dalam bentuk Kartesius ($z = x + i.y$) atau bentuk polar ($z = r.e^{i.\theta}$), dengan

$$r = \sqrt{x^2 + y^2}$$

disebut *modulus* z , sedangkan

$$\theta = \text{Arg}(z) = \tan^{-1} \left(\frac{y}{x} \right)$$

disebut *argumen* z .

EMT menyediakan beberapa fungsi untuk menghitung modulus bilangan kompleks, baik secara numerik (yakni `abs()`) maupun secara simbolik dengan `Maxima` (yakni `cabs()`). Demikian juga, untuk menghitung argumen suatu bilangan kompleks secara numerik dapat menggunakan fungsi `arg()` (hasilnya dalam radian), sedangkan secara simbolik dengan `carg()`.

```
>// modulus dan argumen z secara numerik
>z=3+4*I, abs(z), arg(z)
      3+4i
      5
      0.927295218002
>fraction sin(arg(z)) // cek dengan fungsi trigonometri
      4/5
>fraction cos(arg(z)), fraction tan(arg(z))
      3/5
      4/3
>$w:1+sqrt(3)*I
```

$$1 + \sqrt{3}i$$

```
>// menghitung argumen (sudut) w secara eksak
>$carg(w)
```

$$\frac{\pi}{3}$$

```
>// modulus dan argumen z secara simbolik
>$z, $abs(z), $r=cabs(z), $theta=carg(z)
```

$$\vartheta = \text{atan2}(b, a)$$

Penjumlahan dan pengurangan dua bilangan kompleks dilakukan dengan penjumlahan/mengurangkan bagian riil kedua bilangan serta menjumlahkan/mengurangkan bagian imajiner keduanya.

$$z_1 = x_1 + i.y_1, \quad z_2 = x_2 + i.y_2, \quad \Rightarrow z_1 \pm z_2 = (x_1 \pm x_2) + i(y_1 \pm y_2).$$

Perkalian dua bilangan kompleks dilakukan dengan cara mengalikan komponen-komponen keduanya.

$$\begin{aligned} z_1 &= x_1 + i.y_1, & z_2 &= x_2 + i.y_2, \\ \Rightarrow z_1.z_2 &= (x_1 + i.y_1)(x_2 + i.y_2) \\ &= (x_1x_2 - y_1y_2) + i(x_1y_2 + x_2y_1). \end{aligned}$$

Pembagian dua bilangan kompleks dilakukan dengan cara mengalikan penyebut dan pembilang dengan sekawan (konjugat) penyebut, sehingga penyebutnya menjadi riil.

$$\begin{aligned} z_1 &= x_1 + i.y_1, & z_2 &= x_2 + i.y_2, \\ \Rightarrow \frac{z_1}{z_2} &= \frac{z_1 \overline{z_2}}{z_2 \overline{z_2}} \\ &= \frac{(x_1 + i.y_1)(x_2 - i.y_2)}{(x_2 + i.y_2)(x_2 - i.y_2)} \\ &= \frac{(x_1x_2 + y_1y_2) + i.(x_2y_1 - x_1y_2)}{x_2^2 + y_2^2}. \end{aligned}$$

```
>z=2+3i, w=3-2i, z+w, z-w, z*w, z/w
2+3i
3-2i
5+1i
-1+5i
12+5i
0+1i
>$z:x+y*I, $w:a+b*I, $z+w, $z-w, $expand(z*w), $z/w
```

$$\frac{x+iy}{a+ib}$$

Untuk menyatakan bilangan kompleks dalam bentuk baku $a + b*i$ dengan Maxima dapat digunakan perintah/fungsi `rectform()`.

```
>$rectform(z+w), $rectform(z-w), $rectform(z*w)
```

$$ax - by + i(bx + ay)$$

```
>$rectform(z/w)
```

$$\frac{i(-(bx) + ay)}{a^2 + b^2} + \frac{ax + by}{a^2 + b^2}$$

```
> $ '(2-3*I) / (1+I) = rectform( (2-3*I) / (1+I) )
```

$$\frac{2-3i}{i+1} = -\left(\frac{5i}{2}\right) - \frac{1}{2}$$

3.9.1 Bentuk Polar Bilangan Kompleks

Dari hubungan

$$z = x + i.y = r.e^{i.\theta}$$

dan

$$x = r.\cos(\theta) \text{ dan } y = r.\sin(\theta),$$

maka diperoleh rumus

$$e^{i.\theta} = \cos(\theta) + i.\sin(\theta).$$

Rumus ini dikenal dengan rumus Euler yang sangat terkenal dalam matematika. Karena

$$\cos(\theta) = \cos(\theta + k.2\pi), \text{ dan } \sin(\theta) = \sin(\theta + k.2\pi),$$

maka

$$e^{i\theta} = e^{i(\theta + k.2\pi)}$$

untuk setiap k bilangan bulat.

Rumus Euler memberikan hasil yang juga sangat terkenal dalam matematika, yakni

$$e^{i\pi} = -1.$$

Perhatikan, dari rumus Euler diperoleh

$$e^{-i.\theta} = \cos(\theta) - i.\sin(\theta),$$

sehingga dengan menggabungkan keduanya diperoleh

$$\cos(\theta) = \frac{e^{i.\theta} + e^{-i.\theta}}{2},$$

dan

$$\sin(\theta) = \frac{e^{i.\theta} - e^{-i.\theta}}{2i}.$$

Hasil ini dapat diperluas untuk mendefinisikan fungsi trigonometri bilangan kompleks, yakni

$$\cos(z) = \frac{e^{i.z} + e^{-i.z}}{2},$$

dan

$$\sin(z) = \frac{e^{i.z} - e^{-i.z}}{2i}.$$

Rumus Euler juga akan menghasilkan rumus lain yang sangat terkenal dalam matematika, yakni rumus De Moivre:

$$(\cos(\theta) + i.\sin(\theta))^n = (e^{i.\theta})^n = e^{i.n\theta} = \cos(n\theta) + i.\sin(n\theta).$$

Untuk mengubah bilangan kompleks ke bentuk polar dapat digunakan fungsi Maxima `polarform()`.

```
>$'1+sqrt(2)*I=polarform(1+sqrt(2)*I)
```

$$\sqrt{2}i + 1 = \sqrt{3}e^{i \arctan \sqrt{2}}$$

```
>&assume(r1>0, r2>0);
>$z: a+b*I
```

$$ib + a$$

```
>$'z=polarform(z)
```

$$ib + a = \sqrt{b^2 + a^2} e^{i \arctan(b/a)}$$

Berikut adalah rumus untuk menghitung nilai fungsi trigonometri bilangan kompleks.

$$\begin{aligned} \cos(z) &= \cos(x + iy) = \cos(x)\cos(iy) - \sin(x)\sin(iy) \\ &= \cos(x)\frac{e^y + e^{-y}}{2} - \sin(x)\frac{e^{-y} - e^y}{2i} \\ &= \cos(x)\frac{e^y + e^{-y}}{2} - i\sin(x)\frac{e^y - e^{-y}}{2} \\ &= \cos(x)\cosh(y) - i\sin(x)\sinh(y). \end{aligned}$$

$$\begin{aligned} \sin(z) &= \sin(x + iy) = \sin(x)\cos(iy) + \cos(x)\sin(iy) \\ &= \sin(x)\frac{e^y + e^{-y}}{2} + \cos(x)\frac{e^{-y} - e^y}{2i} \\ &= \sin(x)\frac{e^y + e^{-y}}{2} + i\cos(x)\frac{e^y - e^{-y}}{2} \\ &= \sin(x)\cosh(y) + i\cos(x)\sinh(y). \end{aligned}$$

Untuk mendapatkan kesamaan di atas dengan Maxima dapat digunakan tiga fungsi, yakni `polarform()`, `demoivre()`, dan `radcan()`.

```
>$'cos(z)=radcan(demoivre(polarform(cos(z))))
```

$$\cos(a+ib) = \cos a \cosh b - i \sin a \sinh b$$

```
>$'cos(%pi/3+I)=...
> rectform(radcan(demoivre(polarform(cos(%pi/3+I)))))
```

$$\cos\left(i + \frac{\pi}{3}\right) = \frac{\cosh 1}{2} - \frac{\sqrt{3}i \sinh 1}{2}$$

```
>$'sin(z)=radcan(demoivre(polarform(sin(z))))
```

$$\sin(a+ib) = \sin a \cosh b + i \cos a \sinh b$$

```
>$'sin(%pi/3+I)=...
> rectform(radcan(demoivre(polarform(sin(%pi/3+I)))))
```

$$\sin\left(i + \frac{\pi}{3}\right) = \frac{\sqrt{3} \cosh 1}{2} + \frac{i \sinh 1}{2}$$

Rumus-rumus di atas juga digunakan untuk menghitung nilai-nilai fungsi trigonometri bilangan kompleks secara numerik. Berikut adalah contohnya.

```
>z=2+3i
      2+3i
>sin(z)
      9.1545-4.16891i
>sin(re(z))*cosh(im(z))
      9.15449914691
>cos(re(z))*sinh(im(z))
      -4.16890695997
```

Hasil tersebut benar.

```
>cos(z)
-4.18963-9.10923i
>cos(re(z))*cosh(im(z))
-4.18962569097
>sin(re(z))*sinh(im(z))
9.10922789376
```

Nilai-nilai yang diperoleh sesuai.

Untuk menghitung nilai eksponensial bilangan kompleks digunakan persamaan Euler

$$e^{a+bi} = e^a \cdot e^{bi} = e^a \cdot (\cos(b) + i \cdot \sin(b)).$$

```
>z
2+3i
>exp(z)
-7.31511+1.04274i
>// cek dengan rumus di atas
>E^(re(z))*(cos(im(z))+sin(im(z))*I)
-7.31511+1.04274i
```

Hasilnya sama, jadi fungsi eksponensial hasilnya benar.

Untuk menampilkan hasil perhitungan fungsi eksponensial secara simbolik perlu digunakan fungsi `demoivre()`.

```
>&powerdisp:true;
>$z:a+b*I
```

$$a + ib$$

```
>$'exp(z)=demoivre(exp(z))
```

$$e^{a+ib} = e^a (\cos b + i \sin b)$$

Untuk perhitungan fungsi logaritma bilangan kompleks digunakan bentuk polar bilangan kompleks.

$$a + b.i = r.e^{i.\theta} = \sqrt{a^2 + b^2}.e^{i.\tan^{-1}(b/a)}$$

sehingga:

$$\begin{aligned}\log(a + b.i) &= \log\left(\sqrt{a^2 + b^2} \cdot e^{i \cdot \tan^{-1}(b/a)}\right) \\ &= \log\left(\sqrt{a^2 + b^2}\right) + \tan^{-1}(b/a) \cdot i.\end{aligned}$$

```
>z
      2+3i
>log(z)
      1.28247+0.982794i
>// cek dengan rumus di atas.
>log(abs(z))+atan(im(z)/re(z))*I
      1.28247+0.982794i
```

Untuk menampilkan rumus logaritma bilangan kompleks secara eksak, variabel `logexpand` harus diberi nilai `all` atau `true`.

```
>$z
```

$$a + ib$$

```
>&logexpand=all;
>$'log(z)=log(polarform(z))
```

$$\log(a + ib) = i \operatorname{atan2}(b, a) + \frac{\log(a^2 + b^2)}{2}$$

Hasil ini sama dengan rumus logaritma bilangan kompleks di atas.

Bab 4.

Komputasi Aljabar Lanjut dengan EMT

Bab ini menjelaskan tentang berbagai perhitungan terkait dengan materi atau topik dalam aljabar dengan menggunakan software EMT, sebagai kelanjutan bab sebelumnya. Secara khusus bab ini membahas topik-topik:

1. penyelesaian persamaan dan sistem persamaan,
2. penyelesaian pertidaksamaan dan sistem pertidaksamaan,
3. pengantar aljabar matriks, dan
4. aplikasi aljabar untuk perhitungan bunga majemuk.

4.1 Menyelesaikan Persamaan dan Sistem Persamaan

EMT dapat menyelesaikan persamaan secara numerik (untuk persamaan yang memuat variabel dan koefisien-koefisien variabel berupa bilangan) maupun secara eksak maupun simbolik (untuk persamaan simbolik). Untuk menyelesaikan persamaan

$$f(x) = 0$$

secara numerik dapat digunakan perintah `solve(f(x), a, b, y, tol)` dengan `a` dan `b` adalah dua hampiran awal, `y` adalah nilai ruas kanan dalam persamaan $f(x) = y$ yang aslinya $y=0$, dan `tol` adalah batas toleransi galat (`y` dan `tol` bersifat opsional -- boleh tidak dituliskan). Perintah ini menggunakan metode tali busur untuk menyelesaikan persamaan secara numerik:

$$x_{n+1} = x_n - \left(f(x_n) \cdot \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \right),$$

dengan x_0 dan x_1 diketahui/ditentukan.

Contoh:

Misalkan akan dicari penyelesaian persamaan

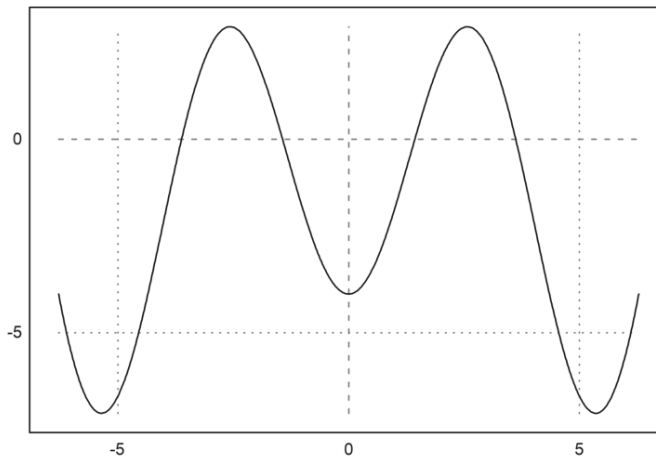
$$x \cdot \sin(x) - 3 \cos(x) = 1.$$

Sebelum menyelesaikan persamaan tersebut, kita dapat menggambar kurva fungsinya untuk melihat pada interval mana akar/solusi persamaan tersebut (jika ada). Kita mulai dengan mendefinisikan fungsinya secara simbolik.

```
>function f(x) &= x*sin(x)-3*cos(x); $'f(x)=f(x)
```

$$f(x) = x \sin x - 3 \cos x$$

```
>plot2d("f(x)-1",-2*pi,2*pi):
```

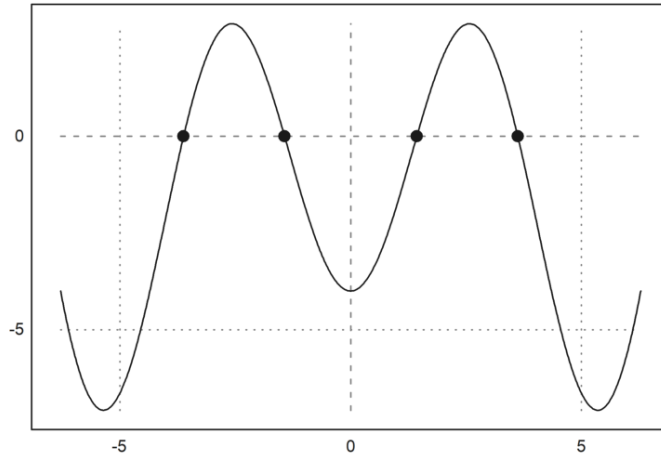


Dari kurva fungsi tersebut terlihat bahwa kurva $y = f(x) - 1$ bersifat simetris terhadap sumbu- y (fungsi genap -- karena $f(-x) = f(x)$), osilatif, dan memotong sumbu- x tak berhingga kali, sehingga persamaan $f(x) = 1$ memiliki tak berhingga solusi. Beberapa solusi adalah di sekitar $x = -4, x = -2, x = 1, x = 3$.

```
>x1=solve("x*sin(x)-3*cos(x)-1",0,2)
1.43123665705
>x2=solve("x*sin(x)-3*cos(x)-1",3,5)
3.61933977605
>x3=solve("x*sin(x)-3*cos(x)-1",-2,0)
-1.43123665705
>x4=solve("x*sin(x)-3*cos(x)-1",-5,-3)
-3.61933977605
```

Berikut ditunjukkan bahwa keempat solusi tersebut merupakan absis titik-titik potong kurva $y = f(x) - 1$ dengan sumbu- x .

```
>plot2d([x1,x2,x3,x4],[f(x1)-1,f(x2)-1,...
>f(x3)-1,f(x4)-1],>points,style="o#",>add):
```



Selanjutnya, bagaimanakah menyelesaikan persamaan di atas secara eksak? Berikut kita coba selesaikan persamaannya dengan menggunakan perintah Maxima solve.

```
>$solve(f(x)=1,x)
```

$$\left[x = \frac{3 \cos x + 1}{\sin x} \right]$$

Maxima gagal menyelesaikan persamaan trigonometri tersebut. Terdapat 2 (dua) paket Maxima yang dapat digunakan untuk menyelesaikan persamaan, termasuk persamaan trigonometri, yakni paket `trigtools` dan paket `to_poly_solve`. Kita akan coba keduanya dan lihat bagaimana hasilnya.

```
>&load(to_poly_solve)
C:/maxima-5.47.0/share/maxima/5.47.0/
share/to_poly_solve/to_poly_solve.mac
>&load(trigtools)
C:/maxima-5.47.0/share/maxima/5.47.0/
share/contrib/trigtools/trigtools.mac
>$eq:f(x)=1
```

$$x \sin x - 3 \cos x = 1$$


```
>$to_poly_solve(eq, x)
```

```
%solve([x sin x - 3 cos x = 1], [x])
```

```
>$trigsolve(eq, -2*%pi, 2*%pi)
```

```
{0, 1}
```

```
>$to_poly_solve(x*cos(x-pi/2)-3*cos(x)=1, x)
```

```
%solve([x sin x - 3 cos x = 1], [x])
```

Baik perintah Maxima `to_poly_solve` dari paket yang sama namanya, maupun perintah `trigsolve` dari paket `trigtools` gagal menyelesaikan persamaan trigonometri di atas. Kenyataannya memang persamaan tersebut tidak dapat diselesaikan secara eksak.

Maxima tidak dapat menyelesaikan persamaan di atas secara eksak, karena memang persamaan tersebut tidak dapat diselesaikan secara eksak. Akan tetapi, Maxima dapat menyelesaikan persamaan tersebut secara numerik dengan perintah `find_root`. Kita coba ini apakah hasilnya sama dengan EMT.

```
>$find_root(f(x)=1, x, 0, 2), $find\_root(f(x)=1, x, 2, 5)
```

```
3.6193397760522332
```

```
>$find_root(f(x)=1, x, -2, 0), $find\_root(f(x)=1, x, -5, -2)
```

```
-3.6193397760522332
```

Hasil yang diperoleh sama dengan yang didapatkan dengan perintah EMT `solve`.

Berikut ditunjukkan beberapa contoh menyelesaikan persamaan secara numerik dan simbolik.

```
>// solusi numerik
>solve("x^2-3*x+1",0,1), solve("x^2-3*x+1",1,3)
    0.38196601125
    2.61803398875
>$solve(x^2-3*x+1,x) // solusi eksak
```

$$\left[x = -\left(\frac{\sqrt{5}-3}{2}\right), x = \frac{\sqrt{5}+3}{2} \right]$$

```
>$float(%) // tampilan numerik solusi eksak
```

$$[x = 0.3819660112501051, x = 2.618033988749895]$$

Untuk menyelesaikan persamaan polinomial, Maxima juga menyediakan perintah `allroots` dan `realroots`. Hasilnya adalah solusi numerik.

```
>$allroots(x^2-3*x+1) // semua solusi x^2-3x+1=0
```

$$[x = 0.38196601125010515, x = 2.618033988749895]$$

```
>solve("3*sin(x)-2*cos(x)+tan(x)",0,3)//solusi numerik
    6.73587780572
>$solve(realroots((3*sin(x)-2*cos(x)+tan(x),x)))
```

$$[x = 0]$$

```
>$ex : sin(x)^2 -2*sin(x) -3
```

$$\sin^2 x - 2 \sin x - 3$$

```
>$solve(ex)
```

$$\left[x = \arcsin 3, x = -\left(\frac{\pi}{2}\right) \right]$$

```
>$find_root(3*sin(x)-2*cos(x)+tan(x), x, 0, 3)
```

0.452692498545336

Berikut adalah contoh-contoh penyelesaian persamaan dengan perintah Maxima di EMT.

$$\cos^{-1}(3x)(f(x) - 1) = 0$$

```
>$solve(asin(cos(3*x)) * (f(x) - 1), x)
```

$$\left[x = \frac{\pi}{6}, f(x) = 1 \right]$$

$$5^{f(x)} = 125$$

```
>$ev(solve(5^f(x) = 125, f(x)), solveradcan)
```

$$\left[f(x) = \frac{\log 125}{\log 5} \right]$$

Menyelesaikan sistem persamaan secara eksak.

$$4x^2 - y^2 = 12$$

$$xy - x = 2$$

```
>$[4*x^2-y^2 = 12, x*y-x = 2] // sistem persamaannya
```

$$[4x^2 - y^2 = 12, xy - x = 2]$$

```
>solek &= solve(%, [x,y]); // solusi eksak
>$solek[1] // solusi pertama
```

$$[x = 2, y = 2]$$

```
>&powerdisp:true;
>$rectform(radcan(solek[2])) // solusi kedua
```

$$\left[x = -\left(\frac{11996778}{90117295}\right) + \frac{21034664i}{40431105}, y = \frac{8390324}{109272081} - \frac{88324608i}{24480191} \right]$$

```
>$rectform(radcan(solek[3])) // solusi ke tiga
```

$$\left[x = -\left(\frac{11996778}{90117295}\right) - \frac{21034664i}{40431105}, y = \frac{8390324}{109272081} + \frac{88324608i}{24480191} \right]$$

```
>$rectform(radcan(solek[4])) // solusi ke empat
```

$$\left[x = -\left(\frac{4695}{2708}\right), y = -\left(\frac{2417}{15739}\right) \right]$$

Penyelesaian persamaan kubik

$$x^3 + ax + 1 = 0.$$

```
>//menyelesaikan persamaan kubik yang memuat parameter a
>solek &=solve(x^3+a*x+1,x);
>$radcan(solek[1]) // solusi pertama
```

$$x = -\left(\frac{\left(-2^{\frac{2}{3}} + 2^{\frac{2}{3}}\sqrt{3}i\right)a + (1 + \sqrt{3}i)\left(-3^{\frac{3}{2}} + \sqrt{27 + 4a^3}\right)^{\frac{2}{3}}}{2^{\frac{4}{3}}\sqrt{3}\left(-3^{\frac{3}{2}} + \sqrt{27 + 4a^3}\right)^{\frac{1}{3}}} \right)$$

```
>$radcan(solek[2]) // solusi kedua
```

$$x = \frac{\left(2^{\frac{2}{3}} + 2^{\frac{2}{3}}\sqrt{3}i\right)a + (-1 + \sqrt{3}i)\left(-3^{\frac{3}{2}} + \sqrt{27 + 4a^3}\right)^{\frac{2}{3}}}{2^{\frac{4}{3}}\sqrt{3}\left(-3^{\frac{3}{2}} + \sqrt{27 + 4a^3}\right)^{\frac{1}{3}}}$$

```
>$radcan(solek[3]) // solusi ketiga
>$radcan(solek[1]) with a=2 //solusi pertama dengan a=2
```

$$x = - \left(\frac{\left(-3^{\frac{3}{2}} + \sqrt{59} \right)^{\frac{2}{3}} (1 + \sqrt{3}i) + 2 \left(-2^{\frac{2}{3}} + 2^{\frac{2}{3}} \sqrt{3}i \right)}{2^{\frac{4}{3}} \sqrt{3} \left(-3^{\frac{3}{2}} + \sqrt{59} \right)^{\frac{1}{3}}} \right)$$

```
>$ratsimp(%) // coba disederhanakan bentuknya
```

$$x = - \left(\frac{-2^{\frac{5}{3}} + 2^{\frac{5}{3}} \sqrt{3}i + \left(-3^{\frac{3}{2}} + \sqrt{59} \right)^{\frac{2}{3}} (1 + \sqrt{3}i)}{2^{\frac{4}{3}} \sqrt{3} \left(-3^{\frac{3}{2}} + \sqrt{59} \right)^{\frac{1}{3}}} \right)$$

```
>// jumlah ketiga akar
>$ratsimp(rhs(solek[1])+rhs(solek[2])+rhs(solek[3]))
```

0

4.1.1 Sistem Persamaan Linier

Persamaan linear adalah sebuah persamaan aljabar, yang setiap sukunya memuat konstanta atau perkalian konstanta dengan suatu variabel. Persamaan ini dikatakan linear sebab hubungan matematis ini dapat digambarkan sebagai garis lurus dalam sistem koordinat Kartesius.

Persamaan Linier Satu Variabel Bentuk umum persamaan linear satu variabel adalah

$$ax + b = 0.$$

Contoh sistem persamaan linier satu variabel

$$10x + 2 = 22.$$

Selesaikan dari persamaan di atas

Jawab:

```
>$solve(10*x+2=22)
```

$$[x = 2]$$

Persamaan Linier Dua Variabel Bentuk umum persamaan linear dua variabel adalah

$$a.x + b.y = c.$$

Contoh sistem persamaan linier dua variabel

(i)

$$\begin{aligned}x + 2y &= 10 \\5x - 3y + 6 &= -9x + 8y + 4.\end{aligned}$$

(ii)

$$\begin{aligned}5x - 3y + 6 &= 0 \\-9x + 8y + 4 &= 0.\end{aligned}$$

Selesaikan kedua sistem persamaan di atas.

Jawab:

```
>$solve([x+2*y=10, 5*x-3*y+6=-9*x+8*y+4])
```

$$\left[\left[y = \frac{142}{39}, x = \frac{106}{39} \right] \right]$$

```
>$solve([5*x-3*y+6=0, -9*x+8*y+4=0])
```

$$\left[\left[y = -\left(\frac{74}{13}\right), x = -\left(\frac{60}{13}\right) \right] \right]$$

Berikut adalah penyelesaian umum SPL dua variabel

$$\begin{aligned}a_1.x + b_1.y &= c_1 \\a_2.x + b_2.y &= c_2\end{aligned}$$

Penyelesaian yang dihasilkan oleh Maxima, sama dengan rumus solusi umum dalam matematika.

```
>$solve([a1*x+b1*y=c1, a2*x+b2*y=c2], [x, y])
```

$$\left[\left[x = -\left(\frac{b_2 c_1 - b_1 c_2}{a_2 b_1 - a_1 b_2}\right), y = \frac{a_2 c_1 - a_1 c_2}{a_2 b_1 - a_1 b_2} \right] \right]$$

4.1.2 Persamaan Linier Tiga Variabel

Bentuk umum persamaan linear tiga variabel adalah

$$a.x + b.y + c.z = d.$$

Contoh sistem persamaan linier tiga variabel.

Jika diketahui sistem persamaan

$$\begin{aligned} 7a - 6b &= -1 + 5c \\ 2a + c &= b - 3a \\ -6 &= 3c + b - 3a, \end{aligned}$$

tentukan:

1. hasil kali semua x yang memenuhi persamaan di bawah ini;
2. jumlah semua x yang memenuhi persamaan di bawah ini;
3. jumlah semua hasil kali setiap dua nilai x berbeda yang memenuhi persamaan di bawah ini.

$$(x-4)(x^2 - a^2) = (x+b)(x+c).$$

Penyelesaian:

Mula-mula, kita selesaikan sistem persamaan di atas.

```
>sol &= solve([7*a-6*b=-1+5*c, 2*a+c=b-3*a, -6=...
>      3*c+b-3*a], [a,b,c]); $sol
```

$$[[a = 1, b = 3, c = -2]]$$

Selanjutnya, jabarkan persamaan polinomialnya dengan mensubstitusikan nilai-nilai a, b, c .

```
>$expand((x-4)*(x^2-a^2)=(x+b)*(x+c)) ...
>      with [a=1,b=3,c=-2]
```

$$4 - x - 4x^2 + x^3 = -6 + x + x^2$$

Kumpulkan di ruas kiri suku-suku yang sejenis.

```
>$x^3-5*x^2-2*x+10=0
```

$$10 - 2x - 5x^2 + x^3 = 0$$

Terakhir, kita faktorkan ruas kiri.

```
>$factor(%)
```

$$(-5+x)(-2+x^2)=0$$

Dari persamaan tersebut, jelas solusinya adalah

$$x = -5, x = -\sqrt{2}, \text{ dan } x = \sqrt{2},$$

dan hasil kali ketiga adalah -10, seperti juga ditunjukkan dari hasil EMT (Maxima) berikut ini.

```
>hasil &= solve(%,x); $hasil
```

$$\left[x = -\sqrt{2}, x = \sqrt{2}, x = 5 \right]$$

```
>$rhs(hasil[1])*rhs(hasil[2])*rhs(hasil[3])
```

$$-10$$

```
>$rhs(hasil[1])+rhs(hasil[2])+rhs(hasil[3])
```

$$5$$

```
>$rhs(hasil[1])*rhs(hasil[2])+...  
>  rhs(hasil[1])*rhs(hasil[3])+ ...  
>  rhs(hasil[2])*rhs(hasil[3])
```

$$-2$$

Kita akan cek hasil di atas secara teoritis. Secara umum, persamaan polinomial berderajat 3

$$ax^3 + bx^2 + cx + d = 0$$

memiliki 3 akar (penyelesaian), misalnya x_1 , x_2 , dan x_3 .

- Hasil kali ketiga akar tersebut memenuhi:

$$x_1 \cdot x_2 \cdot x_3 = \frac{-d}{a}.$$

- Jumlah ketiga akarnya memenuhi:

$$x_1 + x_2 + x_3 = \frac{-b}{a}.$$

- Jumlah semua hasil kali setiap dua akar memenuhi:

$$x_1 \cdot x_2 + x_1 \cdot x_3 + x_2 \cdot x_3 = \frac{c}{a}.$$

Pada persamaan polinomial

$$x^3 - 5x^2 - 2x + 10 = 0,$$

nilai $a = 1$, $b = -5$, $c = -2$, dan $d = 10$, sehingga

1. hasil kali ketiga akarnya adalah:

$$x_1 \cdot x_2 \cdot x_3 = \frac{-d}{a} = \frac{-10}{1} = -10;$$

2. jumlah ketiga akarnya memenuhi:

$$x_1 + x_2 + x_3 = \frac{-b}{a} = \frac{-(-5)}{1} = 5;$$

3. jumlah semua hasil kali setiap dua akar memenuhi:

$$x_1 \cdot x_2 + x_1 \cdot x_3 + x_2 \cdot x_3 = \frac{c}{a} = \frac{-2}{1} = -2;$$

sama seperti hasil EMT di atas.

4.1.3 Persamaan Kuadrat

Persamaan kuadrat disebut juga persamaan suku banyak atau polinomial dengan pangkat tertinggi dua atau polinomial berorde dua. Bentuk umum persamaan kuadrat adalah

$$ax^2 + bx + c = 0.$$

Contoh 1: Faktorkan dan selesaikan bentuk di bawah ini

$$m^2 + (m - 2)m + 2m - 4 = 0.$$

Hitung jumlah dan hasil kali kedua akarnya.

Jawab:

```
>pk1: m^2+(m-2)*m+2*m-4=0;
>$factor(pk1)
```

$$2(-2+m^2)=0$$

Catatan: Maxima tidak dapat memfaktorkan ke bentuk akar.

```
>$sol:solve(pk1)
```

$$\left[m = -\sqrt{2}, m = \sqrt{2} \right]$$

```
>$rhs(sol[1])+rhs(sol[2]) // jumlah kedua akar
```

$$0$$

```
>$rhs(sol[1])*rhs(sol[2]) // hasil kali kedua akar
```

$$-2$$

Contoh 2: Tentukan penyelesaian persamaan kuadrat

$$5x^2 + 7x + 10 = 0.$$

Hitung jumlah dan hasil kali kedua akarnya.

Jawab:

```
>pk2:5*x^2+7*x+10=0;
>$akar:solve(pk2)
>$ratsimp(rhs(akar[1])+rhs(akar[2]))
```

$$-\left(\frac{7}{5}\right)$$

```
>$ratsimp(rhs(akar[1])*rhs(akar[2]))
```

$$2$$

Berikut kita tunjukkan sifat-sifat akar-akar persamaan kuadrat

$$ax^2 + bx + c = 0.$$

```
>$pk:a*x^2+b*x+c=0
```

$$c + bx + ax^2 = 0$$

```
>$akar:solve(pk,x)
```

$$\left[x = - \left(\frac{b + \sqrt{b^2 - 4ac}}{2a} \right), x = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \right]$$

```
>$ratsimp(rhs(akar[1])+rhs(akar[2]))
```

$$- \left(\frac{b}{a} \right)$$

```
>$ratsimp(rhs(akar[1])*rhs(akar[2]))
```

$$\frac{c}{a}$$

Terlihat bahwa jumlah akar-akar persamaan kuadrat

$$ax^2 + bx + c = 0$$

adalah

$$x_1 + x_2 = -\frac{b}{a},$$

dan hasil kali keduanya adalah

$$x_1 \times x_2 = \frac{c}{a}.$$

Kedua hasil sebelumnya cocok dengan teori ini.

```
>$pk : a*x^2 + b*x + c // persamaan kuadrat
```

$$ax^2 + bx + c$$

```
>$akar:solve(pk,x) // akar-akar persamaan kuadrat
```

$$\left[x = - \left(\frac{\sqrt{b^2 - 4ac} + b}{2a} \right), x = \frac{\sqrt{b^2 - 4ac} - b}{2a} \right]$$

```
>$subst(akar[1],pk) // substitusikan akar pertama
```

$$\frac{\left(\sqrt{b^2 - 4ac} + b\right)^2}{4a} - \frac{b\left(\sqrt{b^2 - 4ac} + b\right)}{2a} + c$$

```
>$%=expand(%) // hitung hasilnya
```

$$\frac{\left(\sqrt{b^2 - 4ac} + b\right)^2}{4a} - \frac{b\left(\sqrt{b^2 - 4ac} + b\right)}{2a} + c = 0$$

```
>// substitusikan akar kedua dan hitung
>$subst(akar[2],pk)=expand(subst(akar[2],pk))
```

$$\frac{\left(\sqrt{b^2 - 4ac} - b\right)^2}{4a} + \frac{b\left(\sqrt{b^2 - 4ac} - b\right)}{2a} + c = 0$$

```
>$pk:x^3 + x^2 + x
```

$$x^3 + x^2 + x$$

```
>$akar:solve(pk)
```

$$\left[x = - \left(\frac{\sqrt{3}i + 1}{2} \right), x = \frac{\sqrt{3}i - 1}{2}, x = 0 \right]$$

```
>$subst (akar[1],pk)=expand(subst(akar[1],pk))
```

$$-\left(\frac{(\sqrt{3}i+1)^3}{8}\right) + \frac{(\sqrt{3}i+1)^2}{4} + \frac{-(\sqrt{3}i)-1}{2} = 0$$

```
>$subst (akar[2],pk)=expand(subst(akar[2],pk))
```

$$\frac{(\sqrt{3}i-1)^3}{8} + \frac{(\sqrt{3}i-1)^2}{4} + \frac{\sqrt{3}i-1}{2} = 0$$

4.2 Penyelesaian Langkah demi Langkah

Mungkin Anda protes bahwa kenapa komputer hanya dapat menghasilkan penyelesaian suatu persamaan atau sistem persamaan. Mengapa komputer tidak menunjukkan bagaimana langkah-langkah penyelesaian suatu masalah? Ingat, bahwa komputer adalah benda mati yang cara kerjanya sangat tergantung pada perintah yang ditulis oleh manusia. Jika Anda menginginkan langkah-langkah penyelesaian suatu masalah, maka Anda sendiri yang harus memberikan perintah-perintah agar komputer dapat menunjukkan langkah-langkah tersebut.

Berikut ditunjukkan bagaimana proses penyelesaian suatu sistem persamaan langkah demi langkah. Misalkan kita akan menyelesaikan sistem persamaan (BUKAN linier) dua variabel:

$$x^2 + y^2 + x = 6, \quad x + y = 3,$$

atau

$$\left(x + \frac{1}{2}\right)^2 + y^2 = (2.5)^2, \quad x + y = 3.$$

Mula-mula kedua persamaan kita simpan ke dalam dua variabel simbolik yang berbeda.

```
>p1 &= (x^2+y^2+x=6); p2 &= (x+y=3); $p1, $p2
```

$$x^2 + y^2 + x = 6$$

$$y + x = 3$$

Selanjutnya, kita eliminir variabel x dari persamaan pertama dengan menggunakan persamaan kedua.

```
>$solve(p2,x), $p1 with %, p3 &= expand(%); $p3
```

$$[x = 3 - y]$$

$$y^2 - y + (3 - y)^2 + 3 = 6$$

$$2y^2 - 7y + 12 = 6$$

Diperoleh persamaan kuadrat dalam y . Persamaan kuadrat ini tentu dapat kita selesaikan dengan menggunakan berbagai teknik yang sudah kita ketahui, misalnya dengan cara menyatakan ruas kiri dalam bentuk kuadrat sempurna. Prinsip yang digunakan adalah melakukan operasi yang sama pada kedua ruas.

```
>$expand(p3/2), $%-6+(7/4)^2, $factor(%), $%*16
```

$$y^2 - \frac{7y}{2} + 6 = 3$$

$$y^2 - \frac{7y}{2} + \frac{49}{16} = \frac{1}{16}$$

$$\frac{(4y-7)^2}{16} = \frac{1}{16}$$

$$(4y-7)^2 = 1$$

Dari bentuk terakhir tentu sudah dapat diketahui bahwa solusi diperoleh dari

$$4y - 7 = 1 \text{ atau } 4y - 7 = -1,$$

yang artinya

$$y = 2 \text{ atau } y = \frac{6}{4} = \frac{3}{2}.$$

Hasil ini pun cocok jika dihitung dengan perintah-perintah Maxima.

```
>$solve(%,y); y1 &=y with %[1];  
>y2 &=y with %[2]; $y1, $y2
```

$$\frac{3}{2}$$

Selanjutnya, nilai-nilai y yang diperoleh tersebut disubstitusikan ke persamaan kedua untuk mendapatkan nilai-nilai x .

```
>$solve(p2,x) with y=[y1,y2]
```

$$\left[x = \left[\frac{3}{2}, 1 \right] \right]$$

Jadi, solusi sistem persamaan di atas adalah

$$\left\{ \left(\frac{3}{2}, \frac{3}{2} \right), (1, 2) \right\}.$$

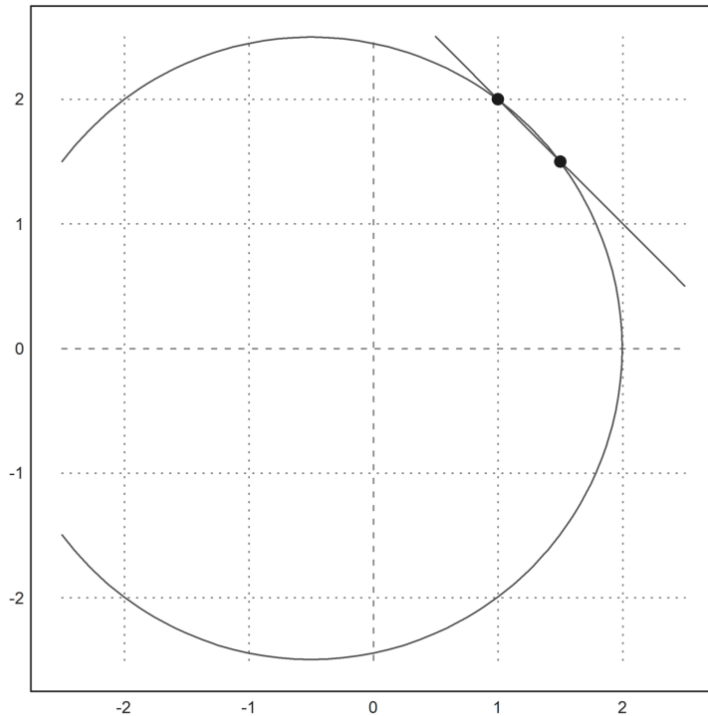
Hasil ini sama dengan solusi yang diperoleh secara langsung dengan perintah Maxima `solve()`.

```
>$solve([p1,p2],[x,y])
```

$$\left[[x = 1, y = 2], \left[x = \frac{3}{2}, y = \frac{3}{2} \right] \right]$$

Gambar berikut juga menunjukkan kedua penyelesaian tersebut.

```
>aspect(1); plot2d("x^2+y^2+x",r=2.5,level=6);  
>plot2d("x+y",r=2.5,level=3,>add);  
>plot2d([1,3/2],[2,3/2],>points,style="o#",>add):
```



4.3 Pertidaksamaan dan Sistem Pertidaksamaan

4.3.1 Pertidaksamaan Linear Satu Variabel

Pertidaksamaan linear satu variabel merupakan bentuk pertidaksamaan dengan satu peubah (variabel) dan pangkat tertingginya 1.

Bentuk umum

$$ax + b < c, \quad ax + b > c, \quad ax + b \leq c, \quad ax + b \geq c.$$

Untuk menyelesaikan pertidaksamaan, EMT tidak akan dapat melakukannya, melainkan dengan bantuan Maxima, artinya secara eksak (simbolik). Perintah Maxima yang digunakan adalah `fourier_elim()`, yang harus dipanggil dengan perintah `load(fourier_elim)` terlebih dahulu.

```
>load(fourier_elim)
      C:/maxima-5.47.0/share/maxima/5.47.0/
share/fourier_elim/fourier_elim.lisp
>$fourier_elim([x + 10 > 0],[x])
```

$$[-10 < x]$$


```
>$fourier_elim([2*x > 15 ], [x])
```

$$\left[\frac{15}{2} < x\right]$$

```
>$fourier_elim([3*x - 18 > 0 ], [x])
```

$$[6 < x]$$

```
>$fourier_elim([x^2 - 1 > 0], [x]) // x^2-1 > 0
```

$$[1 < x] \vee [x < -1]$$

```
>$fourier_elim([x^2 - 1 < 0], [x]) // x^2-1 < 0
```

$$[-1 < x, x < 1]$$

```
>$fourier_elim([x^2 - 1 # 0], [x]) // x^2-1 <> 0
```

$$[-1 < x, x < 1] \vee [1 < x] \vee [x < -1]$$

```
>$fourier_elim([x # 6], [x]) // x <> 6
```

$$[x < 6] \vee [6 < x]$$

```
>// tidak memiliki penyelesaian
>$fourier_elim([x < 1, x > 1], [x])
```

emptyset

```
>// tidak memiliki penyelesaian
>$fourier_elim([minf < x, x < inf], [x])
```

universalset

```
>$fourier_elim([x^3 - 1 > 0], [x])
```

$$[1 < x, x^2 + x + 1 > 0] \vee [x < 1, -x^2 - x - 1 > 0]$$

```
>$fourier_elim([x^2+x+1>0], [x])
```

$$[x^2 + x + 1 > 0]$$

Hasil yang ditampilkan oleh perintah `fourier_elim` bersifat tidak jelas, karena sebenarnya solusi pertidaksamaan

$$x^3 - 1 > 0 \text{ atau } x^3 > 1$$

adalah semua x bilangan riil.

Demikian juga solusi yang diperoleh dari pertidaksamaan

$$x^2 + x + 1 > 0 \text{ atau } (x + \frac{1}{2})^2 + \frac{3}{4} > 0$$

tidak ditampilkan secara eksplisit, padahal sebenarnya solusinya adalah semua bilangan riil karena $x^2 + x + 1$ adalah definit positif.

```
>$fourier_elim([cos(x) < 1/2], [x]) // ??? gagal
```

$$[1 - 2 \cos x > 0]$$

Seperti pada contoh sebelumnya di atas, perintah `fourier_elim` gagal menyelesaikan pertidaksamaan trigonometri.

```
>$fourier_elim(abs((2*x+1)/3)>5, [x])
```

$$[7 < x] \vee [x < -8]$$

Penyelesaian pertidaksamaan

$$|\frac{2x+1}{3}| > 5 \text{ atau } \frac{2}{3}|x - (-\frac{1}{2})| > 5 \text{ atau } |x - (-\frac{1}{2})| > \frac{15}{2}$$

tentu adalah

$$x > \frac{15}{2} - \frac{1}{2} = 7 \text{ atau } x < -\frac{1}{2} - \frac{15}{2} = -8.$$

```
>$fourier_elim(abs(5-2*x)>=1,[x])
```

$$[x = 3] \vee [x = 2] \vee [x < 2] \vee [3 < x]$$

Penyelesaian pertidaksamaan

$$|5 - 2x| \geq 1 \text{ atau } |x - \frac{5}{2}| \geq \frac{1}{2}$$

adalah

$$x \leq \frac{5}{2} - \frac{1}{2} = 2 \text{ atau } x \geq \frac{5}{2} + \frac{1}{2} = 3.$$

```
>$fourier_elim(abs(x-3)+abs(2*x+5)>6,[x])
```

$$[x = 3] \vee [3 < x] \vee [-2 < x, x < 3] \vee \left[x < -\left(\frac{8}{3}\right) \right]$$

Penyelesaian pertidaksamaan

$$|x - 3| + |2x + 5| > 6 \text{ atau } |x - 3| + 2|x - (-\frac{5}{2})| > 6$$

adalah

$$-2 < x \text{ atau } x < -\frac{8}{3}.$$

```
>$fourier_elim([(x+6)/(x-9) <= 6],[x])
```

$$[x = 12] \vee [12 < x] \vee [x < 9]$$

Penyelesaian pertidaksamaan

$$\frac{x+6}{x-9} \leq 6 \text{ atau } \frac{-5x+60}{x-9} \leq 0$$

adalah

$$x \geq 12 \text{ atau } x < 9.$$

4.3.2 Pertidaksamaan Linear Dua Variabel

Pertidaksamaan linear dua variabel adalah bentuk pertidaksamaan yang memuat dua peubah (variabel) dengan pangkat tertinggi variabel tersebut adalah 1.

Bentuk umum

$$ax + by < c, \quad ax + by > c, \quad ax + by \leq c, \quad ax + by \geq c$$

Contoh: Selesaikan sistem pertidaksamaan

$$x + y < 5$$

$$x - y > 0.$$

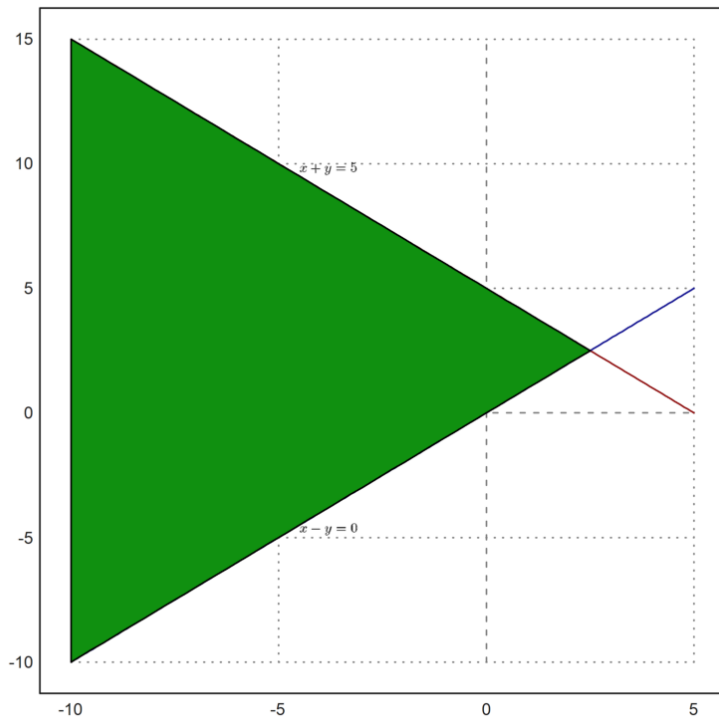
Jawab:

```
>$fourier_elim((x + y < 5) and (x - y > 0), [x, y])
```

$$\left[y < x, x < 5 - y, y < \frac{5}{2} \right]$$

Berikut kita gambar daerah penyelesaian sistem pertidaksamaan di atas.

```
>y1 &= -x+5; y2 &= x;
>aspect(1); plot2d([y1,y2],a=-10,b=5,color=[red,blue]);
>x0 =solve("y1(x)-y2(x)",0);
>x=linspace(-10,x0,50); xb=flipx(x);
>plot2d(x|xb,y1(x)|y2(xb),>filled,fillcolor=green,>add);
>label(latex("x+y=5"),-4.75,10.5);
>label(latex("x-y=0"),-4.75,-4):
```



Contoh: Selesaikan sistem pertidaksamaan

$$\begin{aligned} y - x &< 5 \\ x - y &< 15 \\ y &> 10. \end{aligned}$$

Jawab:

```
>$fourier_elim((y - x < 5) and (x - y < 15) and ...
> (10 < y), [x,y]) // sistem pertidaksamaan
```

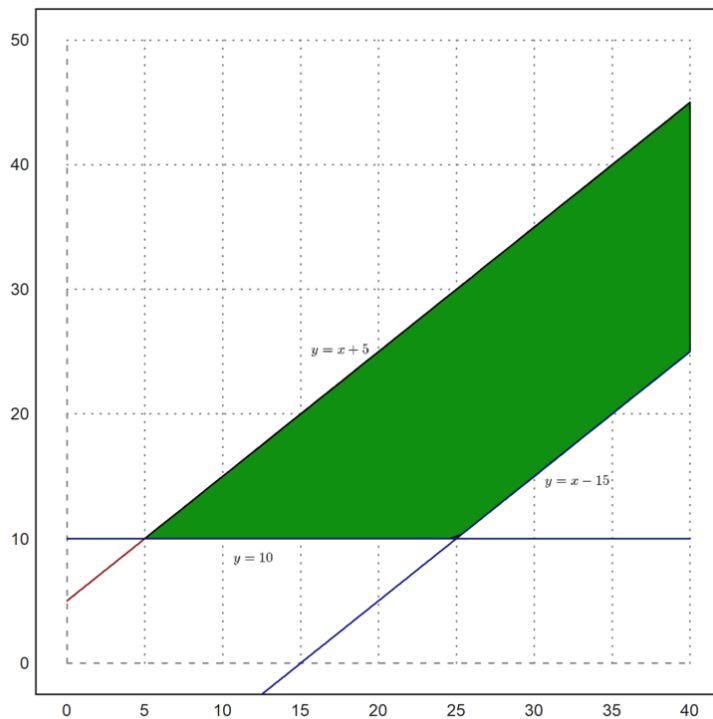
$$[y - 5 < x, 10 < y]$$

Berikut kita gambar daerah penyelesaian sistem pertidaksamaan di atas.

```

>y1 &= x+5;
>function map y2(x) ...
>  if x<=25 then return 10 else return x-15;
  endfunction
>plot2d([y1,"y2(x)"],a=0,b=40,c=0,d=50,color=[red,blue]);
>x=linspace(5,40,50); xb=flipx(x);
>plot2d(x|xb,y1(x)|y2(xb),>filled,fillcolor=green,>add);
>plot2d(["10","x-15"],a=0,b=40,c=0,d=50,color=blue,>add);
>label(latex("y=10"),10,9.75);
>label(latex("y=x+5"),15,26.5);
>label(latex("y=x-15"),30,16):

```



4.3.3 Pertidaksamaan Kuadrat

Pertidaksamaan kuadrat ialah pertidaksamaan yang mempunyai variabel dimana pangkatnya memuat satu atau lebih perubahan serta relasi "lebih dari" ($>$), "lebih dari atau sama dengan" ($\geq$), "kurang dari" ($<$), atau "kurang dari sama dengan" ($\leq$).

Bentuk umum

$$ax^2 + bx + c > 0, \quad ax^2 + bx + c < 0, \quad ax^2 + bx + c \geq 0, \quad ax^2 + bx + c \leq 0$$

Contoh: Selesaikan pertidaksamaan

$$x^2 + 9x + 20 \geq 0.$$

Jawab:

```
>$fourier_elim([x^2 + 9*x + 20 >= 0], [x])
```

$$[x = -5] \vee [x = -4] \vee [-4 < x] \vee [x < -5]$$

Penyelesaian pertidaksamaan

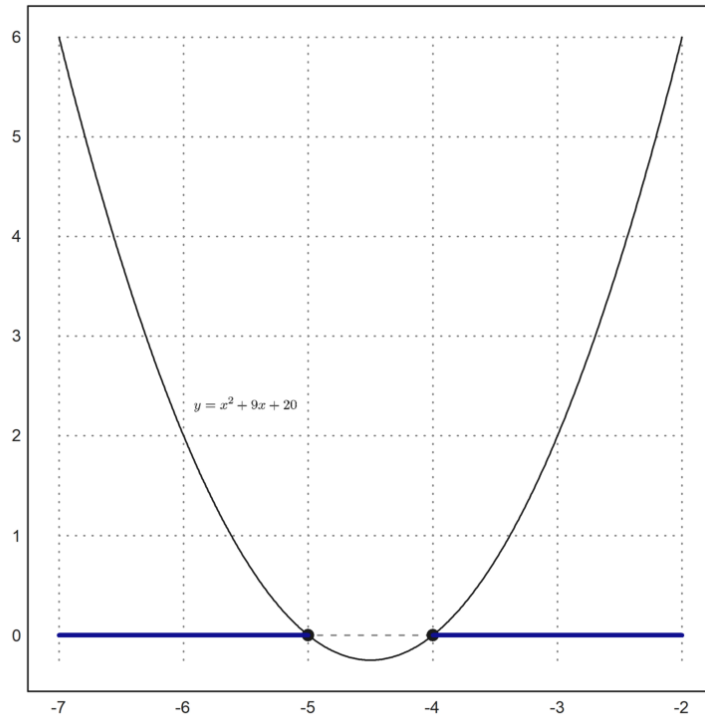
$$x^2 + 9x + 20 = (x + 5)(x + 4) \geq 0$$

adalah

$$\{x | x \leq -5 \vee x \geq -4\}.$$

Berikut kita gambar solusi pertidaksamaan tersebut.

```
>y1 &= x^2+9*x+20;
>plot2d(y1,a=-7,b=-2);
>plot2d([-5,-4],[0,0],>points,style="o#",>add);
>x1=linspace(-7,-5,50); x2=linspace(-4,-2,50);
>y1=zeros(size(x1)); y2=zeros(size(x2));
>label(latex("y=x^2+9x+20"),-6,2.5);
>plot2d(x1,y1,thickness=3,color=blue,>add);
>plot2d(x2,y2,thickness=3,color=blue,>add):
```



Contoh: Selesaikan sistem pertidaksamaan

$$\begin{aligned}x^2 + 8x + 15 &> 0 \\ 2x + 3y &< 10\end{aligned}$$

Jawab:

```
>$fourier_elim([x^2 + 8*x + 15 > 0, 2*x+3*y<10],[x,y])
```

$$\left[-3 < x, x < 5 - \frac{3y}{2}, y < \frac{16}{3} \right] \vee \left[x < \min \left(-5, 5 - \frac{3y}{2} \right) \right]$$

Solusi sistem pertidaksamaan di atas adalah

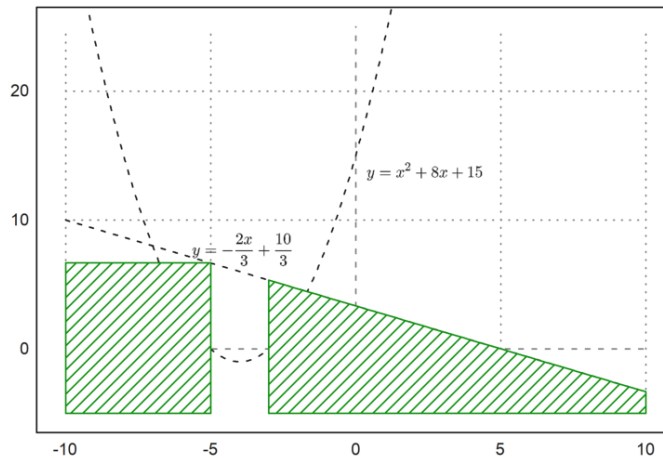
$$\{(x,y) | (x < -5 \wedge y < \frac{20}{3}) \vee (x > -3 \wedge y < -\frac{2x}{3} + \frac{10}{3})\}.$$

Berikut adalah gambar daerah solusi sistem pertidaksamaan tersebut.


```

>y1 &= x^2+8*x+15; y2 &=-2*x/3+10/3;
>function map y3(x) &= 20/3; function map y4(x) &= -5;
>plot2d([y1,y2],a=-10,b=10,c=-5,d=25, style=["--", "--"]);
>x0=linspace(-10,-5,50); xb=flipx(x0);
>plot2d(x0|xb,y4(x0)|y3(xb),>filled,style="/",>add);
>x0=linspace(-3,10,50); xb=flipx(x0); // y3 &= min(x,0);
>label(latex("y=x^2+8x+15"),0,15);
>label(latex("y=-\\frac{2x}{3}+\\frac{10}{3}"),-6,9.5);
>plot2d(x0|xb,y4(x0)|y2(xb),>filled,style="/",>add):

```



4.3.4 Pertidaksamaan Logaritma

Pertidaksamaan logaritma adalah pertidaksamaan yang memuat fungsi logaritma di dalamnya. Bentuk umum persamaan logaritma adalah:

$${}^a\log f(x) < {}^a\log g(x), \quad {}^a\log f(x) > {}^a\log g(x),$$

$${}^a\log f(x) \leq {}^a\log g(x), \quad {}^a\log f(x) \geq {}^a\log g(x).$$

Untuk menyelesaikan pertidaksamaan logaritma, pertidaksamaannya diubah ke salah satu bentuk umum pertidaksamaan logaritma di atas, dan penyelesaiannya diperoleh dengan menyelesaikan pertidaksamaan yang hanya memuat $f(x)$ dan $g(x)$, dengan ketentuan:

- Jika $a > 1$, tanda pertidaksamaannya tetap (tidak berubah);
- Jika $a < 1$, tanda pertidaksamaannya berubah (dibalik).

Contoh:

Selesaikan pertidaksamaan

$${}^2\log(x+1) > 3.$$

Penyelesaian:

Pertidaksamaan tersebut dapat diubah menjadi:

$${}^2\log(x+1) > {}^2\log 2^3,$$

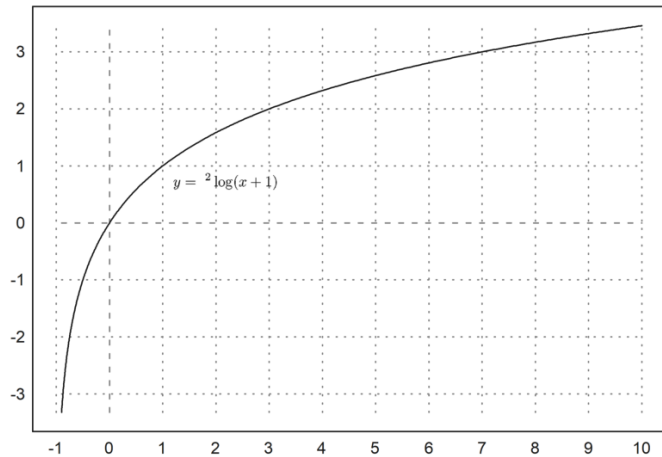
atau

$$x+1 > 8.$$

```
>$fourier_elim([x + 1 > 8], [x])
```

$$[7 < x]$$

```
>plot2d("logbase(x+1,2)",-0.9,10);  
>label(latex("y=\ ^2\log(x+1)"),1,1):
```



Dari grafik fungsi

$$y = {}^2\log(x+1)$$

terlihat bahwa nilai nilai fungsi lebih dari 3 (kurva berada di atas garis $y = 3$) untuk $x > 7$.

```
>$fourier_elim ([log(x + 1)/log(2) > 3], [x])
```

$$[\log(x+1) - 3 \log 2 > 0]$$

Perintah `fourier_elim` tidak berhasil menyelesaikan pertidaksamaan dalam bentuk logaritma. Berikut adalah contoh lain menyelesaikan pertidaksamaan logaritma.

$$\frac{1}{3} \log(2x-3) \geq \frac{1}{3} \log(x+1).$$

```
>$fourier_elim ([2*x - 3 <= x + 1], [x])
```

$$[x = 4] \vee [x < 4]$$

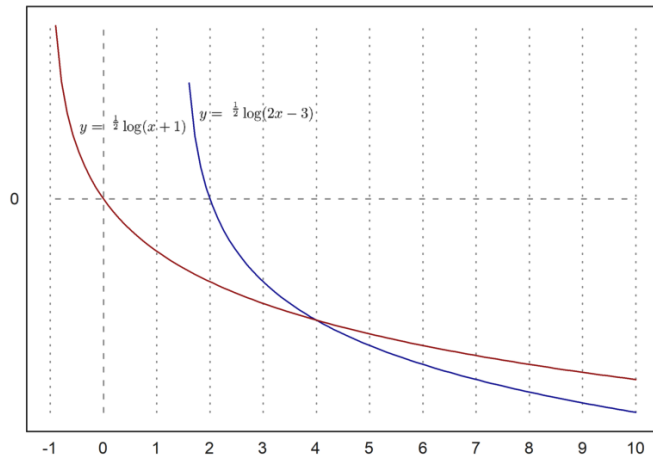
Karena $a = 1/3 \leq 1$ maka tanda pertidaksamaannya berubah menjadi

$$2x - 3 \leq x + 1,$$

sehingga solusinya

$$x \leq 4.$$

```
>plot2d(["logbase(2*x-3,1/2)", "logbase(x+1,1/2)"], ...  
>-0.9,10, color=[blue,red]);  
>label(latex("y=\frac{1}{2}\log(2x-3)"),1.6,2);  
>label(latex("y=\frac{1}{2}\log(x+1)"),-0.65,1.75):
```



Dari grafik fungsi

$$y = \frac{1}{2} \log(2x-3) \text{ dan } y = \frac{1}{2} \log(x+1)$$

terlihat bahwa kurva

$$y = \frac{1}{2} \log(2x-3) \text{ berada di atas kurva } y = \frac{1}{2} \log(x+1)$$

untuk $x \leq 4$.

4.3.5 Pertidaksamaan Trigonometri

Pertidaksamaan trigonometri merupakan pertidaksamaan yang memuat fungsi-fungsi trigonometri, misalnya sinus, kosinus, tangen, kotangen, sekan dan kosekan.

Perintah Maxima `fourier_elim` dari paket yang sama namanya tidak dapat menyelesaikan pertidaksamaan trigonometri secara langsung. Untuk menyelesaikan pertidaksamaan trigonometri, kita dapat menggambar fungsi trigonometri yang terkait dan menyelesaikan persamaan trigonometrinya.

Misalkan kita akan menyelesaikan pertidaksamaan

$$2\sin x \leq 1, \quad 0 \leq x \leq 2\pi.$$

```
>$fourier_elim([2*sin(x)<=1],[x])//solusi tdk eksplisit
```

$$[2\sin x - 1 = 0] \vee [1 - 2\sin x > 0]$$

```
>$solve(realroots(2*sin(x)-1=0),x) // solusi sin(x)=1/2
```

$$\left[x = \frac{\pi}{6} \right]$$

Dengan memperhatikan kurva

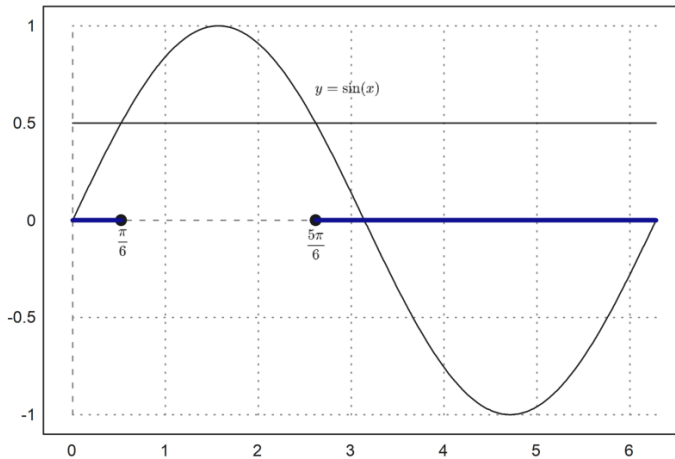
$$y = \sin(x) \text{ pada interval } 0 \leq x \leq 2\pi,$$

solusi pertidaksamaan di atas adalah

$$\{x \mid x \leq \frac{\pi}{6} \vee x \geq \frac{5\pi}{6}\}.$$

Berikut adalah gambar himpunan penyelesaian tersebut.

```
>plot2d(["sin(x)", "1/2"], 0, 2*pi);
>plot2d([pi/6, 5*pi/6], [0, 0], >points, style="o#", >add);
>x1=linspace(0, pi/6, 50); x2=linspace(5*pi/6, 2*pi, 50);
>y1=zeros(size(x1)); y2=zeros(size(x2));
>plot2d(x1, y1, thickness=3, color=blue, >add);
>plot2d(x2, y2, thickness=3, color=blue, >add);
>label(latex("y=\sin(x)"), 2.5, 0.75);
>label(latex("\frac{\pi}{6}"), pi/6-0.15, -0.01);
>label(latex("\frac{5\pi}{6}"), 5*pi/6-0.2, -0.01);
```



4.3.6 Pertidaksamaan Eksponen

Pertidaksamaan eksponen adalah pertidaksamaan yang memuat variabel dalam eksponen (pangkat). Bentuk umum pertidaksamaan eksponen dibagi menjadi dua, yakni sebagai berikut.

a) Untuk bilangan pokok $a > 1$:

$$a^{f(x)} < a^{g(x)} \text{ berlaku } f(x) < g(x)$$

b) Untuk bilangan pokok $0 \leq a \leq 1$:

$$a^{f(x)} < a^{g(x)} \text{ berlaku } f(x) > g(x)$$

Contoh: Tentukan penyelesaian pertidaksamaan

$$27^{x-3} < 81^{x+5}.$$

Penyelesaian:

Langkah pertama adalah menyamakan basis kedua bentuk perpangkatan, yang dalam hal ini adalah 3, sehingga bentuk pertidaksamaannya menjadi

$$(3^3)^{x-3} < (3^4)^{x+5}, \text{ atau } 3^{3(x-3)} < 3^{4(x+5)}.$$

Selanjutnya, solusi pertidaksamaan diperoleh dengan menyelesaikan pertidaksamaan

$$3(x-3) < 4(x+5) \text{ atau } x > -29.$$

Hasil ini sesuai dengan penyelesaian EMT menggunakan perintah `fourier_elim`.

```
>$fourier_elim ([3*x-9 < 4*x+20],[x])
```

$$[-29 < x]$$

Contoh lain: Selesaikan pertidaksamaan

$$\left(\frac{2}{3}\right)^{5x-7} < \left(\frac{2}{3}\right)^{x-2}.$$

Penyelesaian:

```
>$fourier_elim ([5*x-7 > x+2],[x])
```

$$\left[\frac{9}{4} < x\right]$$

Karena basisnya, $0 \leq a \leq 1$, maka tanda pertidaksamaan dibalik untuk mendapatkan solusinya.

4.4 Manipulasi dan Perhitungan Matriks dan Vektor

EMT menyediakan bahasa matriks, yang merupakan inti kemampuan numerik software Euler. Dengan bahasa matriks pembuatan tabel fungsi tidak perlu dilakukan menggunakan *loop*. Sudah tentu, kemampuan ini juga dapat digunakan untuk menghitung nilai suatu ekspresi dengan vektor dan matriks.

Perhitungan matriks secara lengkap adalah bagian dari materi aljabar linier. Pada bagian ini hanya akan dijelaskan secara ringkas berbagai perhitungan dasar matriks dan vektor.

Di dalam EMT, vektor dan matriks ditulis menggunakan tanda kurung siku, elemen dipisahkan dengan koma, baris dipisahkan dengan titik koma. Berikut adalah beberapa contoh mendefinisikan matriks di EMT.

```
>fixedformat(4,0); A=[1,2,3;4,5,6;7,8,9]
1    2    3
4    5    6
7    8    9
```

Apabila matriks ditulis secara tidak lengkap, maka elemen-elemen yang tidak ditulis akan dianggap bernilai nol. Untuk mengakses suatu matriks digunakan nama matriks sesuai definisinya. Nama matriks di EMT dapat menggunakan huruf besar, huruf kecil, string, gabungan huruf dan angka (namun harus diawali

dengan huruf dan tanpa spasi). Untuk mengakses elemen-elemen suatu matriks digunakan nama matriks, tanda kurung siku ([,]), nomor baris, nomor kolom. Nomor baris/kolom dapat tunggal atau berupa rentang nilai yang ditentukan dengan tanda titik dua (:). Selain cara tersebut, EMT juga menyediakan fungsi `mget()` untuk mengakses elemen-elemen suatu matriks.

Terdapat perintah/fungsi `mget(A,k)` yang berguna untuk mendapatkan elemen-elemen matriks *A* dengan indeks *k* (dalam hal ini *k* harus berupa matriks $n \times 2$ dan hasil fungsi `mget()` adalah matriks baris).

```
>A[2,3] //elemen baris ke-2 dan kolom ke-3 matriks A
      6
>mget(A,[2,3])
      6
>A[3,:] // elemen-elemen pada baris ke-3
      7,  8,  9]
>mget(A,[3,1;3,2;3,3])
      7,  8,  9]
>A[:,2] // elemen-elemen pada kolom ke-2
      2
      5
      8
>mget(A,[1,2;2,2;3,2])
      2,  5,  8]
>// elemen2 pada baris ke-2 sd 3, kolom ke-1 sd 2
>A[2:3,1:2]
      4  5
      7  8
>mget(A,[2,1;2,2;3,1;3,2])
      4,  5,  7,  8]
```

Elemen-elemen suatu matriks dapat diganti dengan menggunakan rujukan indeks.

```
>A[-1,-1] // indeks -1 berarti elemen terakhir
      9
>A
      1  2  3
      4  5  6
      7  8  9
```

```

>// mengganti elemen pd baris ke-2 kolom ke-1
>A[2,1]=5; A
      1      2      3
      5      5      6
      7      8      9
>// ganti kolom ke-2 matriks A dengan elemen-elemen B
>B=1:3; A[:,2]=B'
      1      1      3
      5      2      6
      7      3      9

```

Terdapat fungsi `EMT` yang juga dapat digunakan untuk mengganti elemen-elemen suatu matriks, yakni fungsi `mset()`.

Perintah `mset(A, k, B)` berguna untuk menghasilkan matriks baru dari matriks `A` dengan cara mengganti elemen-elemen matriks `A` pada indeks `k` (dalam hal ini, `k` adalah matriks `nx2`) dengan elemen-elemen `B` pada indeks `k`. Matriks `A` sendiri tidak berubah.

```

>mset(A, [2,2], 10) // A[2,2]=10
      1      1      3
      5     10      6
      7      3      9
>A
      1      1      3
      5      2      6
      7      3      9
>B=ones(3,3)
      1      1      1
      1      1      1
      1      1      1
>// A[i,j]=B[i,j] untuk i = j = 1, 2, 3
>mset(A, [1,1;2,2;3,3], B)
      1      1      3
      5      1      6
      7      3      1
>A
      1      1      3
      5      2      6
      7      3      9

```

Matriks yang terdiri atas satu baris merupakan vektor baris. Vektor baris berbeda dengan vektor kolom (matriks yang hanya terdiri atas satu kolom).


```
>v=[1,2,3]
      [1, 2, 3]
>u=[1;2;3]
      1
      2
      3
```

Tanda (operator) titik dua (:) akan menghasilkan elemen-elemen vektor baris dengan selisih sama (aslanya 1).

```
>1:5
      [1, 2, 3, 4, 5]
```

Selisih antar elemen dapat ditentukan dengan menggunakan dua tanda titik dua (:).

```
>1:2:10
      [1, 3, 5, 7, 9]
>10:-3:1
      [10, 7, 4, 1]
>1:0.1:2
      [1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2]
```

EMT menyediakan beberapa fungsi untuk menghasilkan matriks khusus, di antaranya sebagai berikut.

<code>zeros(n)</code> , <code>zeros(m,n)</code> :	menghasilkan matriks nol (semua elemennya nol)
<code>ones(n)</code> , <code>ones(m,n)</code> :	menghasilkan matriks yang semua elemennya 1
<code>id(n)</code> , <code>id(m,n)</code> :	menghasilkan matriks identitas
<code>eye(n)</code> , <code>eye(m,n)</code> :	menghasilkan matriks yang elemen-elemen di diagonalnya 1, lainnya 0
<code>diag(m,n,k,v)</code> :	menghasilkan matriks $m \times n$ dengan vektor v pada diagonal ke- k , elemen-elemen lain nol ($k=0$ berarti diagonal utama, $k < 0$ di bawah diagonal utama, $k > 0$ di atas diagonal utama)

<code>diagmatrix(v, k):</code>	menghasilkan matriks persegi dengan vektor v pada diagonal ke- k , elemen-elemen lain nol (jika k tidak ditulis dianggap $k=0$)
<code>getdiag(A, k):</code>	menghasilkan vektor yang isinya diagonal ke- k matriks A (jika k tidak ditulis dianggap $k=0$)
<code>band(A, d1, d2):</code>	mengganti elemen-elemen matriks A di bawah diagonal $d1$ atau di atas diagonal $d2$ dengan 0
<code>setdiag(A, k, x):</code>	menghasilkan matriks baru dari matriks A dengan cara mengganti elemen-elemen pada diagonal ke- k dengan x
<code>hilb(n):</code>	menghasilkan matriks Hilbert $n \times n$ yang diskala (dengan skala $1.4557e + 11$)
<code>linspace:</code>	untuk menghasilkan matriks baris dengan selisih elemen sama
<code>random:</code>	untuk menghasilkan matriks acak berdistribusi uniform pada $(0, 1)$
<code>random():</code>	menghasilkan sebuah bilangan acak
<code>random(n):</code>	menghasilkan vektor $1 \times n$ bilangan acak
<code>random(m, n):</code>	menghasilkan matriks $m \times n$ bilangan acak
<code>random([m, n]):</code>	menghasilkan matriks $m \times n$ bilangan acak
<code>inrandom:</code>	menghasilkan matriks bilangan bulat secara acak
<code>inrandom(k):</code>	menghasilkan sebuah bilangan bulat acak dari 1 sampai k
<code>magic(n):</code>	menghasilkan matriks bujur sangkar ajaib $n \times n$.
<code>inrandom(n, k):</code>	menghasilkan vektor $1 \times n$ bilangan bulat secara acak dari 1 sampai k
<code>inrandom(m, n, k):</code>	menghasilkan matriks $m \times n$ bilangan acak dari 1 sampai k
<code>inrandom([m, n], k):</code>	menghasilkan matriks $m \times n$ bilangan acak dari 1 sampai k
<code>primes(n):</code>	menghasilkan (vektor) bilangan-bilangan prima yang kurang atau sama dengan n
<code>unique(v):</code>	menghasilkan elemen-elemen unik (berbeda), diurutkan dari terkecil sampai terbesar, pada vektor v (harus berupa matriks baris).

Berikut adalah contoh-contoh menghasilkan matriks dengan fungsi-fungsi tersebut.

```

>zeros(3)
    [0,  0,  0]
>zeros(2,3)
    0  0  0
    0  0  0
>ones(3)
    [1,  1,  1]
>ones(3,2)
    1  1
    1  1
    1  1
>id(3)
    1  0  0
    0  1  0
    0  0  1
>id(2,3)
    1  0  0
    0  1  0
>eye(3)
    1  0  0
    0  1  0
    0  0  1
>eye(3,2)
    1  0
    0  1
    0  0
>diag(3,3,0,1:3)
    1  0  0
    0  2  0
    0  0  3
>diag(4,4,1,1:3)
    0  1  0  0
    0  0  2  0
    0  0  0  3
    0  0  0  0
>diag(3,3,-1,1:3)
    0  0  0
    1  0  0
    0  2  0
>diagmatrix(1:3,0)
    1  0  0
    0  2  0
    0  0  3

```

```

>diagmatrix(1:3)
      1   0   0
      0   2   0
      0   0   3
>diagmatrix(1:3,1)
      0   1   0   0
      0   0   2   0
      0   0   0   3
      0   0   0   0
>diagmatrix(1:3,-2)
      0   0   0   0   0
      0   0   0   0   0
      1   0   0   0   0
      0   2   0   0   0
      0   0   3   0   0
>setdiag(A,1,0)
      1   0   3
      5   2   0
      7   3   9
>A
      1   1   3
      5   2   6
      7   3   9
>setdiag(A,0,0)
      0   1   3
      5   0   6
      7   3   0
>band(A,0,2)
      1   1   3
      0   2   6
      0   0   9
>band(A,-1,1)
      1   1   0
      5   2   6
      0   3   9
>// matriks Hilbert 3x3 asli (skala=1)
>fraction hilb(3,1)
      1      1/2      1/3
      1/2      1/3      1/4
      1/3      1/4      1/5
>linspace(1,10,3) // sama dengan 1:3:10
      [1,  4,  7, 10]

```

```

>shortformat, random()
0.010438
>// matriks acak 1x3 berdistribusi uniform (0,1)
>random(3)
[0.35663, 0.52143, 0.42889]
>// matriks acak 2x2 berdistribusi uniform (0,1)
>random(2,2)
0.16813 0.18274
0.28805 0.75004
>inrandom(10)
5
>// matriks acak bulat antara 0 dan 5 berukuran 1x2
>fixedformat(6,0); inrandom(2,5)
[3, 1]
> // matriks acak bulat antara 0 dan 10 berukuran 2x3
>inrandom(2,3,10)
6 10 10
5 2 5
> // matriks acak bulat antara 0 dan 10 berukuran 3x3
>inrandom(3,3,10)
8 3 9
5 4 8
4 3 1
>magic(3)
8 1 6
3 5 7
4 9 2
>magic(4)
16 2 3 13
5 11 10 8
9 7 6 12
4 14 15 1
>magic(5)
17 24 1 8 15
23 5 7 14 16
4 6 13 20 22
10 12 19 21 3
11 18 25 2 9
>primes(10)
[2, 3, 5, 7]
>v=inrandom(1,10,10)
[5, 1, 1, 9, 3, 4, 7, 3, 7, 3]

```

```

>unique(v)
      [1,  3,  4,  5,  7,  9]
>A=inrandom(10,1,10)
      6
      9
      8
      4
      7
      6
      4
      7
      8
      1
>unique(A)
      Need real row vector for unique
      Try "trace errors" to inspect local variables
        after errors.
      unique:
        return _unique(v);
      Error in:
      unique(A) ...
        ^

>A=1:3
      [1,  2,  3]

```

Selain menggunakan fungsi-fungsi penghasil matriks, suatu matriks juga dapat diperoleh dengan memanipulasi matriks lain yang sudah diketahui. Terdapat beberapa operator dan fungsi untuk memanipulasi matriks di EMT. Yang perlu diketahui, di EMT semua operator dan fungsi matriks dilakukan terhadap elemen-elemen matriks (operasi elemen demi elemen).

```

>B=3:5
      [3,  4,  5]
>A+B // jumlah kedua matriks
      [4,  6,  8]
>B-A // selisih matriks A dan B
      [2,  2,  2]
>A*B //perkalian antar elemen (bukan perkalian matriks)
      [3,  8, 15]
>A^2 // kuadrat elemen-elemen matriks A
      [1,  4,  9]

```

```

>2^A // 2 dipangkatkan elemen-elemen matriks A
[2, 4, 8]
>A^B // perpangkatan antar elemen A dan elemen B
[1, 16, 243]
>shortformat; sqrt(A) // akar kuadrat elemen-elemen A
[1, 1.4142, 1.7321]
>A' // transpose A
1
2
3
>transpose(A) // sama dengan A'
1
2
3
>A.A' // hasil kali matriks A dan A'
14
>C=A'.A // hasil kali matriks A' dan A
1      2      3
2      4      6
3      6      9
>sum(C) // jumlah elemen-elemen setiap baris matriks C
6
12
18
>matrixpower(C,2) // C*C
14      28      42
28      56      84
42      84      126
>power(C,2) // sama dengan `matrixpower`
14      28      42
28      56      84
42      84      126
>D=[1,2;3,4]
1      2
3      4
>fraction matrixpower(D,-1) // D^(-1)
-2      1
3/2      -1/2
>fraction inv(D) // invers matriks: inv(D)=D^(-1)
-2      1
3/2      -1/2

```

```

>fraction inv(inv(D)) // hasilnya matriks D
      1      2
      3      4
>fraction matrixpower(D,-2) // D^(-2)
      11/2      -5/2
      -15/4      7/4
>// D^(-2)D^2=D^0=I
>fraction matrixpower(D,-2).matrixpower(D,2)
      1      0
      0      1
>matrixpower(D,0)
      1      0
      0      1
>A/A //pembagian elemen-elemen matriks yang seletak
      [1,  1,  1]
>A
      [1,  2,  3]
>C
      1      2      3
      2      4      6
      3      6      9
>C/A // pembagian antar elemen matriks C dan A
      1      1      1
      2      2      2
      3      3      3
>b=A'
      1
      2
      3
>A=hilb(3,1); fraction A // matriks Hilbert
      1      1/2      1/3
      1/2      1/3      1/4
      1/3      1/4      1/5
>inv(A)
      9      -36      30
      -36      192      -180
      30      -180      180
>A\\b // hasilkali invers C dan b, C^(-1)b
      27
      -192
      210

```

Suatu matriks juga dapat dibentuk dari sebuah vektor dengan cara menduplikasi

vektor tersebut secara vertikal atau horisontal, sesuai vektornya berupa vektor kolom atau vektor baris. Untuk melakukan hal ini digunakan perintah/fungsi `dup(v, n)`, yang akan menghasilkan matriks baru dari vektor v diulang sebanyak n kali. Dalam hal ini v harus berupa vektor (matriks baris atau matriks kolom). Perhatikan kedua contoh di bawah ini.

```
>A=intrandom(3,1,6), B=dup(A,2)
```

```
1
4
5
1      1
4      4
5      5
```

```
>A=intrandom(1,3,6), B=dup(A,2)
```

```
[4, 5, 6]
4      5      6
4      5      6
```

Terdapat fungsi `multdup(x, n)` yang berguna untuk mengulang baris-baris atau kolom-kolom x sebanyak yang ditentukan pada n .

- Jika n berupa skalar, maka hasilnya akan mengulang (ke kanan) setiap elemen matriks x sebanyak n kali.
- Jika n berupa matriks baris dengan elemen sebanyak kolom matriks x , maka setiap elemen pada setiap kolom matriks x akan diulang (ke kanan) sebanyak nilai elemen pada kolom yang bersesuaian dengan kolom tersebut.
- Jika n berupa matriks kolom dengan elemen sebanyak baris matriks x , maka setiap elemen pada setiap baris matriks x akan diulang (ke bawah) sebanyak nilai elemen pada baris yang bersesuaian dengan baris tersebut.

```
>multdup(A,3) //setiap elemen A diulang 3 kali ke kanan
```

```
[4, 4, 4, 5, 5, 5, 6, 6, 6]
```

```
>// elemen baris 1 diulang 3 kali ke bawah
```

```
>multdup(A',3)
```

```
4
4
4
```

```

>// elemen2 A diulang 2, 3, dan 4 kali ke bawah
>multdup(A',[2;3;4])
    4
    4
    5
    5
    5
    6
    6
    6
    6

>B=intrandom(2,3,10)
    5    7    10
    4    4     5

>multdup(B,2) //setiap elemen B diulang ke kanan 2 kali
    5    5    7    7    10    10
    4    4    4    4     5     5

>//kolom 1-3 matriks B diulang 1, 2, dan 2 kali ke kanan
>multdup(B,[2,3,2])
    5    5    7    7    7    10    10
    4    4    4    4    4     5     5

>// kolom 1-2 matriks B diulang 2 dan 3 kali,
>// kolom ke-3 diulang nol kali!
>multdup(B,[2,3])
    5    5    7    7    7
    4    4    4    4    4

>//baris 1 dan 2 matriks B diulang ke bawah 2 dan 3 kali
>multdup(B,[2;3])
    5    7    10
    5    7    10
    4    4     5
    4    4     5
    4    4     5

>//tidak cocok karena B hanya memiliki 2 baris
>multdup(B,[2;3;4])
Row index 3 out of bounds!
Try "trace errors" to inspect local variables
after errors.
multdup:
  if cols(n)==1 then return x[ind,:];
Error in:
multdup(B,[2;3;4])
^

```

Di EMT, suatu matriks dapat diubah (dibalik) urutan kolomnya dengan menggunakan fungsi `flipx()` atau `fliplr()`. Untuk mengubah (membalik) urutan baris suatu matriks dapat digunakan fungsi `flipy()` atau `flipud()`. Untuk menukar baris-baris suatu matriks dapat digunakan fungsi `swapRows()`, dengan sintaks `swapRows(A, i, j)`, hasilnya baris ke-*i* ditukar dengan baris ke-*j*. Perhatikan penulisan fungsinya menggunakan huruf besar!

Berikut adalah contoh-contoh penggunaan fungsi-fungsi EMT tersebut.

```
>A, flipx(A)
  [4,  5,  6]
  [6,  5,  4]
>B
   5   7  10
   4   4   5
>flipx(B) // balik susunan kolom
  10   7   5
   5   4   4
>fliplr(B) // balik susunan kolom
  10   7   5
   5   4   4
>flipy(B) // balik susunan baris
   4   4   5
   5   7  10
>flipud(B) // balik susunan baris
   4   4   5
   5   7  10
>rotright(B) // geser (putar) kolom-kolom ke kiri
   7  10   5
   4   5   4
>M=magic(5)
   17   24    1    8   15
   23    5    7   14   16
    4    6   13   20   22
   10   12   19   21    3
   11   18   25    2    9
>swapRows(M, 2, 5)
   17   24    1    8   15
   11   18   25    2    9
    4    6   13   20   22
   10   12   19   21    3
   23    5    7   14   16
```

Selain dibalik, posisi kolom-kolom suatu matriks dapat digeser (diputar) ke kiri

atau ke kanan dengan fungsi `rotleft()` atau `rotright()`. Suatu matriks juga dapat diputar (bayangkan matriks sebagai suatu persegi panjang). Fungsi EMT untuk memutar matriks adalah `rot()` dan `rot90()`. Perintah `rot(B)` dan `rot90(B)` akan memutar matriks B berlawanan arah jarum jam 90 derajat. Perintah `rot(B, k)` akan memutar matriks B berlawanan arah jarum jam k kali 90 derajat.

Selain dengan cara dibalik atau diputar susunannya, elemen-elemen suatu matriks juga dapat diacak posisinya dengan menggunakan fungsi `shuffle()`.

```
>// geser kolom-kolom ke kiri,
>// kolom terakhir menjadi 0
>shiftleft(B)
      7      10      0
      4       5      0
>rotright(B) // geser (putar) kolom-kolom ke kanan
      10      5      7
      5       4      4
>rot(B)
      10      5
      7       4
      5       4
>B'
      5       4
      7       4
      10      5
>rot90(B)
      10      5
      7       4
      5       4
>rot(B,2)
      5       4      4
      10      7      5
>shuffle(1:5) // acak posisi elemen-elemennya
      [1,  4,  3,  2,  5]
```

EMT juga menyediakan fungsi-fungsi untuk menghapus beberapa elemen suatu matriks. Fungsi-fungsi ini adalah `drop`, `droprows`, `dropcols`, `butfirst`, dan `butlast`.

<code>drop(v, k):</code>	menghapus elemen ke-k (k dapat berupa vektor) pada vektor v;
<code>droprows(A, k):</code>	menghapus baris ke-k (k dapat berupa vektor) pada matriks A;
<code>dropcols(A, k):</code>	menghapus kolom ke-k (k dapat berupa vektor) pada matriks A;
<code>butfirst(A):</code>	jika A berupa vektor, maka menghapus elemen pertama; jika A berupa matriks dengan banyak baris lebih besar atau sama dengan 3, maka akan menghapus baris pertama matriks A;
<code>butlast(A):</code>	jika A berupa vektor, maka menghapus elemen terakhirnya; jika A berupa matriks dengan banyak baris lebih besar atau sama dengan 3, maka akan menghapus baris terakhirnya;

```

>A
    [4,  5,  6]
>B
    5      7     10
    4      4      5
>drop(A, 2)
    [4,  6]
>P=magic(5)
    17      24      1      8      15
    23       5      7     14     16
     4       6     13     20     22
    10      12     19     21      3
    11      18     25      2      9
>droprows(P, [1, 3])
    23       5      7     14     16
    10      12     19     21      3
    11      18     25      2      9
>dropcols(P, [1, 3])
    24       8     15
     5      14     16
     6      20     22
    12      21      3
    18       2      9
>butfirst(A)
    [5,  6]
>butfirst(A')
    5
    6

```

```

>butfirst(P)
    23     5     7     14     16
     4     6    13    20    22
    10    12    19    21     3
    11    18    25     2     9
>butlast(A)
    [4,  5]
>butlast(B)
     5     7    10
     4     4     5
>butlast(P)
    17    24     1     8    15
    23     5     7    14    16
     4     6    13    20    22
    10    12    19    21     3

```

Terdapat operator `_` dan `|` untuk menggabungkan dua matriks menjadi matriks baru. Perintah `v_w` akan menghasilkan matriks baru yang dibentuk dengan menyusun matriks `v` di atas matriks `w`. Jika salah satu matriks memiliki kolom lebih sedikit daripada matriks yang lain, maka kolom matriks lebih sedikit akan disamakan dengan menambahkan kolom-kolom nol. Apabila `v` atau `w` berupa skalar, nilai ini digunakan sebagai elemen-elemen baris yang ditambahkan. Jadi `v_5` akan menambahkan satu baris berisi 5 semua dibawah `v`.

Jadi, pada matriks baru `v_w` banyaknya baris adalah jumlah baris `v` dan `w`, banyaknya kolom adalah maksimum banyaknya kolom `v` dan `w`. Perintah `v|w` akan menghasilkan matriks baru yang dibentuk dengan menyusun matriks `v` di sebelah kiri matriks `w`. Analog dengan `v_5`, `v|5` akan menambahkan kolom baru yang terdiri elemen-elemen 5 di sebelah kanan `v`. Jadi, pada matriks baru `v|w` banyaknya kolom adalah jumlah kolom `v` dan `w`, banyaknya baris adalah maksimum banyaknya baris `v` dan `w`.

Perhatikan contoh-contoh di bawah ini.

```

>inrandom(2,3,6)\_[1,2]
     2     2     3
     5     4     4
     1     2     0
>inrandom(3,1,6)\_[1,2]
     2     0
     3     0
     2     0
     1     2

```

```

>intrandom(3,2,6) |[1,2]
    6      1      1      2
    3      5      0      0
    2      6      0      0
>intrandom(3,1,6) |[1,2]
    2      1      2
    6      0      0
    2      0      0
>(1:3)\_2
    1      2      3
    2      2      2
>(1:3) '|2
    1      2
    2      2
    3      2

```

Selain dua cara di atas, terdapat cara lain untuk menggabungkan dua matriks menjadi sebuah matriks baru. Untuk menggabungkan dua matriks menjadi matriks baru secara horisontal, tuliskan kedua matriks di dalam kurung siku dan pisahkan keduanya dengan tanda koma, misalnya $[v, w]$. Apabila banyaknya baris keduanya tidak sama, maka akan disamakan dengan menambah baris nol. Penggabungan dua matriks secara vertikal menggunakan tanda titik koma hanya dapat dilakukan terhadap dua vektor baris. Untuk menggabungkan dua vektor baris menjadi matriks baru secara vertikal, tuliskan kedua vektor di dalam kurung siku dan pisahkan keduanya dengan tanda titik koma, misalnya $[v; w]$. Apabila banyaknya elemen keduanya tidak sama, maka akan disamakan dengan menambah elemen-elemen nol. Perhatikan contoh-contoh berikut ini.

```

>v=[1,3], w=[-3,-1], u=[v;w]
    1      3
   -3     -1
    1      3
   -3     -1
>x=[v',w']
    1     -3
    3     -1
>[u,v]
    1      3      1      3
   -3     -1      0      0

```

```
>[u;v]
Semicolon not allowed in matrix of column vectors
or matrices.Error in:
[u;v] ...
^
```

Di sini terjadi kesalahan karena vektor baris tidak dapat ditambahkan sebagai baris baru pada suatu matriks selain matriks baris.

```
>[v,w;v]
      1      3      -3      -1
      1      3       0       0
```

Penggabungan dua vektor dapat digunakan untuk membuat tabel menggunakan vektor ekspresi aljabar yang akan disubstitusikan dengan elemen-elemen suatu vektor. Perhatikan contoh berikut ini untuk membuat tabel perpangkatan.

```
>"[x,x^2,x^3]"((1:5)')
      1      1      1
      2      4      8
      3      9     27
      4     16     64
      5     25    125
>"[x;x^2;x^3]"((1:5))
      1      2      3      4      5
      1      4      9     16     25
      1      8     27     64    125
```

Suatu matriks juga dapat disusun menggunakan elemen-elemen yang persis sama dengan elemen-elemen matriks lain dengan mengubah dimensi (ordo)-nya. Perintah/fungsi EMT untuk melakukan hal ini adalah `redim(A,m,n)` atau `redim(A,[m,n])`. Fungsi ini akan menyusun ulang elemen-elemen matriks A menjadi matriks baru berukuran $m \times n$. Penyusunan ulang dilakukan dengan membaca elemen-elemen matriks A baris demi baris sampai terbentuk matriks $m \times n$. Apabila banyaknya elemen matriks A kurang dari $m \times n$, maka kekurangan elemen akan diisi dengan nol.


```
>A=intrandom(4,3,10)
```

```
4      9      9
3      7      7
1      5      8
7      4      9
```

```
>redim(A,4,4)
```

```
4      9      9      3
7      7      1      5
8      7      4      9
0      0      0      0
```

Matriks membutuhkan lebih banyak elemen sehingga dilengkapi dengan elemen-elemen nol.

```
>redim(A,3,3)
```

```
4      9      9
3      7      7
1      5      8
```

Matriks baru membutuhkan lebih sedikit elemen sehingga beberapa elemen terakhir matriks A tidak digunakan.

```
>redim(A,3,4)
```

```
4      9      9      3
7      7      1      5
8      7      4      9
```

Matriks baru membutuhkan elemen sama banyak dengan banyak elemen matriks A, hanya berbeda ukuran (ordo).

EMT juga menyediakan perintah/fungsi yang dapat dipandang sebagai kebalikan fungsi `redim()`, yakni fungsi/perintah `flatten()`. Perintah ini akan mengubah suatu matriks menjadi vektor baris. Berikut adalah contohnya.

```
>flatten (A)
```

```
[4, 9, 9, 3, 7, 7, 1, 5, 8, 7, 4, 9]
```

Operator-operator perbandingan `<`, `>`, `==`, `<=`, `>=` dapat digunakan untuk menghasilkan matriks khusus (matriks biner, yang elemen-elemennya nol dan 1) dari matriks lain. Perhatikan beberapa contoh di bawah ini.

```

>A=intrandom(3,3,10)
      5      1      2
      8      4     10
      3      5      9
>A<3 //menguji elemen2 A yang < 3
      0      1      1
      0      0      0
      0      0      0
>A>=5 //menguji elemen2 A yang >= 5
      1      0      0
      1      0      1
      0      1      1
>A==5 //menguji elemen-elemen A yang = 5
      1      0      0
      0      0      0
      0      1      0

```

Seperti Anda lihat, hasil operator-operator perbandingan terhadap matriks adalah matriks biner yang berordo sama dengan matriks yang diuji. Elemen 1 menunjukkan elemen memenuhi syarat perbandingan, sedangkan elemen 0 menunjukkan elemen yang tidak memenuhi syarat perbandingan. Untuk mendapatkan indeks elemen-elemen yang memenuhi syarat perbandingan (elemen matriks hasil pengujian bernilai 1) dapat digunakan fungsi `mnonzeros()`. Fungsi ini akan menghasilkan matriks dua kolom yang setiap barisnya menunjukkan indeks (nomor baris dan kolom) elemen yang memenuhi syarat perbandingan/pengujian.

```

>idx=mnonzeros(A>=5)
      1      1
      2      1
      2      3
      3      2
      3      3

```

Seperti terlihat pada matriks hasil $A \geq 5$, elemen-elemen matriks A yang nilainya lebih besar atau sama dengan 5 adalah

$$A(1,1) = 5, A(2,1) = 8, A(2,3) = 10, A(3,2) = 5, \text{ dan } A(3,3) = 9.$$

(Tentu hasil ini akan berubah jika Anda menekan kembali tombol [Enter] mulai dari baris yang mendefinisikan matriks A di atas, karena A adalah matriks acak bulat.)

EMT juga menyediakan fungsi `nonzeros()` untuk menguji elemen-elemen suatu vektor yang bernilai bukan nol. Fungsi ini sangat bermanfaat untuk

menyelidiki sifat-sifat suatu barisan. Berikut adalah dua contoh khusus. Yang pertama mencari bilangan-bilangan antara 1 dan 1000 yang merupakan kelipatan 9 dan 13. Contoh kedua menemukan bilangan-bilangan prima antara 1 dan 1000.

```
>n=1:1000; nonzeros(mod(n,9)==0 && mod(n,13)==0)
[117, 234, 351, 468, 585, 702, 819, 936]
>nonzeros(isprime(n))
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
79, 83, 89, 97, 101, 103, 107, 109, 113,
127, 131, 137, 139, 149, 151, 157, 163, 167,
173, 179, 181, 191, 193, 197, 199, 211, 223,
227, 229, 233, 239, 241, 251, 257, 263, 269,
271, 277, 281, 283, 293, 307, 311, 313, 317,
331, 337, 347, 349, 353, 359, 367, 373, 379,
383, 389, 397, 401, 409, 419, 421, 431, 433,
439, 443, 449, 457, 461, 463, 467, 479, 487,
491, 499, 503, 509, 521, 523, 541, 547, 557,
563, 569, 571, 577, 587, 593, 599, 601, 607,
613, 617, 619, 631, 641, 643, 647, 653, 659,
661, 673, 677, 683, 691, 701, 709, 719, 727,
733, 739, 743, 751, 757, 761, 769, 773, 787,
797, 809, 811, 821, 823, 827, 829, 839, 853,
857, 859, 863, 877, 881, 883, 887, 907, 911,
919, 929, 937, 941, 947, 953, 967, 971, 977,
983, 991, 997]
>length(nonzeros(isprime(n)))
168
```

Jadi terdapat 168 bilangan prima antara 1 dan 1000.

EMT juga menyediakan fungsi yang dapat dipandang sebagai kebalikan dari fungsi `nonzeros()` atau `mnonzeros()`, yakni `any()` dan `all()`.

`any(A)`: menghasilkan nilai 1 jika terdapat elemen A yang bernilai tidak nol (dan nol jika sebaliknya)

`all(A)`: menghasilkan nilai 1 jika semua elemen A bukan nol (dan nol jika sebaliknya). Jadi, `all(A)` ekuivalen dengan `!any(A==0)`. Tanda seru (!) artinya negasi dari pernyataan.

Untuk menemukan elemen pertama dan elemen terakhir yang bukan nol pada suatu matriks dapat digunakan fungsi `firstnonzero()` dan `lastnonzero()`.

`firstnonzero(A)`: indeks elemen pertama yang bukan nol pada setiap baris matriks A
`lastnonzero(A)`: indeks elemen terakhir yang bukan nol pada setiap baris matriks A.

```
>A=round(5*random(3,3)), any(A), all(A)
      1      4      2
      4      2      5
      1      2      1
1
1
>any(A>5)
0
>all(A<=5)
1
>firstnonzero(A)
      1
      1
      1
```

Sesuai hasil tersebut, untuk matriks A di atas, elemen pertama yang bukan nol pada baris ke-1, 2, dan 3 adalah berturut-turut $A(1,1)$, $A(2,2)$, dan $A(3,1)$.

```
>lastnonzero(A)
      3
      3
      3
```

4.4.1 Fungsi-fungsi Dasar Matriks

EMT menyediakan banyak fungsi untuk berbagai perhitungan matriks, mulai dari fungsi-fungsi dasar sampai dengan fungsi-fungsi lanjut terkait Aljabar Linier. Berikut adalah beberapa fungsi dasar untuk berbagai perhitungan matriks.

`rows(A)`: banyaknya baris matriks A
`cols(A)`: banyaknya kolom matriks A
`size(A)`: menghasilnya ukuran (nilai banyaknya baris dan kolom) matriks A
`size(A,B,C,...)`: maksimum ukuran matriks A, B, C, ...
`length(A)`: nilai maksimum banyaknya baris dan banyaknya kolom matriks A

`min(A)`: nilai minimum elemen-elemen setiap baris matriks A (hasilnya matriks kolom)
`max(A)`: nilai maksimum elemen-elemen setiap baris matriks A (hasilnya matriks kolom)
`extrema(A)`: [nilai minimum, posisi minimum, nilai maksimum, posisi maksimum] di setiap baris matriks A (hasilnya matriks kolom)
`sum(A)`: jumlah elemen-elemen setiap baris matriks A (hasilnya matriks kolom)
`colsum(A)`: jumlah elemen-elemen setiap kolom matriks A (hasilnya matriks baris)
`cumsum(A)`: matriks dengan elemen-elemen jumlah kumulatif setiap baris matriks A
`cummax(A)`: matriks dengan elemen-elemen nilai maksimum kumulatif pada baris-baris A
`cummin(A)`: matriks dengan elemen-elemen nilai minimum kumulatif pada baris-baris A
`prod(A)`: hasil kali elemen-elemen setiap baris matriks A (hasilnya matriks kolom)
`cumprod(A)`: matriks dengan elemen-elemen hasil kali kumulatif pada baris-baris A

```

>A=intrandom(4,3,10)
      3      2      6
      1      9      5
      2      5      3
      1      8      3
>rows(A)
      4
>cols(A)
      3
>size(A)
      [4,  3]
>length(A)
      4
>min(A)//nilai2 minimum pada setiap baris matriks A
      2
      1
      2
      1
>max(A)//nilai2 maksimum pd setiap baris A
  
```

```

6
9
5
8
>//nilai2 min, maks, & pos pada setiap baris matriks A
>extrema(A)
      2      2      6      3
      1      1      9      2
      2      1      5      2
      1      1      8      2
>//sort() hanya berlaku untuk vektor
>sort(A)
      Need 1xn or nx1 vector for sort
      Error in:
      sort(A) //sort() hanya berlaku untuk vektor
      ^
>sort([3,-2,5,10,-4])
      [-4,  -2,   3,   5,  10]
>A
      3      2      6
      1      9      5
      2      5      3
      1      8      3
>sum(A) // jumlah setiap baris matriks A
      11
      15
      10
      12
>colsum(A) // jumlah setiap kolom matriks A
      [7,  24,  17]
>// matriks jumlah kumulatif setiap baris matriks A
>cumsum(A)
      3      5      11
      1     10     15
      2      7     10
      1      9     12
>// matriks maksimum kumulatif pada baris-baris A
>cummax(A)
      3      3      6
      1      9      9
      2      5      5
      1      8      8

```

```

> //matriks minimum kumulatif pada baris-baris A
> cummin(A)
      3      2      2
      1      1      1
      2      2      2
      1      1      1
> //hasil kali elemen2 setiap baris matriks A
> prod(A)
      36
      45
      30
      24
> //matriks hasil kali kumulatif setiap baris matriks A
> cumprod(A)
           3           6           36
           1           9           45
           2          10           30
           1           8           24

```

4.4.2 Matriks Simbolik

Dengan bantuan Maxima, EMT juga dapat bekerja dengan matriks simbolis. Matriks simbolik didefinisikan dengan simbol $\& :=$ atau $\& =$ dan elemen-elemen ditulis di dalam tanda kurung siku $[\dots]$, kemudian dapat digunakan dalam setiap ekspresi simbolis. Simbol $\& :=$ digunakan untuk mendefinisikan matriks yang dapat dihitung secara simbolik maupun numerik. Elemen-elemen suatu matriks simbolik dapat berupa bilangan (konstanta) maupun variabel (huruf/symbol). Berikut ditunjukkan contoh mendefinisikan sebuah matriks simbolik dan melakukan perhitungan-perhitungan matriks tersebut.

```

> // mendefinisikan matriks simbolik konstanta A
> A &:= [1,2;3,4]; $A

```

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$$

```

> //determinan dan invers A (simbolik dan numerik)
> $det(A), $invert(A), $invert(A)()

```

$$\begin{pmatrix} -2 & 1 \\ \frac{3}{2} & -\left(\frac{1}{2}\right) \end{pmatrix}$$

```
>// matriks simbolik yang memuat elemen pi
>A &:= [1,pi;4,5]
      1      3.14159
      4      5
>$A
```

$$\begin{pmatrix} 1 & \frac{1146408}{364913} \\ 4 & 5 \end{pmatrix}$$

Apabila matriks simbolik memuat elemen berupa bilangan desimal, maka pada tampilan simbolik matriks tersebut elemen bilangan desimal disajikan dalam bentuk pecahan rasional. Untuk menghindari hal ini dapat digunakan fungsi Euler `mxmset()`.

```
>mxmset(A), $A
```

$$\begin{pmatrix} 1 & 3.141592653589793 \\ 4 & 5 \end{pmatrix}$$

```
>// mendefinisikan dan menampilkan matriks simbolik A
>A &= [a,1,1;1,a,1;1,1,a]; $A
```

$$\begin{pmatrix} a & 1 & 1 \\ 1 & a & 1 \\ 1 & 1 & a \end{pmatrix}$$

Berikut adalah contoh matriks simbolik yang memuat elemen simbolik dan contoh perhitungan matriks (dalam Aljabar Linier).

```
>//determinan A (simbolik) dan memfaktorkannya
>$det(A), $factor(%)
```

$$(a-1)^2 (a+2)$$

Hasilnya cocok dengan perhitungan matematika:

$$\begin{aligned}
 \begin{vmatrix} a & 1 & 1 \\ 1 & a & 1 \\ 1 & 1 & a \end{vmatrix} &= a \cdot \begin{vmatrix} a & 1 \\ 1 & a \end{vmatrix} - 1 \cdot \begin{vmatrix} 1 & 1 \\ 1 & a \end{vmatrix} + 1 \cdot \begin{vmatrix} 1 & a \\ 1 & 1 \end{vmatrix} \\
 &= a(a^2 - 1) - 1(a - 1) + 1(1 - a) \\
 &= (a - 1)(a^2 + a - 2) \\
 &= (a - 1)(a - 1)(a + 2) \\
 &= (a - 1)^2(a + 2).
 \end{aligned}$$

```
>$invert(A) with a=0 // invers A untuk nilai a=0
```

$$\begin{pmatrix} -\left(\frac{1}{2}\right) & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\left(\frac{1}{2}\right) & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\left(\frac{1}{2}\right) \end{pmatrix}$$

```
>// mengganti elemen tertentu matriks A
>&A[2,2] := 2+b; $A
```

$$\begin{pmatrix} a & 1 & 1 \\ 1 & b+2 & 1 \\ 1 & 1 & a \end{pmatrix}$$

```
>//menyubstitusikan variabel di dalam matriks simbolik
>$A with [a=2, b=-3]
```

$$\begin{pmatrix} 2 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

```
>$A // matriks simboliknya tidak berubah
```

$$\begin{pmatrix} a & 1 & 1 \\ 1 & b+2 & 1 \\ 1 & 1 & a \end{pmatrix}$$

Berikut adalah perhitungan nilai-nilai eigen suatu matriks yang merupakan solusi persamaan karakteristik matriks tersebut. Persamaan karakteristik matriks A adalah:

$$|A - x.I| = 0 \text{ dengan } I \text{ adalah matriks identitas yang seukuran matriks } A.$$

Nilai x yang memenuhi persamaan karakteristik disebut **nilai eigen**. Vektor v yang memenuhi $(A - x.I).v = 0$ disebut **vektor eigen**.

```
>$det(A-x*ident(3))|factor, eigen &= ...
>      solve(%,x) with [a=2, b=-3]; $eigen
```

$$\left[x = -\left(\frac{2\sqrt{6}-2}{2}\right), x = \frac{2\sqrt{6}+2}{2}, x = 1 \right]$$

```
>lambda1 &= rhs(eigen[1]);
>lambda2 &= rhs(eigen[2]); lambda3 &= rhs(eigen[3]);
>$lambda1, $lambda2, $lambda3
```

$$\begin{array}{c} \frac{2\sqrt{6}-2}{2} \\ \frac{2\sqrt{6}+2}{2} \\ 1 \end{array}$$

```
>$det(A-lambda1*ident(3)) with [a=2, b=-3]|...
>      expand |simplify
```

$$0$$

```
>$det(A-lambda2*ident(3)) with [a=2, b=-3]|....
>      expand |simplify
```

$$0$$

```
>$det(A-lambda3*ident(3)) with [a=2, b=-3]|...
>      expand |simplify
```

0

Ketiga nilai eigen tersebut memenuhi persamaan karakteristik matriks A (dengan nilai $a = 2$, $b = -3$).

Maxima menyediakan perintah `eigenvalues()` untuk menghitung secara otomatis nilai-nilai eigen suatu matriks, dan perintah `eigenvectors()` untuk menghasilkan vektor-vektor eigen suatu matriks. Hasil perintah `eigenvalues()` berupa nilai-nilai eigen dan pengalinya. Hasil perintah `eigenvectors()` adalah nilai-nilai eigen, pengalinya, dan vektor-vektor eigen untuk masing-masing nilai eigen.

```
>$eigenvalues(A) with [a=2, b=-3]
```

$$\left[\left[-\left(\frac{2\sqrt{6}-2}{2} \right), \frac{2\sqrt{6}+2}{2}, 1 \right], [1, 1, 1] \right]$$

```
>$[eig, veig]:eigenvectors(A) with [a=2, b=-3];  
>$eig, $veig
```

$$\left[\left[\left[1, -\left(\frac{2\sqrt{6}+4}{2} \right), 1 \right], \left[1, \frac{2\sqrt{6}-4}{2}, 1 \right], [1, 0, -1] \right] \right]$$

Berikut kita ekstrak ketiga vektor eigen tersebut.

```
>v1 &= transpose(matrix([veig[1][1][1], ...  
> veig[1][1][2], veig[1][1][3]])); $v1
```

$$\begin{pmatrix} 1 \\ -\left(\frac{2\sqrt{6}+4}{2} \right) \\ 1 \end{pmatrix}$$

```
>v2 &= transpose(matrix([veig[2][1][1], ...  
> veig[2][1][2], veig[2][1][3]])); $v2
```

$$\begin{pmatrix} 1 \\ \frac{2\sqrt{6}-4}{2} \\ 1 \end{pmatrix}$$

```
>v3 &= transpose(matrix([veig[3][1][1], ....
>     veig[3][1][2], veig[3][1][3]))); $v3
```

$$\begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}$$

Selanjutnya, berikut kita periksa kebenaran vektor-vektor eigen tersebut sesuai dengan nilai-nilai eigennya.

```
>$(A-lambda1*ident(3)).v1 with [a=2, b=-3]|...
>     expand|simplify
```

$$\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

```
>$(A-lambda2*ident(3)).v2 with [a=2, b=-3]|...
>     expand|simplify
```

$$\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

```
>$(A-lambda3*ident(3)).v3 with [a=2, b=-3]|...
>     expand|simplify
```

$$\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

4.5 Contoh Aplikasi Aljabar: Perhitungan Bunga Majemuk

Berikut ini ditunjukkan penggunaan Euler Math Toolbox (EMT) untuk perhitungan suku bunga, baik secara numerik maupun simbolik. Asumsikan Anda memiliki modal 500 (katakanlah dalam juta rupiah) yang Anda tabung di bank pada awal bulan.

```
>M=500 // modal awal bulan
      500.000000
```

Sekarang kita asumsikan tingkat suku bunga 3% per tahun, atau 0.25% per bulan. Berikut ini adalah contoh perhitungan hasil modal beserta bunga pada akhir bulan pertama (atau awal bulan ke dua). Terdapat beberapa rumus yang dapat kita gunakan.

```
>M*1.0025//modal akhir bulan jika bunga 0.25% per bulan
      501.250000
>M+M*0.25% // cara perhitungan lain
      501.250000
>q=1+0.25%, M*q // cara lain lagi
      1.002500
      501.250000
```

Cara terakhir ini memudahkan perhitungan modal beserta bunganya pada akhir bulan-bulan selanjutnya. Misalnya, untuk mendapatkan modal akhir setelah 10 bulan, kita cukup mengalikan modal awal dan perpangkatan 10 pengali modalnya.

Modal pada akhir bulan ke-

1	2	3	...	n
$M.q$	$M.q^2$	$M.q^3$	...	$M.q^n$

```
>M*q^10
      512.641567
```

Karena nilai nominalnya dalam jutaan, maka kita pilih format tampilan 6 digit di belakang koma.

```
>format(12,6); M*q^10
      512.641567
```

Anda juga dapat menampilkan nilai-nilai modal dari awal bulan pertama sampai awal bulan ke-10 dengan mudah.

```
>largematrixes on; M*q^(0:9)
Column 1 to 4:
    500.000000    501.250000    502.503125    503.759383
Column 5 to 8:
    505.018781    506.281328    507.547032    508.815899
Column 9 to 10:
    510.087939    511.363159
```

Dalam praktik (kenyataan) ketika kita menabung di bank, pihak bank akan menarik biaya administrasi setiap bulan dan mengenakan pajak sebesar 20% setiap bunga tabungan. Jadi, nilai q akan menjadi

$$q = 1 + 80\% \times 0,25\%,$$

dan rumus nilai modal pada akhir bulan ke- n menjadi (misalnya biaya administrasi rekening adalah Rp. 10.000,00 (sepuluh ribu rupiah)):

$$M \times q^n - 0,01n.$$

Berikut dapat kita definisikan fungsi untuk menghitung nilai tabungan pada akhir bulan (atau awal bulan berikutnya), jika diketahui tabungan di awal bulan adalah M juta, suku bunga tabungan sebesar $r\%$ per bulan, pajak bunga 20%, dan biaya administrai bulanan Rp.10.000,00 atau 0,01 juta.

```
>function tabulan(M,r) := M*(1+0.8*r/100)-0.01
```

Selanjutnya, di EMT kita dapat menggunakan perintah `iterate` untuk menghitung nilai tabungan pada n bulan pertama.

```
>//nilai tabungan bersih pada awal bulan pertama sd 11
>tabungan=iterate("tabulan",500,10;0.25)
Column 1 to 4:
    500.000000    500.990000    501.981980    502.975944
Column 5 to 8:
    503.971896    504.969840    505.969779    506.971719
Column 9 to 11:
    507.975662    508.981614    509.989577
```

Nilai-nilai tabungan tersebut dapat ditampilkan secara vertikal.

```

>tabungan'
  500.000000
  500.990000
  501.981980
  502.975944
  503.971896
  504.969840
  505.969779
  506.971719
  507.975662
  508.981614
  509.989577
>tabungan[-1] // tabungan terakhir
  510

```

Berikut ini kita hitung berapa minimal Anda harus menabung di bank, supaya uang Anda tidak habis tanpa Anda tarik, dengan asumsi besar bunga tabungan 0,25% per bulan, pajak bunga 20%, dan biaya administrasi bulanan Rp.10.000,00= 0.01 juta rupiah.

$$\begin{aligned}
 M(1 + 0.8 \times 0.25/100) - 0.01 &= M \\
 1.002M - 0.01 &= M \\
 0.002M &= 0.01 \\
 2M &= 10 \\
 M &= 5.
 \end{aligned}$$

Jadi, minimal Anda harus menabung 5 juta rupiah agar tabungan Anda tidak berkurang. Apabila tabungan awal kurang dari 5 juta (tanpa menabung lagi dan tanpa ditarik selamanya), maka lama-lama tabungan Anda akan habis. Berikut beberapa contoh simulasinya.

```

>tabungan=iterate("tabulan",4,100;0.25); tabungan[-1]
  3.778841

```

Tabungan 4 juta setelah 100 bulan akan tinggal Rp 3.778.884,00. Tanpa pernah ditarik! Jadi jelas jika menabung 4 juta, lama-lama uang akan habis! Kapan? Berikut kita coba hitung.

```

>tabungan=iterate("tabulan",4,1000;0.25); tabungan[-1]
  -2.374312
>min(nonzeros(tabungan<0))
  807.000000

```

Tabungan 4 juta rupiah akan habis tanpa pernah ditarik dalam waktu 807 bulan atau 67 tahun 3 bulan.

```
>tabungan=iterate("tabulan",5,600;0.25); tabungan[-1]
5.000000
```

Nilai tabungan sebesar 5 juta rupiah tidak akan pernah berubah. Bunganya pas untuk membayar pajak bunga dan biaya administrasi bulanan.

Berikut kita tunjukkan bukti penyelesaian di atas dengan perhitungan simbolik di EMT (menggunakan Maxima).

```
>function tabulas(M) &= M*(1+0.8*0.25/100)-0.01
1.002 M - 0.01
>$solve(tabulas(M)=M,M)
```

$$[M = 5]$$

Hasil penyelesaian secara simbolik sama, yakni minimal tabungan awal adalah 5 juta rupiah agar tabungan tidak akan berkurang.

Dalam praktik, ketika Anda menabung, Anda dapat saja menambah tabungan atau menarik uang dari tabungan Anda. Asumsikan Anda menabung kembali atau menarik uang tabungan di awal bulan-bulan berikutnya sebesar P (penambahan atau penarikan) rupiah dan selalu sama. Maka, kita dapat mendefinisikan fungsi untuk menghitung besar tabungan di awal bulan berikutnya apabila di awal bulan ini saldo tabungan kita sebesar M juta dan di awal bulan berikutnya kita menabung sebesar P rupiah, dan asumsi bunga serta besar pajak dan biaya administrasi seperti sebelumnya.

```
>function saldo(M) := M*(1+0.8*r/100)-0.01+P
```

Nilai P dan r tidak ditentukan pada definisi fungsi tersebut, namun ketika hendak memanggil (menggunakan) fungsi `saldo`, sebelumnya harus ditentukan nilai P dan r . Jika nilai P positif, artinya Anda menambah tabungan, sedangkan apabila P negatif, artinya Anda menarik uang dari tabungan. Nilai r adalah tingkat bunga tabungan (dalam persen atau pecahan desimal ekuivalensinya) per bulan. Ingat, karena M dalam jutaan, maka P juga harus dinyatakan dalam jutaan.

Berikut adalah contoh perhitungan saldo selama 10 bulan dengan asumsi tabungan awal 5 juta, setiap bulan menabung 250 ribu, dan bunga tabungan 0,25% per bulan.


```
>P=0.25; r=0.25; iterate("saldo",5,10)
Column 1 to 4:
    5.000000    5.250000    5.500500    5.751501
Column 5 to 8:
    6.003004    6.255010    6.507520    6.760535
Column 9 to 11:
    7.014056    7.268084    7.522620
```

Bagaimana seandainya mulai bulan kedua Anda tidak menambah tabungan, melainkan menarik uang sebesar 2 juta dan tabungan awal adalah 500 juta? Berikut adalah perhitungan saldo selama 10 bulan berikutnya.

```
>P=-2; iterate("saldo",500,10)
Column 1 to 4:
    500.000000  498.990000  497.977980  496.963936
Column 5 to 8:
    495.947864  494.929760  493.909619  492.887438
Column 9 to 11:
    491.863213  490.836940  489.808613
```

Terlihat bahwa saldo tabungan semakin berkurang apabila setiap bulan ditarik sebesar 2 juta. Apakah tabungan lama-lama akan habis? Berikut kita cek selama 100 bulan, berapa saldo akhirnya.

```
>tabungan=iterate("saldo",500,100); tabungan[-1]
388.314792
```

Ternyata saldo semakin berkurang. Lantas kapan saldo akan menjadi nol? Seperti contoh sebelumnya, kita dapat menghitungnya dengan perintah sebagai berikut.

```
>tabungan=iterate("saldo",500,400); tabungan[-1]
-118.000606
>min(nonzeros(tabungan<0))
346.000000
>tabungan[346]
-1.132798
>tabungan[345]
0.875451
```

Diperoleh hasil bahwa mulai awal bulan ke 346, saldo tabungan sudah negatif, karena pada awal bulan ke 345 saldo tabungan tinggal Rp. 875.451,00 sehingga jika pada bulan berikutnya diambil 2 juta tentu akan menjadi negatif (Artinya tidak mungkin menarik tabungan 2 juta rupiah!). Hal ini dapat dicek dengan

rumus aslinya.

```
>tabungan[345]*(1+0.8*r/100)-0.01+P
-1.132798
```

Berikut adalah cara lain menghitung kapan tabungan akan habis, dan hasilnya sama dengan hasil di atas.

```
>{x,n}=iterate("saldo",500,till="x<0"); x, n
-1.132798
345.000000
```

Sekarang kita ingin mengetahui berapa bunga per bulan agar saldo menjadi nol setelah 300 bulan, misalnya.

Pertanyaan ini hanya dapat dijawab secara numerik. Berikut kita turunkan rumus yang diperlukan. Pertama kita definisikan fungsi untuk melakukan iterasi sebanyak n kali, dan kita tambahkan semua parameter yang terkait ke dalam fungsi ini. Nilai fungsi tersebut adalah hasil iterasi terakhir.

```
>function f(M,P,r,n) := ...
>   iterate("x*(1+0.8*r/100)-0.01+P",M,n;r,P)[-1]
```

Fungsi tersebut tidak lain adalah menghitung iterasi

$$x_{n+1} = x_n \cdot \left(1 + 0.8 \cdot \frac{r}{100}\right) - 0.01 + P.$$

Akan tetapi, kita tidak lagi menggunakan nilai P secara global. Penggunaan fungsi EMT `iterate()` memerlukan trik khusus. Nilai-nilai variabel r dan P dapat dituliskan sebagai parameter setelah titik koma (;).

Sekarang kita coba gunakan fungsi tersebut dalam perhitungan dengan menggunakan parameter-parameter sebelumnya. Hasilnya sama!

```
>f(500,-2,0.25,345)
-1.132798
```

Sekarang kita dapatkan jawaban pertanyaan semula.

```
>solve("f(500,-2,x,300)",0.5)
0.160818
```

Diperoleh jawaban bahwa, jika besar bunga 0,160818% per bulan, maka tabungan akan habis setelah 300 bulan.

Kita juga dapat menjawab berapa seharusnya penarikan setiap bulan dilakukan agar tabungan habis setelah 300 bulan dengan bunga 0,25% per bulan, sebagai berikut.

```
>solve("f(500, x, 0.25, 300)", -1)
-2.207986
```

Diperoleh jawaban bahwa penarikan per bulan adalah sebesar Rp. 2.207.986,00.

Perlu dicatat bahwa kita tidak dapat mencari berapa bulan tabungan akan habis dengan menggunakan fungsi ini, karena fungsi tersebut mengasumsikan nilai n adalah bulat.

4.6 Penyelesaian Simbolik Masalah Bunga Bank

Kita dapat menyelesaikan masalah bunga bank secara simbolik di EMT dengan menggunakan Maxima. Pertama, kita definisikan fungsi `saldos()` sebelumnya secara simbolik.

```
>function saldos(M) &= M*q-0.01+P; $'saldos(M)=saldos(M)
```

$$\text{saldos}(M) = Mq + P - 0.01$$

Kita dapat menggunakan fungsi simbolik `saldos()` secara berulang. Misalnya, berikut adalah salah satu cara menghitung nilai saldo setelah 5 bulan.

```
>&powerdisp:true
true
>$saldos(saldos(saldos(saldos(saldos(M))))), ...
> $ev(expand(%), simpsum)
```

$$-0.01 + P - 0.01q + Pq - 0.01q^2 + Pq^2 - 0.01q^3 + Pq^3 - 0.01q^4 + Pq^4 + Mq^5$$

Kita lihat pada ekspresi di atas adanya pola berupa deret geometri dengan rasio q . Setelah n bulan akan kita peroleh saldo sebesar:

$$M_n = (P - 0.01)(1 + q + q^2 + \dots + q^{n-1}) + Mq^n = (P - 0.01)\frac{q^n - 1}{q - 1} + Mq^n.$$

Ingat di dalam rumus tersebut, $q = 1 + 0.8r/100$, dengan r adalah tingkat bunga bulanan, dan P adalah nilai yang ditabung atau ditarik dari tabungan per bulan.

Jumlah deret geometri tersebut dapat juga dihitung dengan menggunakan *Maxima* di EMT, sebagai berikut.

```
>powerdisp: false;
>sum(q^k,k,0,n-1); $ % = ev(%,simpsum)
```

$$\sum_{k=0}^{n-1} q^k = \frac{q^n - 1}{q - 1}$$

Selanjutnya kita dapat mendefinisikan fungsi yang sama dengan fungsi f sebelumnya secara simbolik sebagai berikut.

```
>function fs(M,P,r,n) &= (q: 1+0.8*r/100,...
> (P-0.01)*(q^n-1)/(q-1)+M*q^n);
>$'fs(M,P,r,n)=fs(M,P,r,n)
```

$$fs(M,P,r,n) = \frac{125.0(P-0.01)((0.008r+1)^n-1)}{r} + M(0.008r+1)^n$$

Fungsi ini sama persis dengan fungsi f di atas, namun lebih efektif untuk perhitungan. Berikut kita lihat hasilnya.

```
>longest f(500,-2,0.25,47), longest fs(500,-2,0.25,47)
450.2794050712018
450.2794050712009
```

Sekarang dengan fungsi tersebut kita dapat menghitung berapa bulan saldo tabungan akan habis, jika menabung pertama kali 500 juta dan mulai bulan berikutnya ditarik sebesar 3 juta per bulan, bunga tabungan $r\%$ per bulan, pajak bunga 20%, biaya administrasi rekening bulanan Rp. 10.000,00. Dalam perhitungan berikut ini kita berikan nilai taksiran saldo akan habis dalam waktu 100 bulan.

```
>solve("fs(500,-3,0.25,x)",100)
202.10451379
```

Penyelesaian tersebut mengatakan bahwa saldo tabungan akan menjadi negatif setelah 203 bulan (= 16 tahun 11 bulan).

Berikut kita cek saldo tabungan pada bulan ke-202.

```
>fs(500,-3,0.25,202)
0.314239528205
```

Saldonya tinggal Rp. 314.239,528205. Jelas jika bulan berikutnya ditarik 3 juta rupiah, nilai tabungan akan menjadi negatif.

Masalah bunga bank selain bunga tabungan adalah bunga pinjaman. Dalam praktik, tarif bunga pinjaman biasanya lebih tinggi daripada tarif bunga tabungan. Biasa, bank tidak mau mendapatkan untung kecil!

Misalkan pada bulan pertama, Anda meminjam M juta kepada bank dan akan dikembalikan dengan cara mengangsur setiap bulan (mulai bulan berikutnya) sebesar A juta, tarif bunga pinjaman $r\%$ per bulan. Berapakah nilai A ?

Sebenarnya fungsi yang dapat digunakan sama dengan fungsi untuk perhitungan tabungan. Akan tetapi pada masalah pinjaman tidak ada pajak (yang dikenakan kepada bank atas bunga pinjaman yang Anda berikan ke bank) dan bank tidak dipotong biaya administrasi (melainkan biasanya justru peminjam yang dikenai biaya administrasi dan provisi). Dengan demikian rumus fungsinya sedikit berbeda. (Jika dalam perhitungan bunga tabungan tidak diperhitungkan bunga tabungan dan biaya administrasi sebenarnya rumusnya persis sama.)

```
>function hutang(M,A,r,n) &= (q: 1+r/100,...
>   A*(q^n-1)/(q-1)+M*q^n);
>$'hutang(M,A,r,n)=hutang(M,A,r,n)
```

$$hutang(M,A,r,n) = \frac{100A \left(\left(\frac{r}{100} + 1 \right)^n - 1 \right)}{r} + M \left(\frac{r}{100} + 1 \right)^n$$

Ini adalah rumus sisa hutang setelah Anda membayar angsuran sebesar A juta sebanyak n kali (bulan).

Sekarang kita coba gunakan fungsi tersebut untuk menghitung besar angsuran jika Anda meminjam bank sebesar 500 juta rupiah dengan bunga pinjaman 0,5% per bulan dan jangka waktu pelunasan 60 bulan. Berapakah besar angsuran per bulan?

```
>solve("hutang(500,x,0.5,60)",5)
-9.66640076471
```

Dari penyelesaian tersebut terlihat bahwa besar angsuran per bulan adalah Rp. 9.666.400,00.

```
>%*60
-579.984045883
```

Dengan kata lain, total yang harus Anda bayar ke bank dalam waktu 5 tahun dengan meminjam 500 juta adalah Rp. 579.984.045,588. Anda memberikan keuntungan sebesar Rp. 79.984.045,588 kepada bank dalam waktu 5 tahun.

Rumus perhitungan di atas sebenarnya masih menguntungkan Anda sebagai peminjam kepada bank. Dalam praktiknya, perhitungan tidak dilakukan dengan rumus di atas, melainkan lebih tinggi lagi, karena pihak bank akan menghitung total pinjaman selama 60 bulan, kemudian dibagi 60 sebagai angsuran per bulan. Nilainya akan jauh lebih tinggi.

```
>angsuran:= 500*(1+0.5/100)^60/60
11.2404179379
```

Angsuran yang harus Anda bayar per bulan sebesar Rp. 11.240.417,94. Bandingkan dengan besar angsuran yang berdasarkan bunga majemuk sebelumnya!

```
>hutang &= (hutang(M,A,r,n)=Hn); $ hutang
```

$$\frac{100A \left(\left(\frac{r}{100} + 1 \right)^n - 1 \right)}{r} + M \left(\frac{r}{100} + 1 \right)^n = Hn$$

Rumus di atas dapat digunakan untuk menghitung nilai A atau n secara simbolik dengan menentukan nilai $Hn = 0$ (hutang lunas).

```
>angsuran &= solve(hutang with Hn=0,A); $angsuran
```

$$\left[A = - \left(\frac{M \left(\frac{r}{100} + 1 \right)^n r}{100 \left(\frac{r}{100} + 1 \right)^n - 100} \right) \right]$$

Berikut kita gunakan rumus angsuran tersebut untuk menghitung besar angsuran seperti sebelumnya.

```
>$ angsuran with [M=500, r=0.5, n=60]
```

$$[A = -9.666400764714137]$$

Hasilnya sama!

Berikut kita coba hitung nilai n (banyaknya angsuran).

```
>tenor &= solve(hutang with Hn=0,n); $tenor
```

$$\left[\frac{(r+100)^n}{100^n} = \frac{100C}{Mr+100C} \right]$$

EMT (Maxima) ternyata tidak berhasil menyatakan penyelesain untuk n . Akan tetapi, dari persamaan yang dihasilkan dapat kita hitung nilai n , yakni

$$n = \frac{\log(100C/(M.r+100C))}{\log((100+r)/100)}.$$

Berikut kita definisikan fungsi simbolik untuk menghitung nilai n tersebut.

```
>&powerdisp:true
                                true
>function tenor(M,r,A) &= log((100*A)/(M*r+100*A))/...
>    log((100+r)/100); $'tenor(M,r,A)=tenor(M,r,A)
```

$$tenor(M,r,A) = \frac{\log\left(\frac{100A}{100A+Mr}\right)}{\log\left(\frac{100+r}{100}\right)}$$

```
>$tenor(500,0.5,-9.66640076471)
```

60.00000000029935

Diperoleh nilai $n = 60$ bulan, seperti yang kita harapkan dengan besar angsuran bulanan seperti hasil perhitungan sebelumnya.



Bab 5.

Visualisasi Grafik 2 Dimensi (2D) dengan EMT

Dalam komputasi matematika sering sekali diperlukan visualisasi diagram maupun grafik 2 dimensi untuk menyajikan data maupun fungsi matematika. EMT menyediakan fungsi/perintah `plot2d()` untuk menggambar berbagai kurva dan grafik dua dimensi (2D). Bab ini menjelaskan berbagai kurva/grafik dan diagram yang dapat dihasilkan dengan menggunakan EMT dan bagaimana memampilkannya di dalam layar EMT. Gambar dapat ditampilkan dan dipadukan langsung dengan teks naskah uraian penjelasan proses komputasi, sehingga penulis/peneliti tidak perlu melakukan *cropping* dan *copy and paste* gambar ke dalam teks naskah tulisan. Inilah salah satu perbedaan software EMT dengan software aplikasi matematika lain, yang tidak memadukan hasil komputasi dan visualisasi dengan teks naskah dokumen.

Di antara kemampuan software EMT untuk visualisasi 2 dimensi adalah:

- menggambar grafik fungsi satu variabel (rumus fungsi ditulis sebagai ekspresi matematis, rumus fungsi disimpan dalam variabel ekspresi, fungsinya didefinisikan sebagai fungsi numerik, fungsinya didefinisikan sebagai fungsi simbolik);
- menggambar beberapa kurva sekaligus (beberapa kurva pada bidang koordinat berbeda, beberapa kurva dalam satu bidang koordinat, sekumpulan kurva dengan satu perintah `plot2d()`);
- menuliskan label sumbu koordinat, label kurva, dan keterangan kurva (*legend*);
- mengatur ukuran/rasio gambar, format (*style*), dan warna kurva;
- membuat kurva yang interaktif;
- menggambar kurva fungsi parametrik, fungsi implisit, dan fungsi kompleks;
- menggambar daerah yang dibatasi oleh beberapa kurva;
- menggambar segi banyak;
- menggambar grafik statistika (plot pasangan data, diagram batang, histogram, boxplot, dll.)

5.1 Layar untuk menggambar

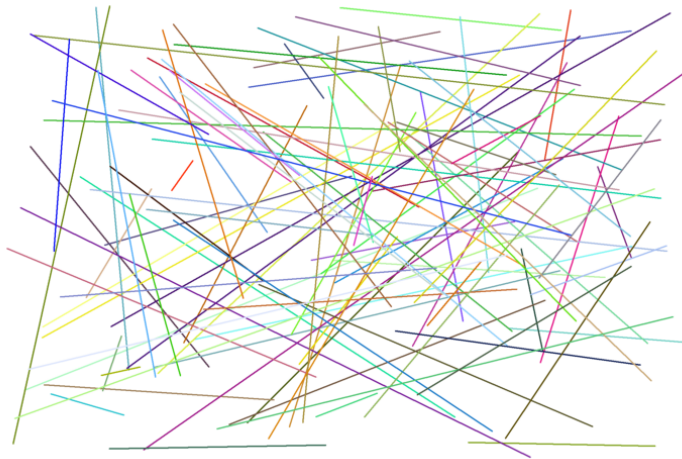
Layar EMT berfungsi seperti buku tulis, dapat digunakan untuk menulis teks penjelasan, perintah-perintah untuk melakukan komputasi dan menggambar, serta menampilkan hasil perintah-perintah tersebut, baik yang berupa nilai bilangan, ekspresi matematika, maupun gambar. Gambar hasil perintah ditampilkan dengan menggunakan bidang koordinat pada layar/jendela terpisah (layar grafik) atau menyatu dengan layar utama EMT (layar *notebook*). Ukuran bidang koordinat berkisar dari 0 hingga 1024 piksel pada setiap sumbu, tidak peduli apakah layar berbentuk persegi atau tidak. Ukuran bidang koordinat (nilai minimum dan maksimum sumbu x dan sumbu y) dapat diatur dengan perintah `setplot()`. Pemetaan antara koordinat bergantung pada ukuran koordinat jendela plot terakhir. Misalnya, perintah `shrinkwindow()` (yang otomatis efektif) akan mengecilkan jendela gambar untuk menyisakan ruang guna menampilkan label sumbu koordinat dan judul gambar. Untuk mendapatkan ruang yang lebih luas, gunakan perintah `fullwindow()`. Jika diperlukan label yang lebih besar pada sumbu y , gunakan perintah `smaller`. Untuk mendapatkan bidang koordinat persegi dengan rasio 1:1, gunakan perintah `square`.

Pada contoh berikut ini, disajikan perintah-perintah untuk menggambar beberapa garis acak dengan berbagai warna, menggunakan perintah dasar `plot()`.

```
>clc; // membersihkan layar gambar
>// menggunakan layar gambar/bidang koordinat penuh
>window(0,0,1024,1024);
>// batas-batas sumbu koordinat: 0 ≤ x, y ≤ 1
>setplot(0,1,0,1);
>// modus menggambar pada bidang koordinat yang sama
>hold on;
>// 100 pasang titik dengan koordinat acak
>n=100; P=random(n,2); Q=random(n,2);
```

```
>// menyimpan 100 warna acak
>colors=rgb(random(n),random(n),random(n));
>// 100 ruas garis acak tersebut
>loop 1 to n; color(colors[#]); plot(P[#],Q[#]); end;
>// modus menggambar pada bidang koordinat terpisah
>hold off;
>insimg; // menyisipkan gambar ke layar notebook
```

Contoh hasil perintah-perintah di atas adalah sebagai berikut.



Perintah `plot()` otomatis akan menghapus/membersihkan isi layar gambar, sehingga untuk menampilkan beberapa gambar pada bidang koordinat yang sama harus digunakan perintah `hold on`. Untuk membersihkan segala sesuatu yang dilakukan sebelumnya pada bidang koordinat, digunakan perintah `reset()`.

```
>reset;
```

Perintah `plot()` adalah perintah dasar untuk menggambar pasangan data. Hasilnya tergantung pasangan data inputnya. Apabila inputnya berupa pasangan vektor, hasilnya berupa garis (ruas garis) yang menghubungkan antar titik dengan koordinat pasangan elemen yang bersesuaian pada pasangan vektor tersebut. Apabila inputnya berupa pasangan matriks $n \times 2$ atau $2 \times n$, maka hasilnya berupa ruas-ruas garis yang menghubungkan setiap pasang titik dengan koordinat berupa elemen-elemen yang bersesuaian dari setiap matriks tersebut, seperti pada contoh di atas.

5.2 Menggambar grafik 2D dengan `plot2d()`

Selain menggunakan perintah `plot()`, untuk menggambar grafik/kurva 2 dimensi secara umum dapat digunakan perintah `plot2d()`. Fungsi ini merupakan fungsi plot yang paling penting untuk menggambar kurva dua dimensi (fungsi satu variabel). Fungsi tersebut diimplementasikan dalam bahasa Euler di dalam file "plot.e", yang dimuat otomatis ketika program EMT dijalankan.

Sama seperti perintah dasar untuk menggambar, aslinya gambar ditampilkan pada layar/jendela gambar yang terpisah dari layar *notebook* di EMT. Untuk menampilkan gambar hasil plot di layar *notebook*, perintah `plot2d()` dapat diakhiri dengan titik dua (:). Cara lain adalah perintah `plot2d()` diakhiri dengan titik koma (;), kemudian menggunakan perintah `insimg()` untuk menampilkan gambar hasil plot di layar *notebook*.

Fungsi/perintah `plot2d()` dapat digunakan untuk menggambar grafik data maupun fungsi. Perintah ini dapat digunakan untuk:

- menggambar grafik fungsi satu variabel (rumus fungsi ditulis sebagai ekspresi matematis, rumus fungsi disimpan dalam variabel ekspresi, fungsinya didefinisikan sebagai fungsi numerik, fungsinya didefinisikan sebagai fungsi simbolik);
- menggambar titik-titik pada bidang koordinat (pasangan vektor x dan y);
- menggambar beberapa kurva sekaligus (beberapa kurva pada bidang koordinat berbeda, beberapa kurva dalam satu bidang koordinat, sekumpulan kurva dengan satu perintah `plot2d()`);
- menuliskan label sumbu koordinat, label kurva, dan keterangan kurva (*legend*);
- mengatur ukuran/rasio gambar, format (*style*) titik/garis/kurva/daerah arsiran/diagram batang, dan warna kurva;
- membuat kurva yang interaktif;
- menggambar kurva fungsi parametrik, fungsi implisit, dan fungsi kompleks;
- menggambar daerah yang dibatasi oleh beberapa kurva;
- menggambar segi banyak;
- menggambar grafik statistika (plot pasangan data, diagram batang, histogram, boxplot, dll.)

Gambar yang dihasilkan EMT dan ditampilkan pada layar teks (notebook) disimpan pada folder yang sama tempat file notebook disimpan, dan aslinya di dalam subfolder "**images**". Gambar-gambar tersebut juga digunakan ketika file notebook dikonversi menjadi file HTML.

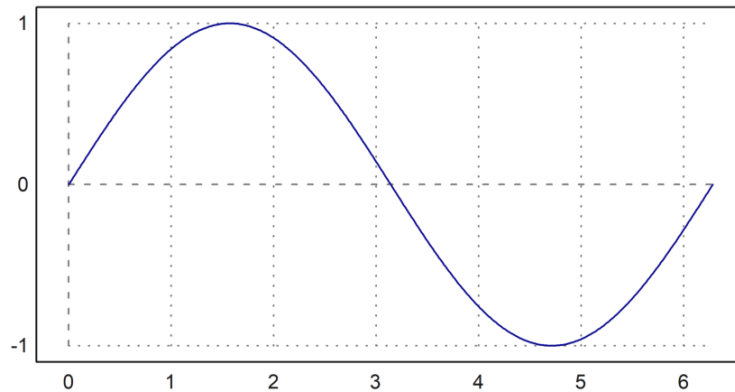
Anda dapat mengklik salah satu gambar pada layar teks dan jika ingin menyalin ke memori (untuk ditempel pada aplikasi lain, misalnya) dengan menekan tombol `[Ctrl]+[C]`. Tentu saja, Anda juga dapat mengeksport grafik tersebut melalui menu **File**.

5.3 Pengaturan rasio bidang koordinat

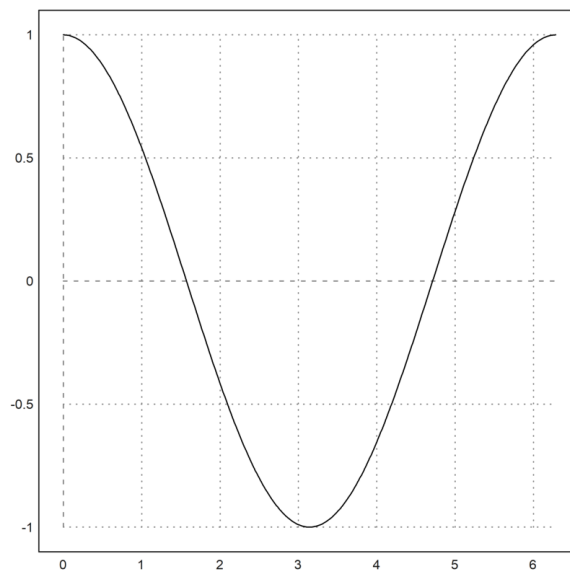
Aslinya bidang koordinat untuk menggambar grafik/kurva berupa persegi (panjang dan lebar sama). Untuk mengubah rasio tersebut dapat digunakan perintah `aspect()` dengan *input* adalah rasio panjang dan lebar (tinggi) bidang koordinat. Pengaturan ini dapat diberlakukan untuk semua gambar sesudahnya, atau hanya pada gambar tertentu. Untuk mengembalikan rasio bidang gambar sesuai aslinya gunakan perintah `aspect()` (tanpa nilai atau dengan nilai 1) atau `reset`. Perintah terakhir, selain mengembalikan rasio bidang gambar ke aslinya juga mengembalikan semua pengaturan bidang gambar ke pengaturan aslinya.

Perhatikan dua contoh berikut ini.

```
>aspect(1.5); // rasio panjang dan lebar 2:1  
>//kurva sinus warna biru pada interval [0, 2pi]  
>plot2d("sin(x)",0,2pi,color=blue):
```



```
>aspect();  
>reset; // pengaturan gambar aslinya  
>// kurva cosinus pada interval [0, 2pi]  
>plot2d("cos(x)",0,2pi):
```



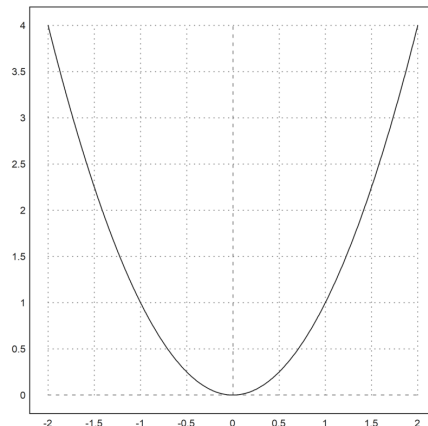
5.4 Menggambar grafik/kurva $y = f(x)$

Untuk menggambar grafik/kurva $y = f(x)$ dengan menggunakan perintah `plot2d()` di EMT dapat dilakukan dengan beberapa cara, yakni:

1. ekspresi yang menyatakan $f(x)$ ditulis langsung sebagai input, diapit dengan tanda kutip dubel (" ");
2. ekspresi yang menyatakan $f(x)$ disimpan dalam suatu variabel, kemudian nama variabel tersebut digunakan sebagai input, diapit dengan tanda kutip dubel (" ");
3. fungsi f didefinisikan sebagai fungsi numerik, kemudian nama fungsi tersebut digunakan sebagai input, diapit dengan tanda kutip dubel (" ");
4. fungsi f didefinisikan sebagai fungsi simbolik, kemudian nama fungsi tersebut digunakan sebagai input, tanpa diapit tanda kutip dubel; atau
5. pasangan vektor yang menyimpan nilai-nilai x pada interval tertentu dan nilai-nilai $y = f(x)$ digunakan sebagai input.

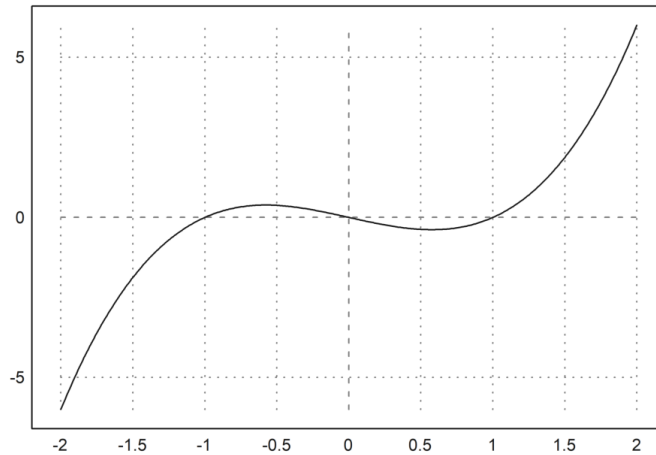
Berikut adalah beberapa contoh menggambar kurva $y = f(x)$. Yang pertama adalah menggambar kurva $y = x^2$.

```
>plot2d("x^2") :
```



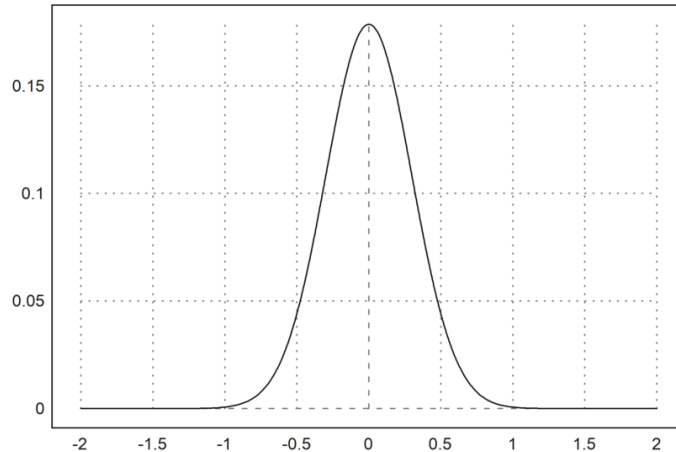
Berikut ini adalah menggambar kurva $y = x^3 - x$ dengan rasio gambar 3:2.

```
>aspect(1.5); plot2d("x^3-x") :
```



Berikut ini adalah menggambar kurva $y = \frac{e^{-ax^2}}{a}$ dengan nilai $a = 5.6$. Gambar tidak langsung ditampilkan dengan perintah `plot2d()` (perintah ini diakhiri dengan titik koma), melainkan menggunakan perintah `insimg()`.

```
>a:=5.6; plot2d("exp(-a*x^2)/a");
>insimg(25); // gambar hasil plot setinggi 25 baris
```



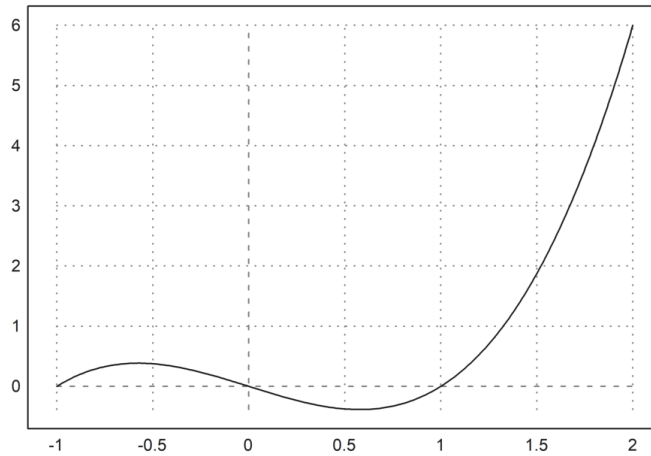
Dari tiga contoh terakhir dapat dilihat bahwa, aslinya gambar plot menggunakan sumbu x dengan rentang nilai dari -2 sampai dengan 2. Untuk mengubah rentang nilai x dan y , pada perintah `plot2d()` dapat ditambahkan nilai-nilai batas x (dan y) di belakang ekspresi yang digambar. Rentang nilai untuk menggambar kurva $y = f(x)$ dapat ditentukan dengan menggunakan parameter-parameter sebagai berikut:

- a, b : rentang nilai x (nilai aslinya -2,2),

- c, d : rentang nilai y (nilai aslinya: nilai $f(-2)$ sampai dengan $f(2)$),
- r : cara lain menentukan rentang nilai x dan y , yakni nilai jari-jari lingkaran dengan pusat $(0,0)$ yang dapat digambar pada bidang koordinat,
- cx, cy : koordinat pusat plot (nilai aslinya: $0,0$).

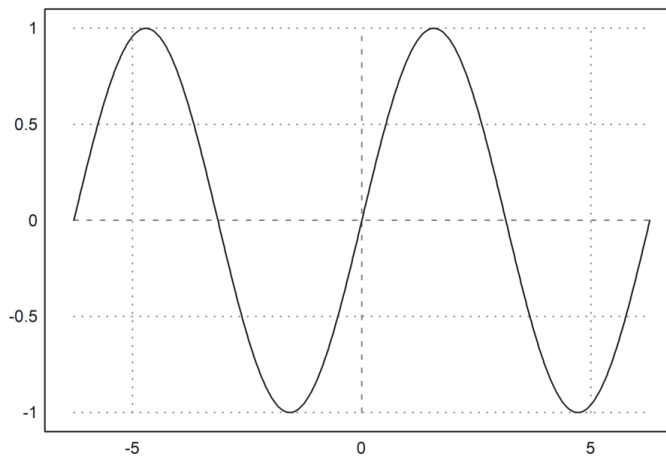
Berikut ini adalah menggambar kurva $y = x^3 - x$ pada interval $[-1, 2]$. Perhatikan bedanya dengan gambar fungsi yang sama sebelumnya.

```
>plot2d("x^3-x",-1,2):
```



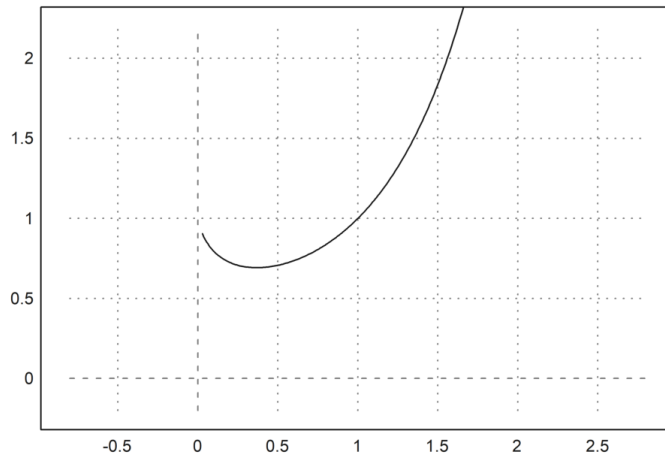
Berikutnya adalah menggambar kurva $y = \sin(x)$ pada interval $[-2\pi, 2\pi]$.

```
>plot2d("sin(x)",-2*pi,2*pi):
```



Berikut adalah contoh menggambar kurva $y = x^x$ dengan pusat plot $(1, 1)$ dan "jari-jari" bidang koordinat $r = 1.2$.

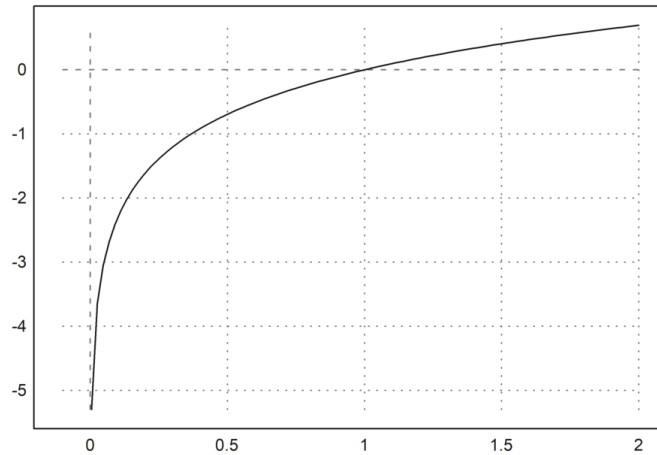
```
>plot2d("x^x", r=1.2, cx=1, cy=1) :
```



Perhatikan, bahwa fungsi $f(x) = x^x$ tidak terdefinisi untuk $x \leq 0$. Fungsi `plot2d` menyadari keadaan ini dan memplot kurva mulai dari titik di mana fungsi terdefinisi. Hal ini berlaku untuk semua fungsi yang menghasilkan nilai `NAN` di luar domainnya.

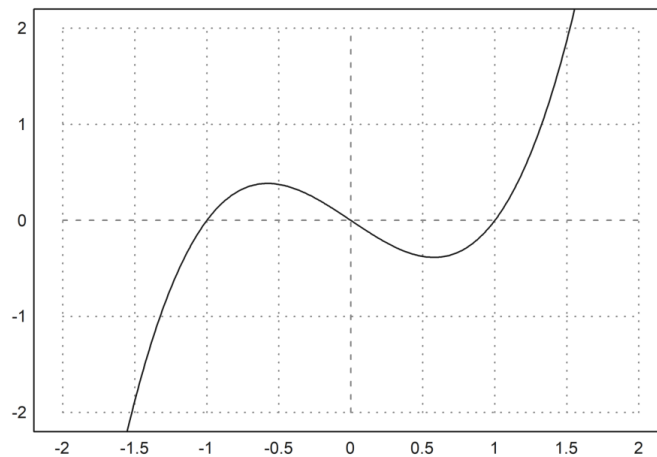
Sudah tentu, apabila domain fungsinya terbatas, kita dapat menentukan rentang nilai x pada perintah `plot2d()`, seperti contoh berikut ini untuk menggambar kurva $y = \log(x)$, yang memiliki domain $(0, \infty)$.

```
>plot2d("log(x)", -0.1, 2) :
```

Parameter (opsi) `square=true` (atau `>square`) pada perintah `plot2d()` akan memilih rentang y secara otomatis sehingga hasilnya layar gambar berbentuk persegi. Aslinya, Euler menggunakan layar gambar persegi.

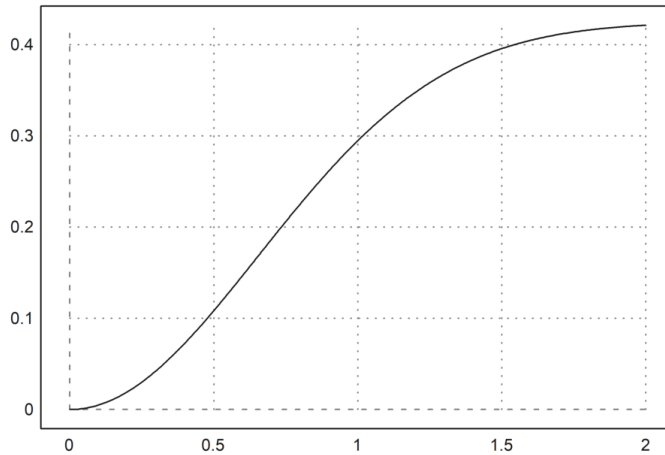
```
>plot2d("x^3-x",>square) :
```



EMT juga mampu menggambar kurva fungsi hasil integral tak tentu (antiderivatif), misalnya

$$y = \int \sin(x) \cdot e^{-x^2} dx.$$

```
>plot2d('// plot integral
>plot2d('integrate("sin(x)*exp(-x^2)",0,x)','',0,2):
```

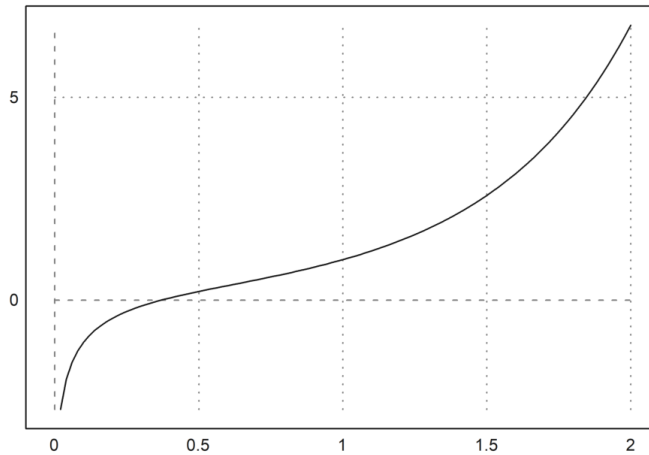


Awas, harus hati-hati, pada perintah tersebut terdapat dua macam tanda kutip, yang pertama tanda kutip dobel " " untuk mengapit ekspresi string yang akan diintegalkan, kedua berupa 2 tanda kutip tunggal ' ' untuk mengapit perintah `integrate()`. Kesalahan menuliskan kedua tanda ini akan menghasilkan pesan `error`.

Fungsi `plot2d()` juga menerima fungsi simbolik (fungsi Maxima). Berikut adalah contoh menggambar kurva $y = f(x)$ dengan

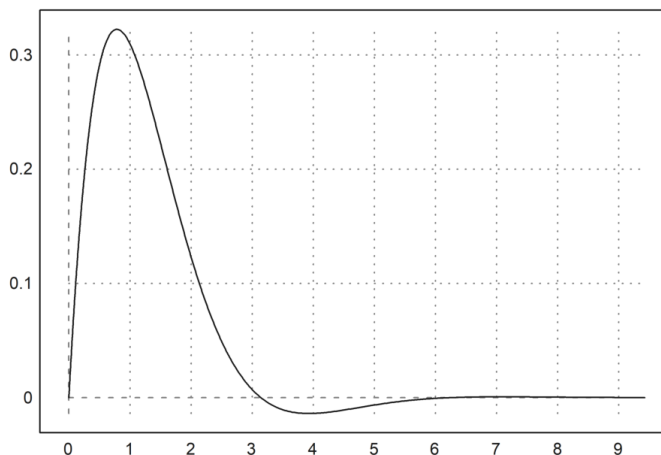
$$f(x) = \frac{d}{dx}(x^x).$$

```
>function f(x) &= diff(x^x,x)
      x
      x (log(x) + 1)
>plot2d(f,0,2):
```



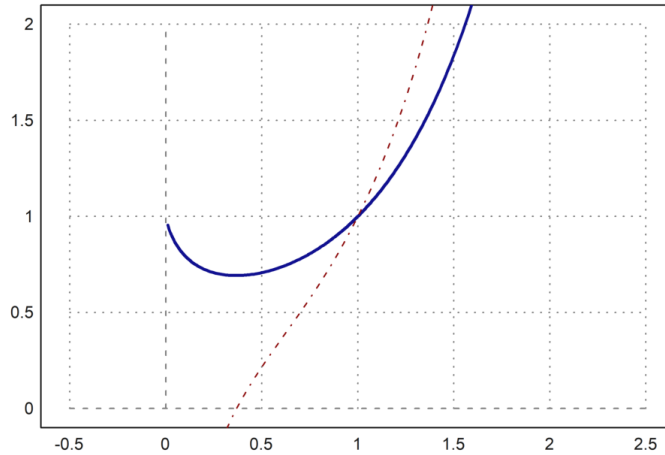
Ekspresi simbolik maupun variabel yang menyimpan ekspresi simbolik juga dapat digunakan di dalam perintah `plot2d()`. Berikut adalah contoh menggambar kurva $y = \sin(x).e^{-x}$. Perhatikan, untuk ekspresi maupun variabel simbolik tidak perlu diapit dengan tanda kutip.

```
>fungsi &= sin(x)*exp(-x)
          -x
          E sin(x)
>plot2d(fungsi,0,3pi):
```



Berikut adalah contoh menggambar kurva $y = x^x$ dan kurva turunannya dengan model garis dan warna berbeda pada bidang koordinat dengan "jari-jari" 1 dan pusat gambar (1,1).

```
>function f(x) &= x^x;
>plot2d(f,r=1,cx=1,cy=1,color=blue,thickness=2);
>plot2d(&diff(f(x),x),>add,color=red,style="-.-") :
```



Nilai-nilai fungsi maupun ekspresi pada perintah `plot2d()` dihitung secara adaptif. Untuk mempercepat waktu, menggambar plot secara adaptif dapat dimatikan dengan opsi `<adaptive` dan nyatakan banyaknya subinterval menggunakan opsi `n=...`. Hal ini hanya perlu untuk kasus-kasus yang langka, misalnya untuk menggambar kurva fungsi berikut ini.

$$f(x) = \begin{cases} -e^{-x^2}, & x < 0 \\ 0, & x = 0 \\ e^{-x^2}, & x > 0 \end{cases}$$

Fungsi di atas dapat dituliskan sebagai fungsi dengan satu rumus, yakni

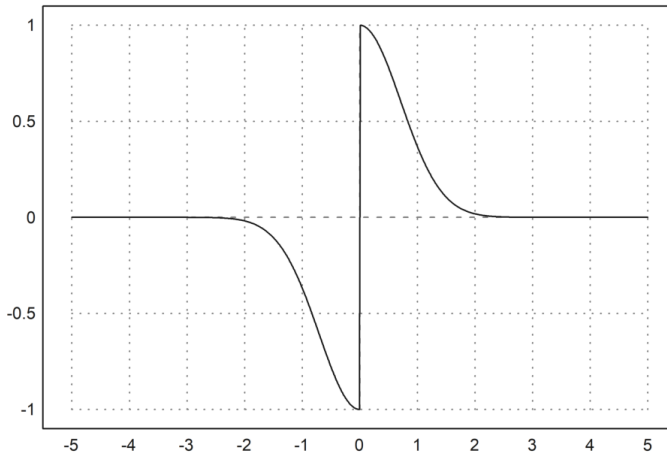
$$f(x) = \text{sign}(x) \cdot e^{-x^2}$$

dengan

$$\text{sign}(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0, \\ 1, & x > 0 \end{cases}$$

sehingga kurva $y = f(x)$ dapat digambar dengan perintah sebagai berikut.

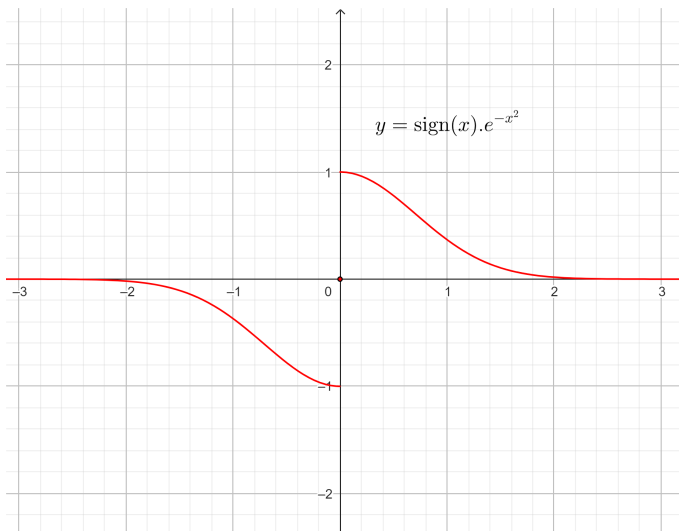
```
>plot2d("sign(x)*exp(-x^2)", -5, 5, <adaptive, n=1000) :
```



Gambaranya tidak terlalu sempurna, karena fungsinya kelihatan kontinu di $x = 0$, padahal seharusnya diskontinu di $x = 0$. Ingat, untuk fungsi di atas,

$$\lim_{x \rightarrow 0^-} f(x) = -1, \quad \lim_{x \rightarrow 0^+} f(x) = 1, \quad f(0) = 0.$$

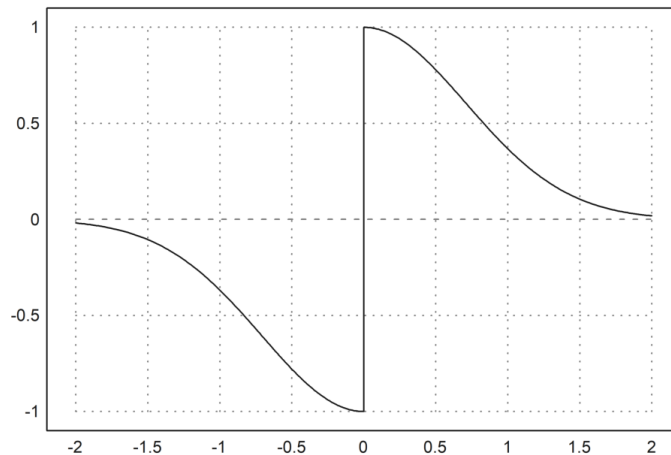
Hal itu tampaknya merupakan suatu kelemahan software EMT. Gambar plot fungsi tersebut yang tepat adalah sebagai berikut.



Cara lain menggambar kurva fungsi di atas adalah dengan mendefinisikan fungsi/program f sebagai fungsi numerik di EMT, kemudian menggunakan

nama fungsinya sebagai input perintah `plot2d()`. Berikut adalah perintah-perintahnya.

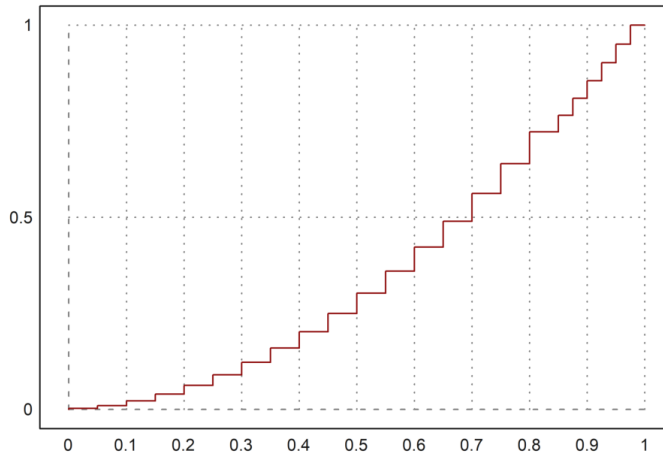
```
>function f(x) ...  
  if x<0 then return -exp(-x^2)  
  else if x==0 then return 0  
  else return exp(-x^2)  
  endif;  
endfunction  
>plot2d("f"):
```



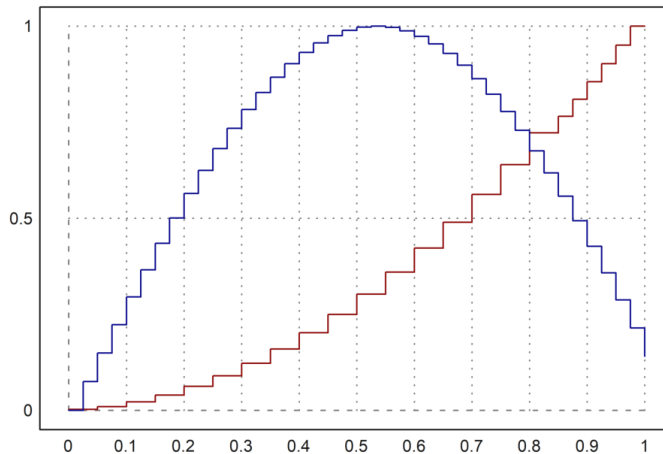
Kurvanya masih tampak ada kekurangtepatan karena masih terdapat garis tegak yang menunjukkan seolah-olah fungsinya kontinu di $x = 0$, padahal seharusnya fungsinya diskontinu. Mungkin ini adalah memang salah satu keterbatasan program EMT.

Berikut adalah contoh menggambar menggambar fungsi tangga dengan opsi `steps` pada perintah `plot2d()`.

```
>// menggambar grafik fungsi tangga  
>plot2d("x^2",0,1,steps=1,color=red,n=10):
```



```
>// gabung grafik fungsi tangga lain
>plot2d("sin(3*x)",0,1,steps=2,color=blue,n=10,>add):
```



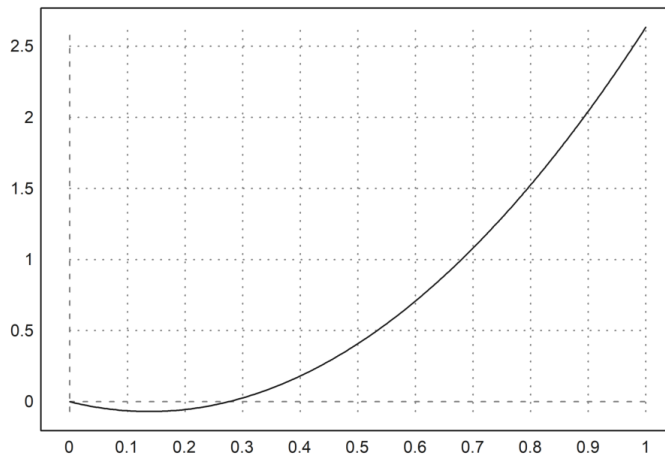
5.5 Grafik fungsi yang memuat parameter

Berikut adalah beberapa contoh menggambar kurva yang fungsinya didefinisikan sebagai fungsi (atau program) Euler. Fungsi di dalam Euler, selain dapat menerima masukan variabel bebasnya, misalnya x , juga dapat menerima parameter, yang merupakan variabel lokal (non global), dipisahkan dengan titik koma dengan variabel global (bebas). Perhatikan contoh berikut ini. Kita akan menggambar kurva fungsi

$$f(x) = \frac{x^2}{a} + ax^2 - x,$$

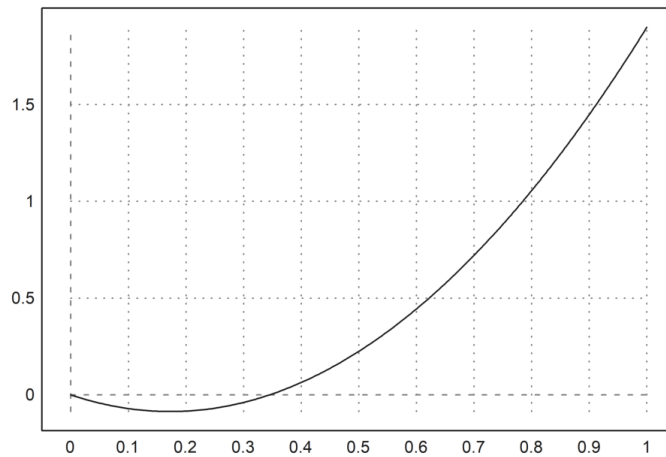
yang memuat satu parameter a . Untuk dapat menggambar kurvanya, parameter a harus ditentukan nilainya.

```
>// mendefinisikan sebuah fungsi  
>function f(x,a) := x^2/a+a*x^2-x;  
>a=0.3; plot2d("f",0,1;a): // plot dengan a=0.3
```

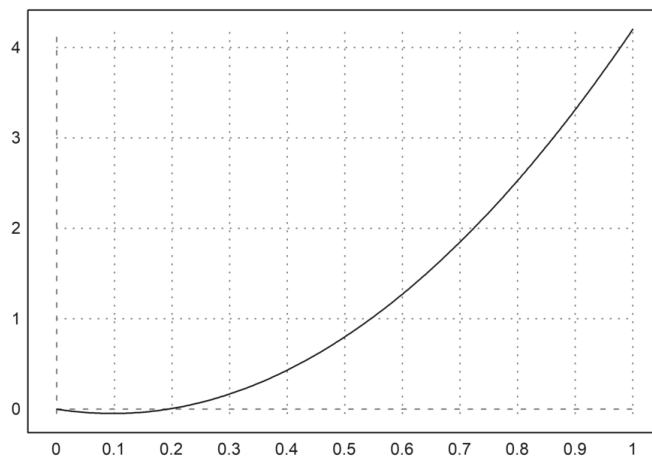


Nilai parameter juga dapat dituliskan langsung di dalam perintah `plot2d()`, dipisahkan dengan nilai batas-batas x menggunakan titik koma. Berikut diberikan beberapa contoh cara menggunakan fungsi yang sudah didefinisikan dan penentuan nilai parameternya untuk menggambar kurva fungsinya.

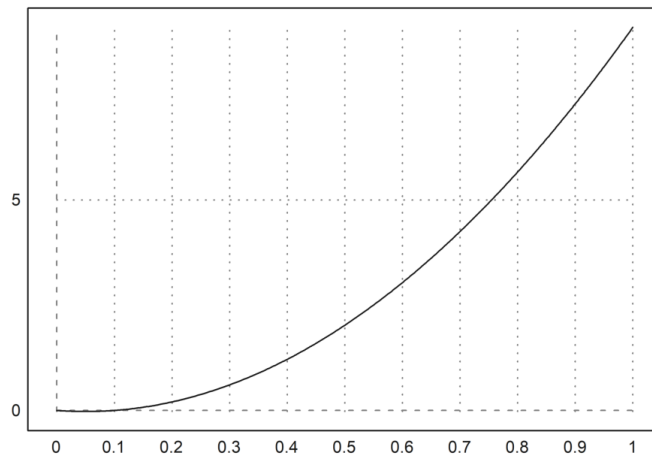
```
>plot2d("f",0,1;0.4): // plot dengan a=0.4
```

```
>plot2d({"f",0.2}},0,1): // plot dengan a=0.2
```



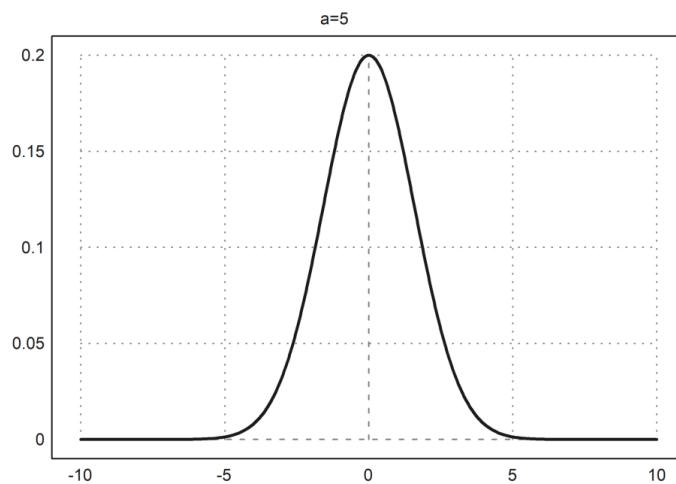
```
>// plot dengan nilai parameter 0.1  
>plot2d({"f(x,b)",b=0.1}},0,1):
```



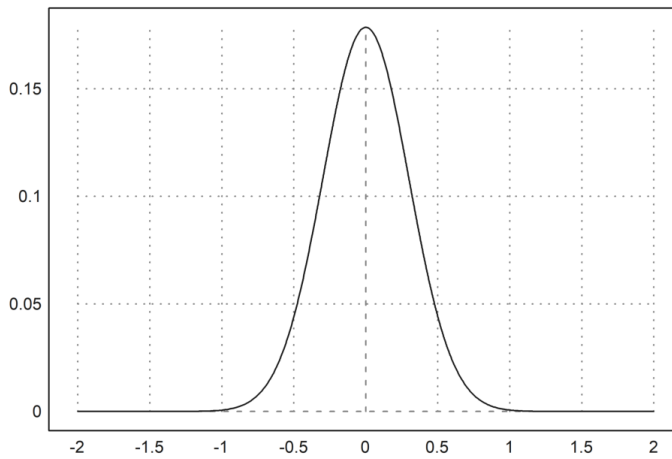
Berikut adalah contoh fungsi lain yang memuat parameter:

$$f(x) = \frac{1}{a} \cdot e^{-\frac{x^2}{a}}.$$

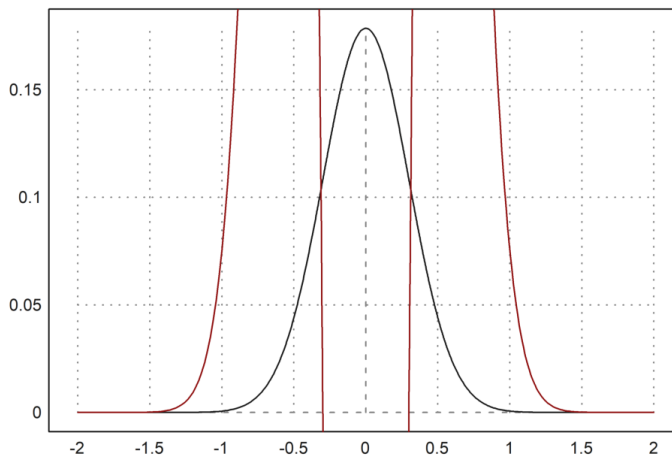
```
>function f(x,a) := 1/a*exp(-x^2/a); ...
  plot2d("f",-10,10;5,thickness=2,title="a=5"):
```



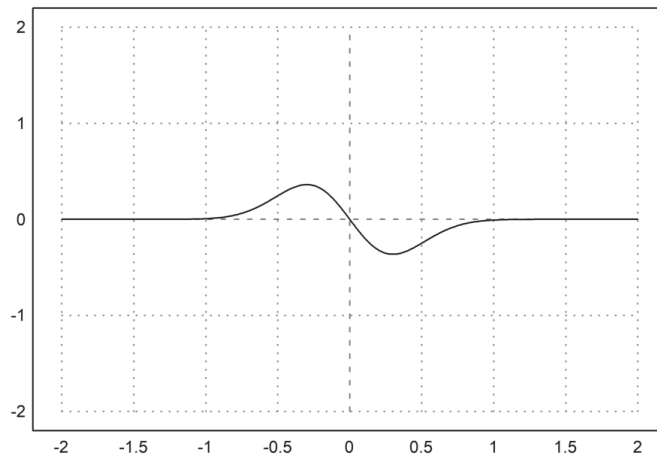
```
>// ekspresi dengan parameter disimpan sebagai variabel
>a:=5.6; fx &= exp(-a*x^2)/a;
>// kurva digambar untuk x dari -2 sampai 2
>plot2d(fx,-2,2):
```



```
>// tambah kurva turunan
>plot2d(&diff(fx,x,2),a=-2,b=2,c=-2,d=1,color=red,>add) :
```

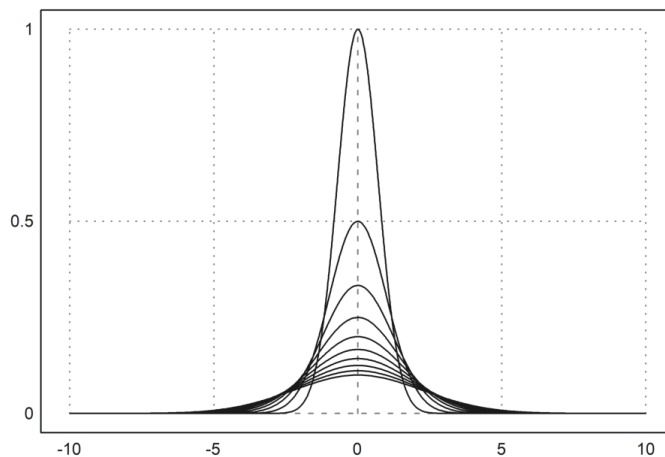


```
>// plot di dalam persegi
>plot2d(&diff(fx,x),a=-2,b=2,>square) :
```



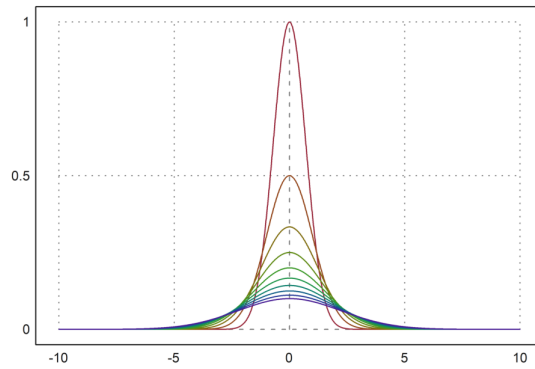
Parameter fungsi juga dapat digunakan untuk menggambar keluarga kurva, dengan menentukan nilai-nilai parameter sebagai vektor (matriks), seperti contoh-contoh berikut ini.

```
>plot2d({{"f",1}},-10,10);
>for a=2:10; plot2d({{"f",a}},>add); end:
```



Pada contoh kedua berikut ini, kita juga menggunakan nilai-nilai parameter untuk menentukan warna kurva dengan perintah `getspectral()`.

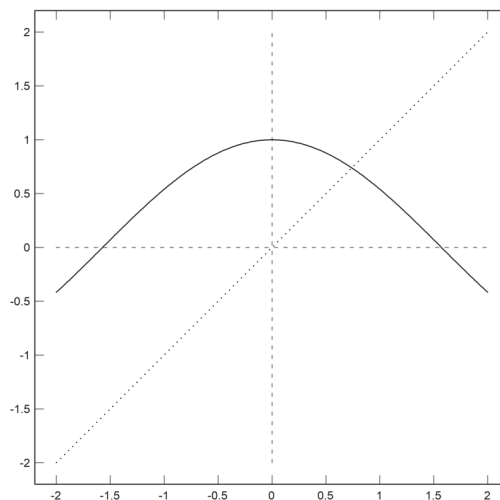
```
>x=-10:0.01:10; a=(1:10)';
>plot2d(x,f(x,a),color=getspectral(a/10)):
```



5.6 Menggambar beberapa kurva sekaligus

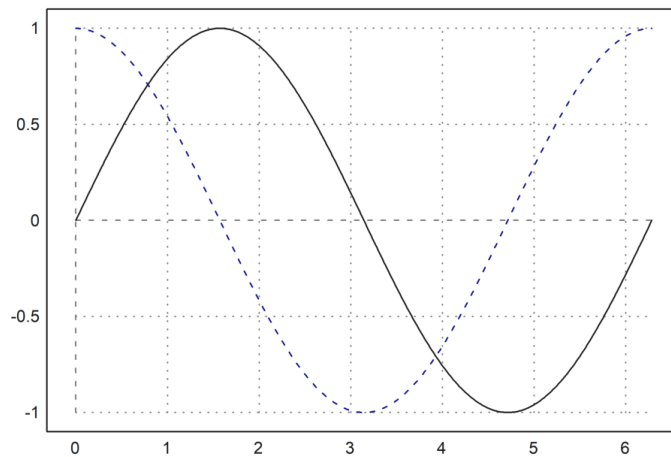
EMT memungkinkan kita menggambar beberapa kurva (beberapa fungsi berbeda) sekaligus, baik pada bidang koordinat maupun pada bidang-bidang koordinat yang berlainan (fungsinya dapat sama, rentang x boleh berbeda). Untuk menggambar beberapa kurva pada bidang koordinat yang sama dapat dilakukan dengan beberapa cara. Salah satu metode adalah menggunakan opsi `>add` pada beberapa perintah `plot2d()`, kecuali pemanggilan pertama. Berikut adalah contoh menggambar kurva $y = \cos(x)$ dan $y = x$ pada bidang koordinat yang sama dengan menggunakan dua perintah `plot2d()`.

```
>aspect(); plot2d("cos(x)",r=2,grid=6);
>plot2d("x",style=".",>add):
```



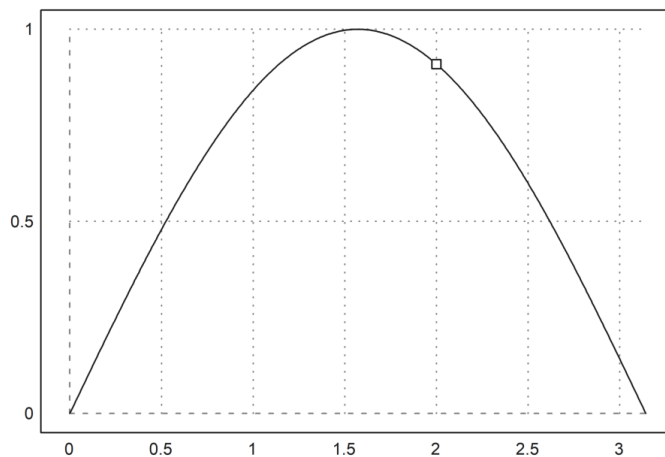
Contoh berikutnya adalah menggambar kurva $y = \sin(x)$ dan $y = \cos(x)$ pada bidang koordinat yang sama dengan model garis dan warna yang berbeda.

```
>aspect(1.5); plot2d("sin(x)",0,2pi);
>plot2d("cos(x)",color=blue,style="--",>add):
```



Salah satu kegunaan `>add` adalah untuk menambahkan titik pada kurva. Berikut adalah contoh menggambar titik pada kurva $y = \sin(x)$.

```
>plot2d("sin(x)",0,pi); plot2d(2,sin(2),>points,>add):
```



Pada contoh berikut ini digambar kurva $y = \text{sinc}(x) = \frac{\sin(x)}{x}$ beserta kurva ham-pirannya menggunakan deret Taylor ke-8 dan ke-16. Perhitungan deret Taylor dilakukan menggunakan Maxima melalui ekspresi simbolik. Gambar plot diha-silkan dengan menggunakan tiga perintah `plot2d()`. Perintah kedua dan ke tiga menggunakan opsi `>add` untuk menggambar 2 plot terakhir pada bidang

koordinat yang sama dengan plot pertama. Gambar juga dilengkapi dengan kotak legenda untuk menjelaskan kurangnya.

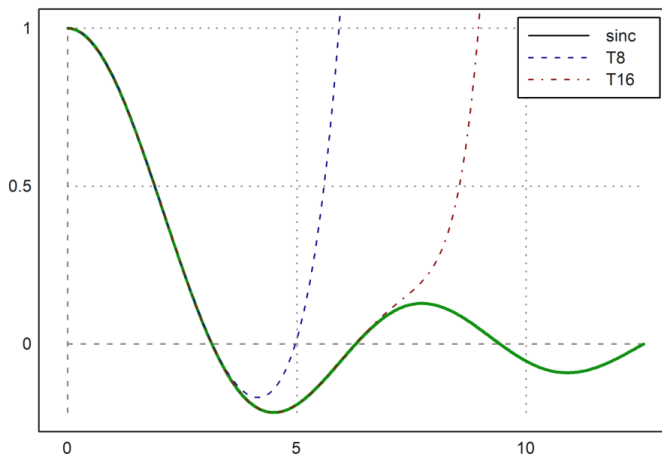
Pertama, menampilkan deret Taylor untuk $f(x) = \frac{\sin(x)}{x}$ dengan pusat $x = 0$ dan sampai suku yang memuat x^{10} dengan menggunakan perintah Maxima `taylor`. Tanda $\$$ di depan perintah menampilkan hasilnya menggunakan $\text{\LaTeX}$. Perintah `&powerdisp:true;` membuat tampilan deret dimulai dari suku berpangkat terkecil. Aslinya EMT menampilkan deret dari suku berpangkat terbesar.

```
>&powerdisp:true;
>\$'sin(x)/x=taylor(sin(x)/x,x,0,10)
```

$$\frac{\sin x}{x} = 1 - \frac{x^2}{6} + \frac{x^4}{120} - \frac{x^6}{5040} + \frac{x^8}{362880} - \frac{x^{10}}{39916800}$$

Selanjutnya, digambar kurva $y = \frac{\sin(x)}{x}$ dan 2 kurva hampirannya menggunakan deret Taylor (tepatnya, deret Maclaurin) sampai suku x^8 dan sampai suku x^{16} dengan model garis berbeda, dilengkapi kotak legenda grafik.

```
>plot2d("sinc(x)",0,4pi,color=green,thickness=2); ...
plot2d(&taylor(sin(x)/x,x,0,8), ...
>add,color=blue,style="--"); ...
plot2d(&taylor(sin(x)/x,x,0,16), ...
>add,color=red,style="-.-"); ...
labelbox(["sinc","T8","T16"],styles=["-","--","-.-"],...
colors=[black,blue,red]):
```



Pada contoh selanjutnya akan digambar kurva-kurva polinomial Bernstein yang didefinisikan sebagai

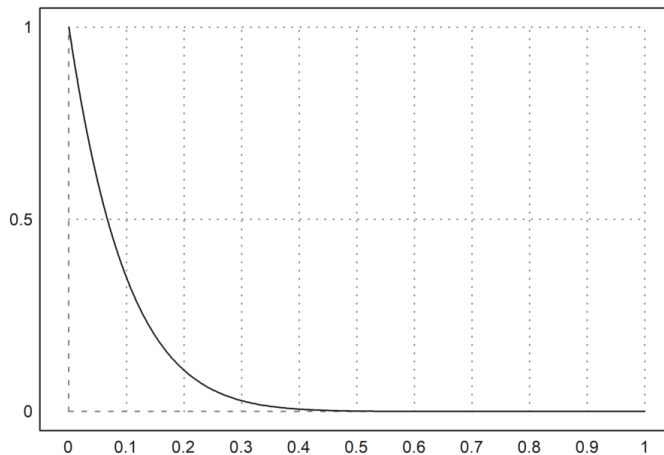
$$B_{i,n}(x) = \binom{n}{i} x^i (1-x)^{n-i}.$$

Pertama, digambar kurva

$$y = (1-x)^{10} = \sum_{i=0}^{10} \binom{10}{i} (-x)^{10-i}$$

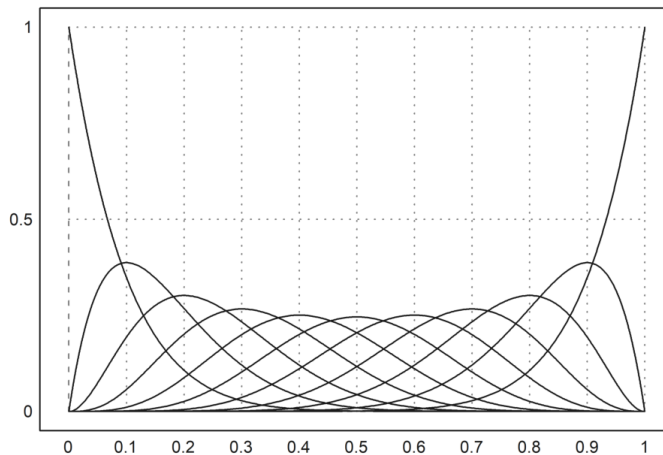
pada interval $[0, 1]$.

```
>plot2d("(1-x)^10", 0, 1):
```



Selanjutnya digambar 10 kurva $y = B_{i,10}(x)$ untuk $i = 1, 2, 3, \dots, 10$ pada bidang koordinat yang sama. Di sini digunakan *loop for* dan untuk menyisipkan 10 kurva digunakan *insimg*.

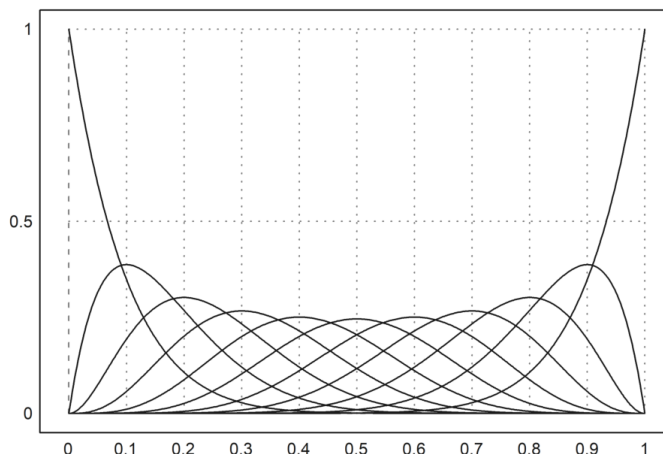
```
>for i=1 to 10;
>plot2d("bin(10,i)*x^i*(1-x)^(10-i)",>add); end;
>insimg;
```

Metode kedua untuk menggambar beberapa kurva pada bidang koordinat yang sama adalah dengan menggunakan pasangan matriks nilai-nilai x dan matriks nilai-nilai y dengan ordo (ukuran) sama.

Untuk menghasilkan gambar plot yang sama seperti contoh terakhir, mula-mula didefinisikan matriks nilai-nilai polinomial Bernstein pada setiap baris.

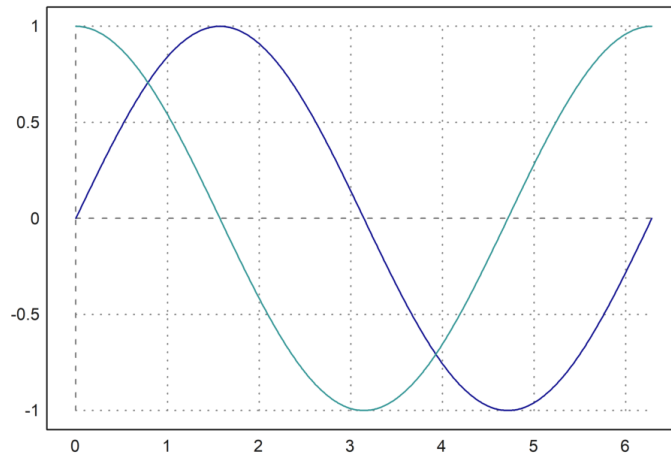
```
>x=linspace(0,1,500); // x adalah vektor baris 1x500
>n=10; k=(0:n)'; // k adalah vektor kolom 11x1
>y=bin(n,k)*x^k*(1-x)^(n-k); // y adalah matriks 11x500
>plot2d(x,y): // plot 11 kurva
```



Metode ketiga untuk menggambar kurva beberapa fungsi pada bidang koordinat yang sama adalah menggunakan vektor ekspresi (*string*). Untuk pengaturan masing-masing kurva dapat digunakan vektor warna dan vektor ketebalan kurva

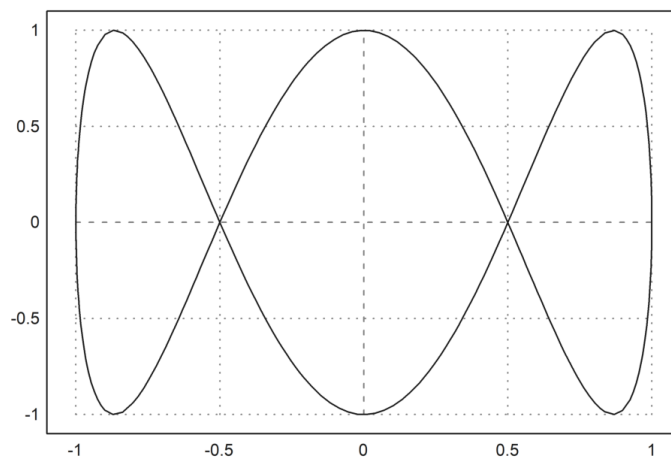
dengan ukuran sama. Berikut adalah contoh menggambar kurva $y = \sin(x)$ dan $y = \cos(x)$ dengan satu perintah `plot2d()`.

```
>plot2d(["sin(x)", "cos(x)"], 0, 2pi, color=4:5):
```



Berikut adalah cara menggambar plot kurva $y = \cos(x)$ dan $y = \sin(3x)$ pada interval $[0, 2\pi]$ yang menghasilkan gambar seolah-olah seperti sebuah kurva.

```
>plot2d("cos(x)", "sin(3*x)", xmin=0, xmax=2pi):
```



Vektor ekspresi dapat didefinisikan menggunakan perintah Maxima `makelist()` dan `mxm2str()`. Perintah Maxima berikut ini mendefinisikan vektor simbolik v dengan elemen-elemen $B_{i,10}(x)$, untuk $i = 0, 1, 2, \dots, 10$.

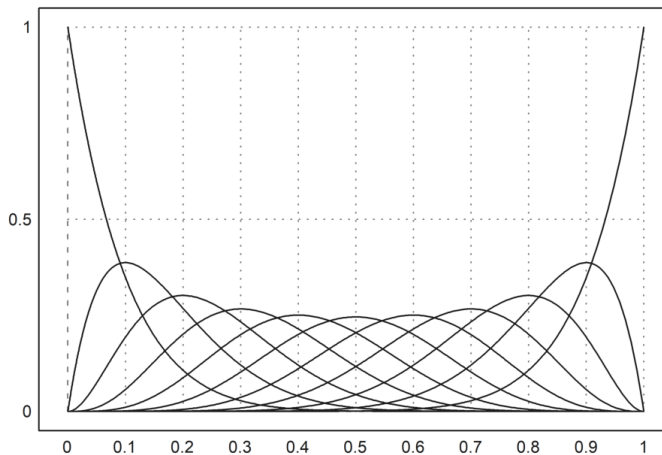
```
>v &= makelist(binomial(10,i)*x^i*(1-x)^(10-i),i,0,2)
               10         9         8 2
      [(1-x)  , 10(1-x) x , 45(1-x) x ]
```

Selanjutnya, vektor simbolik tersebut diubah menjadi vektor string dengan perintah `mxm2str(v)`.

```
>vs=mxm2str(v)
(1-x)^10
10*(1-x)^9*x
45*(1-x)^8*x^2
120*(1-x)^7*x^3
210*(1-x)^6*x^4
252*(1-x)^5*x^5
210*(1-x)^4*x^6
120*(1-x)^3*x^7
45*(1-x)^2*x^8
10*(1-x)*x^9
x^10
```

Terakhir, gunakan vektor string tersebut untuk menggambar kesebelas kurva polinomial Bernstein.

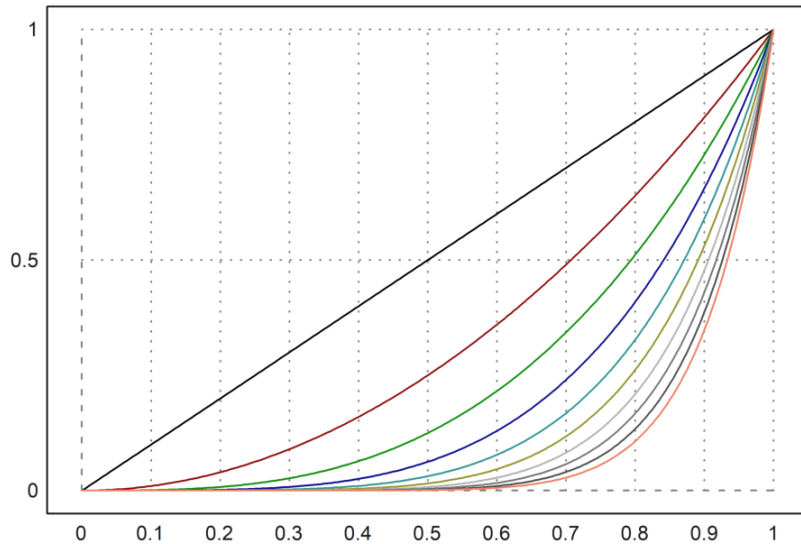
```
>plot2d(vs,0,1):
```



Metode yang lain untuk menggambar beberapa kurva pada satu bidang koordinat adalah menggunakan bahasa matriks pada Euler (EMT). Apabila suatu ekspresi menghasilkan sebuah matriks (vektor) fungsi, dengan satu fungsi pada

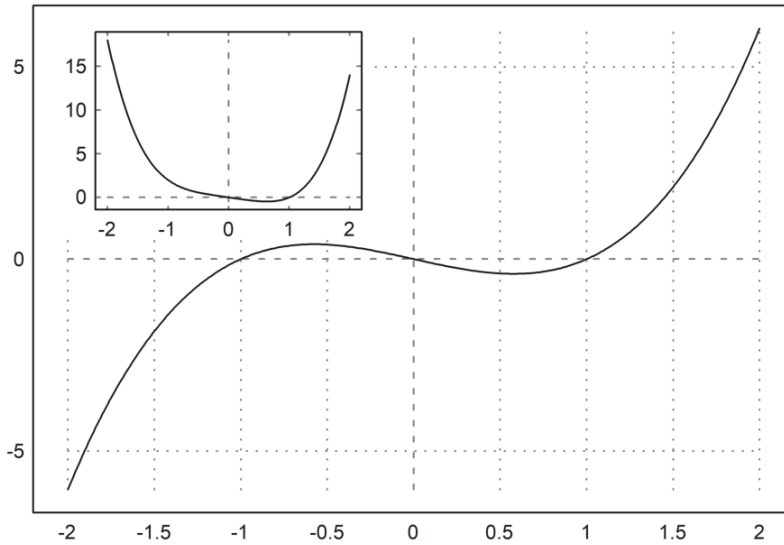
setiap baris, semua fungsi tersebut akan ditampilkan kurangnya dengan satu perintah `plot2d()`. Dalam hal ini, ekspresi yang digunakan sebagai input memuat sebuah vektor kolom. Selanjutnya, apabila digunakan vektor warna akan membuat kurangnya berbeda warna, sesuai vektor warna yang digunakan.

```
>n=(1:10)'; plot2d("x^n",0,1,color=1:10):
```



Terkadang, diperlukan sebuah gambar (plot) disisipkan pada gambar plot lain. Hal ini dapat dilakukan dengan mendefinisikan jendela plot yang lebih kecil. Jendela plot yang disisipkan ini tidak menyediakan tempat yang cukup untuk menuliskan label sumbu-sumbu koordinat. Kita harus menambahkan sembir sesuai kebutuhan. Perintah-perintah pada contoh berikut ini menunjukkan cara menyisipkan sebuah gambar plot pada gambar plot lain.

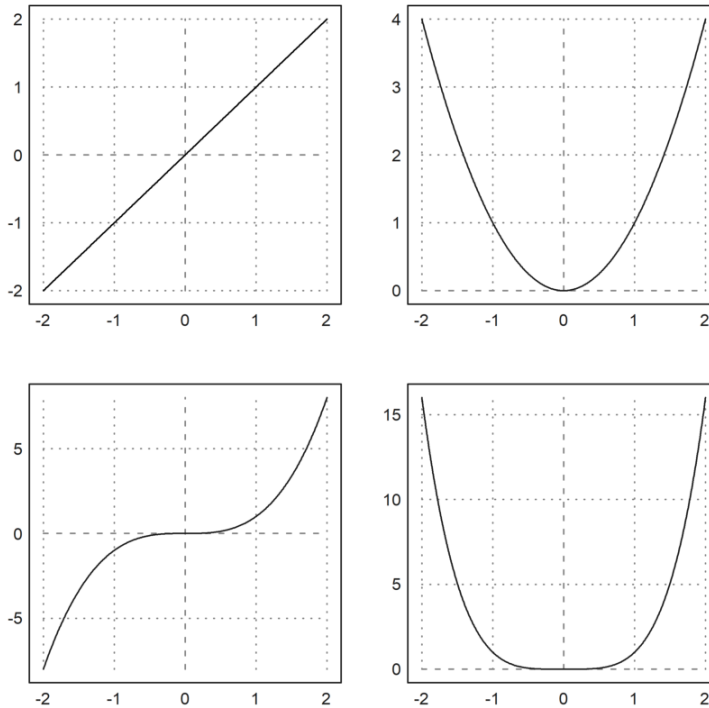
```
>plot2d("x^3-x");
>xw=200; yw=100; ww=300; hw=300;
>ow=window();
>window(xw,yw,xw+ww,yw+hw);
>hold on;
>barclear(xw-50,yw-10,ww+60,ww+60);
>plot2d("x^4-x",grid=6):
```



```
>hold off;
>window(ow);
```

Untuk menggambar beberapa kurva pada bidang koordinat terpisah namun dalam satu gambar dapat digunakan perintah/fungsi `figure()`. Perintah ini berguna untuk menyiapkan beberapa bidang koordinat pada satu layar gambar. Contoh berikut ini menunjukkan cara menggambar kurva $y = x^k$, untuk $k = 1, 2, 3, 4$, pada bidang koordinat berbeda dalam satu layar gambar. Perintah `figure(2,2)` membagi layar gambar menjadi 4 ($= 2 \times 2$) bidang koordinat. Perintah `figure(0)` mengembalikan kondisi layar gambar seperti aslinya.

```
>reset;
>figure(2,2); ...
>for n=1 to 4; figure(n); plot2d("x^"+n); end; ...
>figure(0):
```



5.7 Gaya/model (*style*) plot 2D

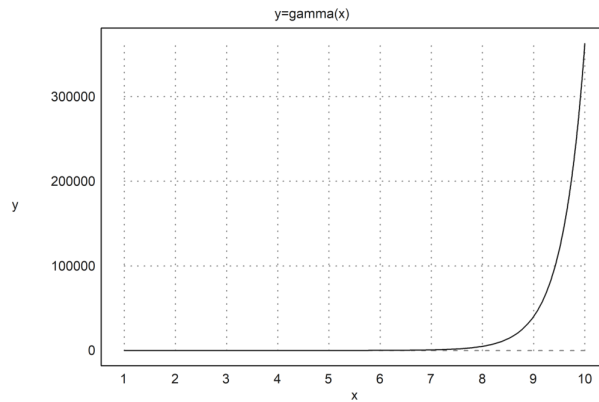
Pengaturan gaya/model (*style*) gambar plot meliputi pengaturan sumbu-sumbu koordinat, grid bidang koordinat, bingkai bidang koordinat, label, judul, legenda plot, model garis, warna, dan lain-lain. perintah `plot2d()` menyediakan parameter (opsi) yang sangat lengkap untuk pengaturan tersebut.

Untuk menampilkan label pada sumbu-sumbu koordinat pada gambar plot dapat digunakan perintah `shrinkwindow()` dengan nilai parameter yang lebih kecil, atau menentukan nilai positif untuk opsi `smaller` pada `plot2d()`. Berikut adalah contoh menggambar kurva fungsi Gamma ($\Gamma(x)$), yang didefinisikan sebagai

$$\Gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt.$$

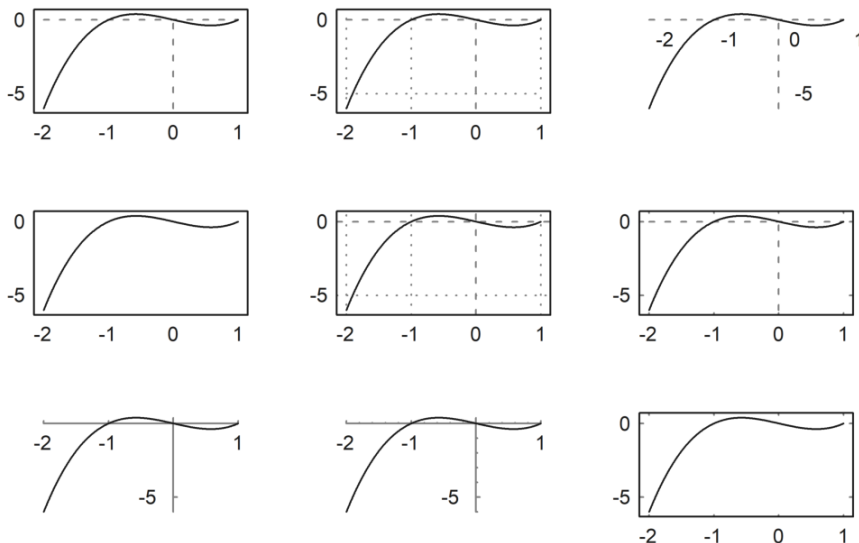
Euler menyediakan fungsi `gamma()` yang mendefinisikan fungsi tersebut.

```
>plot2d("gamma(x)", 1, 10, xl="x", yl="y", ...
      title="y=gamma(x)", smaller=1, <vertical):
```



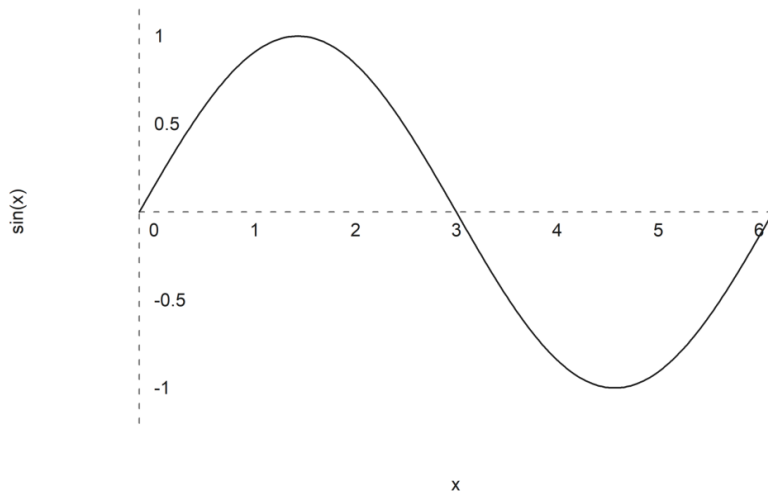
Aslinya, EMT menghitung secara otomatis titik-titik pada sumbu-sumbu koordinat dan menandai dengan label nilai-nilai x dan y . Pengaturan ini dapat diubah dengan menggunakan opsi (parameter) `grid`. Selain itu, label pada sumbu-sumbu koordinat dan judul plot dapat ditambahkan secara manual. Untuk mengembalikan ke pengaturan aslinya digunakan perintah `reset()`. Contoh berikut ini menunjukkan berbagai model plot grafik fungsi dengan menggunakan opsi `grid=k` untuk nilai-nilai k yang berbeda-beda. Perhatikan dan bandingkan hasilnya. Model `grid=0` tidak ditunjukkan, ini akan menggambar pada bidang koordinat tanpa sumbu-sumbu koordinat dan bingkai.

```
>figure(3,3);
>for k=1:9; figure(k); plot2d("x^3-x",-2,1,grid=k); end;
>figure(0):
```



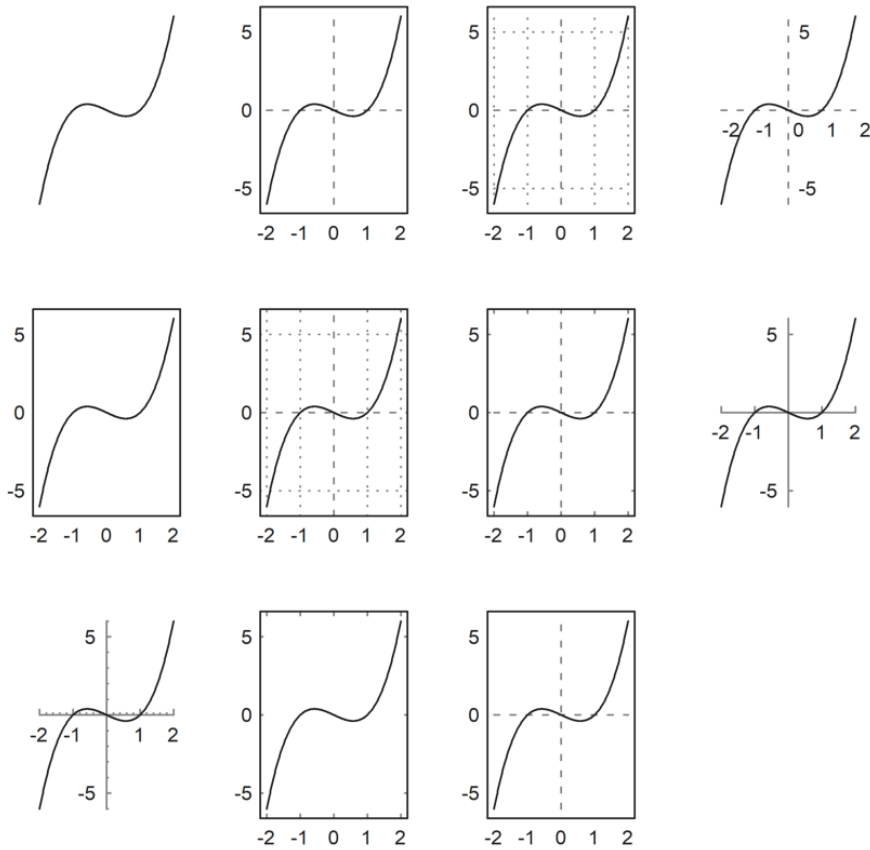
Pada contoh berikut ini kita ubah model grid bidang koordinat, kita tambahkan label sumbu-sumbu koordinat, dan label sumbu y ditulis tegak.

```
>plot2d("sin(x)",0,2pi,-1.2,1.2,grid=3,xl="x", ...
      yl="sin(x) ") :
```



Berikut ini ditunjukkan 11 model grid dan bingkai bidang koordinat berbeda untuk menggambar plot kurva.

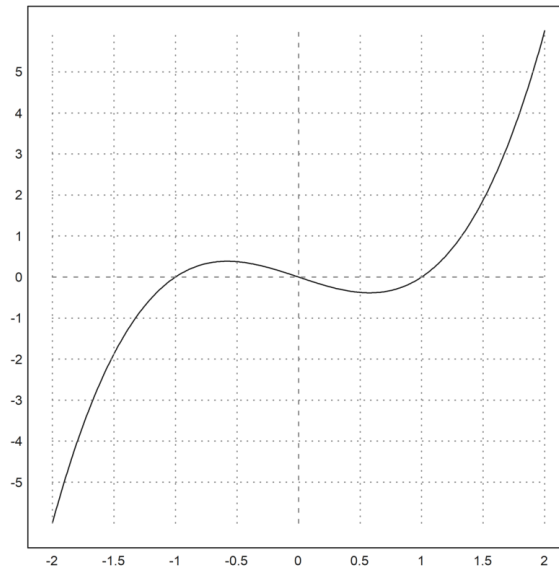
```
>aspect();
>figure(3,4); ...
figure(1); plot2d("x^3-x",grid=0);
figure(2); plot2d("x^3-x",grid=1);
figure(3); plot2d("x^3-x",grid=2);
figure(4); plot2d("x^3-x",grid=3);
figure(5); plot2d("x^3-x",grid=4);
figure(6); plot2d("x^3-x",grid=5);
figure(7); plot2d("x^3-x",grid=6);
figure(8); plot2d("x^3-x",grid=7);
figure(9); plot2d("x^3-x",grid=8);
figure(10); plot2d("x^3-x",grid=9);
figure(11); plot2d("x^3-x",grid=10);
figure(0):
```

Parameter `<frame` menghilangkan bingkai plot, dan `framecolor=blue` menentukan warna biru untuk bingkai.

Untuk mengatur sendiri skala pada sumbu-sumbu koordinat dapat digunakan opsi `grid=0`, kemudian menambahkan skalanya sendiri.

```
>// plot kurva tanpa sumbu dan bingkai
>plot2d("x^3-x",grid=0);
>//tambah bingkai dan skala pada sumbu x
>frame; xgrid([-2,-1.5,-1,-0.5,0,0.5,1,1.5,2]);
>// skala pada sumbu y
>ygrid([-5,-4,-3,-2,-1,0,1,2,3,4,5]):
```



Untuk pengaturan garis (kurva) terdapat beberapa opsi:

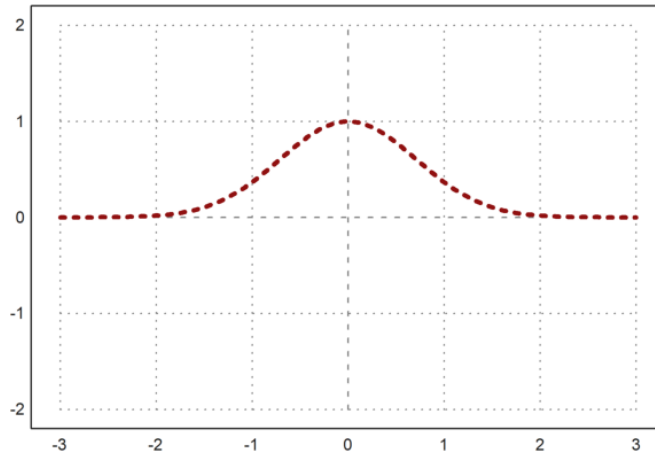
- `style="..."` dengan pilihan "-", "--", ":", ".", "-.", "-.-";
- `color`: lihat pilihan warna di bawah;
- `thickness`: aslinya bernilai 1.

Warna dapat dipilih dari warna asli yang sudah didefinisikan atau menggunakan kombinasi warna merah hijau biru (RGB).

- 0..15: nomor-nomor warna asli yang sudah terdefinisi
- **nama warna**: white, black, red, green, blue, cyan, olive, lightgray, gray, darkgray, orange, lightgreen, turquoise, lightblue, lightorange, yellow
- `rgb(red,green,blue)`: dengan nilai red, green, blue bilangan riil pada interval [0,1].

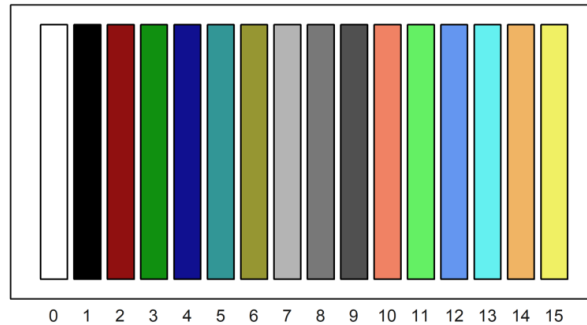
Berikut adalah contoh menggambar kurva dengan beberapa pilihan pengaturan bidang koordinat, warna, dan model garis.

```
>plot2d("exp(-x^2)", r=2, color=red, thickness=3, ...
style="--") :
```



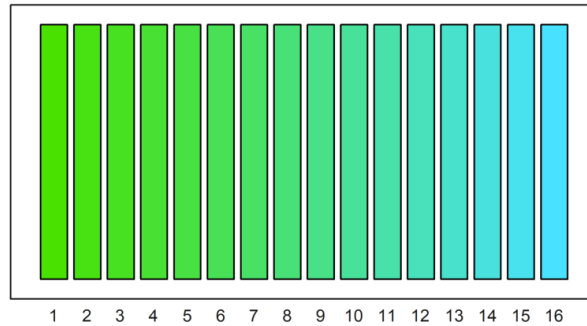
Daftar warna asli yang sudah terdefinisi di EMT dengan nilai 0 sampai 15 adalah sebagai berikut.

```
>aspect(2); columnsplot(ones(1,16),lab=0:15,grid=0, ...
  color=0:15):
```



Berikut adalah contoh menghasilkan warna dengan `rgb()`, yang dapat menghasilkan lebih banyak warna.

```
>columnsplot(ones(1,16),grid=0,color=rgb(random(1), ...
  random(1),linspace(0,1,15))):
```



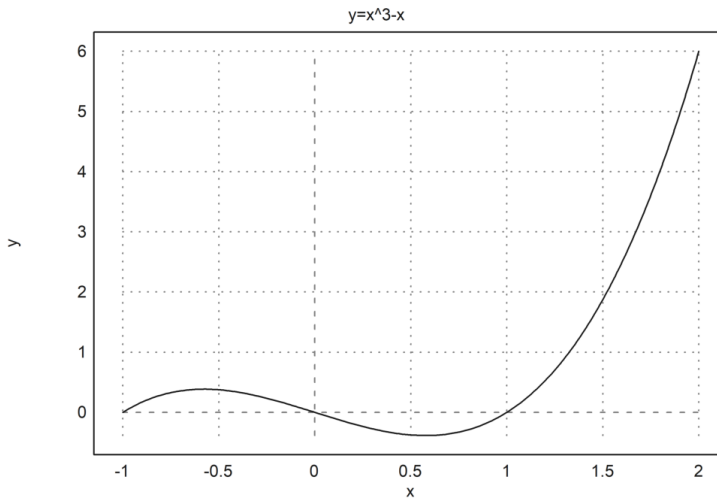
5.7.1 Teks label

Label pada gambar plot digunakan untuk memberi keterangan sumbu-sumbu koordinat, label kurva, judul kurva/diagram dan keterangan kurva (*legend*). Beberapa opsi pada perintah `plot2d()` untuk menuliskan teks label adalah:

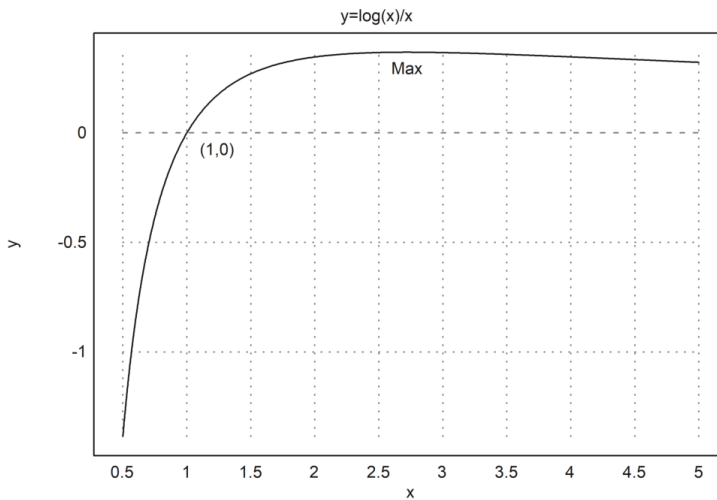
- `title="..."`: untuk memberi judul gambar plot,
- `xl="...", yl="..."`: untuk memberi label pada sumbu x dan sumbu y ,
- `label("...", x, y, ...)`: untuk menuliskan teks keterangan pada posisi (x, y) di bidang koordinat,
- `labelbox([...], styles=[...], colors=[...], w=...)`: untuk me-nampilkan kotak yang berisi keterangan masing-masing kurva (*legenda*),
- `textbox()`: untuk menuliskan keterangan gambar di dalam kotak, mirip *legenda*.

Berikut adalah contoh-contoh penggunaan opsi-opsi tersebut.

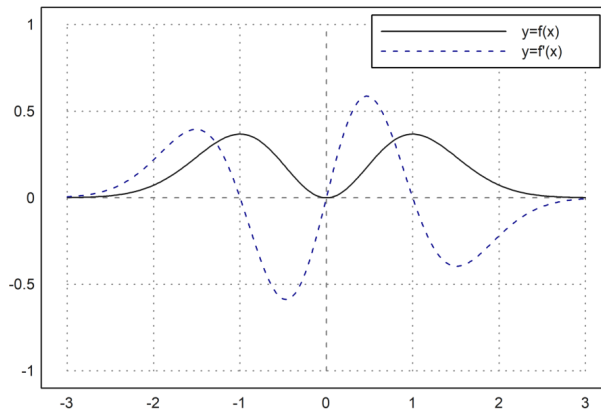
```
>aspect(1.5); plot2d("x^3-x",-1,2,title="y=x^3-x", ...
  yl="y",xl="x"):
```



```
>fs := "log(x)/x"; ...
>plot2d(fs,0.5,5,title="y="+fs,xl="x",yl="y"); ...
>label("(1,0)",1,0); label("Max",E,fs(E),pos="lc"):
```



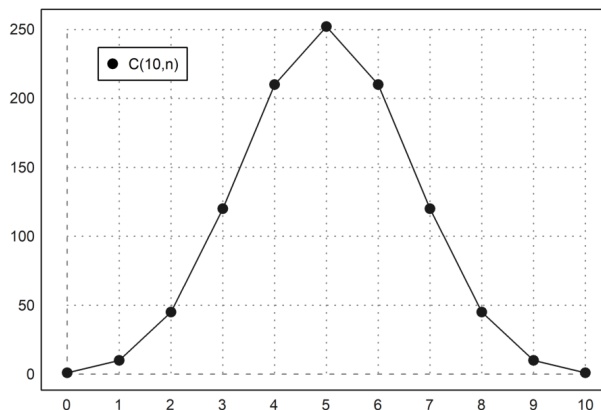
```
>function f(x) &= x^2*exp(-x^2); ...
plot2d(&f(x),a=-3,b=3,c=-1,d=1); ...
plot2d(&diff(f(x),x),>add,color=blue,style="--"); ...
labelbox(["y=f(x)", "y=f'(x)"],styles=["-", "--"], ...
colors=[black,blue],w=0.4):
```



Untuk kotak legenda, aslinya ditampilkan di pojok kanan atas, opsi `>left` dapat digunakan untuk mengubah posisi ke kiri atas. Kotak legenda ini dapat dipindah pada posisi sesuai keinginan atau yang sesuai. Kunci untuk meletakkan kotak legenda adalah pojok kanan atas kotak, bilangan titik koordinat yang digunakan menyatakan pecahan (proporsi) dari ukuran jendela grafik, yakni $0 < x, y < 1$. Lebar kotak legenda otomatis menyesuaikan isinya. Opsi `>points` pada `leblbox()` dapat digunakan untuk menjelaskan grafik titik.

Contoh berikut menunjukkan grafik titik-titik $(n, \binom{n}{k})$, untuk $k = 0, 1, 2, \dots, n$, yang dilengkapi dengan legenda.

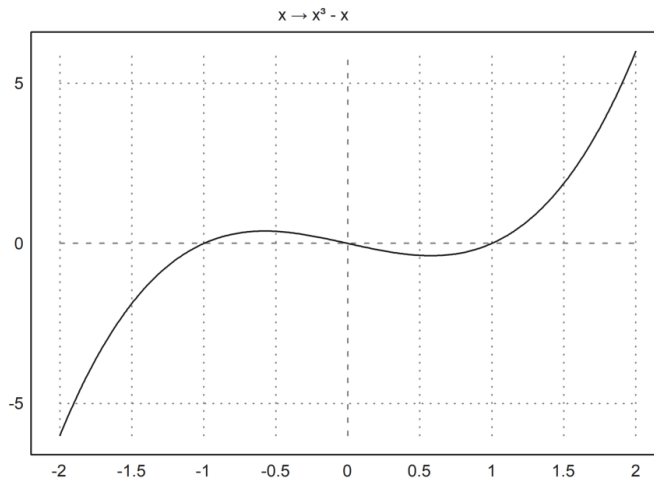
```
>n=10; plot2d(0:n,bin(n,0:n),>points,style="o#"); ...
plot2d(0:n,bin(n,0:n),>add);...
labelbox("C(10,n)",styles="o#",>points,x=0.1,y=0.1, ...
tcolor=black,>left):
```



Teks label, judul, kotak label dan teks lain pada gambar dapat memuat kode/

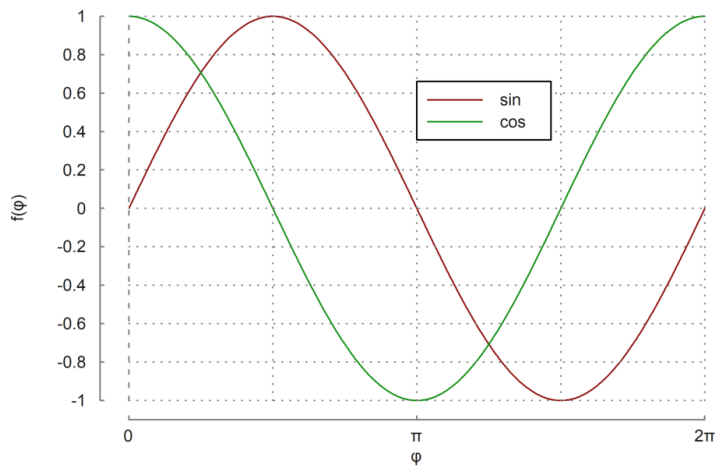
string **Unicode** yang diawali dengan huruf `u` dan diapit dengan tanda kutip dubel (`" "`). Setiap kode/string diawali tanda `&` dan diakhiri tanda titik koma (`;`). Perhatikan beberapa contoh di bawah ini.

```
>plot2d("x^3-x",title=u"x \rarr; x^sup3; - x"):
```



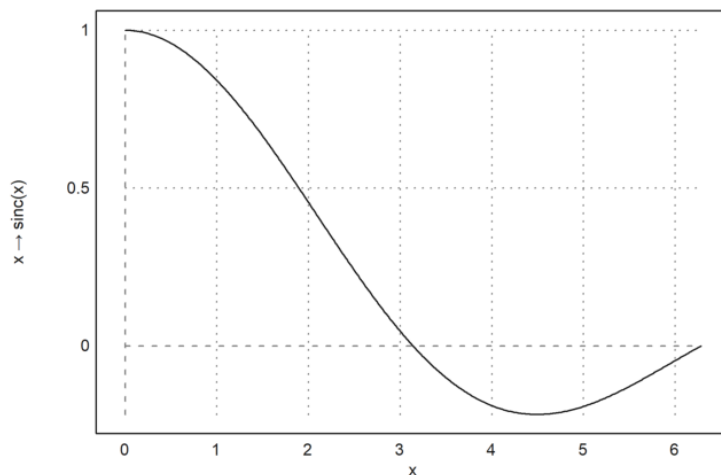
Berikut adalah contoh lain penggunaan kode **Unicode** dan menggambar sumbu koordinat di luar bidang koordinat.

```
>aspect(1.5);
>plot2d(["sin(x)", "cos(x)"], 0, 2pi, color=[red, green], ...
  <grid, <frame);
>xaxis(-1.1, (0:2)*pi, xt=["0", u"&pi; ", u"2&pi; "], ...
  style="-", >ticks, >zero);
>xgrid((0:0.5:2)*pi, <ticks);
>yaxis(-0.1*pi, -1:0.2:1, style="-", >zero, >grid);
>labelbox(["sin", "cos"], colors=[red, green], ...
  x=0.5, y=0.2, >left);
>xlabel(u"&phi;"); ylabel(u"f(&phi;)"):
```



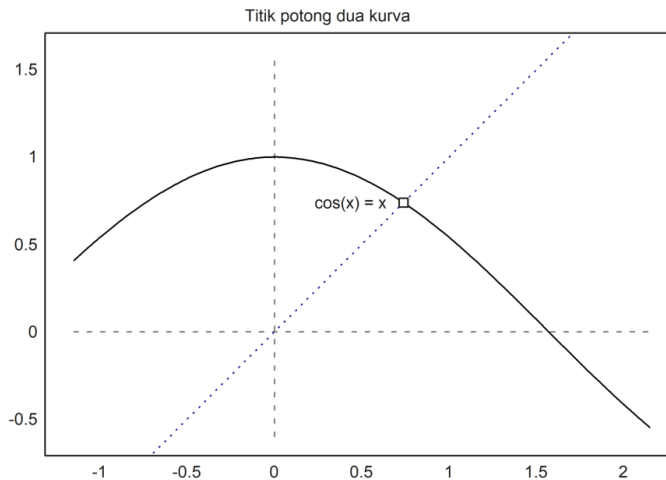
Pada contoh berikut ini ditunjukkan cara menggambar grafik plot dengan label sumbu y ditulis vertikal menggunakan opsi `>vertical`.

```
>plot2d("sinc(x)", 0, 2pi, xl="x", ...
  yl=u"x &rarr; sinc(x)", >vertical):
```



Pada contoh berikut ini, ditunjukkan bagaimana cara menggambar titik potong dua kurva dan memberi teks label pada titik potong. Untuk mencari absis titik potong digunakan perintah `solve()`. Posisi label titik potong menggunakan `"cl"`, artinya center left.

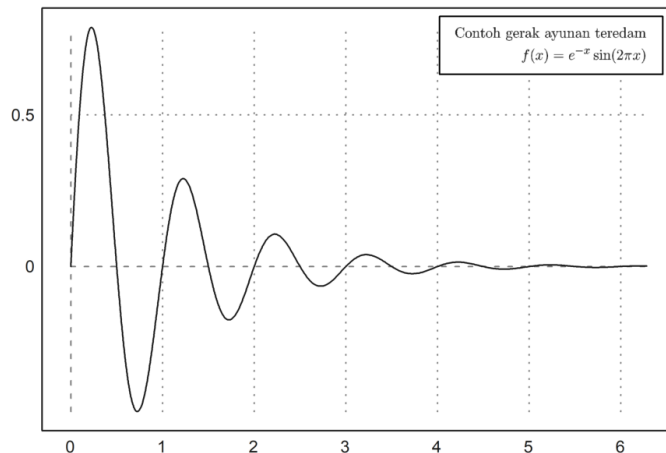

```
>plot2d(["cos(x)", "x"], r=1.1, cx=0.5, cy=0.5, ...
  color=[black, blue], style=["-", "."], ...
  grid=1);
>x0=solve("cos(x)-x", 1); ...
  plot2d(x0, x0, >points, >add, ...
  title="Titik potong dua kurva"); ...
  label("cos(x) = x", x0, x0, pos="cl", offset=20):
```



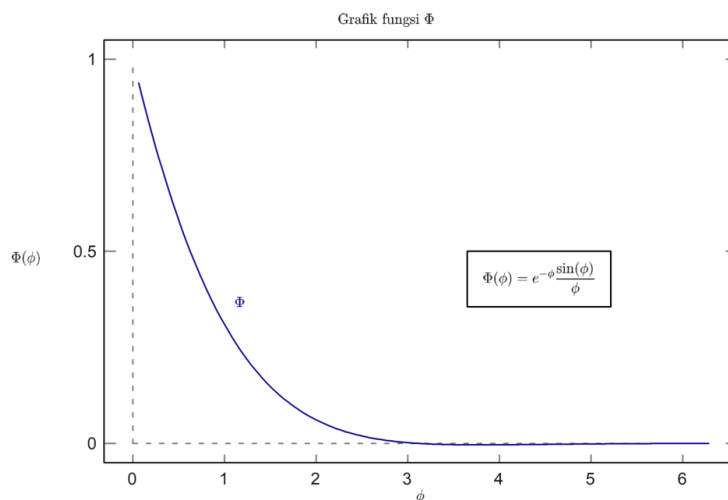
5.7.2 Teks label, keterangan, judul dengan LaTeX

Penulisan label, teks keterangan, dan judul grafik yang memuat ekspresi matematis dapat menggunakan LaTeX, asalkan komputer sudah dipasang sistem LaTeX. Caranya adalah dengan menuliskan semua label/teks di dalam perintah `latex()` dan diapit dengan tanda kutip dobel (" "). Sudah tentu untuk menampilkan ekspresi matematis dengan LaTeX, masing-masing harus ditulis dengan perintah LaTeX (diawali dengan tanda garis miring kiri (\)). Berikut adalah beberapa contoh memberi keterangan pada gambar plot menggunakan LaTeX. Biasanya proses menampilkan gambarnya menjadi agak lama, karena EMT harus menjalankan LaTeX untuk menampilkan ekspresi matematika dengan LaTeX. Sebagai imbalannya, tampilan teks matematis menjadi lebih baik dan enak dilihat dan dibaca.

```
>function f(x) &= exp(-x)*sin(2*pi*x); ...
  plot2d("f(x)", 0, 2pi); ...
  textbox(latex("\begin{aligned}...
  \text{Contoh gerak ayunan teredam}&\backslash\ ...
  f(x)=e^{-x}\sin(2\pi x)&\end{aligned}"))):
```



```
>plot2d("exp(-x)*sin(x)/x",a=0,b=2pi,c=0,d=1, ...
  grid=6,color=blue, ...
  title=latex("\text{Grafik fungsi }\Phi$"), ...
  xl=latex("\phi"),yl=latex("\Phi(\phi)"),<vertical);...
  textbox(latex("\Phi(\phi) = e^{-\phi}. ...
  textbox(latex( \frac{\sin(\phi)}{\phi}"), ...
  textbox(latex( x=0.8,y=0.5); ...
  label(latex("\Phi",color=blue),1,0.4):
```

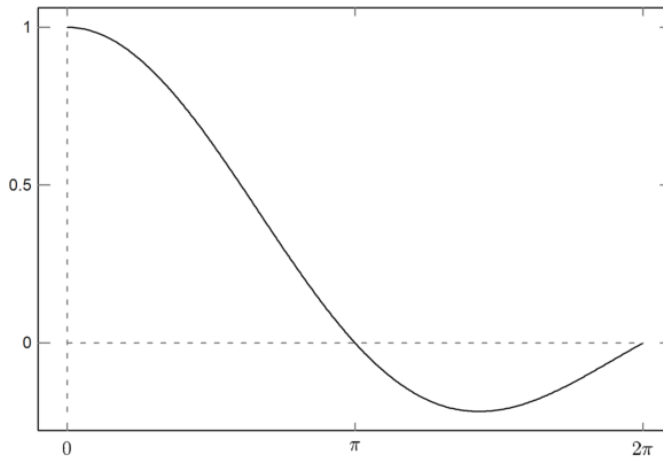


Seringkali, kita menginginkan spasi dan teks label secara khusus pada sumbu x . Hal ini dapat diatur dengan menggunakan opsi `xaxis()` dan `yaxis()`.

Cara termudah adalah membuat plot kosong dengan bingkai menggunakan opsi `grid=4`, kemudian menambahkan grid dengan `ygrid()` dan `xgrid()`. Pada

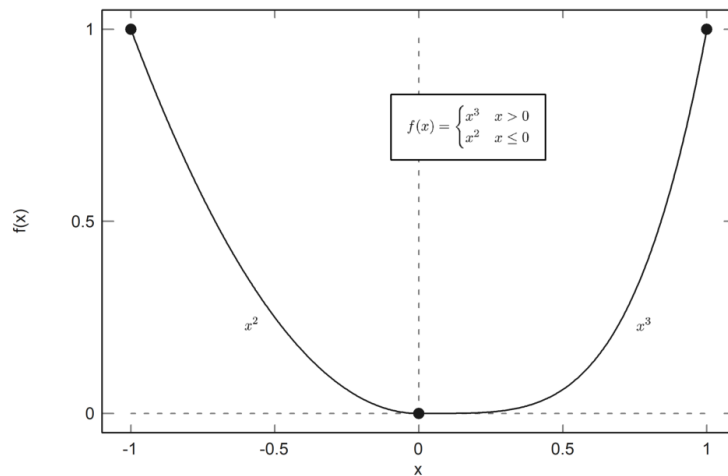
contoh berikut ini digunakan tiga string LaTeX untuk memberi label pada sumbu x dengan `xtick()`.

```
>plot2d("sinc(x)", 0, 2pi, grid=4, <ticks); ...
ygrid(-2:0.5:2, grid=6); ...
xgrid([0:2]*pi, <ticks, grid=6); ...
xtick([0, pi, 2pi], ["0", "\pi", "2\pi"], >latex):
```



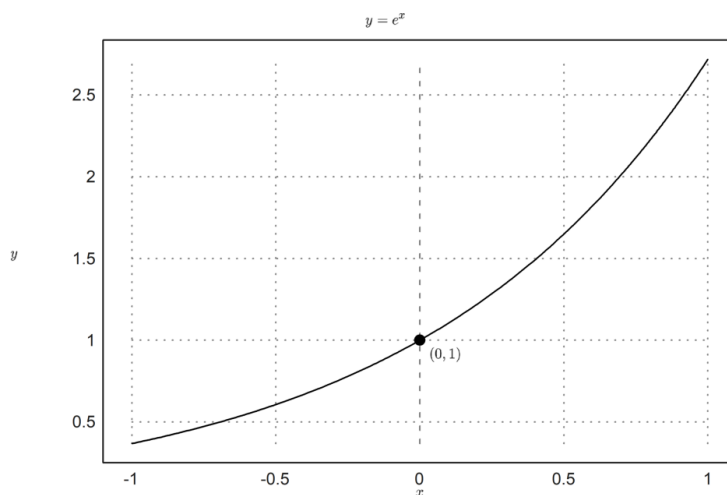
Berikut adalah contoh menampilkan grafik fungsi dengan dua rumus berbeda untuk interval yang berbeda, disertai teks keterangan menggunakan LaTeX.

```
>function map f(x) ...
  if x>0 then return x^4
  else return x^2
endif
endfunction
>plot2d("f", -1, 1, xl="x", yl="f(x)", grid=6, <vertical); ...
plot2d([-1, 0, 1], f([-1, 0, 1]), >points, >add); ...
label(latex("x^3"), 0.72, f(0.72)); ...
label(latex("x^2"), -0.52, f(-0.52), pos="l1"); ...
textbox(latex("f(x)=\begin{cases} x^3 & x>0 \\ ...
x^2 & x \le 0 \end{cases}"), ...
x=0.7, y=0.2):
```



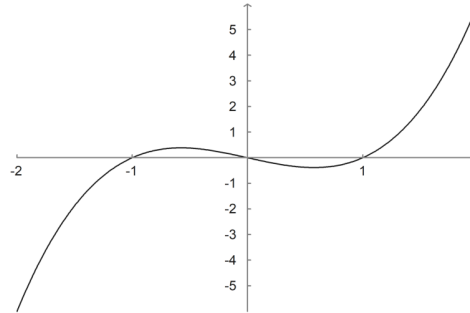
Berikut adalah contoh menggunakan LaTeX untuk menandai judul kurva, label sumbu-x, sumbu-y, dan menandai koordinat suatu titik pada kurva.

```
>plot2d("exp(x)", -1, 1);
>textcolor(black);
>title(latex("y=e^x"));
>xlabel(latex("x"));
>ylabel(latex("y"), <vertical);
>plot2d(0, 1, >points, >add, style="o#");
>label(latex("(0,1)"), 0, 1, color=blue):
```

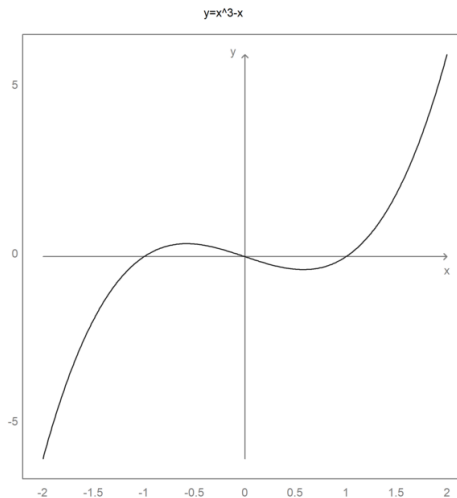


Berikut adalah contoh menggambar sumbu koordinat dengan anak panah dan skala sesuai keinginan kita menggunakan `xaxis()` dan `yaxis()`.

```
>plot2d("x^3-x", <grid, <frame);
>xaxis(0, xx=-2:1, style="->");
>yaxis(0, yy=-5:5, style="->"):
```



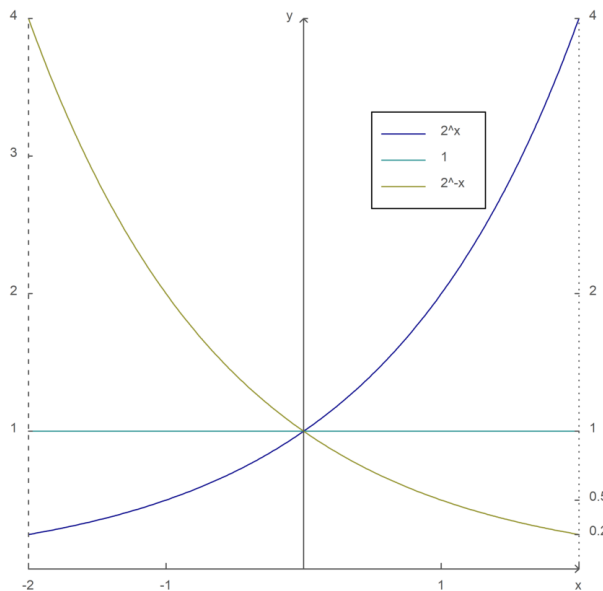
```
>gridstyle("->", color=gray, textcolor=gray, ...
  framecolor=gray); ...
plot2d("x^3-x", grid=1); ...
settitle("y=x^3-x", color=black); ...
label("x", 2, 0, pos="bc", color=gray); ...
label("y", 0, 6, pos="cl", color=gray); ...
reset():
```



Apabila tidak diperlukan label untuk sumbu-sumbu koordinat, perintah `fullwindow()` dapat digunakan untuk memperluas jendela plot. Untuk mengembalikan ke kondisi aslinya dapat digunakan perintah `shrinkwindow()`

atau `reset()` seperti contoh berikut ini.

```
>fullwindow;
>gridstyle(color=darkgray,textcolor=darkgray);
>plot2d(["2^x", "1", "2^(-x)"], a=-2, b=2, c=0, d=4, ...
  <grid, color=4:6, <frame);
>xaxis(0, -2:1, style="->"); xaxis(0, 2, "x", <axis);
>yaxis(0, 4, "y", style="->");
>yaxis(-2, 1:4, >left);
>yaxis(2, 2^(-2:2), style=".", <left);
>labelbox(["2^x", "1", "2^-x"], colors=4:6, x=0.8, y=0.2);
>reset:
```



5.8 Menggambar grafik interaktif

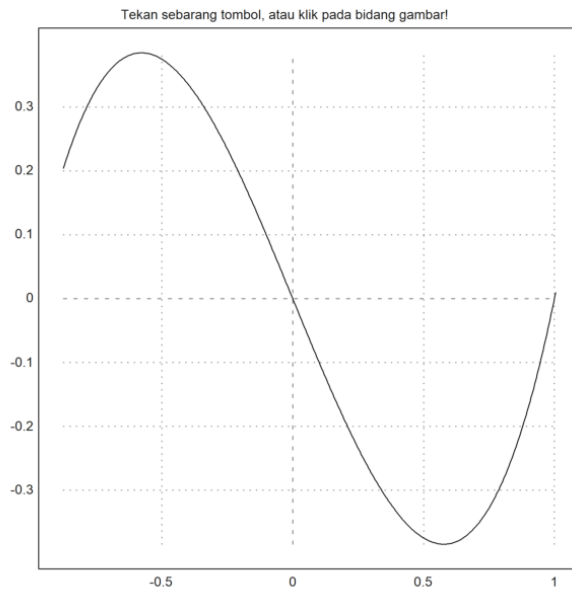
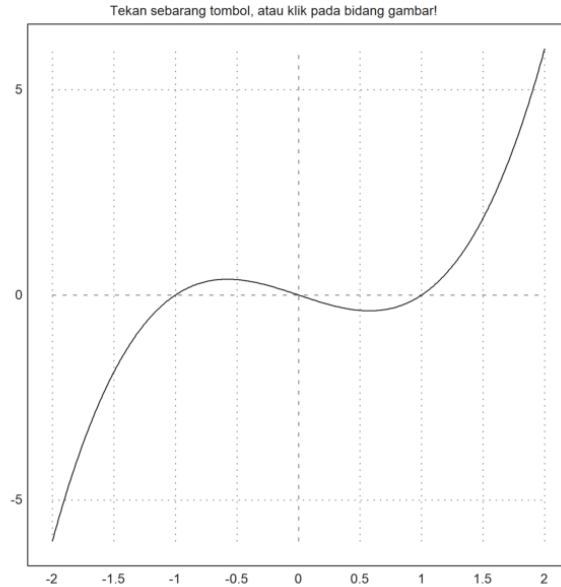
Pada perintah `plot2d()` dapat digunakan opsi `>user` untuk memungkinkan pengguna berinteraksi dengan gambar plot menggunakan *mouse* atau tombol, yakni:

- memperbesar atau memperkecil gambar dengan tombol `[+]` atau `[-]`,
- menggeser gambar dengan tombol panah,
- memilih jendela plot dengan *mouse*,
- mengembalikan tampilan gambar dengan tombol spasi,
- keluar interaksi dengan tombol `[Enter]`.

Cobalah perintah di bawah ini di layar notebook EMT (ditulis pada baris perintah yang bertanda `>`) dan perhatikan apa yang terjadi, kemudian lakukan sesuai

petunjuk yang muncul.

```
>plot2d({"x^3-a*x",a=1}},>user,...
  title="Tekan sebarang tombol, atau klik pada ...
  bidang gambar!":
```



Klik di daerah gambar (atau tekan tombol $[+]$ atau $[-]$) untuk melihat gambar lebih dekat/jauh. Tekan spasi untuk mengembalikan gambar seperti semula. Jika ditekan tombol $[Enter]$, gambar akan hilang dan muncul pesan di bawah

ini.

```
Row index 2 out of bounds!
Try "trace errors" to inspect local variables after
errors.
plot2d:
    if auto then k2[2]=k1[2]; endif;
```

Berikut ini ditunjukkan cara menggambar plot interaktif lebih lanjut. Fungsi bawaan `mousedrag()` dapat digunakan untuk menunggu interaksi menggunakan *mouse* maupun tombol. Interaksi dapat dilakukan dengan memindahkan salah satu titik dengan *mouse* ke bawah, atas, kiri, kanan, atau menggunakan tombol. Fungsi `dragpoints()` memungkinkan pengguna menyeret titik mana pun di dalam bidang plot.

Mula-mula kita definisikan fungsi `plotf()` dengan input pasangan nilai-nilai koordinat titik-titik data dan nilai indeks titik yang dipilih pengguna untuk digeser/diseret. Fungsi ini melakukan langkah-langkah:

1. menghitung koefisien-koefisien polinomial interpolasi menggunakan metode selisih terbagi Newton;
2. menggambar kurva polinomial interpolasi beserta titik-titik yang diinterpolasikan;
3. memindahkan titik yang diseret pengguna dan mengganti warnanya menjadi merah, dan kurva polinomial interpolasinya otomatis menyesuaikan.

Catatan:

Polinomial yang menginterpolasikan titik-titik (x_1, y_1) , (x_2, y_2) , ..., (x_n, y_n) dengan koefisien hasil selisih terbagi Newton d_1 , d_2 , ..., d_n dapat ditulis dalam bentuk:

$$P_{n-1}(x) = d_1 + d_2(x - x_1) + d_3(x - x_1)(x - x_2) + \dots \\ + d_n(x - x_1)(x - x_2) \dots (x - x_{n-1}).$$

Untuk menghitung nilai $P_{n-1}(x)$ dapat digunakan fungsi EMT `interpval()` dengan input data absis (x), matriks koefisien hasil selisih terbagi Newton (d), dan nilai-nilai di mana P_{n-1} akan dihitung nilainya. Perhatikan, penggunaan fungsi `interpval()` di dalam fungsi/perintah `plot2d()` harus dipisah menggunakan titik koma (;), bukan koma, dengan parameter berikutnya. Tanpa titik koma ini, diperlukan fungsi `plotinterp()` yang harus didefinisikan terlebih dahulu, untuk mengakses nilai secara global.


```

>function plotf(xp,yp,select) ...
    d=interp(xp,yp); ...
    plot2d("interpval(xp,d,x)";d,xp,r=2); ...
    plot2d(xp,yp,>points,>add); ...
    if select>0 then ...
    plot2d(xp[select],yp[select],color=red,>points,>add);...
    endif; ...
    title("Seret satu titik, atau tekan tombol spasi ...
    atau Enter!");
endfunction

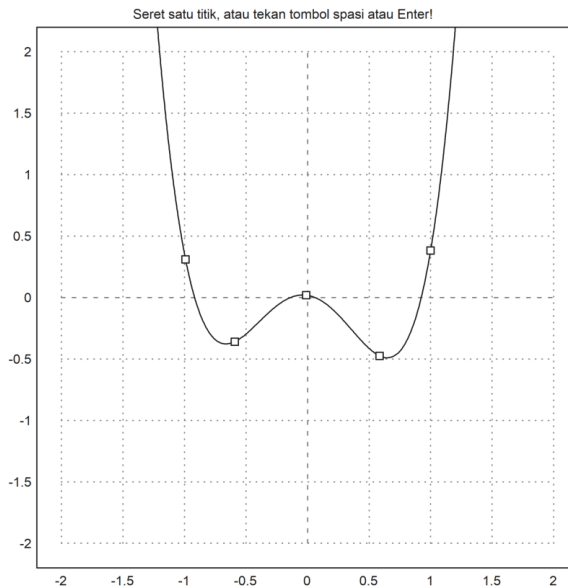
```

Fungsi tersebut selanjutnya digunakan sebagai input pada perintah `dragpoints()` untuk menghasilkan plot interaktif. Pada demonstrasi berikut ini digunakan lima titik acak yang diinterpolasikan dan dapat dipindah-pindah oleh pengguna setelah plot ditampilkan.

```

>t=-1:0.5:1; dragpoints("plotf",t,random(size(t))-0.5):

```



Kita juga dapat mendefinisikan suatu fungsi dengan input/parameter berupa vektor yang elemen-elemennya merupakan parameter fungsi matematika yang kurvanya dapat diubah oleh pengguna dengan cara mengubah nilai elemen-elemen vektor tersebut. Sebagai contoh, berikut adalah cara menggambar kurva fungsi

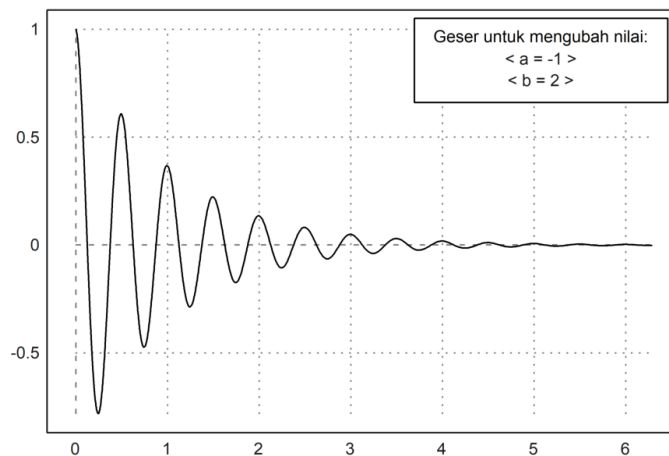
$$y = e^{ax} \cdot \cos(2\pi \cdot b \cdot x),$$

dengan nilai-nilai a dan b dapat diubah oleh pengguna. Fungsi dimaksud dapat kita definisikan sebagai berikut.

```
>function kurvatif([a,b]) ...
  plot2d("exp(a*x)*cos(2pi*b*x)",0,2pi;a,b); ...
endfunction;
```

Selanjutnya, fungsi tersebut digunakan sebagai input/parameter perintah/fungsi `dragvalues()`. Fungsi ini, selain memerlukan input nama fungsi untuk menggambar plot fungsi matematika yang memuat parameter, juga memerlukan vektor string untuk menamai parameter-parameter fungsi matematika tersebut, vektor nilai awalnya, dan matriks $n \times 2$ yang berisi batas-batas rentang nilai masing-masing parameter, serta opsi-opsi lain, termasuk memberi judul di atas nama parameter yang dapat diubah. Pada contoh berikut, kurva fungsi di atas mula-mula digambar dengan $a = -1$ dan $b = 2$, kemudian dapat diubah oleh pengguna pada rentang $-2 \leq a \leq 2$ dan $1 \leq b \leq 10$, masing-masing dengan 20 stop.

```
>aspect(1.5)
>dragvalues("kurvatif",["a","b"],...
  [-1,2],[[-2,2];[1,10]],20,...
  heading="Geser untuk mengubah nilai:",hcolor=blue):
```



Selanjutnya, pengguna dapat melakukan perubahan nilai a atau b dan kurva otomatis akan berubah. Apabila pengguna menekan tombol `[Esc]`, maka gambar akan ditutup dan muncul pesan di bawah ini.

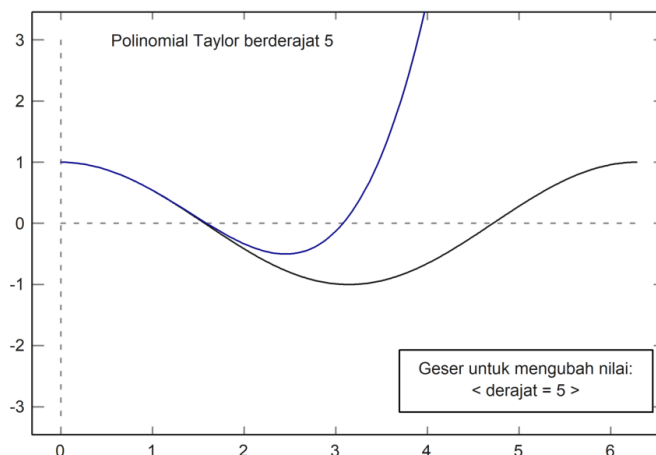
```
Interrupt with Escape key!
Try "trace errors" to inspect local variables after ...
errors.
dragvalues:
    {flag,m,t}=mousedrag(status);
```

Perubahan nilai juga dapat dibatasi pada bilangan bulat. Sebagai contoh, misalkan kita akan menggambar kurva polinomial Taylor berderajat n untuk hampiran fungsi $\sin(x)$.

```
>function tay4sin(n) ...
    plot2d("sin(x)", 0, 3pi);
    plot2d(&"taylor(sin(x), x, 0, @n)", color=red, >add);
    textbox("Polinomial Taylor berderajat "...
    +n, 0.3, 0.02, style="t", >left);
endfunction
```

Pada perintah `dragvalues()` di bawah ini, nilai "derajat" mula-mula 2 kemudian dapat diubah oleh pengguna mulai dari 1 sampai 20 dengan 19 stop.

```
>dragvalues("tay4cos", "derajat", 2, [1, 20], 19, y=0.8, ...
    heading="Geser untuk mengubah nilai:"):
    < derajat = 5 >
```

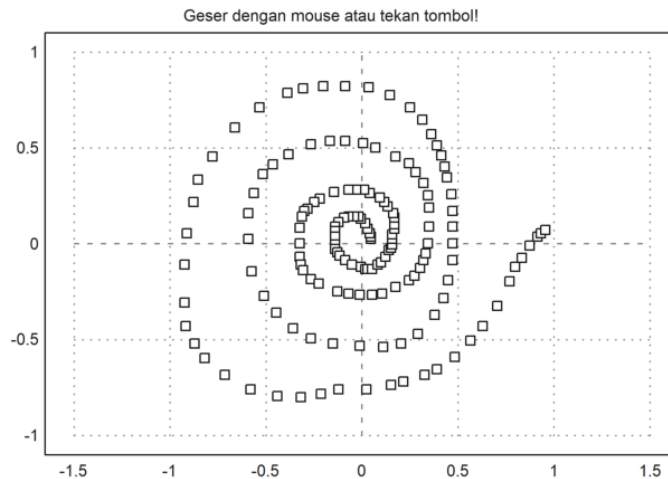


Sebagai contoh terakhir membuat grafik interaktif, berikut adalah fungsi untuk menggambar bidang koordinat kosong, kemudian pengguna dapat menggambar beberapa titik dengan cara menggeser mouse pada bidang koordinat.

```

>function gambar ...
plot2d(none,r=1,...
title="Geser dengan mouse atau tekan tombol!");
start=0;
repeat
  {flag,m,time}=mousedrag();
  if flag==0 then return; endif;
  if flag==2 then
    hold on; mark(m[1],m[2]); hold off;
  endif;
end
endfunction
>gambar // lihat hasilnya dan cobalah lakukan!

```



5.9 Menggambar data 2D (titik)

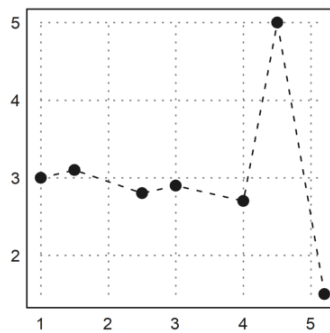
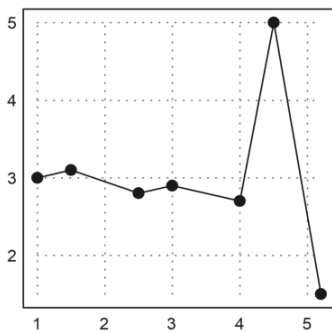
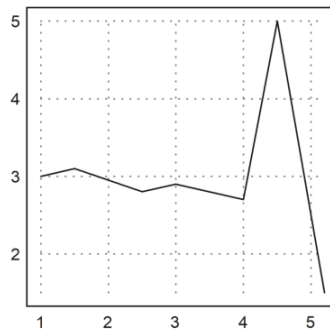
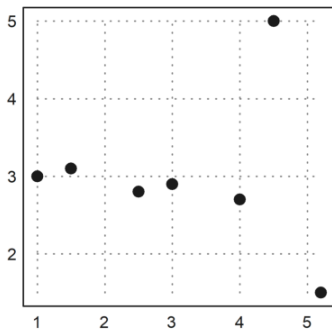
Seperti halnya perintah dasar untuk menggambar kurva, yakni `plot()`, perintah `plot2d` juga dapat digunakan untuk menggambar kurva yang melalui titik-titik (x_k, y_k) , dengan $k = 1, 2, 3, \dots, n$, untuk suatu nilai n bilangan asli. Dalam hal ini, nilai-nilai x_k dan y_k dapat disimpan di dalam vektor baris atau matriks x dan y yang berukuran $1 \times n$. Sebenarnya, ketika menggambar kurva menggunakan ekspresi atau fungsi, EMT juga melakukan hal yang sama:

1. Mula-mula ditentukan nilai-nilai x_k , berdasarkan rentang nilai aslinya atau sesuai yang ditentukan di dalam parameter perintah `plot2d()`;
2. Menghitung nilai-nilai y_k menggunakan rumus di dalam ekspresi yang ditulis atau rumus fungsi yang namanya digunakan sebagai parameter;
3. Menggambar ruas-ruas garis yang menghubungkan titik-titik (x_k, y_k) ;
4. Apabila digunakan opsi `>points` yang digambar hanya titik-titik saja, dan

apabila digunakan opsi `>addpoints` yang digambar garis dan titik.

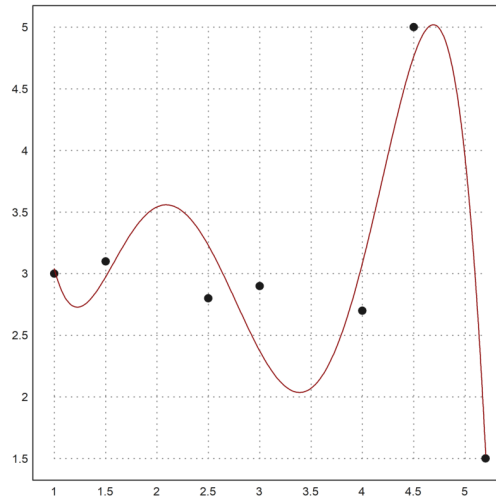
Berikut adalah contoh yang menunjukkan 4 cara berbeda menggambar plot data x dan y .

```
>x=[1,1.5,2.5,3,4,4.5,5.2]; y=[3,3.1,2.8,2.9,2.7,5,1.5];
>figure(2,2);
>//diagram titik
>figure(1); plot2d(x,y,>points, style="o#");
>// diagram (ruas) garis
>figure(2); plot2d(x,y, style="-");
>// diagram garis dan titik
>figure(3); plot2d(x,y,>addpoints, pointstyle="o#");
>// diagram garis dan titik
>figure(4);
>plot2d(x,y,style="--",>addpoints, pointstyle="o#");
>figure(0):
```



Berikut adalah contoh menggambar data titik bersama kurva regresi polinomial berderajat

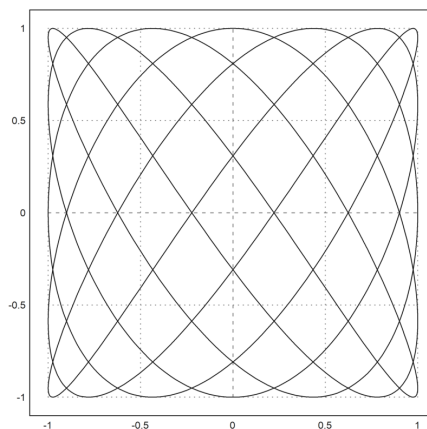
```
>// simpan regresi polinomial berderajat 5
>p=polyfit(x,y,5);
>plot2d(x,y,>points,style="o#");
> // diagram titik bersama garis regresi
>plot2d("polyval(p,x)",>add,color=red):
```



Ekspresi simbolik juga dapat digunakan untuk mendefinisikan nilai-nilai x dan y . Berikut adalah contoh menggambar pasangan nilai-nilai

$$x = \sin(5t), \quad y = \cos(7t), \quad \text{dengan } 0 \leq t \leq 2\pi.$$

```
>t=linspace(0,2pi,1000); plot2d(sin(5t),cos(7t)):
```



Bab 6.

Visualisasi Grafik 2 Dimensi (2D) Lebih Lanjut

Bab ini adalah lanjutan bab sebelumnya, menjelaskan visualisasi grafik 2 dimensi dengan menggunakan fungsi implisit, fungsi logaritmik, fungsi parametrik dan persamaan polar, fungsi kompleks, dan menggambar daerah yang dibatasi oleh dua kurva, serta menggambar segi banyak.

6.1 Kurva fungsi implisit

Fungsi implisit adalah fungsi yang dinyatakan dalam bentuk

$$f(x,y) = k,$$

dengan k suatu konstanta (bilangan/nilai) tertentu. Untuk menggambar grafik/kurva fungsi implisit, nilai k digunakan sebagai nilai opsi/parameter `level=k` pada perintah `plot2d()`. Nilai k tersebut dapat berupa bilangan tunggal atau berupa vektor. Jika digunakan `level="auto"`, maka akan digambar nc garis kontur (ketinggian) mulai dari nilai minimum sampai nilai maksimum fungsi secara merata. Untuk memberi warna lebih gelap atau lebih terang, yang menunjukkan nilai fungsi, dapat ditambahkan opsi `>hue`. Untuk fungsi implisit, `xv` harus berupa fungsi atau ekspresi dari parameter x dan y , atau alternatifnya, `xv` bisa berupa matriks nilai. Parameter n menentukan tingkat kehalusan plot.

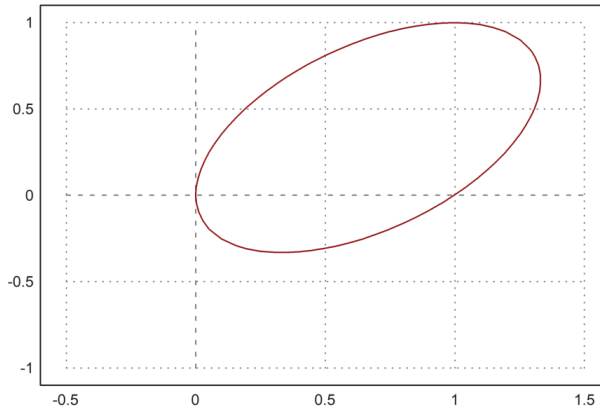
Euler dapat menandai garis ketinggian $f(x,y) = k$ untuk fungsi apa pun. Berikut ini ditunjukkan berbagai cara menggambar grafik fungsi implisit (kurva ketinggian).

Contoh pertama adalah menggambar kurva

$$x^2 + y^2 - xy - x = 0,$$

yang merupakan elips.

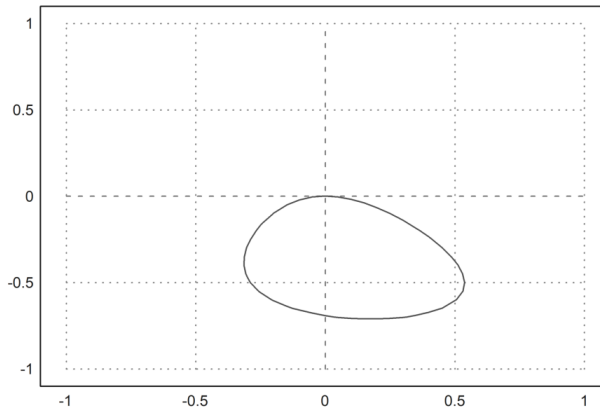

```
>aspect(1.5);
>plot2d("x^2+y^2-x*y-x",a=-.5,b=1.5,level=0, ...
  contourcolor=red):
```



Berikut adalah contoh lain yang menunjukkan kurva

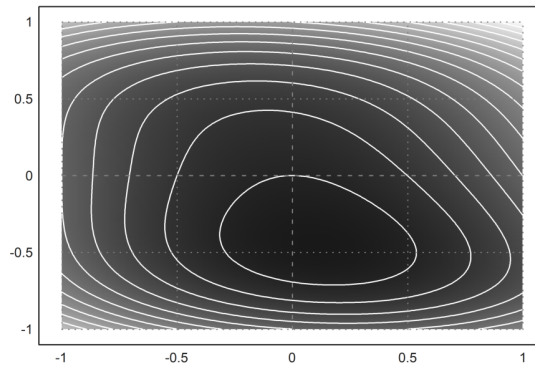
$$2x^2 + xy + 3y^4 + y = 0.$$

```
>fxy := "2*x^2+x*y+3*y^4+y"; //menyimpan ekspresi f(x,y)
>plot2d(fxy,level=0): // Menggambar kurva f(x,y)=0
```

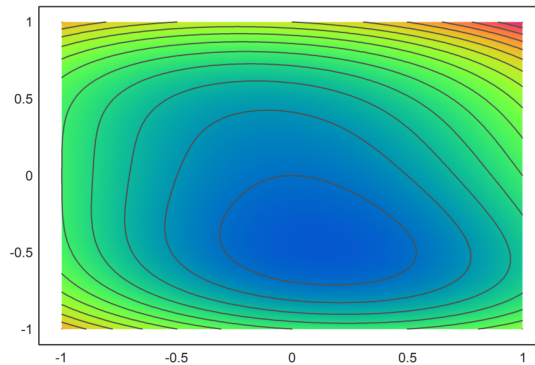


Berikut ini ditunjukkan kurva ketinggian dengan berbagai spektrum warna yang menunjukkan tingkat ketinggian.

```
>plot2d(fxy,level=0:0.5:20,>hue, ...
  contourcolor=white,n=200): // peta kontur hitam putih
```

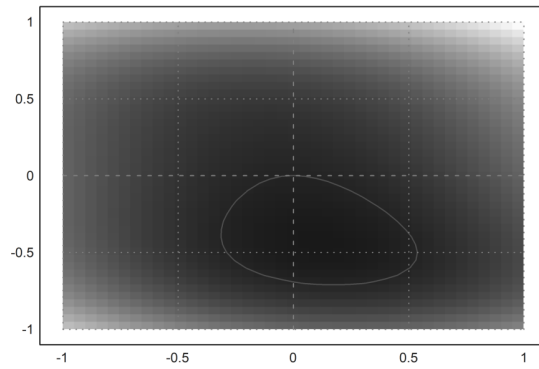


```
>plot2d(fxy,level=0:0.5:20,>hue, ...
>spectral,n=200,grid=4): // peta kontur berwarna
```

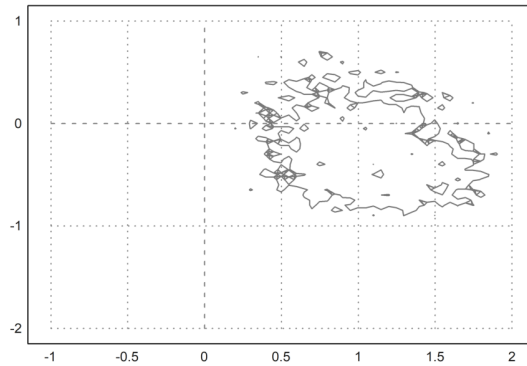


Nilai-nilai x dan y pada fungsi implisit dapat diberikan dalam bentuk data. Dalam hal ini, pada perintah `plot2d()` harus dinyatakan batas-batas x dan y .

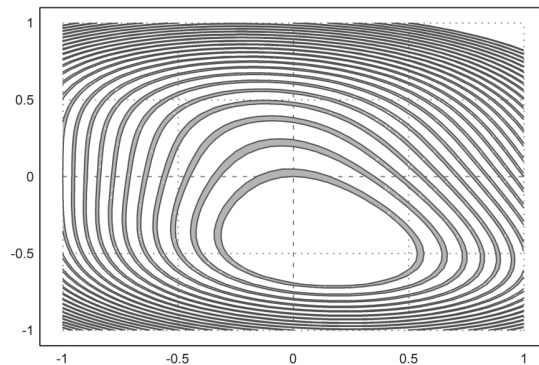
```
>x=-1:0.05:1; y=x'; z=fxy(x,y);
>plot2d(z,level=0,a=-1,b=1,c=-1,d=1,>hue):
```



```
>z=z+normal(size(z))*0.2;
>plot2d(z,level=0.5,a=-1,b=2,c=-2,d=1):
```



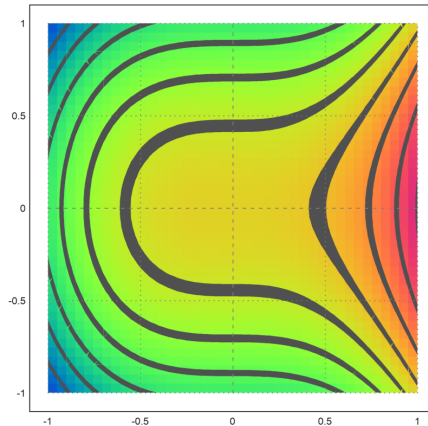
```
>plot2d(fxy,level=[0:0.2:5;0.05:0.2:5.05], ...
color=lightgray):
```



Ekspresi/rumus fungsi implisit juga dapat langsung dituliskan di dalam perin-

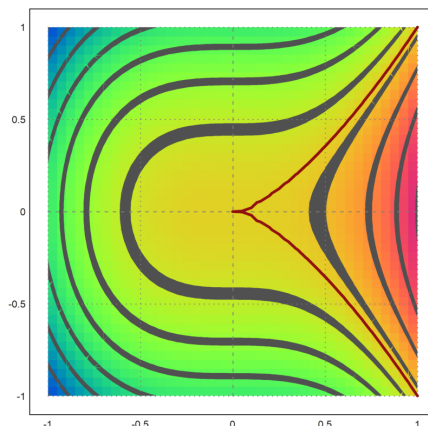
tah `plot2d()`. Opsi `>contour` akan menghasilkan peta kontur warna secara otomatis.

```
>plot2d("x^3-y^2",>contour,>hue,>spectral):
```

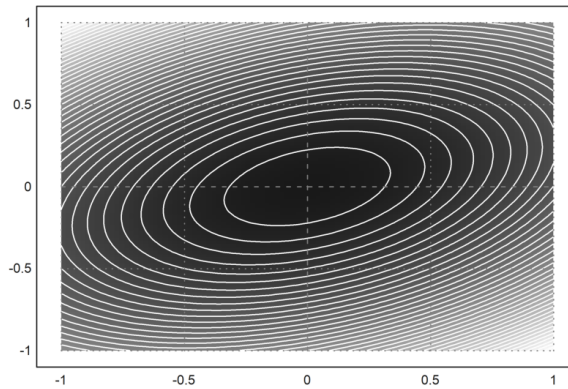


Berikut adalah contoh menggambar peta kontur dengan ketebalan dan warna kontur ditentukan.

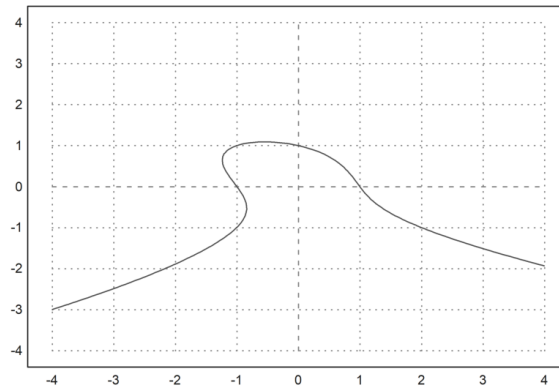
```
>plot2d("x^3-y^2",level=0,contourwidth=3,>add, ...
  contourcolor=red):
```



```
>plot2d("x^2+2*y^2-x*y",level=0:0.1:10,n=100, ...
  contourcolor=white,>hue):
```



```
>plot2d("x^2+y^3+x*y", level=1, r=4, n=100) :
```

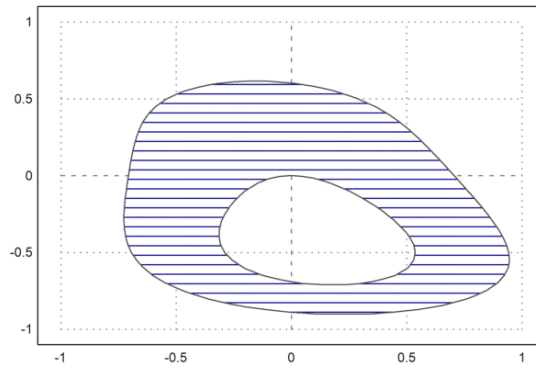


Kita juga dapat menggambar daerah

$$a \leq f(x,y) \leq b$$

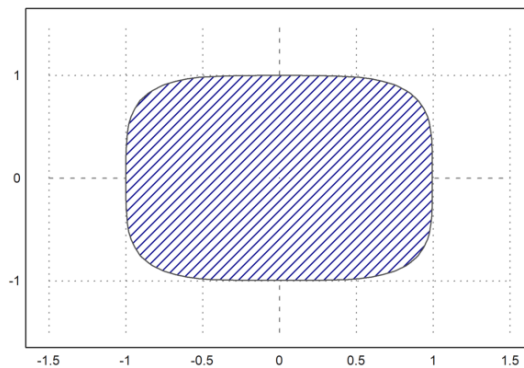
dengan menggunakan nilai a dan b sebagai batas-batas nilai `level`. Dalam hal ini, nilai `level` harus berupa matriks $2 \times n$. Baris pertama adalah nilai-nilai batas bawah, dan baris kedua adalah nilai-nilai batas atas. Opsi `style` dapat digunakan untuk menentukan model arsiran. Berikut adalah beberapa contoh menggambar daerah antara dua kontur dengan menggunakan fungsi implisit.

```
>// 0 <= f(x,y) <= 1
>plot2d(fxy, level=[0;1], style="-", color=blue) :
```

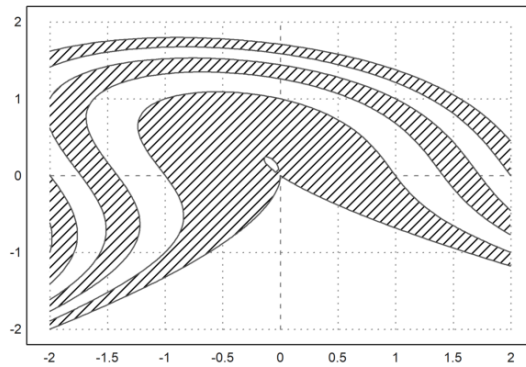


Kurva fungsi implisit juga dapat menunjukkan rentang level. Dalam hal ini level harus berupa matriks interval $2 \times n$, dengan baris pertama berisi batas bawah dan baris kedua berisi batas atas setiap interval.

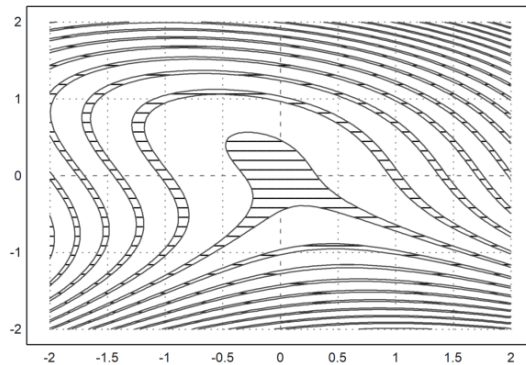
```
>plot2d("x^4+y^4",r=1.5,level=[0;1],color=blue, ...
style="/") :
```



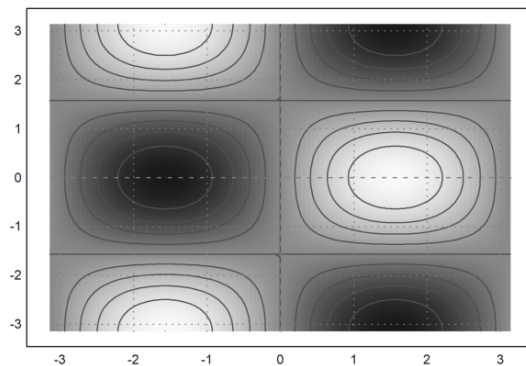
```
>plot2d("x^2+y^3+x*y",level=[0,2,4;1,3,5], ...
style="/",r=2,n=100) :
```



```
>plot2d("x^2+y^3+x*y",level=-10:20,r=2,style="-", ...
  dl=0.1,n=100):
```



```
>plot2d("sin(x)*cos(y)",r=pi,>hue,>levels,n=100):
```

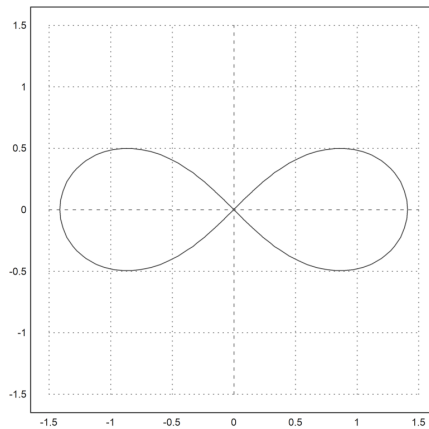


Salah satu kurva yang sangat terkenal adalah *lemniscate* dengan persamaan

implisit

$$(x^2 + y^2)^2 = 2a^2(x^2 - y^2).$$

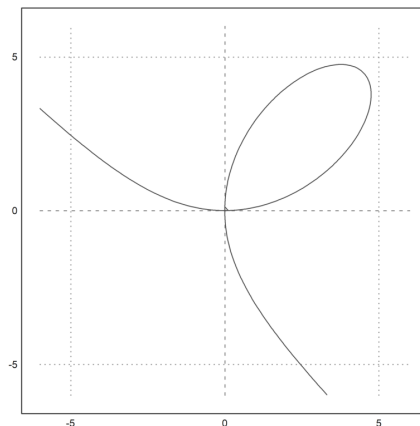
```
>aspect(1);
>a=1; plot2d("(x^2 + y^2)^2 - 2*a^2*(x^2 - y^2)", ...
  r=1.5, level=0) :
```



Kurva fungsi implisit lain yang terkenal adalah *folium Descartes* dengan persamaan

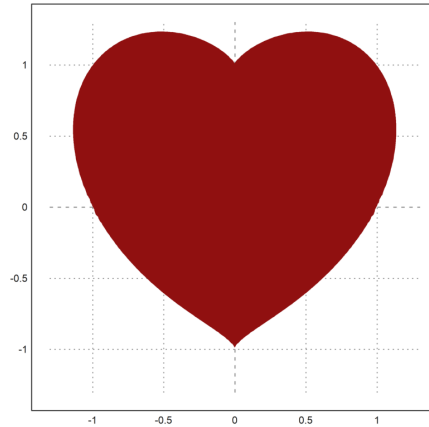
$$x^3 + y^3 = 3axy.$$

```
>a=3; plot2d("x^3+y^3-3*a*x*y", r=6, level=0, n=100) :
```



Berikut adalah contoh menggambar suatu bentuk yang menarik (hati/jantung) menggunakan kurva fungsi implisit sebagai pembatas bangun/bentuk tersebut.


```
>plot2d("(x^2+y^2-1)^3-x^2*y^3",r=1.3, ...
style="#",color=red,<outline, ...
level=[-2;0],n=100):
```



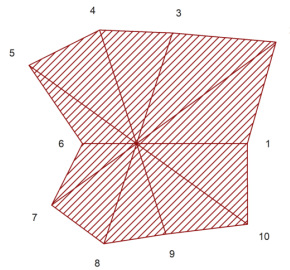
Berikut ini adalah definisi suatu fungsi/program untuk menggambar daerah segi banyak yang diagonal-diagonalnya berpotongan di satu titik. Dalam hal ini tidak digunakan perintah `plot2d()` dan sumbu koordinat, namun digunakan perintah dasar untuk menggambar. Jendela untuk menggambar adalah layar penuh.

```
>function starplot1(v,style="/",color=green,lab=none) ...
if !holding() then clg; endif;
w=window(); window(0,0,1024,1024);
h=holding(1);
r=max(abs(v))*1.2;
setplot(-r,r,-r,r);
n=cols(v); t=linspace(0,2pi,n);
v=v|v[1]; c=v*cos(t); s=v*sin(t);
cl=barcolor(color); st=barstyle(style);
loop 1 to n
  polygon([0,c[#],c[#+1]], [0,s[#],s[#+1]],1);
  if lab!=none then
    rlab=v[#]+r*0.1;
    {col,row}=toscreen(cos(t[#])*rlab,sin(t[#])*rlab);
    ctext(""+lab[#],col,row-textheight()/2);
  endif;
end;
end;
```

```

barcolor(cl); barstyle(st);
holding(h);
window(w);
endfunction
>reset; starplot1(normal(1,10)+5,color=red,lab=1:10):

```



Terkadang, kita ingin menggambar grafik yang tidak dapat dihasilkan dengan perintah `plot2d()` secara persis, melainkan hanya grafik yang serupa. Sebagai contoh, perintah `plot2d()` dapat digunakan untuk menggambar kurva logaritma dan diagram batang, namun tidak dapat digunakan untuk menggambar diagram impuls. Fungsi berikut ini didefinisikan untuk menggambar diagram impuls logaritmik.

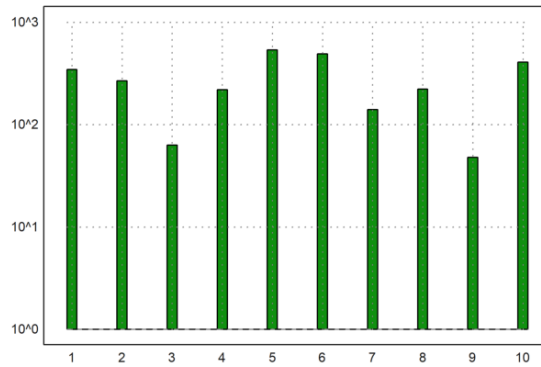
```

>function logimpulseplot1 (x,y) ...
    {x0,y0}=makeimpulse(x,log(y)/log(10));
    plot2d(x0,y0,>bar,grid=0);
    h=holding(1);
    frame();
    xgrid(ticks(x));
    p=plot();
    for i=-10 to 10;
        if i<=p[4] and i>=p[3] then
            ygrid(i,yt="10^"+i);
        endif;
    end;
    holding(h);
endfunction

```

Berikut kita gunakan program tersebut untuk menggambar diagram impuls dengan nilai-nilai berdistribusi eksponensial.

```
>aspect(1.5); x=1:10; y=-log(random(size(x)))*200; ...
logimpulseplot1(x,y):
```



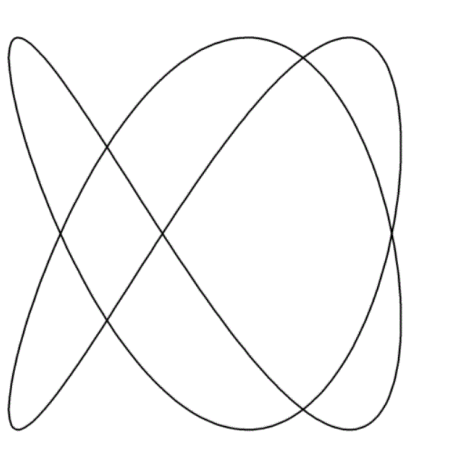
Sebelum beralih ke topik berikutnya, mari kita membuat animasi kurva 2D menggunakan perintah `plot()` langsung. Perintah `plot(x,y)` menggambar kurva pada jendela grafik. Perintah `setplot(a,b,c,d)` dapat digunakan untuk mengatur jendela gambar. Fungsi `wait(0)` memaksa kurva untuk muncul di jendela grafik. Jika tidak digunakan, kurva akan digambar secara berulang dengan interval waktu yang agak jarang untuk menghasilkan seperti animasi.

```
>function animliss (n,m) ...
t=linspace(0,2pi,500);
f=0;
c=framecolor(0);
l=linewidth(2);
setplot(-1,1,-1,1);
repeat
    clg;
    plot(sin(n*t),cos(m*t+f));
    wait(0);
    if testkey() then break; endif;
    f=f+0.02;
end;
framecolor(c);
linewidth(l);
endfunction
```

Berikut adalah salah satu contoh menggunakan program tersebut untuk menampilkan animasi kurva.

```
>animliss(2,3);
>// lihat hasilnya, jika sudah puas, tekan ENTER
```

Animasi kurva ditampilkan pada jendela/layar grafik. Untuk menghentikan animasi (sekaligus menutup layar grafik), tekan sebarang tombol.



6.2 Grafik fungsi logaritmik

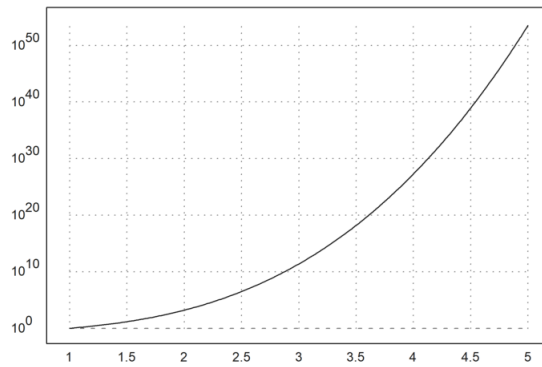
Untuk menggambar grafik fungsi logaritma, EMT tetap menggunakan fungsi/perintah `plot2d()` dengan parameter `logplot`. Kurva logaritma dapat digambar dengan menggunakan skala logaritma dalam y (menggunakan parameter `logplot=1`), atau skala logaritma dalam x dan y (`logplot=2`), atau skala logaritma dalam x (`logplot=3`). Perhatikan beberapa contoh di bawah ini.

Pertama, kita akan tunjukkan kurva

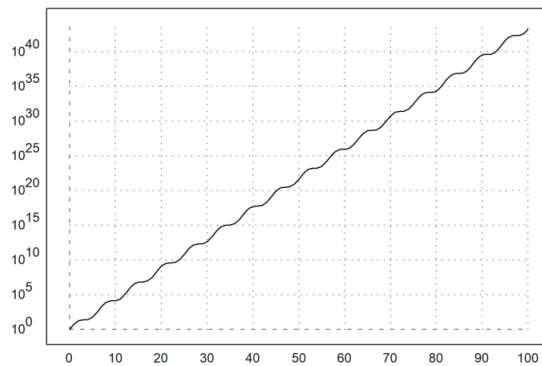
$$y = x^2 \cdot e^{x^3 - x}$$

pada interval $[1, 5]$. Jika digunakan skala normal, kurva tidak terlihat begitu membantu.

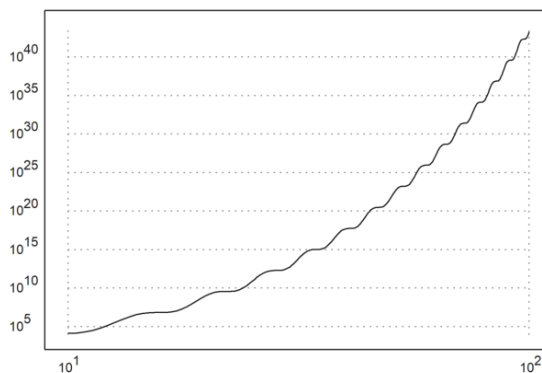
```
>//plot2d("exp(x^3-x)*x^2",1,5):
>// kurva terlihat tidak membantu dengan skala normal
>//skala logaritma pada sumbu-y
>plot2d("exp(x^3-x)*x^2",1,5,logplot=1):
```



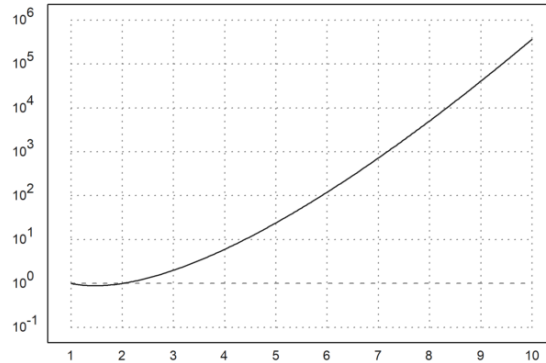
```
>plot2d("exp(x+sin(x))",0,100,logplot=1):
```



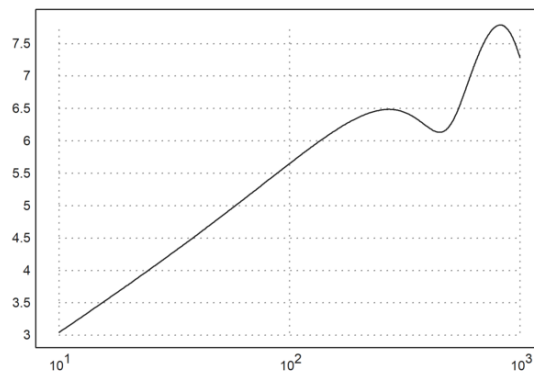
```
>// skala logaritma pada sumbu-x, y
>plot2d("exp(x+sin(x))",10,100,logplot=2):
```



```
>plot2d("gamma(x)",1,10,logplot=1):
```



```
>// skala logaritma pada sumbu-x  
>plot2d("log(x*(2+sin(x/100)))",10,1000,logplot=3):
```

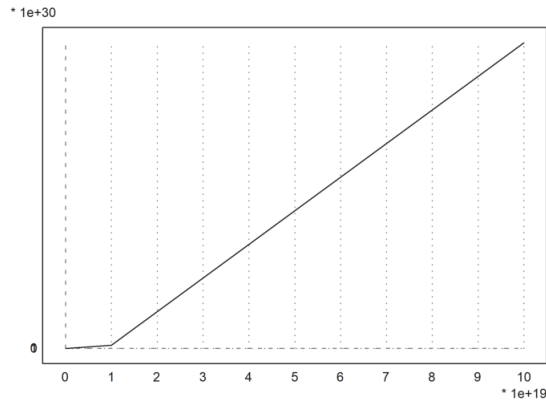


Skala logaritma juga dapat digunakan pada plot pasangan data, seperti pada contoh berikut ini untuk menggambar kurva

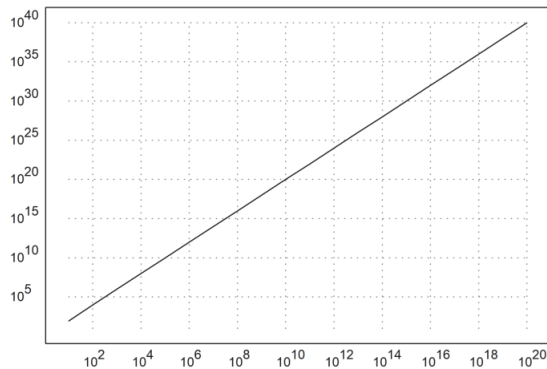
$$y = x^2 - x \text{ untuk } x = 10, 10^2, 10^3, \dots, x^{20}.$$

Perhatikan perbedaan grafik yang tanpa skala logaritma dan yang menggunakannya.

```
>x=10^(1:20); y=x^2-x;  
>plot2d(x,y): // skala normal pada kedua sumbu koordinat
```



```
>// kedua sumbu menggunakan skala logaritma
>plot2d(x,y,logplot=2):
```



6.3 Grafik fungsi parametrik dan persamaan polar

Seperti sudah diketahui, pada fungsi parametrik, nilai-nilai x dan y merupakan fungsi dalam suatu parameter, misalnya t , yakni:

$$x = f(t), y = g(t).$$

Untuk menggambar kurva fungsi parametrik, nilai-nilai x dan y yang dihitung menggunakan rumus masing-masing digunakan sebagai input fungsi/perintah `plot2d()`. Artinya, himpunan

$$\{(x,y)|x = f(t), y = g(t)\}$$

mendefinisikan sebuah kurva. Apabila nilai-nilai x diurutkan, maka kurvanya merupakan grafik suatu fungsi $y = h(x)$. Untuk mendapatkan kurva yang tampak mulus diperlukan banyak titik (menggunakan rentang nilai t yang cukup banyak

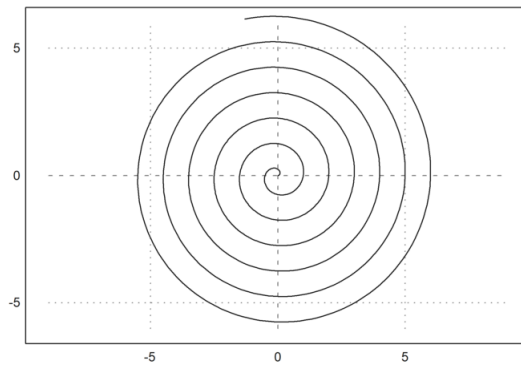
dan cukup saling berdekatan) atau menggunakan perintah/fungsi `adaptive()` dalam menghitung nilai-nilai x dan y tersebut.

Berikut adalah contoh menggambar spiral

$$\gamma(t) = (t \cdot \cos(2\pi t), t \cdot \sin(2\pi t)).$$

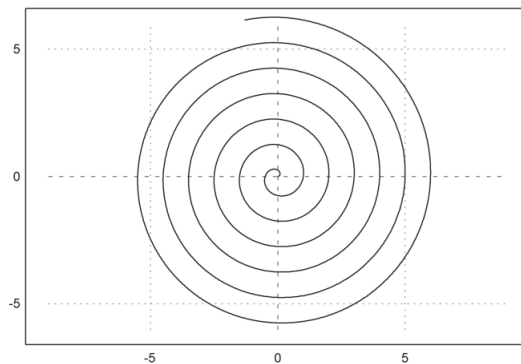
Cara pertama, kita tulis nilai-nilai x dan y sebagai ekspresi dalam t tanpa diapit tanda kutip double (" ") pada perintah/fungsi `plot2d()`.

```
>t=linspace(0,2pi,1000); ...  
plot2d(t*cos(2*pi*t),t*sin(2*pi*t),r=6):
```



Cara kedua, nilai-nilai x dan y ditulis sebagai ekspresi dalam x (parameter t diganti dengan x) dan diapit dengan tanda kutip double (" ") pada perintah/fungsi `plot2d()`.

```
>plot2d("x*cos(2*pi*x)","x*sin(2*pi*x)",xmin=0, ...  
xmax=2*pi,r=6):
```



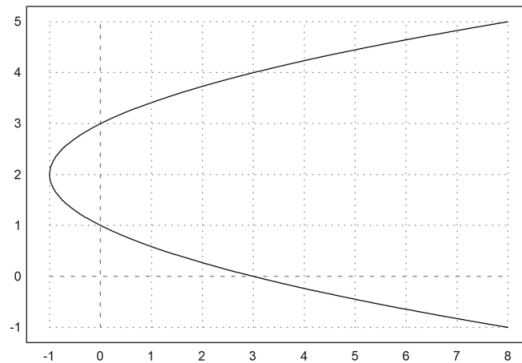
Berikut adalah contoh lain, yakni menggambar kurva

$$x = t^2 - 2t, y = t + 1,$$

yang jika dinyatakan dalam bentuk eksplisit menjadi

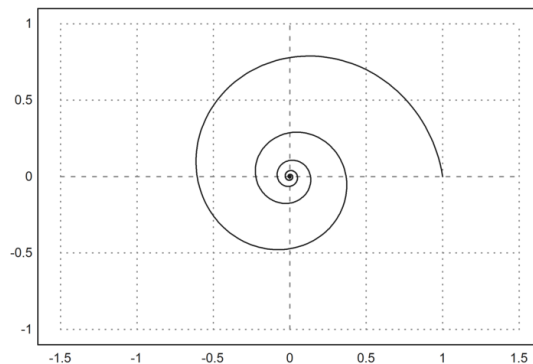
$$x = (y - 2)^2 - 1 = y^2 - 4y + 3.$$

```
>plot2d("x^2-2*x", "x+1", xmin=-2, xmax=4) :
```



Ekspresi simbolik juga dapat digunakan sebagai input untuk fungsi `plot2d()`, karena ekspresi simbolik disimpan sebagai ekspresi *string*. Perhatikan, dalam hal ini variabel yang digunakan untuk mendefinisikan nilai-nilai x dan y harus didefinisikan sebagai vektor dengan rentang yang sesuai.

```
>t=linspace(0,2pi,1000); r=exp(-t); x=r*cos(2pi*t); ...  
y=r*sin(2pi*t);  
>plot2d(x,y,r=1) :
```

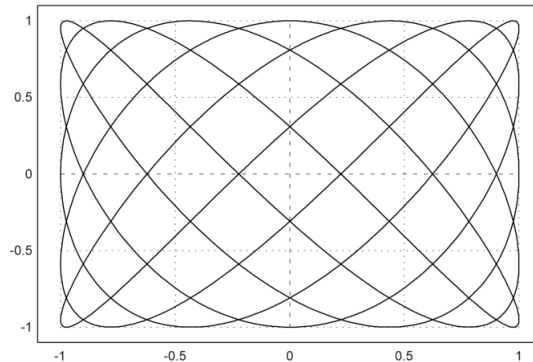


Berikut adalah contoh menggambar kurva parametrik dengan persamaan

$$x = \sin(5x)y = \cos(7x)$$

pada interval $[0, 2\pi]$.

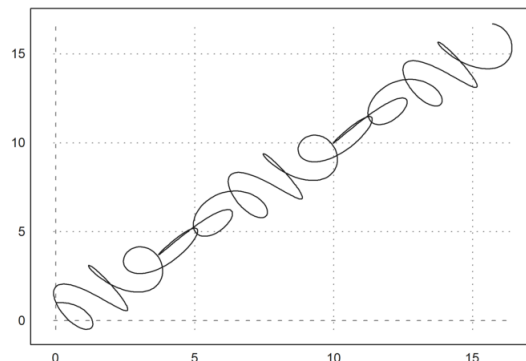
```
>// sama saja x=0:0.001:2*pi
>x=linspace(0,2pi,1000); plot2d(sin(5x),cos(7x)):
```



Berikut adalah kurva unik dengan persamaan parametrik

$$x = t + \sin(5t), y = t + \cos(6t).$$

```
>t=linspace(0,5pi,1000); x=t+sin(5*t); y=t+cos(6*t); ...
plot2d(x,y,0,10):
```

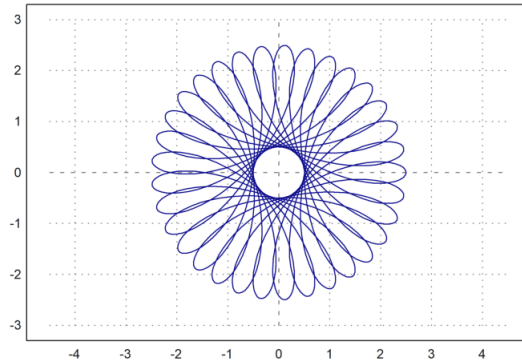


Berikut adalah kurva yang berbentuk bunga matahari, dengan persamaan parametrik

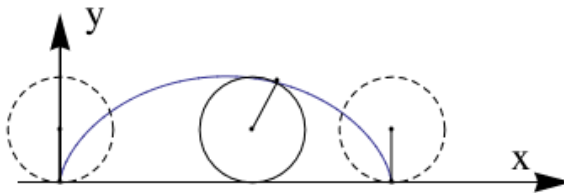
$$x = 1.5 \times \cos(10t) + \cos(23t), y = 1.5 \times \sin(10t) - \sin(23t).$$

Cobalah mengubah nilai-nilai konstanta (parameter)-nya untuk mendapatkan bentuk-bentuk yang lain.

```
>t=linspace(0,2pi,1000); x=1.5*cos(10*t)+cos(23*t); ...
y=1.5*sin(10*t)-sin(23*t); ...
plot2d(x,y,r=3,color=blue):
```



Berikut ini adalah kurva *sikloid* yang merupakan lintasan titik pada keliling sebuah lingkaran yang berputar pada garis lurus (mendatar).

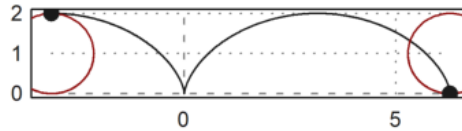


Kurva *sikloid* mempunyai persamaan parametrik

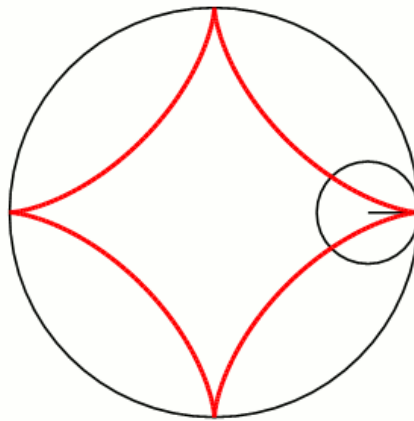
$$x = r \times (t - \sin(t)), \quad y = r \times (1 - \cos(t))$$

dengan r adalah panjang jari-jari lingkaran yang berputar.

```
>aspect(5); t=linspace(-pi,2pi,1000); r=1;
>x=r*(t-sin(t)); y=r*(1-cos(t)); plot2d(x,y,[-5,8]);
>plot2d("-pi+r*cos(x)","r+r*sin(x)",color=red,>add);
>plot2d(r*(-pi-sin(-pi)),r*(1-cos(-pi)),>points, ...
style="o#",>add);
>plot2d("2*pi+r*cos(x)","r+r*sin(x)",color=red,>add);
>plot2d(r*(2pi-sin(2pi)),r*(1-cos(2pi)),>points, ...
style="o#",>add):
```



Apabila tempat kedudukan sebuah titik pada keliling suatu lingkaran yang bergerak melintasi garis lurus merupakan *sikloid*, maka jika lintasannya berupa lingkaran lain yang lebih besar, dan Bergeraknya di dalam, kurva yang dihasilkan dinamakan *hiposikloid*. Apabila lingkarannya bergerak di luar lingkaran lain, kurva lintasannya dinamakan *episkloid*.



Persamaan *hiposikloid* adalah

$$x = (R - r) \cdot \cos(t) + r \cos\left(\frac{(R - r)}{r} \cdot t\right),$$

$$y = (R - r) \cdot \sin(t) - r \sin\left(\frac{(R - r)}{r} \cdot t\right),$$

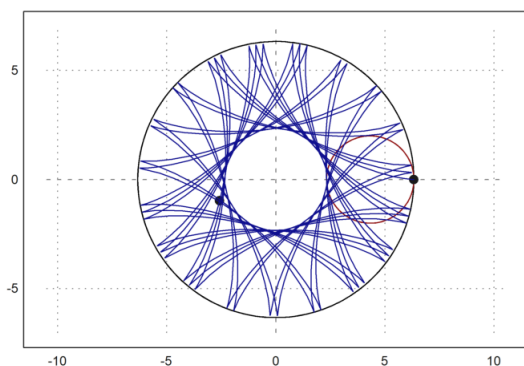
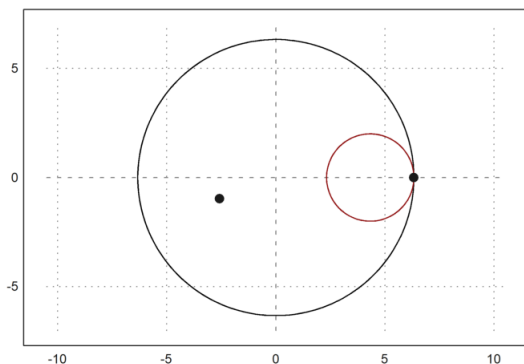
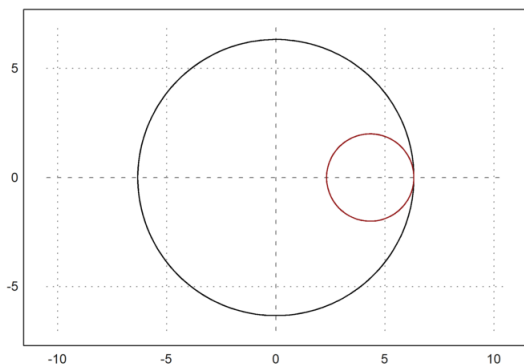
dengan R adalah panjang jari-jari lingkaran lintasan, r adalah panjang jari-jari lingkaran yang berputar. Berikut adalah salah satu bentuk kurva yang menarik, di mana titik yang bergerak tidak pernah kembali ke posisi semula. Apakah syaratnya agar titik yang bergerak dapat kembali ke posisi semula? Cobalah mengubah nilai-nilai konstanta (parameter), R , dan r untuk mendapatkan kurva lintasan yang berbeda-beda.

```
>function xf(R,r,t) := (R-r)*cos(t)+r*cos((R-r)/r*t);
>function yf(R,r,t) := (R-r)*sin(t)-r*sin((R-r)/r*t);
>R=sqrt(40); r =2;
>plot2d("R*cos(x)", "R*sin(x)", r=7);
```

```

>plot2d(" (R-r)+r*cos(x) ", "r*sin(x) ", r=7, >add, color=red) :
>plot2d(xf(R, r, 0), yf(R, r, 0), >points, style="o#", >add);
>plot2d(xf(R, r, pi), yf(R, r, pi), >points, style="o#", >add) :
>plot2d("xf(R, r, x) ", "yf(R, r, x) ", xmin=0, ...
      xmax=100000000*pi, >add, color=blue) :

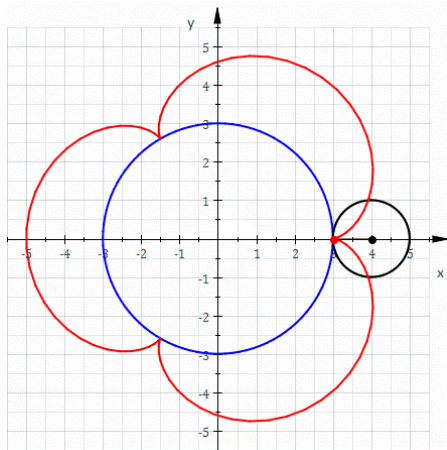
```



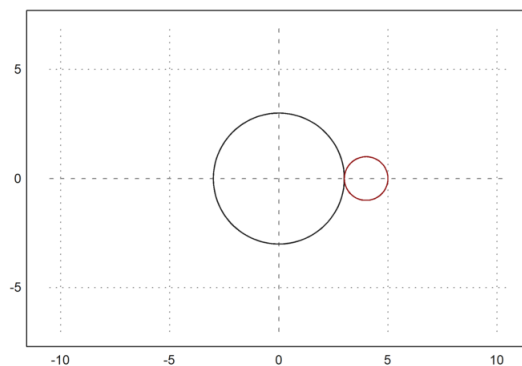
Episikloid (lintasan pergerakan titik pada lingkaran di luar lingkaran lain) mempunyai persamaan parametrik

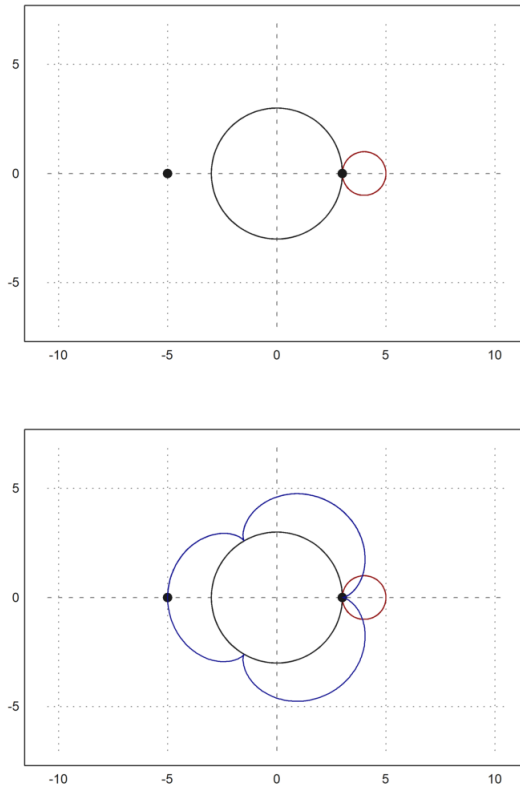
$$x = (R+r) \cdot \cos(t) - r \cos\left(\frac{(R+r)}{r} \cdot t\right),$$

$$y = (R+r) \cdot \sin(t) - r \sin\left(\frac{(R+r)}{r} \cdot t\right).$$



```
>function xf(R,r,t) := (R+r)*cos(t)-r*cos((R+r)/r*t);
>function yf(R,r,t) := (R+r)*sin(t)-r*sin((R+r)/r*t);
>R=3; r =1;
>plot2d("R*cos(x)", "R*sin(x)", r=7);
>plot2d("(R+r)+r*cos(x)", "r*sin(x)", r=7, >add, color=red):
>plot2d(xf(R,r,0), yf(R,r,0), >points, style="o#", >add);
>plot2d(xf(R,r,pi), yf(R,r,pi), >points, style="o#", >add):
>plot2d("xf(R,r,x)", "yf(R,r,x)", xmin=0, xmax=3*pi, ...
>add, color=blue):
```





Cobalah mengubah nilai-nilai konstanta (parameter), R , dan r untuk mendapatkan kurva lintasan yang berbeda-beda. Kapan titik yang bergerak dapat kembali ke posisi semula dan kapan titik tersebut tidak akan pernah kembali ke posisi sebelumnya?

Fungsi polar memiliki persamaan

$$r = f(\theta).$$

Fungsi tersebut dapat diubah ke persamaan dalam koordinat Kartesius sebagai fungsi parameterik

$$x = r \cos \theta = f(\theta) \cos \theta, \quad y = r \sin \theta = f(\theta) \sin \theta.$$

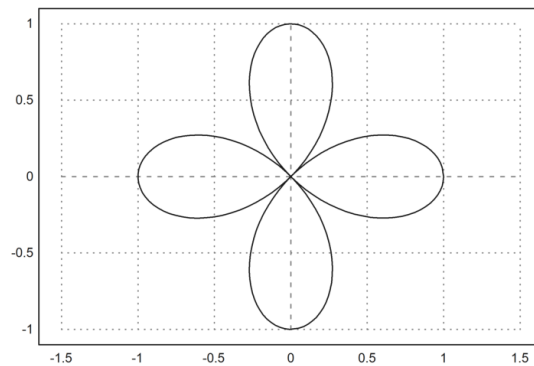
Sebagai contoh, kurva **mawar** memiliki persamaan kutub

$$r(t) = a \cos(nt), \text{ atau } r(t) = a \sin(nt).$$

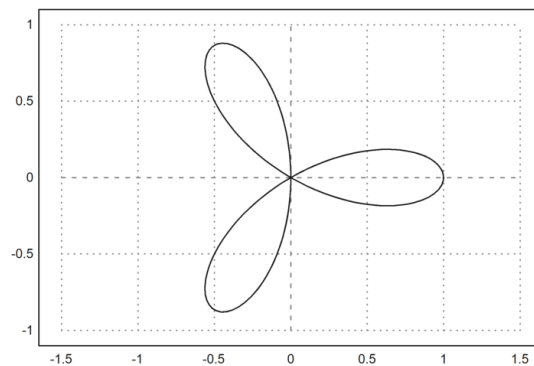
Untuk menggambar kurva mawar di EMT dapat digunakan pasangan nilai x dan y dengan

$$x = a \cos(nt) \cos(t), \quad y = a \cos(nt) \sin(t).$$

```
>plot2d("cos(2*x)*cos(x)", "cos(2*x)*sin(x)", ...
        xmin=0, xmax=2*pi, r=1):
```



```
>plot2d("cos(3*x)*cos(x)", "cos(3*x)*sin(x)", ...
        xmin=0, xmax=2*pi, r=1):
```



Berikut kita gambar kurva parametrik

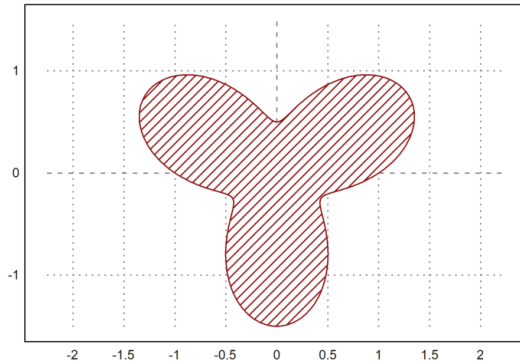
$$\gamma(t) = (r(t) \cos(t), r(t) \sin(t))$$

dengan

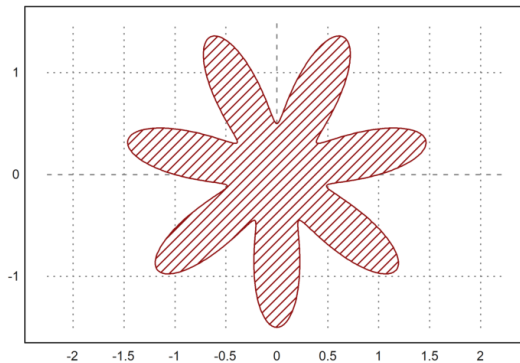
$$r(t) = 1 + \frac{\sin(3t)}{2}.$$

Berikut ini ditunjukkan dua kurva dengan variasi yang berbeda. Lakukan variasi lain untuk mendapatkan kurva yang berbeda.


```
>t=linspace(0,2pi,1000); r=1+sin(3*t)/2;
>x=r*cos(t); y=r*sin(t);
>plot2d(x,y,>filled,fillcolor=red,style="/",r=1.5):
```



```
>t=linspace(0,2pi,1000); r=1+sin(7*t)/2;
>x=r*cos(t); y=r*sin(t);
>plot2d(x,y,>filled,fillcolor=red,style="/",r=1.5):
```



Kurva fungsi polar juga banyak yang menarik dan terkenal. Salah satunya adalah kurva *limason* yang memiliki persamaan kutub

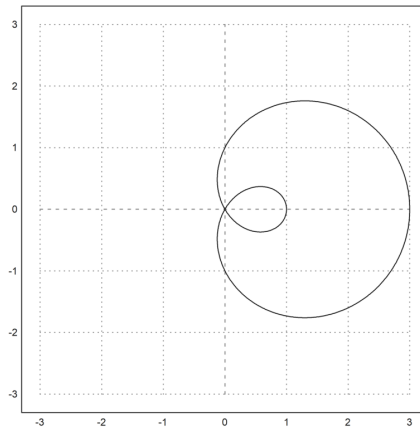
$$r(t) = a + b\cos(t) \text{ atau } r(t) = a + b\sin(t).$$

Dalam koordinat Kartesius kurva *limason* dapat dinyatakan secara implisit

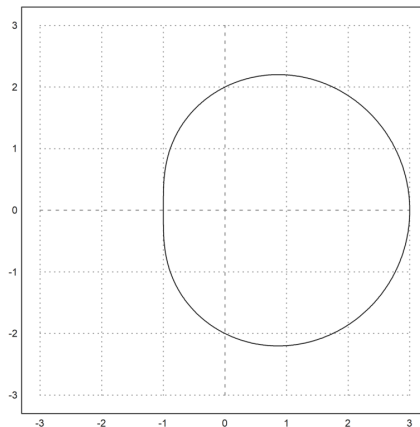
$$(x^2 + y^2 - 2a.x)^2 = b^2(x^2 + y^2).$$

Berikut adalah beberapa bentuk kurva *limason*.

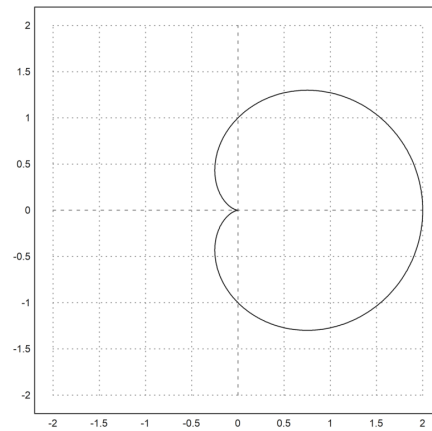
```
>aspect(1); t=linspace(0,2pi,1000);  
>a=1; b=2; r=a+b*cos(t); x=r*cos(t); y=r*sin(t);  
>plot2d(x,y,r=3):
```



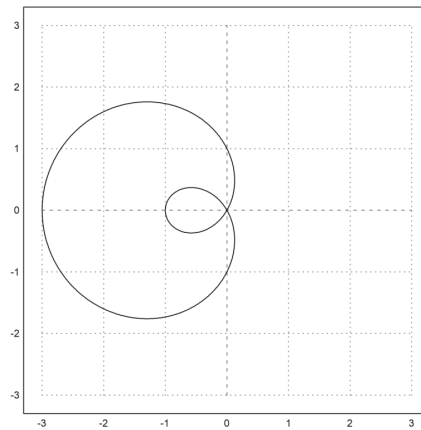
```
>a=2; b=1; r=a+b*cos(t); x=r*cos(t); y=r*sin(t);  
>plot2d(x,y,r=3):
```



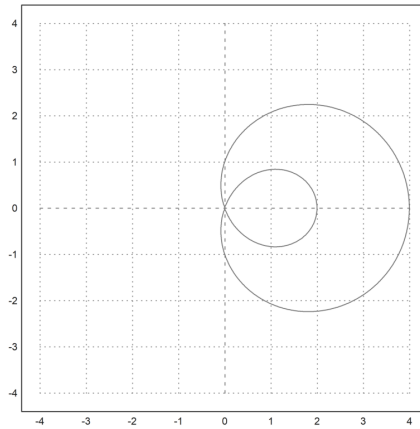
```
>a=1; b=1; r=a+b*cos(t); x=r*cos(t); y=r*sin(t);  
>plot2d(x,y,r=2):
```



```
>a=1; b=-2; r=a+b*cos(t); x=r*cos(t); y=r*sin(t);
>plot2d(x,y,r=3):
```



```
>a=1.5; b=1;
>plot2d("(x^2+y^2-2*a*x)^2-b^2*(x^2+y^2)", ...
  level=0,r=4, n=300):
```



6.4 Menggambar grafik bilangan kompleks

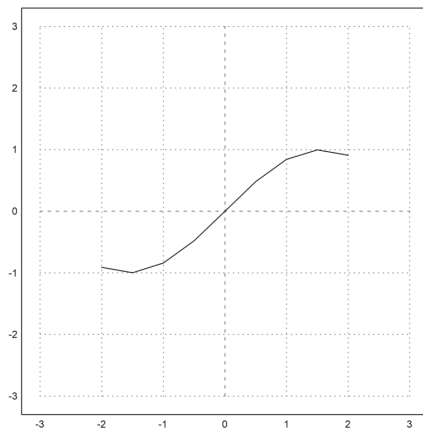
Bilangan kompleks z dinyatakan dengan dua komponen, yakni komponen riil dan komponen imajiner, dituliskan sebagai

$$z = x + y.i$$

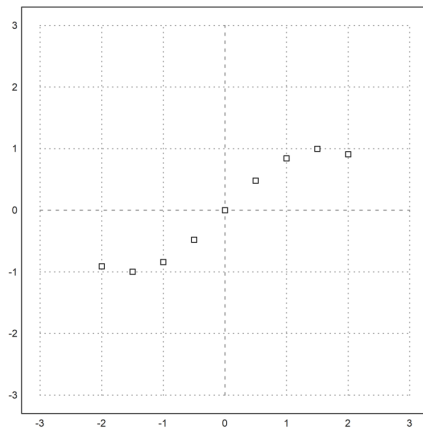
dengan i adalah bilangan imajiner $\sqrt{-1}$.

Oleh karena itu, bilangan kompleks dapat digambar sebagai titik di EMT dengan perintah `plot2d`. Himpunan (sekumpulan atau matriks) bilangan kompleks akan digambar sebagai ruas-ruas garis yang menghubungkan titik-titik posisi bilangan-bilangan kompleks tersebut. Apabila di dalam perintah `plot2d` digunakan opsi `>points`, maka bilangan-bilangan kompleks tersebut digambar sebagai titik-titik.

```
>aspect(1); r=linspace(0,1,50);
>a=linspace(0,2pi,80)'; z=r*exp(I*a);
>x=-2:0.5:2; y=sin(x); z=x+y*I; plot2d(z,r=3):
```



```
>x=-2:0.5:2; y=sin(x); z=x+y*I; plot2d(z,>points,r=3):
```



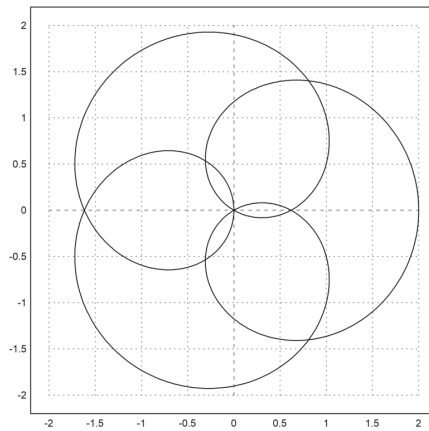
Lingkaran satuan dapat digambar dengan menggunakan fungsi

$$\gamma(t) = e^{it}, \text{ dengan } 0 \leq t \leq 2\pi.$$

Berikut adalah kurva

$$f(t) = e^{it} + e^{4it}.$$

```
>t=linspace(0,2pi,1000);  
>plot2d(exp(I*t)+exp(4*I*t),r=2):
```



Cara lain menggambar bilangan kompleks adalah menggunakan bentuk polar, yakni

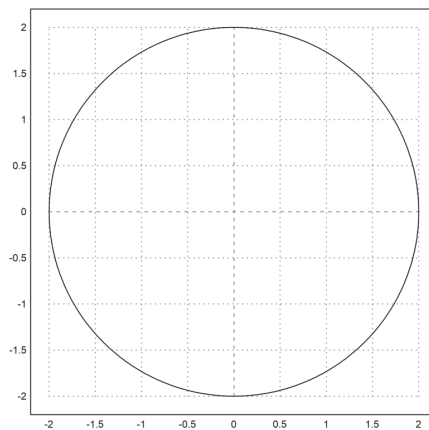
$$z = r(\cos(t) + i \sin(t)),$$

sehingga untuk menggambar plot bilangan kompleks dilakukan seperti pada menggambar fungsi polar. Berikut adalah contoh menggambar bilangan kompleks

$$z = 2(\cos(t) + i \sin(t)), \quad 0 \leq t \leq 2\pi,$$

yang merupakan lingkaran berjari-jari 2 dengan pusat (0,0).

```
>t=linspace(0,2pi,1000); r= 2; x=r*cos(t); y=r*sin(t);  
>z=x+y*I; plot2d(z,r=2):
```



Untuk meningkatkan penulisan bilangan kompleks dapat digunakan notasi Euler

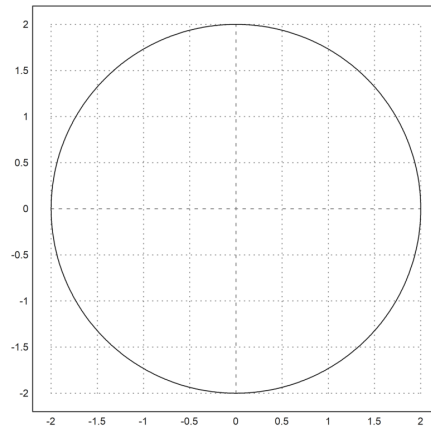
$$e^{iy} = \cos(y) + i \sin(y),$$

sehingga

$$e^{x+iy} = e^x(\cos(y) + i \sin(y)).$$

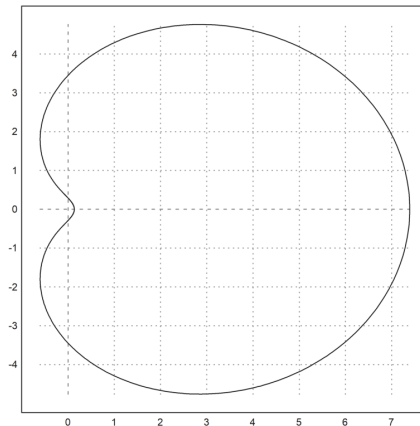
Lingkaran seperti di atas dapat digambar dengan cara sebagai berikut.

```
>t=linspace(0,2pi,1000); z=exp(log(2)+I*t);
>plot2d(z, r=2):
```



Fungsi-fungsi bilangan kompleks dapat digambar dengan mudah setelah domain (nilai-nilai z) diketahui. Misalnya, dari nilai-nilai z di atas dapat digambar kurva fungsi $f(z) = e^z$, misalnya.

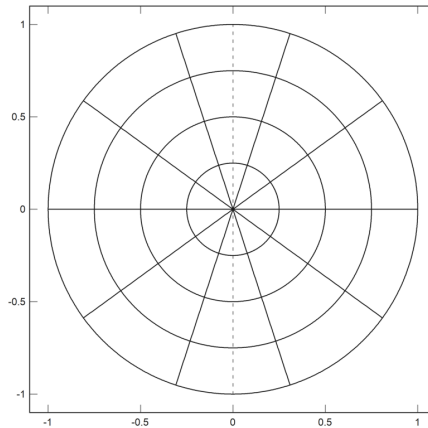
```
>plot2d(exp(z), a=-1, b=8):
```



Bidang kompleks dapat digambar dengan menggunakan parameter `cgrid`. Apabila digunakan parameter `cgrid=n`, maka bidang kompleks digambar dengan

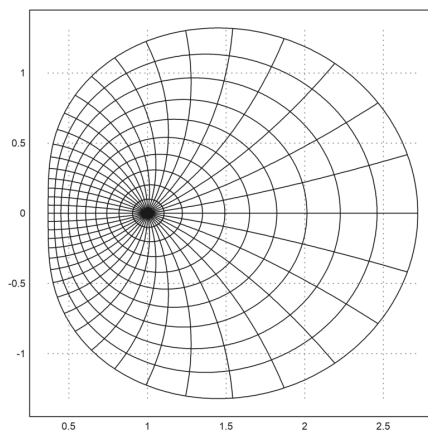
n jari-jari dan n lingkaran. Apabila digunakan parameter `cgrid=[n,m]`, maka bidang kompleks digambar dengan n jari-jari dan m lingkaran.

```
>r=linspace(0,1,100); t=linspace(0,2pi,1000)';
>z=r*exp(I*t); //domain z
>plot2d(z,r=1,grid=6,cgrid=[10,4])://plot bidang domain
```



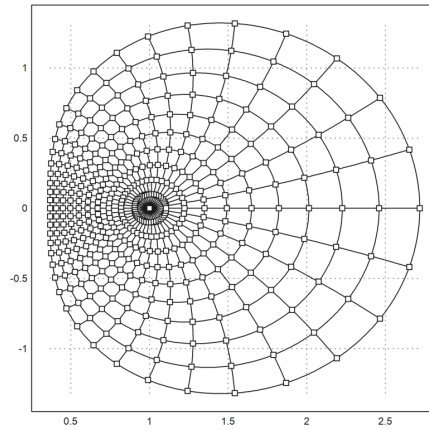
Berikut adalah kurva fungsi $f(z) = e^z$ dengan domain tersebut.

```
>plot2d(exp(z),cgrid=[40,10]):
```



Kita juga dapat menambahkan titik-titik kompleks hasil pemetaan tersebut pada bidang kompleks.

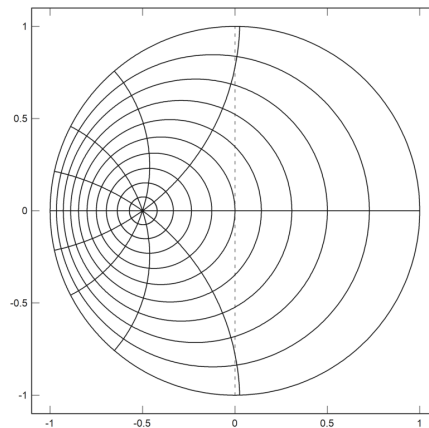

```
>r=linspace(0,1,10); t=linspace(0,2pi,40)';
>z=r*exp(I*t);
>plot2d(exp(z),>points,pointstyle="o",>add):
```



Berikut adalah contoh pemetaan konformal dengan domain bidang kompleks di dalam lingkaran satuan

$$z \rightarrow \frac{z - 1/2}{1 - z/2}, \text{ dengan } |z| \leq 1.$$

```
>r=linspace(0,1,100); t=linspace(0,2*pi,1000)';
>z=r*exp(t*I); // bidang domain
>plot2d((z-0.5)/(1-0.5*z),a=-1,b=1,c=-1,d=1,grid=6, ...
  cgrid=10): //bidang konformal
```

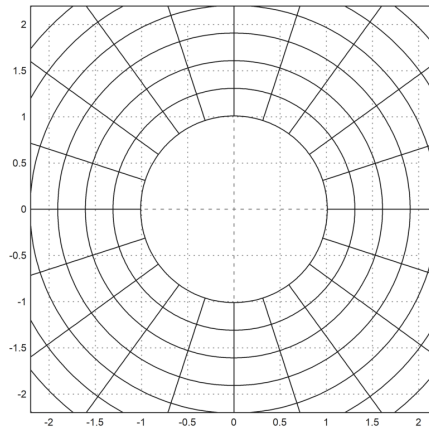


Pemetaan berikut ini dikenal dengan nama pemetaan Joukowski, memetakan satu-satu titik-titik pada bidang kompleks di luar lingkaran satuan ke titik-titik

pada bidang kompleks di luar interval $[-1, 1]$.

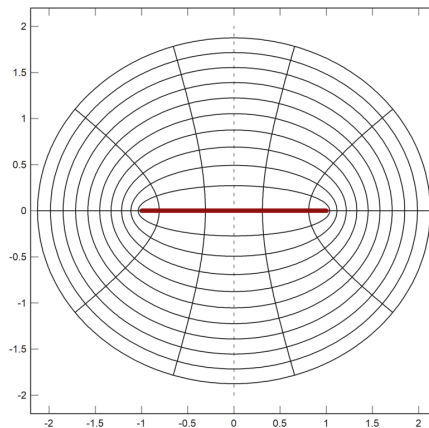
$$z \rightarrow \frac{z + 1/z}{2}.$$

```
>z=linspace(1.01,4,100)*exp(I*linspace(0,2*pi,400)');  
>plot2d(z,a=-2,b=2,c=-2,d=2,grid=2,cgrid=[20,10]):
```



Berikut kita gambar hasil pemetaan tersebut beserta interval $[-1, 1]$ (dengan warna merah) yang tidak termasuk pada daerah hasil.

```
>function J(z):=(z+1/z)/2;  
>plot2d(J(z),a=-2,b=2,c=-2,d=2,grid=6,cgrid=10);  
>plot2d([-1,1],0,color=red,thickness=5,>add):
```



Karena fungsi Joukowski satu-satu, maka memiliki invers. Berikut kita definisikan invers fungsi Joukowski.

```

>function map invJ(z) ...
    w=sqrt(z^2-1);
    return z+(re(z)>=0)*w-(re(z)<0)*w;
endfunction
>$solve((z+1/z)/2=w, z)

```

$$\left[z = w - \sqrt{w^2 - 1}, z = \sqrt{w^2 - 1} + w \right]$$

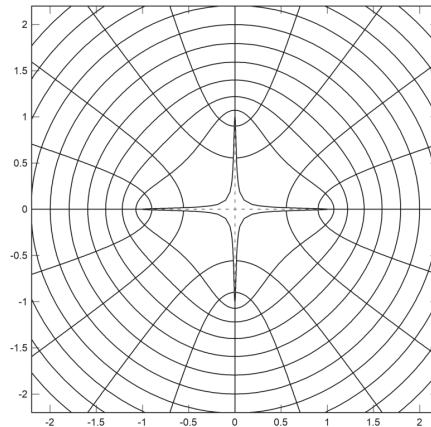
Dengan menggunakan fungsi-fungsi ini, kita petakan bidang di luar lingkaran satuan ke daerah di luar sumbu-sumbu x dan y .

$$w = J(iJ^{-1}(\sqrt{2}J(z)))$$

```

>plot2d(J(I*invJ(J(z)*sqrt(2))), a=-2, b=2, c=-2, d=2, ...
    grid=6, cgrid=[20,10]):

```



Contoh berikutnya adalah fungsi kompleks yang memetakan daerah di luar lingkaran satuan ke bidang di atas sumbu x .

$$w = \frac{iz + 1}{i + z}$$

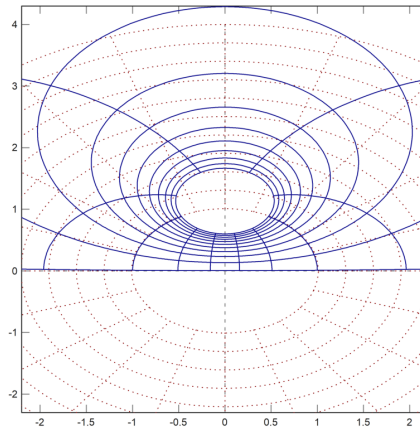
```

>function H(z) &= (I*z+1)/(I+z); $H(z)
    \frac{iz+1}{z+i}

```

Berikut adalah peta (mulus) dari grid di luar lingkaran satuan (putus-putus) di atas. Perhatikan, tak hingga dipetakan ke i .

```
>plot2d(z,a=-2,b=2,c=-2,d=4,grid=6,cgrid=[20,10], ...
  style=".",color=red); // domain
>plot2d(H(z),a=-2,b=2,c=-2,d=4,grid=6,cgrid=10, ...
  color=blue,>add):
```



Selanjutnya dapat didefinisikan invers fungsinya, sebagai berikut, yang memetakan daerah setengah bidang atas ke daerah di luar lingkaran satuan.

```
>$solve(w=H(z),z)
```

$$\left[z = \frac{1 - iw}{w - i} \right]$$

```
>function invH(z) &= (I*z-1)/(I-z); $invH(z)
```

$$\frac{iz - 1}{i - z}$$

```
>$H(2), $H(1+I), $H(2*I)
```

$$\frac{2i+1}{i+2}$$

$$\frac{i(i+1)+1}{2i+1}$$

$$\frac{i}{3}$$

```
>invH(H(2)), invH(H(1+I)), invH(H(2*I))
2+0i
1+1i
0+2i
```

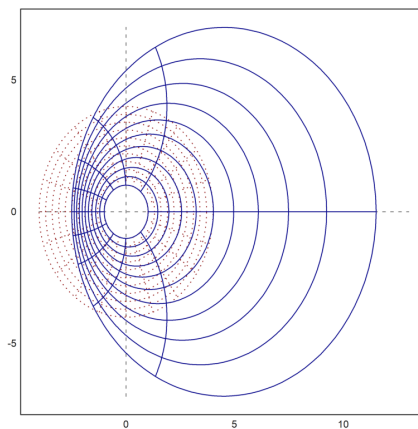
Contoh berikut ini menunjukkan fungsi kompleks yang memetakan daerah di luar lingkaran satuan ke dirinya sendiri, dan memetakan a ke ∞ .

$$w = L(z, a) = \frac{a}{\bar{a}} \times \frac{(\bar{a}z - 1)}{(a - z)}$$

```
>function L(z,a) &= (a/conj(a))*(conj(a)*z-1)/(a-z);
>$L(z,a)
```

$$\frac{a(\text{conj}(a)z - 1)}{\text{conj}(a)(a - z)}$$

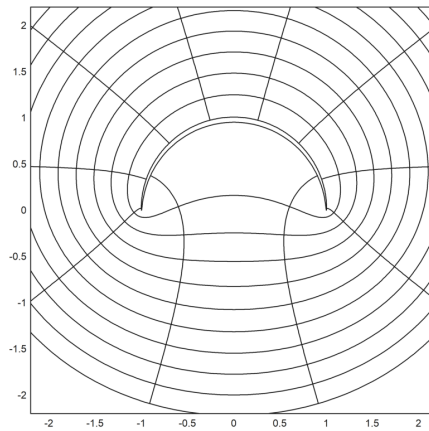
```
>z=linspace(1.01,4,100)*exp(I*linspace(0,2*pi,400)');
>plot2d(z,a=-4,b=13,c=-7,d=7,grid=1,cgrid=[20,10], ...
style=".",color=red); // domain
>plot2d(L(z,6),grid=1,cgrid=10,color=blue,>add):
```



Terakhir kita memetakan daerah di luar lingkaran satuan ke daerah di luar busur dengan sudut π .

$$w = H^{-1}JL(z, -J^{-1}(i))$$

```
>plot2d(invH(J(L(z,-invJ(1i))))),a=-2,b=2,c=-2,d=2, ...
grid=4,cgrid=10):
```



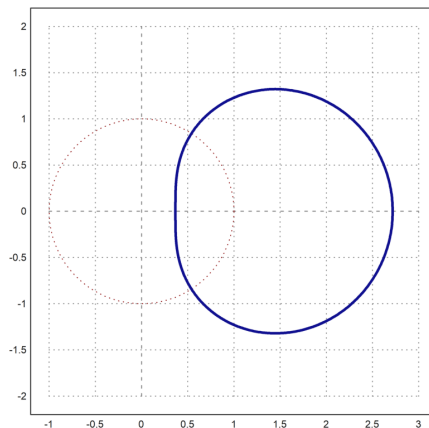
Sebagai contoh terakhir terkait kurva fungsi kompleks, kita sajikan contoh interpolasi pada bidang kompleks. Misalkan kita akan menginterpolasikan fungsi

$$\gamma(z) = e^z$$

pada lingkaran satuan di titik-titik $z = 1$, $z = -1$, $z = i$, dan $z = -i$, kemudian kita bandingkan dengan hampiran menggunakan deret pangkat terpankas

$$e^z \approx 1 + z + \frac{z^2}{2} + \frac{z^3}{6}.$$

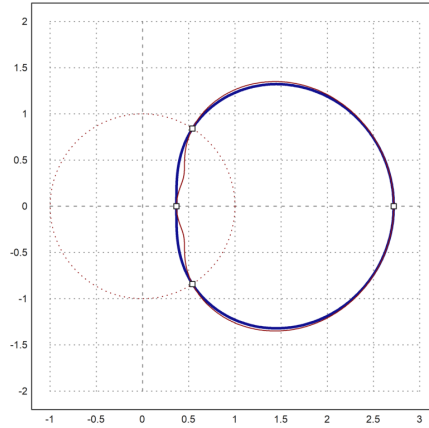
```
>t=linspace(0,2pi,1000); z=exp(I*t);
>plot2d(z,a=-1,b=3,c=-2,d=2,style=".",color=red);
>plot2d(exp(z),color=blue,thickness=3,>add):
```



```

>// titik-titik interpolasi
>xp=[1,-1,I,-I]; yp=exp(xp);
>// polinomial interpolasi Newton
>d=divdif(xp,yp); z1=evaldivdif(z,d,xp);
>// kurva polinomial interpolasi
>plot2d(z1,color=red,>add);
>// titik-titik interpolasi
>plot2d(exp(xp),>points,>add):

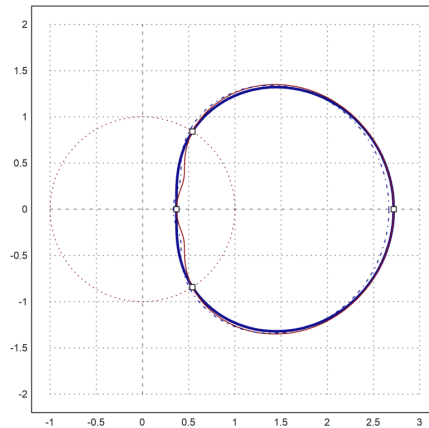
```



```

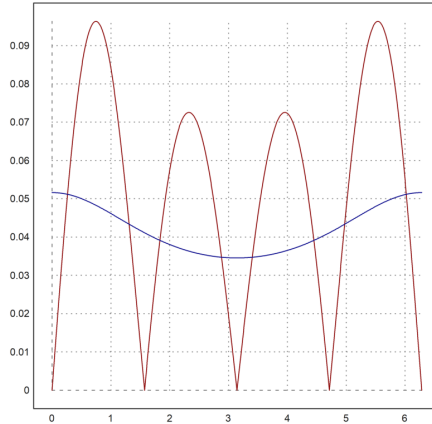
>//hampiran dengan deret pangkat berderajat 3
>z2=1+z+z^2/2+z^3/6;
>//tambahkan kurva deret pangkat
>plot2d(z2,style="-.-",color=blue,>add):

```



Berikut adalah kurva perbandingan galat antara hampiran dengan interpolasi Newton dan hampiran dengan deret pangkat berderajat 3.

```
>// galat mutlak interpolasi
>plot2d(t,abs(exp(z)-z1),color=red);
>// galat mutlak deret pangkat berderajat 3
>plot2d(t,abs(exp(z)-z2),color=blue,>add):
```



6.5 Menggambar daerah yang dibatasi dua kurva dan segi banyak

Misalkan kita akan menggambar daerah yang dibatasi oleh dua kurva

$$y_1 = f(x) \text{ dan } y_2 = g(x).$$

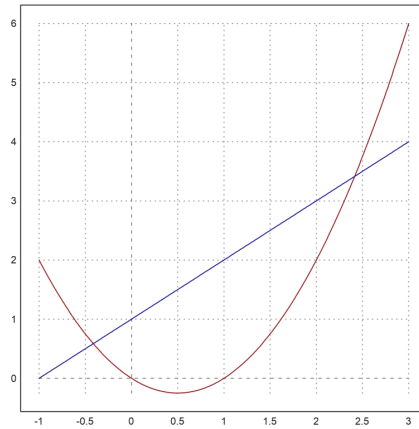
Setelah menggambar kedua kurva, langkah selanjutnya adalah menentukan titik-titik potong kedua kurva, misalnya di x_1 dan x_2 . Daerah yang dibatasi kedua kurva tersebut digambar dengan memplot pasangan nilai-nilai (x, y_1) untuk x dari x_1 sampai x_2 dan (x, y_2) untuk x dari x_2 sampai x_1 , kemudian diarsir. Dalam hal ini dapat digunakan notasi $\text{EMT } x_1 | x_2$ untuk menggabungkan matriks x_1 dan x_2 serta perintah `flipx()` untuk membalik elemen-elemen suatu matriks atau vektor berdasarkan kolom.

Sebagai contoh pertama kita akan menggambar daerah yang dibatasi oleh kurva

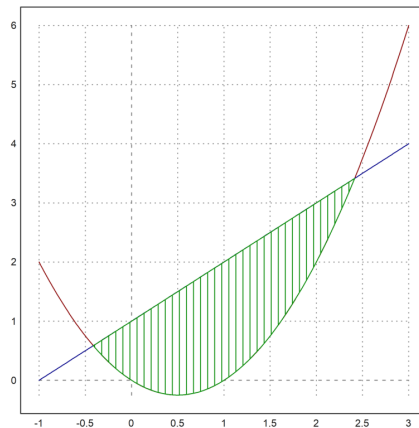
$$y_1 = x^2 - x \text{ dan } y_2 = x + 1.$$

Kedua kurva berpotongan di dua titik.

```
>y1 &= x^2-x; y2 &= x+1;
>plot2d([y1,y2],a=-1,b=3,color=[red,blue]):
```

```
>x1=solve("y1(x)-y2(x)",-0.5), x2=solve("y1(x)-y2(x)",2.5)
-0.414213562373
2.41421356237
>x=linspace(x1,x2,50); xb=flipx(x);
>plot2d(x|xb,y1(x)|y2(xb),>filled,style="|",fillcolor=3,>add):
```

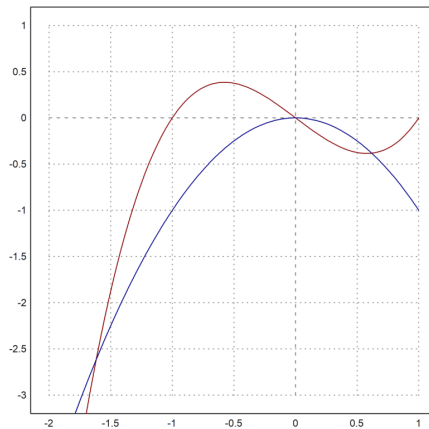


Pada contoh kedua berikut ini kita akan menggambar daerah yang dibatasi oleh kurva

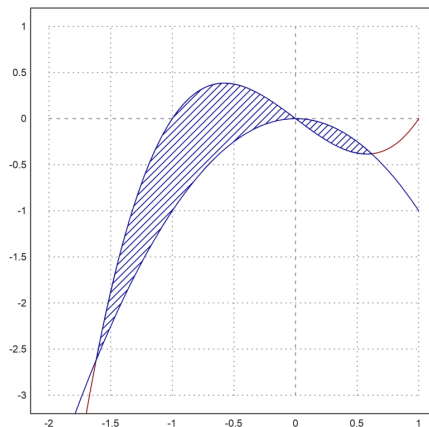
$$y = x^3 - x \text{ dan } y = -x^2.$$

Kedua kurva berpotongan di 3 titik. Kita cukup mencari titik-titik potong di ujung kiri dan kanan, yang di tengah sudah jelas.

```
>y1&=x^3-x; y2&=-x^2;
>plot2d([y1,y2],a=-2,b=1,c=-3,d=1,color=[red,blue]):
```



```
>x1=solve("y1(x)-y2(x)",-1.5), x2=solve("y1(x)-y2(x)",0.5)
-1.61803398875
0.61803398875
>x=linspace(x1,x2,50); xb=flipx(x);
>plot2d(x|xb,y1(x)|y2(xb),>filled,style="/", ...
    fillcolor=blue,>add):
```



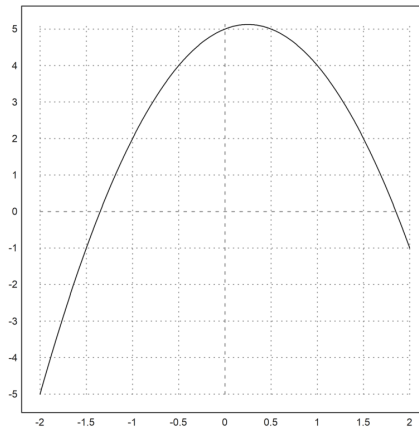
Untuk menggambar daerah di bawah suatu kurva dan di atas sumbu x tentu dapat dilakukan dengan cara serupa, karena dalam hal ini kurva keduanya adalah $y_2 = 0$. Berikut adalah dua contoh menggambar daerah yang dibatasi oleh kurva dengan sumbu x . Kurva pertama adalah

$$y = -2x^2 + x + 5.$$

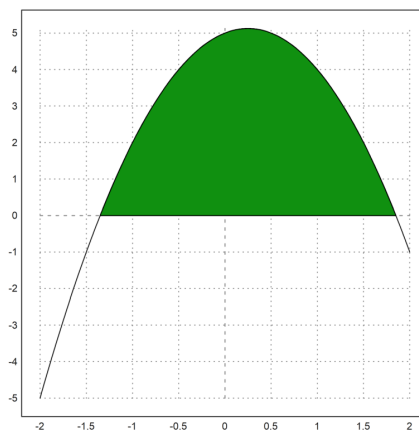
Kurva kedua adalah

$$y = x^3 - x.$$

```
>y &= -2*x^2+x+5;  
>plot2d(y):
```



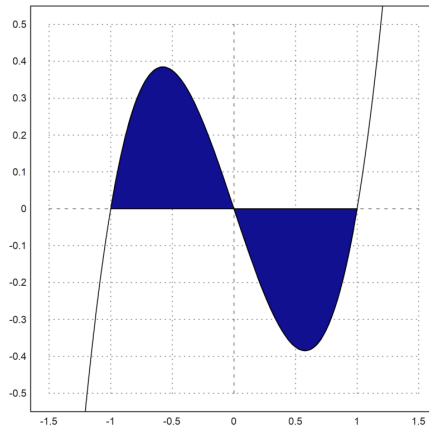
```
>x1=solve("y(x)",-1.5), x2=solve("y(x)",2)  
-1.35078105936  
1.85078105936  
>x=linspace(x1,x2,50);  
>plot2d(x,y(x),>filled,fillcolor=3,>add):
```



```

>y &= x^3-x;
>plot2d(y,a=-1.5,b=1.5,c=-0.5,d=0.5);
>x1=solve("y(x)",-1.5), x2=solve("y(x)",2)
-1
1
>x=linspace(x1,x2,50);
>plot2d(x,y(x),>filled,fillcolor=4,>add):

```

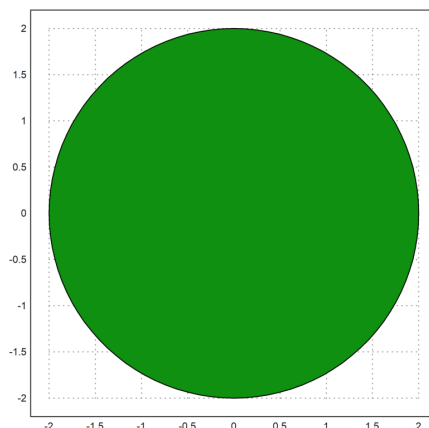


Daerah pada kurva tertutup seperti lingkaran dan elips dapat diwarnai dengan menggunakan parameter atau opsi `>filled` pada perintah `plot2d()`. Dalam hal ini lebih mudah menggunakan fungsi parametrik untuk menggambar daerah di dalam kurva tertutup.

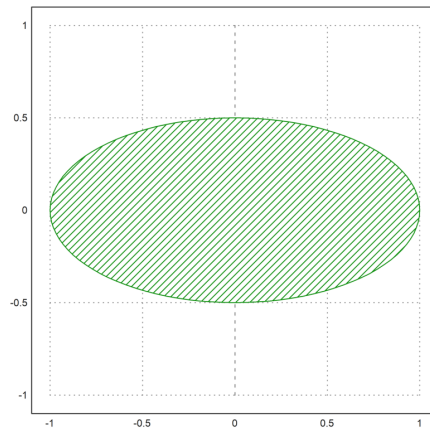
```

>t=linspace(0,2pi,100); plot2d(2*cos(t),2*sin(t),>filled):

```



```
>x=linspace(0,2pi,100); plot2d(sin(x),cos(x)*0.5,r=1,...  
>filled,style="/"):
```

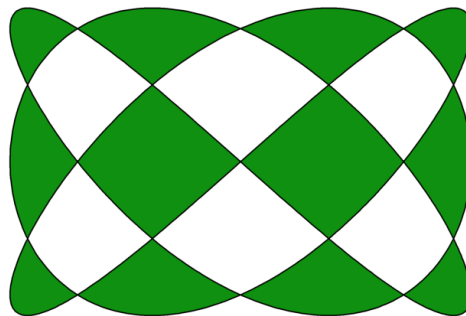


Berikut ini ditunjukkan sebuah gambar yang sangat menarik dengan mewarnai beberapa daerah di dalam kurva parametrik

$$x = \cos(3t), \quad y = \sin(4t), \quad 0 \leq t \leq 2\pi.$$

Agar terlihat menarik gambar dibuat tanpa sumbu-sumbu koordinat dan bingkai.

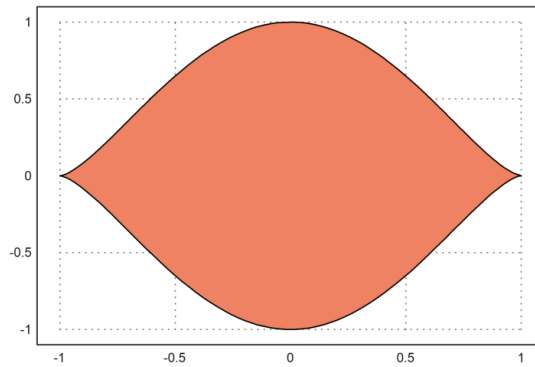
```
>t=linspace(0,2pi,1000); x=cos(3*t); y=sin(4*t);  
>aspect(1.5); plot2d(x,y,<grid,<frame,>filled):
```



Contoh berikutnya berupa daerah di dalam kurva tertutup berbentuk bibir yang memiliki persamaan parametrik

$$x = \cos(t), \quad y = \sin^3(t), \quad 0 \leq t \leq 2\pi.$$

```
>plot2d("cos(x)", "sin(x)^3", xmin=0, xmax=2pi, ...
>filled, fillcolor=10):
```



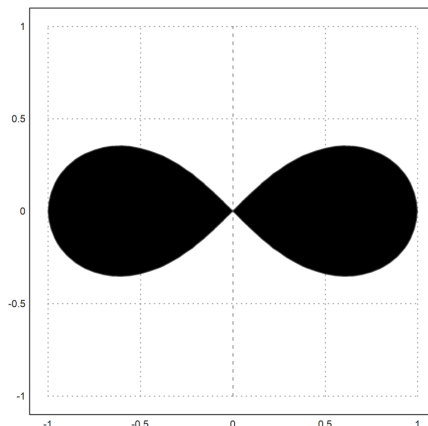
Untuk menggambar daerah nilai-nilai suatu fungsi pada interval tertentu dapat digunakan fungsi implisit dan parameter `level` pada perintah `plot2d()` berupa matriks $2 \times n$, dengan baris pertama adalah batas bawah dan baris kedua adalah batas fungsi.

$$a \leq f(x,y) \leq b.$$

Hal ini sudah ditunjukkan pada pembahasan mengenai cara menggambar kurva fungsi implisit. Sebagai contoh khusus, berikut adalah cara mengarsir/mewarnai daerah sebuah *lemniscate* dengan menggunakan rentang nilai

$$-1 \leq (x^2 + y^2)^2 - x^2 + y^2 \leq 0.$$

```
>aspect(1);
>plot2d("(x^2+y^2)^2-x^2+y^2", r=1, level=[-1;0]):
```

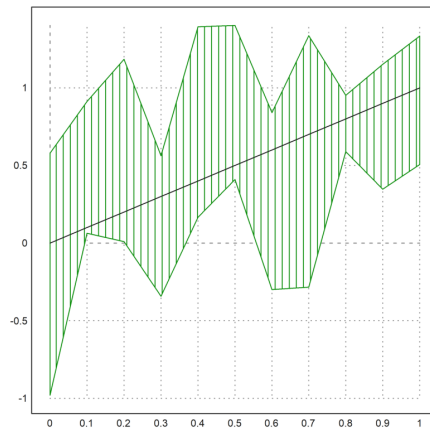


Daerah di dalam *lemniscate* adalah di mana fungsinya bernilai negatif, kurva *lemniscate* adalah titik-titik pembuat nol fungsi, dan daerah di luar kurva adalah di mana fungsinya bernilai positif.

Di EMT, himpunan titik akan digambar sebagai ruas-ruas garis yang menghubungkan titik-titik tersebut. Oleh karena itu, vektor interval (atau matriks dua baris dengan baris pertama adalah nilai-nilai batas bawah interval dan baris kedua adalah nilai-nilai batas atas interval) dapat digunakan untuk menggambar suatu daerah pada interval-interval tersebut. Hal ini sangat berguna untuk menggambarkan galat dalam komputasi atau galat statistika.

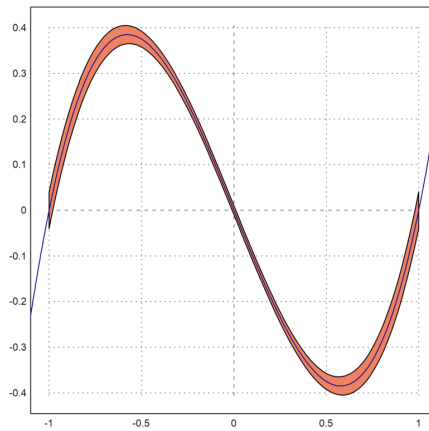
Misalkan kita memiliki interval dengan batas bawah $x - u(0, 1)$ dan batas atas $x + u(0, 1)$ dengan $u(0, 1)$ adalah bilangan acak berdistribusi uniform $(0, 1)$. Apabila x berupa vektor, maka kita akan memiliki matriks/vektor interval. Pasangan nilai-nilai $(x, x - u(0, 1))$ akan digambar sebagai batas bawah dan pasangan nilai-nilai $(x, x + u(0, 1))$ digambar sebagai batas atas. Daerah antara keduanya dapat diarsir.

```
>aspect(1);
>x=0:0.1:1; ...
plot2d(x,interval(x-random(size(x)), ...
x+random(size(x))),style="|"); ...
plot2d(x,x,>add):
```



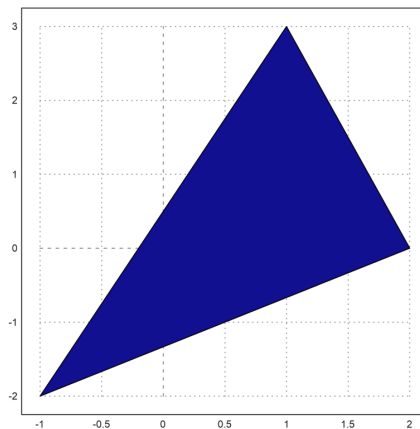
Vektor interval juga dapat digunakan untuk menggambar interval nilai-nilai suatu fungsi yang dihitung pada vektor interval tersebut.

```
>x=-1:0.01:1; t=~x-0.01,x+0.01~; y=t^3-t;
>plot2d(x,y,fillcolor=10); plot2d("x^3-x",r=1.5, ...
color=blue,>add):
```



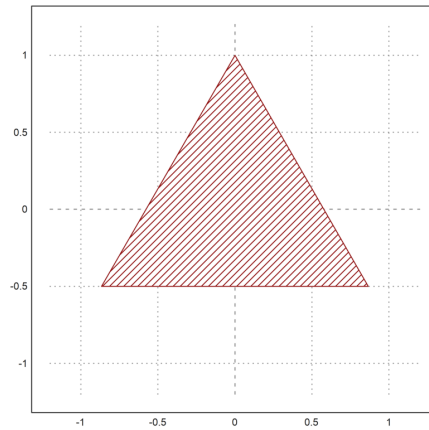
Sebagai akhir dari pembahasan tentang visualisasi dua dimensi di EMT adalah menggambar segi banyak. Terdapat tiga cara menggambar segi banyak di EMT. Cara pertama adalah dengan menggunakan himpunan koordinat titik-titik sudut segi banyak yang akan digambar. Ingat, bahwa di dalam EMT, himpunan titik digambar sebagai ruas-ruas garis yang menghubungkan titik-titik tersebut. Jadi apabila titik pertama dan titik terakhir sama, maka hasilnya berupa segi banyak.

```
>//pasangan koordinat suatu segitiga
>x=[-1, 2, 1, -1]; y=[-2,0,3,-2];
>plot2d(x,y,>filled,fillcolor=blue):
```

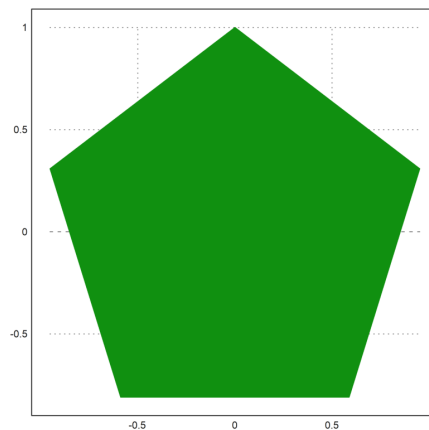


Cara kedua adalah menggunakan persamaan parametrik lingkaran atau elips dengan nilai parameter berupa nilai-nilai sudut sebanyak cacah sisi segi banyak yang akan digambar. Berikut ini ditunjukkan beberapa contoh menggambar segi banyak beraturan.

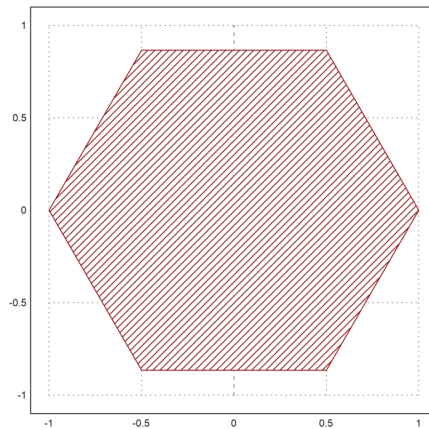

```
>t=linspace(0,2pi,3); ...
>plot2d(cos(t-pi/6),sin(t-pi/6),>filled,style="/", ...
  fillcolor=red,r=1.2):
```



```
>t=linspace(0,2pi,5); plot2d(cos(t+pi/10), ...
  sin(t+pi/10),>filled,style="#"):
```



```
>t=linspace(0,2pi,6); ...
>plot2d(cos(t),sin(t),r=1,>filled,style="/",fillcolor=red):
```

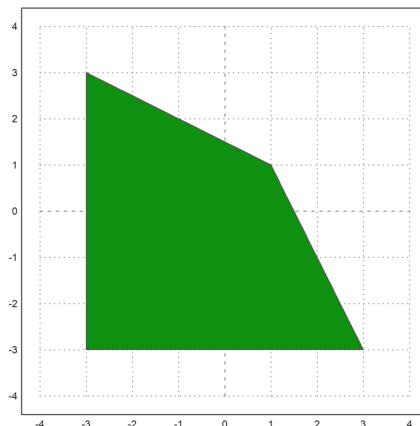


Cara ketiga untuk menggambar segi banyak adalah menggunakan nilai maksimal bentuk-bentuk linier

$$\max(ax + by) \leq c.$$

Untuk mendapatkan titik sudut yang lebih runcing digunakan nilai parameter n pada perintah `plot2d()` yang relatif besar.

```
>A=[2,1;1,2;-1,0;0,-1];
>function f(x,y) := max([x,y].A');
>plot2d("f",r=4,level=[0;3],color=green,n=200):
```



6.6 Rujukan lengkap fungsi `plot2d()`

```
function plot2d (xv, yv, btest, a, b, c, d, xmin,...
  xmax, r, n, logplot, grid, frame, framecolor,...
  square, color, thickness, style, auto, add, user, ...
  delta, points, addpoints, pointstyle, bar,histogram,...
  distribution, even, steps, own, adaptive, hue,...
  level, contour, nc, filled, fillcolor, outline,...
  title, xl, yl, maps, contourcolor, contourwidth,...
  ticks, margin, clipping, cx, cy, insimg,...
  spectral, cgrid, vertical, smaller, dl, niveau, levels)
```

Fungsi/perintah `plot2d()` merupakan fungsi serbaguna EMT untuk menggambar plot (kurva) pada bidang koordinat (2D). Perintah ini dapat menggambar kurva fungsi satu variabel, data, kurva pada bidang, kurva kontur, kurva fungsi kompleks, dan kurva fungsi implisit dua variabel.

Jika fungsi atau ekspresi `xv` diberikan, `plot2d()` akan menghitung nilai dalam rentang tertentu menggunakan fungsi atau ekspresi tersebut. Ekspresi harus dinyatakan dalam variabel `x`. Rentang nilai `x` dapat ditentukan menggunakan parameter `a` dan `b` atau menggunakan rentang nilai bawaan `[-2, 2]`. Rentang nilai `y` akan dihitung secara otomatis, kecuali digunakan parameter `c` dan `d`, atau menggunakan parameter radius `r`, yang menghasilkan rentang nilai `[-r, r]` untuk `x` dan `y`. Untuk menggambar kurva suatu fungsi, aslinya `plot2d()` akan menghitung nilai-nilai fungsi secara adaptif (titik demi titik). Untuk mempercepat menggambar kurva fungsi yang rumit, matikan ini dengan `<adaptive`, dan secara opsional mengurangi banyak interval `n`. Selain itu, `plot2d()` aslinya akan menggunakan pemetaan. Apabila ekspresi `xv` dapat menerima masukan vektor, penggunaan pemetaan dapat dinonaktifkan dengan pilihan `<maps` untuk mempercepat perhitungan.

Apabila digunakan fungsi atau ekspresi untuk `xv` dan `yv`, `plot2d()` akan menggunakan nilai `xv` sebagai absis `x` dan nilai `yv` sebagai ordinat `y`. Dalam hal ini, rentang nilai `x` dapat ditentukan dengan menggunakan parameter `xmin` dan `xmax`. Ekspresi yang ditulis dalam bentuk string (diapit oleh tanda " ") harus dinyatakan dalam variabel `x`.

Parameter/input pada fungsi `plot2d()`

Parameter	Penjelasan
<code>x, y</code>	ekspresi, persamaan, fungsi atau vektor data
<code>a, b, c, d</code>	batas-batas sumbu <code>x</code> dan <code>y</code> (aslinya <code>a=2, b=2</code>)
<code>r</code>	jika nilai <code>r</code> ditentukan, maka <code>a=cx-r, b=cx+r, c=cy-r, d=cy+r</code>

Parameter	Penjelasan
	<code>r</code> dapat menjadi vektor <code>[rx, ry]</code> atau vektor <code>[rx1, rx2, ry1, ry2]</code> .
<code>xmin, xmax</code>	batas-batas nilai sumbu x
<code>auto</code>	jangkauan nilai y otomatis (ini adalah nilai aslinya)
<code>square</code>	jika <code>square=true</code> atau <code>>square</code> , bidang koordinat dibuat persegi
<code>n</code>	banyaknya subinterval (aslinya bersifat adaptif)
<code>adaptif</code>	menggunakan kurva adaptif (<code>n</code> adalah minimal banyak langkah) dan ini adalah nilai aslinya. Di daerah curam digunakan lebih banyak titik plot, yaitu grid yang lebih halus.
<code>grid:</code>	0 = bidang koordinat tanpa sumbu koordinat dan bingkai 1 = sumbu x dan y, dan bingkai, skala pada bingkai 2 = sumbu koordinat normal, ini adalah opsi aslinya 3 = sumbu x dan y dengan skala, tanpa bingkai 4 = hanya bingkai dengan label 5 = sama seperti aslinya, namun dengan sembir 6 = label skala pada bingkai, sama dengan <code>grid=1</code> 7 = mirip dengan <code>grid=3</code>, garis sumbu mulus 8 = mirip dengan <code>grid=3</code>, garis sumbu mulus, skala lebih detil 9 = hanya bingkai dengan skala dan label 10 = sumbu koordinat, label tanpa skala pada bingkai
<code>frame</code>	menggambarkan bingkai di sekitar plot (nilai aslinya <code>>frame</code>)
<code>framecolor</code>	warna bingkai dan sumbu koordinat
<code>margin</code>	nilai antara 0 dan 0.4 untuk nilai margin di sekitar plot
<code>color</code>	warna kurva. Vektor warna dapat digunakan untuk setiap baris matriks plot.
	Dalam kasus plot titik, vektor warna harus berupa vektor kolom.
	Vektor baris atau matriks warna digunakan digunakan untuk setiap titik data.
<code>thickness</code>	ketebalan kurva (nilai <code>< 1</code> berarti garisnya sangat tipis).
<code>style</code>	model untuk kurva, titik, dan arsiran.
	Pilihan untuk model titik:
	<code>"[]"</code> , <code>"<>"</code> , <code>"."</code> , <code>".."</code> , <code>"..."</code> , <code>"*"</code> , <code>"+"</code> , <code>" "</code> , <code>"-"</code> , <code>"o"</code>
	<code>"[]#"</code> , <code>"<>#"</code> , <code>"o#"</code> (bulatan hitam)
	<code>"[]w"</code> , <code>"<>w"</code> , <code>"aw"</code> (tidak transparan)
	Pilihan model garis/kurva:
	<code>"-"</code> , <code>--"</code> , <code>"-."</code> , <code>"."</code> , <code>"-."</code> , <code>"-."</code> , <code>"->"</code>

Parameter	Penjelasan
points	menggambar titik (>points)
addpoints	menggambar ruas garis dan titik (>addpoints)
pointstyle	model titik
add	menambahkan plot pada plot sebelumnya pada bidang koordinat yang sama.
user	untuk membuat grafik kurva interaktif (>user). Interaksi dapat dilakukan dengan tombol-tombol: [kiri], [kanan] : mengubah interval [+]/[-] : memperbesar/kecil kurva klik mouse : memilih daerah pada kurva [spasi] : mengakhiri interaksi
delta	ukuran langkah untuk interaksi pengguna
bar	menggambar diagram batang (x adalah batas-batas interval, dan y adalah nilai-nilai interval)
histogram	menggambar diagram frekuensi x (x = vektor variabel acak) dalam n subinterval (>bar)
distribution=n	menggambar diagram distribusi x dengan n subinterval.
even	gunakan nilai bulat untuk histogram otomatis.
steps	menggambar kurva fungsi langkah (steps =1, 2)
adaptif	memplot secara adaptif (titik demi titik)
level	menggambar garis/kurva ketinggian fungsi implisit dua variabel. Jika nilai levelnya adalah matriks $2 \times n$, maka rentang level akan digambarkan dalam warna menggunakan gaya isian yang diberikan. Jika outline bernilai benar, maka garis ketinggian akan digambar dengan warna kontur. Dengan menggunakan fitur ini, daerah $f(x,y)$ dapat ditandai. Selain menggunakan matriks $2 \times n$ dapat digunakan vektor ketinggian yang lebih sederhana dan opsi dl dapat digunakan untuk menentukan ketebalan garis kontur.
outline	menggambar batas-batas jangkauan tingkat.
hue	menambahkan gradasi warna pada kurva ketinggian untuk menunjukkan nilai fungsi
contour	menggambar kurva ketinggian dengan pemilihan tingkat ketinggian otomatis
nc	banyaknya kurva ketinggian otomatis

Parameter	Penjelasan
<code>contourcolor</code>	warna kurva ketinggian
<code>contourwidth</code>	lebar kurva ketinggian
<code>title</code>	judul gambar/grafik (aslinya kosong)
<code>x1, y1</code>	label untuk sumbu x dan y
<code>smaller</code>	jika >0 , terdapat lebih banyak ruang di sebelah kiri untuk menuliskan label.
<code>vertikal</code>	mengaktifkan atau mematikan label vertikal. Ini mengubah variabel global
	<code>verticallabels</code> secara lokal untuk satu plot. Nilai 1 berarti teks vertikal,
	nilai 2 menggunakan label numerik vertikal pada sumbu y .
<code>filled</code>	mengarsir daerah kurva
<code>fillcolor</code>	warna diagram batang dan arsiran daerah kurva
<code>outline</code>	batas untuk poligon yang diarsir
<code>logplot</code>	mengatur kurva fungsi logaritma
	1 = logplot dalam y ,
	2 = logplot dalam x dan y ,
	3 = logplot dalam x .
<code>own</code>	Sebuah string, yang menunjuk ke rutinitas plotnya sendiri. Dengan opsi <code>>user</code> , Anda mendapatkan interaksi pengguna yang sama seperti di <code>plot2d</code> . Kisarannya akan ditentukan sebelum setiap panggilan ke fungsi Anda.
<code>maps</code>	ekspresi peta (0 lebih cepat), fungsi selalu dipetakan.
<code>clipping</code>	Tampilkan kliping (nilai aslinya)
<code>cgrid</code>	Menentukan banyak jari-jari dan lingkaran yang digambar pada bidang kompleks.
	<code>cgrid</code> dapat bernilai vektor $[cx, cy]$ dengan cx banyak jari-jari yang digambar dan cy banyak lingkaran yang digambar.

Bab 7.

Visualisasi Grafik 3 Dimensi (3D) dengan EMT

Bab ini menjelaskan tentang berbagai perhitungan dan visualisasi grafik tiga dimensi (fungsi dua variabel) dengan menggunakan software EMT. (Sedang dalam proses)

Daftar Pustaka

- Anonymous (2023a). URL: <https://www.mathsisfun.com/definitions/computation.html> (diakses pada 9 - 8 - 2023).
- Anonymous (2023b). URL: <https://www.storyofmathematics.com/glossary/computation/> (diakses pada 9 - 8 - 2023).
- Contributors, Scilab (2022). *Scilab Documentation*. URL: https://scilab.gitlab.io/legacy_wiki/Documentation.html (diakses pada 4 - 4 - 2024).
- Contributors, Wikipedia (2022). "FreeMat". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/FreeMat> (diakses pada 2 - 10 - 2023).
- Contributors, Wikipedia (2023a). "C.a.R.". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/C.a.R.> (diakses pada 1 - 8 - 2023).
- Contributors, Wikipedia (2023b). "CarMetal". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/CaRMetal> (diakses pada 2 - 10 - 2023).
- Contributors, Wikipedia (2023c). "Comparison of computer algebra system.". In: *Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/Comparison_of_software_calculators (diakses pada 10 - 6 - 2023).
- Contributors, Wikipedia (2023d). "Geometer's Sketchpad". In: *Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/The_Geometer's_Sketchpad (diakses pada 2 - 10 - 2023).
- Contributors, Wikipedia (2023e). "List of computer algebra system". In: *Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/List_of_computer_algebra_systems (diakses pada 2 - 10 - 2023).
- Contributors, Wikipedia (2023f). "List of interactive geometry software". In: *Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/List_of_interactive_geometry_software (diakses pada 2 - 10 - 2023).

- Contributors, Wikipedia (2023g). "Mathematical software". In: *Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/Mathematical_software (diakses pada 21 - 10 - 2023).
- Contributors, Wikipedia (2024a). "DrGeo". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/DrGeo> (diakses pada 2 - 3 - 2024).
- Contributors, Wikipedia (2024b). "GeoGebra". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/GeoGebra> (diakses pada 12 - 3 - 2024).
- Contributors, Wikipedia (2024c). "GNU Octave". In: *Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/GNU_Octave (diakses pada 2 - 3 - 2024).
- Contributors, Wikipedia (2024d). "Maple (software)". In: *Wikipedia, The Free Encyclopedia*. URL: [https://en.wikipedia.org/wiki/Maple_\(software\)](https://en.wikipedia.org/wiki/Maple_(software)) (diakses pada 2 - 3 - 2024).
- Contributors, Wikipedia (2024e). "Mathcad". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/Mathcad> (diakses pada 15 - 4 - 2024).
- Contributors, Wikipedia (2024f). "MATLAB". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/MATLAB> (diakses pada 2 - 3 - 2024).
- Contributors, Wikipedia (2024g). "SageMath". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/SageMat> (diakses pada 15 - 3 - 2024).
- Contributors, Wikipedia (2024h). "Scilab". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/Scilab> (diakses pada 2 - 3 - 2024).
- Contributors, Wikipedia (2024i). "Scilab". In: *Wikipedia, The Free Encyclopedia*. URL: <https://en.wikipedia.org/wiki/Scilab> (diakses pada 4 - 4 - 2024).
- Contributors, Wikipedia (2024j). "Software calculator". In: *Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/Software_calculator (diakses pada 14 - 3 - 2024).
- Contributors, Wikipedia (2023h). "Wolfram Mathematica". In: *Wikipedia, The Free Encyclopedia*. URL: https://en.wikipedia.org/wiki/Wolfram_Mathematica (diakses pada 29 - 12 - 2023).
- Grothmann, René (2006). *About C.a.R.* URL: <http://compute.ku-eichstaett.de/MGF/wikis/caruser/doku.php?id=history> (diakses pada 30 - 3 - 2023).
- Grothmann, René (2015). *Skriptum Numerik*. URL: <http://euler.rene-grothmann.de/german/%20Numerik-Skript.pdf> (diakses pada 12 - 4 - 2023).
- Grothmann, René (march 2016). "Discover Euler Math Toolbox". in *International Journal of Computer Discovered Mathematics (IJCDM)*: 1.1, pages 62–75. URL: https://drive.google.com/file/d/1NnbgY8zmPVnWntSghhF_7aVM4dJDmsOU/view (diakses pada 12 - 4 - 2023).

- Grothmann, René (2021). *Euler Math Toolbox, An Introduction*. URL: https://euler-math-toolbox.de/euler_en.pdf (diakses pada 12 - 4 - 2024).
- Grothmann, René (2023). *C.a.R. (Compass and Ruler)*. **version** 2023-03. URL: <http://car.rene-grothmann.de/> (diakses pada 14 - 3 - 2023).
- Grothmann, René (2024). *EMT Documentation*. URL: <http://euler.rene-grothmann.de/docs/> (diakses pada 16 - 3 - 2024).
- Kortenkamp Lembach, Richter & Akira Iritani (2011). *Cinderella Documentation*. URL: <https://doc.cinderella.de/tiki-index.php> (diakses pada 2 - 10 - 2023).
- Papadimitriou, Stergios (2020). GroovyLab. URL: <https://github.com/sterglee/GroovyLab> (diakses pada 2 - 10 - 2023).
- Rowland, Todd (2024). "Computation." In: *MathWorld – A Wolfram Web Resource*. URL: <https://mathworld.wolfram.com/Computation.html> (diakses pada 16 - 3 - 2024).
- Woollet, Edwin Ted (2023). *Maxima by Example*. URL: <https://home.csulb.edu/~woollett/mbe.html> (diakses pada 2 - 10 - 2023).